

[1-6]- Data Science

CODE

CADEMY

—

CODECAMPY

Computers, Compute!

In the 20th century, there was a growing need for mathematical computation. Before the rise of powerful machines, people would perform these mathematical computations by hand (e.g. the movie, *Hidden Figures*), and these people were called computers. They would sit in large rooms and solve complex equations with long, complicated decimals. Imagine your hardest math class without a calculator!

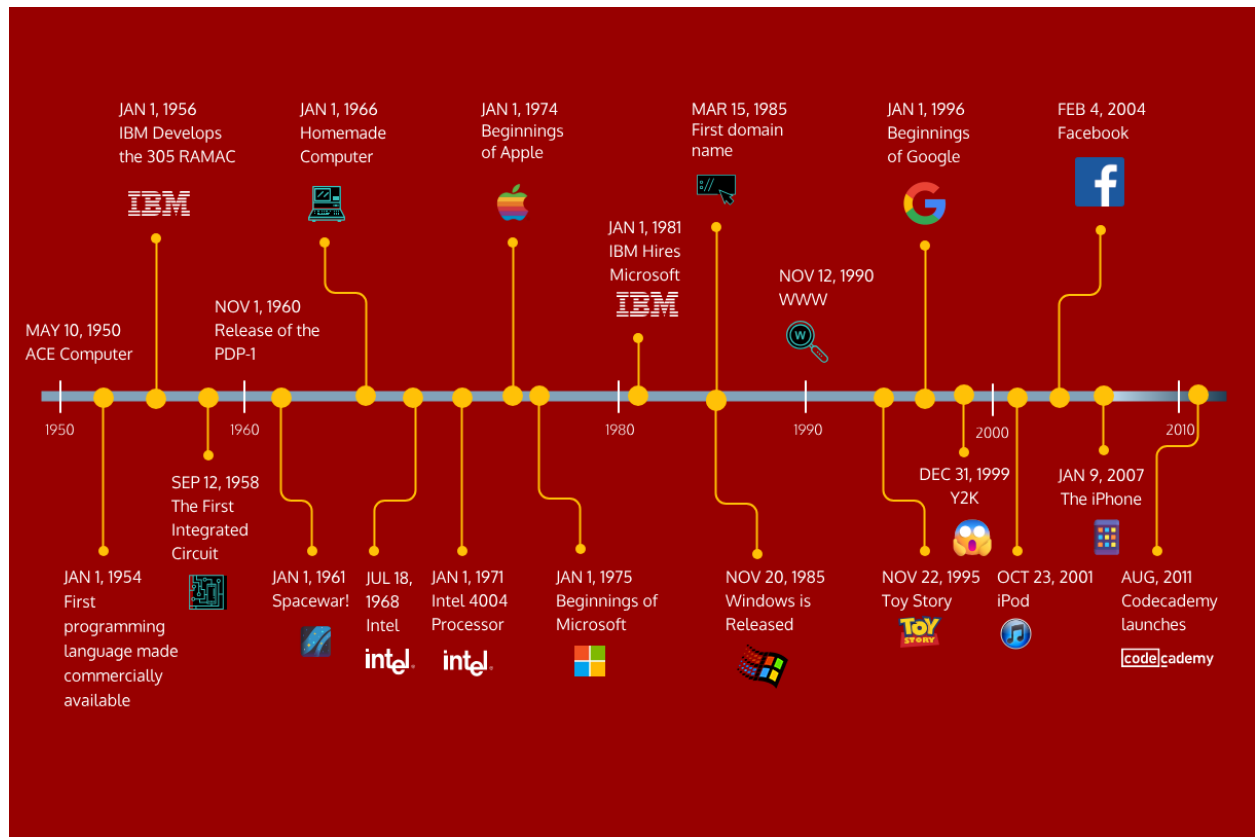
However, with the rise of technological advancements, these people were replaced by machines that were named “computers” (surprise, surprise). These machines were improvements on Babbage’s original proposal and had the ability to perform even more complex mathematical operations. NASA used these machines for their rocket computations during the famous Space Race. These computers took up entire rooms and were operated with punch cards.



However, as cool as it sounds, we don’t use punch cards to write programs today. In 1971, a chip was developed that allowed users to type programs. This started the rise of

personal computers with the [IBM Selectric Typewriter](#). Today, computer scientists still type their programs using various programming languages.





MOORE'S LAW

You're telling me this is a Computer?

Remember

Preview: Docs Loading link description

[Ada Lovelace](#)

and her first algorithm? In the same paper where she described her program, Lovelace predicted that one day computers could do much more than crunch numbers. Today, that prediction has held true! Programs can now make predictions about what tv shows you like, beat a grandmaster at chess, and personalize advertisements. But how did we get here?

In 1947, Bell Labs invented something that would change computer science forever - *transistors*. Transistors are devices used to switch and amplify electronic signals: the basic building blocks of computers. The old

Preview: Docs Loading link description

[method](#)

of switching and amplifying signals used unwieldy, fragile glass tubes. Transistors were sturdier, cheaper, and smaller, and scientific advances have only enhanced those traits. The first transistors were the size of the palm of your hand. Now, the average transistor is only the size of a couple of molecules!

In fact, the computer scientist

Preview: Docs Gordon Moore is the co-founder of Intel Corporation, the largest manufacturer of various computer parts like microprocessors for companies like Acer, Lenovo, HP, and Dell. He was also the primary advocate for Moores Law.

[Gordon Moore](#)

predicted that transistors would get exponentially smaller. Since transistors are grouped into chips called integrated circuits, the prediction is expressed in those terms: Moore predicted that the amount of transistors in an integrated circuit would double every two years, and that prediction has held true! This is popularly known as [Moore's law](#).

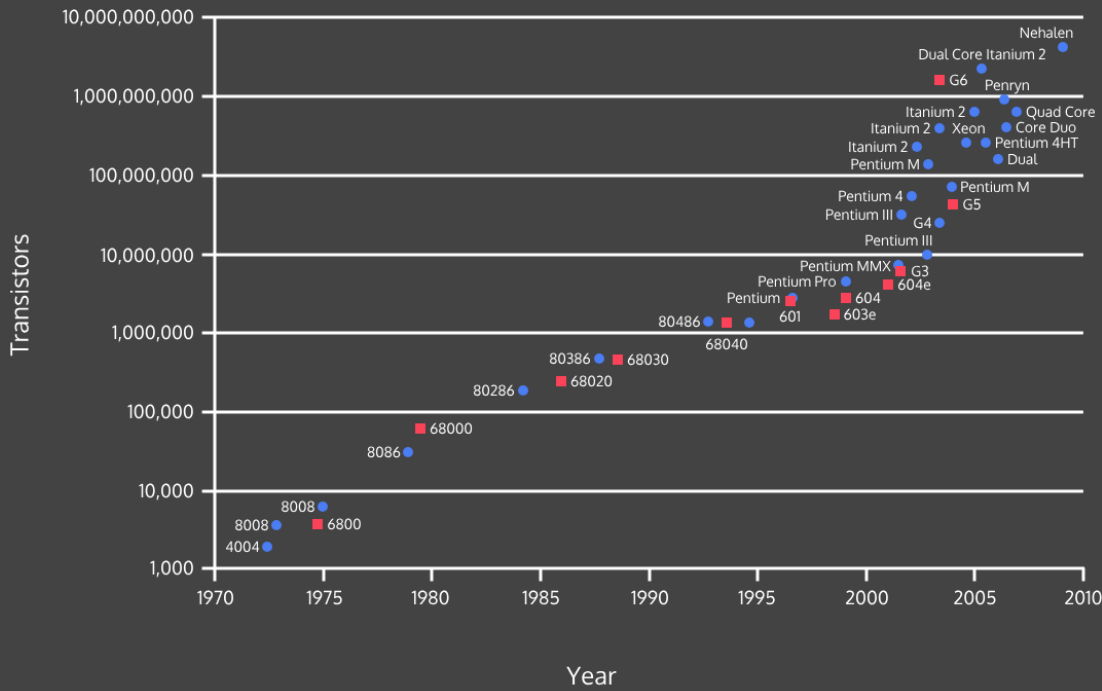
With better computers, programmers can solve more advanced problems, which leads to new and interesting jobs (we will get more into this in Lesson 3). For instance, if you are a programmer who enjoys educating the world you could get a job at Codecademy. If you are a TV and film aficionado, you could join Netflix or Hulu. If you love cars, you could join any car company (how about Tesla?! Every industry needs programmers, and over time, you can develop the skills to become a high-valued programmer.

Instructions

Moore's law is shown on the right, with the year on the x-axis and a logarithmic scale of the number of transistors on the y-axis. Over time, computers have become smaller and smaller with the exponential increase of transistors that could fit in a computer.

Look around you right now. How many computers can you see? Chances are you are vastly underestimating that number.

Moore's Law



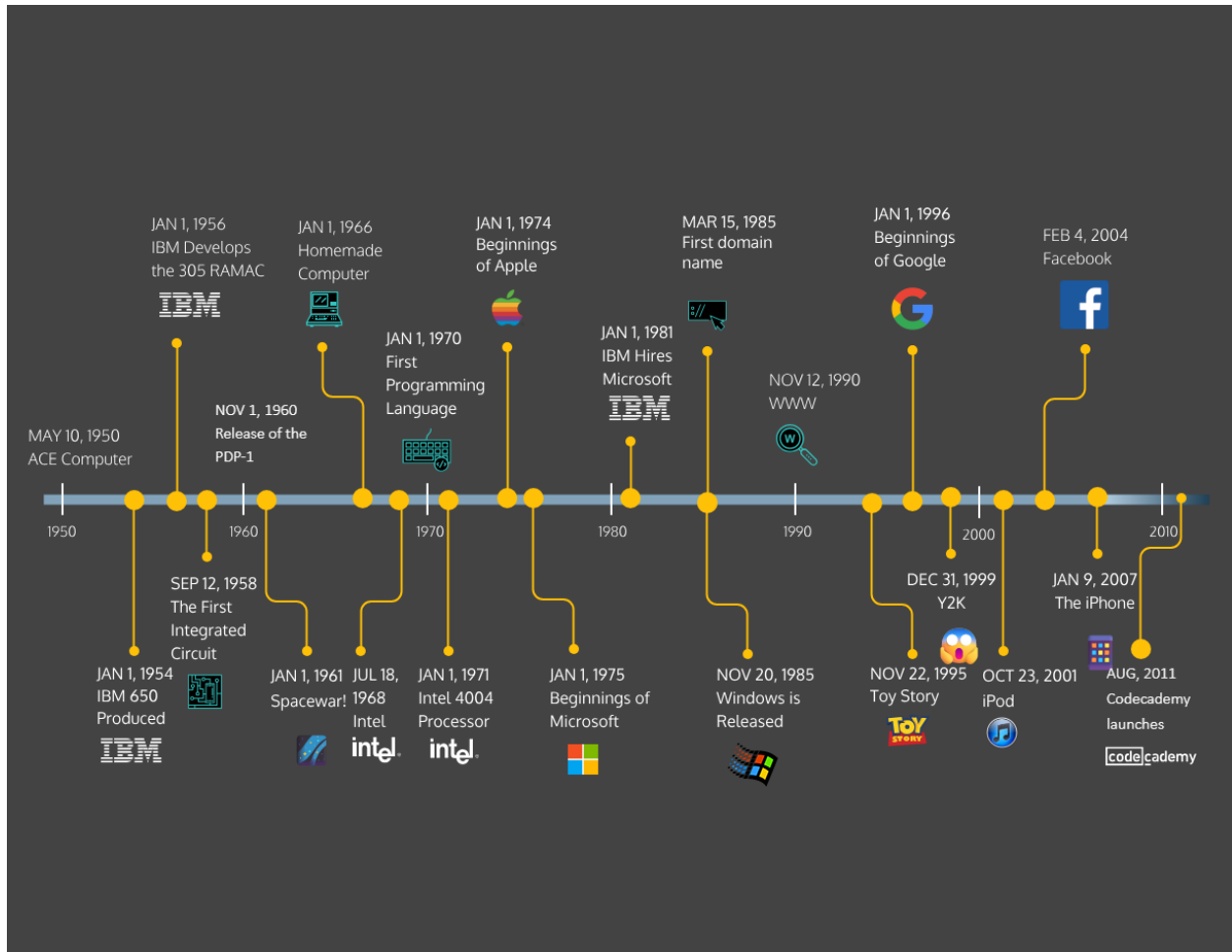
History of Computer Science

What Now?

The role of programming and computers has evolved over time, from the simple abacus to the complex laptops and supercomputers of today.

Today, computer science is one of the fastest growing college degrees, and half of the top-earning jobs in the United States—those pulling in \$57,000 per year or higher—require the ability to code. On the Codecademy platform, you will have the ability to learn these programming skills to score a great job.

In the next lesson, we will explore how programming exactly works, for in the words of Bill Gates, “programming teaches you how to think”.



Survey of Computer Science - The Tale of Kenny

What is Computer Science?

Meet Kenny. Kenny is a bright young man who recently graduated from a prestigious university with a degree in Computer Science. But Kenny is a little disillusioned. He isn't sure that he wants to go into

Preview: Docs Software engineering is a field of study that involves using engineering principles to design, build, test, and maintain software systems.

[software engineering](#)

or even be a part of the technology industry at all.

Kenny thinks to himself, “how will I ever use any of these computer science skills out in the real world?”

Kenny is not unique. Often the realm of computer science can feel indescribably abstract and arcane, even to those who study it. But in fact, the same abstractness that makes computer science feel inaccessible also makes it incredibly useful and applicable to a wide range of fields, disciplines, and industries. Computer Science isn't just about learning how to code and program, it's about learning how to view the world in a different way, to see patterns that emerge from the mundane, and to reveal opportunities for optimization and improvement around every corner.

Over the course of this lesson, we will follow Kenny as he pursues a variety of different careers and hobbies and discovers that his computer science background is more relevant to his everyday life than he could have ever imagined.

```
# This is a Python Dictionary that contains all of the students in Kenny's
class as well as their grades.

student_grades = {"Jeremy" : 87, "Kyla" : 82, "Ayesha" : 90, "Aleida" :
94, "Todd" : 79, "Maxwell" : 98, "Yolonda" : 81, "Kiyoko" : 71, "Dagmar" :
73, "Laura" : 91, "Shimeah" : 81, "Songqiao" : 92, "Frankie" : 87,
"Natalia" : 95, "Gonzalo" : 82, "Pavel" : 78}

# This is a function that determines the student with the highest grade
given a dictionary
```

```

def highest_grade(grades):
    top_of_class = list(grades.keys())[0]
    for student_a in grades:
        for student_b in grades:
            if (grades[student_a] > grades[student_b]) and (grades[student_a] >
grades[top_of_class]):
                top_of_class = student_a
            elif (grades[student_b] > grades[student_a]) and (grades[student_b]
> grades[top_of_class]):
                top_of_class = student_b
    return top_of_class

# This is a function that determines the student with the lowest grade
given a dictionary
def lowest_grade(grades):
    bottom_of_class = list(grades.keys())[0]
    for student_a in grades:
        for student_b in grades:
            if (grades[student_a] < grades[student_b]) and (grades[student_a] <
grades[bottom_of_class]):
                bottom_of_class = student_a
            elif (grades[student_b] < grades[student_a]) and (grades[student_b]
< grades[bottom_of_class]):
                bottom_of_class = student_b
    return bottom_of_class

# This is Kenny's Algorithm! It sorts the students into pairs based on
their grades.
def kennys_algorithm(grade_dict):
    student_pairs = []
    while len(grade_dict) > 0:

```

```
    student_pairs.append([highest_grade(grade_dict),
lowest_grade(grade_dict)])
    grade_dict.pop(highest_grade(grade_dict))
    grade_dict.pop(lowest_grade(grade_dict))
print(student_pairs)
```

Paste the code below!

KENNY'S ALGORITHM

1. Get list of unpaired students
2. Create empty list of paired students
3. Until the list of unpaired students is empty, repeat the following steps:
 4. Determine the student with the highest grade in the unpaired list
 5. Determine the student with the lowest grade in the unpaired list
 6. Pair those students and add that list to the list of paired students
 7. Remove those students from the list of unpaired students
8. Print the list of paired students to the terminal

strawberry_picking(empty_container):

1. While the container is not full:

2. Check a strawberry

3. If the strawberry is red:

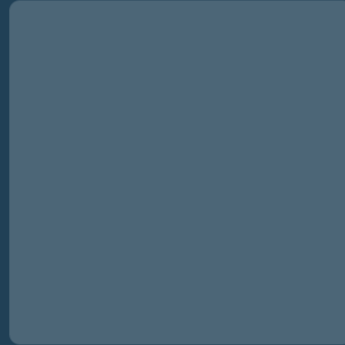
3a. pick the strawberry

3b. place it in the container

4. Else, if the strawberry is green:

4a. leave it on the plant

5. Once the container is full, return the container



Control Flow and Logic

Although it was interesting and fun, Kenny didn't feel that the farm life was quite right for him. But his time on the farm made him interested in food and food safety, so Kenny became a health inspector.

In his job as a health inspector, he had to give the restaurants he inspected a grade. In order to grade them fairly, Kenny knew he had to develop a system to decide their score based on a set of conditions of the restaurant.

Kenny would check each area in the restaurant in sequence: first the dining room, then the bar, then the servers station, and finally the kitchen.

If he found any pests, dirt, or something unsanitary at any of those locations he would subtract a letter from the overall grade of the restaurant. The restaurants always started with an *A*, but by the end of the inspection sometimes they ended up having an *F*, and no one wants to eat at a restaurant with an *F* health grade.

Kenny knew that what he had developed was a

Preview: Docs Control flow refers to the order in which statements and instructions are executed in a program.

[control flow](#)

. He had a set sequence he followed, and throughout that sequence, based on some

Preview: Docs Loading link description

[conditionals](#)

he would lower the health grade of the establishment. This is a way of thinking that he learned when developing software. Kenny had turned his job as a health inspector into a sequence of if/then statements within a control flow. This allowed him to grade each restaurant fairly and consistently.

Instructions

Checkpoint 1 Enabled

1.

In the code editor, you'll find a short text-based adventure game where you can play as health inspector Kenny and give your daily quota of restaurants their health grades.

Click run, then copy and paste `python kenny_the_health_inspector.py` into the terminal on the far right titled `bash` and hit return or enter on your keyboard.

Games like this, and in fact most functions and programs, rely heavily on *control flow* to guide the user through the application. Take a look at the code for the game itself and see if you can see examples of control flow logic.

Anywhere you see `if` statements, or comparative operators like `==`, `>=`, `>`, or `<`, there's logic at work!

```
import textwrap
import random
```

```
# This function makes up the main body of the game and controls a players
general flow through each of the options.
def health_inspector_kenny(visited):
    if visited == 0:
        print(textwrap.fill(lb + "Welcome Health Inspector Kenny.\nToday you
have to inspect three restaurants and give them ratings.", width=60)
        + "\nWhere would you like to go first?\n" + lb)
        choice = location_selector(locations)
        if locations[choice] == "Billy's Big Burgers":
            locations.remove("Billy's Big Burgers")
            visited += 1
            billys_grade = billys_burgers()
            print(textwrap.fill("You've finished inspecting \"Billy's Big
Burgers\" and have given it a grade of {0}.".format(grades[billys_grade]),
width=60))
            health_inspector_kenny(visited)
        elif locations[choice] == "Lucy's Lemonade and Licorice":
            locations.remove("Lucy's Lemonade and Licorice")
            visited += 1
            lucys_grade = lucys_lemonade()
            print(textwrap.fill("You've finished inspecting \"Lucy's Lemonade
and Licorice\" and have given it a grade of
{0}.".format(grades[lucys_grade]), width=60))
            health_inspector_kenny(visited)
        elif locations[choice] == "Laura's Luncheonette":
            locations.remove("Laura's Luncheonette")
            visited += 1
            lauras_grade = lauras_luncheonette()
            print(textwrap.fill("You've finished inspecting \"Laura's
Luncheonette\" and have given it a grade of
{0}.".format(grades[lauras_grade]), width=60))
            health_inspector_kenny(visited)
```

```
elif visited < 3:
    print("Where do you want to go next?" + lbb)
    choice = location_selector(locations)
    if locations[choice] == "Billy's Big Burgers":
        locations.remove("Billy's Big Burgers")
        visited += 1
        billys_grade = billys_burgers()
        print(textwrap.fill("You've finished inspecting \"Billy's Big
Burgers\" and have given it a grade of {0}.".format(grades[billys_grade]),
width=60))
        health_inspector_kenny(visited)
    elif locations[choice] == "Lucy's Lemonade and Licorice":
        locations.remove("Lucy's Lemonade and Licorice")
        visited += 1
        lucys_grade = lucys_lemonade()
        print(textwrap.fill("You've finished inspecting \"Lucy's Lemonade
and Licorice\" and have given it a grade of
{0}.".format(grades[lucys_grade]), width=60))
        health_inspector_kenny(visited)
    elif locations[choice] == "Laura's Luncheonette":
        locations.remove("Laura's Luncheonette")
        visited += 1
        lauras_grade = lauras_luncheonette()
        print(textwrap.fill("You've finished inspecting \"Laura's
Luncheonette\" and have given it a grade of
{0}.".format(grades[lauras_grade]), width=60))
        health_inspector_kenny(visited)
    else:
        print(textwrap.fill(lb + "Congratulations! You've finished a day of
fullfilling health inspecting. Head home, pop open your favorite beverage,
turn on old reruns of America's Next Top Model, and enjoy your evening." +
lbb, width=60))
```

```

# Helper functions
def location_selector(locations):
    for i in range(0, len(locations)):
        print("(" + str(i + 1) + ") " + locations[i] + "\n")
    selection = chooser(locations)
    return selection

def chooser(locations):
    try:
        selection = int(input("Enter the number of your choice: ")) - 1
        if selection >= len(locations):
            print("Sorry, invalid selection. Please try again.")
            selection = chooser(locations)
        return selection
    except (NameError, SyntaxError):
        print("Sorry, invalid selection. Please try again.")
        selection = chooser(locations)
    return selection

def responder():
    response = int(input("Enter the number of your choice: "))
    if response not in [1, 2]:
        print("Please enter a valid choice.")
        responder()
    return response

def format_text(text):
    print(textwrap.fill(lb + text, width=60, replace_whitespace=False) +
lb)

# Locations

```

```

def billys_burgers():
    # 3 = A, 2 = B, 1 = C, 0 = F

    billys_grade = 3

    areas = ["Dining Room", "Bar", "Kitchen"]

    print(textwrap.fill(lb + "You enter the rustic yet charming all-American
diner \"Billy's Big Burgers\". Billy himself, a large swarthy man in a
striped apron, comes up to you and gruffly shakes your hand.\n\"Welcome
Mr. Health Inspector, please take a look around, let me know if you need
anything.\" Billy retreats back to the kitchen, keeping an eye on you as
he backs away. Seems like a pretty weird guy.", width=60,
replace_whitespace=False))

    for i in range(len(areas)):

        print("What area of the restaraunt would you like to check?\n"+ lb)

        choice = location_selector(areas)

        if areas[choice] == "Dining Room":

            format_text("You enter the dining room and are immediately hit with
the smell of unwashed laundry. \"Hmm, that's not a good sign\" you think
to yourself. Upon inspecting several tables, you find stains all over most
of the tablecloths. You peer under the top of one of the booths only to
find more dried gum than an elementary school's cafeteria.")

            print("How clean is the {0}? \n\n(1) {1} (pass this area).\n\n(2) {2}
(fail this area).\n".format(areas[choice],
clean_responses[random.randint(0,6)],
dirty_responses[random.randint(0,6)]))

            response = responder()

            if response == 2:

                billys_grade -= 1

            print(textwrap.fill(lb + "You mark your grade for the {0} on your
clipboard. The grade for Billy's Big Burgers is currently
{1}".format(areas[choice], grades[billys_grade]) + lbb, width=60))

            areas.remove("Dining Room")

        elif areas[choice] == "Bar":

```

```
        format_text("Walking behind the bar, you see everything appears  
fairly organized and clean. You notice a small picture of a little girl  
next to the glasses. Billy notices you looking at the picture and comes  
over. \"That's my girl Sasha\", he taps the picture and looks directly in  
your eyes, \"she'd be really crushed if the restaurant didn't pass the  
health inspection. You wouldn't disappoint Sasha would you?\")  
        print("How clean is the {0}?\\n\\n(1) {1} (pass this area).\\n\\n(2) {2}  
(fail this area).\\n".format(areas[choice],  
clean_responses[random.randint(0,6)],  
dirty_responses[random.randint(0,6)]))  
        response = responder()  
        if response == 2:  
            billys_grade -= 1  
            print(textwrap.fill(lb + "You mark your grade for the {0} on your  
clipboard. The grade for Billy's Big Burgers is currently  
{1}".format(areas[choice], grades[billys_grade]) + lbb, width=60))  
            areas.remove("Bar")  
        elif areas[choice] == "Kitchen":  
            format_text("The first thing you see in the kitchen is a massive  
pile of dishes sitting in the sink. \"I haven't had a chance to clean  
those yet\", states Billy, standing in your way. Pushing past him you see  
more concerning things. Open packs of American cheese sitting on counters  
in the heat. A mysterious mound of grey meat sits on top of a cold grill.  
A strange orange fluid seems to be leaking from the bottom of the fridge.  
To top it off, you see an empty container of plastic gloves sitting above  
a workstation, stuffed with what appears to be used tissues.")  
            print("How clean is the {0}?\\n\\n(1) {1} (pass this area).\\n\\n(2) {2}  
(fail this area).\\n".format(areas[choice],  
clean_responses[random.randint(0,6)],  
dirty_responses[random.randint(0,6)]))  
            response = responder()  
            if response == 2:
```

```

        billys_grade -= 1

        print(textwrap.fill(lb + "You mark your grade for the {0} on your
clipboard. The grade for Billy's Big Burgers is currently
{1}".format(areas[choice], grades[billys_grade]) + lbb, width=60))

        areas.remove("Kitchen")

    return billys_grade

def lauras_luncheonette():
    # 3 = A, 2 = B, 1 = C, 0 = F

    lauras_grade = 3

    areas = ["Dining Room", "Bar", "Kitchen"]

    print(textwrap.fill(lb + "You pull up to \"Laura's Luncheonette\", a
plain white building with fading blue paint on the window trim. Laura
meets you outside, bouncing from one foot to the other, appearing quite
nervous. She leads you inside and gestures wildy. \"Please, please,
inspect away, inspect it all it's perfect I promise!\" She nods
enthusiastically at you to begin.", width=60))

    for i in range(len(areas)):

        print("What area of the restaurant would you like to check?" + lb)

        choice = location_selector(areas)

        if areas[choice] == "Dining Room":

            format_text("As you start to inspect the dining room you are
interrupted as a family of mice dart across the room, running across your
shoes. Clearly these mice are not afraid of people. Laura doesn't seem to
notice the mice as she has begun frantically weaving between tables
emptying ashtrays into a pocket on her apron.")

            print("How clean is the {0}?\\n\\n(1) {1} (pass this area).\\n\\n(2) {2}
(fail this area).\\n".format(areas[choice],
clean_responses[random.randint(0,6)],
dirty_responses[random.randint(0,6)]))

            response = responder()

            if response == 2:

```

```
        lauras_grade -= 1

        print(textwrap.fill(lb + "You mark your grade for the {0} on your
clipboard. The grade for Laura's Luncheonette is currently
{1}".format(areas[choice], grades[lauras_grade]) + lbb, width=60))

        areas.remove("Dining Room")

    elif areas[choice] == "Bar":

        format_text("The bar is crowded with half finished drinks, some
appear to have formed some sort of weird film on them. You want to inspect
behind the bar but your foot gets stuck in something extremely sticky that
is coating the floor. Oh boy, it's that kind of day.")

        print("How clean is the {0}?\\n\\n(1) {1} (pass this area).\\n\\n(2) {2}
(fail this area).\\n".format(areas[choice],
clean_responses[random.randint(0,6)],
dirty_responses[random.randint(0,6)]))

        response = responder()

        if response == 2:

            lauras_grade -= 1

            print(textwrap.fill(lb + "You mark your grade for the {0} on your
clipboard. The grade for Laura's Luncheonette is currently
{1}".format(areas[choice], grades[lauras_grade]) + lbb, width=60))

            areas.remove("Bar")

        elif areas[choice] == "Kitchen":

            format_text("The 'kitchen' is actually a series of cafeteria tables
filled with rusty hot plates surrounded by cupboards. This place looks
more like a high school chemistry lab than it does an eating
establishment. Upon opening one of the cupboards, you see several
waterbugs quickly scamper out of the light. You've seen enough here, this
is bad.")

            print("How clean is the {0}?\\n\\n(1) {1} (pass this area).\\n\\n(2) {2}
(fail this area).\\n".format(areas[choice],
clean_responses[random.randint(0,6)],
dirty_responses[random.randint(0,6)]))
```



```

    response = responder()

    if response == 2:
        lauras_grade -= 1

        print(textwrap.fill(lb + "You mark your grade for the {0} on your
clipboard. The grade for Laura's Luncheonette is currently
{1}".format(areas[choice], grades[lauras_grade]) + lbb, width=60))

        areas.remove("Kitchen")

    return lauras_grade

def lucys_lemonade():
    # 3 = A, 2 = B, 1 = C, 0 = F

    lucys_grade = 3

    areas = ["Dining Room", "Bar", "Kitchen"]

    print(textwrap.fill(lb + "Lucy's Lemonade in Licorice, a small hole in
the wall restaraunt, is teeming with people waiting outside when you
arrive. You push your way to front of the line and ask to speak to Lucy. A
woman comes out, pauses to wash her hands, and then turns to you. \n\n
\"Oh, I didn't expect an inspection today. Sorry for the crowd, we
launched a brand new flavor of licorice today, Matcha! But everything
should be in order, come in and look.\", width=60,
replace_whitespace=False))

    for i in range(len(areas)):
        print("What area of the restaraunt would you like to check?" + lbb)

        choice = location_selector(areas)

        if areas[choice] == "Dining Room":
            format_text("You squeeze into the small dining room. All six of the
tables are filled with customers, munching on pale green strands of
licorice and drinking glasses of lemonade. 'Interesting color of licorice'
you think to yourself 'must be the new flavor.' \n\n Despite being
crowded, the waiters are moving through smoothly and the floors and tables
seem quite clean. A group of patrons gets up to leave and the table is
promptly wiped down and cleaned.")

```

```
print("How clean is the {0}?\\n\\n(1) {1} (pass this area).\\n\\n(2) {2}
(fail this area).\\n".format(areas[choice],
clean_responses[random.randint(0,6)],
dirty_responses[random.randint(0,6)]))

response = responder()

if response == 2:

    lucys_grade -= 1

    print(textwrap.fill(lb + "You mark your grade for the {0} on your
clipboard. The grade for Lucy's Lemonade and Licorice is currently
{1}".format(areas[choice], grades[lucys_grade]) + lbb, width=60))

    areas.remove("Dining Room")

elif areas[choice] == "Bar":

    format_text("Dominating the wall behind the bar are rows and rows of
jars of different flavors of licorice. Upon inspection, each jar is
sparklingly clean and closed with an airtight seal. \\n\\nOn the bar itself
lies three large glass barrels of lemonade. Next to the barrels are lines
of cleaned glasses. You see the various lemonade garnishes properly
refrigerated underneath the bartop.")

    print("How clean is the {0}?\\n\\n(1) {1} (pass this area).\\n\\n(2) {2}
(fail this area).\\n".format(areas[choice],
clean_responses[random.randint(0,6)],
dirty_responses[random.randint(0,6)]))

    response = responder()

    if response == 2:

        lucys_grade -= 1

        print(textwrap.fill(lb + "You mark your grade for the {0} on your
clipboard. The grade for Lucy's Lemonade and Licorice is currently
{1}".format(areas[choice], grades[lucys_grade]) + lbb, width=60))

        areas.remove("Bar")

    elif areas[choice] == "Kitchen":

        format_text("In the kitchen you see a large strange machine. As you
go to investigate it Lucy comes over to explain it's function. \\\"See
```

```

here,\" she points at a row of nozzles, \"this is where the strands of
licorice come out and this here twirls them.\" Cool, you think to yourself
as you note that this machine is also very clean. In fact, everything in
the kitchen seems spotless. You've never seen such a well-run lemonade and
licorice establishment.")

    print("How clean is the {0}?\\n\\n(1) {1} (pass this area).\\n\\n(2) {2}
(fail this area).\\n".format(areas[choice],
clean_responses[random.randint(0,6)],
dirty_responses[random.randint(0,6)]))

    response = responder()

    if response == 2:

        lucys_grade -= 1

    print(textwrap.fill(lb + "You mark your grade for the {0} on your
clipboard. The grade for Lucy's Lemonade and Licorice is currently
{1}".format(areas[choice], grades[lucys_grade]) + lbb, width=60))

    areas.remove("Kitchen")

    return lucys_grade


# Global Objects

lbb = "\\n-----\\n"
lb = "-----\\n"

grades = ["F", "C", "B", "A"]

locations = ["Billy's Big Burgers", "Lucy's Lemonade and Licorice",
"Laura's Luncheonette"]

# use clean_responses[randint(0,6)] to generate a random response
clean_responses = ["It looks pretty clean", "Clean enough", "Very clean",
"I'd eat food prepared here", "Cleaner than my house", "Squeaky clean",
"The cleanest I've ever seen"]

dirty_responses = ["Oh god, it's so gross", "I can't breathe in here",
"Quite dirty", "Very dirty", "Really disgusting", "Dirtier than a
dumpster", "I wouldn't eat here"]

visited = 0

```

```
health_inspector_kenny(visited)
```

```
# This binary search function will look for a book on a bookshelf by  
iteratively narrowing down the area where it might find the book, until it  
finds the book.
```

```
def binary_search(lst, search_val):
```

```
    if len(lst) == 0:
```

```
        print("Book is not on the shelf!")
```

```
        return
```

```
    print("Checking the bookshelf and finding: {0}".format(lst[0]))
```

```
    mid = len(lst) // 2
```

```
    if lst[mid] == search_val:
```

```
        print("\nFound the book: {0}\n\n\n\n\n\n\n\n".format(search_val))
```

```
        return
```

```
    elif lst[mid] > search_val:
```

```
        binary_search(lst[:mid], search_val)
```

```
    else:
```

```
        binary_search(lst[(mid + 1):], search_val)
```

```
# This linear search looks through a bookshelf book by book until it finds  
the book it is looking for.
```

```
def linear_search(lst, search_val):
```

```
    count = 0
```

```
while len(lst) != 0:

    print("Checking the bookshelf and finding: {}".format(lst[0]))

    if lst[0] == search_val:

        print("\nFound the book: {}\n\n\n\n\n\n\n".format(search_val))

        return

    lst = lst[1:]

    count += 1

with open("books.csv") as f:

    books = f.read()

    bookshelf = sorted(books.split(","))

# Copy-paste the search code below:

linear_search(bookshelf, "Do Androids Dream of Electric Sheep?")

binary_search(bookshelf, "Do Androids Dream of Electric Sheep?")
```

Programming / Web Dev

HTML
CSS
JavaScript

CSS
JavaScript

JavaScript

Data Scientists

Statistics

Normal Distribution

Mean = 0

Std = 1

Probability

```
# We have hidden code in another file. If you're curious, open the folder
to the left and inspect the simulate.py file
```

```
from simulate import simulate

num_people_in_room = 2          #Change This Number (keep it smaller than 100
to save processing power)

simulate(num_people_in_room)
```

```
import random
#Simulate a room with a certain number of people
def simulate(num_people):
    birthdays = []
    print("Here's what our room looks like:\n")
    months = ["January", "February", "March", "April", "May", "June", "July", "August", "September", "October", "November", "December"]
    #Assign a random birthday to each person
    for i in range(0, num_people):
        #Choose a random month
        month_choice = random.choice(months)
        #Choose a random day based on month
        if month_choice == "February":
            day = random.randint(1, 29)
        elif month_choice == "April" or month_choice == "June" or month_choice == "September" or month_choice == "November":
            day = random.randint(1, 30)
        else:
            day = random.randint(1, 31)
        birthday = month_choice + " " + str(day)
        #Store the birthday
        birthdays.append(birthday)
        print("Person {0}'s birthday: {1}".format(i + 1, birthday))
    calculate_probability(num_people)
    match = False
    #Check for matching birthdays
    for i in range(len(birthdays)):
        if find_duplicates(birthdays, birthdays[i], i):
            match = True
            break
    if not match:
```

```

        print("\n\nIn our simulation, no two people have the same birthday")

#Calculate the probability of there being 2 people with the same birthday
def calculate_probability(num_people):
    #Check there is at least 2 people in the room
    if num_people < 2:
        print("\n\nNot enough people in the room!")
        return
    else:
        #Calculate the probability
        numerator = 365
        countdown = 364
        for i in range(2, num_people + 1):
            numerator = numerator * countdown
            countdown -= 1
        denominator = 365 ** num_people
        probability = 1 - numerator/float(denominator)
        #Change probability to percentage
        rounded = round(probability*100, 2)
        print("\n\nThe probability that two people in a room of {0} people have the same birthday is nearly {1}%".format(num_people, rounded))

#Find the same birthday within our list of birthdays
def find_duplicates(birthdays_list, birthday, index):
    people = []
    for i in range(len(birthdays_list)):
        if birthdays_list[i] == birthday and i != index:
            people.append(i + 1)
    if people:
        people.append(index + 1)
        print("\n\nIn our simulation, the following people have the same birthdays: ")
        for person in people:
            print("Person {0}".format(person))
        return True
    else:
        return False

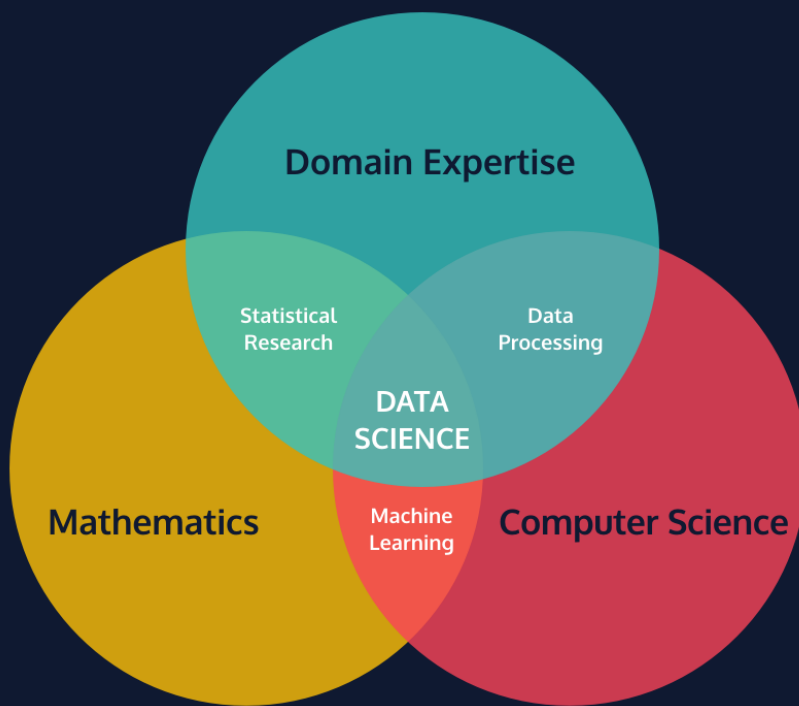
```

Programming

Domain Expertise

Company	How they use your data
	• To predict traffic patterns by location of other users • Autocomplete searches based on search patterns of other users
	• To personalize advertisements based on your interests • To organize news feed based on your interests
	• To provide recommendations of tv shows and movies based on your interests
	• To provide song recommendations based on your interests and likes of other users

Data Scientist Expertise



The Data Science Process



- Read Data
- Clean Data
- Explore the Data
 - Stats Calculations
 - Visualizations
- Modeling and Analysis
- Communication Findings
 - Visualizing
 - Storetelling
-
- Reproducibility and Automation

Clean Data

```
import codecademylib3_seaborn
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Import the CSV files and create the DataFrames:
user_data = pd.read_csv("user_data.csv")
pop_data = pd.read_csv("pop_data.csv")

# Paste print code here:
print(user_data.head(15))

# Paste merge code here:
new_df = pd.merge(user_data, pop_data)
print(new_df.head(15))

# Paste location code here:
new_df.loc[new_df.population_proper < 100000, "location"] = "rural"
new_df.loc[new_df.population_proper >= 100000, "location"] = "urban"
print(new_df.head(15))
```

Netflix Recommendation Engine

Learn how machine learning algorithms impact our daily lives by analyzing one of the most influential algorithms there is: the Netflix recommendation engine!

What's Data Got To Do With It?

You may have heard the term “big data” thrown around, but do you understand what it is? Big data refers to massive datasets that can be used to reveal patterns and trends. Any large quantity of information, from sports scores to social media posts, can be considered big data. And since big data is so, well, big, we need optimized algorithms and high-powered computers to sift through it.

Advances in data science have changed the way we communicate, share, and receive information. Consider how big data has changed our TV and movie experiences. Companies like Netflix collect thousands of data points from several places to make suggestions to users with the help of a tool known as a recommender engine. Gone are the days of browsing the shelves in a Blockbuster on a Friday night (that is, if you even were alive when Blockbusters were around).

But how did the Netflix engineering team build a recommender engine? Netflix uses machine learning, a subset of artificial intelligence, to help their algorithms “learn” without human assistance. Machine learning gives the platform the ability to automate millions of decisions based off of user activities. When Netflix recommends *The Office* because I like *Parks and Recreation*, machine learning was behind that decision. This suggestion is the Netflix recommendation engine at work: it uses your past activity and returns movies and shows it thinks you will enjoy.

The need for recommendation engines and personalization is a result of a phenomenon known as the “era of abundance”. We have a huge variety of choices because of how much is available through the Internet. In this article, we’ll learn how data science has enhanced our ability to choose and, frankly, our Saturday night binging options. Let’s take a deep dive into the Netflix recommendation system.

How Netflix Slays the Recommendation Game

Let's not date ourselves, but some may remember a time when we frequented video rental stores. These stores were a hit! That is, until the market was tired of limited selections and other physical constrictions.

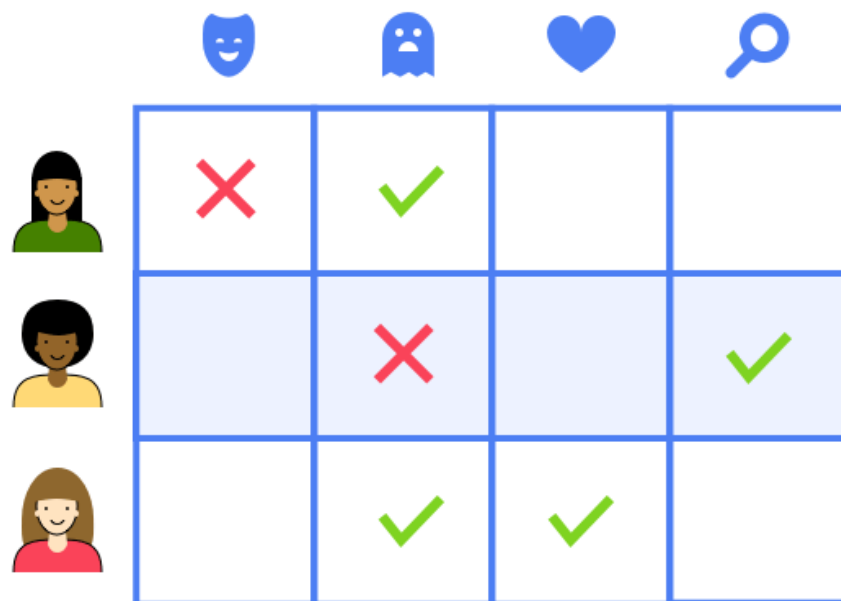
Today, online platforms like Netflix offer thousands of movies and shows. However, this much choice can be overwhelming for users! With over 7,000 movies and shows in the Netflix catalog, it is nearly impossible for users to find movies they'll like on their own. The large platform needs a recommendation engine algorithm to automate the search process for users.

There are multiple potential methods for creating a recommendation engine. The method you choose simply depends on the size of the user base, the size of the catalog, and the goals of the platform. A basic implementation of a recommendation engine would be the *editorial method*. In the editorial method, the platform would make recommendations based on a relatively small amount of individuals. Another easy one is the aptly named *simple collection method* where the platform makes suggestions based on the top products across the platform.

Netflix doesn't use those recommendation methods because they don't allow for personalization, or cover the breadth of the movie catalogs and user preferences. Instead, Netflix uses the *personalized method* where movies are suggested to the users who are most likely to enjoy them based on a metric like major actors or genre. Machine learning is necessary for this method because it uses user data to make informed suggestions. This way Netflix methodology accounts for the diversity in its audiences and its very large catalog.

Netflix is All Probability

Machine learning is able to create “smart” platforms because it uses probability to discover the likelihood of a user liking a product. To understand the probability aspect of recommendation engines, let’s look at an example of a *utility matrix*, a probability model which places a score on the relationship between a user and a movie type in order to predict their preferences.



The diagram illustrates a utility matrix for three Netflix users (Fatima, Maya, and Leslie) across four movie genres. The genres are represented by icons: a smiling face (Comedy), a ghost (Horror), a heart (Romance), and a magnifying glass (Mystery). The users are represented by avatars on the left. The matrix cells contain checkmarks (✓) for positive ratings and red X's for negative ratings. Cells with no rating are empty.

				
	✗	✓		
		✗		✓
		✓	✓	

In this example, we have three Netflix users: Fatima, Maya, and Leslie. Each user has watched a few movies on Netflix and rated them and each user has movies in the catalog they haven’t watched or rated yet. The user ratings are *utility scores* which represent the relationship between the movie and the user. In this example, our utility scores are represented by the checkmark and X symbols. The checkmarks represent

movies that they've seen and liked and the Xs represent movies they've seen and not liked. The question marks represent movies that they haven't seen yet.

Given this utility matrix, what do you think the recommendation engine will suggest?

As we've learned, in some recommendation engines, the probability of a user liking a movie is based on the previous movies that they've seen and liked. So, Netflix may suggest Fatima watch a who-done-it since she enjoyed the thrilling plot of a horror movie she watched last week. The engine may recommend more light-hearted films to Maya, like a comedy or romance, because she didn't enjoy the horror film. Because Leslie equally enjoyed a horror movie and a romance movie, Netflix will give her a more niche, curated experience, like romantic thrillers. Now each user will receive a suggestion that's personalized for them!

Summary

Machine learning algorithms are designed to work diligently to provide you with everything you want, and some things you don't even know you want yet. At Netflix, they even go beyond recommending movie titles and use recommendation engines to curate the preview images you're seeing on your feed. If you rated Johnny Depp movies highly, you're likely to see an image of Captain Jack Sparrow when Netflix recommends Pirates of the Caribbean to you. If you tend to like romance movies, your feed will likely have preview images of scenes with actors embracing one another.

Machine learning techniques at Netflix have completely disrupted the way television and movie industries operate. We have data science to thank for having personalized experiences like Netflix. The field is growing rapidly in this era of abundance so we have guidance when sorting through thousands of options to find the perfect products.

Data science in our daily lives gives us access to educated choices and more curated experiences. The computer power and know-how of data scientist results in seamlessly accurate decision-making. Think about that during your binge fest this weekend!

And if you want to learn how to build your own recommender engine, check out our [Data Scientist: Machine Learning Specialist career path](#) and other [data science courses](#).

PYTHON

- Hello World
-
- Control Flow
-
- Lists
-
- Loops
-
- Functions
-
- Python: Code Challenges (Optional)
-
- Strings
-
- Modules
-
- Dictionaries
-
- Files
-
- Classes
-
- Python: Code Challenges II (Optional)

Files

- # Reading a File
- # Iterating Through Lines
- # Reading a Line
- # Writing a File
- # Appending to a File
- # What's With "with"?
- # CSV File
- # Reading a CSV File
- # Writing a CSV File
- # Reading a JSON File
- # Writing a JSON File

Types

1. Call type() on the integer 5 and print the result
print(type(5))

2. Define an empty dictionary called my_dict
my_dict = {}

3. Print out the type of my_dict
print(type(my_dict))

4. Define a list called my_list
my_list = []

5. Print out the type of my_list
print(type(my_list))

Class

```
class Facade:
    pass

facade_1 = Facade()

facade_1_type = type(facade_1)
print(facade_1_type)
```

```
# Class Variables
class Grade:
    minimum_passing = 65
```

```
# Methods

class Rules:
    def washing_brushes(self):
        return "Point bristles towards the basin while washing your brushes."
```

```
# Methods with Arguments

class Circle:
    pi = 3.14

    def area(self, radius):
        return self.pi * radius ** 2

# Create an instance of Circle
circle = Circle()

# Calculate the area of the three circles
pizza_area = circle.area(6)    # 12 / 2 = 6
teaching_table_area = circle.area(18)    # 36 / 2 = 18
round_room_area = circle.area(5730)    # 11460 / 2 = 5730
```

```
# Constructors
```

```

class Circle:
    pi = 3.14

    # Add constructor here:
    def __init__(self, diameter):
        print("New circle with diameter: {}".format(diameter))

# Create a new Circle instance
teaching_table = Circle(36)

```

```

# Instance Variables

class Store:
    pass

alternative_rocks = Store()
isabelles_ices = Store()

alternative_rocks.store_name = "Alternative Rocks"
isabelles_ices.store_name = "Isabelle's Ices"

```

```

# Attribute Functions

can_we_count_it = [{'s': False}, "sassafrass", 18, ["a", "c", "s", "d", "s"]]

for element in can_we_count_it:
    if hasattr(element, "count"):
        print(str(type(element)) + " has the count attribute!")
    else:
        print(str(type(element)) + " does not have the count attribute :(")

```

```

# Self

class Circle:
    pi = 3.14
    def __init__(self, diameter):
        print("Creating circle with diameter {}".format(d=diameter))
        # Add assignment for self.radius here:

        self.radius = diameter / 2

```

```
def circumference(self):
    return 2 * self.pi * self.radius

medium_pizza = Circle(12)
teaching_table = Circle(36)
round_room = Circle(11460)

print(medium_pizza.circumference())
print(teaching_table.circumference())
print(round_room.circumference())
```

```
# Everything is an Object

# Checkpoint 1: Call dir() on the number 5
print(dir(5))

# Checkpoint 2: Define a function
def this_function_is_an_object():
    return "Yes, I am an object!"

# Checkpoint 3: Call dir() on the function
print(dir(this_function_is_an_object))
```

```
# String Representation

class Circle:
    pi = 3.14

    def __init__(self, diameter):
        self.radius = diameter / 2

    def area(self):
        return self.pi * self.radius ** 2

    def circumference(self):
        return self.pi * 2 * self.radius

    def __repr__(self):
        return "Circle with radius {radius}".format(radius=self.radius)

medium_pizza = Circle(12)
teaching_table = Circle(36)
round_room = Circle(11460)

print(medium_pizza)
```

```
print(teaching_table)
print(round_room)
```

Review

So far we've covered what a data type actually is in Python. We explored what the functionality of Python's built-in types (also referred to as *primitives*) are. We learned how to create our own

Preview: Docs Python is a strongly typed language. At runtime, it prevents typing errors and engages in little implicit type conversion.

[data types](#)

using the

Preview: Docs Used for defining classes in Python.

`class`

keyword.

We explored the relationship between a class and an object — we create objects when we instantiate a class, we find the class when we check the

Preview: Docs Loading link description

`type()`

of an object. We learned the difference between class

Preview: Docs Loading link description

[variables](#)

(the same for all objects of a class) and instance variables (unique for each object).

We learned about how to define an object's functionality with methods. We created multiple objects from the same class, all with similar functionality, but with different internal data. They all had the same methods, but produced different output because they were different instances.

Take a moment to congratulate yourself, object-oriented programming is a complicated concept.

Instructions

Checkpoint 1 Passed

1. Define a class named `Student` that will be our data model at *Jan van Eyck High School and Conservatory*.

Add a constructor for `Student`. Have the constructor take in two parameters: a `name` and a `year`. Save those two as attributes `.name` and `.year`.

Stuck? Get a hint

Checkpoint 2 Passed

2. Create three instances of the `Student` class:

Roger van der Weyden, year 10

Sandro Botticelli, year 12

Pieter Bruegel the Elder, year 8

Save them into the variables `roger`, `sandro`, and `pieter`.

Stuck? Get a hint

Checkpoint 3 Enabled

3. Create a `Grade` class, with `.minimum_passing` as an attribute set to `65`.

Stuck? Get a hint

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4. Give `Grade` a constructor. Take in a parameter `score` and assign it to `self.score`.

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5. In the body of the constructor for `Student`, declare `self.grades` as an empty list.

Checkpoint 6 Step instruction is unavailable until previous steps are completed

6. Add an `.add_grade()` method to `Student` that takes a parameter, `grade`.

`.add_grade()` should verify that `grade` is of type `Grade` and if so, add it to the `Student`'s

`.grades.`

If `grade` isn't an instance of `Grade` then `.add_grade()` should do nothing.

Checkpoint 7 Step instruction is unavailable until previous steps are completed

7. Create a new `Grade` with a score of `100` and add it to `pieter`'s `.grades` attribute using `.add_grade()`.

Checkpoint 8 Step instruction is unavailable until previous steps are completed

8. Great job! You've created two classes and defined their interactions. This is object-oriented programming! From here you could:

Write a `Grade` method `.is_passing()` that returns whether a `Grade` has a passing `.score`.

Write a `Student` method `.get_average()` that returns the student's average score. Add an instance variable to `Student` that is a dictionary called `.attendance`, with dates as keys and booleans as values that indicate whether the student attended school that day.

Write your own classes to do whatever logic you want!

```
class Student:
    def __init__(self, name, year):
        self.name = name
        self.year = year
        self.grades = [] # Step 5

    def add_grade(self, grade): # Step 6
        if isinstance(grade, Grade):
            self.grades.append(grade)
    # Optional extra method
    # def get_average(self):
    #     if not self.grades:
    #         return 0
    #     return sum(grade.score for grade in self.grades) /
len(self.grades)

class Grade:
    minimum_passing = 65 # Step 3

    def __init__(self, score): # Step 4
        self.score = score
```



```

# Optional extra method
# def is_passing(self):
#     return self.score >= self.minimum_passing

# Step 2: Create students
roger = Student("Roger van der Weyden", 10)
sandro = Student("Sandro Botticelli", 12)
pieter = Student("Pieter Bruegel the Elder", 8)

# Step 7: Add a grade
pieter.add_grade(Grade(100))

# You can check results like this:
# print(pieter.grades[0].score) # Should output 100
# print(pieter.get_average())   # If you enable the optional method

```

Next Steps

You've completed the Learn Python 3 course! What's next?

Congratulations, you've successfully completed the Learn Python 3 course! You got an introduction to fundamental programming concepts and the Python programming language syntax. More specifically, you learned about:

- How to write Python syntax to print and work with user input
- How to use Python control flow, boolean variables, and logic operators to create programs that can make dynamic decisions
- How to utilize the list data structure to store and manipulate ordered groups of data
- How to use loops to reduce code repetition and effectively execute a group of statements several times
- How to create functions to organize code into reusable blocks

How to work with the string data type to automatically create, rearrange, reassign, disassemble, and reassemble blocks of text

How code in Python is grouped and shared across a codebase via modules

How to utilize the dictionary data structure to organize information in a key-value format

How to work with files in an automated way

How the relationship between a class and an object works to create object-oriented code

Your learning journey into Python isn't over yet! There are plenty of other topics that you can dive into to continue learning. Here are our recommendations for the next steps:

Intermediate Python

Feeling comfortable with the basics of Python? Intermediate Python 3 is a stepping stone to take your skills to the next level. This course will provide you with the knowledge to make your Python code cleaner, more efficient, and more manageable. We'll also dive under the hood of fundamental concepts that will deepen your understanding of the Python language.

This course is a deep dive into the inner workings of some of Python's most popular features to teach you how to use the language's unique features to get closer to becoming a Python expert.

While completing Learn Intermediate Python 3, you'll learn about:

The various ways to utilize functions to create cleaner and more manageable code

The core pillars of one of the most popular programming paradigms - Object-Oriented Programming

How Python iteration works under the hood and how to create custom iterators using generator functions

How to use specialized Python collections as alternatives to Python's general-purpose built-in containers

How to better manage resources using context managers

How to use the Python Unittest library to test complex applications

[Get started now!](#)

[Pass the Technical Interview with Python](#)

You'll need to pass a technical interview if you want to be hired for a technical role. Don't worry — these interviews are pretty predictable, and the same kinds of problems appear again and again. Even if you don't have a technical interview scheduled just yet, practicing these common problems will help you grow as a programmer and problem solver, and will help you write cleaner more efficient code.

In this path, you'll get a broad overview of computer science concepts: data structures, algorithms, and common applications of both. You'll build implementations of each data structure and algorithm from scratch in modern Python.

[Jump into interview prep now!](#)

More Python!

Don't worry, if the above courses don't pique your interest, we have plenty of other options:

[Build Python Web Apps with Flask](#)

In this path, you will learn the foundations of Python so you can use it to create fully-featured, interactive web applications. Along your journey, you will learn how

to code in Python, design and access databases, create interactive web applications, and share your apps with the world.

Alternatively, try out [Build Python Web Apps with Django](#) another popular web framework for Python!

[Analyze Data with Python](#)

Data is everywhere. That means more companies are tracking, analyzing, and using the insights they find to make better decisions. In this Skill Path, you'll learn the fundamentals of data analysis while building Python skills.

[Build Chatbots with Python](#)

It turns out, you don't need to know linear algebra to make advanced chatbots with artificial intelligence. In this Skill Path, we'll take you from being a complete Python beginner to creating chatbots that teach themselves. Say hello to your next cutting-edge skill.

Contribute to Docs!

Want to build your portfolio and help others? Contribute to Codecademy Docs! You can go to our [Contribution Guide](#) to get started.

Exploring Data with Python

Sorting and Filtering Rows

Combining Booleans with And

The US Federal Government created an electric vehicle tax credit in 2010. What if we wanted a DataFrame of all electric vehicle models introduced after 2010?

We would need to combine two different comparisons:

```
vehicles['Year'] > 2010
```

Copy to Clipboard
and

```
vehicles['fuel'] == 'electric'
```

Copy to Clipboard

Boolean operators allow us to combine comparisons into a single True/False value.

Boolean operators: and

In English, we might say `This vehicle is electric and it was introduced after 2010`. The word `and` indicates that the statement is true only if both components are true:

the vehicle is indeed electric

the vehicle was in fact introduced after 2010

Python behaves exactly the same way: two comparisons can be combined with `and` to test if both are simultaneously true.

For example, we can combine `2011 > 2010` with `'electric' == 'electric'`, using parentheses `()` to mark the two components for readability:

```
(2011 > 2010) and ('electric' == 'electric')  
# Output: True
```

Copy to Clipboard

On the other hand, combining `2011 > 2010` with `'biodiesel' == 'electric'` produces `False` since the second comparison is false:

```
(2011 > 2010) and ('biodiesel' == 'electric')  
> False
```

Copy to Clipboard

And in pandas

Pandas also provides a version of `and`, although pandas uses the symbol `&`. Let's perform our combined comparison on the following mini DataFrame `df`:

year	fuel
2009	electric
2010	hybrid electric
2011	electric
2012	propane

While we can combine the comparisons on a single line, it is more readable to do this piece-by-piece:

```
is_electric = df['fuel'] == 'electric'  
year_gt2010 = df['year'] > 2010  
combined = is_electric & year_gt2010
```

Copy to Clipboard

Here's the output `combined` next to the original DataFrame:

Original DataFrame		Combined Mask
year	fuel	output
2009	electric	False
2010	hybrid electric	False
2011	electric	True
2012	propane	False

Lastly, we can select only the `True` rows from the DataFrame

```
df[combined]
```

Copy to Clipboard

producing

year	fuel
2011	electric

Usually, instead of creating a `combined` mask, we just combine the individual masks when we pass them to the DataFrame:

```
df[is_electric & year_gt2010]
```

Copy to Clipboard

Technically, we can perform the entire process within the `[]`, using parentheses to separate the individual comparisons:

```
df[(df['fuel']=='electric') & (df['year'] > 2010)]
```

Copy to Clipboard

But this can very quickly become challenging to read and debug. Unless we're particularly worried about efficiency, it can be helpful to use "extra" variables to build the filter one step at a time.

[Boolean](#) operators: or

Combining Booleans with Or

Our vehicle dataset has more than one type of electric vehicle:

electric

hybrid electric

plug-in hybrid electric

To analyze all electric vehicles, we might need to select rows that have any of these three.

Preview: Docs Represents the logical values of true and false.

[Boolean](#)

operators: or

In English, we might say `this vehicle is electric or hybrid electric`. This statement is

true if the vehicle is electric (the first property is true)

true if the vehicle is hybrid electric (the second property is true)

false only when the vehicle is neither electric nor hybrid electric

In Python, `or` behaves the same way: it outputs `True` as long as one of the combined comparisons is `True`.

For example, we can combine the comparison `'electric' == 'electric'` with the comparison `'biodiesel' == 'hybrid electric'`:

```
('electric' == 'electric') or ('biodiesel' == 'hybrid electric')  
# Output: True, since the first comparison is True
```

Copy to Clipboard

On the other hand, combining `'propane' == 'electric'` with `'biodiesel' == 'hybrid electric'` will be false, since neither comparison is true:

```
('propane' == 'electric') or ('biodiesel' == 'hybrid electric')  
> False
```

Copy to Clipboard

Pandas uses the symbol `|` instead of `or`. Take the following mini DataFrame `df`:

	fuel
0	electric
1	hybrid electric
2	plug-in hybrid electric
3	propane

Let's construct a Boolean mask that is `True` whenever `fuel` is `electric` or `hybrid electric`:

```
is_electric = df['fuel'] == 'electric'  
is_hybrid = df['fuel'] == 'hybrid electric'  
combined = is_electric | is_hybrid
```

Copy to Clipboard

The first two rows will be `True`, since each matches one of the comparisons. The last two, on the other hand, will be false.

<code>fuel</code>	<code>combined output</code>
<code>electric</code>	<code>True</code>
<code>hybrid electric</code>	<code>True</code>
<code>plug-in hybrid electric</code>	<code>False</code>
<code>propane</code>	<code>False</code>

Note that we can combine more than two options using `|`. For example, to include `plug-in hybrid electric`, we would add it to the comparisons:

```
is_electric = df['fuel'] == 'electric'
is_hybrid = df['fuel'] == 'hybrid electric'
is_plugin = df['fuel'] == 'plug-in hybrid electric'
combined = is_electric | is_hybrid | is_plugin
```

Copy to Clipboard

Lastly, we can filter the DataFrame down to only `True` rows by passing `combined` to `df`:

```
df[combined]
```

Copy to Clipboard

or by constructing the combined comparison within the brackets:

```
df[is_electric | is_hybrid | is_plugin]
```

Copy to Clipboard

In either case, we'd receive the filtered DataFrame

Inverting Booleans with Not

In the last exercise, we created a

Preview: Docs Loading link description

[Boolean](#) filter to find any rows involving electric fuel:

```
is_electric = df['fuel'] == 'electric'
is_hybrid = df['fuel'] == 'hybrid electric'
is_plugin = df['fuel'] == 'plug-in hybrid electric'
electric = is_electric | is_hybrid | is_plugin
```

What if we wanted all **nonelectric** vehicles? We could create a new statement using multiple `is`s. But it would be easier to use our pre-created filter.

Pandas uses the symbol `~` for `not/non`. In our example, we could create a new Boolean mask `not_electric` using the pre-existing mask:

```
not_electric = ~electric
```

LESSONS covered in this lesson, including

1. sorting DataFrames by columns and index
2. selecting rows from DataFrames using labels and position
3. comparing variable values using Python comparison operators
4. creating
5. Preview: Docs Represents the logical values of true and false.
6. [Boolean](#)
masks using pandas and comparison operators
7. combining comparisons using Boolean operators

8. filtering a DataFrame using a Boolean mask

Cleaning and Transforming Columns

Real datasets rarely come in exactly the format we need. So one of the first steps in data science is typically **data wrangling**: transforming raw data into an analytics-ready format.

In this lesson, we'll focus on transforming the *columns* of a raw dataset into columns we can use in analysis.

As an example, we've loaded a real dataset of National Park visitor counts in the Learning Environment. As data scientists, we might be interested in answering questions like:

- How has the number of visitors changed over the years?
- Which states have the most National Parks?
- Do larger parks tend to get more visitors?

Our dataset has enough information to answer these questions, but not without first performing some calculations and other data transformations. For example, we'll need to combine the information from the columns `Year2019` and `Year2020` to see if the number of visitors increased or decreased from 2019 to 2020.

In this lesson, you will learn how to:

- Create new calculated columns in a DataFrame

- Rename and drop columns
- Perform variable operations in Python and pandas
- Clean and modify text data
- Investigate columns with missing data
- Correct data type errors

Python for Data Science: Working with Data

Merge - pd

Inner Merges

Pandas Syntax

The syntax in pandas is

```
pd.merge(left = X,  
         right = Y,  
         left_on = 'Category',  
         right_on = 'Category',  
         how = 'inner')
```

where

`left` is the name of the left DataFrame

`right` is the name of the right DataFrame

`left_on` is the column from the left DataFrame that will be matched to

`right_on`

`right_on` is the column from the right DataFrame that will be matched to

`left_on`

`how` tells pandas which type of merge to use

Left Merges

Pandas Syntax

The syntax for a left merge is almost identical to an inner merge. We just need to change the `how` parameter:

```
pd.merge(left = X,  
         right = Y,  
         left_on = 'Category',  
         right_on = 'Category',  
         how = 'left')
```

Right Merges

Pandas Syntax

The syntax for a right merge is the same as a left merge, but with `how = 'right'`:

```
pd.merge(left = X,  
         right = Y,  
         left_on = 'Category',  
         right_on = 'Category',  
         how = 'right')
```

Aggregating Data - pd

Right Merges

Not Tidy Data

The titles of some columns form a group and should be encoded as a characteristic (Location). One characteristic (Alone) is encoded in multiple columns.

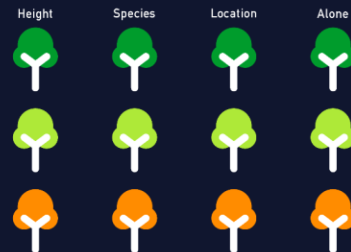


Tree Census (not tidy)

ID	Height (ft)	Species	City	Highway	Undeveloped
19433	15.00	London Plane	Alone	nan	nan
13132	24.70	Silver Maple	nan	nan	Group
12034	26.30	Silver Maple	nan	nan	Group
20143	6.70	American Linden	nan	Group	nan
11246	5.90	Tulip	Group	nan	nan
11584	6.10	Honeylocust	nan	Group	nan
13280	14.60	Red Oak	nan	Alone	nan
11562	14.10	Red Oak	nan	Group	nan

Tidy Data

Every column represents a variable, and every variable appears in only one column.



Tree Census (tidy)

ID	Height (ft)	Species	Location Type	Single	Prettiness
12139	10.40	Pin Oak	Undeveloped Area	0	2
13281	11.60	Honeylocust	City	1	5
17461	19.10	Pin Oak	Highway	1	3
13132	24.70	Silver Maple	Undeveloped Area	1	1
11246	5.90	Tulip	City	0	2
11562	14.10	Red Oak	Highway	0	1
12438	5.40	Green Ash	Undeveloped Area	0	5
11584	6.10	Honeylocust	Highway	0	3

Tree Census

ID	Height (ft)	Species	Location Type	Single	Distance (ft)	Prettiness
11246	5.90	Tulip	City	0	nan	3
11562	14.10	Red Oak	Highway	0	nan	5
11584	6.10	Honeylocust	Highway	0	nan	3
12139	10.40	Pin Oak	Undeveloped Area	nan	3.10	3
13280	nan	Red Oak	Highway	nan	nan	2
13281	nan	Honeylocust	City	1	3.20	5
15156	9.50	nan	Undeveloped Area	0	nan	3
19325	12.20	Tulip	City	0	nan	nan
20143	6.70	American Linden	nan	0	nan	nan
21110	27.50	London Plane	City	1	3.30	1

Statistical Thinking

Introduction

Let's imagine we are working for the city government of the fictional city of Melody Metropolis. The mayor of Melody Metropolis wants to know more about the musicians who currently live in the city. The learning environment shows a dataset we have on musicians living in the city as of last year. How would you describe this dataset? See if you can answer any of the following questions:

What does a typical musician's income look like?

Is there a wide range of musician ages?

What proportion of the musicians in the dataset play guitar?

We can try to make generalizations by looking over the rows and columns, but it's difficult to answer these questions precisely. We need some kind of "data vocabulary" that can help us measure and describe the

Preview: Docs Loading link description

[variables](#)

in the dataset. *Summary statistics* can be used for exactly this purpose!

With a basic understanding of summary

Preview: Docs Loading link description

[statistics](#)

, we can communicate and understand a lot more specific information about the musicians in the city. But learning statistics is often associated with a lot of negativity:

Memorization of lots of math formulas

Long calculations done by hand

Confusing or meaningless interpretations

None of these struggles need to be part of learning to use statistics. In this lesson, we'll gain a conceptual understanding of how summary statistics can easily help us communicate and interpret our dataset.

Instructions

1. Before moving to the next exercise, familiarize yourself with the following names and descriptions of the variables in the dataset:
 - `age`: age in years
 - `income`: yearly income in US dollars
 - `title`: primary job title
 - `experience`: years of experience in the field of music
 - `instrument`: primary instrument
 - `band`: whether in a band (1 = `yes`, 0 = `no`)

Musician Dataset

[Code ▾](#)

age	income	title	experience	instrument	band
40	40840	performer	13	guitar	1
54	21689	composer	11	voice	1
37	43153	composer	28	piano	0
47	58113	composer	31	violin	0
52	24518	producer	13	guitar	1
52	43574	composer	27	drums	1
58	27637	performer	9	violin	1
57	25707	performer	8	piano	1
37	26169	composer	8	piano	0
28	30852	educator	9	guitar	1
65	48403	composer	22	guitar	1
51	40874	educator	15	guitar	0
49	29925	performer	9	piano	1
37	20350	manager	9	voice	1
38	25049	performer	17	drums	1
41	33209	producer	29	voice	1
37	35711	performer	14	piano	0
40	61691	performer	28	saxophone	1
43	24854	performer	6	voice	1
57	24583	performer	31	drums	0
39	45258	performer	24	violin	1

Statistical Thinking

Describing Categorical Variables

To start our summary for the mayor, let's describe some of the categorical variables in the musician dataset — those variables that contain qualitative information on the city's musicians. First, let's look at information about the `title` variable, which tells us the job title each musician holds.

The following table shows:

- **frequency:** the count of musicians for each job title
- **proportion:** the frequency divided by the total number of musicians
- **percentage:** the proportion converted from a decimal to a percentage

Job Title Categories

TITLE	FREQUENCY	PROPORTION	PERCENTAGE
performer	333	0.35	35%
manager	113	0.12	12%
producer	87	0.09	9%
educator	239	0.25	25%
composer	186	0.19	19%
TOTAL	958	1.00	100%

From the table, we can learn about the different job titles of musicians in the city. There are 333 performers out of a total of 958 musicians. The proportion of performers is $333 \div 958 = 0.35$. To make this even easier to understand, we can convert the proportion to a percentage by multiplying it by 100: 35% of musicians in the city are performers.

We can also compare one category to another by checking the ratio of their frequencies. For example, there are far fewer managers than performers. Their ratio is 333 performers to 113 managers, which can be simplified by dividing: $333 \div 113 = 2.95$. This means there are almost 3 performers for every manager in the city.

Describing Numeric Variables

Now that we've learned about some of the categorical

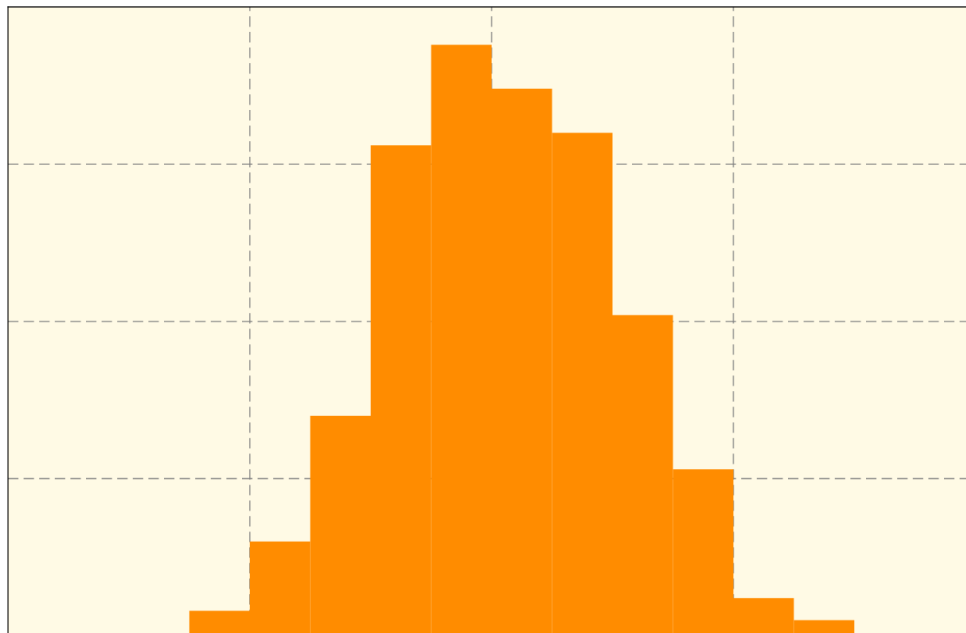
Preview: Docs Loading link description
[variables](#)

in our musician dataset, it's time to explore some numeric variables — those with quantitative data. There are a lot of ways we can describe the **distribution** of a numeric variable. A distribution is a function that shows all possible values of a variable and how frequently each value occurs. This may sound pretty technical, but visualizing the distribution can make it easy to understand.

In the learning environment, the distribution of musician ages is plotted with age on the x-axis and frequency on the y-axis. From this plot, we can see:

- Ages range from about 15 to 70.
- There are few musicians under 30 or over 50 years old.
- There are a lot of musicians between the ages of 30 and 50.

This distribution might be considered bell-shaped or hill-shaped and symmetrical. This is actually a very common pattern and is called a **normal distribution**.



Viewing a plot or knowing a variable is normally distributed gives us some general information, but still nothing specific. We need exact measurements to describe where the center of the distribution is and how wide the values are spread away from that center. There are several sets of

Preview: Docs Loading link description

[statistics](#)

we may use for these measurements, and we will need to know when to use which combination.

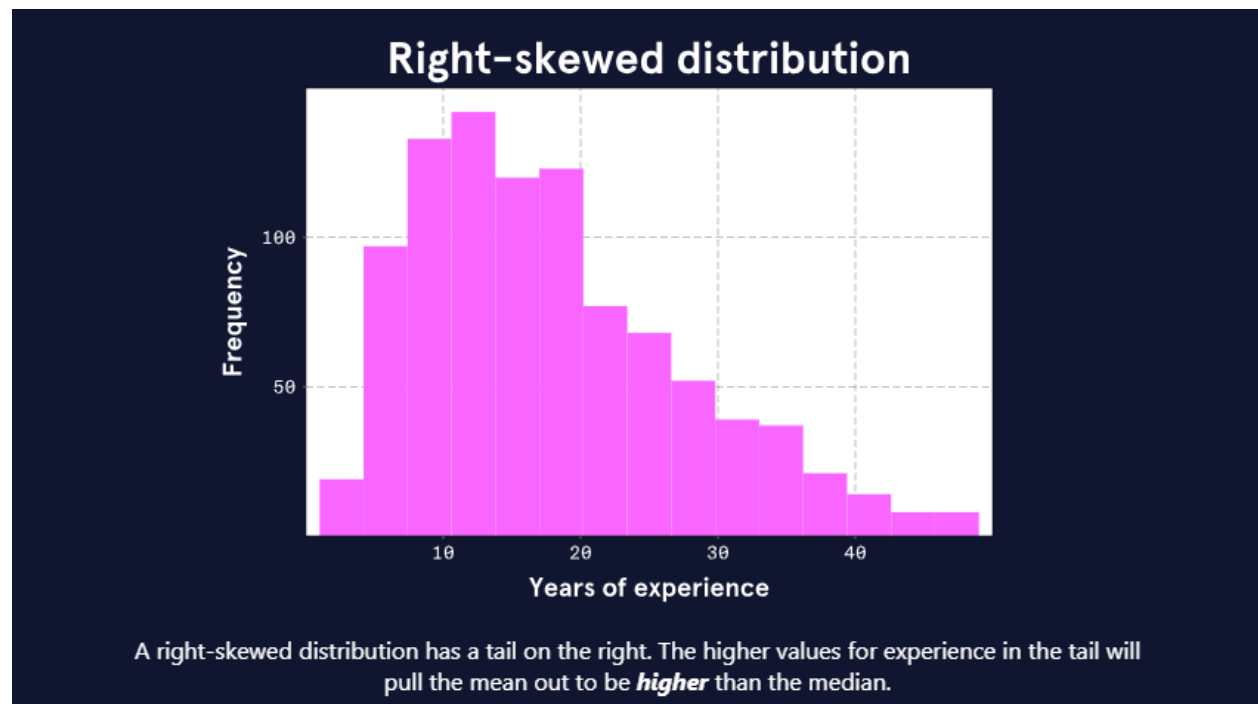
Skewed Distributions

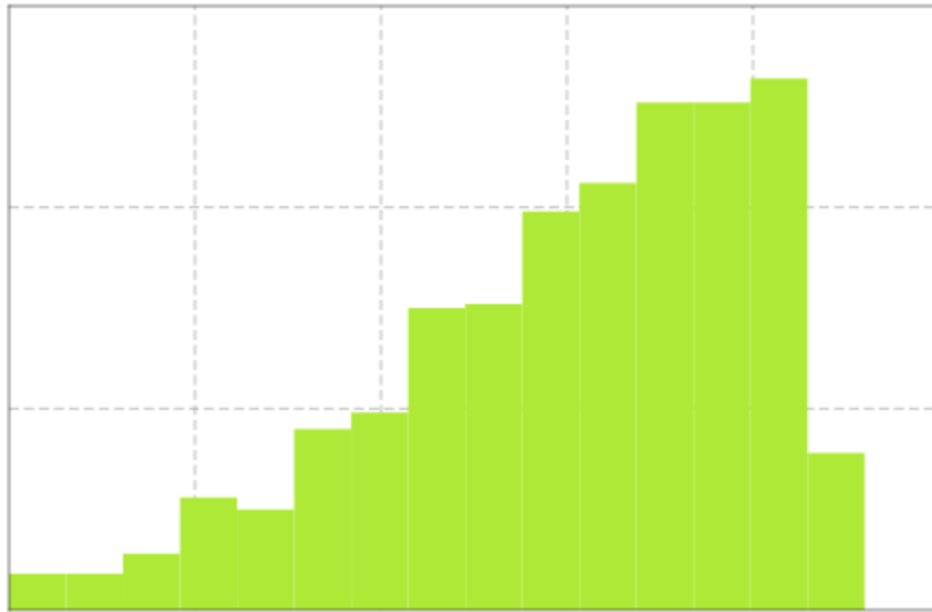
As we're moving through the numeric

Preview: Docs Loading link description

[variables](#)

in our musician dataset, we come across some interesting details when we inspect the income variable.





We notice that the shape of the distribution is different than the shape of the age distribution. There are quite a few musicians with higher incomes that are creating a longer tail on the right side.

We also notice that the mean indicates that the typical income is \$34,795. This value seems a little high since most of the incomes seem to be between \$15,000 and \$40,000.

What we have learned is that the income distribution is skewed. A **skewed** distribution is asymmetrical with a steep change in frequency on one side and a flatter, trailing change in frequency on the other. Specifically, the income distribution is right-skewed (also called positively-skewed) because the tail is on the right side.

So why does the mean seem wrong? Remember, the mean is the sum of all the values in the dataset divided by the total count. That sum is made very large by all the higher incomes in that right tail. This makes the mean a greater number than we would like it

be. When the data are skewed, the mean may not be the best measure of a typical observation.

There are a number of ways to deal with this issue. We will handle the problem with the income data by taking some alternative measurements.

Median and IQR

Let's find an alternative measure to the mean. We want to find a value that represents the typical musician income, but we don't want to use the actual values in the computation because the data are skewed.

One

Preview: Docs Loading link description

[method](#)

would be to find the middle value when all values are arranged from smallest to largest. This value is called the **median**, but it's also referred to as the 50th percentile or the second quartile (Q2).

Let's look at some simple data in the learning environment where the median (Q2) is 13. Half the data points are less than 13, and half are greater than 13.

These data span 22 values, ranging from 6 to 28. We could use this as our measure of spread, but what if the highest number wasn't 28 but 280? The median would still be 13, but now the range is 274 (280-6), which doesn't tell us a lot about the bulk of the data.

A better measurement might be the **interquartile range (IQR)**. A quartile is simply a marker for a quarter (25%) of the data.

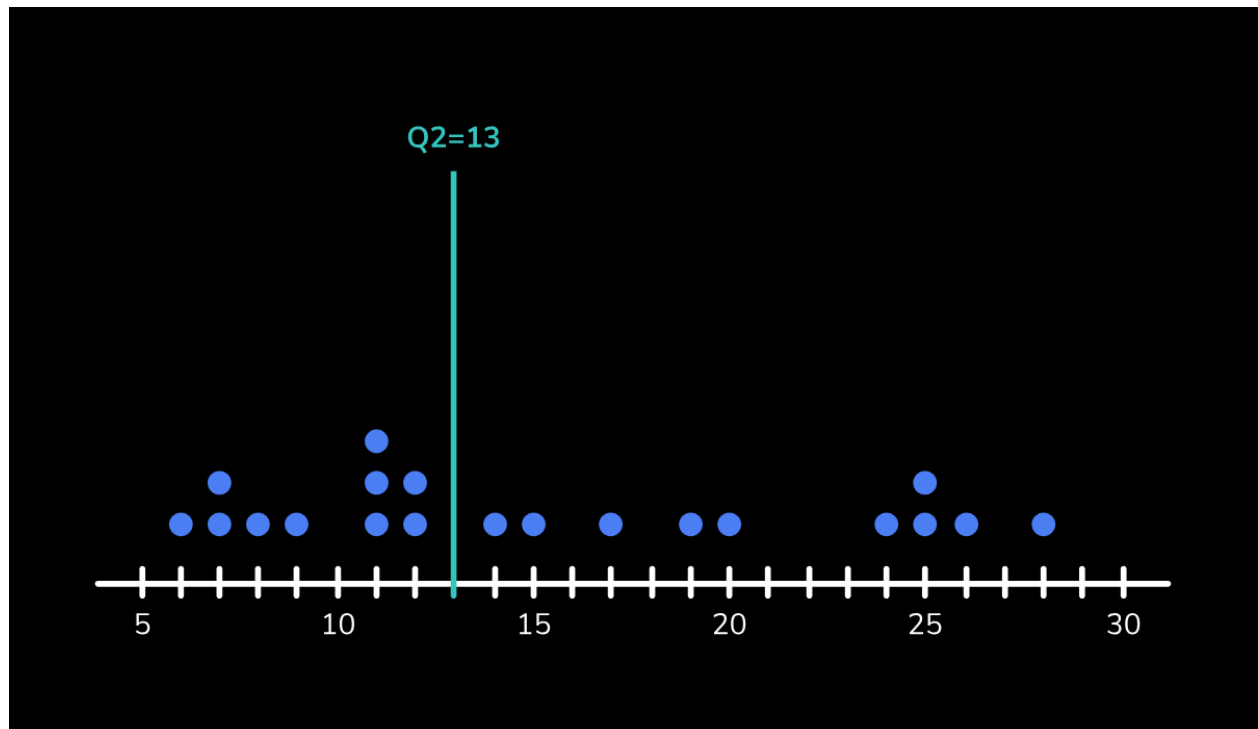
The first quartile marks 25% (Q1 = 10).

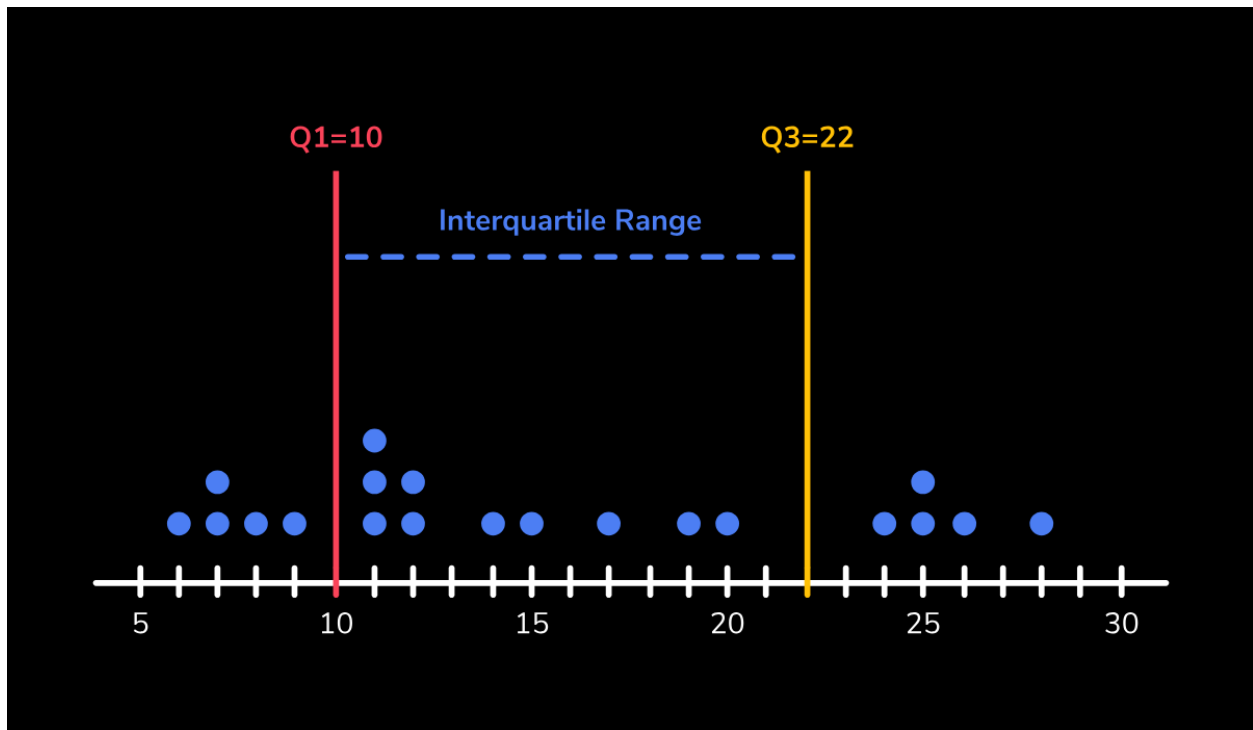
The second quartile marks 50% (Q2 = 13 — the median)

The third quartile marks 75% ($Q3 = 22$)

The IQR is the difference between $Q3$ and $Q1$ ($22 - 10 = 12$), marking the range for just the middle 50% of the data.

Let's find out how the median and IQR work out for our income data.





This looks better — the median of \$32,978 is lower than the mean of \$34,795 and seems more typical.

Outliers and Robust Measures

For our income data, the difference between the mean (\$34,795) and median (\$32,978) was only about \$2,000. You may be wondering: Is the difference ever larger?

Let's imagine some very famous celebrity musicians have all decided to move to Melody Metropolis. We know celebrities make much more money than the typical musician in our dataset. In the learning environment, we've added three new incomes to the distribution:

\$48 million: Paul McCartney, British musician of the Beatles

\$57 million: BTS, South Korean K-pop band

\$81 million: Beyoncé, American singer-songwriter

The second plot shows that the median appears almost unaffected by the addition of these three gigantic incomes: the median moves from \$32,978 to \$33,011. However, the mean makes a drastic change from \$34,795 to \$228,235. The mean is now well beyond even the maximum in the original distribution. An income of \$228,235 is definitely not a great measure of the center of our income distribution.

These celebrity incomes are examples of outliers, extreme values that are distant from the rest of the distribution. Just as with skewness, outliers tend to more heavily influence the mean than the median. This same pattern occurs with measures of spread: the standard deviation is more influenced by outliers and skewness than the interquartile range (IQR).

Because the median and IQR are NOT heavily influenced by extreme values, we say they are robust. Robust

Preview: Docs Loading link description

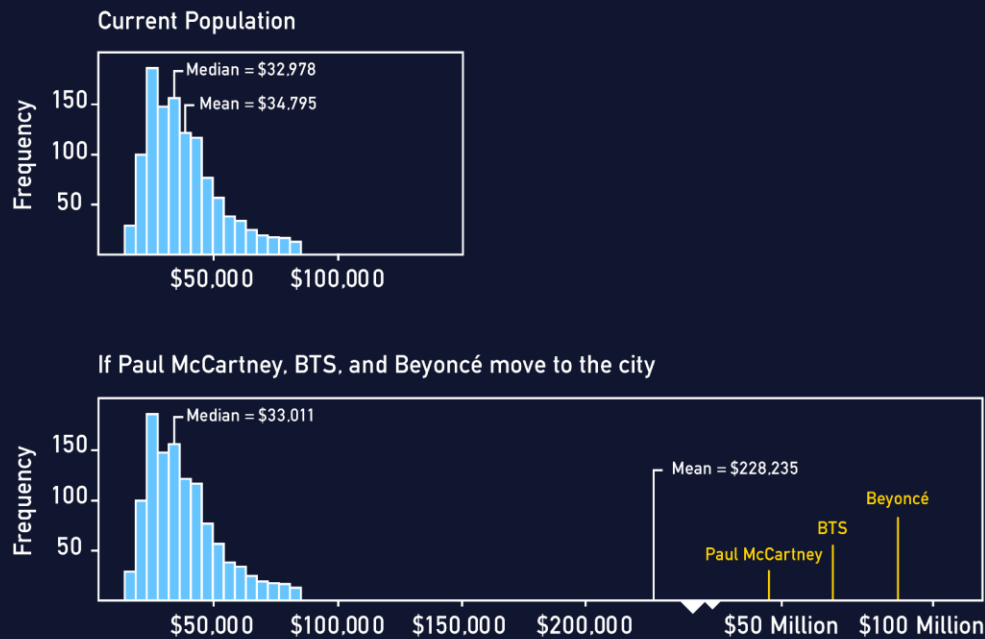
[statistics](#)

are often a better choice to measure the center and spread of a distribution that is skewed or has outliers.

Source:

[The World's Top-Earning Musicians Of 2019](#)

Income Distribution



Aggregate Data

One measure that we haven't covered that is usually talked about alongside the mean and median is the mode. The mode is defined as the value with the highest frequency, but we can also think of the mode as the value where the peak of the distribution occurs. While not great for computations, the mode can help us identify interesting features in a variable.

For instance, there might be more than one mode, such as in our distribution of years of experience. In the following plot, we can see there's one peak near the 10-year mark and another near the 30-year mark. We would call this distribution *bimodal* because it has two modes.

Sometimes bimodal distributions occur when there are differences across categories of another variable. Given that the city seems to have a lot of young people in bands, let's see if this pattern is reflected when we find the mean of each category of the `band` variable.

These means are very different and very close to the locations of the modes in our plot. This indicates that there may be some differences in experience level between these two groups that are showing up in our distribution plots as two peaks.

By making this separation and then summarizing with the mean, we have **aggregated** our data. In this case, we have aggregated by summarizing a numeric variable (`experience`) across each value of a categorical variable (`band`).

We have aggregated some other data in tables in the learning environment. Do you see any interesting patterns?

Aggregate Statistics

Income By Title

TITLE	MEDIAN
Performer	\$40,387
Manager	\$27,984
Producer	\$36,897
Educator	\$29,387
Composer	\$35,027

Income By Band Membership

BAND	MEDIAN	IQR
Yes	\$31,683	\$22,234
No	\$32,370	\$15,984

Experience By Instrument

INSTRUMENT	MEAN
Voice	19 Years
Guitar	14 Years
Piano	34 Years
Drums	10 Years
Saxophone	28 Years
Violin	36 Years

Variable Relationships

Aggregating data is a way of exploring variable relationships. We specifically looked at relationships between a numeric variable and a categorical variable, but we should also examine relationships between two numeric

Preview: Docs Stores data that can be manipulated and referenced throughout the code.

[variables](#)

.

For example, we might wonder: Does musician income vary with years of experience?

To start, we can take a look at a **scatter plot** with experience on the x-axis and income on the y-axis. Each point in the plot represents a musician, and the coordinates of that point are the musician's experience (x) and income (y).

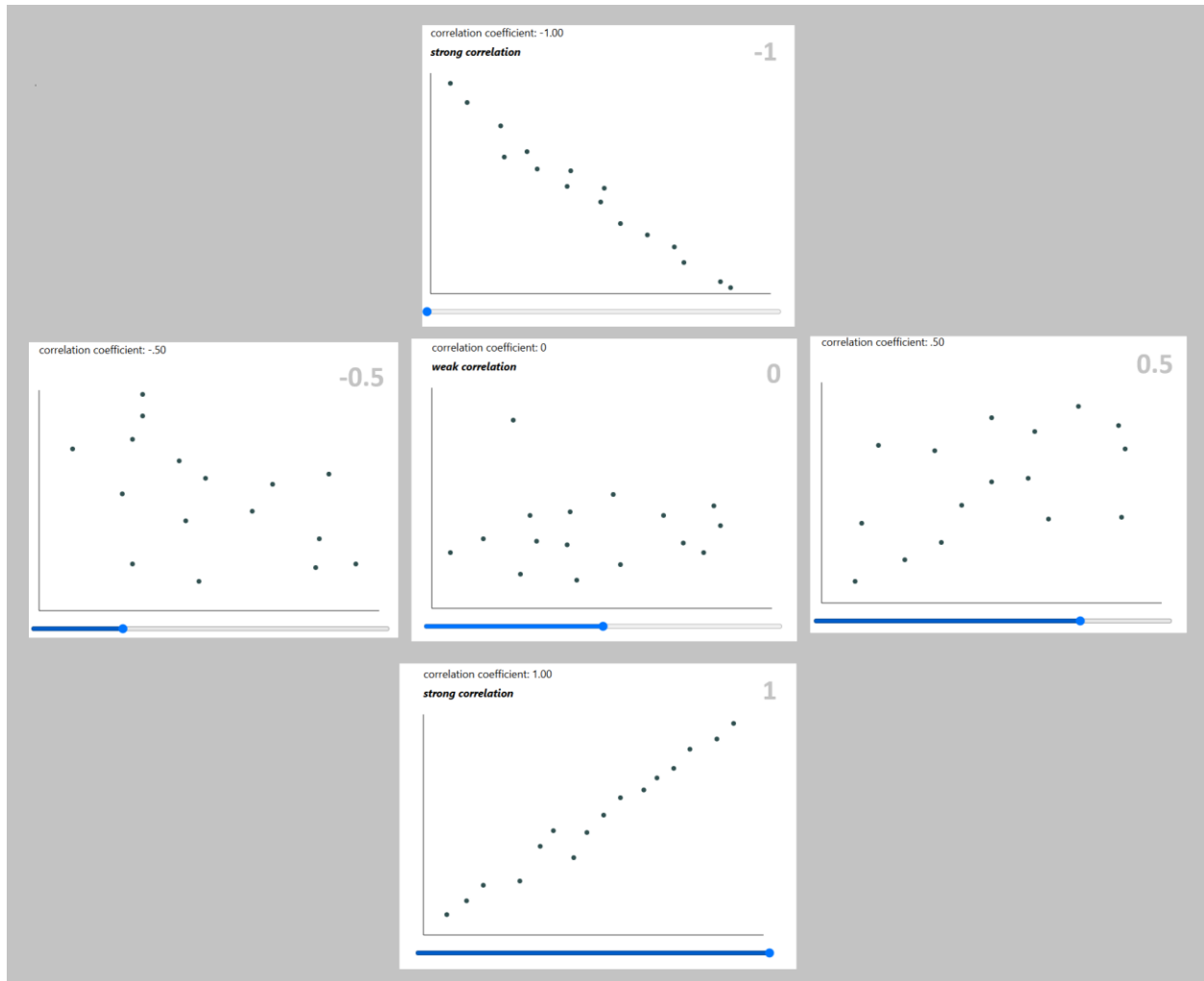
The cloud of points in the plot has a pattern. The points move from the lower left to the upper right part of the plot. In other words, lower levels of experience tend to be associated with lower incomes, and higher levels of experience tend to be associated with higher incomes. The points don't form a perfect line though — there is some variation.

We can describe this relationship more precisely by measuring the **correlation coefficient**. This number ranges from -1 to +1 and tells us two things about a linear relationship:

- **Direction:** A positive coefficient means that higher values in one variable are associated with higher values in the other. A negative coefficient means higher values in one variable are associated with lower values of the other.

- **Strength:** The farther the coefficient is from 0, the stronger the relationship and the more the points in a scatter plot look like a line.

The correlation coefficient for income and experience is 0.74 – the relationship is positive and moderately strong.



Movie Statistics Project

Explore Netflix data with your new understanding of summary statistics!

Overview

In this project, you'll practice using summary statistics from real data.

You will:

As an added bonus, some toggle prompts are labeled “*Just for Fun*” and don't have to be answered. These prompts provide extra interesting details about the data for your enjoyment!

Motivation

You just got a very cool job in the film industry! Your first assignment is to do some research on the content produced by streaming services in the last few years. You're putting together information for a report on the kinds of films being produced as well as any patterns that might be worth exploring further.

Dataset

You decide to start your research by exploring some data about films and documentaries produced by Netflix. The dataset you'll be using is a modified version of one found on the website [Kaggle](#). Your dataset includes 503 films with the following variables:

title: title of the film

`genre`: genre of the film

`language`: primary language of the film

`year`: year the film premiered

`runtime`: length of the film in minutes

`score`: film rating of 1 to 10 (worst to best) from the website [IMDb](https://www.imdb.com/)

Individual Variables

Language

You decide to start with the `language` variable. The table that follows gives the count of films in each language.

Using the table, try answering the following individual questions. Then use your answers to write up a brief summary about the primary languages of the films.

There are clearly a lot of films that have English as their primary language. Of the 503 films, what proportion have English as their primary language? (Click to Toggle Correct Answer)

What is the ratio of English-language films to films in a single language that is NOT English? (Click to Toggle Correct Answer)

What proportion of the films has multiple primary languages? (Click to Toggle Correct Answer)

Free response

Based on the questions you've just answered, write a few sentences summarizing what you found out about film languages. Give all proportions as percentages to help readers of your research report better understand your findings.

Submit Response

Stuck? Get a Hint

Just for Fun: About 4% of the films have multiple primary languages. Find out which genres tend to have this interesting feature. (Click to Toggle the Information)

IMDb Score

You are excited to take a look at all the IMDb scores! Are most films rated around the same score? Are there some extremely low or high scores?

Mean: 6.3

Standard Deviation: 1.0

Free response

Using the plot and summary statistics, describe the distribution of IMDb scores.

Submit Response

Stuck? Get a Hint

Just for Fun: Find out which films got the highest and lowest IMDb scores! (Click to Toggle the Information)

Genre

Your colleague is helping you with your research. They will be writing up the summary for the `genre` variable.

Free response

Which summary statistics might your colleague include in their summary of the film genres?

Submit Response

Stuck? Get a Hint

Runtime

Your colleague used analytics software to create a summary of the `runtime` variable. The software program has a default setting for numeric variables that outputs a

distribution plot, the mean, and the standard deviation. The analytics software produced the following plot and statistics for the runtimes:

Mean: 92.5

Standard Deviation: 28.4

Free response

Based on the distribution plot, what concerns might you have with the default summary statistics used by the analytics software?

Submit Response

Stuck? Get a Hint

Free response

Which alternative statistics could your colleague use in this case?

Submit Response

Stuck? Get a Hint

Just for Fun: Based on the distribution plot, what is your guess for the median and IQR of runtimes? Find out if your guess was close! (Click to Toggle the Median and IQR)

Just for Fun: Find out which movie has that really high runtime. (Click to Toggle the Information)

Relationships

Next, you want to explore some relationships between variables. Aggregating data across genres and languages may give you some insights.

IMDb Score by Genre

You want to know if any genres have particularly high IMDb scores, so you look at a table of means and standard deviations of IMDb scores for each genre.

Free response

Describe what you learn about IMDb scores across genres from the means and standard deviations in the table.

Submit Response

Stuck? Get a Hint

Runtime by Language

You are wondering if there are any differences in the length of films across languages.

Free response

Describe what you learn about film runtimes across languages from the means and standard deviations in the table.

Submit Response

Stuck? Get a Hint

Runtime and IMDb Score

For your final exploration, you've just added the following plot and summary statistic to your report.

Correlation Coefficient: 0.92

Your colleague is concerned there may be an error. The plot and the correlation coefficient don't match.

Free response

What does the plot tell you about the relationship between runtime and IMDb score?

Submit Response

Stuck? Get a Hint

Free response

What does the correlation coefficient tell you about the relationship between runtime and IMDb score?

Submit Response

Stuck? Get a Hint

Just for Fun: The correlation coefficient was incorrect. What do you think the true correlation coefficient is? Find out if your guess is correct! (Click to Toggle the Information)

Final Thoughts

You finished your first movie research report! You did a lot of exploring and learned a lot about the different features of Netflix content. These findings could be the final step or the starting point of a deeper analysis. Great job!

Back

Next

-

Data Visualization Basics

Introduction

Welcome!

The field of data visualization has taken off in the last twenty years or so. People in all kinds of professional fields make data viz – from news media to business analysis – and most of us consume multiple visualizations per day, whether we notice it or not!

Data visualizations are how we communicate data. We don't read the numbers off a spreadsheet or list every number in a trend to communicate a point – we make a visualization to show them.

In this module, we'll cover two major ideas:

We visualize data to accomplish a goal – to explore and see something new in the data, to strengthen an

Preview: Docs An argument is the actual value of a variable passed into a function.

[argument](#)

with a visual aid, to share a data-driven idea with someone else, and more.

Our design decisions can either help or hinder how effectively a data viz communicates. We tell stories with data viz (yes, even if it's a simple visualization) using good design choices in chart type, color, labels, context, and more.

There is so much we could say about data viz! It definitely won't all fit here though, so in this Data Viz Basics lesson, we're only talking about visualizations that are static (no moving parts or animations) and fixed (no interactivity).

From Data Type to Chart Type

The first step of making a data visualization is choosing a chart type. Chart type isn't our only tool when it comes to visualizing data, but it's an important one for communicating about the relationship we want to show.

In this context, a "relationship" in the data could mean something like...

- "the shop's sales of Gouda were higher in 2021 than any year since 2006"
- "30% of people ordered pizza with pineapple"
- "most people in the sample have a shoe size between 6 and 10.5"
- "as temperature increases, ice cream sales increase"

The first example is a change over time – that can be perfect for a line chart or a bar chart.

The second example compares a part to the whole: 30% of people got pizza with pineapple, out of 100% of people who ordered pizza. A pie chart is the classic (sometimes controversial) choice, but newer options include waffle and donut charts. Yum!

The third example is a distribution – the spread of data points in one variable. A histogram is the classic choice for visualizing a distribution.

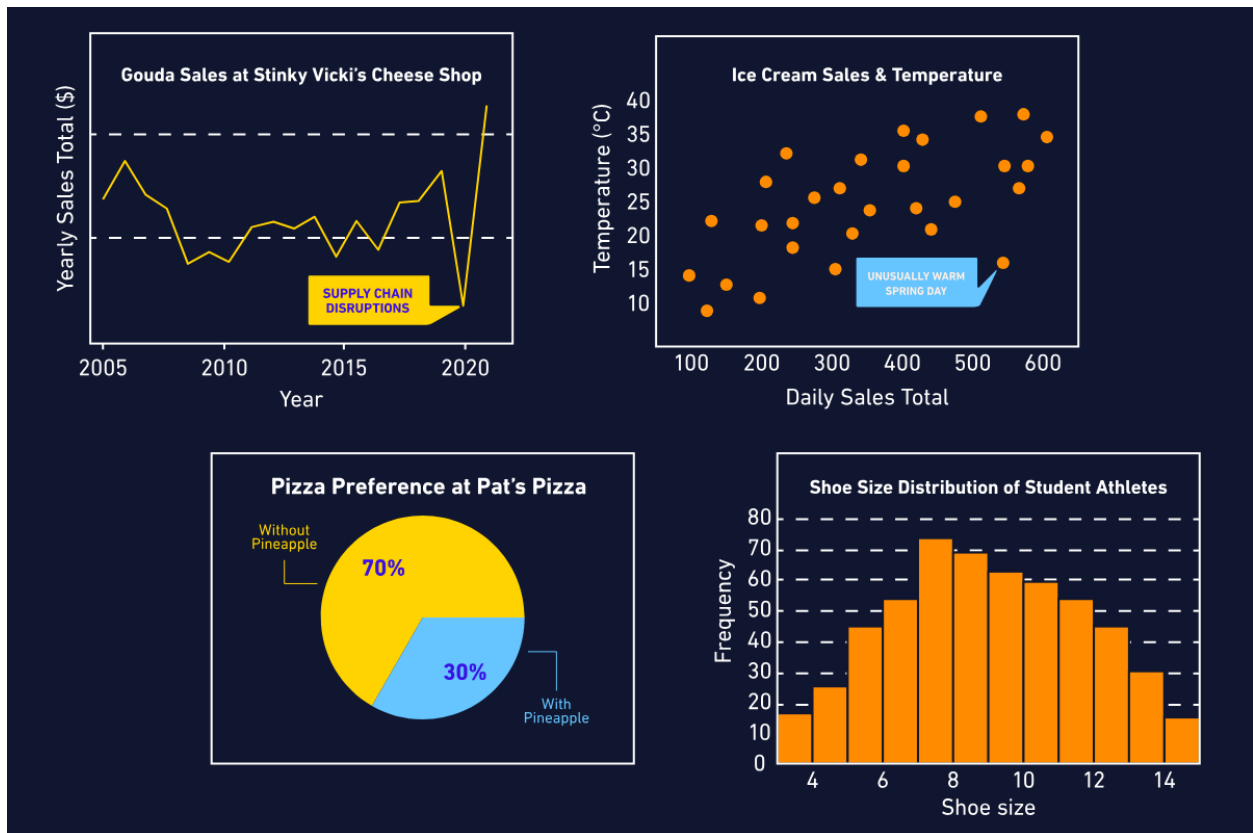
The fourth example is a direct comparison of two

Preview: Docs Loading link description

[variables](#)

to help understand a trend. This is perfect for a scatterplot, with or without a trend line. There's often more than one possible chart we can use for a dataset. But different charts emphasize different questions, arguments, or relationships in the data, and whichever we choose should help translate that data relationship into a visual relationship.

Next up, we'll get to know these charts a little better.



Univariate Charts

One big consideration when choosing a chart type is how many

Preview: Docs Loading link description

[variables](#)

we're comparing. **Univariate** charts help us visualize a change in **one variable**. Often that means measuring "how much," which can either be a **count** or a **distribution**.

A common chart for counts is the bar graph. If we want to compare an amount between different categories, like "how many of each coin is in the piggy bank" or "how many birds were saved by species," a bar chart translates the difference in count to a difference in bar height. Remember, **the data relationship is translated to a visual relationship**.

Another common univariate chart is the histogram. Histograms measure the distribution or spread, of a variable.

Histograms are a great way to show the concept of a normal (or skewed) distribution.

We can visualize the answer to questions like...

“how does foot size vary across the population?”

“what is the distribution of pregnancy length across the human population?”

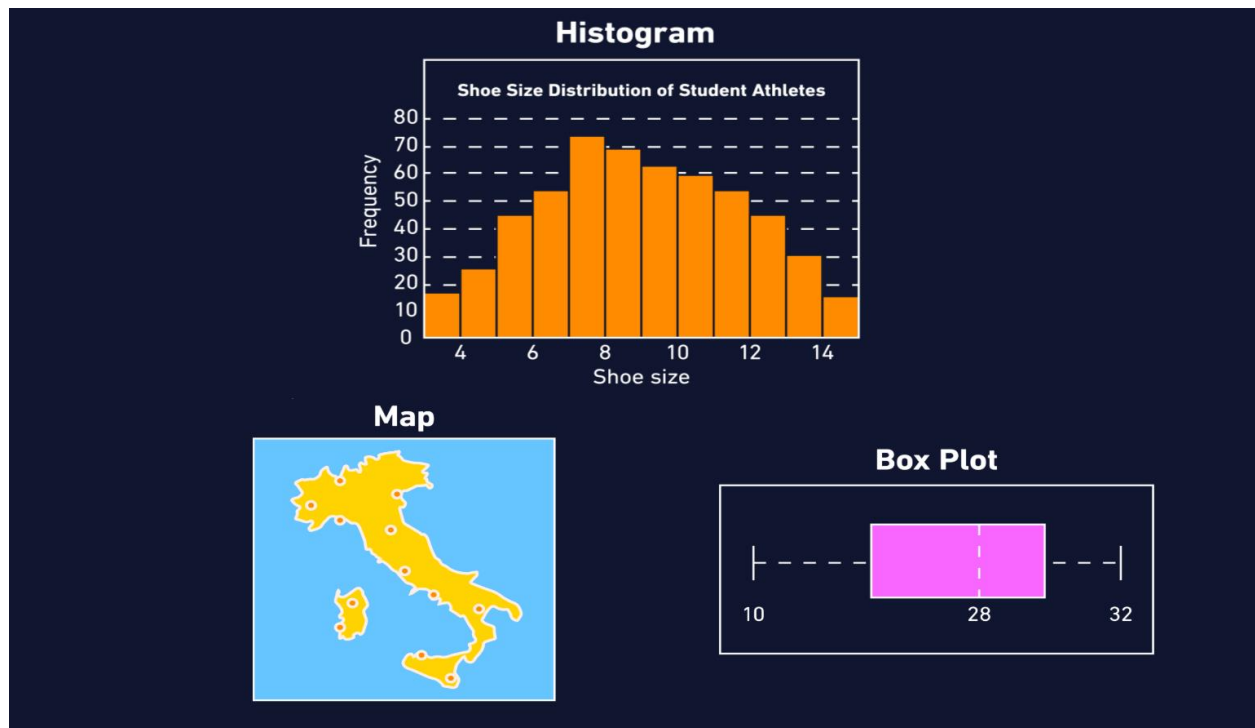
“how is income distributed in my country?”

A density curve also visualizes a distribution, without putting data in bins like a histogram does.

A more “math-forward” way to visualize distributions is a box plot or violin plot. These visualizations make percentile and quartile values obvious.

Last up, outside the counting and distribution category, let’s consider a univariate map.

This would be a map where the only variable is geographic, i.e. a map that just shows us location and distance.



Bi- and Multivariate Charts

Next up, bivariate and multivariate charts! These charts show the relationships between two or more

Preview: Docs Loading link description
[variables](#)

.

The classic bivariate example is the scatter plot – one variable on the x-axis, another on the y-axis, and each point helps us compare the two variables by its position on the graph.

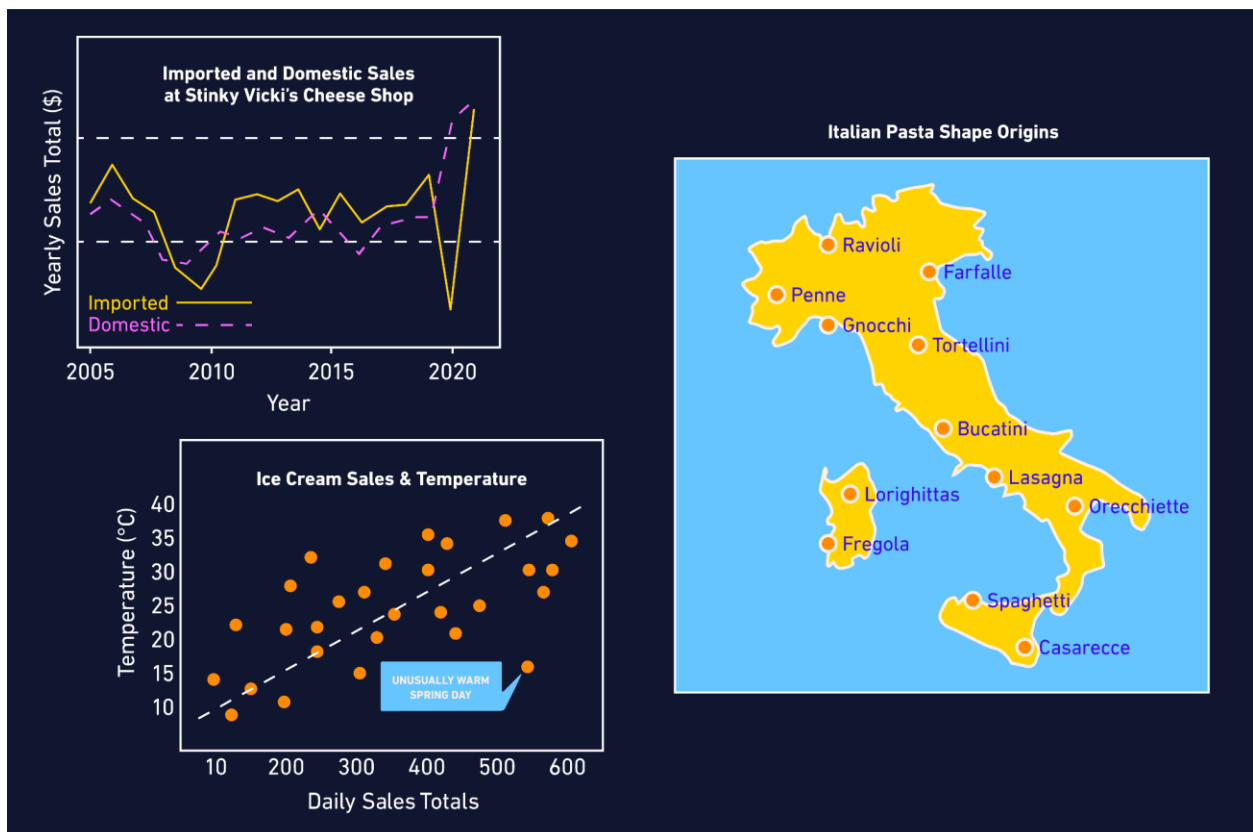
Scatterplots translate the relationship between two variables in the data into an easy-to-see spatial relationship. Because we're relying on the idea that each variable increases as we move up the X or Y axis, the scatterplot only makes sense for numeric variables, not categorical.

A line chart is another common bivariate chart, often measuring a variable changing over time. A stock chart, for example, measures the value of a company over time.

A line chart with multiple lines for different variables is a multivariate chart. For an example, check out the line chart that plots both imported and domestic cheese sales.

Last but not least, let's think about a bivariate map. It shows a basic geographical map plus an additional variable — this example shows roughly where different pasta shapes originated in Italy. We can also map precipitation, altitude or depth, median income, museum locations, or combinations of variables... the list is endless.

Charts often rely on visual signifiers besides chart type to visualize additional variables in the data. For example, the lines on a multivariate line chart are distinguished by pattern and color, and a scatter plot can use color, shape, or dot size to make a third variable apparent. Read on for info on color, shape, and more!



Aesthetic Properties I: the Menu

We've covered how we use chart type to highlight a relationship in the data. Now we'll talk about how we use aesthetic properties to further clarify and visualize the "details" of the data.

Aesthetic properties are the attributes we use to communicate data visually:

- Position
- Size
- Shape
- Color / pattern

Most of us are already familiar with this concept, even if we've never seen it put into words before. Let's walk through a visualization together to see how its aesthetic properties help us understand the data story.

This visualization published by The New York Times in 2012 shows different tech IPOs (initial public offering, or when a formerly-private company becomes available as a publicly traded stock).

Take a minute to look at the graph and see how position, size, shape, and color are used. What does it mean for a data point to be up and to the right versus down and to the left? What do bigger and smaller signify? How does shape come into play? What does color mean?

Okay, let's walk through it together now.

We can start with position: from left to right (the x-axis), the graph measures time. From bottom to top (the y-axis) the graph measures company value in billions of dollars.

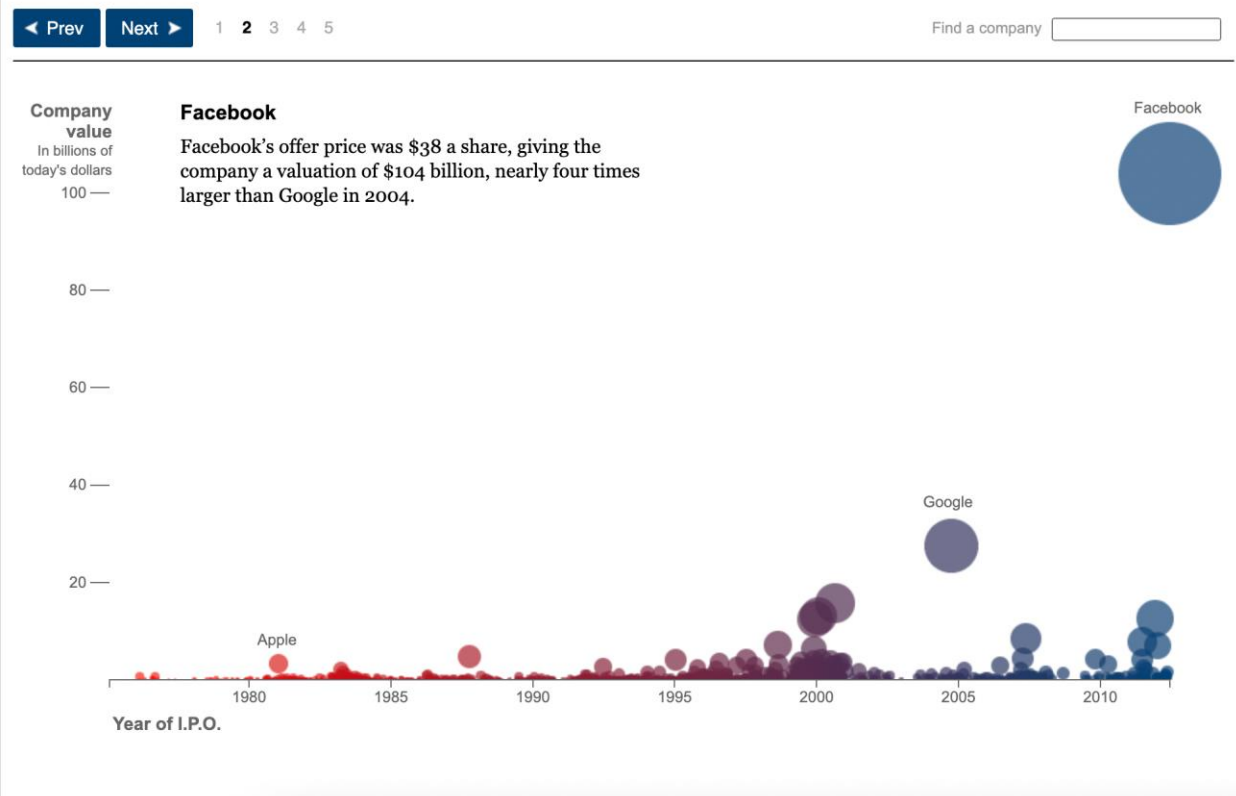
As far as shape goes, there's only one here – circles.

Moving on to size, it looks like the size of each circle tracks with its company value.

Companies with a larger IPO amount get a bigger circle.

Finally, color corresponds to time. Earlier corresponds to red, and later to blue. The middle portion, around 1995, is purple. This visually separates the three decades into three general zones.

The Facebook Offering: How It Compares



Aesthetic Properties II: Information Redundancy

Let's take a closer look at this graph. There's a connection here between size and y-position (how high or low a circle is): they actually tell us the same information twice!

This is an example of **information redundancy**, or encoding the same information in different visual properties. We already know that Facebook has the largest company value because it's the highest circle on the chart. Its large size gives us another way to visually compare it to the other data points.

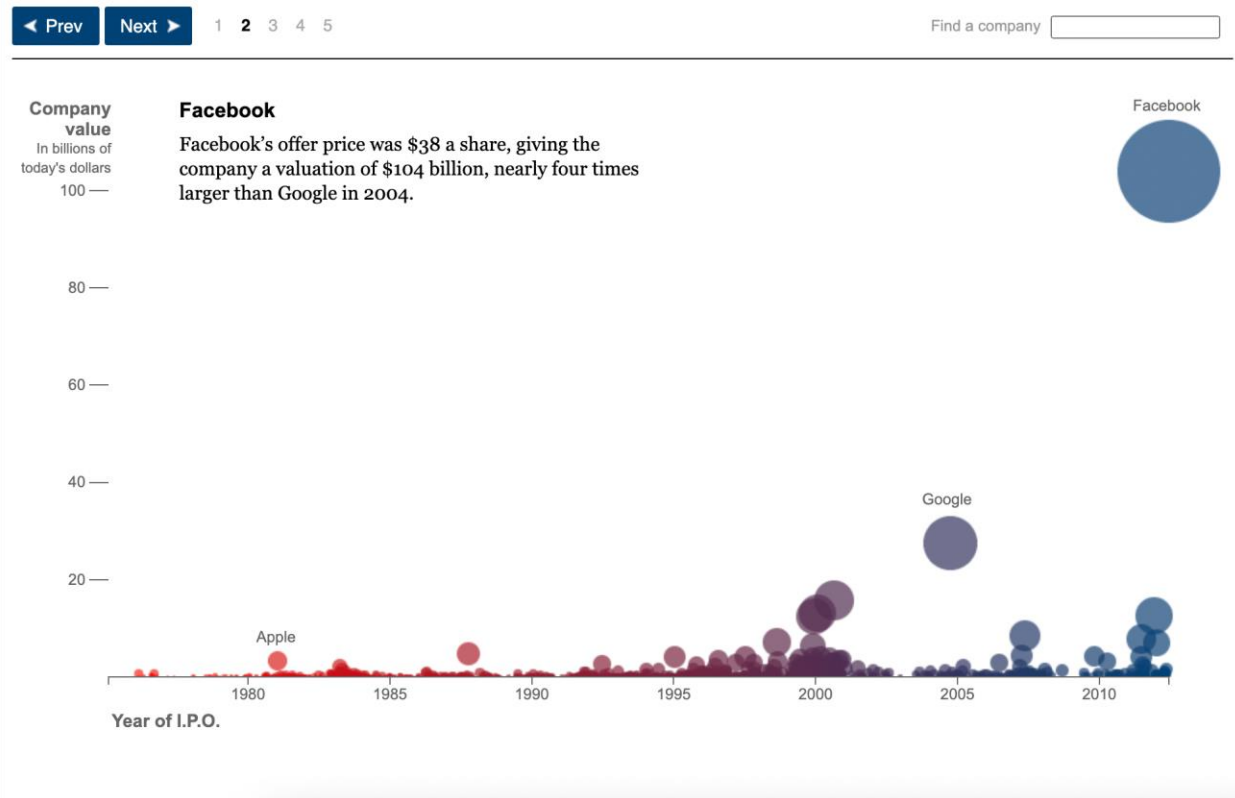
Info redundancy is also helpful for prioritizing values. There are lots and lots of smaller companies on this graph – if every circle were the size of Google's circle, the bottom part of the graph would be an unreadable ball pit. Or, if all the circles were the size of the smallest ones, the chart would lose some of its emphasis on Facebook's large IPO value. **Information redundancy helps key data points to stand out.**

Color and x-position are also redundant on this graph, making the chart a little easier and faster to interpret. The three color groups in the graph help break up the three-ish decades shown, giving us a sense in one glance that red circles are part of an early group, purples are in a middle group, and blues are the latest.

We'll dive deeper into accessibility later, but for now note that information redundancy is also an important practice to ensure that colorblind viewers can access all the information in a chart.

To sum up, information redundancy visualizes the same information using multiple different aesthetic properties. It's important for readability, organization and prioritization of information, and accessibility.

The Facebook Offering: How It Compares



Consider the Audience

The best data visualizations help us to understand what's in the data, draw meaningful conclusions, and make decisions about the next steps. This requires context and **different context is appropriate for different audiences.**

Let's walk through an imaginary-world example:

Shinji, Paola and Raj work together in an ecology lab. Their lab is applying for funding for a field research trip next year. This week, each of them will present the lab's work and data to a different potential funder:

Sir Avon Rattleborough, retired ecologist and expert field researcher

Claude Tuber, board game developer and eccentric venture capitalist

Milana Diamante, heiress and amateur biologist

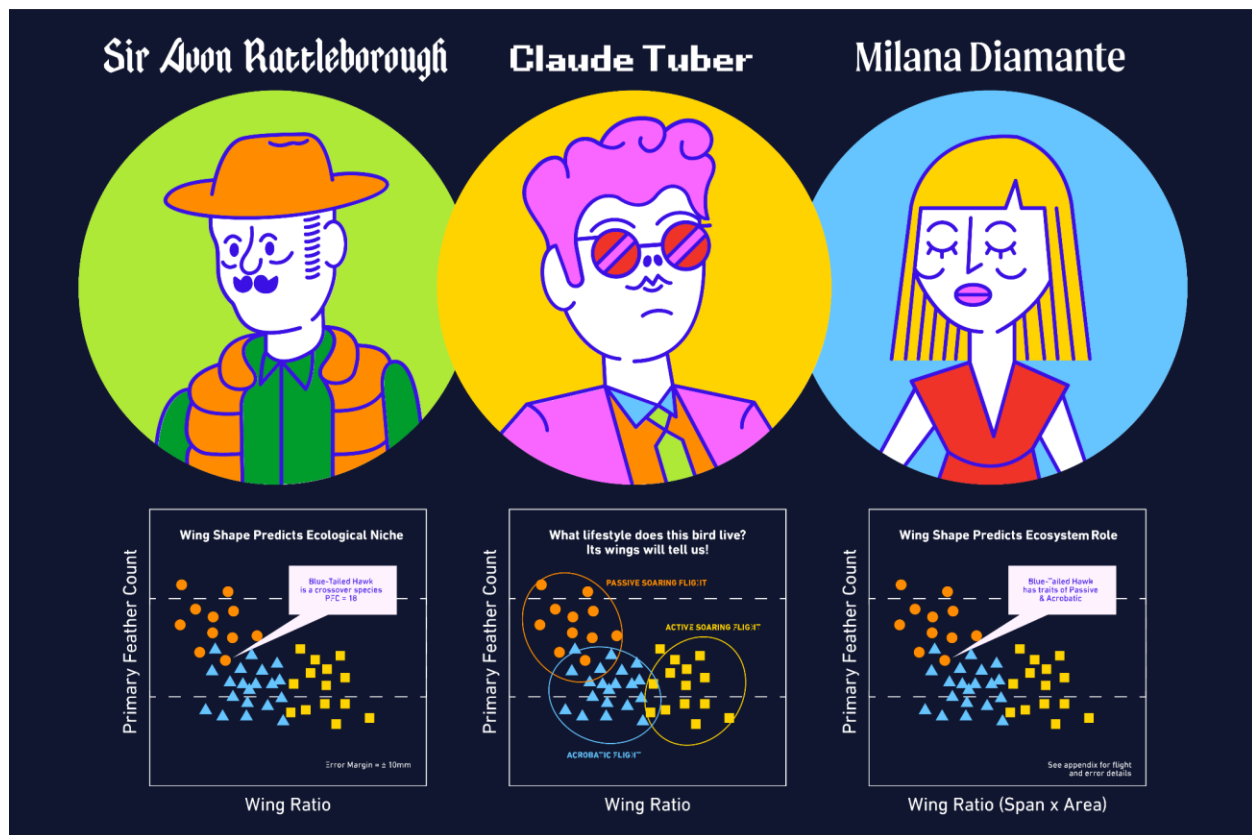
The three labmates know they'll have to communicate differently to each of these potential funders. **They'll communicate the same information, but each lab member will personalize their chart with a title and annotations that work best for their intended audience.**

With this in mind, let's get some background on the funders and see which title and annotation is most effective for each graph...

Sir Avon has over 50 years of ecology research under his belt, and lives for the details. He's not familiar with the lab's work specifically, but he knows the lingo. Sir Avon always wants to see proof.

Claude is a big picture thinker: he's wondering if the lab's research can be turned into a fun and educational board game for a broad audience. He's excited about it but knows nothing about ecology. Claude has a limited attention span.

Milana is an enthusiastic citizen scientist in her free time, and is eager to support a worthy cause. She's never done field research herself, but has some background knowledge and has been following the lab's work for a couple years. Milana loves to ask follow-up questions.



Context is Key

We tend to think of context as “outside” a data visualization, but Shinji, Paola and Raj had the right idea by including appropriate context using titles and annotations. Each of them also did a great job of **considering their specific audiences when making decisions about what context to include or not.**

Viewers need context to understand what a data visualization means and why it matters.

Paola predicted that, as a scientist, Sir Avon would want to know the technical details like the amount of error in the measurements. She didn’t take up space on the graph with definitions, since she knew they weren’t necessary in this case.

Shinji decided to use a question and answer format for their title to help communicate the takeaway of the graph to Claude in an accessible way. Shinji went for a more aggregated, less detailed approach to help keep the conclusion simple and digestible.

Finally, Raj made good choices in his visualization for Milana: a descriptive but slightly-less-technical title, and a pointer towards definitions of terms he thinks Milana may not know and would be interested to learn.

In each case, the lab member made sure to...

- Provide necessary details
- Include context that's helpful for the specific audience
- Avoid "chart junk": excess graphics, annotations, and general lines that don't actually contain information



Accessibility Basics

Now that we can make some visualizations, let's talk accessibility to make sure our work reaches everyone who wants to interact with it!

The most commonly discussed accessibility concern is color, since color blindness affects 1 in 12 males and 1 in 200 females. That's more common than we tend to think! There are a few different types of color blindness: check out the images for simulations of each form.

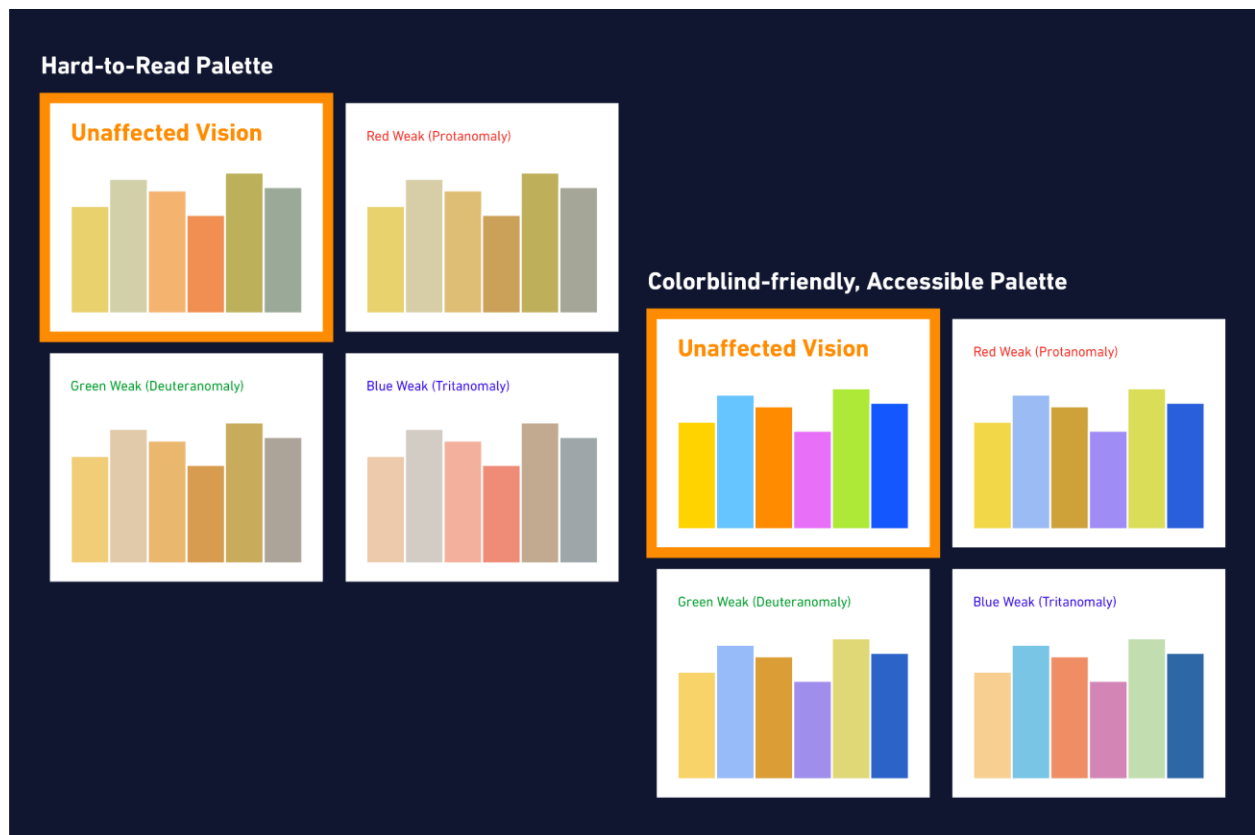
The big takeaway when designing for color accessibility is to think not only about the **hue** of a color (e.g. red, green, or purple), but the **value** as well (e.g. bright red, light green, dark purple). **Good color comparisons use high contrast values, not just different hues.**

It's also important to use readable fonts in readable sizes, and make sure they're web-accessible if online. (If you don't do this, ☹ ☹ ☹ ☹ ☹ ☹ will happen!)

Finally, for online data visualizations, make sure to include alt text as we would for any other web image. Alt text ensures that users experiencing a visualization through a screen reader won't miss out on whatever information it contains.

To recap, here's a checklist for baseline accessibility:

- Colorblind-friendly color palettes
- Large enough font size
- Readable, web-accessible font type
- Alt text on data visualization images online



Further Accessibility: Universal Design

In the last exercise we covered accessibility guidelines specifically related to vision access – using color palettes, fonts, and alt-text to ensure that people across the vision spectrum can access our work.

The same accessibility goal – making our work available and easier to access for more people – is a great principle to keep in mind no matter what. (This is actually called “universal design.”)

We can apply it when it comes to...

- Readability: keep the reading level to a high school level whenever possible
- Prior knowledge: define unfamiliar terms and avoid unnecessary jargon

- Information overload: introduce new information with intentional pacing and organization

Remember the ecology lab's pitches for funding? Each lab member took a different approach based on their audience. The difference between the pitches for Sir Avon and Milana is a perfect example of "leveling" content for the intended audience: swapping the more technical term "ecological niche" with the synonym "ecosystem role" makes the graph's title readable with more everyday words, without changing the meaning or even sacrificing detail.

Finally, with any of these practices, there may be situations where it's not the right call.

Trinh probably shouldn't avoid all technical language on the

Preview: Docs Loading link description

[graphs](#)

in his peer-reviewed paper, and there may be no need for Anabelle to define industry terms when presenting a chart to work colleagues in her department. But for most audiences, and especially for broad or unknown audiences, keeping accessibility in mind will help everyone get the most out of our visualizations.

Owning the Role of Author

Last up, we're talking authorship.

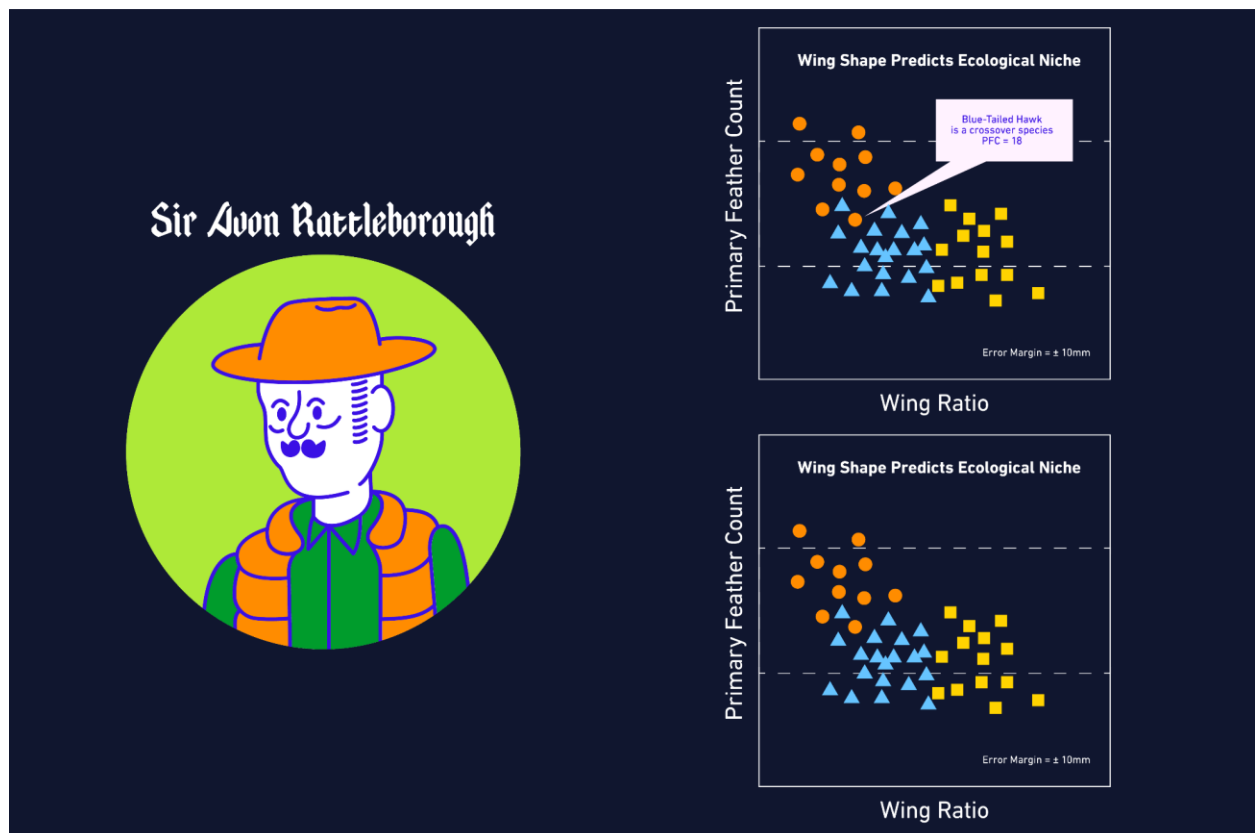
When Paola made the visualization for Sir Avon, at first she included fewer details and took out her annotations about points of interest. She assumed there was no way she – a student scientist – would know something that Sir Avon didn't already know.

But she was wrong! By the time Paola was able to make a data visualization about bird wing shapes, she was becoming an expert on the subject. Sir Avon, while a celebrated ecologist in general, did not share her depth of specific, current knowledge on this topic.

Dr. Dinkle, the lab's leader, helped Paola to own her role as the author – to recognize that the amount of time she had invested in data cleaning and data analysis, as well as data visualization, qualified her to make decisions about what should be included. It was not only worth sharing her opinion, the graph actually suffered without those annotations! Her perspective on the data helped Sir Avon to grasp the key takeaways.

Paola's experience is true for many people. It can feel unnatural to "speak for" the data, and many authors worry that they will influence their audiences by including annotations. But that's just it! **Data does not speak for itself**, and often, the author of a data viz is the person in the best position to create insightful, helpful annotations.

In fact, this context isn't only helpful – very often, it's the ethical way to present information. Providing context or even a written summary of what a graph shows helps to limit false conclusions, misinterpretations, and misinformation.



Conclusion

We made it! This lesson covered everything you need to know to start designing data visualizations that look great, communicate effectively, and treat data ethically.

Here's a recap of what we talked about...

- Choosing the right chart type for our data

- Mapping

- Preview: Docs Loading link description

- [variables](#)

- onto aesthetic properties to communicate using position, shape, size, color

- Making effective color choices for data, design, and accessibility

- Designing accessible visualizations

- Scaffolding the visualization with context for the audience

- Owning the role of author

That's a big chunk of information. Pat yourself on the back for making it to the end of this lesson, then go forth and visualize!

Misleading and Confusing Graphs

Introduction

Have you ever used a map that was just... wrong? As in, “this entire building doesn’t exist on the map” wrong, or “it almost had me drive down a one-way street” wrong. You notice pretty quickly, right?

What about a map where countries are the wrong size? How long do you think it would take to notice that something was off?

Anyone who has ever learned geography from a map has learned something “wrong.” Mapmakers have known for hundreds of years that it’s just not possible to transfer the earth’s 3D information to a 2D map without changing the shape or size of the continents and oceans – something has to be sacrificed.

(Check out this GIF showing a Mercator projection versus the actual size of countries. The Mercator projection, invented in **1569** and widely used to teach geography through the 20th century and today, severely distorts the size of land masses near the poles.)

The same idea is true in data visualization. Data visualizations take information from the living, breathing world that we inhabit and show it to us in just a few square inches of screen or paper. That always involves prioritizing speed, or accuracy, or sample size, or cost, or another factor, at the expense of something else.

While the best

Preview: Docs Graphs are a way of modeling systems based on a node and edge structure for representing the relationships between items.

[graphs](#)

really do teach us something new, and help us understand a deeper truth using data, graphs can also be misleading, both intentionally or unintentionally. We shouldn’t ever assume that a data visualization shows us the truth, the whole truth, and nothing but the truth.

This lesson will help us recognize the elements of misleading and confusing graphs so that we can avoid making them ourselves.

Axes

Axes and scaling are like the page layout and spacing of a paper book: they're not the most exciting parts, but they do present plenty of opportunities to make it harder to read.

Let's start with axes – the x-axis (horizontal, left-right) and y-axis (vertical, top-bottom). A common misleading aspect of an axis is a **break**. A break starts the count at a number that's not zero, or jumps ahead – this can distort the amount of difference between data points by removing context, and make small differences in data seem bigger.

Here's an illustration of that idea: it would be almost impossible to tell at a glance if there were 100, 105, or 110 people standing in a room – but you'd be able to easily tell the difference between 0, 5, and 10 people standing in a room. Using a break on an axis can have the same effect, amplifying the **change** rather than the **context** because it alters the proportions in the visualization. Check out the

Preview: Docs Graphs are a way of modeling systems based on a node and edge structure for representing the relationships between items.

[graphs](#)

to the right to see what this looks like in practice.

So what to do? If you're looking at a graph, take a second to check where the axis starts. If there's a break, factor that in as you think about what the numbers mean.

If you're making the graph, instead of using a big break...

Keep enough context to view differences **in proportion** to a meaningful amount,
OR

Make two graphs, one without a break and one “zoomed in”, OR

Choose a visualization type that shows the change, rather than the raw numbers



Scaling

Scaling refers to the distances between numbers on an axis. Almost all

Preview: Docs Loading link description

[graphs](#)

use a **linear scale**, where the numbers count up by a consistent interval – tenths of a centimeter or millions of dollars, if it's the same interval, it's a linear scale.

The other scaling option is a **logarithmic scale**, a.k.a. log scale. The log scale is common for showing exponential growth that won't fit on the page with a linear scale,

but it's almost never a good choice for a general audience. Unless people use log scales regularly, they tend to have trouble interpreting them correctly.

Check out the graphs in the LE to see how the pharmaceutical company Purdue infamously used this misinterpretation to their advantage in the early 2000s. The linear scale shows how the concentration of a painkiller drug spikes sharply in the bloodstream at higher doses – the log scale makes it look like all doses behave pretty similarly. (These are reproductions of the original graphs, but we can definitely see how differently they represent the same numbers.)

In general, just like it's always worth checking for a break, it's always worth checking how a graph is scaled.

Last thing about axes and scaling: **generally, we measure time horizontally**, putting that variable on the x-axis. For the vast majority of circumstances, this makes the most sense and helps readers to intuit what the graph measures.



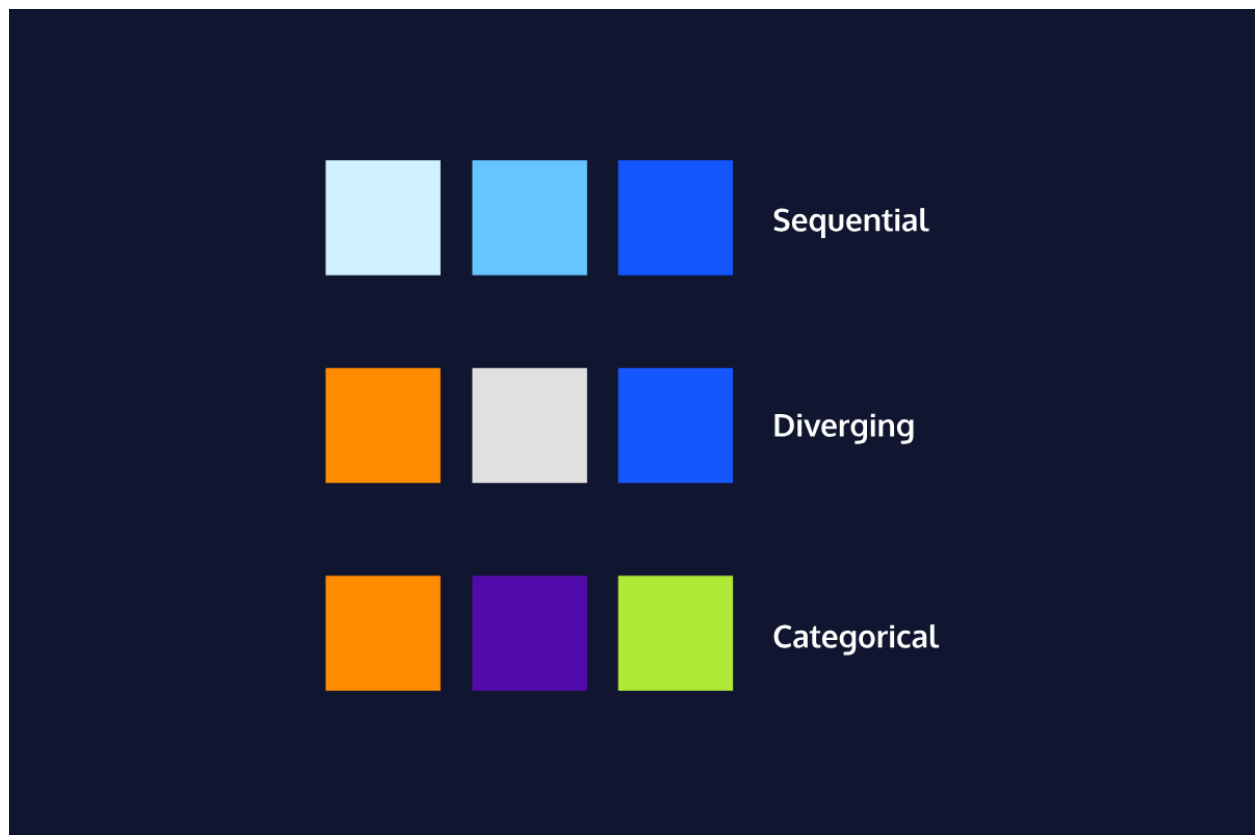
Color Scales

Color is often the first thing we register when looking at data visualizations. There are three types of color scales, used for the three major types of relationships we can visualize with color.

Sequential scales are colors in a sequence – often, this is the same hue with more and more white added to or taken away from the color. Sequential scales are used to show a variable increasing or decreasing in intensity or amount, like income, depth, or percent of population that owns a chinchilla.

Divergent scales are anchored by colors from opposite sides of the color wheel, a.k.a. complementary colors. A divergent scale is used to visualize data where the middle is a baseline, and either side represents a contrasting change. For example, divergent scales do a good job of showing a positive/negative swing in voting or polling, temperatures above and below freezing, or gains and losses over time.

Categorical scales use a variety of colors to differentiate categories without assigning a rank or order to them. In other words, “purple” doesn’t necessarily mean more than “green” – the two are just different colors. Categorical scales are for categorical data, like types of vegetables in a supermarket, or different treatments tested in a controlled study, or organizational blocks on a calendar.



Color Associations

Once we've picked the right color scale, there are still a few more considerations to be made to reduce confusion.

First up, we tend to view darker colors as “more” and lighter colors as “less.” For example, if we're visualizing which US states have the most pet ferrets, California – with the most pet ferrets of any state – should be the darkest state on the map. When this scale is reversed, people will tend to just read the graph wrong rather than reading the legend carefully.

We also come to data visualizations with pre-existing associations for certain colors. These can be culturally specific (red means bad vs. red means lucky), or influenced by the norms for a particular field (red means negative financial balance).

Sometimes it's good to stick with what's recognizable: it would be confusing for US voters if a major newspaper decided to visualize Democrats in red and Republicans in blue, since these colors are overwhelmingly associated with the opposite party.

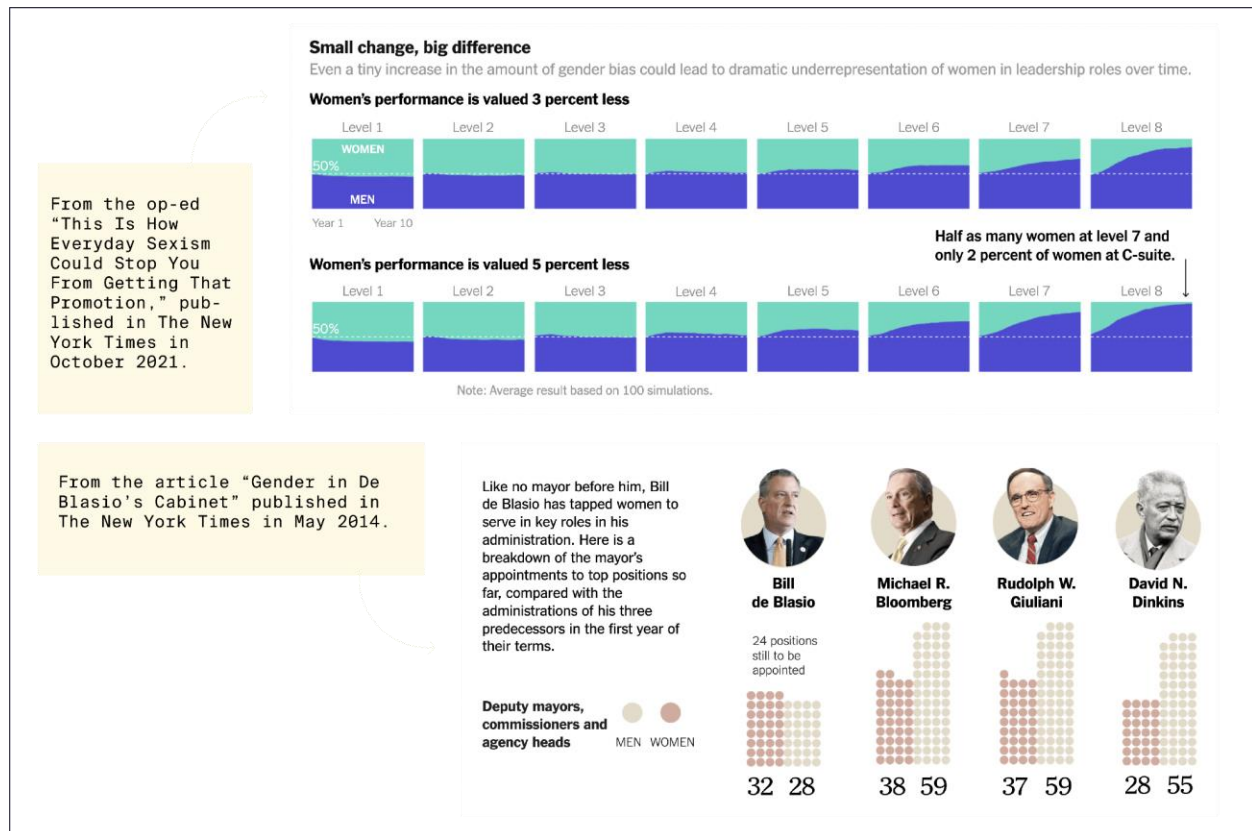
But in other cases, switching up colors that have existing cultural associations can reduce harmful stereotyping. Using pink for women and blue for men reinforces an outdated,

Preview: Docs Loading link description

[binary](#)

view of women as soft and passive, and men as strong and unemotional. This design choice will not only turn off some viewers, it may also distract on a graph where gender is a relevant variable but not the whole focus.

It would be confusing to just reverse this stereotypical color palette, but there are plenty of good alternatives – check out two examples from The New York Times on the right. The important thing is to be consistent with the alternative palette.



Labels and Titles

A good title is one of the best and fastest tools for making a more understandable visualization. Lots of confusion can be saved with a descriptive title.

If the graph doesn't have a good title (or even a title!), viewers have to do more legwork to first figure out what each axis measures and then what the data points show.

The title can be a question that visualization answers, like, "Who speaks more in Disney movies, male or female characters?"

Titles can also be a statement of what the visualization shows, like "Comparing denim inseam lengths through the decades" or "Millennials really do spend more on rent than on avocados" or "The effect of hunger on mood level."

Like a good title, annotations on a graph also help the viewer to understand what's going on. Annotations are perfect for calling out points of interest, explaining outliers, or including background information that a viewer won't necessarily know from just looking at the graph.

Check out this *Live Births* graph from FlowingData to see how much value the annotations add. They...

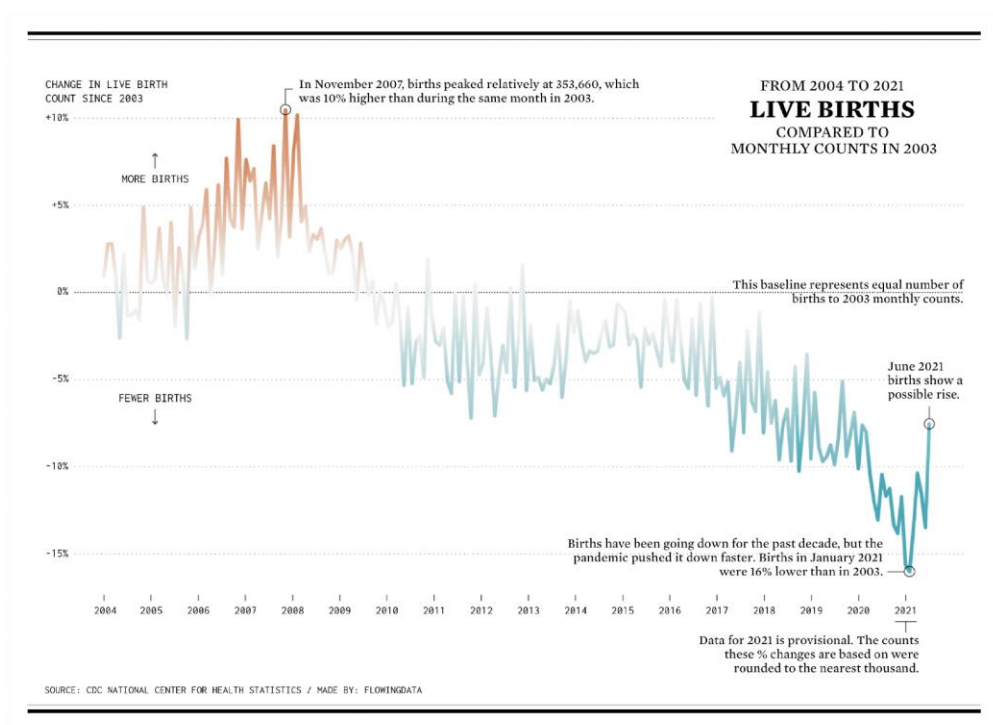
add detail to the highest and lowest points on the graph

explain what the 0% baseline means

provide a caveat for the 2021 data

reinforce in words that the percents on the y-axis show “more births” and “fewer births”

Just a few lines of thoughtful annotation here and there give the audience so much more ability to interpret the graph at a deeper level!



Conclusion

That was a short primer on some common misleading parts of data visualizations – pitfalls that get between **the actual truth of the data** and **how the viewer understands it**.

People make misleading or confusing

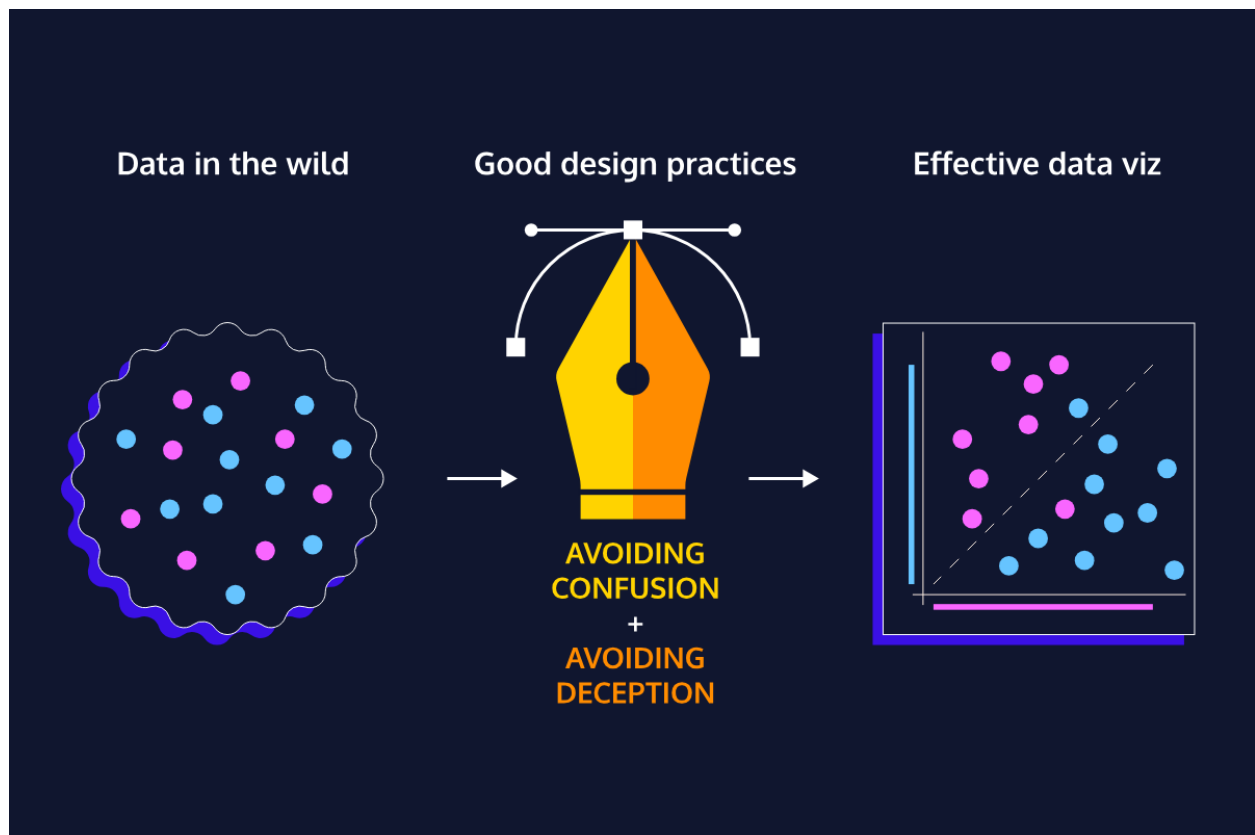
Preview: Docs Loading link description

[graphs](#)

on purpose and by accident. We've seen how intentional changes to the scaling can really distort what the graph seems to say, and also how going with the status quo on color palettes can be helpful or harmful. Most of us aren't out here trying to make misleading graphs – very often, it's just a question of making a better or worse decision while designing the data viz.

And improving our design skills – knowing when to use different color palettes and how to label our axes clearly – not only makes our graphs better, **it also reduces bias!** When we follow best practices in design, we're not "going rogue" and just making design decisions that make the graph look the most extreme, or suit our needs at that moment. We're following established practices that help to standardize graphs so that we can focus on what the data's really saying.

Long story short: good data viz is always a combination of quality data and effective design choices. If we can avoid confusion or missteps along the way, that'll always be a good thing!



Data Viz Basics

Project

Building Effective Data Visualizations

You've joined Shinji, Paola, and Raj in Dr. Dinkle's ecology lab! The lab is swamped with its latest project after a successful round of funding (award-winning board game designer Claude Tuber wants to turn the lab's work into the tabletop role-playing game of a generation). They're excited to have your data visualization skills!

In a few weeks the lab will present data on birdfeeder birds to the Backyard Birders Association of nearby Dandelion City. One component is a visualization of the wingspans of the three most common backyard birds in Dandelion City: the Northern Cardinal, American Robin, and Blue Jay.

The lab's paid intern Janis is trying her chops at data visualization, and will be drafting all of the visuals for this presentation. With your knowledge of data visualization, your task is to help her make the most effective visualization of wingspan data to present to the backyard birders of Dandelion City.

Chart Type

Color Palette

Now that we've gotten the chart type to align with our data, let's move on to the next important consideration: color.

Annotations

With chart type and color palette sorted, let's look on to annotations that will help our audience make the most of the final chart.

Design

And finally, let's finish up with some design considerations.

Data Analyses and Conclusions

Introduction

When working with data, our ultimate goal is to draw conclusions. The point of **data analysis** is to discover useful information, inform conclusions, and support decision-making. In general, data analysis lets us evaluate the presence, strength, and direction of relationships between different

Preview: Docs Loading link description

[variables](#)

in our data. It lets us see patterns and make connections that would be impossible to find looking at raw data.

Importantly, different types of data analysis allow us to draw different types of conclusions. In this lesson, we will learn about **five types of data analysis**:

- **Descriptive analysis**
- **Exploratory analysis**
- **Inferential analysis**
- **Causal analysis**

- **Predictive analysis**

When applied appropriately, these analyses might let us determine why something happened, why something is the way it is, or what is likely to happen next.

Descriptive Analysis

Let's imagine we're looking at a field of flowers like the one in the learning environment. There are flowers with different shapes, colors, and heights. If you had to describe these flowers, how would you quantify all these characteristics? What does a typical blue flower look like? Are blue flowers all similar in height? It's hard to say just looking at a mix of all the flowers, just like it's hard to describe a big dataset just by looking at a spreadsheet of values.

Descriptive analysis lets us describe, summarize, and visualize data so that patterns can emerge. Sometimes we'll only do a descriptive analysis, but most of the time a descriptive analysis is the first step in our analysis process.

Descriptive analyses include measures of central tendency (e.g., mean, median, mode) and spread (e.g., range, quartiles, variance, standard deviation, distribution), which are referred to as **descriptives** or **summary statistics**.

Typically, data visualization is also included in descriptive analysis.

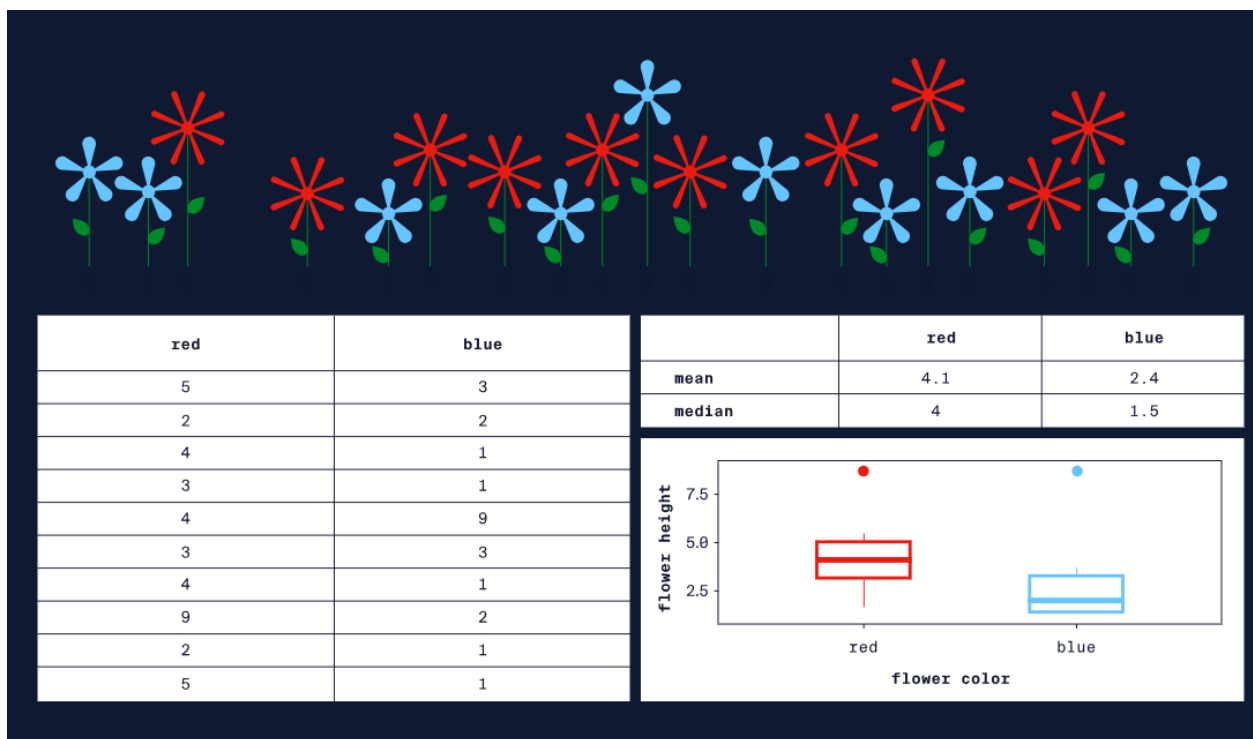
Descriptive analysis helps us understand our data and decide what steps to take next, but we cannot extend what we learn to other datasets.

Instructions

1. Take a look at the different stages of descriptive analysis of the flowers in the learning environment by selecting 'Next' or playing the slides using the center button. Consider the following:

1. Does calculating the mean and median height for blue and red flowers make it easier to see trends in the data? What about the boxplot?
 2. Which part of the descriptive analysis shown is most useful?
 3. Based on our analysis of red and blue flowers, can we conclude that red flowers are taller than blue flowers *everywhere*?
2. Remember:
1. Descriptive analyses let us draw basic conclusions about the data at hand.
 2. We cannot extend findings from descriptive analyses to other datasets.

In the next exercise, we will learn about another type of data analysis: exploratory analysis!



Exploratory Analysis

Exploratory analysis is the next step after descriptive analysis. With exploratory analysis, we look for relationships between

Preview: Docs Loading link description

[variables](#)

in our dataset.

While our exploratory analyses might uncover some fascinating patterns, we should keep in mind that exploratory analyses cannot tell us *why* something happened: correlation is not the same as causation.

Let's look at an exploratory analysis with a real dataset! Check out the exploratory plots in the learning environment (you can pause if you need to!). These plots use annual income data for the United States from the government's [Social Security Administration Wage Data](#).

We are exploring trends in annual income over time using two descriptives (mean and median). We see that the difference between these two descriptives seems to be growing. This suggests that annual income in the U.S. is becoming more *skewed*: a few very high earners are pulling the average up.

Next, we plot the difference between mean and median annual income over time. It sure does look like it's increasing with time!

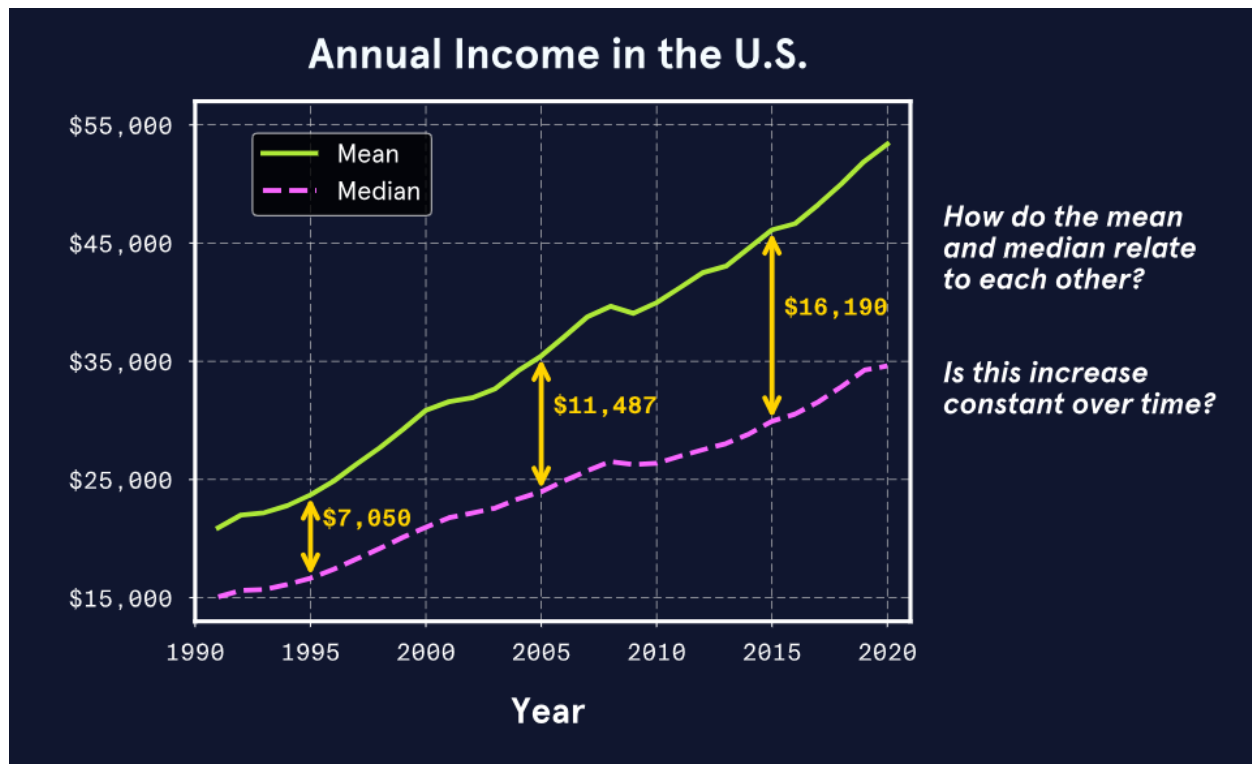
Finally, we add a trend line and calculate the R-squared, which gives us an indication of how well the line fits the data. With an R-squared of 0.99 out of a maximum of 1.00, we can be very sure that the gap between mean and median is increasing.

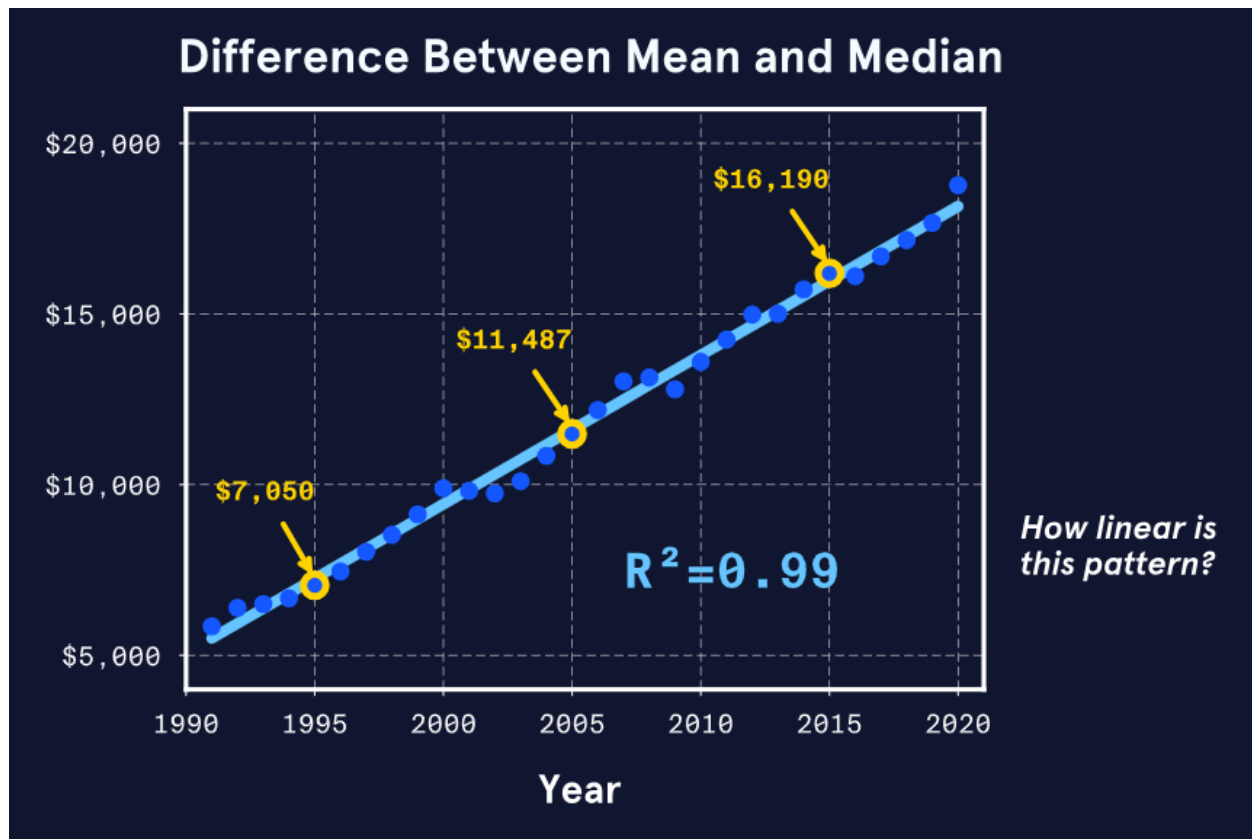
Instructions

1. Now, consider the following:
 - Can these data tell you *why* the wage gap is increasing in the U.S.?
 - Can you draw conclusions about wages in other countries from these data?
2. Remember:

- Exploratory analyses show us underlying patterns and relationships within datasets.
- Exploratory analyses cannot determine causation.

Next, we will take a closer look at another kind of exploratory analysis: cluster analysis!





More Exploratory Analysis: Clustering

Unsupervised machine learning techniques, such as **clustering** algorithms, are useful tools for exploratory analysis. These techniques “learn” patterns from untagged data, or data that do not have classifications already attached to them, and they help us see relationships between many [variables](#) at once.

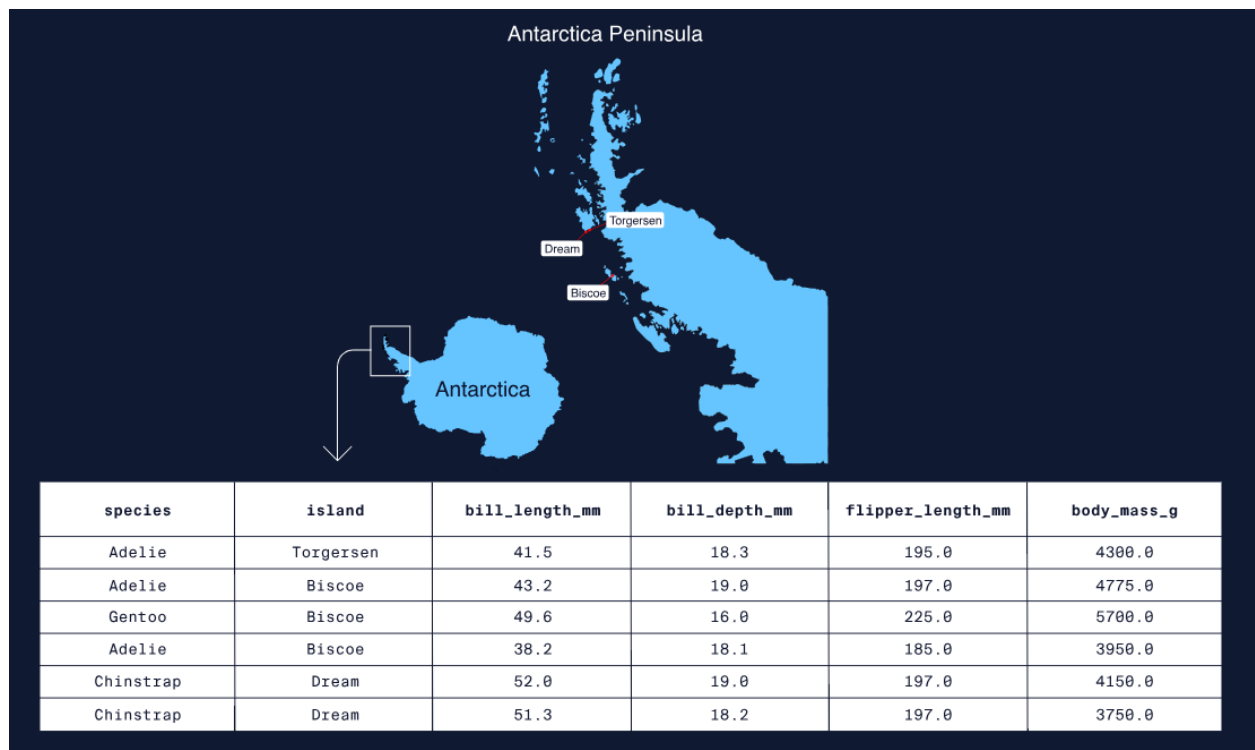
Let’s see how cluster analysis works with penguin data that scientists collected in Antarctica! Take a look at the data in the learning environment, which comes from the [palmer penguins dataset](#).

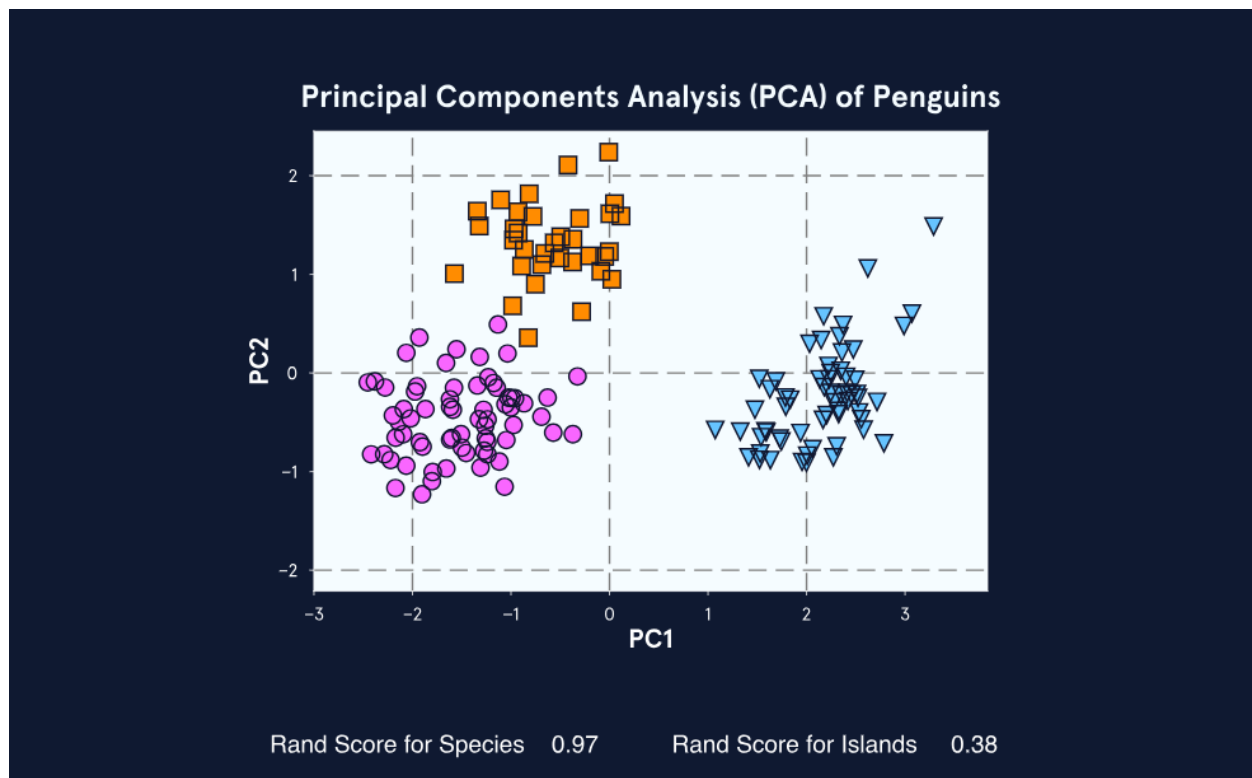
We have four measurements (weight, beak length, beak depth, and flipper length) from Gentoo, Adelie, and Chinstrap penguins living on three islands (penguin artwork by @allison_horst). It’s pretty difficult to see relationships between the measurements looking at the raw data.

Ideally, we want to see how all the measurements relate to each other, but plotting them all would be a lot to look at. Instead, we can perform a **Principal Component Analysis** or PCA, which compresses the variables into principal components that can be plotted against each other. After PCA, we can use **k-means clustering** to look for trends in the data. We see that the penguins fall into three distinct clusters in the PCA plot!

It's cool that our analysis shows three clusters because we know that our data include three penguin species living on three islands. We can use the Rand statistic to see how well species and islands match the k-means clusters. This is similar to the R-squared statistic we use to check the fit of a trend line. With a Rand score of 0.97 out of 1.0, species have a stronger relationship to clustering!

Next up, we'll check out inferential analysis!





Inferential Analysis

A/B tests are a popular business tool that data scientists use to optimize websites and other online platforms. A/B tests are a type of **inferential analysis**. Inferential analysis lets us test a hypothesis on a sample of a population and then extend our conclusions to the whole population.

Imagine we want to know if a blue or a green button will get more clicks on a website. We hypothesize that the green button will be more successful and we run an A/B test on a sample of people that visit our site. Half the sample sees the blue button (option A) and half see the green button (option B). At the end of the test, 90% of people that saw the green button clicked it, whereas 60% of the people that saw the blue button clicked it.

Now we need to ask, “If color wasn’t related to click rate, how likely was a 30% difference just by chance?” We can use

Preview: Docs Loading link description

[statistics](#)

to calculate that probability. If there is less than a 5% probability that our results happened by chance, we have good evidence that green buttons get more clicks! We can extend these results to everyone that visits our site (the whole population), so it makes sense to use green buttons on our website.

This is a powerful thing to be able to do! But since it’s so powerful, there are some rules about how to do it:

Sample size must be big enough compared to the total population size (10% is a good rule-of-thumb).

Our sample must be randomly selected and representative of the total population.

We can only test one hypothesis at a time.

Instructions

1. Based on what we know about A/B tests, consider the following scenarios:

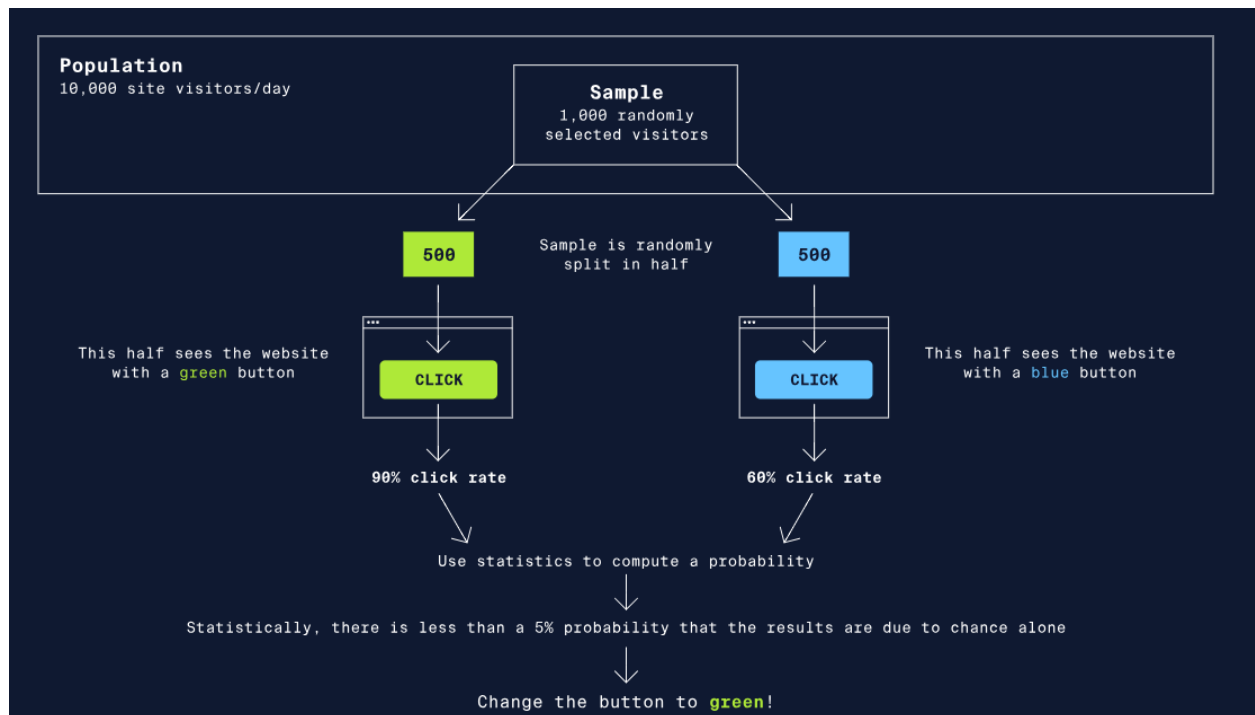
If the A/B test was being run for a site with 1 million visitors per day, would a sample size of 100 be big enough to extend conclusions from the sample to the whole population?

The site in the A/B test has visitors from all over the planet. If you ran the A/B test for 12 hours when only one-half of the planet was awake, would your sample be representative of the whole population?

When assigning people to see either the blue or green button, would it be better to randomly assign people or only assign people to the green group if they say their favorite color is green?

If instead of testing a green button versus a blue button, the test was of a blue button with small font vs. a green button with large font, would we be able to conclude that the color of the button was influencing whether visitors clicked the button?

Next up, causal analysis!



Causal Analysis

We know that correlation does not mean causation. This is an important limitation in data analysis. We should be cautious to believe any studies or headlines claiming that one thing caused another without knowing their research methods. However, we often really want to know why something happened. In these cases, we turn to **causal analysis**. Causal analysis generally relies on carefully designed experiments, but we can sometimes also do causal analysis with observational data.

Experiments that support causal analysis:

Only change one variable at a time

Carefully control *all* other variables

Are repeated multiple times with the same results

These are pretty high standards to meet. However, experiments that meet these standards are the clearest way to figure out if one thing caused another.

Instructions

1. Take a look at the breakdown of good experimental design in the learning environment and think about the following questions:

Could you determine if drinking more water caused better sleep by monitoring your own sleep for a month and comparing results from days that you drank 8 glasses of water and days that you did not?

Could you determine if drinking more water caused better sleep if you expanded your study to include 1,000 subjects and asked all men to record their sleep on days they drank 8 glasses of water and all women to record their sleep on days that they did not drink 8 glasses of water?

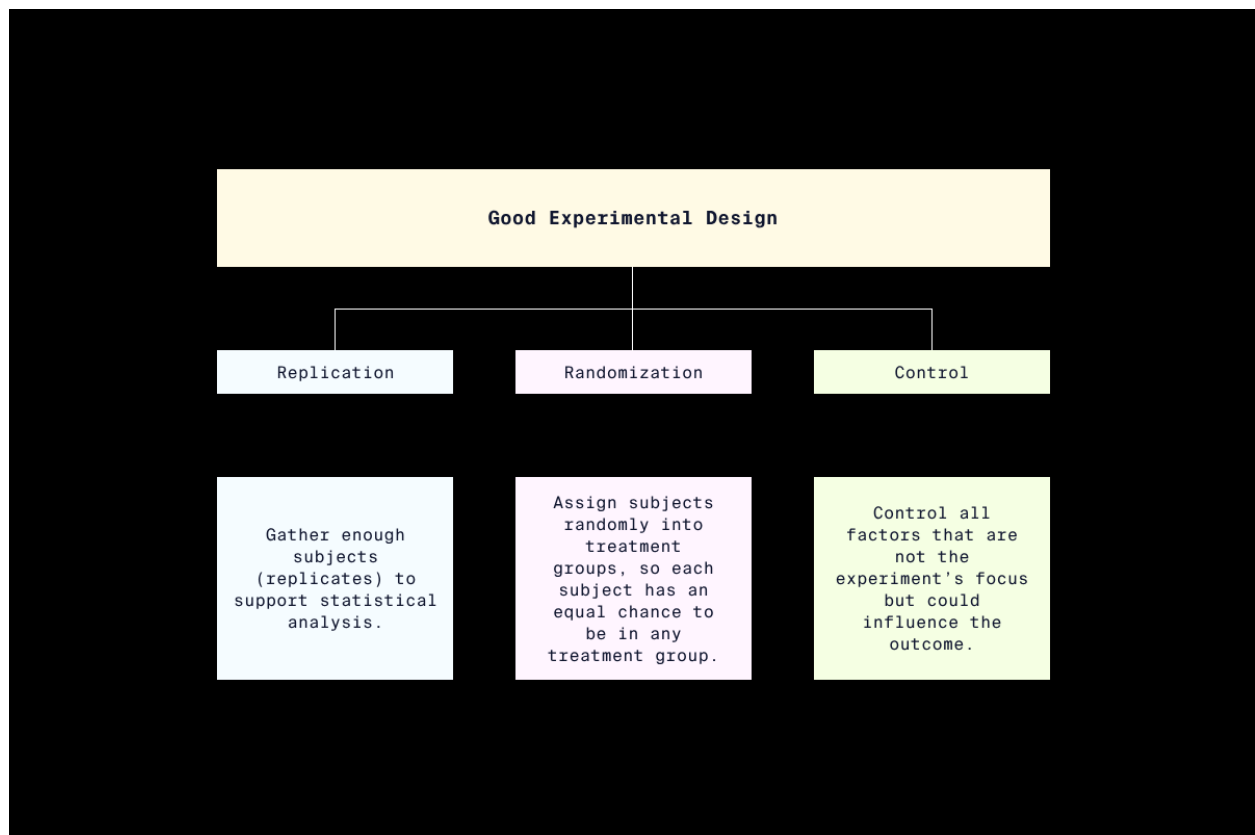
2. Remember:

Correlation does not equal causation.

Proving causation is tricky and generally requires very careful experimental design.

Replication, randomization, and control are key components of good experimental design.

In the next exercise, we will consider when and how to do causal analysis if an experiment is impossible.



Causal Analysis with Observational Data

Sometimes we need to establish causation when actual experimentation is impossible. This could be due to a variety of reasons. For example, we might want to know why something happened that we really don't want to repeat (e.g., why did a product flop?), or the necessary experiments may be too difficult, too expensive, or unethical. In such cases, we can sometimes apply causal inference techniques to observational data, but we need to be very careful.

Causal inference with observational data requires:

- Advanced techniques to identify a causal effect
- Meeting very strict conditions
- Appropriate statistical tests

Let's think about this in terms of climate change. It is important to know if climate change is causing more frequent and intense hurricanes. But, we can't do a controlled, replicated experiment on multiple planets with and without climate change. Instead, climate scientists carefully use causal analysis on observational data to determine whether climate change contributes to bigger hurricanes happening more often.

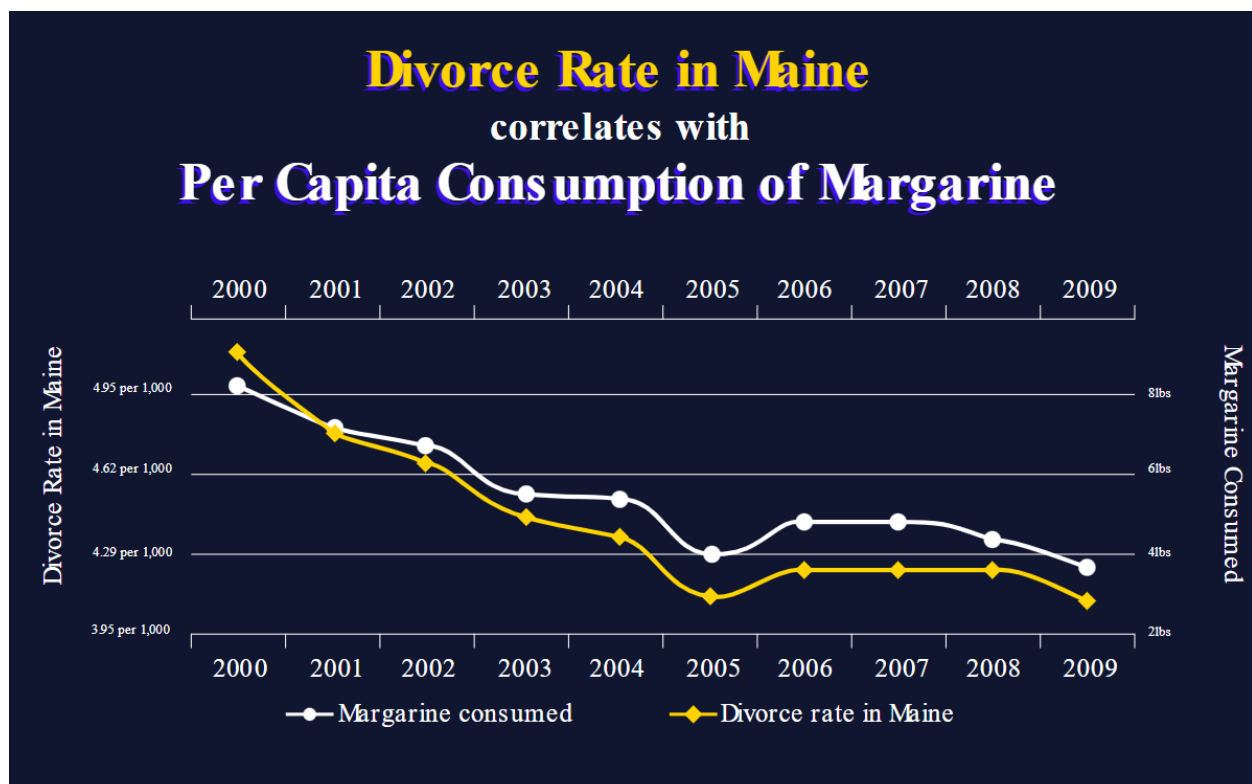
Instructions

1. Take a look at the observational data in the learning environment that comes from the site [Spurious Correlations](#). Consider the following questions:

Is margarine consumption correlated with divorce rate?

Does consuming less margarine *cause* fewer people to get divorced in Maine? Why or why not?

Next, we will discover a popular form of analysis that is used to make predictions!



Predictive Analysis

We interact with **predictive analysis** in everyday life when we text a friend using text completion or watch a suggested TV show on Netflix. Predictive analysis also underlies computer vision, which is applied in facial recognition software and self-driving cars.

Predictive analysis uses data and **supervised machine learning** techniques to identify the likelihood of future outcomes.

Some popular supervised

Preview: Docs Loading link description

[machine learning](#)

techniques include **regression models**, **support vector machines**, and **deep learning convolutional neural networks**. The actual

Preview: Docs Loading link description

[algorithm](#)

used with each of these techniques is different, but each requires training data. That is, we have to provide a set of already-classified data that the algorithm can “learn” from. Once the algorithm has learned from the features of the training data, it can make predictions about new data.

An important point here is that the algorithm can only be as good as the data used to train the algorithm. Maybe you’ve heard the catchphrase, “garbage in, garbage out”?

That is certainly true for predictive analysis: a predictive model trained on poor-quality data will make poor-quality predictions.

Instructions

1. Take a look at the graphic in the learning environment and consider the following questions:

How is *supervised* machine learning used in predictive analysis different from *unsupervised* machine learning used in exploratory analysis?

If training data used to train an algorithm had many mistakes or mislabeled data, would the output of the algorithm be trustworthy?

Think about times in your life when you interact with predictive algorithms.

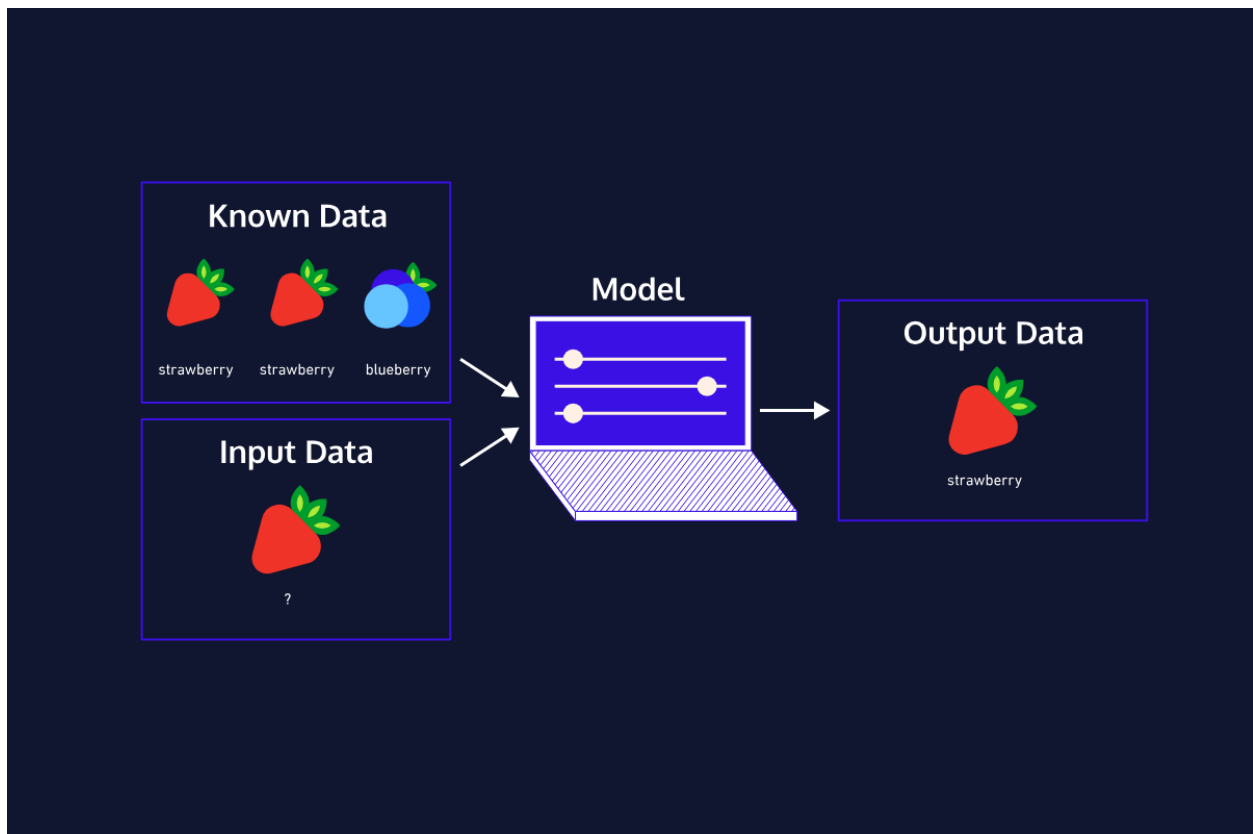
How accurate are they?

2. Remember:

Supervised machine learning algorithms are trained with labeled data and predict the likelihood of future outcomes.

Supervised machine learning algorithms can only be as good as the data used to train them.

Next, we will think more about how predictive analysis is used in real life.



Predictive Analysis: Challenges

Predictive analyses have tremendous power. As a result, we need to be careful about when we use them and when we trust them.

Recommendation algorithms are excellent for making low-risk predictions. When Netflix makes recommendations for you, the predictions are relatively low-risk. For the most part, no one will be hurt if the recommendation is wrong. In contrast, predicting whether someone will commit a crime is a high-risk prediction, especially if the prediction is used for criminal sentencing or deciding to grant someone parole.

Let's think more about predictive analyses with an example from [an article published by WIRED in 2020](#). When the COVID-19 pandemic started in 2020, high-school students had to finish the school year virtually. Standardized tests, like the International Baccalaureate (IB) exam, were canceled. International high school students wanting to study in the U.S. must take the IB exam to attend competitive U.S. colleges and universities.

Since the exam was canceled, the IB board used a supervised

Preview: Docs Loading link description

[machine learning](#)

Preview: Docs An algorithm is a formal process used to solve a problem. They can be represented in several formats but are usually represented in pseudocode in order to communicate the process by which the algorithms solve the problems they were created to tackle.

[algorithm](#)

to predict scores for each student. Unfortunately, many students were surprised by the scores the algorithm predicted for them. The students thought they were going to do much better on the test than the algorithm predicted they would!

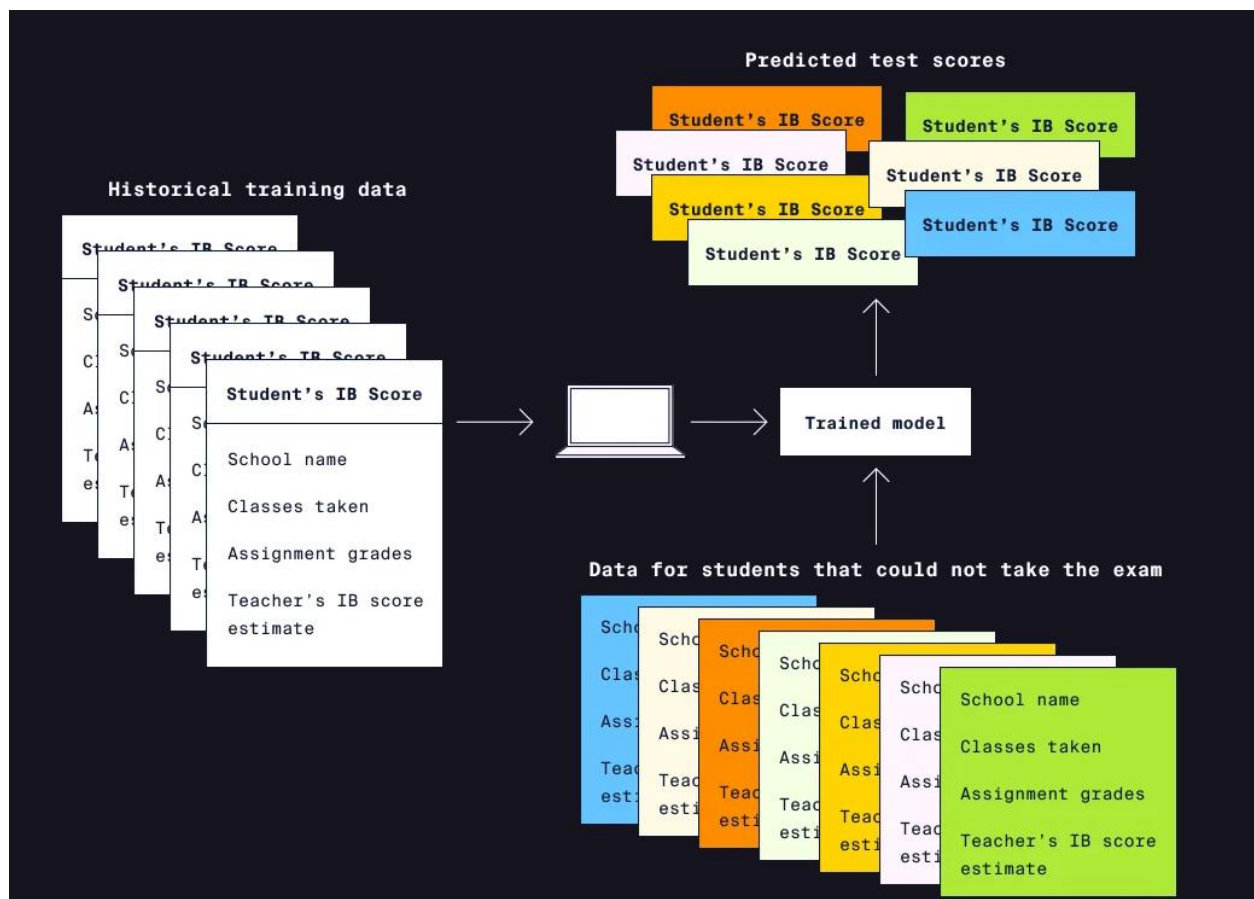
Instructions

1. The animation in the learning environment shows how the IB board used historical training data to predict IB test scores for students that could not take the IB exam. Check it out and then consider the following questions:

Is the IB test score predicting algorithm high or low risk?

Who might be more likely to be accidentally discriminated against by the model: students at large schools with a long history of students taking the exam or students at small schools that only recently began offering the IB program?

Is it fair to grant or deny college acceptance based on algorithm-predicted test scores?



Bias in Data Analysis

Bias is everywhere in data. The key to combatting bias is knowing what to look out for.

Would you follow your GPS anywhere? Even into a lake? That may sound ridiculous, but a quick [google search](#) brings up dozens of cases where drivers drove into lakes and rivers because their GPS instructed them to. Following GPS instructions against your better judgment is one example of **automation bias**.



As humans, we have many biases, both implicit and explicit. Biases are systematic errors in thinking influenced by cultural and personal experiences. Biases distort our perception and cause us to make incorrect decisions. One bias that many humans share is automation bias. Automation bias stems from the idea that computers or machines are more trustworthy than humans because they are more objective. Automation bias is at the root of why people follow their GPS into trouble, even when contradictory information is available.

Computers, data, and [algorithms](#) are not actually completely objective. It is true that data analysis can help us make better decisions, but it is not immune to bias. Humans create technologies and algorithms. As a result, they often have human biases encoded into them. It's clear that we need to pay attention to other information streams (our eyes and ears) when we drive with GPS. Similarly, we need to look at more information sources when we evaluate data analysis results or reports.

If we want to be responsible when we use data and algorithms, we need to understand the different types of bias that show up at each stage of analyzing data. Let's take a closer look at some types of bias that impact data analysis and data-driven decision-making.

Bias in data collection

Before we can analyze data or use machine learning algorithms, we need to collect data. Data collection is subject to **selection bias** (also called sample bias). Selection bias occurs when study subjects (*i.e.*, the sample) are not

representative of the population. Selection bias can be due to poor study design if the sample is too small or is not randomized. Selection bias can also crop up when the only data available is influenced by **historical bias** — systematic influence based on historic social and cultural beliefs.

[A Reuters article from 2018](#) highlights how the company Amazon produced a machine-learning algorithm that suffered from such a selection bias. The company designed the algorithm to help recruiters hire top talent. The model was trained on thousands of resumes from people that were or were not hired by Amazon. It learned 50,000 phrases associated with resumes and began to ignore common phrases, such as the names of programming languages. However, the algorithm also learned to downgrade resumes that contained the word “women’s.” This included resumes that referenced women’s colleges, teams, or committees.

This is an example of selection bias because the data used to train the algorithm were not representative of the modern applicant pool. The majority of Amazon’s past applicants and employees were male. This means a larger proportion of the successful resumes in the training data came from male applicants. Amazon did not explicitly train the algorithm to use gender. Yet, the algorithm still found and used gender-associated terms to weed out women candidates.

We can do our best to avoid selection bias by doing everything possible to have a representative sample, not just a convenient one. For example, it’s a good idea to include data inputs from multiple sources to diversify data. This is easier said than done, however, and we need to acknowledge and address historical bias in data sources and work towards building frameworks to increase inclusivity.

Bias in building and optimizing algorithms

Algorithmic bias arises when an algorithm produces systematic and repeatable errors that lead to unfair outcomes, such as privileging one group over another. Algorithmic bias can be initiated through selection bias and then reinforced and perpetuated by other bias types.

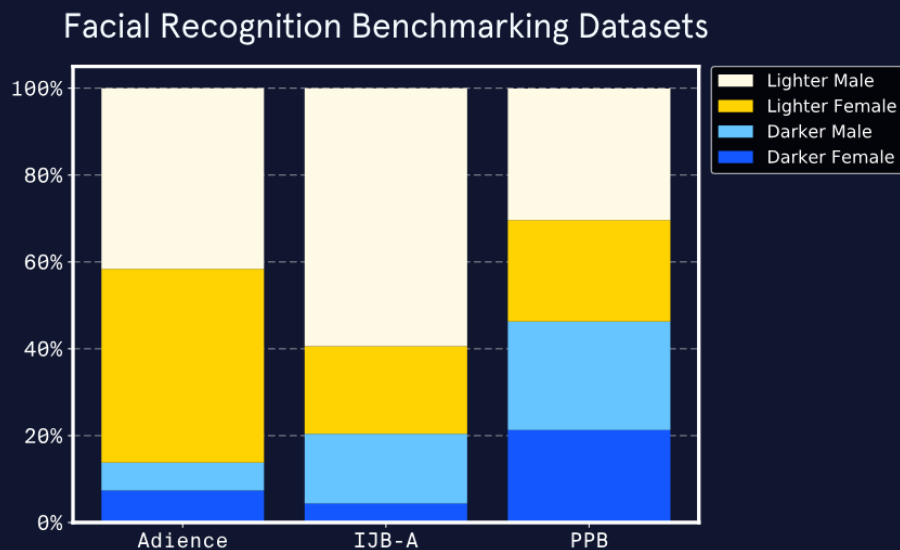
Facial recognition software is an area where algorithmic bias can do a lot of harm. This software is sold to police departments and used to recognize criminals in surveillance footage. If the software systematically makes more mistakes depending on race or gender, people in some groups will be incorrectly pursued more often, which has serious, negative outcomes for individuals.

[The Gender Shades project](#) tested commercial facial recognition software for these kinds of biases. IBM, Microsoft, and Face++ are three companies that offer facial recognition software with a binary gender classifier feature. Researchers assessed the accuracy of these algorithms and discovered that they suffered from algorithmic bias. The algorithms were good at identifying lighter males, okay at identifying darker males and lighter females, and very bad at identifying darker females.

Each software used proprietary algorithms and did not report performance results with benchmarking datasets. However, the developers probably tested the software on one of two commonly-used benchmarking datasets: Adience or IJB-A. These datasets include few dark-skinned people and especially low proportions of dark-skinned females. Testing an algorithm with a non-representative dataset leads to **evaluation bias**. Testing with a non-representative benchmarking dataset would give high overall accuracy scores, even if the algorithms were inaccurate for certain groups.

Another key point when it comes to algorithmic bias in facial recognition software is that the algorithms are proprietary, making them “black boxes”. In addition to not knowing what data were used to train and test the algorithm, we can’t know how it was designed or how it works. As a result, it’s impossible to evaluate the algorithms themselves.

Avoiding algorithmic bias relies on transparency, especially concerning data used for training and testing an algorithm. In response to the poor performance of facial recognition with darker females, a new benchmarking dataset was developed (PPB) that is more representative of the full spectrum of humanity. This is a big step forward, as long as the new dataset is actually used by companies making and selling facial recognition software.



Data for this plot came from [Buolamwini et al. 2018, *Proceedings of Machine Learning Research*](#).

Bias in interpreting results and drawing conclusions

Bias also influences the final stages of data analysis: interpreting results and drawing conclusions. The following bias types are ones we should watch out for when evaluating or generating data reports:

Confirmation bias is our tendency to seek out information that supports our views. Confirmation bias influences data analysis when we consciously or unconsciously interpret results in a way that supports our original hypothesis. To limit confirmation bias, clearly state hypotheses and goals before starting an analysis, and then honestly evaluate how they influenced our interpretation and reporting of results.

Overgeneralization bias is inappropriately extending observations made with one dataset to other datasets, leading to overinterpreting results and unjustified extrapolation. To limit overgeneralization bias, be thoughtful when interpreting data, only extend results beyond the dataset used to generate them when it is justified, and only extend results to the proper population.

Reporting bias is the human tendency to only report or share results that affirm our beliefs or hypotheses, also known as “positive” results. Editors, publishers, and readers are also subject to reporting bias as positive results are published, read, and cited more often. To limit reporting bias, report negative results and cite others who do, too.

Conclusions

Data and machine learning algorithms are now ubiquitous. They influence decisions about who is hired or fired, accepted into schools, or allowed to rent houses. They even influence which neighborhoods are more heavily policed and who is granted parole. Therefore, we must recognize that data and algorithms can be biased, just like the humans who create and train them. Learning more about the types of bias that influence how algorithms function will improve our ability to perform and interpret data analyses and will help us make more informed decisions.

The Role of Databases in Data Science

Gear up for the fun you'll have in SQL Fundamentals!

As a data scientist or data analyst, you will likely be solving business problems with data (and therefore databases) regularly. Even if you aren't the person deep into the data, you will have to understand how databases work in order to understand how data is stored. This unit will get you ready for answering business questions with the most popular database language – SQL!

What will SQL Fundamentals cover?

- Manipulating Data
- Querying Data
- Aggregating Data
- Working with Multiple Tables

- Data Acquisition

Why did we build this?

Databases are at the core of Data Science and are where most data is stored.

Understanding databases is an essential skill for everyone who works with data.

We decided to introduce you to databases before Python because databases are more foundational to data science and there is a lower barrier to entry. Once you complete this track, you will immediately be able to apply your skills to real data problems and query data to answer questions.

If you would prefer to get started with Python first, feel free to jump ahead and come back to SQL and databases later. It will be here. Databases have been at the core of data science for decades, they'll be here when you get back.

Who are my classmates?

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the forums. Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the projects category, which can help to reinforce what you've learned.

A Day in the Life - Data Analyst

It's Catherine's first day working as a Data Analyst for Codecademy. She's excited to use some of the tools that she learned in [Analyze Data with SQL](#), such as:

- Writing basic queries
- Calculating aggregates
- Combining data from multiple tables
- Determining web traffic attribution
- Creating usage funnels
- Analyzing user churn

Query Results		
user_id	browse_date	item_id
336f9fdc-aaeb-48a1-a773-e3a935442d45	2017-12-20	3
336f9fdc-aaeb-48a1-a773-e3a935442d45	2017-12-20	22
336f9fdc-aaeb-48a1-a773-e3a935442d45	2017-12-20	25
336f9fdc-aaeb-48a1-a773-e3a935442d45	2017-12-20	24
4596bb1a-7aa9-4ac9-9896-022d871cdcde	2017-12-20	0
4596bb1a-7aa9-4ac9-9896-022d871cdcde	2017-12-20	2
2fdb3958-ffc9-4b84-a49d-5f9f40e9469e	2017-12-20	26
2fdb3958-ffc9-4b84-a49d-5f9f40e9469e	2017-12-20	24
fc394c75-36f1-4df1-8665-23c32a43591b	2017-12-20	12
fc394c75-36f1-4df1-8665-23c32a43591b	2017-12-20	24
Database Schema		
browse		
name	type	
user_id	TEXT	
browse_date	TEXT	
item_id	INTEGER	
Rows: 2352		
items		
name	type	
id	INTEGER	
item	TEXT	
price	REAL	
Rows: 33		
checkout		
name	type	
user_id	TEXT	
checkout_date	TEXT	
item_id	INTEGER	
Rows: 539		
purchase		

Database Schema	
browse	
name	type
user_id	TEXT
browse_date	TEXT
item_id	INTEGER
Rows: 2352	
items	
name	type
id	INTEGER
item	TEXT
price	REAL
Rows: 33	
checkout	
name	type
user_id	TEXT
checkout_date	TEXT
item_id	INTEGER
Rows: 539	
purchase	
name	type
user_id	TEXT
purchase_date	TEXT
item_id	INTEGER
Rows: 472	

Creating Usage Funnels

Visitors to Codecademy's website follow a simple workflow:

Browse items available for sale

Click an icon to begin the checkout process

Enter payment information to complete their purchase

Not all users who browse on the website will find something that they like enough to checkout, and not all users who begin the checkout process will finish entering their payment information to make a purchase.

This type of multi-step process where some users leave at each step is called a *funnel*.

Catherine wants to determine what percent of users make it through each step of the funnel so that she can recommend improvements to Codecademy's website.

Instructions

Checkpoint 1 Passed

1.

Catherine is going to combine data from three different tables:

`browse` - gives the timestamps of users who visited different item description pages

`checkout` - gives the timestamps of users who visited the checkout page

`purchase` - gives the timestamps of when users complete their purchase

Using SQL, she finds that 24% of all users who browse move on to checkout. 89% of those who reach checkout purchase.

Click Run, to see Catherine's analysis.

test.sqlite

```

1 SELECT ROUND(
2     100.0 * COUNT(DISTINCT c.user_id) /
3     COUNT(DISTINCT b.user_id)
4 ) AS browse_to_checkout_percent,
5 ROUND(
6     100.0 * COUNT(DISTINCT p.user_id) /
7     COUNT(DISTINCT c.user_id)
8 ) AS checkout_to_purchase_percent
9 FROM browse b
10 LEFT JOIN checkout c
11     ON b.user_id = c.user_id
12 LEFT JOIN purchase p
13     ON c.user_id = p.user_id;

```

Run

Query Results

browse_to_checkout_percent	checkout_to_purchase_percent
24.0	89.0

Database Schema

browse

name	type
user_id	TEXT
browse_date	TEXT
item_id	INTEGER

Rows: 2352

items

name	type
id	INTEGER
item	TEXT
price	REAL

Rows: 33

checkout

name	type
user_id	TEXT
checkout_date	TEXT
item_id	INTEGER

Rows: 539

purchase

name	type
user_id	TEXT
purchase_date	TEXT
item_id	INTEGER

Rows: 472

Back
Next

3/6

Analyzing User Churn

Next, Catherine wants to take a look at the churn rate.

A *churn rate* is the percent of subscribers to a monthly service who have canceled. For example, in January, let's say Codecademy has 1,000 customers. In February, 200 learners sign up, and 250 cancel.

The churn rate for February would be:

$$\frac{\text{cancellations}}{\text{total subscribers}} = \frac{250}{1000 + 200} = 20.8\%$$

$$\text{Cancellations} = \frac{250}{1000 + 200} = 20.8\%$$

Catherine wants to analyze the churn rates for Codecademy for the past few months so she writes another SQL query.

Instructions

Checkpoint 1 Passed

1.

Click Run, to see Catherine's analysis for the churn rate in March 2017.

What recommendations would you make to Codecademy based on Catherine's analysis?

(This query might take some time to load because the `pro_users` table has 118,135 rows!)

test.sqlite

```
1 SELECT COUNT(DISTINCT user_id) AS enrollments,
2 COUNT(CASE
3     WHEN strftime("%m", cancel_date) = '03'
4     THEN user_id
5 END) AS march_cancellations,
6 ROUND(100.0 * COUNT(CASE
7     WHEN strftime("%m", cancel_date) = '03'
8     THEN user_id
9 END) / COUNT(DISTINCT user_id)) AS churn_rate
10 FROM pro_users
11 WHERE signup_date < '2017-04-01'
12 AND (
13     (cancel_date IS NULL) OR
14     (cancel_date > '2017-03-01')
15 );
```

Run

Query Results

enrollments	march_cancellations	churn_rate
16435	4165	25.0

Database Schema

pro_users	
name	type
user_id	TEXT
signup_date	TEXT
cancel_date	TEXT

Rows: 118135

4/6

BackNext

What is a Relational Database Management System?

Learn about RDBMS and the language used to access large datasets – SQL.

What is a Database?

A *database* is a set of data stored in a computer. This data is usually structured in a way that makes the data easily accessible.

What is a Relational Database?

A *relational database* is a type of database. It uses a structure that allows us to identify and access data *in relation* to another piece of data in the database. Often, data in a relational database is organized into tables.

Tables: Rows and Columns

Tables can have hundreds, thousands, sometimes even millions of rows of data. These rows are often called *records*.

Tables can also have many *columns* of data. Columns are labeled with a descriptive name (say, *age* for example) and have a specific *data type*.

For example, a column called *age* may have a type of *INTEGER* (denoting the type of data it is meant to hold).

NAME	AGE	COUNTRY
Natalia	11	Iceland
Ned	6	United States
Zenas	14	Ireland
Laura	8	Kenya

In the table above, there are three columns (*name*, *age*, and *country*).

The `name` and `country` columns store string data types, whereas `age` stores integer data types. The set of columns and data types make up the schema of this table.

The table also has four rows, or records, in it (one each for Natalia, Ned, Zenas, and Laura).

What is a Relational Database Management System (RDBMS)?

A relational database management system (RDBMS) is a program that allows you to create, update, and administer a relational database. Most relational database management systems use the SQL language to access the database.

What is SQL?

SQL (**Structured Query Language**) is a **programming language** used to communicate with data stored in a relational database management system. SQL syntax is similar to the English language, which makes it relatively easy to write, read, and interpret.

Many RDBMSs use SQL (and variations of SQL) to access the data in tables. For example, SQLite is a relational database management system. SQLite contains a minimal set of SQL commands (which are the same across all RDBMSs). Other RDBMSs may use other variants.

(SQL is often pronounced in one of two ways. You can pronounce it by speaking each letter individually like “S-Q-L”, or pronounce it using the word “sequel”.)

Popular Relational Database Management Systems

SQL syntax may differ slightly depending on which RDBMS you are using. Here is a brief description of popular RDBMSs:

MySQL

MySQL is the most popular open source SQL database. It is typically used for web application development, and often accessed using PHP.

The main advantages of MySQL are that it is easy to use, inexpensive, reliable (has been around since 1995), and has a large community of developers who can help answer questions.

Some of the disadvantages are that it has been known to suffer from poor performance when scaling, open source development has lagged since Oracle has taken control of MySQL, and it does not include some advanced features that developers may be used to.

PostgreSQL

PostgreSQL is an open source SQL database that is not controlled by any corporation. It is typically used for web application development.

PostgreSQL shares many of the same advantages of MySQL. It is easy to use, inexpensive, reliable and has a large community of developers. It also provides some additional features such as foreign key support without requiring complex configuration.

The main disadvantage of PostgreSQL is that it can be slower in performance than other databases such as MySQL. It is also slightly less popular than MySQL.

For more information about PostgreSQL including installation instructions, read [this](#) article.

Oracle DB

Oracle Corporation owns Oracle Database, and the code is not open sourced.

Oracle DB is for large applications, particularly in the banking industry. Most of the world's top banks run Oracle applications because Oracle offers a powerful combination of technology and comprehensive, pre-integrated business applications, including essential functionality built specifically for banks.

The main disadvantage of using Oracle is that it is not free to use like its open source competitors and can be quite expensive.

SQL Server

Microsoft owns SQL Server. Like Oracle DB, the code is close sourced.

Large enterprise applications mostly use SQL Server.

Microsoft offers a free entry-level version called *Express* but can become very expensive as you scale your application.

SQLite

SQLite is a popular open source SQL database. It can store an entire database in a single file. One of the most significant advantages this provides is that all of the data can be stored locally without having to connect your database to a server.

SQLite is a popular choice for databases in cellphones, PDAs, MP3 players, set-top boxes, and other electronic gadgets. The SQL courses on Codecademy use SQLite.

For more info on SQLite, including installation instructions, read [this](#) article.

Using An RDBMS On Codecademy

On Codecademy, we use both SQLite and PostgreSQL. While this may sound confusing, don't worry! We want to stress that the basic syntax you will learn can be used in both systems. For example, the syntax to create tables, insert data into those tables, and retrieve data from those tables are all identical. That's one of the nice parts of learning SQL — by learning the fundamentals with one RDBMS, you can easily begin work in another.

That being said, let's take a look at some of the more subtle details:

File extensions — when working with databases on Codecademy, take a look at the name of the file you're writing in. If your file ends in `.sqlite`, you're using a SQLite database. If your file ends in `.sql`, you're working with PostgreSQL.

Data types — You'll learn about data types very early into learning a RDBMS. One thing to note is that SQLite and PostgreSQL have slightly different data types. For example, if you want to store text in a SQLite database, you'll use the `TEXT` data type. If you're working with PostgreSQL, you have many more options. You could use `varchar(n)`, `char(n)`, or `text`.

Each type has its own subtle differences. This is a good example of PostgreSQL being slightly more robust than SQLite, but the core concepts remaining the same.

Built-in tables — As you work your way through more complicated lessons on databases, you'll start to learn how to access built-in tables. For example, if you take our lesson on indexes, you'll learn how to look at the table that the system automatically creates to keep track of what indexes exist. Depending on which RDBMS system you are using (in that lesson we're using PostgreSQL), the syntax for doing that will be different. Any time you're writing SQL about the database itself, rather than the data, that syntax will likely be unique to the RDBMS you're using.

Conclusion

Relational databases store data in tables. Tables can grow large and have a multitude of columns and records. Relational database management systems (RDBMSs) use SQL (and variants of SQL) to manage the data in these large tables. The RDBMS you use is your choice and depends on the complexity of your application.

Queries

Select

Previously, we learned that

Preview: Docs Every SQL query will begin with the SELECT command to fetch data from one or more tables.

SELECT

is used every time you want to query data from a database and `*` means *all* columns.

Suppose we are only interested in two of the columns. We can select individual columns by their names (separated by a comma):

```
SELECT column1, column2
FROM table_name;
```

To make it easier to read, we moved `FROM` to another line.

Line breaks don't mean anything specific in SQL. We could write this entire query in one line, and it would run just fine.

As

Knowing how

Preview: Docs Every SQL query will begin with the `SELECT` command to fetch data from one or more tables.

`SELECT`

works, suppose we have the code below:

```
SELECT name AS 'Titles'  
FROM movies;
```

Can you guess what `AS` does?

Preview: Docs Renames a column or table with an alias for display, does not permanently change table/column names in the database.

`AS`

is a keyword in SQL that allows you to *rename* a column or table using an alias. The new name can be anything you want as long as you put it inside of single quotes. Here we renamed the `name` column as `Titles`.

Some important things to note:

Although it's not always necessary, it is considered best practice to surround your aliases with single quotes.

Note that this practice is specific to SQLite, the RDBMS used in this exercise. When you work with other RDBMSs, notably PostgreSQL, [no quotes or double quotes](#) may be required in place of single quotes.

When using `AS`, the columns are not being renamed in the table. The aliases only appear in the result.

Distinct

When we are examining data in a table, it can be helpful to know what *distinct* values exist in a particular column.

Preview: Docs Returns unique values in the specified column(s). `DISTINCT` is used to return unique values in the output. It filters out all duplicate values in the specified column(s).

For instance,

```
SELECT tools
FROM inventory;
```

By adding `DISTINCT` before the column name,

```
SELECT DISTINCT tools
FROM inventory;
```

Filtering the results of a query is an important skill in SQL. It is easier to see the different possible `genres` in the `movie` table after the data has been filtered than to scan every row in the table.

Where

We can restrict our query results using the

Preview: Docs Loading link description `WHERE` clause in order to obtain only the information we want.

Following this format, the statement below filters the result set to only include top rated movies (IMDb ratings greater than 8):

```
SELECT *
FROM movies
WHERE imdb_rating > 8;
```

How does it work?

The `WHERE` clause filters the result set to only include rows where the following *condition* is true.

`imdb_rating > 8` is the condition. Here, only rows with a value greater than 8 in the `imdb_rating` column will be returned.

The `>` is an *operator*. Operators create a condition that can be evaluated as either *true* or *false*.

Comparison operators used with the `WHERE` clause are:

- `=` equal to
- `!=` not equal to
- `>` greater than
- `<` less than
- `>=` greater than or equal to
- `<=` less than or equal to

There are also some special operators that we will learn more about in the upcoming exercises.

Like I

Preview: Docs Returns ``TRUE`` if its first text argument matches the wildcard pattern in its second argument. `LIKE` can be a useful operator when you want to compare similar values.

The `movies` table contains two films with similar titles, 'Se7en' and 'Seven'.

How could we select all movies that start with 'Se' and end with 'en' and have exactly one character in the middle?

```
SELECT *  
FROM movies  
WHERE name LIKE 'Se_en';
```

`LIKE` is a special operator used with the

Preview: Docs Loading link description

`WHERE`

clause to search for a specific pattern in a column.

`name LIKE 'Se_en'` is a condition evaluating the `name` column for a specific pattern.

`Se_en` represents a pattern with a *wildcard* character.

The `_` means you can substitute any individual character here without breaking the pattern. The names `Seven` and `Se7en` both match this pattern.

Like II

The percentage sign `%` is another wildcard character that can be used with

Preview: Docs Returns `TRUE` if its first text argument matches the wildcard pattern in its second argument. `LIKE`. This statement below filters the result set to only include movies with names that begin with the letter 'A':

```
SELECT *
FROM movies
WHERE name LIKE 'A%';
```

`%` is a wildcard character that matches zero or more missing characters in the pattern. For example:

`A%` matches all movies with names that begin with letter 'A'

`%a` matches all movies that end with 'a'

We can also use `%` both before and after a pattern:

```
SELECT *
FROM movies
WHERE name LIKE '%man%';
```

Here, any movie that *contains* the word 'man' in its name will be returned in the result. `LIKE` is not case sensitive. 'Batman' and 'Man of Steel' will both appear in the result of the query above.

What are the different use cases of the `LIKE` operator?

- `LIKE c%` finds any values that start with the letter 'c'
- `LIKE %c` finds any values that end with the letter 'c'
- `LIKE %re%` finds values that have 're' in any position

- `LIKE _a%` finds any values that have the letter 'a' in the second index
- `LIKE a_%_%` finds any values that start with 'a' and are at least 3 characters in length.
- `LIKE a%r` finds any values that start with 'a' and end with 'r'.

Is Null

By this point of the lesson, you might have noticed that there are a few missing values in the `movies` table. More often than not, the data you encounter will have missing values.

Unknown values are indicated by `NULL`.

It is not possible to test for `NULL` values with comparison operators, such as `=` and `!=`.

Instead, we will have to use these operators:

Preview: Docs Checks if a value is NULL.

`IS NULL`

Preview: Docs Checks if a value is not NULL.

`IS NOT NULL`

To filter for all movies *with* an IMDb rating:

```
SELECT name
FROM movies
WHERE imdb_rating IS NOT NULL;
```

Between

The BETWEEN works with numbers, text, or date data types.

`BETWEEN` operator is used in a `WHERE` clause to filter the result set within a certain *range*. It accepts two values that are either numbers, text or dates.

For example, this statement filters the result set to only include movies with `year`s from 1990 up to, *and including* 1999.

```
SELECT *  
FROM movies  
WHERE year BETWEEN 1990 AND 1999;
```

When the values are text, `BETWEEN` filters the result set for within the alphabetical range.

In this statement, `BETWEEN` filters the result set to only include movies with `name`s that begin with the letter 'A' up to, *but not including* ones that begin with 'J'.

```
SELECT *  
FROM movies  
WHERE name BETWEEN 'A' AND 'J';
```

However, if a movie has a name of simply 'J', it would actually match. This is because `BETWEEN` goes *up to* the second value — up to 'J'. So the movie named 'J' would be included in the result set but not 'Jaws'.

And

Sometimes we want to *combine multiple conditions* in a `WHERE` clause to make the result set more specific and useful.

One way of doing this is to use the

Preview: Docs Loading link description `AND` operator. Here, we use the `AND` operator to only return 90's romance movies.

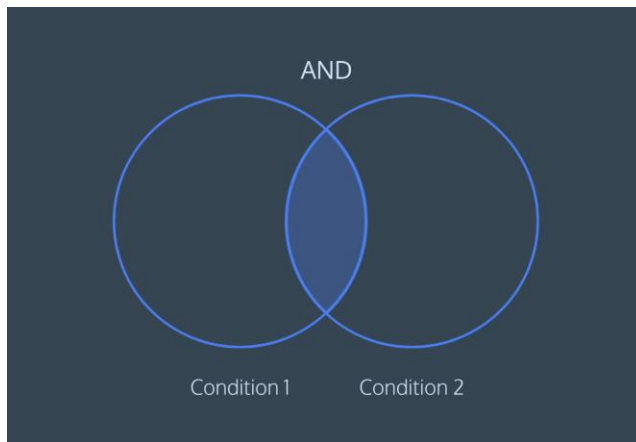
```
SELECT *  
FROM movies  
WHERE year BETWEEN 1990 AND 1999
```

```
AND genre = 'romance';
```

`year BETWEEN 1990 AND 1999` is the 1st condition.

`genre = 'romance'` is the 2nd condition.

`AND` combines the two conditions.



With `AND`, *both* conditions must be true for the row to be included in the result.

Or

Similar to `AND`, the `OR` operator can also be used to combine multiple conditions in

Preview: Docs Loading link description

WHERE

, but there is a fundamental difference:

Preview: Docs Loading link description

AND

operator displays a row if *all* the conditions are true.

Preview: Docs Loading link description

OR

operator displays a row if *any* condition is true.

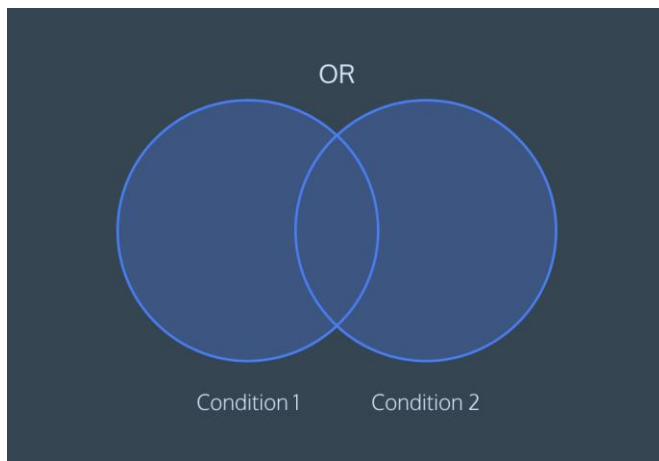
Suppose we want to check out a new movie or something action-packed:

```
SELECT *  
FROM movies  
WHERE year > 2014  
OR genre = 'action';
```

`year > 2014` is the 1st condition.

`genre = 'action'` is the 2nd condition.

`OR` combines the two conditions.



With `OR`, if any of the conditions are true, then the row is added to the result.

Order By

That's it with `WHERE` and its

[operators](#)

. Moving on!

It is often useful to list the data in our result set in a particular order.

We can *sort* the results using

Preview: Docs Loading link description

`ORDER BY`

, either alphabetically or numerically. Sorting the results often makes the data more useful and easier to analyze.

For example, if we want to sort everything by the movie's title from A through Z:

```
SELECT *  
FROM movies  
ORDER BY name;
```

`ORDER BY` is a clause that indicates you want to sort the result set by a particular column.

`name` is the specified column.

Sometimes we want to sort things in a decreasing order. For example, if we want to select all of the well-received movies, sorted from highest to lowest by their year:

```
SELECT *  
FROM movies  
WHERE imdb_rating > 8  
ORDER BY year DESC;
```

`DESC` is a keyword used in `ORDER BY` to sort the results in *descending order* (high to low or Z-A).

`ASC` is a keyword used in `ORDER BY` to sort the results in *ascending order* (low to high or A-Z).

The column that we `ORDER BY` doesn't even have to be one of the columns that we're displaying.

Note: `ORDER BY` always goes after `WHERE` (if `WHERE` is present).

Limit

We've been working with a fairly small table (fewer than 250 rows), but most SQL tables contain hundreds of thousands of records. In those situations, it becomes important to cap the number of rows in the result.

For instance, imagine that we just want to see a few examples of records.

```
SELECT *  
FROM movies  
LIMIT 10;
```

Copy to Clipboard

Preview: Docs Specifies, or limits, the maximum number of rows the result set will have.

LIMIT

is a clause that lets you specify the maximum number of rows the result set will have. This saves space on our screen and makes our queries run faster.

Here, we specify that the result set can't have more than 10 rows.

LIMIT always goes at the very end of the query. Also, it is not supported in all SQL databases.

Case

A CASE statement allows us to create different outputs (usually in the SELECT statement). It is SQL's way of handling [if-then](#) logic.

Suppose we want to condense the ratings in `movies` to three levels:

If the rating is above 8, then it is Fantastic.

If the rating is above 6, then it is Poorly Received.

Else, Avoid at All Costs.

```
SELECT name,  
CASE  
  WHEN imdb_rating > 8 THEN 'Fantastic'  
  WHEN imdb_rating > 6 THEN 'Poorly Received'  
  ELSE 'Avoid at All Costs'  
END  
FROM movies;
```


Each `WHEN` tests a condition and the following `THEN` gives us the string if the condition is true.

The `ELSE` gives us the string if *all* the above conditions are false.

The `CASE` statement must end with `END`.

In the result, you have to scroll right because the column name is very long. To shorten it, we can rename the column to 'Review' using

Preview: Docs Renames a column or table with an alias for display, does not permanently change table/column names in the database.

AS

|

```
SELECT name,  
CASE  
  WHEN imdb_rating > 8 THEN 'Fantastic'  
  WHEN imdb_rating > 6 THEN 'Poorly Received'  
  ELSE 'Avoid at All Costs'  
END AS 'Review'  
FROM movies;
```

Let's summarize:

- `SELECT` is the clause we use every time we want to query information from a database.
- `AS` renames a column or table.
- `DISTINCT` returns unique values.
- `WHERE` is a popular command that lets you filter the results of the query based on conditions that you specify.
- `LIKE` and `BETWEEN` are special operators.

- AND and OR combine multiple conditions.
- ORDER BY sorts the result.
- LIMIT specifies the maximum number of rows that the query will return.
- CASE creates different outputs.

```

0
SELECT *
FROM nomnom;

-1Select all:
-SELECT DISTINCT neighborhood
-FROM nomnom;

2
-ELECT DISTINCT cuisine
-FROM nomnom;

3
SELECT * FROM nomnom
WHERE cuisine = 'Chinese';

4
SELECT * FROM nomnom
WHERE REVIEW >= 4;

5
SELECT * FROM nomnom
WHERE cuisine = 'Italian' AND price = '$$$';

6
SELECT * FROM nomnom
WHERE name LIKE '%meatball%';

7
SELECT * FROM nomnom
WHERE neighborhood = 'Midtown'
   OR neighborhood = 'Downtown'
   OR neighborhood = 'Chinatown';

SELECT * FROM nomnom
WHERE health IS NULL;

SELECT * FROM nomnom
ORDER BY review DESC
LIMIT 10;

SELECT name,
CASE
  WHEN review > 4.5 THEN 'Extraordinary'
  WHEN review > 4 THEN 'Excellent'
  WHEN review > 3 THEN 'Good'
  WHEN review > 2 THEN 'Fair'
  ELSE 'Poor'
END AS 'Review'
FROM nomnom;

```

Aggregate Functions

QL Queries don't just access raw data, they can also perform calculations on the raw data to answer specific data questions.

Calculations performed on multiple rows of a table are called **aggregates**.

In this lesson, we have given you a table named `fake_apps` which is made up of fake mobile applications data.

Here is a quick preview of some important aggregates that we will cover in the next five exercises:

- `COUNT()`: count the number of rows
- `SUM()`: the sum of the values in a column
- `MAX()`/`MIN()`: the largest/smallest value
- `AVG()`: the average of the values in a column
- `ROUND()`: round the values in the column

Count

The fastest way to calculate how many rows are in a table is to use the `COUNT()` function.

Preview: Docs Loading link description

`COUNT()`

is a function that takes the name of a column as an argument and counts the number of non-empty values in that column.

```
SELECT COUNT(*)  
FROM table_name;
```

Here, we want to count every row, so we pass `*` as an argument inside the parenthesis.

```
SELECT COUNT(*)  
FROM fake_apps;
```

```
SELECT COUNT(*)
```

```
FROM fake_apps  
WHERE price = 0;
```

Sum

SQL makes it easy to add all values in a particular column using `SUM()`.

`SUM()` is a function that takes the name of a column as an argument and returns the sum of all the values in that column.

What is the total number of downloads for all of the apps combined?

```
SELECT SUM(downloads)  
FROM fake_apps;
```

This adds all values in the `downloads` column.

Max / Min

The `MAX()` and `MIN()` functions return the highest and lowest values in a column, respectively.

How many downloads does the most popular app have?

```
SELECT MAX(downloads)  
FROM fake_apps;
```

The most popular app has 31,090 downloads!

`MAX()` takes the name of a column as an argument and returns the largest value in that column.

Here, we returned the largest value in the `downloads` column.

`MIN()` works the same way but it does the exact opposite; it returns the smallest value.

Average

SQL uses the

`AVG()` function to quickly calculate the average value of a particular column.

The statement below returns the average number of downloads for an app in our database:

```
SELECT AVG(downloads)
FROM fake_apps;
```

The `AVG()` function works by taking a column name as an argument and returns the average value for that column.

Round

By default, SQL tries to be as precise as possible without rounding. We can make the result table easier to read using the `ROUND()` function.

`ROUND()` function takes two arguments inside the parenthesis:

- a column name
- an integer

It rounds the values in the column to the number of decimal places specified by the integer.

```
SELECT ROUND(price, 0)
FROM fake_apps;
```

Here, we pass the column `price` and integer `0` as arguments. SQL rounds the values in the column to 0 decimal places in the output.

```
SELECT ROUND(AVG(download, 2))
FROM fake_apps;
```

Group By I

Oftentimes, we will want to calculate an aggregate for data with certain characteristics.

For instance, we might want to know the mean IMDb ratings for all movies each year. We could calculate each number by a series of queries with different `WHERE` statements, like so:

```
SELECT AVG(imdb_rating)
FROM movies
WHERE year = 1999;

SELECT AVG(imdb_rating)
FROM movies
WHERE year = 2000;

SELECT AVG(imdb_rating)
FROM movies
WHERE year = 2001;
```

and so on.

Luckily, there's a better way!

We can use `GROUP BY` to do this in a single step:

```
SELECT year,
       AVG(imdb_rating)
FROM movies
GROUP BY year
ORDER BY year;
```

`GROUP BY` is a clause in SQL that is used with

[aggregate functions](#)

. It is used in collaboration with the `SELECT` statement to arrange identical data into *groups*.

The `GROUP BY` statement comes after any `WHERE` statements, but before `ORDER BY` or `LIMIT`

```
SELECT price, COUNT(*)
FROM fake_apps
GROUP BY price;
```

? In the previous query, add a `WHERE` clause to count the total number of apps that have been downloaded more than 20,000 times, at each price.

```
SELECT price, COUNT(*)
FROM fake_apps
```

```
WHERE downloads > 20000  
GROUP BY price;
```

Write a new query that calculates the total number of downloads for each category.

Select `category` and `SUM(downloads)`.

```
SELECT category, SUM(downloads) AS 'total'  
  
FROM fake_apps  
  
GROUP BY category;  
  
-- ORDER BY 2; - Will order based on values of SUM (second column)
```

Group By II

Sometimes, we want to `GROUP BY` a calculation done on a column.

For instance, we might want to know how many movies have IMDb ratings that round to 1, 2, 3, 4, 5.

We could do this using the following syntax:

```
SELECT ROUND(imdb_rating),  
       COUNT(name)  
FROM movies  
GROUP BY ROUND(imdb_rating)  
ORDER BY ROUND(imdb_rating);
```

However, this query may be time-consuming to write and more prone to error.

SQL lets us use column reference(s) in our `GROUP BY` that will make our lives easier.

- 1 is the first column selected
- 2 is the second column selected
- 3 is the third column selected

and so on. The following query is equivalent to the one above:

```
SELECT ROUND(imdb_rating),  
       COUNT(name)  
FROM movies  
GROUP BY 1  
ORDER BY 1;
```

Here, the `1` refers to the first column in our `SELECT` statement, `ROUND(imdb_rating)`.

```
SELECT category,
       price,
       AVG(downloads)
FROM fake_apps
GROUP BY category, price;
```

Note: Even if you use column names instead of numbers, it will still be correct because these two queries are exactly the same!

```
SELECT category,
       price,
       AVG(downloads)
FROM fake_apps
GROUP BY 1, 2;
```

Having

In addition to being able to group data using `GROUP BY`, SQL also allows you to filter which groups to include and which to exclude.

For instance, imagine that we want to see how many movies of different genres were produced each year, but we only care about years and genres with at least 10 movies.

We can't use `WHERE` here because we don't want to filter the rows; we want to *filter groups*.

This is where `HAVING` comes in. `HAVING` is very similar to `WHERE`. In fact, all types of `WHERE` clauses you learned about thus far can be used with `HAVING`.

We can use the following for the problem:

```
SELECT year,
       genre,
       COUNT(name)
FROM movies
GROUP BY 1, 2
HAVING COUNT(name) > 10;
```

When we want to limit the results of a query based on values of the individual rows, use `WHERE`.

When we want to limit the results of a query based on an aggregate property, use `HAVING`.

HAVING statement always comes after GROUP BY, but before ORDER BY and LIMIT

```
SELECT price,  
       ROUND(AVG(downloads)),  
       COUNT(*)  
FROM fake_apps  
GROUP BY price  
HAVING COUNT(*) > 10;
```

Review

You just learned how to use aggregate functions to perform calculations on your data. What can we generalize so far?

- COUNT() : count the number of rows
- SUM() : the sum of the values in a column
- MAX() / MIN() : the largest/smallest value
- AVG() : the average of the values in a column
- ROUND() : round the values in the column

users

id	first_name	last_name	email	password
The user's id	The first name of the user	The last name of the user	The user's email	The hashed password of the user

payments

id	user_id	amount	status	pay_date
The id of the payment	The id of the user making the payment	The payment amount	The status of the payment ("paid" or "failed")	The date of the payment

watch_history

id	user_id	watch_date	watch_duration_in_minutes
The id of the watch history	The user who the watch history belongs to	When the watch history occurred	How long the user watched for

Learn AI for SQL

- Structuring an AI Conversation

It can be tempting to spend a lot of time crafting the “perfect prompt” to get the exact output we want. But it is often more efficient (and more powerful!) to converse back-and-forth with the AI assistant throughout the analytics process, iterating and improving upon the AI output as we go.

By using a conversational approach, we

- Avoid repeating the same information in multiple

- Preview: Docs Loading link description

- [prompts](#)

- .

- Save time crafting overly detailed prompts.

- Build the habit of reflecting and iterating on the AI output.

This last step is crucial. Remember, while AI can provide valuable insights, it’s not infallible. We shouldn’t use AI output unless we understand how it works!

We recommend the following outline for interacting with AI:

Explain the goal (I'm working on an analytics report about electric vehicles)

Provide context (I have three tables with the following names)

State your request clearly (Write a query to determine the most efficient vehicle)

Review and refine (Can you explain this piece of the query you generated:)

Continuing the conversation (That's great, what other types of statistics should I look into for this report?)

Alright, buckle up! Let's complete a simple task while having a fun conversation with the AI chatbot.

Important notes:

Throughout the lesson, we include our sample prompts within each **Hint**. Feel free to take a look any time you get stuck!

Some checkpoints only involve chatting with the AI, to receive the check mark and move to the next checkpoint, select **Run** under the

Preview: Docs Loading link description

[code editor](#)

.

Instructions

Checkpoint 1 Passed

1.

We'll explore techniques for writing individual prompts in later exercises, but for this exercise let's play around with having a basic conversation with the AI! Don't worry about getting everything perfect yet, but you may be surprised at how well the AI does even with basic prompts.

Begin by examining the table `models`. Run the query below to return all the columns present inside the `models` table:

```
SELECT * FROM models;
```

Copy to Clipboard

Note that `best_mpg` contains the **mile-per-gallon equivalent** under optimal conditions, a number that allows us to compare the efficiency of electric vehicles to gasoline counterparts, and so forth.

1.

We'll explore techniques for writing individual prompts in later exercises, but for this exercise let's play around with having a basic conversation with the AI! Don't worry about getting everything perfect yet, but you may be surprised at how well the AI does even with basic prompts.

Begin by examining the table `models`. Run the query below to return all the columns present inside the `models` table:



Copy to Clipboard

Note that `best_mpg` contains the **mile-per-gallon equivalent** under optimal conditions, a number that allows us to compare the efficiency of electric vehicles to gasoline counterparts, and so forth.

Hint

Type or copy-and-paste the query into the code editor (not the AI Chat). Select the `Run` button. The output of the query will appear in the Query Results pane.

Checkpoint 2 Passed

2.

Write an overview of the `models` table in the AI chat, providing information on all the column names to the AI.

Note that while this may seem tedious, it will save a lot of time and trouble down the line!

NOTE: to move onto the next checkpoint, you will need to select the `Run` button even though this checkpoint did not involve any new code.

Hint

First, explain that we have a SQL database with a `models` table

Provide the column names

Explain that we'll be asking for SQL prompts based on this table

A sample prompt for inspiration

My SQL database has a table called `models`. It has the following columns:

`model_id | model | year | manufacturer_id | fuel_id | best_mpg`

I will ask some questions based on this table, and you will have to provide SQL queries for them.

Checkpoint 3 Passed

3.

We'd like to know which manufacturer has produced the most efficient vehicles on average.

Write a prompt asking the AI to produce a SQL query returning this information.

Note that in this table, we'll only be able to identify the `manufacturer_id`, but we'll work on joining multiple tables later in the lesson.

Hint

The efficiency of the vehicle is in `best_mpg`, since MPGe is the miles-per-gallon equivalent for non-gasoline vehicles.

A sample prompt for inspiration

Write a query to identify the manufacturer that the largest average "best_mpg"

The code we received:

```
SELECT manufacturer_id, AVG(best_mpg) AS avg_best_mpg
FROM models
GROUP BY manufacturer_id
ORDER BY avg_best_mpg DESC
LIMIT 1;
```

[Copy to Clipboard](#)

Checkpoint 4 Passed

4.

Now, let's find the top ten most efficient models. Write a prompt, and run the code the AI returns!

Hint

What details does the AI already know, and what details do you need to provide at this point in the conversation?

A sample prompt for inspiration

Now, write a SQL query to return the top 10 most efficient models

The code we received:

```
SELECT *
FROM models
```

```
ORDER BY best_mpge DESC
LIMIT 10;
```

Copy to Clipboard

Community Support

Still have questions? Get help from the [Codecademy community](#).

Code Editor

Open file navigation

test.sqlite

Close test.sqlite

Enter Fullscreen Code Editor

```
SELECT * FROM models;
```

SQL Preview

Query Results

Run a query to see results.

GPT-4o

Expand chat

Minimize chat

4. Certainly! Let's assume we're documenting a fictional "Models" table typically used in an AI chat system or platform. Below is an overview of the model's table, detailing each column and its purpose or data type.

5. Models Table Overview

Column Name	Data Type	Description
	Integer (or UUID)	A unique identifier for each model. This is the primary key used to distinguish between different models in the table.

String The name of the model, which may include versioning information (e.g., "GPT-3.5-turbo"). This helps in identifying and referencing the model easily.

String The specific version of the model, indicating iteration or updates. Helps in tracking improvements or changes over time.

DateTime The timestamp denoting when the model entry was created in the database. Useful for historical tracking and audit purposes.

DateTime The timestamp indicating the last time the model entry was updated, allowing for tracking modifications and changes over time.

String

Represents the current status of the model, which might include values like "active," "deprecated," or "experimental." This helps in managing lifecycle and usability of the models.

Integer

Maximum number of input tokens that the model can process efficiently. Critical for understanding the model's capabilities and constraints in terms of input text length.

Integer

Maximum number of output tokens that the model can generate in response. Important for setting expectations around response length and generation capacity.

Text

A brief explanation or summary of the model's functionality or purpose. Provides users with insights into what the model can achieve.

String	URL or endpoint where the model can be accessed via API calls. Essential for developers interfacing with the model programmatically.
--------	--

JSON/Object	A data field storing various performance metrics such as accuracy, latency, etc., in a structured format. Useful for evaluating the model's effectiveness and efficiency.
-------------	---

Text or Reference	Details about the dataset used to train the model, which could include a summary or a reference to another table or data source. Important for transparency and reproducibility in model development.
-------------------	---

6. Notes

7. This table structure provides a comprehensive understanding of each model's metadata necessary for effective management and usage in an AI-driven chat system.
8. Copy

12. Sure! With the table structure and column information you've provided, I can suggest several SQL prompts or challenges that could help you practice or solve specific tasks. Below are some prompts based on the `models` table:

13. **SQL Challenge Prompts**

```
SELECT model, year FROM models;
```

```
SELECT * FROM models WHERE year = 2020;
```

```
SELECT * FROM models ORDER BY best_mpge DESC;
```

```
SELECT * FROM models WHERE manufacturer_id = <specific_id>;
```

```
SELECT AVG(best_mpge) AS average_mpge FROM models;
```

```
SELECT m.model, f.fuel_type  
FROM models AS m  
JOIN fuels AS f ON m.fuel_id = f.fuel_id;
```

```
SELECT year, AVG(best_mpge) AS average_mpge  
FROM models  
GROUP BY year;
```

```
SELECT manufacturer_id, COUNT(*) AS model_count
```

```
FROM models
GROUP BY manufacturer_id;
```

```
SELECT * FROM models
ORDER BY best_mpg DESC
LIMIT 1;
```

14. SELECT DISTINCT model FROM models;
- 15.
16. These prompts can help explore different functionalities and operations you can perform on the `models` table using SQL. Each challenge focuses on a different aspect of SQL, from basic selection and filtering to more complex operations like aggregation and joining tables. Feel free to ask for more specific challenges or for further clarifications on these!
- 17.
18. Copy code
19. Copy

You can send 20 messages per hour
Powered by OpenAI

[Back](#)

Learn AI for SQL: Providing context

Providing context

Alright, now that we have a bit of practice conversing with AI, let's dig into how to craft the best individual

Preview: Docs Loading link description

[prompts](#)

.

AI systems require **context** to produce relevant results.

For example, we sent the prompt `write a select query` to both ChatGPT and Gemini. Here are the responses that we got:

ChatGPT: Of course! Could you please specify the

Preview: Docs Loading link description

[database](#)

schema and the table you'd like to query, as well as any specific conditions or criteria you want to include in the query? This information will help me provide you with an accurate select query.

Gemini: `SELECT column1, column2 FROM table_name;`

These aren't totally unhelpful responses! ChatGPT asked for more details, and Gemini provided a template for a query that we could fill out with our specific database information.

But to get the most helpful responses, we want to provide the kind of context ChatGPT asked for. In this exercise, we'll stick to working with a single table.

For an ideal prompt, we'll want to include:

The name and topic of the table

The relevant column names

The specific output we want

For example, suppose we're analyzing the alternate-fuel vehicle industry. Our database has a `models` table with data on each alternate fuel model (such as a Tesla).

The `models` table includes a column labeled `best_mpge`, which records the mile-per-gallon equivalent for each vehicle model under optimal conditions. This is a value that lets us compare traditional gasoline vehicles to newer vehicles like electric cars. We've been asked to determine the 5 most fuel-efficient vehicles.

Here's a sample prompt:

The database table "models" has data regarding alternative fuel vehicles. I need a SQL query to extract the top 5 models by column "best_mpge".

Notice how we've communicated

The name of the table

The name of the relevant column

The output we want

The output we received from ChatGPT is exactly what we wanted:

```
SELECT * FROM models
ORDER BY best_mpg DESC
LIMIT 5;
```

Copy to Clipboard

Instructions

Checkpoint 1 Passed

1.

Run the SQL query given below to get familiar with all the columns present in the `models` table:

```
Select * from models LIMIT 5;
```

Copy to Clipboard

Stuck? Get a hint

Checkpoint 2 Passed

2.

Alright, buckle up, learners! ☺ Let's explore the `models` table together!

Our goal is to identify the 10 oldest models in our dataset. Write an AI prompt to produce a query. Once you receive a query, run it in the code editor.

Challenge mode: phrase your prompt carefully so that the query you get back needs as little editing as possible!

What was the oldest model?

Stuck? Get a hint

Checkpoint 3 Passed

3.

Has the average fuel efficiency improved over the years? Ask the AI for a query that calculates the average miles-per-gallon equivalent for each year in the `models` table.

Learn AI for SQL Multiple tables

Often, SQL databases have multiple related tables. When working with AI, you'll need to communicate:

What tables are in the

Preview: Docs Loading link description

[database](#)

The columns that connect the tables

For example, suppose we want calculate the average MPG-equivalent for each manufacturer.

We might start the AI conversation with

```
I'm analyzing a database. I'll ask you for SQL queries. Got it?
```

Copy to Clipboard

Then, we could follow up with our specific ask:

```
Here are two tables with relevant columns:
```

```
`models` has columns `manufacturer_id` and `best_mpg`
```

```
`manufacturers` has columns `manufacturer_id` and `manufacturer`
```

```
create a SQL query that calculates the average `best_mpg` by  
`manufacturer`
```

Copy to Clipboard

The resulting query we got:

```
SELECT manufacturers.manufacturer, AVG(models.best_mpg) AS
average_best_mpg

FROM models

JOIN manufacturers ON models.manufacturer_id =
manufacturers.manufacturer_id

GROUP BY manufacturers.manufacturer;
```

Copy to Clipboard

Pretty slick!

A couple important points:

In this case, we only provided the relevant columns to save time. If we're planning on an extended analysis with multiple queries, we might lead with the full table schemas (lists of column names).

The

Preview: Docs Loading link description

[LLM](#)

was able to pick up on `manufacturer_id` as the join column because of the way it was named! This saved us from having to specify. However, we may run into cases where we'll have to be more explicit.

The LLM assumed we wanted an `INNER` join. It is always worth noting the assumptions the LLM makes, because there may be cases where we want a different kind of join.

Note that LLMs will often alias tables (giving them shorter names). This is fine, but if we find it hard to track which table is named what, we can always iterate with a prompt like "please rewrite without table aliases."

Instructions

Checkpoint 1 Enabled

1.

The `fuels` table contains data on which type of fuel each vehicle uses.

Run the query

```
SELECT * FROM fuels;
```

Copy to Clipboard

to learn how this table is structured, and what kinds of fuels are included in the database.

```
/* Checkpoint 1 */
SELECT * FROM fuels;

/* Checkpoint 2 */
SELECT models.model, models.year, fuels.fuel
FROM models
JOIN fuels ON models.fuel_id = fuels.fuel_id
WHERE fuels.fuel = 'electric'
ORDER BY models.year
LIMIT 10;

/*Checkpoint 3 */
SELECT manufacturers.manufacturer, COUNT(models.model) AS
num_hybrid_vehicles
FROM models
JOIN manufacturers ON models.manufacturer_id =
manufacturers.manufacturer_id
JOIN fuels ON models.fuel_id = fuels.fuel_id
WHERE fuels.fuel = 'hybrid electric'
GROUP BY manufacturers.manufacturer
```

```
ORDER BY num_hybrid_vehicles DESC  
  
LIMIT 1;
```

Learn AI for SQL

Specify Output Structure

Sometimes it can feel like writing a prompt is basically just writing SQL

Preview: Docs Loading link description
[pseudocode](#)

.

For example, here's a prompt to count the number of models for each manufacturer and fuel type:

```
Write a query that joins the `manufacturers` and `models` tables to the  
`fuels` table using the `manufacturer_id` and `fuel_id` as the join columns.  
Then, group by `manufacturer` and `fuel`, and count the number of models.'
```

Copy to Clipboard

At this point, we're practically writing the SQL, just in slightly more natural language. Another option is to provide the structure of the query *output*, and let the AI fill in the details of what we want. For example, here's a prompt with the same goal, but without specifying as many details:

```
I have three database tables: models, manufacturers, and fuels. they join  
using the columns fuel_id and manufacturer_id
```

```
I want a SQL query that produces a table structured as follows:
```

```
| manufacturers.manufacturer | fuels.fuel | number_of_models |
```

Copy to Clipboard

A few important points:

- We described the three tables and how they join

- We used `table.column` notation to communicate which column belongs to which table

- We used a descriptive name for the third column (`number_of_models`) in the hope that the AI would calculate this column appropriately

Here's the output we got from this prompt:

```
SELECT m.manufacturer AS manufacturer,
       f.fuel AS fuel,
       COUNT(*) AS number_of_models
FROM manufacturers m
JOIN models mo ON m.manufacturer_id = mo.manufacturer_id
JOIN fuels f ON mo.fuel_id = f.fuel_id
GROUP BY m.manufacturer, f.fuel;
```

Copy to Clipboard

Pretty good, right? Recall that if we're getting confused about which table `m` is, we can always iterate and ask "please rewrite without table aliases."

Instructions

Checkpoint 1 Passed

1.

Suppose we have been asked to analyze how efficient (in terms of `best_mpg`) each manufacturer is at using each type of fuel.

Design a target table structure to answer this question, and prompt the AI to produce SQL.

Try running the SQL – did your prompt communicate all the necessary information?

Hint

One potential table structure has the columns `manufacturer`, `fuel`, and `average_best_mpg`

A sample prompt for inspiration

Prompt: Write a SQL query that transforms the `manufacturer` and `fuel` tables into the table below:

`manufacturers.manufacturer | fuels.fuel | average(models.best_mpg)`
using `fuel_id` as the join column.

```
SELECT m.manufacturer, f.fuel, AVG(mod.best_mpg) AS average_best_mpg
FROM manufacturers m
JOIN models mod ON m.manufacturer_id = mod.manufacturer_id
JOIN fuels f ON mod.fuel_id = f.fuel_id
GROUP BY m.manufacturer, f.fuel;
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Imagine we've been asked to calculate which manufacturers make the most electric vehicles, the most hybrid vehicles, and so on.

Design a table to capture this information using the data from the `models`, `manufacturers`, and `fuels` tables? Write a prompt communicating this information to the AI, and run the returned query.

Hint

One possible output structure is `manufacturer, fuel, count_of_model`

A sample prompt for inspiration

Prompt: Using the same tables, write a SQL query to produce a table with the following structure:

manufacturer	fuel	count_of_models
--------------	------	-----------------

```
SELECT m.manufacturer, f.fuel, COUNT(*) AS count_of_models
FROM manufacturers m
JOIN models mod ON m.manufacturer_id = mod.manufacturer_id
JOIN fuels f ON mod.fuel_id = f.fuel_id
GROUP BY m.manufacturer, f.fuel;
```

Copy to Clipboard

Community Support

Still have questions? Get help from the [Codecademy community](#).

Code Editor

Open file navigation

test.sqlite

Close test.sqlite

Enter Fullscreen Code Editor

```
/* Checkpoint 1 */
```

```
SELECT m.manufacturer, f.fuel, AVG(mod.best_mpge) AS
average_best_mpge
FROM manufacturers m
```

```
JOIN models mod ON m.manufacturer_id = mod.manufacturer_id

JOIN fuels f ON mod.fuel_id = f.fuel_id

GROUP BY m.manufacturer, f.fuel;

/* Checkpoint 2 */

SELECT m.manufacturer, f.fuel, COUNT(*) AS count_of_models

FROM manufacturers m

JOIN models mod ON m.manufacturer_id = mod.manufacturer_id

JOIN fuels f ON mod.fuel_id = f.fuel_id

GROUP BY m.manufacturer, f.fuel;
```

Run

Loading Complete
[View solution](#)

SQL Preview

Query Results
Database Schema
fuels

name	type
fuel_id	INTEGER
fuel	TEXT

Rows: 9

manufacturers	
name	type
manufacturer_id	INTEGER
manufacturer	TEXT

Rows: 54

models	
name	type
model_id	INTEGER
model	TEXT
year	INTEGER
manufacturer_id	INTEGER
fuel_id	INTEGER
best_mpg	REAL

Rows: 2520

GPT-4o

Expand chat
Minimize chat

4. Certainly! To achieve the desired result, you can use SQL joins and aggregation functions. Assuming that the tables are named `manufacturers`, `models`, and `fuels`, and they have the appropriate columns as described, here's an example query that performs the task:
5. `SELECT`
6. `manufacturers.manufacturer,`
7. `fuels.fuel,`
8. `COUNT(models.model_id) AS model_count`
9. `FROM`
10. `manufacturers`
11. `JOIN`

```
12.      models ON manufacturers.manufacturer_id = models.manufacturer_id
      JOIN
      fuels ON models.fuel_id = fuels.fuel_id
      GROUP BY
      manufacturers.manufacturer,
      fuels.fuel
      ORDER BY
      manufacturers.manufacturer,
      fuels.fuel;
```

13. Copy code

14. **Explanation:**

15. Copy

You can send 20 messages per hour

Powered by OpenAI

Back

Learn AI for SQL: Multiple tables

Other uses of AI in SQL

Up until now, our journey has revolved around creating

Preview: Docs Loading link description

[prompts](#)

for SQL queries, exploring problem-solving tasks. AI can also be used to help with

- brainstorming analytics ideas
- explaining how an existing SQL query works in plain language
- debugging

Brainstorming

We recommend telling the AI as much as about the dataset as possible, so that the ideas it generates are relevant. For example, instead of

```
Brainstorm ideas for analyzing the alternate fuel vehicle industry
```

Copy to Clipboard

It would be more effective to say something like:

```
I have a database on alternate fuel vehicles including information on: year,
make and model, what type of fuel it uses, miles per gallon equivalent.
Brainstorm 3-5 actionable ideas for analyzing this data.
```

Copy to Clipboard

Explaining Queries

We can also use the AI to break down difficult queries into generic language. The prompt can look like:

```
Here's a SQL query, break it down and help me understand how this SQL query
works?
```

```
"
```

```
SELECT
```

```
    m.manufacturer AS Manufacturer,
```

```
    f.fuel AS Fuel_Type,
```

```
    AVG(md.best_mpge) AS Average_MPG, 
```

```
    MAX(md.best_mpge) AS Highest_MPG, 
```

```
    MIN(md.best_mpge) AS Lowest_MPG
```

```
FROM
```

```
    models md
```

```
JOIN
```

```
    manufacturers m ON md.manufacturer_id = m.manufacturer_id
```

```
JOIN
```

```
    fuels f ON md.fuel_id = f.fuel_id
```

```
GROUP BY
```

```
    m.manufacturer, f.fuel;
```

```
"
```


Copy to Clipboard

Instructions

Checkpoint 1 Enabled

1.

Run our brainstorming prompt, and see what the AI returns. Are all the ideas actionable? Are there any other pieces of information you would add to improve on the brainstorming output?

Pick one of the ideas, and try to generate a query to answer it. Run the query in the code editor.

Hint

Ideas on ways to improve the brainstorming output:

add more information on the columns in each table

add more specifications on the kind of output we want

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Run the following query. What do you think it is designed to do?

Ask the AI to break it down – was the AI answer helpful?

```
SELECT
    manufacturers.manufacturer,
    fuels.fuel,
    models.year,
    COUNT(*) AS model_count
FROM
    models
JOIN
    manufacturers ON models.manufacturer_id =
manufacturers.manufacturer_id
JOIN
    fuels ON models.fuel_id = fuels.fuel_id
GROUP BY
    manufacturers.manufacturer, fuels.fuel, models.year
ORDER BY
    manufacturers.manufacturer, fuels.fuel, models.year;
```

Learn AI for SQL - AI Risks

Using GPT for analytics can pose the risk of serious pitfalls such as:

Hallucinations

Confirmation bias

Plagiarism

Data privacy and security

Hallucinations: AI models use probability to respond to

Preview: Docs Loading link description

[prompts](#)

. They don't actually know or understand what they are saying, and so it is common for them to "hallucinate" or generate false information. For example, GPT could make up a SQL command that doesn't exist or confidently suggest a analysis strategy that is invalid for the task at hand.

Confirmation Bias: As human beings, we are psychologically prone to believing new things that confirm our existing beliefs. If we run code provided by an AI and it produces a result that "makes sense", we are less likely to catch errors in the AI code. We need to be very stringent with ourselves when using AI generated code, double-checking everything the AI has done for validity. Note that we can't use AI to do these checks, as that is using the same flawed system to check itself.

Plagiarism: AI systems may unintentionally produce text or code that closely resembles existing work from published papers or articles without adding proper citations. This is especially likely for extremely detailed prompts.

Data Privacy and Security: The data we feed into an AI model may be used to train future iterations of the model, or stored in some other way, which can result in data leaks where our information becomes part of the publicly available large language model. This poses privacy and security risks if the data is sensitive or confidential.

Mitigating these common pitfalls for analytics requires careful practice and strategic thinking. Here are some tips on how to avoid them:

- Use human experts to engineer prompts and audit responses for potential errors and biases

- Make a backup of important data before running AI-generated update queries

- Ensure that private and sensitive information is anonymized or excluded from the data used by the model

- Make sure to follow any relevant corporate rules or policies for sharing code and data with third parties

Instructions

1. We ran the following prompt

2. write a sqlite query to change the data type of a column

3.

4. Copy to Clipboard

5. and got the following response (note that if you run this query, you might get a totally different response, since GPT is probabilistic):

6. *To change the data type of a column in SQLite, you can use the ALTER TABLE statement along with the ALTER COLUMN clause. Here is an example query to change the data type of a column named column_name in a table named table_name to new_data_type:*

7. `ALTER TABLE table_name`

8. `MODIFY COLUMN column_name new_data_type;`

9.

10. Copy to Clipboard

11. *Make sure to replace table_name, column_name, and new_data_type with your actual table name, column name, and the new data type you want to change to.*

12. Running this code won't actually work: SQLite doesn't have the ability update a column's datatype using `ALTER TABLE`. In fact, [SQLite's documentation](#) says

13. *The only schema altering commands directly supported by SQLite are the "rename table", "rename column", "add column", "drop column" commands*

14. Which AI pitfall is being exposed here?

15. **Our answer**

16. This is an example of a hallucination! GPT is using information in its training dataset to produce a query that, while it exists in other database systems, does not exist in SQLite.

17. Use the prompt component in this exercise to try to refine our prompt and get a more accurate response from GPT!

Analyze Data with SQL

Analyze AI Systems with SQL

For this project, you are an analyst working in the AI industry. Using a database based on the [Epochai](#) dataset, you will be investigating the rise of Generative AI systems, the major players in the space, and the systems themselves.

Tasks

9/9 complete

Mark the tasks as complete by checking them off

Setup

1.

Run the command `.tables` to become familiar with the data contained in the database.

Hint

In the place of `table_name` insert the table names and run the query to see the columns present in the tables.

Tables and what they are meant to showcase:

problems: problem_id, domain, task

Contains types of AI tasks like “text generation” associated to domains like “Language”

orgs: org_id, org_name, org_type

Contains organizations working in AI and whether they are academic or industry

systems: publication_date, system, access, parameters, org_id, problem_id

Contains data on the actual AI systems (like GPT4)

2.

Write a prompt explaining the overall goal of the project.

Stuck? Get a hint

3.

Write a prompt explaining the overall database structure to the AI, including any details that apply broadly.

Hint

explain what each table is, and its columns

include any helpful context if there are columns whose names don't describe what they contain

Sample prompts for inspiration

I'm working with a database that contains three tables: `problems`, `orgs`, and `systems`.

These tables are linked using the `table_id` column notation.

Table descriptions:

`problems`: `problem_id` | `domain` | `task`

Types of AI tasks like "text generation" associated with domains like "Language"

`orgs`: `org_id` | `org_name` | `org_type` Organizations working in AI whether they are academic or industry

`systems`: `publication_date` | `system` | `access` | `parameters` | `org_id` | `problem_id` The actual AI systems (like GPT4)

4.

Ask the AI to brainstorm some analytical ideas that can be answered using the database for you to work on. Note that it may have already provided some ideas in the response to Task 3!

Feel free to pursue some of the ideas you get! But in the next couple task groups, we'll guide you through some of the ideas we got.

Hint

Ask for suggestions based on the database structure that you gave in the 3rd task.

Sample prompts for inspiration

Can you suggest to me some analytical ideas that can be answered using the above database?

Finding the influential players

5.

When we asked for ideas while developing this project, one of the ideas suggested to us was to identify the most influential players in the AI space. One way to do this would be to count the number of AI Systems created by each organization. Create a prompt asking the AI to produce SQL to answer this question.

Run the code – how did the AI do?

Hint

Try giving the AI the output structure that you want in the new table.

Sample prompts for inspiration

Prompt: I want to find the most influential players in the AI field. We have to find the organizations that have developed the most amount of AI systems. I want the outcome table to look like:

organization_name | number_of_ai_systems_developed

Response:

```
SELECT o.org_name AS organization_name, COUNT(s.system) AS
number_of_ai_systems_developed
FROM orgs o
JOIN systems s ON o.org_id = s.org_id
GROUP BY o.org_name
ORDER BY number_of_ai_systems_developed DESC;
```

Copy to Clipboard

6.

Time to iterate! Are there any refinements to the query you'd like to explore? Here are some ideas we can implement in the current query to make it more precise:

Filtering by AI system: specify the type of AI, such as searching solely for "Image generation" AI.

Sorting based on organization type: introduce a new column indicating the type of organization.

Write a prompt that asks the AI to iterate on the last query in one of these ways.

Hint

Write a prompt that instructs the AI what things it needs to modify in the previous prompt properly.

Sample prompts for inspiration

Prompt: Modify the last prompt to make it return to the organizations that are leading in the field of

Image generation and create a new column that shows org_type.

```
SELECT o.org_name AS organization_name, COUNT(s.system) AS
number_of_ai_systems_developed, o.org_type
FROM orgs o
JOIN systems s ON o.org_id = s.org_id
JOIN problems p ON s.problem_id = p.problem_id
WHERE p.task = 'Image generation'
GROUP BY o.org_name, o.org_type
```

```
ORDER BY number_of_ai_systems_developed DESC;
```

Copy to Clipboard

Analyzing the development over time

7.

Let's investigate the growth of AI development over time.

Write a prompt asking the AI for a SQL query that

- counts the number of AI systems in each year

- reports the maximum number of parameters

If the code you receive includes the `YEAR` keyword, as it typically does, no output will appear when you run the code. No worries, we'll debug in the next task!

Hint

Give information about the table structure you want to see in the output that will help AI build the context. (Columns can be `publication_year` and `number_of_system`).

Sample prompts for inspiration

Prompt: Using the database structure above, help me analyze the trends in AI development over time.

The output table should look like this: `publication_year` | `number_of_ai_systems` | `largest_parameter`

Response query:

```
SELECT YEAR(s.publication_date) AS publication_year,
       COUNT(s.system) AS number_of_ai_systems,
       MAX(s.parameters) AS largest_parameter
FROM systems s
GROUP BY YEAR(s.publication_date)
ORDER BY publication_year;
```

Copy to Clipboard

8.

The query we received for the last task used the `YEAR` function to extract the year from the publication date:

```
SELECT YEAR(s.publication_date) AS publication_year,
       COUNT(s.system) AS number_of_ai_systems,
       MAX(s.parameters) AS largest_parameter
```

```
FROM systems s
GROUP BY YEAR(s.publication_date)
ORDER BY publication_year;
```

Copy to Clipboard

If we run this query, we'll get no output.

Write a prompt explaining to the AI that we're using SQLite and not receiving output with `YEAR`. Ask for a revised query.

Hint

Give the error message in the prompt which will help the AI to debug the query better

Explain that you are using the SQLite database which doesn't support the `YEAR` function.

Sample prompts for inspiration

I'm using SQLite and not getting any output from that query, could you refine?

Response query:

```
SELECT strftime('%Y', s.publication_date) AS publication_year,
       COUNT(s.system) AS number_of_ai_systems,
       MAX(s.parameters) AS largest_parameter
FROM systems s
GROUP BY publication_year
ORDER BY publication_year;
```

Copy to Clipboard

9.

Lastly, let's find the top 5 AI problems organizations focus on. The resulting table should display the AI problems and the number of organizations working on each problem.

Note that there may be some null values! This is due to missing data in the underlying dataset.

Hint

Try giving the AI the output table structure along with a limit of 5.

Sample prompts for inspiration

Find the top 5 AI problems that most organizations are working on. The table should look like:

```
ai_problems | number_of_organization
```

Reponse:


```
SELECT p.task AS ai_problems, COUNT(DISTINCT s.org_id) AS
number_of_organizations
FROM problems p
JOIN systems s ON p.problem_id = s.problem_id
GROUP BY ai_problems
ORDER BY number_of_organizations DESC
LIMIT 5;
```

Copy to Clipboard

Code Review

Get targeted feedback about your code and how well it meets the objective of the project. You can request 1 AI-powered review per project when you complete all tasks. **Request code review**

Code Editor

Open file navigation

script.sqlite

Close script.sqlite

Enter Fullscreen Code Editor

1

Save

SQL Preview

Query Results

Run a query to see results.

Calculating Churn

What is Churn?

A common revenue model for [SaaS](#) (Software as a service) companies is to charge a monthly subscription fee for access to their product. Frequently, these companies aim to continually increase the number of users paying for their product. One metric that is helpful for this goal is churn rate.

Churn rate is the percent of subscribers that have canceled within a certain period, usually a month. For a user base to grow, the churn rate must be less than the new subscriber rate for the same period.

To calculate the churn rate, we only will be considering users who are subscribed at the beginning of the month. The churn rate is the number of these users who cancel during the month divided by the total number:

$$\frac{\text{cancellations}}{\text{total subscribers}}$$

For example, suppose you were analyzing data for a monthly video streaming service called CodeFlix. At the beginning of February, CodeFlix has 1,000 customers. In February, 250 of these customers cancel. The churn rate for February would be:

$$250 / 1000 = 25\% \text{ churn rate}$$

I got a SQL table called subscriptions with following columns: id, subscription_start, subscription_end, segment. So, I would you to help solve the following problem:

Determine the range of months of data provided. Which months will you be able to calculate churn for?

Hint

Use `MIN` and `MAX` to examine the range of `subscription_start`.

Single Month I

Now that we've gone over what churn is, let's see how we can calculate it using SQL. In this example, we'll calculate churn for the month of December 2016.

Typically, there will be data in a `subscriptions` table available in the following format:

`id` - the customer id
`subscription_start` - the subscribe date
`subscription_end` - the cancel date

When customers have a `NULL` value for their `subscription_end`, that's a good thing. It means they haven't canceled!

Remember from the previous exercise that churn rate is:

$$\frac{\text{cancellations}}{\text{total subscribers}}$$

For the numerator, we only want the portion of the customers who cancelled during December:

```
SELECT COUNT(*)
FROM subscriptions
WHERE subscription_start < '2016-12-01'
AND (
    subscription_end
    BETWEEN '2016-12-01' AND '2016-12-31'
);
```

Copy to Clipboard

For the denominator, we only want to be considering customers who were active at the beginning of December:

```
SELECT COUNT(*)  
  
FROM subscriptions  
  
WHERE subscription_start < '2016-12-01'  
  
    AND (  
  
        (subscription_end >= '2016-12-01')  
  
        OR (subscription_end IS NULL)  
  
    );
```

Copy to Clipboard

You might've noticed there are quite a few parentheses in these two queries.

When there are multiple conditions in a

Preview: Docs Loading link description

WHERE

clause using

Preview: Docs Loading link description

AND

and

Preview: Docs Loading link description

OR

, it's the best practice to always use the parentheses to enforce the order of execution. It reduces confusion and will make the code easier to understand. The condition within the brackets/parenthesis will always be executed first.

Anyways, now that we have the users who canceled during December, and total subscribers, let's divide the two to get the churn rate.

When dividing, we need to be sure to multiply by `1.0` to cast the result as a float:

```

SELECT 1.0 *
(
    SELECT COUNT(*)
    FROM subscriptions
    WHERE subscription_start < '2016-12-01'
    AND (
        subscription_end
        BETWEEN '2016-12-01'
        AND '2016-12-31'
    )
) / (
    SELECT COUNT(*)
    FROM subscriptions
    WHERE subscription_start < '2016-12-01'
    AND (
        (subscription_end >= '2016-12-01')
        OR (subscription_end IS NULL)
    )
)
AS result;

```

Copy to Clipboard

Here, we have the numerator divided by the denominator, and then multiplying the answer by 1.0. At the very end, we are renaming the final answer to `result` using

Preview: Docs Loading link description

Exercise

1.

We've imported 4 months of data for a company from when they began selling subscriptions. This company has a minimum commitment of 1 month, so there are no cancellations in the first month.

The `subscriptions` table contains:

```
id
subscription_start
subscription_end
```

Use the methodology provided in the narrative to calculate the churn for January 2017.

Hint

The table names should be the same, but the month and year dates need to be adjusted:

Every `2016` should be `2017`
Every `12` should be `01`

The result is roughly 12.6%

Here, we are using `*` as an arithmetic operator.

For example:

```
SELECT 2.0 * 3;
```

Copy to Clipboard
 $2.0 \times 3 = 6.0$

Here is the

```
SELECT 1.0 *
(
  -- customers who canceled
  -- during January 2017
```

```
) / (  
  
    -- customers who were active  
  
    -- at the beginning o
```

Solution

```
SELECT 100.0 *  
  
(  
  
    SELECT COUNT(*)  
  
    FROM subscriptions  
  
    WHERE subscription_start < '2017-01-01'  
  
    AND (  
  
        subscription_end  
  
        BETWEEN '2017-01-01'  
  
        AND '2017-01-31'  
  
    )  
) / (  
  
    SELECT COUNT(*)  
  
    FROM subscriptions  
  
    WHERE subscription_start < '2017-01-01'  
  
    AND (  
  
        (subscription_end >= '2017-01-01')  
  
        OR (subscription_end IS NULL)  
  
    )  
)  
  
AS result;
```

Calculating Churn

Single Month II

The previous method worked, but you may have noticed we selected the same group of customers twice for the same month and repeated a number of conditional statements.

Companies typically look at churn data over a period of many months. We need to modify the calculation a bit to make it easier to mold into a multi-month result. This is done by making use of

Preview: Docs Loading link description

WITH

and

Preview: Docs Loading link description

CASE

.

To start, use WITH to create the group of customers that are active going into December:

```
WITH enrollments AS

(SELECT *

FROM subscriptions

WHERE subscription_start < '2016-12-01'

AND (

    (subscription_end >= '2016-12-01')

    OR (subscription_end IS NULL)

)),
```


Copy to Clipboard

Let's create another temporary table that contains an `is_canceled` status for each of these customers. This will be 1 if they cancel in December and 0 otherwise (their cancellation date is after December or `NULL`).

```
status AS

(SELECT

CASE

    WHEN (subscription_end > '2016-12-31')

        OR (subscription_end IS NULL) THEN 0

        ELSE 1

    END as is_canceled,

...


```

Copy to Clipboard

We could just

Preview: Docs Loading link description

`COUNT()`

the rows to determine the number of users. However, to support the multiple month calculation, let's add a `is_active` column to the `status` temporary table. This uses the same condition we created `enrollments` with:

```
status AS

...

CASE


```

```
WHEN subscription_start < '2016-12-01'

    AND (

        (subscription_end >= '2016-12-01')

        OR (subscription_end IS NULL)

    ) THEN 1

ELSE 0

END as is_active

FROM enrollments

)
```

Copy to Clipboard

This tells us if someone is active at the beginning of the month.

The last step is to do the math on the `status` table to calculate the month's churn:

```
SELECT 1.0 * SUM(is_canceled) / SUM(is_active)

FROM status;
```

Copy to Clipboard

We make sure to multiply by `1.0` to force a float result instead of an integer.

Instructions

Checkpoint 1 Enabled

1.

Use the methodology provided in the narrative to calculate the churn for January 2017.

The `subscriptions` table contains:

```
id
subscription_start
subscription_end
```

Hint

The table names are the same, but the month and year dates will need to be adjusted.

```
WITH enrollments AS
(
  ...
),
status AS
(
  ...
)
SELECT 1.0 * SUM(is_canceled) / SUM(is_active)
FROM status;
```

Copy to Clipboard

`WITH` is capable of creating multiple temporary tables by separating query names with a comma.

Here, we are creating the first temporary table and give it the name `enrollments`, and then creating the second temporary table and give it the name `status`.

Exercise

```
WITH enrollments AS

(SELECT *
FROM subscriptions
```

```

WHERE subscription_start < '2017-01-01'
AND (
    (subscription_end >= '2017-01-01')
    OR (subscription_end IS NULL)
)),
status AS
(SELECT
CASE
    WHEN (subscription_end > '2017-01-31')
        OR (subscription_end IS NULL) THEN 0
    ELSE 1
    END as is_canceled,
CASE
    WHEN (subscription_start < '2017-01-01')
        AND (
            (subscription_end >= '2017-01-01')
            OR (subscription_end IS NULL)
        ) THEN 1
    ELSE 0
    END as is_active
FROM enrollments
)
SELECT 1.0 * SUM(is_canceled)/SUM(is_active) FROM status;

```

Multiple Month: Create Months Temporary Table

Our single month calculation is now in a form that we can extend to a multiple month result. But first, we need months!

Some SQL table schemes will contain a prebuilt table of months. Ours doesn't, so we'll need to build it using

Preview: Docs Loading link description

UNION

. We'll need the first and last day of each month.

Our churn calculation uses the first day as a cutoff for subscribers and the last day as a cutoff for cancellations.

This table can be created like:

```
SELECT

    '2016-12-01' AS first_day,

    '2016-12-31' AS last_day

UNION

SELECT

    '2017-01-01' AS first_day,

    '2017-01-31' AS last_day;
```

Copy to Clipboard

Instructions

Checkpoint 1 Passed

1.

We will be using the months as a temporary table (using `WITH`) in the churn calculation.

Create the `months` temporary table using `WITH` and `SELECT` everything from it so that you can see the structure.

We need a table for January, February, and March of 2017.

Hint

```
WITH months AS (...)  
SELECT *  
FROM months;
```

```
WITH months as  
  
(SELECT  
  
    '2017-01-01' as first_day,  
  
    '2017-01-31' as last_day  
  
UNION  
  
SELECT  
  
    '2017-02-01' as first_day,  
  
    '2017-02-28' as last_day  
  
UNION  
  
SELECT  
  
    '2017-03-01' as first_day,  
  
    '2017-03-31' as last_day  
  
)
```

```
SELECT *  
  
FROM months;
```

Multiple Month: Cross Join Months and Users

Now that we have a table of months, we will join it to the subscriptions table. This will result in a table containing every combination of month and subscription.

Ultimately, this table will be used to determine the status of each subscription in each month.

Instructions

Checkpoint 1 Passed

1.

The workspace contains the `months` temporary table from the previous exercise. Create a `cross_join` temporary table that is a `CROSS JOIN` of `subscriptions` and `months`.

We've added:

```
SELECT *  
  
FROM cross_join  
  
LIMIT 100;
```

Copy to Clipboard

at the bottom of this exercise so you can visualize the temporary table you create.

It should `SELECT` all the columns from the temporary table.

Hint

The pattern for creating a temporary table using a `CROSS JOIN` is:

```
desired_temp_table AS  
(SELECT *  
FROM table1  
CROSS JOIN table2)
```

```
WITH months AS

(
SELECT

    '2017-01-01' as first_day,

    '2017-01-31' as last_day

UNION

SELECT

    '2017-02-01' as first_day,

    '2017-02-28' as last_day

UNION

SELECT

    '2017-03-01' as first_day,

    '2017-03-31' as last_day

),

cross_join AS

(
SELECT *

FROM subscriptions

CROSS JOIN months)

SELECT *

FROM cross_join
```



```
LIMIT 100;
```

Multiple Month: Determine Active Status

We now have a cross joined table that looks something like:

id	subscription_start	subscription_end	month
1	2016-12-03	2017-02-15	2016-12-01
1	2016-12-03	2017-02-15	2017-01-01
1	2016-12-03	2017-02-15	2017-02-01
1	2016-12-03	2017-02-15	2017-03-01

If you remember our single month example, our ultimate calculation will make use of the `status` temporary table. The first column of this table was used in the denominator of our churn calculation:

`is_active`: if the subscription started before the given month and has not been canceled before the start of the given month

For the example above, this column would look like:

month	is_active
2016-12-01	0
2017-01-01	1
2017-02-01	1

Instructions

Checkpoint 1 Passed

1.

Add a `status` temporary table. This table should have the following columns:

`id` - selected from the `cross_join` table

`month` - this is an alias of `first_day` from the `cross_join` table. We're using the first day of the month to represent which month this data is for.

`is_active` - 0 or 1, derive this column using a `CASE WHEN` statement

The `is_active` column should be 1 if the `subscription_start` is before the month's `first_day` and if the `subscription_end` is either after the month's `first_day` or is `NULL`.

We've added:

```
SELECT *  
FROM status  
LIMIT 100;
```

Copy to Clipboard

at the bottom of this exercise so you can visualize the temporary table you create.

Hint

One version of the required conditional is:

```
(subscription_start < first_day)  
AND (  
    subscription_end > first_day  
    OR subscription_end IS NULL  
) THEN 1  
ELSE 0
```

```
WITH months AS  
  
(SELECT  
  
    '2017-01-01' as first_day,  
  
    '2017-01-31' as last_day  
  
UNION  
  
SELECT  
  
    '2017-02-01' as first_day,  
  
    '2017-02-28' as last_day  
  
UNION  
  
SELECT
```

```

    '2017-03-01' as first_day,

    '2017-03-31' as last_day

),

cross_join AS

(SELECT *

FROM subscriptions

CROSS JOIN months),

status AS

(SELECT id, first_day as month,

CASE

    WHEN (subscription_start < first_day)

        AND (

            subscription_end > first_day

            OR subscription_end IS NULL

        ) THEN 1

    ELSE 0

END as is_active

FROM cross_join)

SELECT *

FROM status

LIMIT 100;

```

Multiple Month: Determine Cancellation Status

For our calculation, we'll need one more column on the `status` temporary table: `is_canceled`

This column will be `1` only during the month that the user cancels.

From the last exercise, the sample user had a `subscription_start` on `2016-12-03` and their `subscription_end` was on `2017-02-15`. Their complete status table should look like:

month	is_active	is_canceled
2016-12-01	0	0
2017-01-01	1	0
2017-02-01	1	1
2017-03-01	0	0

In our examples, our company has a minimum subscription duration of one month. This means that the `subscription_start` always falls before the beginning of the month that contains their `subscription_end`. Outside of our examples, this is not always the case, and you may need to account for customers canceling within the same month that they subscribe.

Instructions

Checkpoint 1 Enabled

1.

Add an `is_canceled` column to the `status` temporary table. Ensure that it is equal to `1` in months containing the `subscription_end` and `0` otherwise.

Derive this column using a `CASE WHEN` statement. You can use the `BETWEEN` function to check if a date falls between two others.

We've added:

```
SELECT *  
FROM status  
LIMIT 100;
```

Copy to Clipboard

at the bottom of this exercise so you can visualize the status table.

Hint

```
WHEN date1 BETWEEN date2 AND date3
```

```

WITH months AS

(SELECT

  '2017-01-01' as first_day,

  '2017-01-31' as last_day

UNION

SELECT

  '2017-02-01' as first_day,

  '2017-02-28' as last_day

UNION

SELECT

  '2017-03-01' as first_day,

  '2017-03-31' as last_day

),

cross_join AS

(SELECT *

FROM subscriptions

CROSS JOIN months),

status AS

(SELECT id, first_day as month,

CASE

  WHEN (subscription_start < first_day)

    AND (

      subscription_end > first_day

      OR subscription_end IS NULL

    ) THEN 1

  ELSE 0

END as is_active,

```

```
CASE
  WHEN subscription_end BETWEEN first_day AND last_day THEN 1
  ELSE 0
END as is_canceled
FROM cross_join)

SELECT *
FROM status
LIMIT 100;
```

Multiple Month: Sum Active and Canceled Users

Now that we have an active and canceled status for each subscription for each month, we can aggregate them.

We will `GROUP BY` month and create a

Preview: Docs Loading link description

`SUM()`

of the two columns from the `status` table, `is_active` and `is_canceled`.

This provides a list of months, with their corresponding number of active users at the beginning of the month and the number of those users who cancel during the month.

Instructions

Checkpoint 1 Enabled

1.

Add a `status_aggregate` temporary table. This table should have the following columns:

`month` - selected from the `status` table

active - the SUM() of active users for this month

canceled - the SUM() of canceled users for this month

We've added:

```
SELECT *  
FROM status_aggregate;
```

Copy to Clipboard

at the bottom of this exercise so you can visualize the temporary table you create.

Hint

Be sure to GROUP BY month to get a result for each month.

```
WITH months AS  
  
(SELECT  
  '2017-01-01' as first_day,  
  '2017-01-31' as last_day  
UNION  
SELECT  
  '2017-02-01' as first_day,  
  '2017-02-28' as last_day  
UNION  
SELECT  
  '2017-03-01' as first_day,  
  '2017-03-31' as last_day  
) ,  
cross_join AS  
  
(SELECT *  
FROM subscriptions
```



```

CROSS JOIN months),

status AS

(SELECT id, first_day as month,

CASE

    WHEN (subscription_start < first_day)

        AND (

            subscription_end > first_day

            OR subscription_end IS NULL

        ) THEN 1

    ELSE 0

END as is_active,

CASE

    WHEN subscription_end BETWEEN first_day AND last_day THEN 1

    ELSE 0

END as is_canceled

FROM cross_join),

status_aggregate AS

(SELECT

    month,

    SUM(is_active) as active,

    SUM(is_canceled) as canceled

FROM status

GROUP BY month)

SELECT *

FROM status_aggregate;

```

Multiple Month: Churn Rate Calculation

Now comes the moment we've been waiting for - the actual churn rate.

We use the number of canceled and active subscriptions to calculate churn for each month:

```
churn_rate = canceled / active
```

Instructions

Checkpoint 1 Passed

1.

Add a `SELECT` statement to calculate the churn rate. The result should contain two columns:

`month` - selected from `status_aggregate`

`churn_rate` - calculated from `status_aggregate.canceled` and `status_aggregate.active`.

Hint

Remember to cast the result to a float by multiplying by `1.0`

```

WITH months AS

(SELECT

  '2017-01-01' as first_day,

  '2017-01-31' as last_day

UNION

SELECT

  '2017-02-01' as first_day,

  '2017-02-28' as last_day

UNION

SELECT

  '2017-03-01' as first_day,

  '2017-03-31' as last_day

),

cross_join AS

(SELECT *

FROM subscriptions

CROSS JOIN months),

status AS

(SELECT id, first_day as month,

CASE

  WHEN (subscription_start < first_day)

    AND (

      subscription_end > first_day

      OR subscription_end IS NULL

    ) THEN 1

  ELSE 0

END as is_active,

```

```

CASE
    WHEN subscription_end BETWEEN first_day AND last_day THEN 1
    ELSE 0
END as is_canceled
FROM cross_join),
status_aggregate AS
(SELECT
    month,
    SUM(is_active) as active,
    SUM(is_canceled) as canceled
FROM status
GROUP BY month)
SELECT
    month,
    1.0 * canceled/active AS churn_rate
FROM status_aggregate;

```

Calculating Churn Review

You can now calculate a churn rate over time for a company:

month	churn_rate
2017-01-01	0.127

2017-02-01	0.125
------------	-------

2017-03-01	0.237
------------	-------

It looks like our sample company did something in March that doubled the churn rate. They might want to look into that...

In this lesson you learned:

- The churn rate is a percent of subscribers at the beginning of a period that cancel within that period. "Monthly churn" is a typical metric and what was used in the examples.

- How to calculate this metric using SQL for a single month. This used `COUNT()` and conditions to determine the number of subscribers that were active and how many canceled.

- A more complex method to track the subscriber churn rate over many months.

```

WITH months AS (

    SELECT

        '2017-01-01' AS first_day,

        '2017-01-31' AS last_day

    UNION

    SELECT

        '2017-02-01' AS first_day,

        '2017-02-28' AS last_day

    UNION

    SELECT

        '2017-03-01' AS first_day,

        '2017-03-31' AS last_day

),

cross_join AS (

    SELECT *

    FROM subscriptions

    CROSS JOIN months

),

status AS (

    SELECT

        id,

        first_day AS month,

        CASE

            WHEN (subscription_start < first_day)

                AND (

                    subscription_end > first_day

                    OR subscription_end IS NULL

```

```

        ) THEN 1

    ELSE 0

END AS is_active,

CASE

    WHEN subscription_end BETWEEN first_day AND last_day THEN 1

    ELSE 0

END AS is_canceled

FROM cross_join

),

status_aggregate AS (

    SELECT

        month,

        SUM(is_active) AS active,

        SUM(is_canceled) AS canceled

    FROM status

    GROUP BY month

)

SELECT

    month,

    1.0 * canceled / active AS churn_rate

FROM status_aggregate;

```

The Attribution Query I

We just learned how to attribute a user's first and last touches. What if we want to attribute the first and last touches for ALL users? This is where SQL comes in handy – with one query we can find all first- or last-touch attributions (the first and last versions are nearly identical). We can save this query to run it later, or modify it for a subset of users. Let's learn the query...

In order to get first-touch attributions, we need to find the first time that a user interacted with our website. We do this by using a `GROUP BY`. Let's call this table `first_touch`:

```
SELECT user_id,  
  
       MIN(timestamp) AS 'first_touch_at'  
  
FROM page_visits  
  
GROUP BY user_id;
```

Copy to Clipboard

This tells us the first time that each user visited our site, but does not tell us how they got to our site – the query results have no UTM parameters! We'll see how to get those in the next exercise.

The Attribution Query II

To get the UTM parameters, we'll need to `JOIN` these results back with the original table.

We'll join tables `first_touch`, aka `ft`, and `page_visits`, aka `pv`, on `user_id` and `timestamp`.

```
ft.user_id = pv.user_id  
  
AND ft.first_touch_at = pv.timestamp
```

Copy to Clipboard

Remember that `first_touch_at` is the earliest timestamp for each user. Here's the simplified query:


```

WITH first_touch AS (

    /* ... */

)

SELECT *

FROM first_touch AS 'ft'

JOIN page_visits AS 'pv'

    ON ft.user_id = pv.user_id

    AND ft.first_touch_at = pv.timestamp;

```

Copy to Clipboard

Now fill in the

Preview: Docs Loading link description

WITH

clause using the `first_touch` query from the previous exercise. We'll also specify the columns to

Preview: Docs Loading link description

SELECT

.

```

WITH first_touch AS (

    SELECT user_id,

        MIN(timestamp) AS 'first_touch_at'

    FROM page_visits

    GROUP BY user_id)

```

```
SELECT ft.user_id,  
  
       ft.first_touch_at,  
  
       pv.utm_source  
  
FROM first_touch AS 'ft'  
  
JOIN page_visits AS 'pv'  
  
   ON ft.user_id = pv.user_id  
  
   AND ft.first_touch_at = pv.timestamp;
```

Copy to Clipboard

Instructions

1. The diagram on the right illustrates the `JOIN` we need to get the UTM parameters of each first touch:

On the left is `page_visits` (just three columns from the original table). We get the UTM parameters from there.

On the right is `first_touch` (the result of the `GROUP BY` query). We get the first touches from there.

```
SELECT user_id,
       timestamp,
       utm_source
FROM page_visits;
```

```
SELECT user_id,
       MIN(timestamp) as first_touch_at
FROM page_visits
GROUP BY user_id;
```

user_id	timestamp	utm_source
10048	2018-01-16 04:17:46	medium
10048	2018-01-16 07:28:46	medium
10048	2018-01-19 00:00:46	email
10069	2018-01-02 23:14:01	buzzfeed
10069	2018-01-02 23:55:01	buzzfeed
10069	2018-01-04 08:12:01	facebook
10069	2018-01-04 08:13:01	facebook
10734	2018-01-05 13:17:16	buzzfeed

first_touch_at	user_id
2018-01-16 04:17:46	10048
2018-01-02 23:14:01	10069
2018-01-05 13:17:16	10734

The Attribution Query III

We can easily modify the first-touch attribution query to get last-touch attribution: use

`MAX(timestamp)` instead of `MIN(timestamp)`.

For reference, the first-touch attribution query is shown below.

Now that you've seen how it works, it's time to practice!

Instructions

Checkpoint 1 Passed

1.

Using the query above as a guide, write the LAST-touch attribution query and run it.

Hint

Your query should follow this structure:

```
WITH last_touch AS (  
    SELECT user_id,  
           MAX(timestamp) AS 'last_touch_at'  
    FROM page_visits  
    GROUP BY user_id)  
SELECT lt.user_id,  
       lt.last_touch_at,  
       pv.utm_source  
FROM last_touch AS 'lt'  
JOIN page_visits AS 'pv'  
    ON lt.user_id = pv.user_id  
    AND lt.last_touch_at = pv.timestamp;
```

Copy to Clipboard

You should recognize the query in the `WITH` clause from the previous exercises.

Checkpoint 2 Passed

2.

Make sure June's last touch at `08:13:01` is still there!

Add a `WHERE` clause for `user_id = 10069` to your existing query.

Hint

Your query should follow this structure:

```
WITH last_touch AS (  
    SELECT user_id,  
           MAX(timestamp) AS 'last_touch_at'  
    FROM page_visits  
    GROUP BY user_id)  
SELECT lt.user_id,  
       lt.last_touch_at,  
       pv.utm_source
```

```
FROM last_touch AS 'lt'  
JOIN page_visits AS 'pv'  
    ON lt.user_id = pv.user_id  
    AND lt.last_touch_at = pv.timestamp  
WHERE lt.user_id = 10069;
```

SOLUTION:

```
WITH last_touch AS (  
    SELECT user_id,  
           MAX(timestamp) as last_touch_at  
    FROM page_visits  
    GROUP BY user_id)  
SELECT lt.user_id,  
       lt.last_touch_at,  
       pv.utm_source  
FROM last_touch lt  
JOIN page_visits pv  
    ON lt.user_id = pv.user_id  
    AND lt.last_touch_at = pv.timestamp  
WHERE lt.user_id = 10069;
```

Window Functions

Introduction to Window Functions

<https://www.codecademy.com/paths/analyze-data-with-sql/tracks/advanced-sql/modules/sql-window-functions/lessons/sql-window-functions/exercises/sql-introduction-to-window-functions>

So far we've learned how to retrieve information from tables and perform calculations on them. But we've seen that `GROUP BY` and `SUM` reduce the number of rows in your query results because they are combining or grouping rows.

Window functions, on the other hand, allow you to maintain the values of your original table while displaying grouped or summative information alongside in another column. This is why many Data Scientists and Data Engineers love to use

Preview: Docs Loading link description

[window functions](#)

for complex data analysis.

In this lesson, you will build upon the knowledge you've gained about

Preview: Docs In SQL, aggregate functions perform a calculation on a set of values and return a single value. They are often used with the `GROUP BY` clause of the `SELECT` statement. Note: Except for `COUNT()`, aggregate functions ignore all `NULL` values. List all the years and their number of movies, but only the years with more than 5 movies: `sql SELECT year, COUNT() FROM movies`

[aggregate functions](#)

and other clauses in order to write window functions.

Let's start by looking at data from some of the most followed accounts on Instagram.

Instructions

Checkpoint 1 Passed

1.

Run a query to select all the information in our `social_media` table to see what data we have.
What are the column names?

Hint

```
SELECT *  
FROM social_media;
```

Copy to Clipboard

The table contains changes in followers, following, and posts over an 8 month period.

```
username  
month  
change_in_followers  
change_in_following  
posts
```

Note: The `change_in_followers` column is in millions.

Checkpoint 2 Passed

2.

Delete the query and write a new query to return the `username` column and the `SUM` of the total `change_in_followers` in our `social_media` table, that are from “instagram”.

Don't forget to use `WHERE` and `GROUP BY`.

Hint

```
SELECT  
    username, SUM(change_in_followers)  
FROM  
    social_media  
WHERE  
    username = 'instagram'  
GROUP BY  
    username;
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Copy the following query to see the “running total” of Instagram’s

`change_in_followers` for each month.

A running total is the sum of all the previous rows up to the current one.

```
SELECT
    month,
    change_in_followers,
    SUM(change_in_followers) OVER (
        ORDER BY month
    ) AS 'running_total'
FROM
    social_media
WHERE
    username = 'instagram';
```

Copy to Clipboard

Hint

Copying:

Windows: `ctrl + c`

Mac: `command + c`

Pasting:

Windows: `ctrl + v`

Mac: `command + v`

Checkpoint 4 Passed

4.

How is the `running_total` column different from the `SUM` value you found in the previous query?

Take a look at the hint for the answer.

Click Run again to continue.

Hint

`SUM` is only able to return *one* row of data that represents the sum of all months.

`running_total` is able to show you what the total is after each month.

```
SELECT
    month,
    change_in_followers,
    SUM(change_in_followers) OVER (
        ORDER BY month
    ) running_total
FROM
    social_media
WHERE
    username = 'instagram';
```

Window Function Syntax

In the previous exercise, you ran a window function to see a ‘running total’.

Let’s break down the query into smaller parts:

```
SELECT
    month,
    change_in_followers,
    SUM(change_in_followers) OVER (
        ORDER BY month
    ) AS 'running_total'
```

```
FROM
```

```
    social_media
```

```
WHERE
```

```
    username = 'instagram';
```

Copy to Clipboard

`SELECT month, change_in_followers`: Same as usual, selecting the columns.

`SUM(change_in_followers)`: Here is our aggregate function to find the `SUM` of our chosen column.

`OVER`: This is the clause that designates `SUM` as a window function.

`ORDER BY month`: Here we declare what we would like our window function to do.

This window function is taking the sum of followers for each month.

So for each month, the window function adds the current month's

`change_in_followers` to our running total.

Then name the running total column `'running_total'`.

And lastly, this is all coming from table `social_media` where the username is `instagram`.

Instructions

Checkpoint 1 Passed

1.

In the code editor, modify the example window function:

Use `AVG` instead of `SUM`.

Change the name of the window function column to be `running_avg` instead of `running_total`.

Hint

```
SELECT
    month,
    change_in_followers,
    AVG(change_in_followers) OVER (
```

```
ORDER BY month
) AS 'running_avg'
FROM
    social_media
WHERE
    username = 'instagram';
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Let's now implement a more complex window function that utilizes `SUM`, `COUNT` and `AVG`.

Copy and paste the following query:

```
SELECT
    month,
    change_in_followers,
    SUM(change_in_followers) OVER (
        ORDER BY month
    ) AS 'running_total',
    AVG(change_in_followers) OVER (
        ORDER BY month
    ) AS 'running_avg',
    COUNT(change_in_followers) OVER (
        ORDER BY month
    ) AS 'running_count'
FROM
    social_media
WHERE
    username = 'instagram';
```

Copy to Clipboard

How is this query and its results different from what you're used to seeing with `SUM`, `COUNT`, and `AVG`?

Hint

Here we can see how the sum, average and count of followers is changing each month rather than just the *total* sum, average or count.

```
SELECT
    month,
    change_in_followers,
    SUM(change_in_followers) OVER (
        ORDER BY month
    ) AS 'running_total',
    AVG(change_in_followers) OVER (
        ORDER BY month
    ) AS 'running_avg',
    COUNT(change_in_followers) OVER (
        ORDER BY month
    ) AS 'running_count'
FROM
    social_media
WHERE
    username = 'instagram';
```

PARTITION BY

In our last query, we were able to find the running total, average and count of only Instagram's followers using

Preview: Docs Loading link description

[window functions](#)

. But what about the other accounts in our social media table? What if we wanted to compare all of our users?

This is where we can use another feature of window functions: `PARTITION BY`.

`PARTITION BY` is a subclause of the `OVER` clause and divides a query's result set into parts. It's very similar to `GROUP BY` except it *does not* reduce the number of rows returned.

While using `GROUP BY` only allows one row to be returned for each group, `PARTITION BY` allows you to see all of the resultant rows.

Let's see what this means.

Instructions

Checkpoint 1 Enabled

1.

Run the query below to see the total change in followers for each user.

```
SELECT
    username,
    SUM(change_in_followers) AS 'total_follower_change'
FROM social_media
GROUP BY username;
```

Copy to Clipboard

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

If you modify the query to also return the `month` column, what happens?

(Make sure to add `month` between `username` and `'total_change_in_followers'` and also

add `month` in the `GROUP BY`.)

Are you still seeing the total changes for each account?

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Now run the following query which utilizes `PARTITION BY`. How does this change our results from the `GROUP BY` query we ran?

```
SELECT
    username,
    month,
    change_in_followers,
    SUM(change_in_followers) OVER (
        PARTITION BY username
        ORDER BY month
    ) 'running_total_followers_change'
FROM
    social_media;
```

Copy to Clipboard

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Utilizing `PARTITION BY` allowed us to find the total change in followers for each user up to the current month and display it next to the current month.

Modify the query to have it also return the *average change in followers for each month* and return this as `running_avg_followers_change`.

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

Looking at the results of your query, which user had the largest/smallest increase in new followers? Which user had the steadiest flow of followers?

```
SELECT
    username,
```

```
month,  
change_in_followers,  
SUM(change_in_followers) OVER (  
    PARTITION BY username  
    ORDER BY month  
) 'running_total_followers_change',  
posts,  
AVG(change_in_followers) OVER (  
    PARTITION BY username  
    ORDER BY month  
) 'running_avg_followers_change'  
FROM  
social_media;
```

Window Functions

FIRST_VALUE and LAST_VALUE

In the past, when we wanted to get the first or last value of a query, we might use the

Preview: Docs Loading link description

LIMIT

clause, probably in conjunction with `ORDER BY`, which would return one result showing us the first or last value from our dataset.

With

Preview: Docs A window function performs a calculation over the inputted column and then returns the calculated value for each row. Window functions can be identified by their use of the `OVER` clause. In the simplest syntactic example, a function can be performed over a given column as

shown below: `sql SELECT functionname(column1name) OVER() FROM tablename;` An `ORDER BY` clause can be used to determine in which direction the function should be calculated.

[window functions](#)

, we can return our first or last values alongside our other data by using the `FIRST_VALUE()` or `LAST_VALUE()` functions.

They work exactly as you would imagine:

`FIRST_VALUE()` returns the first value in an ordered set of values.

`LAST_VALUE()` returns the last value in an ordered set of values.

Let's break down a query that is fetching the `FIRST_VALUE()` from our social media dataset:

```
SELECT

    username,

    posts,

    FIRST_VALUE (posts) OVER (

        PARTITION BY username

        ORDER BY posts

    ) fewest_posts

FROM

    social_media;
```

Copy to Clipboard

This query should look familiar overall as it follows the standard window function format, however, we are using `FIRST_VALUE` now for `posts`. This means our window function will pull the first value from the `posts` column.

`OVER (PARTITION BY username ORDER BY posts) fewest_posts`: here we can see that `posts` is going to be pulled based on `username` due to the `PARTITION BY`. We are naming this column `fewest_posts` because of the `ORDER BY` which defaults to ascending order. And all of this is coming from our `social_media` table.

Instructions

Checkpoint 1 Enabled

1.

Copy the following query into the code editor and run it.

```
SELECT username,  
       posts,  
       FIRST_VALUE (posts) OVER (  
         PARTITION BY username  
         ORDER BY posts  
       ) AS 'fewest_posts'  
FROM social_media;
```

Copy to Clipboard

What is the result?

Hint

The result is showing us each of the `usernames` from the table alongside how many posts they posted each month. Our `fewest_posts` column is showing the lowest number of posts that user made for any month.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

If `FIRST_VALUE` is returning the *fewest* posts, then `LAST_VALUE` should return the *most* posts for each user.

Update the query now to use `LAST_VALUE` instead of `FIRST_VALUE`.

Are we seeing the expected result?

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

We saw that `LAST_VALUE` didn't work as we expected. This is because each row in our results set *is* the last row at the time it is outputted.

In order to get `LAST_VALUE` to show us the *most* posts for a user, we need to specify a **frame** for our window function.

```
SELECT
    username,
    posts,
    LAST_VALUE (posts) OVER (
        PARTITION BY username
        ORDER BY posts
        RANGE BETWEEN UNBOUNDED PRECEDING AND
        UNBOUNDED FOLLOWING
    ) most_posts
FROM
    social_media;
```

```
SELECT

    username,

    posts,

    LAST_VALUE (posts) OVER (

        PARTITION BY username

        ORDER BY posts

        RANGE BETWEEN UNBOUNDED PRECEDING AND

        UNBOUNDED FOLLOWING

    ) most_posts
FROM

    social_media;
```

Window Functions

LAG

Now that we understand how to pull the first or last value from our dataset and display it alongside our query results, let's see how to fetch data from other parts of our results.

Window functions can use `LAG` or `LEAD` in order to access information from a row at a specified offset which comes before (`LAG`) or after (`LEAD`) the current row.

This means that by using `LAG` or `LEAD` you can access any row before or after the current row, which can be very useful in calculating the difference between the current and adjacent row. We will first focus on using `LAG`.

Instructions

Checkpoint 1 Enabled

1.

Run a query to select all the information in our new data table `streams`.

What are the column names?

Hint

```
SELECT *  
FROM streams;
```

Copy to Clipboard

```
artist  
week  
streams_millions  
chart_position
```

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Let's look at a window function that uses `LAG`. We will look at the artist's current number of streams and their streams from the previous week.

```

SELECT
    artist,
    week,
    streams_millions,
    LAG(streams_millions, 1, 0) OVER (
        ORDER BY week
    ) previous_week_streams
FROM
    streams
WHERE
    artist = 'Lady Gaga';

```

Copy to Clipboard

`LAG` takes up to *three* arguments:

- column (required)

- offset (optional, default 1 row offset)

- default (optional, what to replace default `null` values with)

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Displaying the previous row value in each row can be useful, but for our purposes let's change our `LAG` function to instead show the *change in streams* for Lady Gaga per week:

```

SELECT
    artist,
    week,
    streams_millions,
    streams_millions - LAG(streams_millions, 1,
streams_millions) OVER (
        ORDER BY week
    ) streams_millions_change
FROM

```

```
streams
WHERE
    artist = 'Lady Gaga';
```

Copy to Clipboard

To do this, we just need to subtract the current week's `streams_millions` by our `LAG` function value (which is returning the previous week's `streams_millions`).

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Build upon our current query to use `PARTITION BY` to show the change in streams for “Lady Gaga” as well as the change in chart position.

Take a look at the hint for the answer.

```
SELECT
    artist,
    week,
    streams_millions,
    streams_millions - LAG(streams_millions, 1, streams_millions) OVER (
        PARTITION BY artist
        ORDER BY week
    ) AS 'streams_millions_change',
    chart_position,
    LAG(chart_position, 1, chart_position) OVER (
        PARTITION BY artist
        ORDER BY week
    ) - chart_position AS 'chart_position_change'
FROM
    streams
WHERE artist = 'Lady Gaga';
```

Window Functions

LEAD

In the previous exercise we used `LAG` to examine how an artist's weekly streams and chart position changed. Your final query should have looked something like this:

```
SELECT

  artist,

  week,

  streams_millions,

  streams_millions - LAG(streams_millions, 1, streams_millions) OVER
(
  PARTITION BY artist

  ORDER BY week

) AS 'streams_millions_change',

  chart_position,

  LAG(chart_position, 1, chart_position) OVER (

    PARTITION BY artist

    ORDER BY week

  ) - chart_position AS 'chart_position_change'

FROM

  streams
```

```
WHERE
```

```
artist = 'Lady Gaga';
```

Copy to Clipboard

We are going to modify this to use `LEAD`, which looks to future rows, instead of `LAG`, which looks to previous rows.

Instructions

Checkpoint 1 Enabled

1.

Because `LEAD` looks to future rows, we need to flip how we subtract in order to find the `streams_millions_change`:

```
SELECT
    artist,
    week,
    streams_millions,
    LEAD(streams_millions, 1) OVER (
        PARTITION BY artist
        ORDER BY week
    ) - streams_millions AS 'streams_millions_change'
FROM
    streams;
```

Copy to Clipboard

We will subtract the current row's `streams_millions` from the *next row's* `streams_millions` in order to see an accurate reflection of how the number of streams has changed.

Hint

Positive means they increased streams while negative means they decreased. If we did not flip our subtraction here, we would get *negative* values for an increase in streams which is not what we want.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now modify your query to make it return the `chart_position` and `chart_position_change` as well, like we did for `LAG`.

```
SELECT
    artist,
    week,
    streams_millions,
    LEAD(streams_millions, 1) OVER (
        PARTITION BY artist
        ORDER BY week
    ) - streams_millions AS 'streams_millions_change',
    chart_position,
    chart_position - LEAD(chart_position, 1) OVER (
        PARTITION BY artist
        ORDER BY week
    ) AS 'chart_position_change'
FROM
    streams;
```

Window Functions

ROW_NUMBER

Now that we have a good understanding of how to fetch data from different rows, let's explore how we can use

Preview: Docs Loading link description

[window functions](#)

to order and rank our results.

The most straight-forward way to order our results is by using the `ROW_NUMBER` function which adds a *sequential* integer number to each row.

Adding a `ROW_NUMBER` to each row can be useful for seeing where in your result set the row falls.

Instructions

Checkpoint 1 Enabled

1.

Here is a query that uses the `ROW_NUMBER` function, which is quite easy!

All we need is to write `ROW_NUMBER() OVER` and then the column you'd like to number by:

```
SELECT
    ROW_NUMBER() OVER (
        ORDER BY streams_millions
    ) AS 'row_num',
    artist,
    week,
    streams_millions
FROM
    streams;
```

Copy to Clipboard

Run the query. Would `row_num = 1` have the most or least streams?

Hint

`ORDER BY` returns numeric values in ascending order (lowest to highest) by default, so `row_num = 1` would be our lowest value.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

What happens for the `row_num` when the `streams_millions` are equal?

Click Run to continue.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Modify our query to return the results with `row_num = 1` being the *most* streams.

```
SELECT

  ROW_NUMBER() OVER (

    ORDER BY streams_millions DESC

  ) row_num,

  artist,

  week,

  streams_millions

FROM

  streams;
```

Window Functions

RANK

Now that we understand how to use `ROW_NUMBER`, there is another function that is similar but provides an actual ranking: `RANK`.

If you were to modify your `ROW_NUMBER` query to use `RANK` instead, it might appear to be exactly the same at first glance. But if you look more closely, you can see that `RANK` will follow standard ranking rules so that when two values are the same, they will have the same rank whereas with `ROW_NUMBER` they would not.

Instructions

Checkpoint 1 Enabled

1.

Run the following query.

```
SELECT
    RANK() OVER (
        ORDER BY streams_millions
    ) AS 'rank',
    artist,
    week,
    streams_millions
FROM
    streams;
```

Copy to Clipboard

What happened to our first 3 results?

Hint

Because we are using `RANK` now and the 7th and 8th spot have the same value (26.3), they are both ranked as 7. Then, in standard ranking fashion, the next value is ranked as 9.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now modify your query to return the results so that the streams are ranked with 1 being the most streams in our dataset.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Lastly, modify the query that it is partitioning our results by week so we can see the most streamed artist of each week.

```
SELECT

    RANK() OVER (

        PARTITION BY week

        ORDER BY streams_millions DESC

    ) AS 'rank',

    artist,

    week,

    streams_millions

FROM

    streams;
```

Window Functions

NTILE

The last window function we are going to learn is called `NTILE` and this can be used to find quartiles, quintiles or whatever ntile your data-driven heart desires.

`NTILE` allows you to break your data into roughly equal groups, based on what ntile you'd like: so if you were using *quartile*, it would divide the data into *four groups* (quarters).

When using `NTILE` you are required to provide a *bucket*, which represents the number of groups you'd like your data broken down into: `NTILE(4)` would be four "buckets" which would represent **quartiles**.

Instructions

Checkpoint 1 Enabled

1.

Run the following query:

```
SELECT
    NTILE(5) OVER (
        ORDER BY streams_millions DESC
    ) AS 'weekly_streams_group',
    artist,
    week,
    streams_millions
FROM
    streams;
```

Copy to Clipboard

Think about the following:

What does each `NTILE` represent?

If you're in `weekly_streams_group 1`, are you the most or least streamed artist?

What artists make up the most streamed grouping?

Hint

Because we have 5 buckets, this `NTILE` would represent *quintiles* or fifths. The top quintile is made almost entirely of Drake. The Weeknd and Bad Bunny also had enough weekly streams to land them a few spots in the top quintile.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Change our `NTILE` query to be a *quartile*, name the column `quartile` and run the query. How does this change our resultant data? What artists are now in the top quartile?

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Now change our query to also `PARTITION BY week`.

How does this change our data?

```
SELECT
  NTILE(4) OVER (
    PARTITION BY week
    ORDER BY streams_millions DESC
  ) AS 'quartile',
  artist,
  week,
  streams_millions
FROM
  streams;
```

Math and Date Functions

Introduction

In this lesson, we will introduce some math

Preview: Docs Loading link description

[operators](#)

and functions to perform calculations on and convert values, along with a few date functions to transform, format, and extract information from dates.

Throughout the lesson, you will be working with the following tables containing mock data:

`bakery`: table storing information about orders made at a local bakery

`guesses`: table storing the values of a guessing game where the guessed value closest to a number wins

Here's a quick preview of the concepts that will be covered in these next six exercises:

Math operators:

- + addition
- subtraction
- / division
- * multiplication
- % modulo (returns the remainder)

Math functions:

Preview: Docs Outputs the positive equivalent of a numeric argument.

ABS ()

: returns the absolute value of the input expression

CAST () : converts an expression into another data type

Date and time functions:

Preview: Docs Returns the both the date and time (YYYY-MM-DD hh:mm:ss).

DATETIME ()

: returns the date and time of a time string

DATE () : returns the date portion of a time string

Preview: Docs Converts timestamps to time (hh:mm:ss).

TIME ()

: returns the time portion of a time string

Preview: Docs Returns a formatted date.

STRFTIME ()

: returns a formatted date

Let's get started!

Instructions

Checkpoint 1 Enabled

1.

Before moving on, take a look at the data in the `bakery` table.

In the code editor, write the query to select the first 5 rows.

What are the column names?

Hint

The `LIMIT` statement is used to limit the number of records returned based on a limit value:

```
SELECT *  
FROM bakery  
LIMIT 5;
```

Copy to Clipboard

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

And now take a look at the data in the `guesses` table.

Delete the old query and write a new query to select the first 5 rows of `guesses`.

What are the column names?

Database Schema	
bakery	
name	type
id	INTEGER
item_name	TEXT
price	REAL
quantity	INTEGER

discount	TEXT
order_date	DATETIME

Rows: 30
guesses

name	type
id	INTEGER
first_name	TEXT
last_name	TEXT
guess	INTEGER
Rows: 30	

Math and Date Functions

Math Operators

SQL supports math

Preview: Docs Loading link description

[operators](#)

to perform calculations.

The `+` operator is for addition and returns the sum of two values.

The `-` operator is for subtraction and returns the difference between two values.

The `/` operator is for division and returns the quotient after dividing one value by another value. If the two values are of type `INTEGER`, then the result will also be an `INTEGER` rounded down. If one or more of the values are of type `REAL`, then the result will be `REAL`.

The `*` operator is for multiplication and returns the product of two numbers.

The `%` operator, or modulo, returns the remainder after division.

```
SELECT (1 + 5); -- 6

SELECT (10 - 6); -- 4

SELECT (4 / 2); -- 2

SELECT (15 * 3); -- 45

SELECT (10 % 3); -- 1
```

Copy to Clipboard

Instructions

Checkpoint 1 Passed

1.

From the `bakery` table, compute the total cost (`price` times `quantity`) of each order.

Hint

```
SELECT price * quantity
FROM bakery;
```

Math and Date Functions

Abs

The Preview: Docs Outputs the positive equivalent of a numeric argument.

`ABS()`

function returns the absolute value of the input expression.

The absolute value of a number is the distance of that number from zero, taken as a positive value. It can also be understood as the value of the number if we ignore its sign, such that negative values become positive, and positive values remain the same.

For example,

```
SELECT ABS (-3); -- 3

SELECT ABS (5); -- 5
```

Copy to Clipboard

Here, the absolute value of -3 is equal to 3, because it is 3 away from 0, and because ignoring the negative sign leaves 3. And, because 5 is a positive value, its absolute value remains 5.

Instructions

Checkpoint 1 Enabled

1.

A school holds an event every month where a glass jar is filled with jelly beans, and students can guess how many jelly beans are contained in the jar. Students who make the closest guess win a pack of new notebooks.

Using the `guesses` table with the `ABS()` function, write a query to find out how close each student's guess is to the actual number of jelly beans in the jar, which is 804. Select the first

name and the absolute difference of each guess from 804.

How close are each of the guesses?

Hint

```
SELECT first_name, ABS(guess - 804)
FROM guesses;
```

Copy to Clipboard

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Find out the absolute value of the difference between the average guess of all students from the actual jelly bean count, which is 804.

Is the average guess close to the actual count?

```
-- Checkpoint 1

SELECT first_name, ABS(guess - 804)

FROM guesses;


-- Checkpoint 2

SELECT ABS(AVG(guess) - 804)

FROM guesses;
```

Math and Date Functions

Cast

The `CAST()` function is used to convert the value of an expression into another data type.

It has the following syntax:

```
SELECT CAST(expr AS type-name);
```

Copy to Clipboard

Here, `expr` is any expression that evaluates to a value, and `type-name` is the data type to convert that value into.

This function can be useful when performing calculations on values but we need to change their

Preview: Docs Loading link description

[data types](#)

. For example, say that we want to divide two `INTEGER` numbers.

```
SELECT 3 / 2; -- 1
```

Copy to Clipboard

In the query above, the result would be rounded down to the nearest `INTEGER` number, 1, because 3 and 2 are both `INTEGER` values. Using `CAST()`, one of the values could be converted to a `REAL` so that the result is also of type `REAL`:

```
SELECT CAST(3 AS REAL) / 2; -- 1.5
```

Copy to Clipboard

In this query, `CAST(3 AS REAL)` will convert 3 to the `REAL` value 3.0. Then it will calculate 3.0 / 2 which

Preview: Docs Loading link description

[equals to](#)

1.5.

`CAST()` can also be used to convert `TEXT` values into a numerical value like `REAL` or `INTEGER`. When doing so, it will only consider any valid prefix at the beginning of the string that represents a numerical value.

For example,

```
SELECT CAST('3.14 is pi' AS REAL);
```

Copy to Clipboard

This query will convert the `TEXT` value `'3.14 is pi'` into type `REAL`, considering only the prefix `'3.14'` and ignoring `' is pi'`, resulting in `3.14`.

Instructions

Checkpoint 1 Enabled

1.

From the `bakery` table, we want to determine the final price of each order after the `discount` is applied. However, the value of each discount is currently stored as `TEXT`, like `'1.00 off'`, so you will need to convert them into numerical values.

Utilize `CAST()` to convert the values of the `discount` column into `REAL` values. Select the `item_name` column and the total cost of each order, after applying the discount for each item.

Hint

```
SELECT item_name, (price - CAST(discount AS REAL)) * quantity
FROM bakery;
```

```
SELECT item_name, (price - CAST(discount AS REAL)) * quantity
FROM bakery;
```

Math and Date Functions

Date and Time Functions I

SQL provides functions to work with dates.

In SQL, dates are typically written in one of the following formats:

Date: `YYYY-MM-DD`

Datetime or Timestamp: `YYYY-MM-DD hh:mm:ss`

These strings that represent dates are also known as time strings.

The

Preview: Docs Returns the both the date and time (YYYY-MM-DD hh:mm:ss).

`DATETIME()`

function will return the entire time string which includes the date and time portions.

```
SELECT DATETIME('2020-09-01 17:38:22');
```

Copy to Clipboard

To obtain the current date and time, you can provide the string `'now'` to the function, which returns the date and time in UTC.

```
SELECT DATETIME('now');
```

Copy to Clipboard

To obtain the date and time converted to your local timezone, you can provide a modifier `localtime`.

```
SELECT DATETIME('now', 'localtime');
```

Copy to Clipboard

The `DATE()` function allows us to extract just the date portion of a time string, which consists of the year, month and day.

```
SELECT DATE('2020-09-01 17:38:22'); -- 2020-09-01
```

Copy to Clipboard

The Preview: Docs Loading link description

`TIME()`

function allows us to extract just the time portion of a time string, which consists of the hour, minute and second.

```
SELECT TIME('2020-09-01 17:38:22'); -- 17:38:22
```

Copy to Clipboard

Instructions

Checkpoint 1 Enabled

1.

Write a query to select *just the dates* for each order's `order_date` column from the `bakery` table.

Hint

The `DATE()` function allows us to extract just the date portion of a time string, which consists of the year, month, and day.

```
SELECT DATE(order_date)
FROM bakery;
```

Copy to Clipboard

```
SELECT DATE(order_date)
FROM orders;
```

Math and Date Functions

Date and Time Functions II

We can provide additional arguments, called modifiers, to date functions in addition to the time string. Modifiers are applied from left to right as they are listed in the function, so order matters.

The syntax is as follows:

```
SELECT DATETIME(timestring, modifier1, modifier2, ...);
```

In the previous exercise, you were introduced to the modifier `'localtime'`, which converts the time string from UTC (Coordinated Universal Time) to the local time zone.

The following modifiers can be used to shift the date backwards to a specified part of the date.

`start of year`: shifts the date to the beginning of the current year.

`start of month`: shifts the date to the beginning of the current month.

`start of day`: shifts the date to the beginning of the current day.

For example, the following returns the beginning of September 2005.

```
SELECT DATE('2005-09-15', 'start of month');
```

The following modifiers add a specified amount to the date and time of the time string.

`'+-N years'`: offsets the year

`'+-N months'`: offsets the month

`'+-N days'`: offsets the day

`'+-N hours'`: offsets the hour

`'+-N minutes'`: offsets the minute

`'+-N seconds'`: offsets the second

After each offset is applied, the result date and time is normalized so that it will always be valid.

For example, we can have the following statement.

```
SELECT DATETIME('2020-02-10', 'start of month', '-1 day', '+7 hours');
```

First, it will apply the modifier `'start of month'` which will shift to the beginning of the month, `'2020-02-01 00:00:00'`. It will include the time portion because we are using the

Preview: Docs Loading link description

DATETIME()

function.

Then, it will apply the modifier `'-1 day'` which will offset the day by -1, resulting in `'2020-01-31 00:00:00'`.

Finally, it will apply the modifier `'+7 hours'`, which will add 7 hours to the time, giving the final result of `'2020-01-31 07:00:00'`.

Instructions

Checkpoint 1 Enabled

1.

For each order in the `bakery` table, the order will be ready for pick up 2 days after the order is made, at 7:00AM, 7 days a week. The order will always be ready 2 days after the order, no matter what time of day the order was made.

Utilizing the date and time functions and some modifiers, find out when each order can be picked up.

Hint

```
SELECT DATETIME(order_date, 'start of day', '+2 days', '+7
hours')
FROM bakery;
```

Math and Date Functions

Date and Time Functions III

The

Preview: Docs Loading link description

STRFTIME ()

function allows you to return a formatted date, as specified in a format string.

It has the following syntax:

```
STRFTIME (format, timestring, modifier1, modifier2, ...)
```

Copy to Clipboard

The first argument, `format`, is the format string.

The second argument is the `timestring`.

The remaining arguments are 0 or more optional `modifiers` to transform the time string.

Let's go into some more detail about the format string. The format string allows you to extract specific parts of the date and time. Recall the time string format of `YYYY-MM-DD HH:MM:SS`. Each part of this time string can be extracted utilizing what are known as substitutions inside the format string.

The substitutions to extract each part of the date and time are the following:

`%Y` returns the year (YYYY)

`%m` returns the month (01-12)

`%d` returns the day of month (01-31)

`%H` returns the hour (00-23)

`%M` returns the minute (00-59)

`%S` returns the second (00-59)

For example, to extract the month and year of the current date,

```
SELECT STRFTIME('%m %Y', 'now');
```

Copy to Clipboard

And to extract just the current day,

```
SELECT STRFTIME('%d', 'now');
```

Copy to Clipboard

Instructions

Checkpoint 1 Enabled

1.

The owners of the bakery want to find out what day most people placed orders.

Use `STRFTIME()` with the `COUNT()` function to find out how many orders were made on each day. Show the results in descending order based on the daily number of orders.

Hint

```
SELECT strftime('%d', order_date) AS 'order_day',  
       COUNT(*)  
FROM bakery  
GROUP BY 1  
ORDER BY 2 DESC;
```

```
select strftime('%d', order_date) AS 'order_day',  
       COUNT(*)  
FROM bakery  
GROUP BY 1  
ORDER BY 2 DESC;
```

Math and Date Functions

Review

Congratulations!

You've just learned several useful math

Preview: Docs Loading link description

[operators](#)

and functions, along with some date functions in SQL.

Let's review everything you've gone over so far.

Math operators:

- + Addition
- Subtraction
- / Division
- * Multiplication
- % Modulo

Math functions:

Preview: Docs Outputs the positive equivalent of a numeric argument.

`ABS()`

: Gives us the absolute value of the input expression

`CAST()`: Converts the value of an expression into another data type

Date functions:

Preview: Docs Loading link description

`DATETIME()`

: Obtains the date and time of a time string

`DATE ()` : Obtains the date portion of a time string

Preview: [Docs Loading link description](#)

`TIME ()`

: Obtains the time portion of a time string

Preview: [Docs Loading link description](#)

`STRFTIME ()`

: Returns a formatted date

Date function modifiers:

Modifiers to shift the date backwards to a specified part:

`'start of year'`

`'start of month'`

`'start of day'`

Modifiers to add a specified amount to the date and time of a time string:

`'+-N years'`

`'+-N months'`

`'+-N days'`

`'+-N hours'`

`'+-N minutes'`

`'+-N seconds'`

Format string substitutions:

`%Y` returns the year (YYYY)

`%m` returns the month (01-12)

`%d` returns the day of month (01-31)

`%H` returns the hour (00-23)

`%M` returns the minute (00-59)

`%S` returns the second (00-59)

Interview Prep Problems

Code Challenge 2

The `nba_matches` table has the following columns:

`id` - the match id

`date` - the date of the match

`home_team` - the home team

`away_team` - the away team

`home_points` - the home team points

`away_points` - the away team points

Instructions

Checkpoint 1 Passed

1.

Given the final scores of several NBA games, use `CASE` to return the results for each game:

If home team won, return 'HOME WIN'.

If away team won, return 'AWAY WIN'.

Select the `id` column and the `CASE` result.

Hint

```
SELECT id,  
       CASE  
         WHEN home_points > away_points  
           THEN 'HOME WIN'  
         ELSE 'AWAY WIN'  
       END  
FROM nba_matches;
```

```
SELECT id,  
       CASE  
         WHEN home points > away points
```

```
        THEN 'HOME WIN'

        ELSE 'AWAY WIN'

    END

FROM nba_matches;
```

SUBQUERY

Code Challenge 8

The `math_students` and `english_students` tables have the following columns:

`student_id` - the student id

`grade` - the grade level of the student

`first_name` - the student's first name

`last_name` - the student's last name

Instructions

Checkpoint 1 Passed

1.

Using a subquery, find all students enrolled in english class who are not also enrolled in math class.

Hint

```
SELECT *
FROM english_students
WHERE student_id
NOT IN (
    SELECT student_id
    FROM math_students
```



```
);
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Using a subquery, find out what grade levels are represented in both the math and english classes.

Hint

```
SELECT grade
FROM math_students
WHERE EXISTS (
    SELECT grade
    FROM english_students
);
```

WINDOWS

Code Challenge 9

The `box_office` table has the following columns:

`id` - the id of each row

`title` - the title of the movie

`week` - the week following the film's release

`gross` - the gross for that week

`to_date_gross` - the total gross of the movie up to each week

Instructions

Checkpoint 1 Passed

1.

Using a window function with `PARTITION BY`, get the running total in `gross` for each movie up to the current `week` and display it next to the current `week` column along with the `title`, `week`, and `gross` columns.

Hint

```
SELECT title, week, gross,  
       SUM(gross) OVER (  
         PARTITION BY title  
         ORDER BY week  
       ) AS 'running_total_gross'  
FROM box_office;
```

```
SELECT title, week, gross,  
       SUM(gross) OVER (  
         PARTITION BY title  
         ORDER BY week  
       ) AS running_total_gross  
FROM box_office;
```

Code Challenge 10

The `box_office` table has the following columns:

`id` - the id of each row

`title` - the title of the movie

`week` - the week following the film's release

`gross` - the gross for that week

`to_date_gross` - the total gross of the movie up to each week

Instructions

Checkpoint 1 Passed

1.

Write a query using a window function with `ROW_NUMBER` and `ORDER BY` to see where each row falls in the amount of `gross`.

Hint

```
SELECT ROW_NUMBER()  
OVER (  
    ORDER BY gross  
) AS 'row_num', title, week, gross  
FROM box_office;
```

OPERATIONS (+-*/)

Interview Prep Problems

Code Challenge 11

The `orders` table has the following columns:

`id` - the order id

`product_id` - the product id

`price` - the price of the individual product item

`quantity` - the quantity of items in the order

Instructions

Checkpoint 1 Passed

1.

Given an `orders` table, calculate the `price` times `quantity` of each order. Include the `id` and `product_id` columns in the result.

Hint

```
SELECT id, product_id, price * quantity
FROM orders;
```

Code Challenge 12

The `weather` table has the following columns:

`id` - the id of each entry

`date` - the date of each entry

`high` - the high temperature of the date, in Fahrenheit

`low` - the low temperature of the date, in Fahrenheit

Instructions

Checkpoint 1 Passed

1.

Utilize `CAST` to calculate the average of the `low` and `high` temperatures for each `date` such that the result is of type `REAL`.

Select the `date` column and alias this result column as 'average'.

Hint

```
SELECT date, (CAST(high AS 'REAL') +
             CAST(low AS 'REAL')) / 2.0 AS 'average'
FROM weather;
```

Code Challenge 13

The `purchases` table has the following columns:

`purchase_id` - the id of the purchase

`purchase_date` - the date of the purchase

Instructions

Checkpoint 1 Passed

1.

After a purchase is created, it can be returned within 7 days for a full refund.

Using modifiers, get the date of each purchase offset by 7 days in the future.

Hint

```
SELECT purchase_id, DATE(purchase_date, '+7 days')
FROM purchases;
```

Interview Prep Problems

Code Challenge 13

The `purchases` table has the following columns:

`purchase_id` - the id of the purchase

`purchase_date` - the date of the purchase

Instructions

Checkpoint 1 Passed

1.

After a purchase is created, it can be returned within 7 days for a full refund.

Using modifiers, get the date of each purchase offset by 7 days in the future.

Hint

```
SELECT purchase_id, DATE(purchase_date, '+7 days')
FROM purchases;
```

Copy to Clipboard

Checkpoint 2 Failed, try again

2.

Which hour had the most purchases made?

The simple query would be:

Copy to Clipboard

```
SELECT strftime('%H',purchase_date) AS 'Hour',
COUNT(strftime('%H',purchase_date)) AS 'Purchases' FROM purchases
GROUP BY 1 ORDER BY 2 desc;
```

Code Challenge 14

`purchase_id` - the id of the purchase

`purchase_date` - the date of the purchase

Instructions

Checkpoint 1 Passed

1.

Using string formatting and substitutions, get the month and day for each purchase in the form 'mm-dd'.

Give this new column a name of 'reformatted'.

Hint

```
SELECT STRFTIME('%m-%d', purchase_date) AS 'reformatted'
FROM purchases;
```

Common SQL Interview Questions

Practice with some common SQL interview questions.

Congrats on making it through the entire course! You've learned about many different concepts in SQL that can be applied in a data analyst role so now it's time to put your knowledge to the test!

This article will provide a list of some common SQL interview questions that you may encounter in a real interview, along with answers.

To get the most out of it, try to answer each question on your own first. Good luck!

1. What is SQL?

SQL, which stands for Structured Query Language, is a programming language to communicate with data stored in a relational database management system.

Its syntax is similar to English, making it easy to write, read, and interpret. It allows you to write queries which define the subset of data you are seeking. You can save these queries, refine them, share them, and run them on different databases.

2. What is a database?

A database is a set of data stored in a computer, where the data is structured in a way that makes it easily accessible.

3. What is a relational database?

A relational database is a type of database. It uses a structure that allows us to identify and access data in relation to other pieces of data in the database.

Data in a relational database is often organized as tables.

4. What is a RDBMS?

A relational database management system (RDBMS) is a program that allows you to create, update, and administer a relational database.

Most relational database management systems use SQL language to access the databases.

The most popular RDBMS is MySQL. Others include PostgreSQL, Oracle DB, SQL Server, and SQLite.

5. What is a table?

A table is a collection of data organized into rows and columns. They are sometimes referred to as "relations".

They can have hundreds, thousands, and sometimes even millions of rows of data.

6. What is a row and column in a table?

A **row** is a single record of data in a table.

A **column** is a set of data values of a particular type.

7. What is a data type?

A data type is an attribute that specifies the type of data a column holds.

Every column of a database has a data type.

Some common data types are `INTEGER`, `TEXT`, `DATE`, `REAL`.

8. What is a primary key and foreign key?

A **primary key** is a column that uniquely identifies each row of a table.

Primary keys must satisfy the following requirements: No value can be `NULL`, each value must be unique, and a table cannot have more than one primary key column.

For example, in a `customers` table, the primary key would be `customer_id`.

A **foreign key** is the primary key for one table that appears in a different table. For example if there was also an `orders` table, each order can store the customer, such that the `customer_id` column would be the foreign key.

9. What is the difference between ALTER and UPDATE?

The `ALTER` statement is used to add a new column to a table. It changes the table structure.

The `UPDATE` statement is used to edit a row in the table. It changes existing records in the table.

10. What is a query?

A query is a SQL statement used to retrieve information stored in a database.

They allow us to communicate with the database by asking questions and returning a result set with data relevant to the question.

11. What is a subquery?

A subquery is an internal query nested inside of an external query.

They can nested inside of `SELECT`, `INSERT`, `UPDATE`, or `DELETE` statements.

When a subquery is present, it will be executed before the external statement is run.

12. What are constraints?

Constraints are a set of rules used to tell the database to limit the type of data that can be stored for the columns. They tell the database to reject inserted data that does not adhere to the restriction.

They add information about how a column can be used, and are invoked after the data type for a column.

Some examples of constraints are:

`PRIMARY KEY`, which uniquely identifies each row and requires each value to be unique. `UNIQUE`, which requires every value in the column to be different. `NOT NULL`, which requires columns to have a value. `DEFAULT`, which takes an additional argument which will be the assumed value for an inserted row if the new row does not specify a value for that column.

13. What is a statement?

A statement is text that the database recognizes as a valid command.

They can be used to perform tasks like altering the structure of a table, updating data, or retrieving data from a database.

The structure of a statement can vary, but each must end with a semi-colon `;`. The number of lines used for a statement does not matter, and it can be written as a single line or split into multiple lines to make it easier to read.

14. How do you check if a field does not have a value or has a value?

When a field has no value, it is indicated with a `NULL` value.

To check if a field does not have a value, you can use `IS NULL` as the condition: `WHERE col IS NULL`.

To check if a field has a value, you can use `IS NOT NULL` as the condition: `WHERE col IS NOT NULL`.

15. What is the difference between `DISTINCT` and `UNIQUE`?

`DISTINCT` is a keyword used when we want to return unique values in the output. It filters out all duplicate values in the specified column.

`UNIQUE` is a constraint used to ensure that all values of a column are different. It is similar to `PRIMARY KEY`, except that a table can have many different `UNIQUE` columns.

16. What are aggregate functions used for?

Aggregate functions are used to perform a calculation on one or more values, and returns a single value of more meaningful information.

Some aggregate functions are `COUNT()`, `SUM()`, `MAX()`, `MIN()`, `AVG()`, and `ROUND()`.

17. What is a join?

A join is a way to combine rows from two or more tables, based on a related column between them.

18. What is the difference between an INNER JOIN and LEFT JOIN?

An `INNER JOIN` is used to include combined rows from two tables that match the `ON` condition. The final result does not include rows with no match for the `ON` condition.

A `LEFT JOIN` is used to keep all rows from the first table, regardless of whether there is a matching row in the second table for the `ON` condition.

19. What is the purpose of windows functions?

Windows functions are used when you want to maintain the values of your original table while displaying grouped or summative information alongside. It is similar to aggregate functions, but does not reduce the number of rows in the result by combining or grouping them into a few result.

20. What are indexes and why are they needed?

Indexes are a powerful tool used in the background of a database to speed up querying, by acting as a lookup table for data.

They are needed to efficiently store data for quicker retrieval, which can be paramount to the success of large tech companies which need to process on the scale of petabytes of data each day.

PIVOT

MERGE

Exploratory Data Analysis

Exploring your data is the first step to understanding it. Preview what you will learn in Exploratory Data Analysis.

Welcome to Exploratory Data Analysis (EDA). EDA is an essential step in the Data Science process, and establishes the foundation for your data science projects. What you learn during EDA will determine what kinds of analysis you can do, and what models you can build.

Sometimes, the results of EDA are the answers to the questions you are asking. Questions about what the data shows are often questions of Summary Statistics, which are part of EDA. So, becoming fluent in when to look at the mean vs. the median of a dataset or when the range vs. the inter-quartile range is the best answer will be essential on your data science journey.

In this unit, we will introduce the essential techniques, most popular tools, and things you should look out for when exploring your data for the first (or the 50th) time.

What will you learn?

In this unit, you will start building proficiency with summary statistics and variable types.

After this course, you will be able to:

- Identify different variable types
- Determine what kinds of analysis are appropriate based on the variable type
- Summarize a single variable
- Summarize the relationship between two variables
- Inspect and clean a dataset.

Why is this important?

Exploratory Data Analysis is an essential tool for everyone working with data. Developing fluency with summary statistics and variable types will make it easier to answer questions about data and decide what kinds of analysis can be run and what types of models can be built.

What is EDA?

Learn about EDA and some of the techniques involved.

Exploration is curiosity put into action.

– [Don Walsh](#), American Oceanographer

Exploratory Data Analysis (EDA for short) is all about getting curious about your data – finding out what is there, what patterns you can find, and what relationships exist. EDA is the important first step towards analysis and model building. When done well, it can help you formulate further questions and areas for investigation, and it almost always helps uncover aspects of your data that you wouldn't have seen otherwise.

Goals of EDA

Depending on what you want to do with your data, EDA can take many different forms; However, the main goals of EDA are generally:

Uncover the data structure and determine how it is coded

Inspect and “get to know” the data by summarizing and visualizing it

Detect outliers, missing data, and other anomalies and decide how/whether to address these issues

Find new avenues for analysis and further research

Prepare for model building or analysis, including the following:

- Check assumptions

- Select features

- Choose an appropriate method

EDA techniques

Just as the goals of EDA may vary, so do the techniques used to accomplish those goals. That said, the EDA process generally involves strategies that fall into the following three categories:

- Data inspection
- Numerical summarization
- Data visualization

Data inspection

Data inspection is an important first step of any analysis. This can help illuminate potential issues or avenues for further investigation.

For example, we might use the pandas `.head()` method to print out the first five rows of a dataset:

```
print(data.head())
```

Student Name	Favorite Class	Age	Hours of Sleep	Hours Spent Studying
--------------	----------------	-----	----------------	----------------------

John	Math	9	9	1
Sophie	Statistics	17	7.5	4
Alex	English	12	8	3
Liam	Statistics	18	nan	4.5
Colin	Math	18	6.5	6.5

Based on this output, we notice that hours of sleep is a quantitative variable. In order to summarize it, we'll need to make sure it is stored as an `int` or `float`.

We also notice that there is at least one instance of missing data, which appears to be stored as `nan`. As a next step, we could investigate further to determine how much missing data there is and what we want to do about it.

Numerical summarization

Once we've inspected our data and done some initial cleaning steps, numerical summaries are a great way to condense the information we have into a more reasonable amount of space. For numerical data, this allows us to get a sense of scale, spread, and central tendency. For categorical data, this gives us information about the number of categories and frequencies of each.

In pandas, we can get a quick collection of numerical summaries using the `.describe()` method:

```
data.describe(include = 'all')
```

	Student Name	Favorite Class	Age	Hours of Sleep	Hours Spent Studying
count	250	250	250	250	250

unique	177	15	NaN	NaN	NaN
top	Kevin	Math	NaN	NaN	NaN
freq	12	23	NaN	NaN	NaN
mean	NaN	NaN	13.75	7.89	4.34
std	NaN	NaN	1.68	0.3	0.6
min	NaN	NaN	8	4.5	0.5
25%	NaN	NaN	10.3	5.6	1.47
50%	NaN	NaN	13.5	7.6	4.32
75%	NaN	NaN	17	9.7	6.5
max	NaN	NaN	23	11	10.5

Based on this table, we can see that there are 177 unique student names in our table, with Kevin being the most common. The average student age is 13.75 years with students as young as 8 years and as old as 23 years.

Data visualization

While numerical summaries are useful for condensing information, visual summaries can provide even more context and detail in a small amount of space.

There are many different types of visualizations that we might want to create as part of EDA. For example, histograms allow us to inspect the distribution of a quantitative feature, providing information about central tendency, spread, and shape (eg., skew or multimodality). The histogram below shows the distribution of the number of characters in each student's name. We see that the average name is about 5-6 characters long and up to 10 characters long.



Other kinds of visualizations are useful for investigating relationships between multiple features. For example, the scatterplot below shows the relationship between hours spent studying and hours spent sleeping.



EDA as a cyclical process

Though EDA is commonly performed at the start of a project — before any analysis or model building — you may find yourself revisiting EDA again and again. It is quite common for more questions and problems to emerge during an analysis (or even EDA itself!). EDA is also a great tool for tuning a predictive model to improve its accuracy. It is therefore useful to think of EDA as a cycle rather than a linear process in a data science workflow.

Conclusion

EDA is a crucial step before diving into any data project because it informs data cleaning, can illuminate new research questions, is helpful in choosing appropriate analysis and modeling techniques, and can be useful during model tuning.

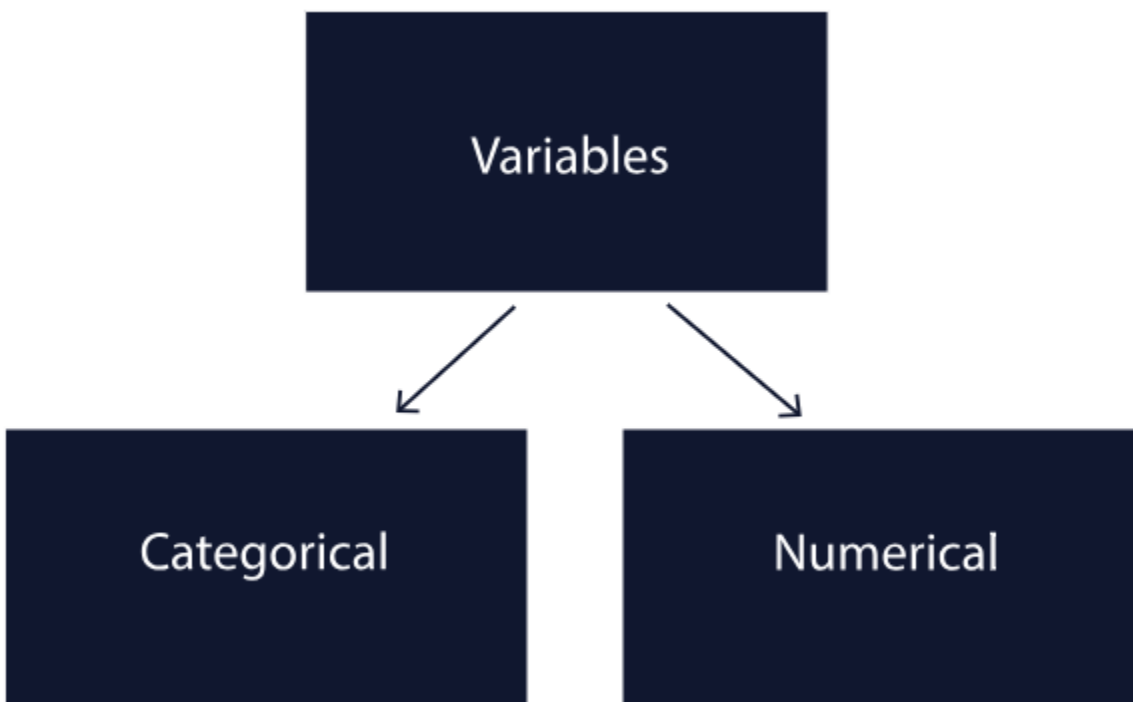
Assessing Variable Types

"It all began with a variable", the storyteller began.

Just kidding, no one starts their stories that way, even though variables are where all data stories begin.

Variables define datasets. They are the characteristics or attributes that we evaluate during data collection. There are two ways to do that evaluation: we can measure or we can categorize.

How we evaluate determines what kind of variable we have. Since there are only two ways to get data, there are only two types of variables: numerical and categorical.



Every observation (the individuals or objects we are collecting data about) is classified according to its characteristics. In “flat” file formats (like tables, csvs, or DataFrames), the observations are the rows, the variables are the columns, and the values are at the intersection.

We’ll go deeper into categorical and numerical variables in the following exercises.

Typically, the best way to understand your data is to look at a sample of it.

In the example dataset about cereal below, we can look at the first few rows with the `.head()` method to get an idea of the variable types that we have.

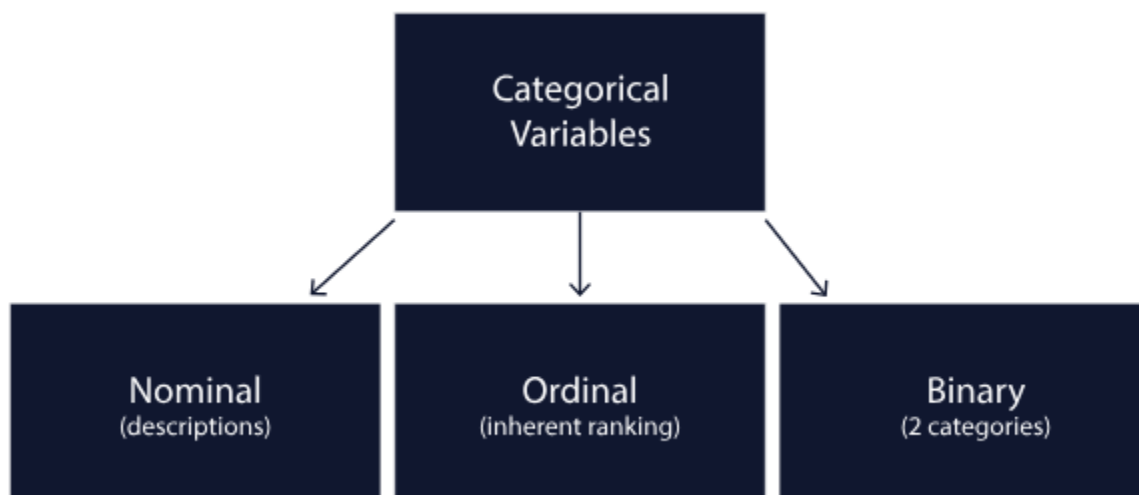
Categorical Variables

Without getting too philosophical about it, “your mind is a categorization machine” - [HBR](#). We move through the world by categorizing things into various groups: safe/unsafe, best/worst, on/off. These categorizations help us process information. They also create major problems for us when we over-categorize, and can lead to bias and unfair assumptions.

However, we still do it, and regardless of the dangers, categorization helps us transform the world around us into data. (Being aware of the dangers is a crucial part of data analysis, but outside the scope of this lesson.)

Categorical variables come in 3 types:

- Nominal variables, which describe something,
- Ordinal variables, which have an inherent ranking, and
- Binary variables, which have only two possible variations.



Let's look at each one separately

Nominal Variables

When we want to describe something about the world, we need a nominal variable. Nominal variables are usually words (i.e., red, yellow, blue or hot, cold), but they can also be numbers (i.e., zip codes or user id's).

Often, nominal variables describe something with a lot of variation. It can be hard to capture all of that variation, so an 'Other' category is often necessary. For example, in the case of color, we could have a lot of different labels, but might still need an 'Other' category to capture anything we missed.

Ordinal variables

When our categories have an inherent order, we need an ordinal variable. Ordinal variables are usually described by numbers like 1st, 2nd, 3rd. Places in a race, grades in school, and the scales in survey responses (Likert Scales) are ordinal variables.

Ordinal variables can be a little tricky because even though they are numbers, it doesn't make sense to do math on them. For example, let's say an Olympian won a Gold medal (1st place) and a Bronze medal (3rd place). We wouldn't say that they averaged Silver medals (2nd place).

Though there is [some debate about whether Likert scales should be treated like intervals or ordinal categories](#), most statisticians agree that they are ordinal categories and therefore should not be summarized numerically.

Binary variables

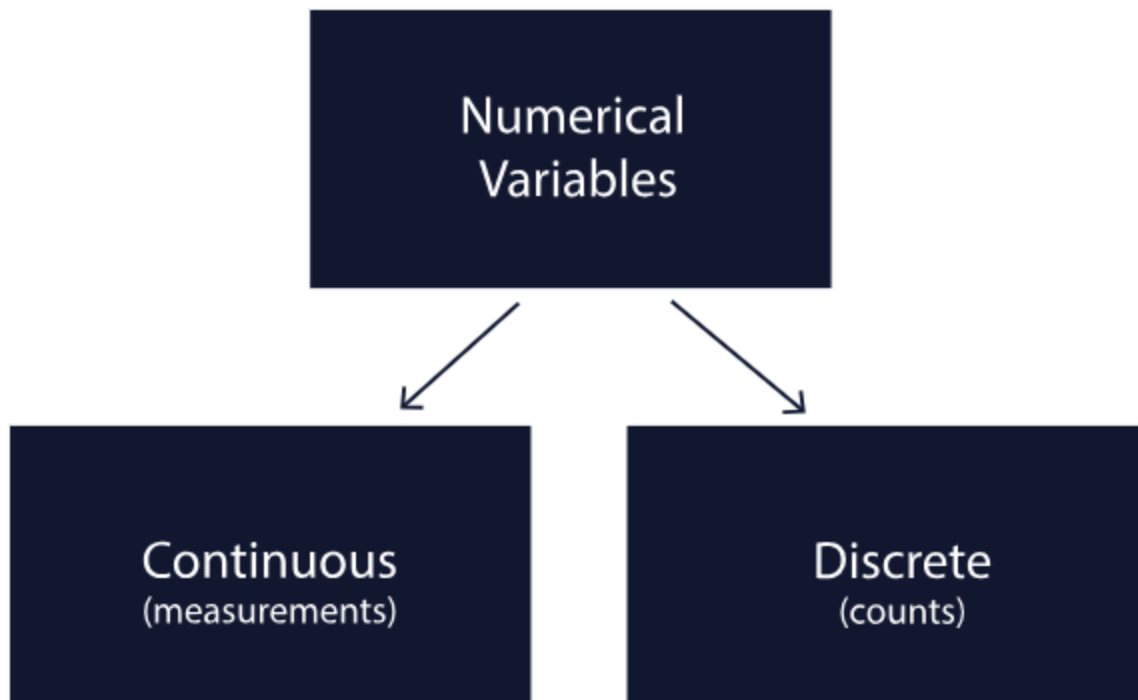
When there are only two logically possible variations, we need a binary variable. Binary variables are things like on/off, yes/no, and TRUE/FALSE. If there is any possibility of a third option, it is not a binary variable.

Quantitative Variables

"One accurate measurement is worth a thousand expert opinions"

– [Grace Hopper](#)

Numerical variables are created two ways: through measurement and counting. While measurement is a [matter of philosophical debate](#), counting is pretty straightforward. The result is continuous and discrete variables.



Continuous variables come from measurements. For a variable to be continuous, there must be infinitely smaller units of measurement between one unit and the next unit. Continuous variables can be represented by decimal places (but because of rounding, sometimes they are whole numbers). Length, time, and temperature are all good examples of continuous variables because they all increase continuously.

Discrete variables come from counting. For a variable to be discrete, there must be gaps between the smallest possible units. People, cars, and dogs are all good examples of discrete variables.

Some variables depend on context to determine if they are continuous or discrete. Money and time can both be measured continuously or discretely.

For money, all currencies have a smallest-possible-unit (i.e., the cent in USD) and are therefore discrete. However, banks and other institutions sometimes measure money in fractions of a cent, treating it like a continuous variable.

It is therefore always essential to understand how your data was created in order to represent it appropriately.

Variable Data Types

```
.astype()
```

- `int32` `int64`
- `float32` `float64`
- `object`
- `string`
- `Bool`

```
cereal['id'] = cereal['id'].astype("string")
```

```
print(cereal.dtypes)
```

Develop your One-Hot Encoding skills with the `pd.get_dummies()` method.

In this lesson, we used a `cereal` dataset [from Kaggle](#) , which was originally created by Chris Crawford and which contains data on various cereal brands in the US. We made alterations to this data for the purposes of the lesson. The other datasets used in this lesson can be found here:

The [movies](#) dataset courtesy of Shivam Bansal via Kaggle.

The [auto](#) dataset courtesy of UCI Machine Learning Repository.

The [titanic](#) dataset courtesy of Khashayar Baghizadeh Hosseini via Kaggle.

The [clothes](#) dataset courtesy of Nicapotato via Kaggle.

EDA: Inspect, Clean, and Validate a Dataset

Introduction

One of the most challenging parts of data cleaning is diagnosing data issues and figuring out HOW to most effectively address them. In order to accomplish this, exploratory data analysis (EDA) can be an extremely useful tool. In this article, we'll walk through an example dataset to demonstrate how EDA can inform the initial data inspection, cleaning, and validation process.

While this article serves as an introduction to EDA for data cleaning, it is important to note that every dataset is different, and therefore will require different exploration. EDA is all about following the data, verifying your assumptions, and investigating anything that is unexpected.

Initial Data Inspection

Before analysis or cleaning, it is useful to print a few rows of data. This helps ensure that the data is properly loaded. It also allows us to compare the observed data to the data dictionary and determine whether the coding appears to match our expectations. For example, let's load and inspect the first

few rows of a dataset of heart disease patients (downloaded from the [UCI Machine Learning Repository](#)).

```
import pandas as pd
heart = pd.read_csv('processed.cleveland.data.csv')
print(heart.head())
```

Output:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	heart_disease
0	63.0	1.0	1.0	145.0	233.0	1.0	2.0	150.0	0.0	2.3	3.0	0.0	6.0	0
1	67.0	1.0	4.0	160.0	286.0	0.0	2.0	108.0	1.0	1.5	2.0	3.0	3.0	2
2	67.0	1.0	4.0	120.0	229.0	0.0	2.0	129.0	1.0	2.6	2.0	2.0	7.0	1
3	37.0	1.0	3.0	130.0	250.0	0.0	0.0	187.0	0.0	3.5	3.0	0.0	3.0	0
4	41.0	0.0	2.0	130.0	204.0	0.0	2.0	172.0	0.0	1.4	1.0	0.0	3.0	0

There are a few things we might want to inspect. For example, the data dictionary gives the following information about the `cp` column:

`cp`: chest pain type

Value 1: typical angina
Value 2: atypical angina
Value 3: non-anginal pain
Value 4: asymptomatic

Based on this information, it's not necessarily clear whether the data is going to be coded as numerical values (eg., 1, 2, 3, or 4) or with strings (eg., 'typical angina'). Data inspection allows us to clarify that this column contains numerical values.

Similarly, there is some conflicting information in the data dictionary about the target column (note: we renamed this column as `heart_disease` before loading it, but it was originally coded as `num`). The list of features contains the following information about this column:

`num`: diagnosis of heart disease (angiographic disease status)

Value 0: < 50% diameter narrowing
Value 1: > 50% diameter narrowing

However, the initial data description suggests that the target field is integer valued from 0-4, where 0 indicates no heart disease, and values 1-4 indicate the presence of heart disease.

By inspecting the first few rows of data, we see at least one instance of the value 2 in the `heart_disease` column. This suggests that the values probably range from 0-4 instead of just 0-1.

We could verify this with further exploration (e.g., by using `heart.heart_disease.value_counts()` to get a table of values in this column).

Data Information

Once we've taken a first look at some data, a common next step is to address questions such as:

- How many (non-null) observations do we have?
- How many unique columns/features do we have?
- Which columns (if any) contain missing data?
- What is the data type of each column?

Using pandas, we can easily address these questions using the `.info()` method. For example:

Exploratory Data Analysis: Summary Statistics

Learn to explore a dataset by choosing a summary statistic.

Introduction

Summary statistics are an important component of Exploratory Data Analysis (EDA) because they allow a data analyst to condense a large amount of information into a small set of numbers that can be easily interpreted. In order to decide what kind of summary statistic to use, it is important to consider two things:

- The question (and how many variables that question involves)
- The data (is it quantitative or categorical?)

Univariate Statistics

Summary statistics that focus on a single variable are called *univariate statistics*. They are useful for answering questions about a single feature in tabular data. For example, the following dataset contains information about used cars listed on cardekho.com:

	name	year	selling_price	km_driven	fuel	transmission	owner	mileage	engine
0	Maruti Swift Dzire VDI	2014	450000	145500	Diesel	Manual	First Owner	23.4 kmpl	1248 CC
1	Skoda Rapid 1.5 TDI Ambition	2014	370000	120000	Diesel	Manual	Second Owner	21.14 kmpl	1498 CC
2	Honda City 2017-2020 EXi	2006	158000	140000	Petrol	Manual	Third Owner	17.7 kmpl	1497 CC
3	Hyundai i20 Sportz Diesel	2010	225000	127000	Diesel	Manual	First Owner	23.0 kmpl	1396 CC
4	Maruti Swift VXi BSIII	2007	130000	120000	Petrol	Manual	First Owner	16.1 kmpl	1298 CC

Univariate statistics can help us answer questions like:

- How much does a typical car cost?
- What proportion of cars have a manual transmission?
- How old is the oldest listed car?

Each of these questions focuses on a single variable (*selling_price*, *transmission*, and *year*, respectively, for the above examples). Depending on the type of variable, different summary statistics are appropriate.

Quantitative Variables

When summarizing quantitative variables, we often want to describe *central location* and *spread*.

Central Location

The central location (also called central tendency) is often used to communicate the “typical” value of a variable. Recall that there are a few different ways of calculating the central location:

- Mean: Also called the “average”; calculated as the sum of all values divided by the number of values.
- Median: The middle value of the variable when sorted.
- Mode: The most frequent value in the variable.
- Trimmed Mean: The mean excluding x percent of the lowest and highest data points.

Choosing an appropriate summary statistic for central tendency sometimes requires data visualization techniques along with domain knowledge. For example, suppose we want to know the typical price of a car in our dataset. If we calculate each of the statistics described above, we'll get the following estimates:

- Mean = Rs. 63827.18
- Median = Rs. 45000.00
- Mode = Rs. 30000.00
- Trimmed Mean = Rs. 47333.61

Because the mean is so much larger than the median and trimmed mean, we might guess that there are some outliers in this data with respect to price. We can investigate this by plotting a histogram of `selling_price`:

```
# Generate a histogram of the selling_price variable  
plt.hist(cars['selling_price'])  
plt.show()
```



Indeed, we see that `selling_price` is highly right-skewed. The very high prices (10 million Rupees for a small number of cars) are skewing the average upwards. By using the median or a trimmed mean, we can more accurately represent a “typical” price.

Spread

Spread, or dispersion, describes the variability within a feature. This is important because it provides context for measures of central location. For example, if there is a lot of variability in car prices, we can be less certain that any particular car will be close to 450000.00 Rupees (the median price). Like the central location measures, there are a few values that can describe the spread:

- Range: The difference between the maximum and minimum values in a variable.
- Inter-Quartile Range (IQR): The difference between the 75th and 25th percentile values.
- Variance: The average of the squared distance from each data point to the mean.
- Standard Deviation (SD): The square root of the variance.
- Mean Absolute Deviation (MAD): The mean absolute value of the distance between each data point and the mean.

Choosing the most appropriate measure of spread is much like choosing a measure of central tendency, in that we need to consider the data holistically. For example, below are measures of spread calculated for `selling_price`:

- Range: Rs. 9970001
- IQR: Rs. 420001
- Variance: 650044550668.61 (Rs²)
- Standard Deviation: Rs. 806253.40
- Mean Absolute Deviation: Rs. 42,213.14

We see that the range is almost 10 million Rupees; however, this could be due to a single 10 million Rupee car in the dataset. If we remove that one car, the range might be much smaller. The IQR is useful in comparison because it trims away outliers.

Meanwhile, we see that variance is extremely large. This happens because variance is calculated using squared differences, and is therefore not in the same units as the original data, making it less interpretable. Both the standard deviation and MAD solve this issue, but MAD is even less impacted by extreme outliers.

For highly skewed data or data with extreme outliers, we therefore might prefer to use IQR or MAD. For data that is more normally distributed, the variance and standard deviation are frequently reported.

Categorical Variables

Categorical variables can be either ordinal (ordered) or nominal (unordered). For ordinal categorical variables, we may still want to summarize central location and spread. However, because ordinal categories are not necessarily evenly spaced (like numbers), we should NOT calculate the mean of an ordinal categorical variable (or anything that relies on the mean, like variance, standard deviation, and MAD).

For nominal categorical variables (and ordinal categorical variables), another common numerical summary statistic is the frequency or proportion of observations in each category. This is often reported using a frequency table and can be visualized using a bar plot.

For example, suppose we want to know what kind of fuel listed cars tend to use. We could calculate the frequency of each fuel type:

```
cars.fuel.value_counts()
```

Output:

```
Diesel      2153
Petrol      2123
CNG          40
LPG          23
Electric      1
Name: fuel, dtype: int64
```

This tells us that 'Diesel' cars are most common, with 'Petrol' cars a close second. Converting these frequencies to proportions can also help us compare fuel types more easily. For example, the following table of proportions indicates that 'Diesel' cars account for almost half of all listings.

```
cars.fuel.value_counts(normalize=True)
```

Output:

```
Diesel      0.496083
Petrol      0.489171
CNG          0.009217
LPG          0.005300
```

```
Electric      0.000230  
Name: fuel, dtype: float64
```

Bivariate Statistics

In contrast to univariate statistics, bivariate statistics are used to summarize the relationship between two variables. They are useful for answering questions like:

- Do manual transmission cars tend to cost more or less than automatic transmission?
- Do older cars tend to cost less money?
- Are automatic transmission cars more likely to be sold by individuals or dealers?

Depending on the types of variables we want to summarize a relationship between, we should choose different summary statistics.

One Quantitative Variable and One Categorical Variable

If we want to know whether manual transmission cars tend to cost more or less than automatic transmission cars, we are interested in the relationship between `transmission` (categorical) and `selling_price` (quantitative). To answer this question, we can use a mean or median difference.

For example, we could calculate that the median price of automatic transmission cars is 100000 Rupees higher than for manual transmission cars.

Two Quantitative Variables

If we want to know whether older cars tend to cost less money, we are interested in the relationship between `year` and `selling_price`, both of which are quantitative. To answer this question, we can use the Pearson correlation.

For example, if we calculate that the correlation between `year` and `selling_price` is 0.4, we can conclude that there is a moderate positive association between these variables (older cars do tend to cost less money).

Two Categorical Variables

If we want to know whether automatic transmission cars are more likely to be sold by individuals or dealers, we are interested in the relationship between `transmission` and `seller_type`, both of which are categorical. We can explore this relationship using a contingency table and the Chi-Square statistic.

For example, based on the following contingency table, we might conclude that, although dealers sell more manual cars overall, the proportion of automatic cars among dealer sales is higher than the proportion of automatic cars among individual sales:

seller_type	Dealer	Individual	Trustmark Dealer
transmission			
Automatic	217	212	19
Manual	777	3032	83

Conclusion

In this article, we've summarized some of the important considerations for choosing a summary statistic based on the question a data analyst wants to answer and the type of data that is available. When it comes to choosing summary statistics, there's no one right answer, but exploring data holistically and systematically is an important component of EDA.

Central Tendency for Quantitative Data

For quantitative

Preview: Docs Loading link description

[variables](#)

, we often want to describe the *central tendency*, or the “typical” value of a variable. For example, what is the typical cost of rent in New York City?

There are several common measures of central tendency:

Mean: The average value of the variable, calculated as the sum of all values divided by the number of values.

Median: The middle value of the variable when sorted.

Mode: The most frequent value of the variable.

Trimmed mean: The mean excluding x percent of the lowest and highest data points.

For our `rentals` DataFrame with a column named `rent` that contains rental prices, we can calculate the central tendency statistics listed above as follows:

```
# Mean
rentals.rent.mean()

# Median
rentals.rent.median()

# Mode
rentals.rent.mode()

# Trimmed mean
from scipy.stats import trim_mean
trim_mean(rentals.rent, proportiontocut=0.1) # trim extreme 10%
```

Spread for Quantitative Data

The *spread* of a quantitative variable describes the amount of variability. This is important because it provides context for measures of central tendency. For example, if there is a lot of variability in New York City rent prices, we can be less certain that the mean or median price is representative of what the typical rent is.

There are several common measures of spread:

Range: The difference between the maximum and minimum values of a variable.

Interquartile range (IQR): The difference between the 75th and 25th percentile values.

Variance: The average of the squared distance from each data point to the mean.

Standard deviation (SD): The square root of the variance.

Mean absolute deviation (MAD): The mean absolute value of the distance between each data point and the mean.

For our `rentals` DataFrame, we can calculate the spread for the `rent` column as follows:

```
# Range
rentals.rent.max() - rentals.rent.min()

# Interquartile range
rentals.rent.quantile(0.75) - rentals.rent.quantile(0.25)

from scipy.stats import iqr
iqr(rentals.rent) # alternative way

# Variance
rentals.rent.var()

# Standard deviation
rentals.rent.std()

# Mean absolute deviation
rentals.rent.mad()
```

Visualizing Quantitative Variables

While summary statistics are certainly helpful for exploring and quantifying a feature, we might find it hard to wrap our minds around a bunch of numbers. This is why data visualization is such a powerful element of EDA.

For quantitative

Preview: Docs Loading link description

[variables](#)

, *boxplots* and *histograms* are two common visualizations. These plots are useful because they simultaneously communicate information about minimum and maximum values, central location,

and spread. Histograms can additionally illuminate patterns that can impact an analysis (e.g., skew or multimodality).

Python's `seaborn` library, built on top of `matplotlib`, offers the `boxplot()` and `histplot()`

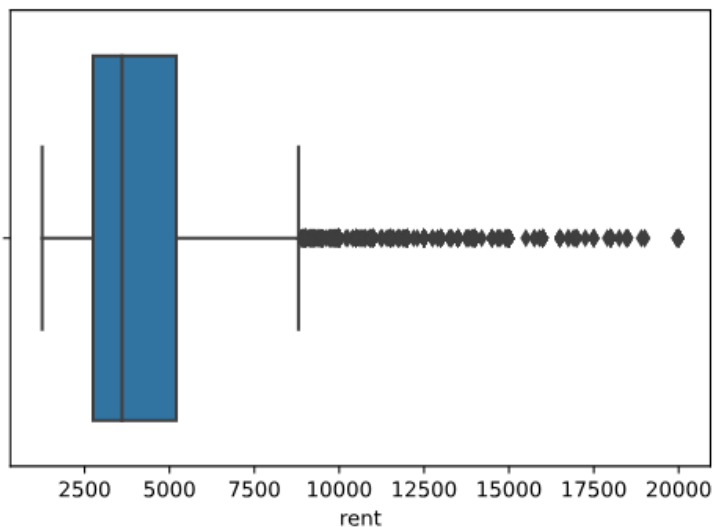
Preview: Docs Loading link description

[functions](#)

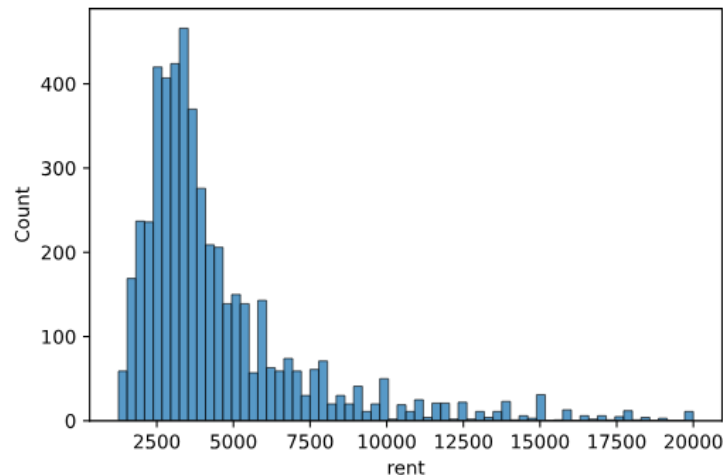
to easily plot data from a `pandas` DataFrame:

```
import matplotlib.pyplot as plt
import seaborn as sns

# Boxplot for rent
sns.boxplot(x='rent', data=rentals)
plt.show()
plt.close()
```



```
# Histogram for rent
sns.histplot(x='rent', data=rentals)
plt.show()
plt.close()
```



Value Counts for Categorical Data

When it comes to categorical

Preview: Docs Loading link description

[variables](#)

, the measures of central tendency and spread that worked for describing numeric variables, like mean and standard deviation, generally becomes unsuitable when we're dealing with discrete values. Unlike numbers, categorical values are not continuous and oftentimes do not have an intrinsic ordering.

Instead, a good way to summarize categorical variables is to generate a frequency table containing the count of each distinct value. For example, we may be interested to know how many of the New York City rental listings are from each borough. Related, we can also find which borough has the most listings.

The `pandas` library offers the `.value_counts()` method for generating the counts of all values in a DataFrame column:

```
# Counts of rental listings in each borough
df.borough.value_counts()
```

Value Proportions for Categorical Data

A counts table is one approach for exploring categorical

Preview: Docs Loading link description

[variables](#)

, but sometimes it is useful to also look at the proportion of values in each category. For example, knowing that there are 3,539 rental listings in Manhattan is hard to interpret without any context about the counts in the other categories. On the other hand, knowing that Manhattan listings make up 71% of all New York City listings tells us a lot more about the relative frequency of this category. We can calculate the proportion for each category by dividing its count by the total number of values for that variable:

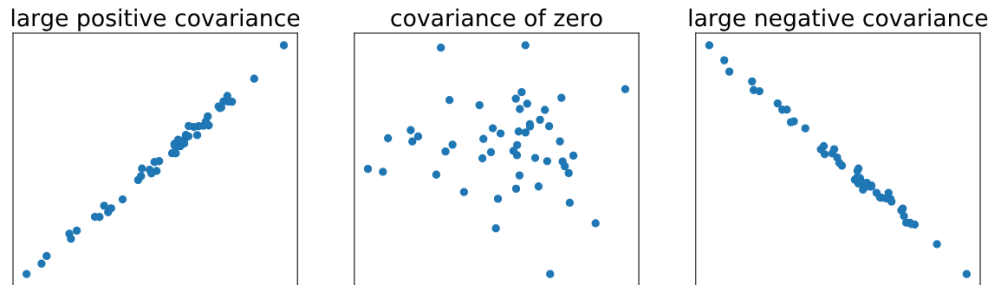
```
# Proportions of rental listings in each borough
rentals.borough.value_counts() / len(rentals.borough)
```



Exploring Covariance

Beyond visualizing relationships, we can also use summary statistics to quantify the strength of certain associations. *Covariance* is a summary statistic that describes the strength of a linear relationship. A linear relationship is one where a straight line would best describe the pattern of points in a scatter plot.

Covariance can range from negative infinity to positive infinity. A positive covariance indicates that a larger value of one variable is associated with a **larger** value of the other. A negative covariance indicates a larger value of one variable is associated with a **smaller** value of the other. A covariance of 0 indicates no linear relationship. Here are some examples:



To calculate covariance, we can use the `cov()` function from NumPy, which produces a covariance matrix for two or more

Preview: Docs Loading link description

[variables](#)

. A covariance matrix for two variables looks something like this:

Correlation- Part 1

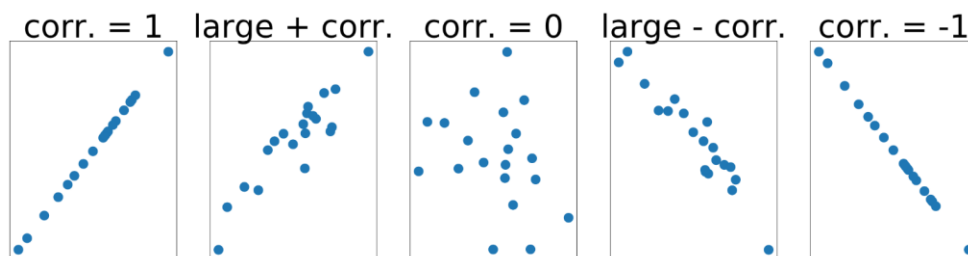
Like covariance, *Pearson Correlation* (often referred to simply as “correlation”) is a scaled form of covariance. It also measures the strength of a linear relationship, but ranges from -1 to +1, making it more interpretable.

Highly associated

Preview: Docs Loading link description

[variables](#)

with a positive linear relationship will have a correlation close to 1. Highly associated variables with a negative linear relationship will have a correlation close to -1. Variables that do not have a linear association (or a linear association with a slope of zero) will have correlations close to 0.



The `pearsonr()` function from `scipy.stats` can be used to calculate correlation as follows:

Correlation Part 2

It's important to note that there are some limitations to using correlation or covariance as a way of assessing whether there is an association between two

Preview: Docs Loading link description

[variables](#)

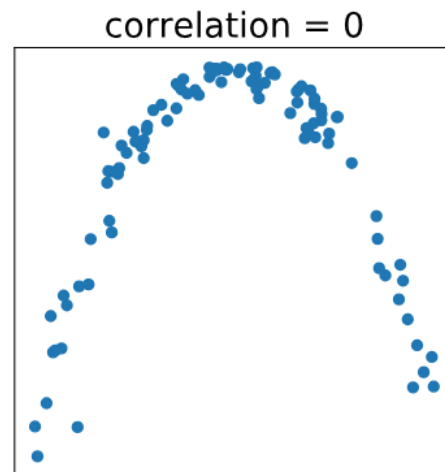
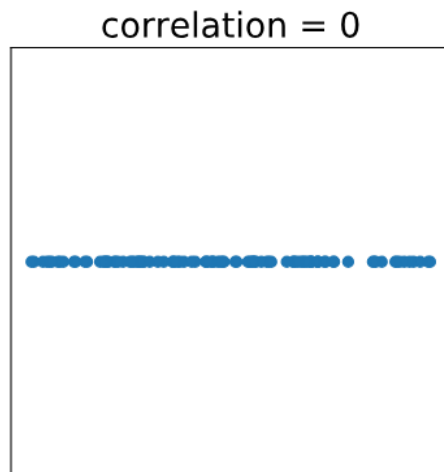
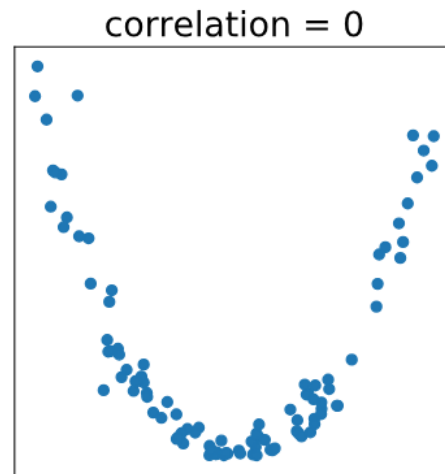
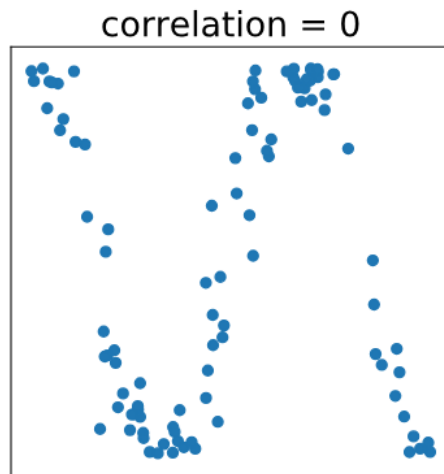
. Because correlation and covariance both measure the strength of **linear** relationships with non-zero slopes, but not other kinds of relationships, correlation can be misleading.

For example, the four scatter plots below all show pairs of variables with near-zero correlations. The bottom left image shows an example of a perfect linear association where the slope is zero (the line is horizontal). Meanwhile, the other three plots show non-linear relationships — if we drew a line through any of these

Preview: Docs Loading link description

[sets](#)

of points, that line would need to be curved, not straight!



Contingency Tables: Frequencies

Contingency tables, also known as two-way tables or cross-tabulations, are useful for summarizing two

Preview: Docs Loading link description

[variables](#)

at the same time. For example, suppose we are interested in understanding whether there is an association between `influence` (whether a person thinks they have a talent for influencing people) and `leader` (whether they see themselves as a leader). We can use the `crosstab` function from pandas to create a contingency table:

```
influence_leader_freq = pd.crosstab(npi.influence, npi.leader)
print(influence_leader_freq)
```

```
npi = pd.read_csv("npi_sample.csv")

# save the table of frequencies as special_authority_freq:
special_authority_freq = pd.crosstab(npi.special, npi.authority)

# save the table of proportions as special_authority_prop:
special_authority_prop = special_authority_freq/len(npi)
```

Expected Contingency Tables

In the previous exercise we calculated the following marginal proportions for the `leader` and `influence` questions:

leader		influence	
no	0.484	no	0.388
yes	0.516	yes	0.612

In order to understand whether these questions are associated, we can use the marginal proportions to create a contingency table of *expected proportions* if there were **no association** between these

Preview: Docs A variable is used to store data that will be used by the program. This data can be a number, a string, a Boolean, a list or some other data type. Every variable has a name which

can consist of letters, numbers, and the underscore character . The equal sign = is used to assign a value to a variable. After the initial assignment is made, the value of a variable can be updated to new values as needed. A variable can have a short name (like x and y) or a more descriptive name (age, grade, grocerylist). Rules for Python variables: - A variable name must start with a letter or the underscore character. It cannot start with a number. - A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _). - Variable names are case-sensitive (num, Num, and NUM are three different variables).

[variables](#). To calculate these expected proportions, we need to multiply the marginal proportions for each combination of categories:

	leader = no	leader = yes
influence = no	$0.484 \times 0.388 = 0.188$	$0.516 \times 0.388 = .200$
influence = yes	$0.484 \times 0.612 = 0.296$	$0.516 \times 0.612 = 0.315$

These proportions can then be converted to frequencies by multiplying each one by the sample size (11097 for this data):

	leader = no	leader = yes
influence = no	$0.188 \times 11097 = 2087$	$0.200 \times 11097 = 2221$
influence = yes	$0.296 \times 11097 = 3288$	$0.315 \times 11097 = 3501$

This table tells us that **if** there were no association between the `leader` and `influence` questions, we would expect 2087 people to answer `no` to both.

In python, we can calculate this table using the `chi2_contingency()` function from SciPy, by passing in the observed frequency table. There are actually four outputs from this function, but for now, we'll only look at the fourth one:

```
from scipy.stats import chi2_contingency

chi2, pval, dof, expected = chi2_contingency(influence_leader_freq)

print(np.round(expected))
```

Copy to Clipboard

Output:

```
[[2087. 2221.]
```

```
 [3288. 3501.]]
```

Note that the ScyPy function returned the same expected frequencies as we calculated “by hand” above! Now that we have the expected contingency table if there’s no association, we can compare it to our observed contingency table:

leader	no	yes
influence		
no	3015	1293
yes	2360	4429

The more that the expected and observed tables differ, the more sure we can be that the variables are associated. In this example, we see some pretty big differences (eg., 3015 in the observed table compared to 2087 in the expected table). This provides additional evidence that these variables are associated.

The Chi-Square Statistic

In the previous exercise, we calculated a contingency table of expected frequencies **if** there were no association between the `leader` and `influence` questions. We then compared this to

the observed contingency table. Because the tables looked somewhat different, we concluded that responses to these questions are probably associated.

While we can inspect these tables visually, many data scientists use the *Chi-Square statistic* to summarize **how** different these two tables are. To calculate the Chi Square statistic, we simply find the squared difference between each value in the observed table and its corresponding value in the expected table, and then divide that number by the value from the expected table; finally add up those numbers:

$$\text{ChiSquare} = \sum ((\text{observed} - \text{expected})^2 / \text{expected})$$

The Chi-Square statistic is also the first output of the SciPy function `chi2_contingency()`:

```
from scipy.stats import chi2_contingency

chi2, pval, dof, expected = chi2_contingency(influence_leader_freq)

print(chi2)

output: 1307.88
```

The interpretation of the Chi-Square statistic is dependent on the size of the contingency table. For a 2x2 table (like the one we've been investigating), a Chi-Square statistic larger than around 4 would strongly suggest an association between the

Preview: Docs Loading link description

[variables](#)

. In this example, our Chi-Square statistic is much larger than that — 1307.88! This adds to our evidence that the variables are highly associated.

Probability

Find out what you'll learn in Probability and why it's important.

Welcome to Probability

In this unit, we will cover the fundamentals of probability. This is useful to quantify uncertainty and describe random events and outcomes.

We'll cover ways of calculating probabilities using formulas along with Python functions, how to visualize these random outcomes on a distribution, and how to represent a population using sample data.

Why is this important?

Probability allows us to measure uncertainty. This is essential in many different industries, such as meteorology, medicine, sports, insurance, and countless more.

When you check the weather on your phone or computer, you are using probability and intuition to gauge what you will wear that day. Doctors use genetics and medical history to gauge certain risk factors a patient might encounter by analyzing how different events affect the probability of certain outcomes.

Knowing how to calculate probabilities based on specific events is essential for data scientists, statisticians, and almost any STEM field!

What will you learn?

At the end of this module, you will be able to:

- Define probability using the law of large numbers
- Quantify uncertain events using:
 - The addition rule
 - The product rule
 - Bayes' Theorem
 - Tree Diagrams
- Describe random events using probability distributions
- Understand use-cases for the following probability distributions:
 - Binomial Distribution
 - Poisson Distribution
- Describe a population using a sampling distribution and the central limit theorem

Probability, Set Theory, and the Law of Large Numbers

Learn about what probability is and the language we use to represent it!

Probability

Probability is a branch of mathematics that allows us to quantify uncertainty. In our daily lives, we often use probability to make decisions, even without thinking about it!

For example, many weather reports give a percent chance that it will rain. If we hear that there is an 80 percent chance of rain, we probably are not going to make many plans outside. However, if there is only a 5 percent chance of rain, we may feel comfortable planning a picnic.

In this article, we are going to build a foundation for understanding probability. To do this, we are going to explore a field of mathematics called *set theory*.

Set Theory

Set theory is a branch of mathematics based around the concept of *sets*. In simple terms, a set is a collection of things. For example, we can use a set to represent items in a backpack. We might have:

$$\{\text{Book, Paper, Folder, Hat, Pen, Snack}\}$$
$$\{\textit{Book, Paper, Folder, Hat, Pen, Snack}\}$$

Notationally, mathematicians often represent sets with curly braces. Sets also follow two key rules:

Therefore, we can say:

$$\{1,2,3,4,5\}=\{5,3,2,4,1\}$$
$$\{1,2,3,4,5\}=\{5,3,2,4,1\}$$

When defining a set, we often use a capital letter. For example:

$$A=\{1,2,3,4,5\}$$
$$A=\{1,2,3,4,5\}$$

Sets can also contain *subsets*. Set *A* is a subset of set *B* if all the elements in *A* exist within *B*. For example:

$$A=\{1,2,3\} B=\{1,2,3,4,5\}$$
$$A=\{1,2,3\}$$

Here, set *A* is a subset of *B* because all elements of *A* are contained within *B*.

Experiments and Sample Spaces

In probability, an *experiment* is something that produces observation(s) with some level of uncertainty. A *sample point* is a single possible outcome of an experiment. Finally, a *sample space* is the set of all possible sample points for an experiment.

For example, suppose that we run an experiment where we flip a coin twice and record whether each flip results in heads or tails. There are four sample points in this experiment: two heads (HH), tails and then heads (TH), heads and then tails (HT), or two tails (TT). We can write the full sample space for this experiment as follows:

$$S = \{HH, TT, HT, TH\}$$

$$S = \{HH, TT, HT, TH\}$$

Suppose we are interested in the probability of one specific outcome: HH . A specific outcome (or set of outcomes) is known as an *event* and is a subset of the sample space. Three events we might look at in this sample space are:

Getting Two Heads $A = \{HH\}$ Getting Two Tails $B = \{TT\}$ Getting Both a Heads and Tails $C = \{HT, TH\}$

Getting Two Heads

The frequentist definition of *probability* is as follows: If we run an experiment an infinite amount of times, the probability of each event is the proportion of times it occurs. Unfortunately, we don't have the ability to flip two coins an infinite amount of times – but we can estimate probabilities by choosing some other large number, such as 1000. Let's give it a try!

Okay, we have flipped two coins 1000 times. Wasn't that FUN? Here are each of the outcomes and the number of times we observed each one:

To calculate the estimated probability of any one outcome, we use the following formula:

$$P(\text{Event}) = \frac{\text{Number of Times Event Occurred}}{\text{Total Number of Trials}}$$

$$P(\text{Event}) =$$

$$\frac{\text{Number of Times Event Occurred}}{\text{Total Number of Trials}}$$

Number of Times Event Occurred

In this scenario, a *trial* is a single run of our experiment (two coin flips). So, the probability of two heads on two coin flips is approximately:

$$P(HH) = \frac{252}{1000} = .252$$

$$P(HH) =$$

$$\frac{252}{1000}$$

$$= .252$$

$$= .252$$

Based on these 1000 trials, we would estimate there is a 25.2 percent chance of getting two heads on two coin flips. This is great! However, if we do this same procedure over and over again, we may get slightly different results. For example, if we repeat the experiment another 1000 times, we might get two heads only 24.2 percent of the time.

If we want to feel confident that we are close to the true probability of a particular event, we can leverage the *law of large numbers*.

Law of Large Numbers

We can't repeat our random experiment an infinite amount of times (as much FUN as that would be!). However, we can still flip both coins a large number of times. As we flip both coins more and more, the observed proportion of times each event occurs will converge to its true probability. This is called the *law of large numbers*.

Let's observe the law of large numbers in real-time. We will use Python to simulate flipping both coins as many times as we want and watch the proportion of two heads converge to its true probability.

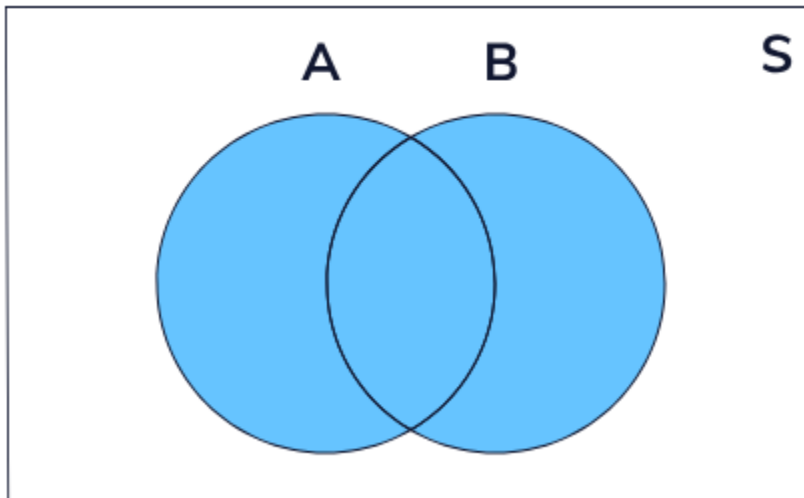
Let's walk through each part of the code below one step at a time. You do not need to worry about every line of code, but understanding the overall objective will help you build your understanding of probability

Union, Intersection, and Complement

Let's dive into some key concepts we will use throughout this lesson: *union*, *intersection*, and *complement*.

Union

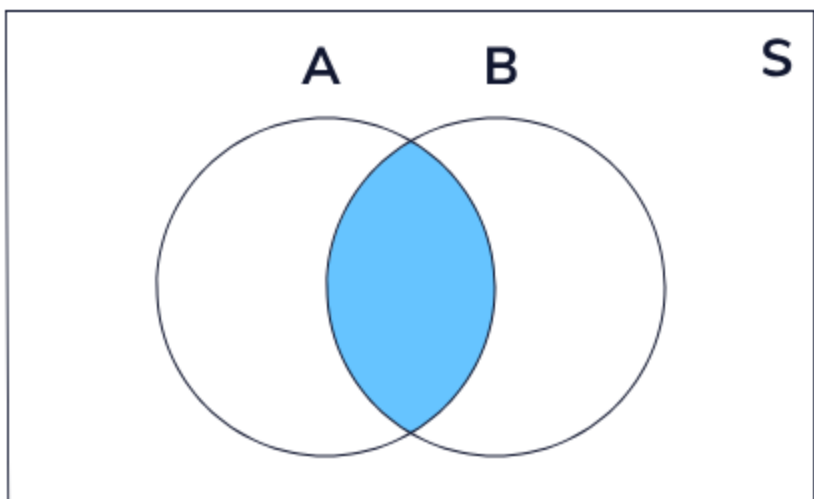
The *union* of two sets encompasses any element that exists in either one or both of them. We can represent this visually as a *Venn diagram*.



For example, let's say we have two sets, *A* and *B*. *A* represents rolling an odd number with a six-sided die (the set $\{1, 3, 5\}$). *B* represents rolling a number greater than two (the set $\{3, 4, 5, 6\}$). The union of these two sets would be everything in either set *A*, set *B*, or both: $\{1, 3, 4, 5, 6\}$. We can write the union of two events mathematically as $(A \text{ or } B)$.

Intersection

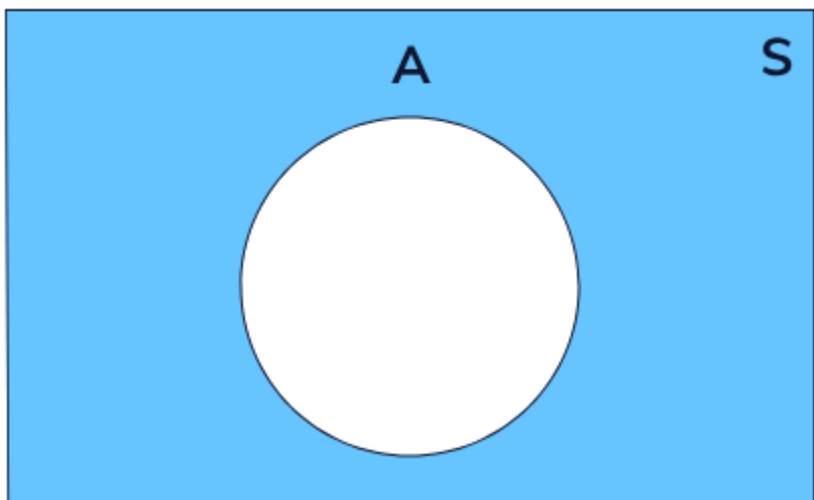
The *intersection* of two sets encompasses any element that exists in both of the sets. Visually:



The intersection of the above sets (A represents rolling an odd number on a six-sided die and B represents rolling a number greater than two) includes any value that appears in both sets: $\{3, 5\}$. We can write the intersection of two events mathematically as (A and B).

Complement

Lastly, the *complement* of a set consists of all possible outcomes outside of the set. Visually:



Consider set A from the above example (rolling an odd number on a 6-sided die). The complement of this set would be rolling an even number: $\{2, 4, 6\}$. We can write the complement of set A as A^c . One key feature of complements is that a set and its complement cover the entire sample space. In this

die roll example, the set of even numbers and odd numbers would cover all possible rolls: $\{1, 2, 3, 4, 5, 6\}$.

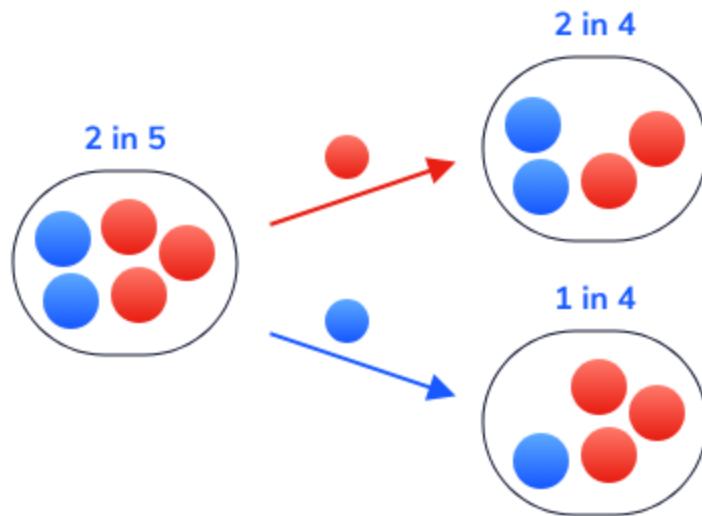
Instructions

1. To the right is an applet that shows two overlapping circles: A and B . The two dropdown menus allow you to choose from different events. Event A can be any of the following:
 - The outcome of a die roll is even
 - The outcome of a die roll is a five
 - The outcome of a die roll is less than or equal to three
2. Event B can be any of the following:
 - The outcome of a die roll is odd
 - The outcome of a die roll is anything but five
 - The outcome of a die roll is greater than or equal to two
3. Using the applet, find the intersection and union between the following two events:
 - The outcome of a die roll is less than or equal to three
 - The outcome of a die roll is anything but five
4. Using the applet, find the complement of the following events:
 - The outcome of a die roll is greater than or equal to two
 - The outcome of a die roll is five

Independence and Dependence

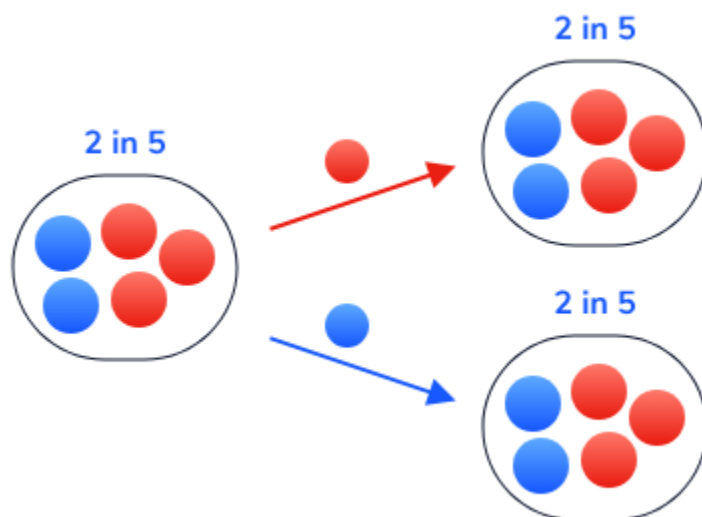
Imagine that we flip a fair coin 5 times and get 5 heads in a row. Does this affect the probability of getting heads on the next flip? Even though we may feel like it's time to see "tails", it is impossible for a past coin flip to impact a future one. The fact that previous coin flips do not affect future ones is called *independence*. Two events are *independent* if the occurrence of one event does not affect the probability of the other.

Are there cases where previous events DO affect the outcome of the next event? Suppose we have a bag of five marbles: two marbles are blue and three marbles are red. If we take one marble out of the bag, what is the probability that the second marble we take out is blue?



This describes two events that are *dependent*. The probability of grabbing a blue marble in the second event *depends* on whether we take out a red or a blue marble in the first event.

What if we had put back the first marble? Is the probability that we pick a blue marble second independent or dependent on what we pick out first? In this case, the events would be independent.



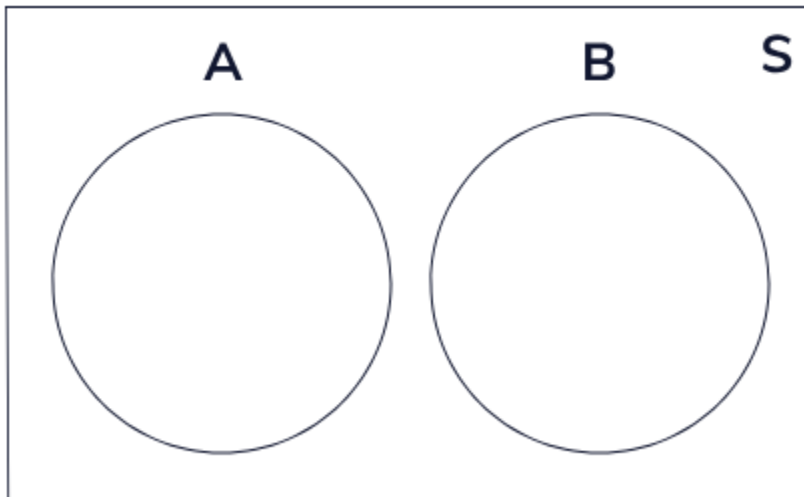
Why do we care if events are independent or dependent? Knowing this helps us quantify the probability of events that depend on preexisting knowledge. This helps researchers understand and predict complex processes such as:

- Effectiveness of vaccines
- The weather on a particular day
- Betting odds for professional sports games

Mutually Exclusive Events

Two events are considered *mutually exclusive* if they cannot occur at the same time. For example, consider a single coin flip: the events “tails” and “heads” are mutually exclusive because we cannot get both tails and heads on a single flip.

We can visualize two mutually exclusive events as a pair of non-overlapping circles. They do not overlap because there is no outcome for one event that is also in the sample space for the other:



Addition Rule

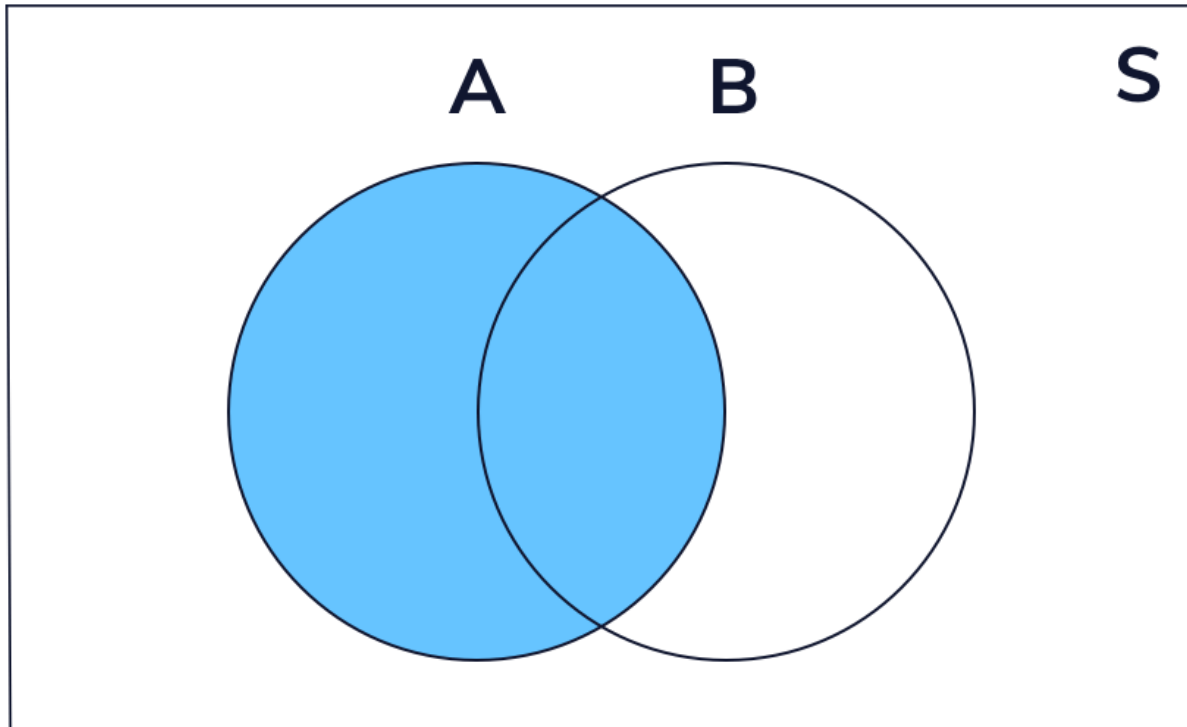
Now, it's time to apply these concepts to calculate probabilities.

Let's go back to one of our first examples: event A is rolling an odd number on a six-sided die and event B is rolling a number greater than two. What if we want to find the probability of one or both events occurring? This is the probability of the union of A and B:

$$P(A \text{ or } B)$$

$$P(A \text{ or } B)$$

We can visualize this calculation as follows:



$$P(A \text{ or } B) = P(A)$$

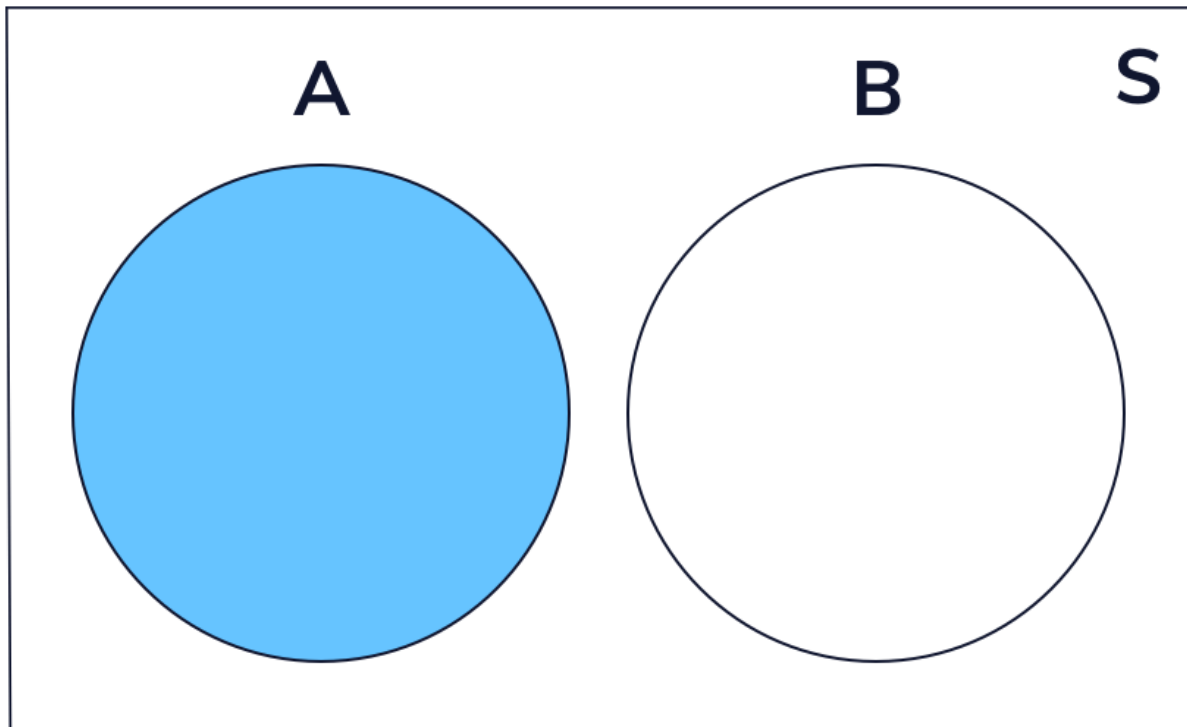
This animation gives a visual representation of the addition rule formula, which is:

$$P(A \text{ or } B) = P(A) + P(B) - P(A \text{ and } B)$$

$$P(A \text{ or } B) = P(A) + P(B) - P(A \text{ and } B)$$

We subtract the intersection of events A and B because it is included twice in the addition of $P(A)$ and $P(B)$.

What if the events are mutually exclusive? On a single die roll, if event A is that the roll is less than or equal to 2 and event B is that the roll is greater than or equal to 5, then events A and B cannot both happen.



$$P(A \text{ or } B) = P(A)$$

For mutually exclusive events, the addition rule formula is:

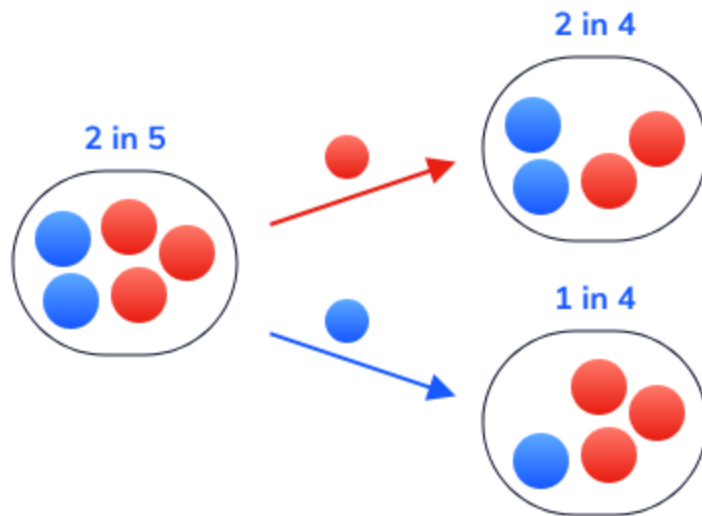
$$P(A \text{ or } B) = P(A) + P(B)$$

$$P(A \text{ or } B) = P(A) + P(B)$$

This is because the intersection is empty, so we don't need to remove any overlap between the two events.

Conditional Probability

If we want to calculate the probability that a pair of dependent events both occur, we need to define conditional probability. Using a bag of marbles as an example, let's remind ourselves of the definition of dependent events:



If we pick two marbles from a bag of five marbles without replacement, the probability that the second marble is red depends on the color of the first marble. We have a special name for this: *conditional probability*. In short, conditional probability measures the probability of one event occurring, given that another one has already occurred.

Notationally, we denote the word “given” with a vertical line. For example, if we want to represent the probability that we choose a red marble given the first marble is blue, we can write:

$$P(\text{Red Second} \mid \text{Blue First})$$

$$P(\text{Red Second} \mid \text{Blue First})$$

From the above diagram, we know that:

$$P(\text{Red Second} \mid \text{Blue First}) = \frac{3}{4}$$

$$P(\text{Red Second} \mid \text{Blue First}) =$$

4

3

What if we picked out two marbles with replacement? What does the conditional probability look like? Well, let’s think about this. Regardless of which marble we pick out first, it will be put back into the bag. Therefore, the probability of picking out a red marble or a blue marble second is unaffected by the first outcome.

Therefore, for independent events, we can say the following:

$$P(A|B) = P(A) \text{ and } P(B|A) = P(B)$$

$$P(A|B) = P(A)$$

and

$$P(B|A) = P(B)$$

Multiplication Rule

We have looked at the addition rule, which describes the probability one event OR another event (or both) occurs. What if we want to calculate the probability that two events happen simultaneously? For two events, A and B , this is $P(A \text{ and } B)$ or the probability of the intersection of A and B .

General Formula

The general formula for the probability that two events occur simultaneously is:

$$P(A \text{ and } B) = P(A) \cdot P(B|A)$$

$$P(A \text{ and } B) = P(A) \cdot P(B|A)$$

However, for independent events, we can simplify this formula slightly.

Dependent Events

Let's go back to our bag of marbles example. We have five marbles: two are blue, and three are red. We pick two marbles without replacement. What if we want to know the probability of choosing a blue marble first AND a blue marble second?

Taking conditional probability into account, the multiplication rule for these two dependent events is:

$$P(\text{Blue 1st and Blue 2nd}) = P(\text{Blue 1st}) \cdot P(\text{Blue 2nd} | \text{Blue 1st})$$
$$P(\text{Blue 1st and Blue 2nd}) = \frac{2}{5} \cdot \frac{1}{4} = \frac{2}{20} = \frac{1}{10}$$

(

This is one potential outcome when picking two marbles out of the bag. One way to visualize all possible outcomes of a pair of events is a *tree diagram*.

Tree diagrams have the following properties:

Each branch represents a specific set of events.

The probabilities the terminal branches (all possible sets of outcomes) sum to one.

We multiply across branches (using the multiplication rule!) to calculate the probability that each branch (set of outcomes) will occur.

In the browser to the right, you will be able to play with one!

Independent Events

For two independent events, the multiplication rule becomes less complicated. The probability of two independent events occurring is:

$$P(A \text{ and } B) = P(A) \cdot P(B)$$

$$P(A \text{ and } B) = P(A) \cdot P(B)$$

This is because the following is true for independent events:

$$P(B | A) = P(B)$$

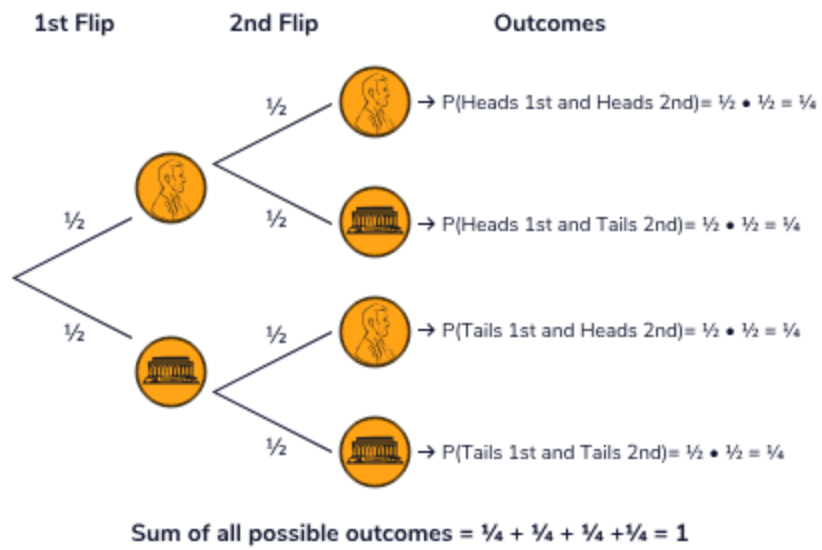
$$P(B | A) = P(B)$$

Let's look at the simplest example: flipping a fair coin twice. Event A is that we get tails on the first flip, and event B is that we get tails on the second flip. $P(A) = P(B) = 0.5$, so according to our formula, the probability of getting tails on both flips would be:

$$P(A \text{ and } B) = 0.5 \cdot 0.5 = 0.25$$

$$P(A \text{ and } B) = 0.5 \cdot 0.5 = 0.25$$

Visually on a tree diagram, we see:



Open the diagram if in a new window [here](#) if you would like to zoom in for a better view.

20 percent of the population has strep throat.

80 percent of the population does not have strep throat.

Now suppose that we test a bunch of people for strep throat. The possible results of these tests are shown in the next set of branches:

If a person has strep throat, there is an 85% chance their test will be positive and a 15% chance it will be negative. This is labeled as:

$$P(+|ST)=0.85 \text{ and } P(-|ST)=0.15$$

(

If a person does not have strep throat, there is a 98% chance their test will be negative and a 2% chance it will be positive. This can be labeled as:

$$P(-|NO\ ST)=0.98 \text{ and } P(+|NO\ ST)=0.02$$

(

Finally, let's look at the four possible pairs of outcomes that form the terminal branches of our diagram:

$$P(ST \text{ and } +)=0.17 \quad P(ST \text{ and } -)=0.03 \quad P(NO\ ST \text{ and } +)=0.016 \quad P(NO\ ST \text{ and } -)=0.784$$

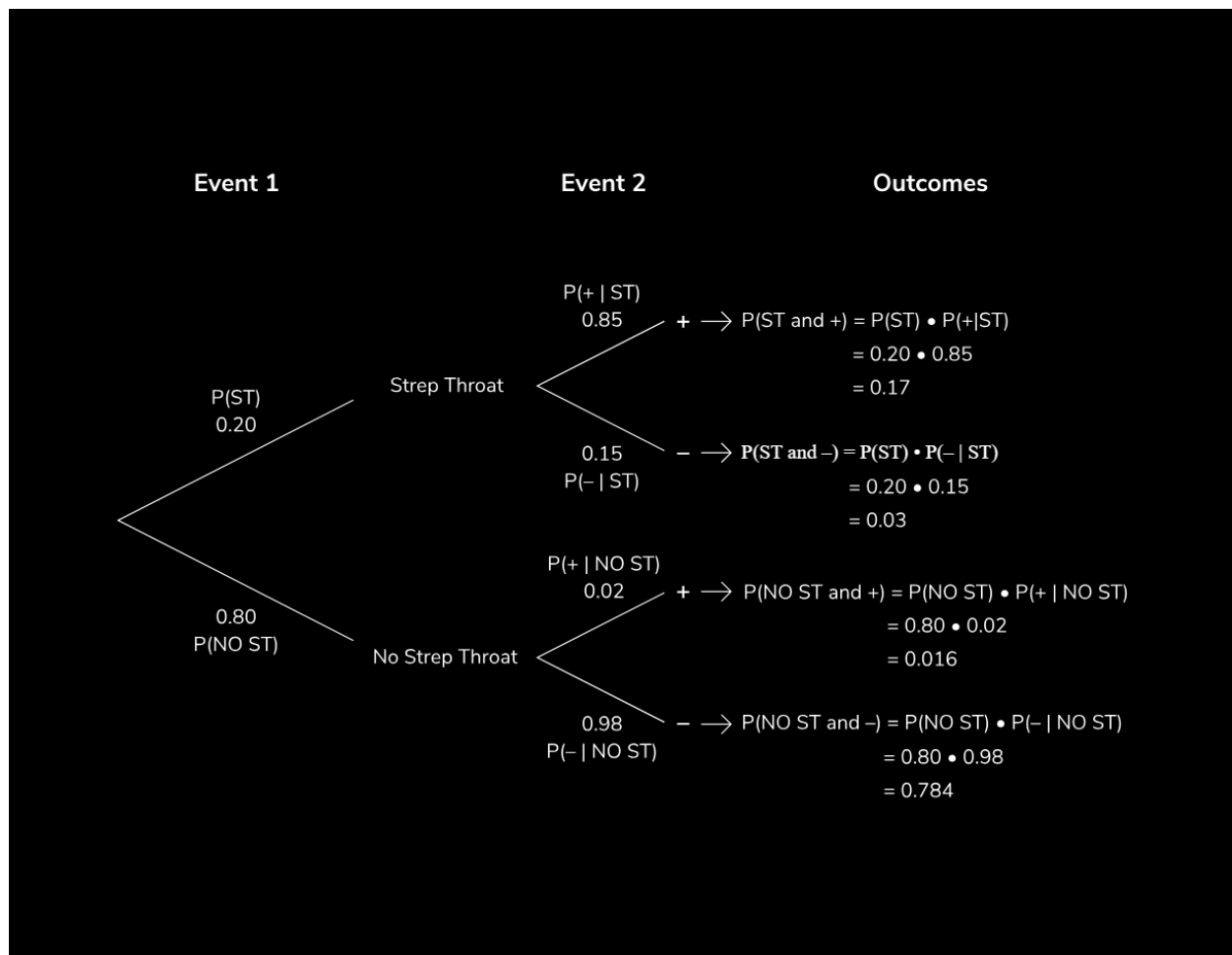
(

Together, these add up to one since they capture all potential outcomes after patients are tested.

It's great that we have all this information. However, we are missing something. If someone gets a positive result, what is the probability that they have strep throat? Notationally, we can write this probability as:

$$P(ST|+)$$

$$P(ST|+)$$



Bayes' Theorem

Imagine that you are a patient who has recently tested positive for strep throat. You may want to know the probability that you HAVE strep throat, given that you tested positive:

$$P(\text{ST} | +)$$

$$P(\text{ST} | +)$$

To calculate this probability, we will use something called *Bayes Theorem*, which states the following:

$$P(B | A) = P(A | B) \cdot P(B) / P(A)$$

$$P(B | A) =$$

$$P(A)$$

$$P(A|B) \cdot P(B)$$

Using Bayes' theorem:

$$P(ST|+) = P(+|ST) \cdot P(ST) / P(+)$$

$$P(ST|+) =$$

$$P(+)$$

$$P(+|ST) \cdot P(ST)$$

We know:

$$P(+|ST) = 0.85$$

$$P(+|ST) = 0.85$$

We also know:

$$P(ST) = 0.20$$

$$P(ST) = 0.20$$

What about $P(+)$? Is this something we know? Well, let's think about this. There are four possible outcomes:

Having strep throat and testing positive

Having strep throat and testing negative

Not having strep throat and testing positive

Not having strep throat and testing negative

We only care about the two outcomes where a patient tests positive for $P(+)$. Therefore, we can say:

$$P(+)=P(ST \text{ and } +)+P(NO ST \text{ and } +)P(+)=0.17+.016P(+)=0.186$$

(

Finally, if we plug all of these into the Bayes' theorem formula, we get:

$$P(ST|+) = 0.85 \cdot 0.200.186 = 0.914$$

$$P(ST|+) =$$

$$0.186$$

$$0.85 \cdot 0.20$$

$$= 0.914$$

There is a 91.4% chance that you actually have strep throat given you test positive. This is not obvious from the information outlined in our tree diagram, but with the power of Bayes theorem, we were able to calculate it!

$$P(ST| -) = ?$$

First, find the probability of a negative result:

$$P(-) + P(+) = 1$$

$$P(-) = 1 - P(+) = 0.814$$

Then, use the following formula:

$$P(ST | -) = \frac{P(- | ST) \cdot P(ST)}{P(-)}$$

Plugging in for known values, we have:

$$P(ST | -) = \frac{0.15 \cdot 0.20}{0.814} = 0.037$$

$$P(\text{NO ST} \mid +) = ?$$

Use the following formula:

$$P(\text{NO ST} \mid +) = \frac{P(+ \mid \text{NO ST}) \cdot P(\text{NO ST})}{P(+)}$$

Plugging in for known values, we have:

$$P(\text{NO ST} \mid +) = \frac{0.02 \cdot 0.80}{0.186} = 0.086$$

Alternative Method

Since the only two events given a patient is positive are ST or NO ST, we can say the following:

$$P(\text{NO ST} \mid +) + P(\text{ST} \mid +) = 1$$

$$P(\text{NO ST} \mid +) = 1 - P(\text{ST} \mid +)$$

$$P(\text{NO ST} \mid +) = 0.086$$

$$P(\text{NO ST} \mid -) = ?$$

Use the following formula:

$$P(\text{NO ST} \mid -) = \frac{P(- \mid \text{NO ST}) \cdot P(\text{NO ST})}{P(-)}$$

Plugging in for known values, we have:

$$P(\text{NO ST} \mid -) = \frac{0.98 \cdot 0.80}{0.814} = 0.963$$

PROBLEMS

The following probabilities map out the event of “Iced Coffee” vs. “No Iced Coffee” in the morning as well as “On-time” vs. “Late” to work given whether or not you got an iced coffee:

$$P(\text{Iced Coffee}) = 0.90$$

$$P(\text{No Iced Coffee}) = 0.10$$

$$P(\text{Late} \mid \text{Iced Coffee}) = 0.85$$

$$P(\text{On-time} \mid \text{Iced Coffee}) = 0.15$$

$$P(\text{Late} \mid \text{No Iced Coffee}) = 0.40$$

$$P(\text{On-time} \mid \text{No Iced Coffee}) = 0.60$$

$$P(\text{Late and Iced Coffee}) = 0.765$$

$$P(\text{On-time and Iced Coffee}) = 0.135$$

$$P(\text{Late and No Iced Coffee}) = 0.04$$

$$P(\text{On-time and No Iced Coffee}) = 0.06$$

The four final outcomes come out to:

Given all of this information, find $P(\text{Iced Coffee} \mid \text{Late})$: The probability that you had iced coffee in the morning given that you were late.

Hint: You won't need to use all the probabilities, and the answers are not in the simplified form!

Answer Choices

This probability cannot be quantified with the given information.

$0.85 \cdot 0.900.765+0.04$

$0.765+0.04$

$0.85 \cdot 0.90$

-

$0.85 \cdot 0.900.765$

0.765

$0.85 \cdot 0.90$

$0.90 \cdot 0.7650.85+0.04$

$0.85+0.04$

$0.90 \cdot 0.765$

Random Variables

A *random variable* is, in its simplest form, a function. In probability, we often use random variables to represent random events. For example, we could use a random variable to represent the outcome of a die roll: any number between one and six.

Random variables must be numeric, meaning they always take on a number rather than a characteristic or quality. If we want to use a random variable to represent an event with non-numeric outcomes, we can choose numbers to represent those outcomes. For example, we could represent a coin flip as a random variable by assigning “heads” a value of 1 and “tails” a value of 0.

In this lesson, we will use `random.choice(a, size = size, replace = True/False)` from the `numpy` library to simulate random variables in python. In this method:

`a` is a list or other object that has values we are sampling from

`size` is a number that represents how many values to choose

`replace` can be equal to `True` or `False`, and determines whether we keep a value in `a` after drawing it (`replace = True`) or remove it from the pool (`replace = False`).

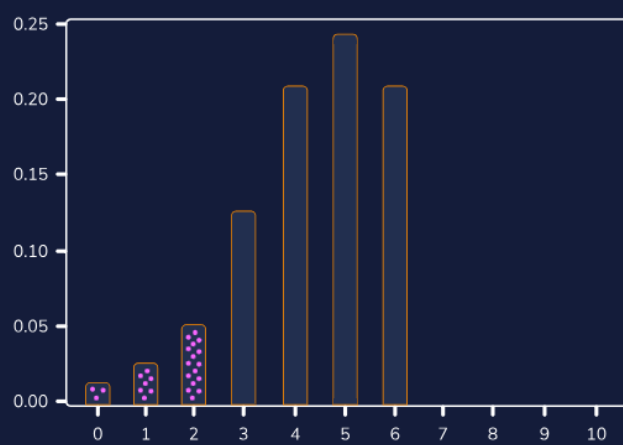
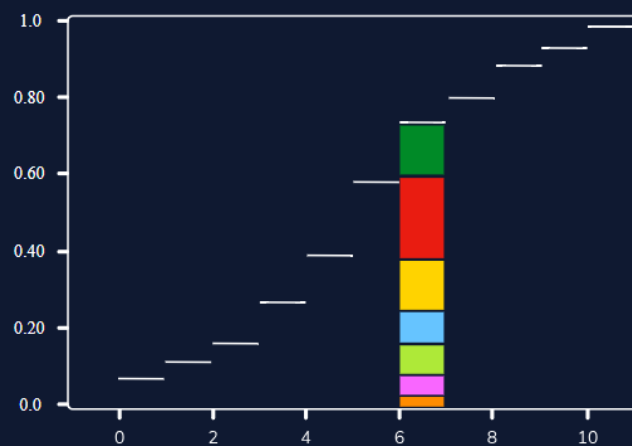
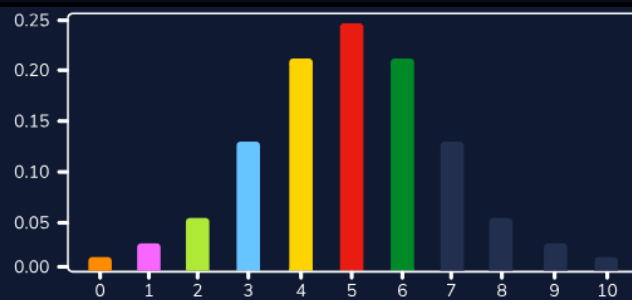
The following code simulates the outcome of rolling a fair die twice using `np.random.choice()`:

```
import numpy as np

# 7 is not included in the range function
die_6 = range(1, 7)

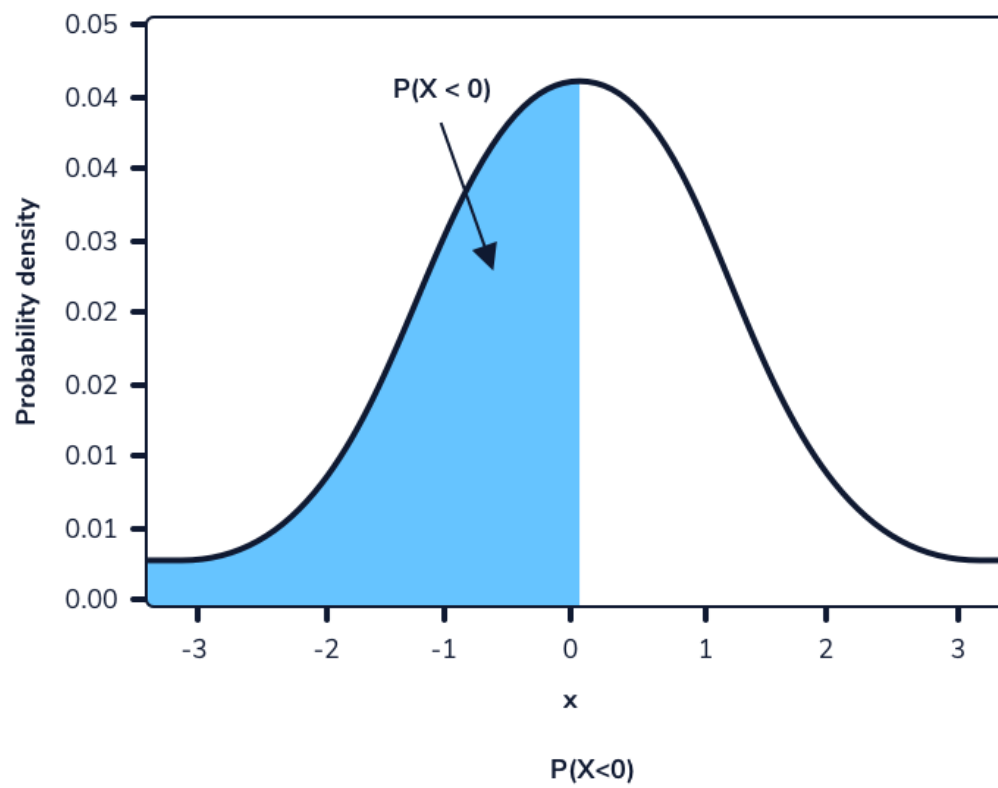
rolls = np.random.choice(die_6, size = 2, replace = True)

print(rolls)
```



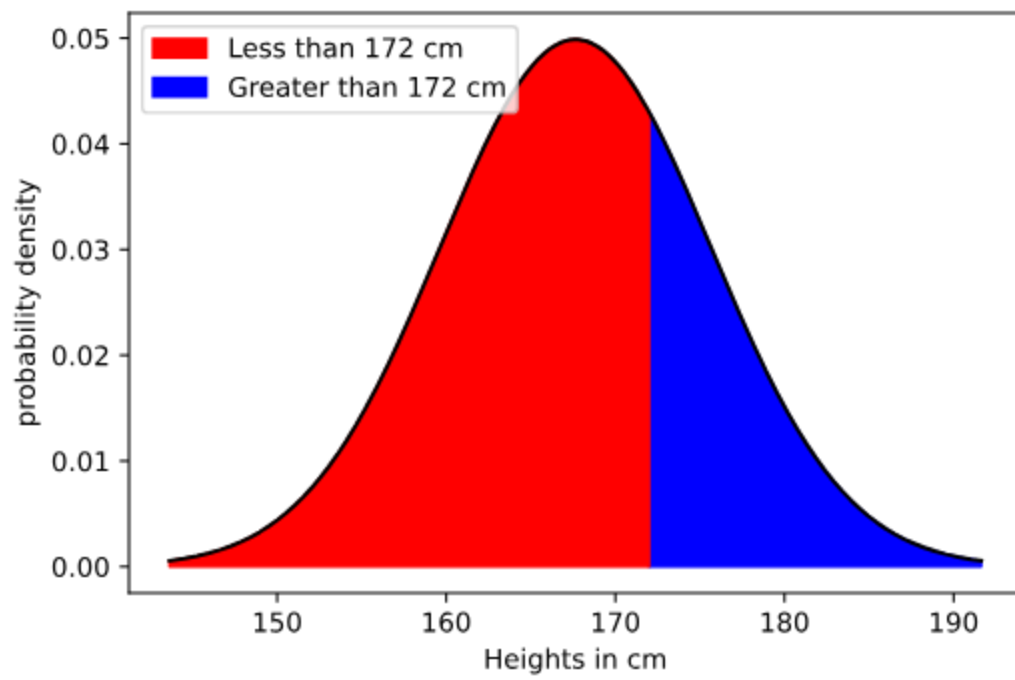
$$P(X \leq 6) = \text{CDF}(X=6) = 0.83$$

$$P(X \leq 2) = \text{CDF}(X=2) = 0.05$$



```
import scipy.stats as stats

# P(X > 172) = 1 - P(X < 172)
# 1 - stats.norm.cdf(x, loc, scale)
1 - stats.norm.cdf(172, 167.74, 8))
```



- **Poisson Distribution**
- **Binomial Distribution**

Central Limit Theory

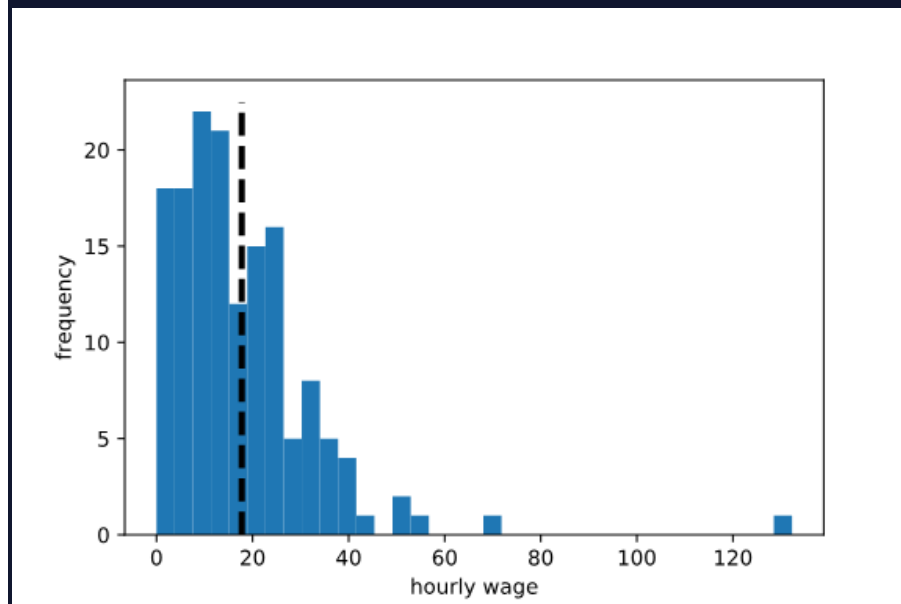
An introduction to the Central Limit Theorem using simulation

What is the Central Limit Theorem?

The *Central Limit Theorem (CLT)* is a powerful statistical tool that is useful in quantifying uncertainty around sample mean estimates. It is also the basis for common hypothesis tests, such as Z- and t-tests. A formal proof of the CLT requires some complex math, but this article will demonstrate it using a simulation!

Building intuition for the CLT

Imagine that a data scientist wants to know the average hourly wage for all working U.S. adults who make under 150 dollars per hour. This is a very large population — it would be unrealistic to collect wage data for every person meeting these criteria. Instead, suppose that the data scientist surveys a random sample of 150 people, records each person's hourly wage, and calculates a sample mean of 17.74 dollars per hour. Here's a histogram of this sample, with a black dotted line at 17.74:

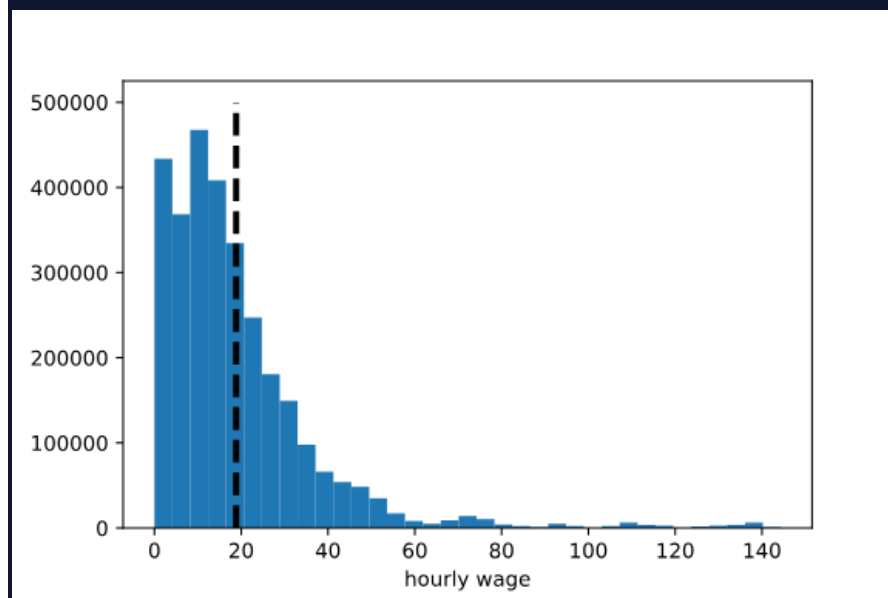


A good data scientist knows that this sample mean is not EXACTLY the same as the population mean, but hopes that it is close ENOUGH.

The next question is:

How far from the population mean could this sample mean realistically be?

To answer this, let's temporarily pretend that we are all-knowing and can actually inspect the hourly wages of all people in the population of interest. Suppose that the true average wage is 18.84 dollars per hour and a histogram of the full population looks like this:



In real life, we usually only observe a single sample — but in order to quantify our uncertainty about that sample, it is useful to think about what WOULD happen if we could observe more. Consider the following thought experiment: imagine that we could take some large number (say, 10,000) random samples of 150 people from the population and calculate the mean hourly wage for each of those samples. We could then inspect the 10,000 sample means to see how much they vary. A large amount of variation would make us less confident that any individual sample mean is representative of the population; less variation would make us more confident.

The Python code below does this in a loop. The `population` object is a list containing all wages in the full population. In each iteration of the loop, we do the following:

```
sample_means
```

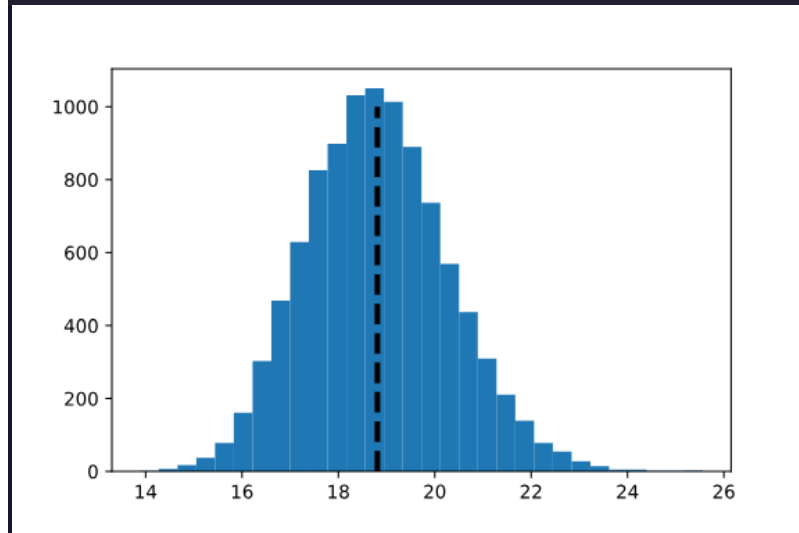
Finally, after collecting 10,000 sample means, we can inspect them using a histogram.

```
import numpy as np
import matplotlib.pyplot as plt
import random

sample_means = []
```

```
for i in range(10000):
    samp = random.sample(population, 150)
    sample_means.append(np.mean(samp))

plt.hist(sample_means, bins = 30)
plt.vlines(np.mean(sample_means), 0, 1000, lw=3, linestyle='dashed')
```



There are a few interesting things to notice about this distribution, which is called the *sampling distribution of the mean*:

Specifically, the NumPy `percentile()` function can be used to calculate that 95% of the sample means from the above simulation fall in a range from 16.14 to 21.87 dollars per hour (plus or minus around 2.87 dollars from the mean):

```
percentiles = np.percentile(sample_means, [2.5, 97.5])
print(percentiles)
# output: array([16.13810156, 21.87180969])
```

Formally defining the CLT

It's now time to formally define the CLT, which tells us that the sampling distribution of the mean:

With respect to the standard error formula described above, note that there are two levers on the width of the sampling distribution:

To build your intuition, the following simulation may be helpful. We can use the `random.normal()` function from NumPy to generate a population of values with a particular standard deviation (`pop_stdev`) and mean (arbitrarily set to 10 in this example). As before, we can do the following:

```
samp_size
```

Coding question

Questions

Try changing `samp_size` to any number between 2 and 20. Then, leaving `samp_size = 2`, try changing the value of `pop_stdev` to any number between 1 and 10 (higher values mean more variation in the population). Observe how higher values of `samp_size` and lower values of `pop_stdev` result in a skinnier distribution (smaller standard error).

```
import matplotlib.pyplot as plt
import codecademylib3

### try changing samp_size and pop_stdev
and re-running the code
# try values of samp_size between 2 and 20
# try values of pop_stdev between 1 and 10
samp_size = 2
pop_stdev = 10

population = np.random.normal(loc = 10,
                              scale = pop_stdev, size = 100000)
population = list(population)

sample_means = []

for i in range(10000):
    samp = random.sample(population,
                          samp_size)
```

```

sample_means.append(np.mean(samp))

plt.hist(sample_means, bins = 30)
plt.title('Sampling Distribution')
plt.xlim(-10, 30)

plt.show()

```

How does this help the data scientist?

In real life, the data scientist is still stuck with their one sample mean as a best guess for the population mean. However, they can leverage the CLT to estimate the standard error — the amount of variation in imagined, repeated samples!

Remember that the CLT tells us that the standard error (SE) can be calculated as follows:

$$SE = \frac{\text{Population Standard Deviation}}{\sqrt{\text{Sample Size}}}$$

$$SE =$$

Sample Size

Population Standard Deviation

While a researcher or data scientist probably does not know the population standard deviation, they can use the standard deviation of their sample to estimate it.

Let's return to the data scientist who collected a single sample of 150 wages and calculated an average wage of 17.74 dollars. To quantify the uncertainty around this sample mean, this data scientist can first estimate the standard error:

```

std_error = np.std(my_sample) / (150 ** 0.5)
print(std_error)
# output: 1.275

```

Then, leveraging the part of the CLT that says the sampling distribution is normally distributed, our data scientist can use a nifty property of normal distributions: 95% of normally distributed values are within about 1.96 standard deviations of the

mean. This allows the data scientist to estimate the width of the sampling distribution above, without actually knowing the population distribution!

First, the data scientist needs to multiply 1.96 by the estimated standard error: $1.96 * 1.275 = 2.50$. The interpretation of this number is as follows:

Imagine taking a large number of samples of size 150 from a population with the same amount of variation as in the observed sample.

95% of those samples would be within about 2.50 dollars from the true population mean.

Therefore, there is about a 95% probability that the observed sample mean of 17.74 is no more than 2.50 dollars away from the population mean. In other words, there is about a 95% probability that the population mean is between 15.24 and 20.24. This is referred to as a *95% confidence interval*.

Note that the estimate of 2.50 is similar to the value of 2.87 that we calculated as all-knowing beings with access to the full population. With only a single sample in hand, the data scientist can express the uncertainty in their sample mean fairly accurately!

Descriptive vs. Inferential Statistics

Learn about the difference between descriptive and inferential statistics.

Introduction

Descriptive and inferential statistics are two subfields of the larger field of statistics. Each is used for distinct purposes. In this article, we will introduce and explore some of the methods associated with each one.

Descriptive Statistics

Descriptive statistics is all about summarizing data. It is useful for making large amounts of information into an interpretable subset of numbers and/or visualizations. Imagine a long and complex spreadsheet; we would not be able to easily understand the data (any trends, patterns, or meaningful summaries) just by looking over the rows and columns. However, with descriptive statistics, we are able to distill that information into numbers and visualizations that we can make sense of.

Commonly used descriptive statistics include the average, median, frequency, standard deviation, and range of a set of values. These numeric descriptive statistics can also be displayed as visual representations, such as tables and graphs.

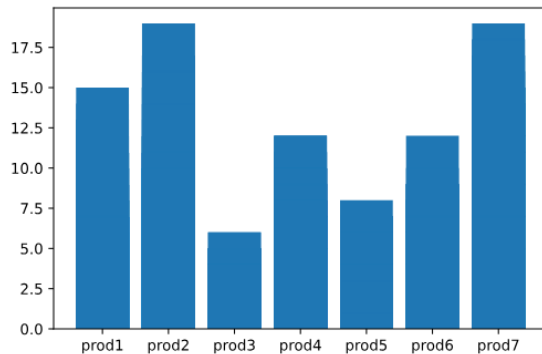
Example: Sales Company

Suppose you work at a large company and you have been given a dataset of sales information from the past month. You could use descriptive statistics to turn the dataset into a one-page report or table that will be more readable and provide more information than the raw data. Some potential descriptive statistics in the report could include:

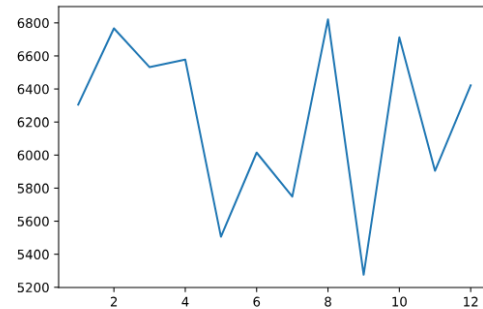
Average # sales per day: 214.0667

% Salespeople that met quota: 60%

Total Sales of Each Product This Month:



Monthly Sales Throughout the Year:



The data is easily interpretable in this form. Instead of trying to make sense of a spreadsheet of the raw data, we can learn specific pieces of information very easily. In this example, we can see that Product 3 is sold the least while Product 7 is sold the most. We can also see how sales varied throughout the year.

Multiple choice

Questions

Which of the following is not true about descriptive statistics?

Answer Choices

Numeric descriptive statistics can be visualized.

Descriptive statistics are limited to average, frequency, and percentages.

Descriptive statistics summarize larger amounts of data.

Descriptive statistics allows us to see information about the data that was not easily visible otherwise.

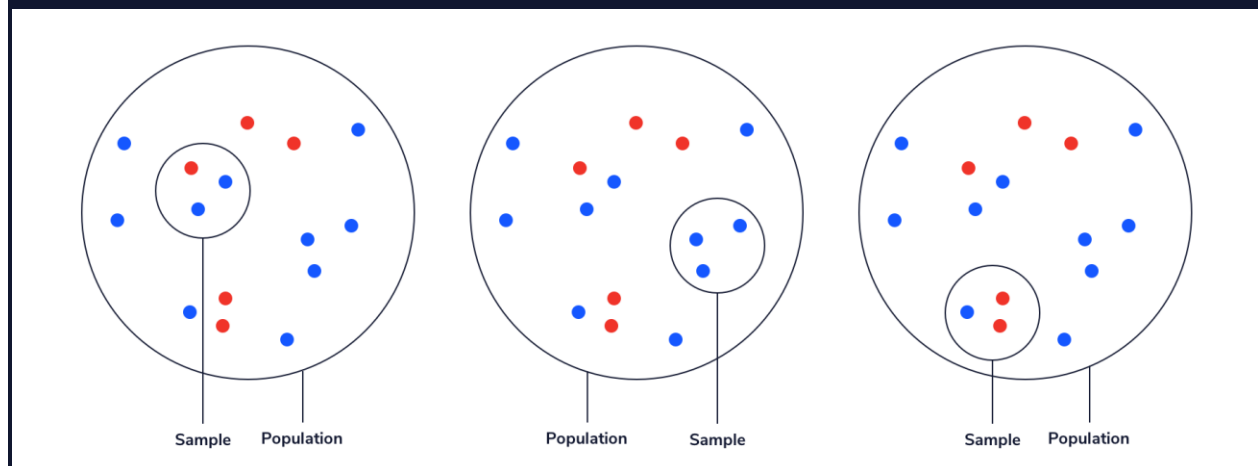
Inferential Statistics

Inferential statistics is all about using a sample (a subset of a population) to make inferences about a larger population of interest. This is useful when we want to know something about a population

but cannot observe every member — often due to time, feasibility, or monetary constraints. Some methods that are used in inferential statistics include hypothesis testing and regression.

The key to inferential statistics is understanding that samples do not always accurately reflect the population they came from. A large part of inferential statistics is quantifying our uncertainty about a population by looking at a smaller sample.

For example, the population shown below is made up of 10 blue dots and 5 red dots, which means that two thirds of the population is blue and one third is red. Suppose we take a random sample of 3 dots and want to use that sample to estimate proportion of the population that is blue. If we're lucky, we'll sample 2 blue dots and 1 red dot, as shown on the left, which would accurately represent the proportions of blue to red dots in the population. However, we could also randomly sample 3 blue dots, or 2 red dots and 1 blue dot — both of which do not match the population. Inferential statistics allows us to look at a sample and then quantify our uncertainty about how similar (or different) the entire population might be.



Multiple choice

Questions

True or False: Because it is a subset of the population, a random sample will always perfectly represent the population.

Answer Choices

True

False

Example: Customer Contacting

Suppose you work at a sales company that is interested in testing two different customer contacting methods to see if one leads to a higher response rate than the other. It is impossible to test both methods with the entire population of every single past, present, and future customer. Instead, you could take a sample of 1,000 customers and randomly assign them to either a text contacting system or a phone calling system. After one month, you could then calculate the difference in response rate (a descriptive statistic) for the two sampled groups.

Suppose you find that the customers who received a text were 12% more likely to respond than the customers who received a phone call. This is a descriptive statistic about the sample – but what you really want to know is: if you had sampled the full population of customers, would you still have found at least a 12% difference in response rate?

This is where inferential statistics methods come in handy! For example, you could use a hypothesis test to estimate the probability that, in the full population, you will observe a higher response rate for texts compared to calls given that you observed a higher rate in your sample.

Example: Test Scores

Suppose you are a researcher studying the relationship between high school students' homework grades and standardized test scores. It would be very difficult and expensive to collect information on homework grades and standardized test scores for every single high school student in the world. Instead, you could find a random sample of students and inspect the relationship between homework grades and standardized test scores among that sample. Finally, you could use a regression analysis (another inferential statistics method) to understand whether a similar relationship is likely to exist in the larger population of all students.

Implementing a One-Sample T-Test

In the last exercise, we inspected a sample of 50 purchase prices at BuyPie and saw that the average was 980 Rupees. Suppose that we want to run a one-sample t-test with the following null and alternative hypotheses:

Null: The average cost of a BuyPie order is 1000 Rupees

Alternative: The average cost of a BuyPie order is **not** 1000 Rupees.

SciPy has a function called `ttest_1samp()`, which performs a one-sample t-test for you.

`ttest_1samp()` requires two inputs, a sample distribution (eg. the list of the 50 observed purchase prices) and a mean to test against (eg. `1000`):

```
tstat, pval = ttest_1samp(sample_distribution, expected_mean)
```

Copy to Clipboard

The function uses your sample distribution to determine the sample size and estimate the amount of variation in the population — which are used to estimate the null distribution. It returns two outputs: the t-statistic (which we won't cover in this course), and the p-value.

Simulating the Null Distribution I

The first step in running a hypothesis test is to form a *null hypothesis*. For the question of whether the purchase rate at Live-it-LIVE.com was different from 10% this month, the null hypothesis describes a world in which the true probability of a visitor making a purchase was exactly 10%, but by random chance, we observed that only 41 visitors (8.2%) made a purchase.

Let's return to the coin flip example from the previous exercise. We can simulate 10 coin flips and print out the number of those flips that came up heads using the following code:

```
flips = np.random.choice(['heads', 'tails'], size=10, p=[0.5, 0.5])
num_heads = np.sum(flips == 'heads')
print(num_heads)
## output: 4
```

Copy to Clipboard

If we run this code a few times, we'll likely see different results each time. This will give us get a sense for the range in the number of heads that could occur by random chance, even if the coin is fair. We're more likely to see numbers like four, five, or six, but maybe we'll see something more extreme every once in a while — ten heads in a row, or even zero!

Simulating the Null Distribution II

In the last exercise, we simulated a random sample of 500 visitors, where each visitor had a 10% chance of making a purchase. When we pressed “Run” a few times, we saw that the number of purchases varied from sample to sample, but was **around** 50.

Similarly, we simulated a single random sample of 10 coin flips, where each flip had a 50% chance of coming up heads. We saw that the number of simulated heads was not necessarily 5, but somewhere around 5.

By running the same simulated experiment **many** times, we can get a sense for how much a particular outcome (like the number of purchases, or heads) varies by random chance. Consider the following code:

```
outcomes = []
for i in range(10000):
    flips = np.random.choice(['heads', 'tails'], size=10, p=[0.5,
0.5])
    num_heads = np.sum(flips == 'heads')
    outcomes.append(num_heads)
print(outcomes)
## output is something like: [3, 4, 5, 8, 5, 6, 4, 5, 3, 2, 8, 5, 7,
4, 4, 5, 4, 3, 6, 5, ...]
```

Copy to Clipboard

In this code chunk, we've done the following:

initialized an empty list named `outcomes` to store the number of 'heads' from simulated samples of coin flips

set up a for-loop to repeat the steps below 10000 times:

- flip a fair coin 10 times

- calculate the number of those 10 flips that came up heads

- append that number onto `outcomes`

Note that 10000 is an arbitrarily chosen large number — it's big enough that it will yield almost all possible outcomes of our experiment, and small enough that the simulation still runs quickly. From inspecting the output, we can see that the number of 'heads' varied between 0 and 10:

```
min_heads = np.min(outcomes)
print(min_heads) #output: 0

max_heads = np.max(outcomes)
print(max_heads) #output: 10
```

Copy to Clipboard

Thus, if we flip a fair coin 10 times, we could observe anywhere between 0 and 10 heads by random chance.

Inspecting the Null Distribution

In the previous exercise, we simulated 10000 different samples of 500 visitors, where each visitor had a 10% chance of making a purchase, and calculated the number of purchases per sample. Upon further inspection, we saw that those numbers ranged from around 25 to 75. This is useful information, but we can learn even more from inspecting the full distribution.

For example, recall our 10000 coin flip experiments: for each experiment, we flipped a fair coin 10 times and recorded the number of heads in a list named `outcomes`. We can plot a histogram of `outcomes` using `matplotlib.pyplot.hist()`. We can also add a vertical line at any x-value using `matplotlib.pyplot.axvline()`:

```
import matplotlib.pyplot as plt
plt.hist(outcomes)
plt.axvline(2, color = 'r')
plt.show()
```

This histogram shows us that, over 10000 experiments, we observed as few as 0 and as many as 10 heads out of 10 flips. However, we were most likely to observe around 4-6 heads. It would be unlikely to observe only 2 heads (where the vertical red line is).

Confidence Intervals

So far, we've inspected the null distribution and calculated the minimum and maximum values. While the number of purchases in each simulated sample ranged roughly from 25 to 75 by random chance, upon further inspection of the distribution, we saw that those extreme values happened very rarely.

By reporting an interval covering 95% of the values instead of the full range, we can say something like: "we are 95% confident that, if each visitor has a 10% chance of making a purchase, a random sample of 500 visitors will make between 37 and 63 purchases." We can use the `np.percentile()` function to calculate this 95% interval as follows:

```
np.percentile(outcomes, [2.5, 97.5])
# output: [37. 63.]
```

Copy to Clipboard

We calculated the 2.5th and 97.5th percentiles so that exactly 5% of the data falls outside those percentiles (2.5% above the 97.5th percentile, and 2.5% below the 2.5th percentile). This leaves us with a range covering 95% of the data.

If our observed statistic falls outside this interval, then we can conclude it is unlikely that the null hypothesis is true. In this example, because 41 falls within the 95% interval (37 - 63), it is still

reasonably likely that we observed a lower purchase rate by random chance, even though the null hypothesis was true.

Calculating a One-Sided P-Value

Calculating a Two-Sided P-Value

MODEL

Linear

Regression

Linear regression is a powerful modeling technique that can be used to understand the relationship between a quantitative variable and one or more other variables, sometimes with the goal of making predictions. For example, linear regression can help us answer questions like:

What is the relationship between apartment size and rental price for NYC apartments?

Is a mother's height a good predictor of their child's adult height?

The first step before fitting a linear regression model is exploratory data analysis and data visualization: is there a relationship that we can model? For example, suppose we collect heights (in cm) and weights (in kg) for 9 adults and inspect a plot of height vs. weight:

```
plt.scatter(data.height, data.weight)
plt.xlabel('height (cm)')
plt.ylabel('weight (kg)')
plt.show()
```

Equation of a Line

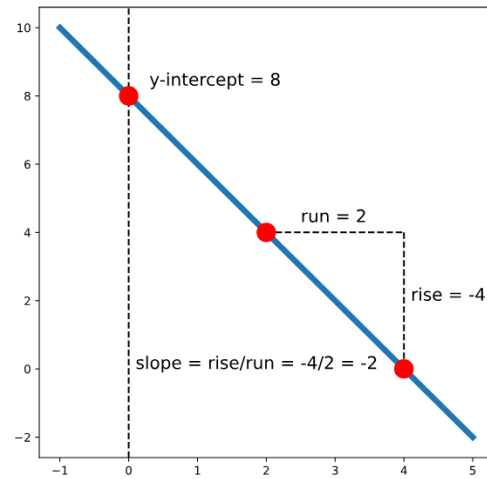
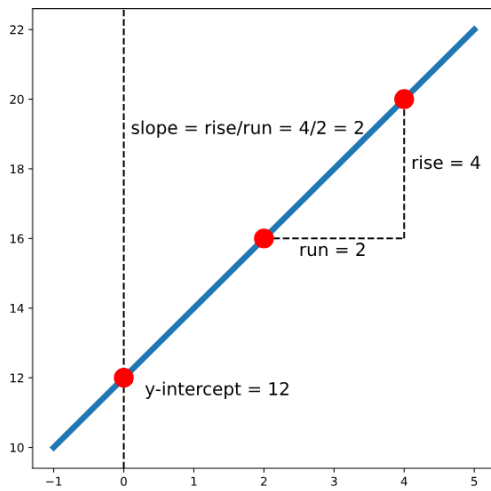
Like the name implies, LINEar regression involves fitting a line to a set of data points. In order to fit a line, it's helpful to understand the equation for a line, which is often written as $y=mx+b$. In this equation:

x and y represent variables, such as height and weight or hours of studying and quiz scores.

b represents the *y-intercept* of the line. This is where the line intersects with the y -axis (a vertical line located at $x = 0$).

m represents the slope. This controls how steep the line is. If we choose any two points on a line, the slope is the ratio between the vertical and horizontal distance between those points; this is often written as rise/run.

The following plot shows a line with the equation $y = 2x + 12$:



Note that we can also have a line with a negative slope. For example, the following plot shows the line with the equation $y = -2x + 8$.

```
# Load libraries

import codecademylib3

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt


# Read in the data

students = pd.read_csv('test_data.csv')


# Write equation for a line

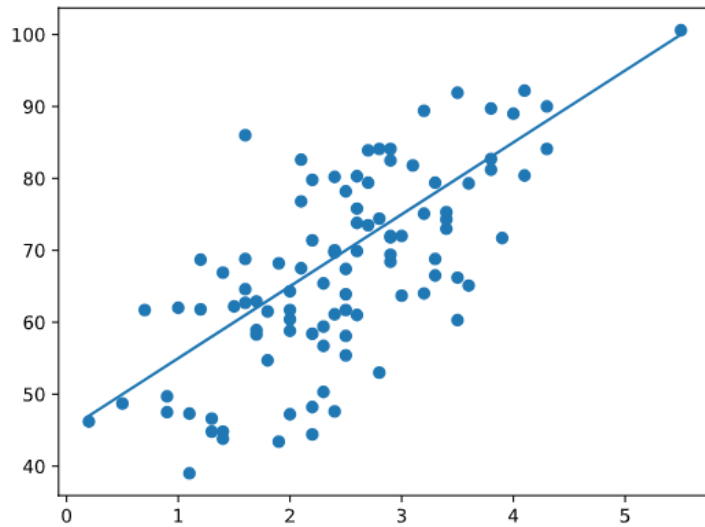
predicted_score = 25 * students.hours_studied + 20


# Create the plot

plt.scatter(students.hours_studied, students.score)

plt.plot(students.hours_studied, predicted_score)

plt.show()
```



Finding the "Best" Line

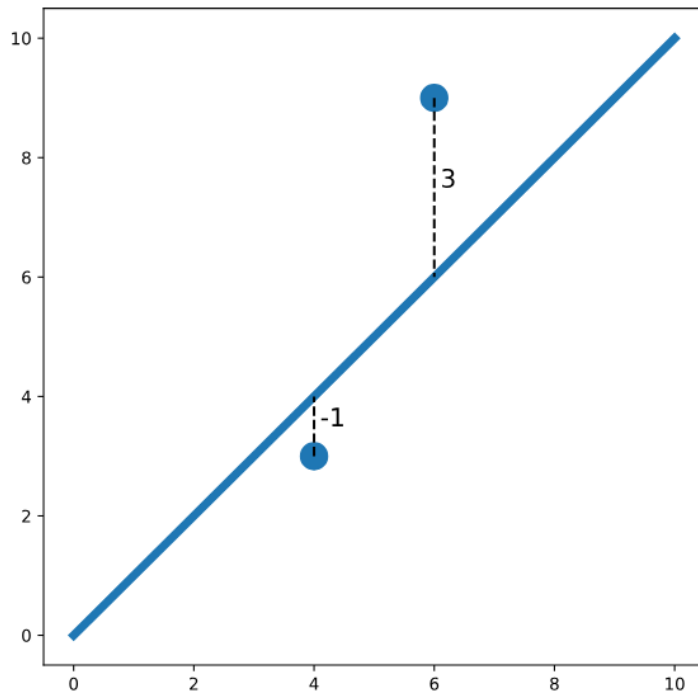
In the last exercise, we tried to eye-ball what the best-fit line might look like. In order to actually choose a line, we need to come up with some criteria for what "best" actually means.

Depending on our ultimate goals and data, we might choose different criteria; however, a common choice for linear regression is *ordinary least squares (OLS)*. In simple OLS regression, we assume that the relationship between two variables x and y can be modeled as:

$$y = mx + b + \text{error}$$

$$y = mx + b + \text{error}$$

We define "best" as the line that minimizes the *total squared error* for all data points. This total squared error is called the *loss function* in machine learning. For example, consider the following plot:



In this plot, we see two points on either side of a line. One of the points is one unit below the line (labeled -1). The other point is three units above the line (labeled 3). The total squared error (loss) is:

$$\text{loss} = (-1)^2 + (3)^2 = 1 + 9 = 10$$

$$\text{loss} = (-1)^2$$

$$2$$

$$+ (3)^2$$

$$2$$

$$= 1 + 9 = 10$$

Notice that we square each individual distance so that points below and above the line contribute equally to loss (when we square a negative number, the result is positive). To find the best-fit line, we need to find the slope and intercept of the line that minimizes loss.

EDA: Inspect, Clean, and Validate a Dataset

Learn how EDA can inform the data cleaning process.

Introduction

One of the most challenging parts of data cleaning is diagnosing data issues and figuring out HOW to most effectively address them. In order to accomplish this, exploratory data analysis (EDA) can be an extremely useful tool. In this article, we'll walk through an example dataset to demonstrate how EDA can inform the initial data inspection, cleaning, and validation process.

While this article serves as an introduction to EDA for data cleaning, it is important to note that every dataset is different, and therefore will require different exploration. EDA is all about following the data, verifying your assumptions, and investigating anything that is unexpected.

Initial Data Inspection

Before analysis or cleaning, it is useful to print a few rows of data. This helps ensure that the data is properly loaded. It also allows us to compare the observed data to the data dictionary and determine whether the coding appears to match our expectations. For example, let's load and inspect the first few rows of a dataset of heart disease patients (downloaded from the [UCI Machine Learning Repository](#)).

```
import pandas as pd
heart = pd.read_csv('processed.cleveland.data.csv')
print(heart.head())
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	heart_disease
0	63.0	1.0	1.0	145.0	233.0	1.0	2.0	150.0	0.0	2.3	3.0	0.0	6.0	0
1	67.0	1.0	4.0	160.0	286.0	0.0	2.0	108.0	1.0	1.5	2.0	3.0	3.0	2
2	67.0	1.0	4.0	120.0	229.0	0.0	2.0	129.0	1.0	2.6	2.0	2.0	7.0	1
3	37.0	1.0	3.0	130.0	250.0	0.0	0.0	187.0	0.0	3.5	3.0	0.0	3.0	0
4	41.0	0.0	2.0	130.0	204.0	0.0	2.0	172.0	0.0	1.4	1.0	0.0	3.0	0

There are a few things we might want to inspect. For example, the data dictionary gives the following information about the `cp` column:

cp: chest pain type

Based on this information, it's not necessarily clear whether the data is going to be coded as numerical values (eg., 1, 2, 3, or 4) or with strings (eg., 'typical angina'). Data inspection allows us to clarify that this column contains numerical values.

Similarly, there is some conflicting information in the data dictionary about the target column (note: we renamed this column as `heart_disease` before loading it, but it was originally coded as `num`). The list of features contains the following information about this column:

num: diagnosis of heart disease (angiographic disease status)

However, the initial data description suggests that the target field is integer valued from 0-4, where 0 indicates no heart disease, and values 1-4 indicate the presence of heart disease.

By inspecting the first few rows of data, we see at least one instance of the value 2 in the `heart_disease` column. This suggests that the values probably range from 0-4 instead of just 0-1.

We could verify this with further exploration (e.g., by using `heart.heart_disease.value_counts()` to get a table of values in this column).

Data Information

Once we've taken a first look at some data, a common next step is to address questions such as:

How many (non-null) observations do we have?

How many unique columns/features do we have?

Which columns (if any) contain missing data?

What is the data type of each column?

Using pandas, we can easily address these questions using the `.info()` method. For example:

```
print(heart.info())
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
#   Column          Non-Null Count  Dtype
---  -
0   age             303 non-null   float64
1   sex             303 non-null   float64
.
.
.
```

Inspecting Missing Data

After identifying that there is some missing data and converting it to a format that Python can recognize, it's often a good idea to take a closer look at those rows. Sometimes, we can find clues as to WHY the data is missing, which can help us make decisions about whether to get rid of the rows altogether or impute the missing values somehow.

```
heart[heart.isnull().any(axis=1)]
```

Exploratory Data Analysis: Summary Statistics

Learn to explore a dataset by choosing a summary statistic.

Introduction

Summary statistics are an important component of Exploratory Data Analysis (EDA) because they allow a data analyst to condense a large amount of information into a small set of numbers that can be easily interpreted. In order to decide what kind of summary statistic to use, it is important to consider two things:

Univariate Statistics

Summary statistics that focus on a single variable are called *univariate statistics*. They are useful for answering questions about a single feature in tabular data. For example, the following dataset contains information about used cars listed on cardekho.com:

	name	year	selling_price	km_driven	fuel	transmission	owner	mileage	engine
0	Maruti Swift	20	450000	14550	Die	Manual	First	23.4	1248
	Dzire VDI	14		0	sel		Owner	kmpl	CC
1	Skoda Rapid 1.5	20	370000	12000	Die	Manual	Second	21.14	1498
	TDI Ambition	14		0	sel		Owner	kmpl	CC
2	Honda City 2017-	20	158000	14000	Pet	Manual	Third	17.7	1497
	2020 EXi	06		0	rol		Owner	kmpl	CC
3	Hyundai i20	20	225000	12700	Die	Manual	First	23.0	1396
	Sportz Diesel	10		0	sel		Owner	kmpl	CC

4	Maruti Swift VXi	20	130000	12000	Pet	Manual	First	16.1	1298
	BSIII	07		0	rol		Owner	kmp1	CC

Univariate statistics can help us answer questions like:

Each of these questions focuses on a single variable (`selling_price`, `transmission`, and `year`, respectively, for the above examples). Depending on the type of variable, different summary statistics are appropriate.

Quantitative Variables

When summarizing quantitative variables, we often want to describe *central location* and *spread*.

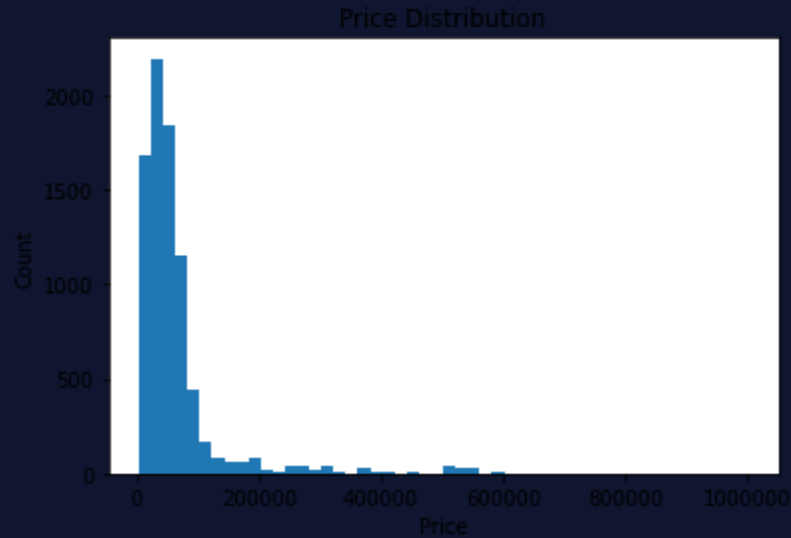
Central Location

The central location (also called central tendency) is often used to communicate the “typical” value of a variable. Recall that there are a few different ways of calculating the central location:

Choosing an appropriate summary statistic for central tendency sometimes requires data visualization techniques along with domain knowledge. For example, suppose we want to know the typical price of a car in our dataset. If we calculate each of the statistics described above, we’ll get the following estimates:

Because the mean is so much larger than the median and trimmed mean, we might guess that there are some outliers in this data with respect to price. We can investigate this by plotting a histogram of `selling_price`:

```
# Generate a histogram of the selling_price variable
plt.hist(cars['selling_price'])
plt.show()
```



Indeed, we see that `selling_price` is highly right-skewed. The very high prices (10 million Rupees for a small number of cars) are skewing the average upwards. By using the median or a trimmed mean, we can more accurately represent a “typical” price.

Spread

Spread, or dispersion, describes the variability within a feature. This is important because it provides context for measures of central location. For example, if there is a lot of variability in car prices, we can be less certain that any particular car will be close to 450000.00 Rupees (the median price). Like the central location measures, there are a few values that can describe the spread:

Choosing the most appropriate measure of spread is much like choosing a measure of central tendency, in that we need to consider the data holistically. For example, below are measures of spread calculated for `selling_price`:

We see that the range is almost 10 million Rupees; however, this could be due to a single 10 million Rupee car in the dataset. If we remove that one car, the range might be much smaller. The IQR is useful in comparison because it trims away outliers.

Meanwhile, we see that variance is extremely large. This happens because variance is calculated using squared differences, and is therefore not in the same units as the original data, making it less interpretable. Both the standard deviation and MAD solve this issue, but MAD is even less impacted by extreme outliers.

For highly skewed data or data with extreme outliers, we therefore might prefer to use IQR or MAD. For data that is more normally distributed, the variance and standard deviation are frequently reported.

Categorical Variables

Categorical variables can be either ordinal (ordered) or nominal (unordered). For ordinal categorical variables, we may still want to summarize central location and spread. However, because ordinal categories are not necessarily evenly spaced (like numbers), we should NOT calculate the mean of an ordinal categorical variable (or anything that relies on the mean, like variance, standard deviation, and MAD).

For nominal categorical variables (and ordinal categorical variables), another common numerical summary statistic is the frequency or proportion of observations in each category. This is often reported using a frequency table and can be visualized using a bar plot.

For example, suppose we want to know what kind of fuel listed cars tend to use. We could calculate the frequency of each fuel type:

```
cars.fuel.value_counts()
```

Output:

```
Diesel      2153
Petrol      2123
CNG         40
LPG         23
Electric     1
Name: fuel, dtype: int64
```

This tells us that 'Diesel' cars are most common, with 'Petrol' cars a close second. Converting these frequencies to proportions can also help us compare fuel types more easily. For example, the following table of proportions indicates that 'Diesel' cars account for almost half of all listings.

```
cars.fuel.value_counts(normalize=True)
```

Output:

```
Diesel      0.496083
Petrol      0.489171
CNG         0.009217
LPG         0.005300
Electric     0.000230
Name: fuel, dtype: float64
```

Bivariate Statistics

In contrast to univariate statistics, bivariate statistics are used to summarize the relationship between two variables. They are useful for answering questions like:

Do manual transmission cars tend to cost more or less than automatic transmission?

Do older cars tend to cost less money?

Are automatic transmission cars more likely to be sold by individuals or dealers?

Depending on the types of variables we want to summarize a relationship between, we should choose different summary statistics.

One Quantitative Variable and One Categorical Variable

If we want to know whether manual transmission cars tend to cost more or less than automatic transmission cars, we are interested in the relationship between `transmission` (categorical) and `selling_price` (quantitative). To answer this question, we can use a mean or median difference.

For example, we could calculate that the median price of automatic transmission cars is 100000 Rupees higher than for manual transmission cars.

Two Quantitative Variables

If we want to know whether older cars tend to cost less money, we are interested in the relationship between `year` and `selling_price`, both of which are quantitative. To answer this question, we can use the Pearson correlation.

For example, if we calculate that the correlation between `year` and `selling_price` is 0.4, we can conclude that there is a moderate positive association between these variables (older cars do tend to cost less money).

Two Categorical Variables

If we want to know whether automatic transmission cars are more likely to be sold by individuals or dealers, we are interested in the relationship between `transmission` and `seller_type`, both of which are categorical. We can explore this relationship using a contingency table and the Chi-Square statistic.

For example, based on the following contingency table, we might conclude that, although dealers sell more manual cars overall, the proportion of automatic cars among dealer sales is higher than the proportion of automatic cars among individual sales:

seller_type	Dealer	Individual	Trustmark	Dealer
transmission				
Automatic	217	212		19
Manual	777	3032		83

Conclusion

In this article, we've summarized some of the important considerations for choosing a summary statistic based on the question a data analyst wants to answer and the type of data that is available. When it comes to choosing summary statistics, there's no one right answer, but exploring data holistically and systematically is an important component of EDA

Fitting a Linear Regression Model in Python

There are a number of Python libraries that can be used to fit a linear regression, but in this course, we will use the `OLS.from_formula()` function from `statsmodels.api` because it uses simple syntax and provides comprehensive model summaries.

Suppose we have a dataset named `body_measurements` with columns `height` and `weight`. If we want to fit a model that can predict weight based on height, we can create the model as follows:

```
model = sm.OLS.from_formula('weight ~ height', data = body_measurements)
```

Copy to Clipboard

We used the formula `'weight ~ height'` because we want to predict `weight` (it is the outcome variable) using `height` as a predictor. Then, we can fit the model using `.fit()`:

```
results = model.fit()
```

Copy to Clipboard

Finally, we can inspect a summary of the results using `print(results.summary())`. For now, we'll only look at the coefficients using `results.params`, but the full summary table is useful because it contains other important diagnostic information.

```
print(results.params)
```

Copy to Clipboard

Output:

```
Intercept    -21.67
height         0.50
dtype: float64
```

Using a Regression Model for Prediction

Suppose that we have a dataset of heights and weights for 100 adults. We fit a linear regression and print the coefficients:

```
model = sm.OLS.from_formula('weight ~ height', data = body_measurements)
results = model.fit()
print(results.params)
```

Copy to Clipboard

Output:

```
Intercept    -21.67
height         0.50
dtype: float64
```

Copy to Clipboard

This regression allows us to predict the weight of an adult if we know their height. To make a prediction, we need to plug in the intercept and slope to our equation for a line. The equation is:

$$\text{weight} = 0.50 * \text{height} - 21.67$$

$$\text{weight} = 0.50 * \text{height} - 21.67$$

To make a prediction, we can plug in any height. For example, we can calculate that the expected weight for a 160cm tall person is 58.33kg:

$$\text{weight} = 0.50 * 160 - 21.67 = 58.33$$

$$\text{weight} = 0.50 * 160 - 21.67 = 58.33$$

In python, we can calculate this by plugging in values or by accessing the intercept and slope from `results.params` using their indices (0 and 1, respectively):

```
print(0.50 * 160 - 21.67)
# Output: 58.33

# OR:

print(results.params[1]*160 + results.params[0])
# Output: 58.33
```

Copy to Clipboard

We can also do this calculation using the `.predict()` method on the fitted model. To predict the weight of a 160 cm tall person, we need to first create a new dataset with `height` equal to 160 as shown below:

```
newdata = {"height": [160]}
print(results.predict(newdata))
```

Assumptions of Linear Regression Part 1

There are a number of assumptions of simple linear regression, which are important to check if you are fitting a linear model. The first assumption is that the relationship between the outcome variable and predictor is linear (can be described by a line). We can check this before fitting the regression by simply looking at a plot of the two variables.

The next two assumptions (normality and homoscedasticity) are easier to check after fitting the regression. We will learn more about these assumptions in the following exercises, but first, we need to calculate two things: *fitted values* and *residuals*.

Again consider our regression model to predict weight based on height (model formula `'weight ~ height'`). The fitted values are the predicted weights for each person in the dataset that was used to fit the model, while the residuals are the differences between the predicted weight and the true weight for each person. Visually:

We can calculate the fitted values using `.predict()` by passing in the original data. The result is a pandas series containing predicted values for each person in the original data

Categorical Predictors: Fit and Interpretation

Now that we've seen what a regression model with a binary predictor looks like visually, we can actually fit the model using `statsmodels.api.OLS.from_formula()`, the same way we did for a quantitative predictor:

```
model = sm.OLS.from_formula('height ~ play_bball', data)
results = model.fit()
print(results.params)
```

Copy to Clipboard

Output:

```
Intercept    169.016
```

```
play_bball    14.628
dtype: float64
```

Copy to Clipboard

Note that this will work if the `play_bball` variable is coded with 0s and 1s, but it will also work if it is coded with `True`s and `False`s, or even if it is coded with strings like 'yes' and 'no' (in this case, the coefficient label will look something like `play_bball[T.yes]` in the `params` output, indicating that 'yes' corresponds to a 1).

To interpret this output, we first need to remember that the intercept is the expected value of the outcome variable when the predictor is equal to zero. In this case, the intercept is therefore the mean height of non-basketball players.

The slope is the expected difference in the outcome variable for a one unit difference in the predictor variable. In this case, a one unit difference in `play_bball` is the difference between not being a basketball player and being a basketball player. Therefore, the slope is the difference in mean heights for basketball players and non-basketball players.

Linear Regression with a Categorical Predictor

Learn how to fit and interpret a linear model with a categorical predictor that has more than two categories.

Introduction

Linear regression is a machine learning technique that can be used to model the relationship between a quantitative variable and some other variable(s). Those other variables can be either quantitative (e.g., height or salary) or categorical (e.g., job industry or hair color). However, if we want to include categorical predictors in a linear regression model, we need to treat them a little differently than quantitative variables. This article will explore the implementation and interpretation of a single categorical predictor with more than two categories.

The Data

As an example, we'll use a [dataset from StreetEasy](#) that contains information about housing rentals in New York City. For now, we'll only focus on two columns of this dataset:

```
rent: the rental price of each apartment
borough: the borough that the apartment is located in, with three possible values ('Manhattan', 'Brooklyn', and 'Queens')
```

The first five rows of data are printed below:

```
import pandas as pd
rentals = pd.read_csv('rentals.csv')
print(rentals.head(5))
```

Output:

	rent	borough
0	5295	Brooklyn

1	4020	Manhattan
2	16000	Manhattan
3	3150	Queens
4	2955	Queens

The X Matrix

To understand how we can fit a regression model with a categorical predictor, it's useful to walk through what happens when we use `statsmodels.api.OLS.from_formula()` to create a model. When we pass a formula to this function (like `'weight ~ height'` or `'rent ~ borough'`), it actually creates a new data set, which we don't see. This new data set is often referred to as the *X matrix*, and it is used to fit the model.

When we use a quantitative predictor, the X matrix looks similar to the original data, but with an additional column of `1s` in front (the reasoning behind this column of `1s` is the subject of a future article — for now, no need to worry about it!). However, when we fit the model with a categorical predictor, something else happens: we end up with additional column(s) of `1s` and `0s`.

For example, let's say we want to fit a regression predicting `rent` based on `borough`. We can see the X matrix for this model using `patsy.dmatrices()`, which is implemented behind the scenes in `statsmodels`:

```
import pandas as pd
import patsy

rentals = pd.read_csv('rentals.csv')
y, X = patsy.dmatrices('rent ~ borough', rentals)

# Print out the first 5 rows of X
print(X[0:5])
```

Output:

```
[[1. 0. 0.]
 [1. 1. 0.]
 [1. 1. 0.]
 [1. 0. 1.]
 [1. 0. 1.]]
```

The first column is all `1s`, just like we would get for a quantitative predictor; but the second two columns were formed based on the `borough` variable. Remember that the first five values of the `borough` column looked like this:

`borough`

Brooklyn

Manhattan

Manhattan

Queens

Queens

Note that the second column of the X matrix `[0, 1, 1, 0, 0]` is an indicator variable for Manhattan: it is equal to `1` where the value of `borough` is `'Manhattan'` and `0` otherwise. Meanwhile, the third column of the X matrix (`[0, 0, 0, 1, 1]`) is an indicator variable for Queens: it is equal to `1` where the value of `borough` is `'Queens'` and `0` otherwise.

The X matrix does not contain an indicator variable for Brooklyn. That's because this data set only contains three possible values of `borough`: `'Brooklyn'`, `'Manhattan'`, and `'Queens'`. In order to recreate the `borough` column, we only need two indicator columns — because any apartment that is not in `'Manhattan'` or `'Queens'` must be `'Brooklyn'`. For example, the first row of the X matrix has `1`s in both indicator columns, indicating that the apartment must be in Brooklyn. Mathematically, we say that a `'Brooklyn'` indicator creates *collinearity* in the X matrix. In regular English: a `'Brooklyn'` indicator does not add any new information.

Because `'Brooklyn'` is missing from the X matrix, it is the *reference category* for this model.

Implementation and Interpretation

Let's now fit a linear regression model using `statsmodels` and print out the model coefficients:

```
import statsmodels.api as sm
model = sm.OLS.from_formula('rent ~ borough', rentals).fit()
print(model.params)
```

Output:

```
Intercept          3327.403751
borough[T.Manhattan] 1811.536627
borough[T.Queens]   -811.256430
dtype: float64
```

In the output, we see two different slopes: one for `borough[T.Manhattan]` and one for `borough[T.Queens]`, which are the two indicator variables we saw in the X matrix. We can use the intercept and two slopes to construct the following equation to predict rent:

$$\text{rent} = 3327.4 + 1811.5 * \text{borough}[T.\text{Manhattan}] - 811.3 * \text{borough}[T.\text{Queens}]$$

$$\text{rent} = 3327.4 + 1811.5 * \text{borough}[T.\text{Manhattan}] - 811.3 * \text{borough}[T.\text{Queens}]$$

To understand and interpret this equation, we can construct separate equations for each borough:

Equation 1: Brooklyn

When an apartment is located in Brooklyn, both `borough[T.Manhattan]` and `borough[T.Queens]` will be equal to zero and the equation becomes:

$$\text{rent} = 3327.4 + 1811.5 * 0 - 811.3 * 0 \text{ rent} = 3327.4$$

$$\text{rent} = 3327.4 + 1811.5 * 0 - 811.3 * 0$$

In other words, the intercept is the predicted (average) rental price for an apartment in Brooklyn (the reference category).

Equation 2: Manhattan

When an apartment is located in Manhattan, `borough[T.Manhattan] = 1` and `borough[T.Queens] = 0`. The equation becomes:

$$\text{rent} = 3327.4 + 1811.5 * 1 - 811.3 * 0 \text{ rent} = 3327.4 + 1811.5 \text{ rent} = 5138.9$$

$$\text{rent} = 3327.4 + 1811.5 * 1 - 811.3 * 0$$

We see that the predicted (average) rental price for an apartment in Manhattan is $3327.4 + 1811.5$: the intercept (which is the average price in Brooklyn) plus the slope on `borough[T.Manhattan]`. We can therefore interpret the slope on `borough[T.Manhattan]` as the difference in average rental price between apartments in Brooklyn (the reference category) and Manhattan.

Equation 3: Queens

When an apartment is located in Queens, `borough[T.Manhattan] = 0` and `borough[T.Queens] = 1`. The equation becomes:

$$\text{rent} = 3327.4 + 1811.5 * 0 - 811.3 * 1 \text{ rent} = 3327.4 - 811.3 \text{ rent} = 2516.1$$

$$\text{rent} = 3327.4 + 1811.5 * 0 - 811.3 * 1$$

We see that the predicted (average) rental price for an apartment in Queens is $3327.4 - 811.3$: the intercept (which is the average price in Brooklyn) plus the slope on `borough[T.Queens]` (which happens to be negative because Queens apartments are less expensive than Brooklyn apartments). We can therefore interpret the slope on `borough[T.Queens]` as the difference in average rental price between apartments in Brooklyn (the reference category) and Queens.

We can verify our understanding of all these coefficients by printing out the average rental prices by borough:

```
print(rentals.groupby('borough').mean())
```

Output:

	rent
borough	
Brooklyn	3327.403751
Manhattan	5138.940379
Queens	2516.147321

The average prices in each borough come out to the exact same values that we predicted based on the linear regression model! For now, this may seem like an overly complicated way to recover mean rental prices by borough, but it is important to understand how this works in order to build up more complex linear regression models in the future.

Changing the Reference Category

In the example above, we saw that `'Brooklyn'` was the default reference category (because it comes first alphabetically), but we can easily change the reference category in the model as follows:

```
model = sm.OLS.from_formula('rent ~ C(borough, Treatment("Manhattan"))',
rentals).fit()
print(model.params)
```

Output:

```
Intercept                    5138.940379
C(borough, Treatment("Manhattan")) [T.Brooklyn]  -1811.536627
C(borough, Treatment("Manhattan")) [T.Queens]    -2622.793057
dtype: float64
```

In this example, the reference category is 'Manhattan'. Therefore, the intercept is the mean rental price in Manhattan, and the other slopes are the mean differences for Brooklyn and Queens in comparison to Manhattan.

Other Python Libraries for fitting Linear Models

There are a few different Python libraries that can be used to fit linear regression models. It is therefore important to understand how this implementation differs for each library. In `statsmodels`, the creation of the X matrix happens completely “behind the scenes” once we pass in a model formula.

In `scikit-learn` (another popular library for linear regression), we actually need to construct the indicator variables ourselves. Note that we do not have to construct the extra column of `1`s that we saw in the X matrix – this also happens behind the scenes in `scikit-learn`. In order to construct those indicator variables, the `pandas get_dummies()` function is extremely useful:

```
import pandas as pd
rentals = pd.get_dummies(rentals, columns = ['borough'], drop_first = True)
print(rentals.head())
```

Output:

	rent	borough_Manhattan	borough_Queens
0	5295	0	0
1	4020	1	0
2	16000	1	0
3	3150	0	1
4	2955	0	1

Setting `drop_first = True` tells Python to drop the first indicator variable (for 'Brooklyn' in this case), which is what we need for linear regression. We can then fit the exact same model using `scikit-learn` as follows:

```
from sklearn.linear_model import LinearRegression

X = rentals[['borough_Manhattan', 'borough_Queens']]
y = rentals[['rent']]

# Fit model
regr = LinearRegression()
regr.fit(X, y)
print(regr.intercept_)
print(regr.coef_)
```

Output:

```
LinearRegression()
[3327.40375123]
[[1811.5366274 -811.25642981]]
```


Matplotlib

- Creating a line graph from data
- Changing the appearance of the line
- Zooming in on different parts of the axis
- Putting labels on titles and axes
- Creating a more complex figure layout
- Adding legends to graphs
- Changing tick labels and positions
- Saving what you've made

```
from matplotlib import pyplot as plt
```

Basic Line Plot

Line graphs are helpful for visualizing how a variable changes over time.

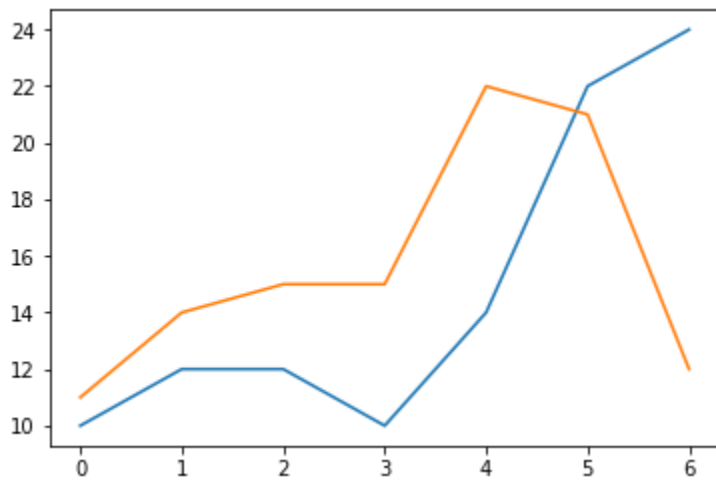
Some possible data that would be displayed with a line graph:

- average prices of gasoline over the past decade
- weight of an individual over the past couple of months
- average temperature along a line of longitude over different latitudes

Using Matplotlib methods, the following code will create a simple line graph using `.plot()` and display it using `.show()` :

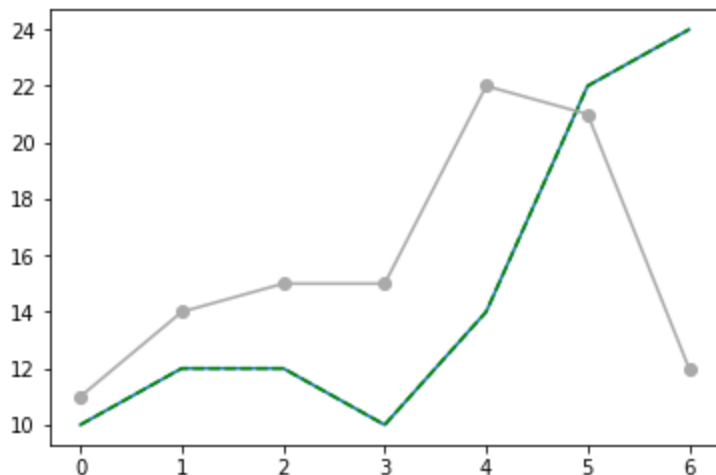
```
x_values = [0, 1, 2, 3, 4]
y_values = [0, 1, 4, 9, 16]
plt.plot(x_values, y_values)
plt.show()
```

```
# Days of the week:
days = [0, 1, 2, 3, 4, 5, 6]
# Your Money:
money_spent = [10, 12, 12, 10, 14, 22, 24]
# Your Friend's Money:
money_spent_2 = [11, 14, 15, 15, 22, 21, 12]
# Plot your money:
plt.plot(days, money_spent)
# Plot your friend's money:
plt.plot(days, money_spent_2)
# Display the result:
plt.show()
```



```
plt.plot(days, money_spent, color='green', linestyle='--')
plt.plot(days, money_spent_2, color='#AAAAAA', marker='o')
```

Copy to Clipboard

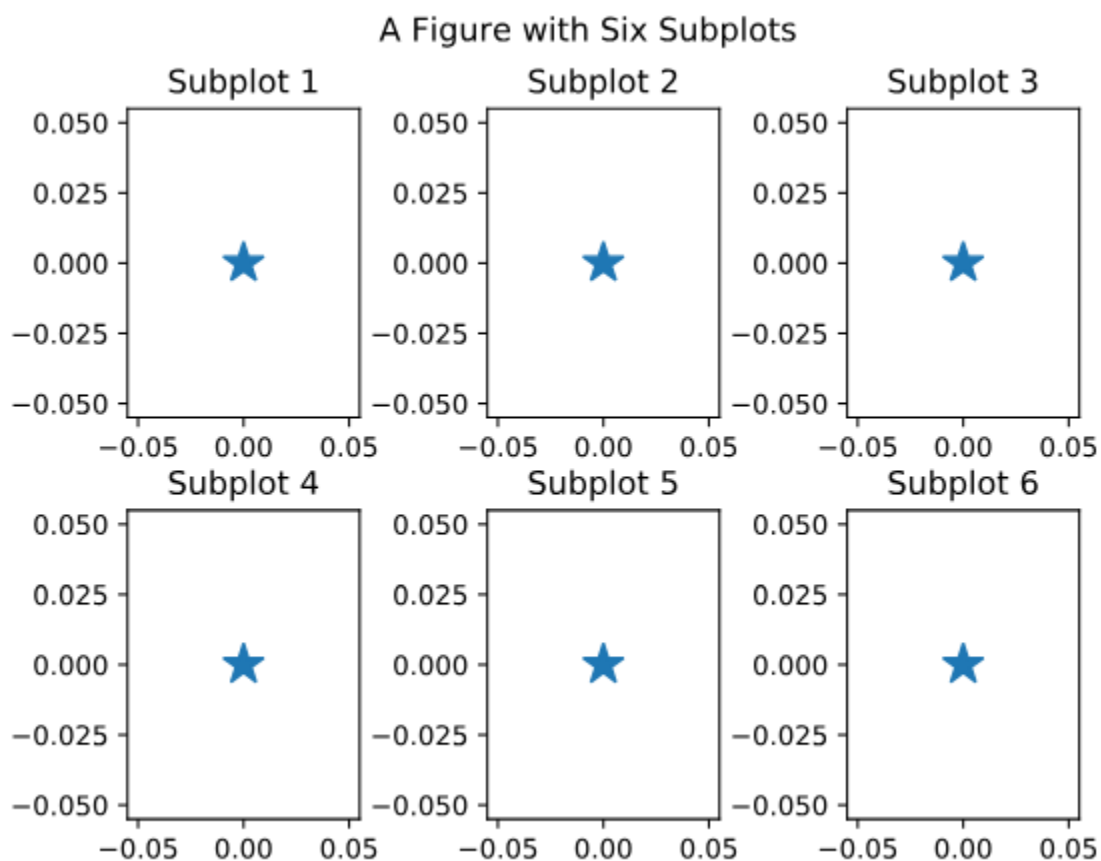


- **Labeling the Axes**
- Axis

Subplots

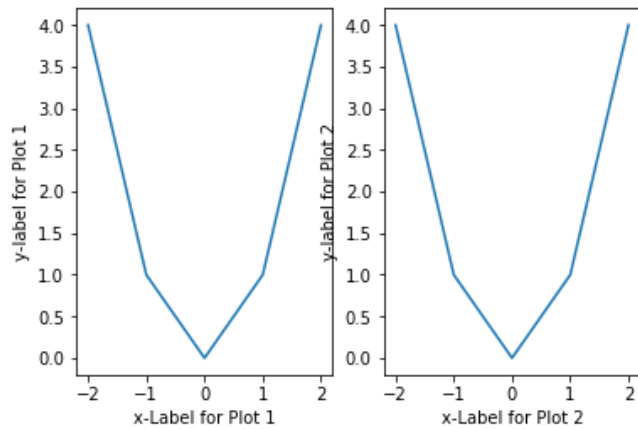
Sometimes, we want to display two lines side-by-side, rather than in the same set of x- and y-axes. When we have multiple axes in the same picture, we call each set of axes a *subplot*. The picture or object that contains all of the subplots is called a *figure*.

We can have many different subplots in the same figure, and we can lay them out in many different ways. We can think of our layouts as having rows and columns of subplots. For instance, the following figure has six subplots split into 2 rows and 3 columns:



Subplots Part II

Sometimes, when we're putting multiple subplots together, some elements can overlap and make the figure unreadable:



```
plt.subplots_adjust(bottom=0.2)
```

Copy to Clipboard

We can also use multiple keyword arguments, if we need to adjust multiple margins. For instance, we could adjust both the `top` and the `hspace`:

```
plt.subplots_adjust(top=0.95, hspace=0.25)
```

Copy to Clipboard

Let's use `wspace` to fix the figure above:

```
# Left Plot

plt.subplot(1, 2, 1)

plt.plot([-2, -1, 0, 1, 2], [4, 1, 0, 1, 4])


# Right Plot

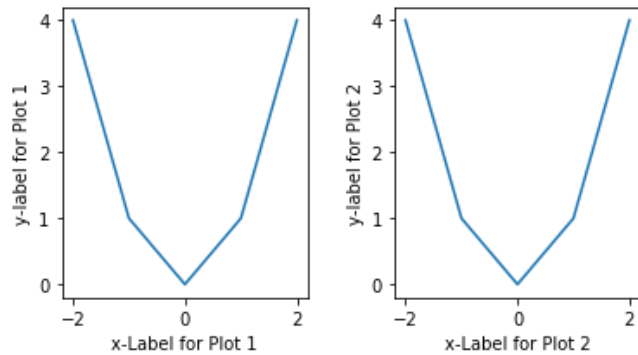
plt.subplot(1, 2, 2)

plt.plot([-2, -1, 0, 1, 2], [4, 1, 0, 1, 4])


# Subplot Adjust
```

```
plt.subplots_adjust(wspace=0.35)
```

```
plt.show()
```



```
x = range(7)

straight_line = [0, 1, 2, 3, 4, 5, 6]

parabola = [0, 1, 4, 9, 16, 25, 36]

cubic = [0, 1, 8, 27, 64, 125, 216]

# Subplot 1

plt.subplot(2, 1, 1)

plt.plot(x, straight_line)

# Subplot 2

plt.subplot(2, 2, 3)

plt.plot(x, parabola)

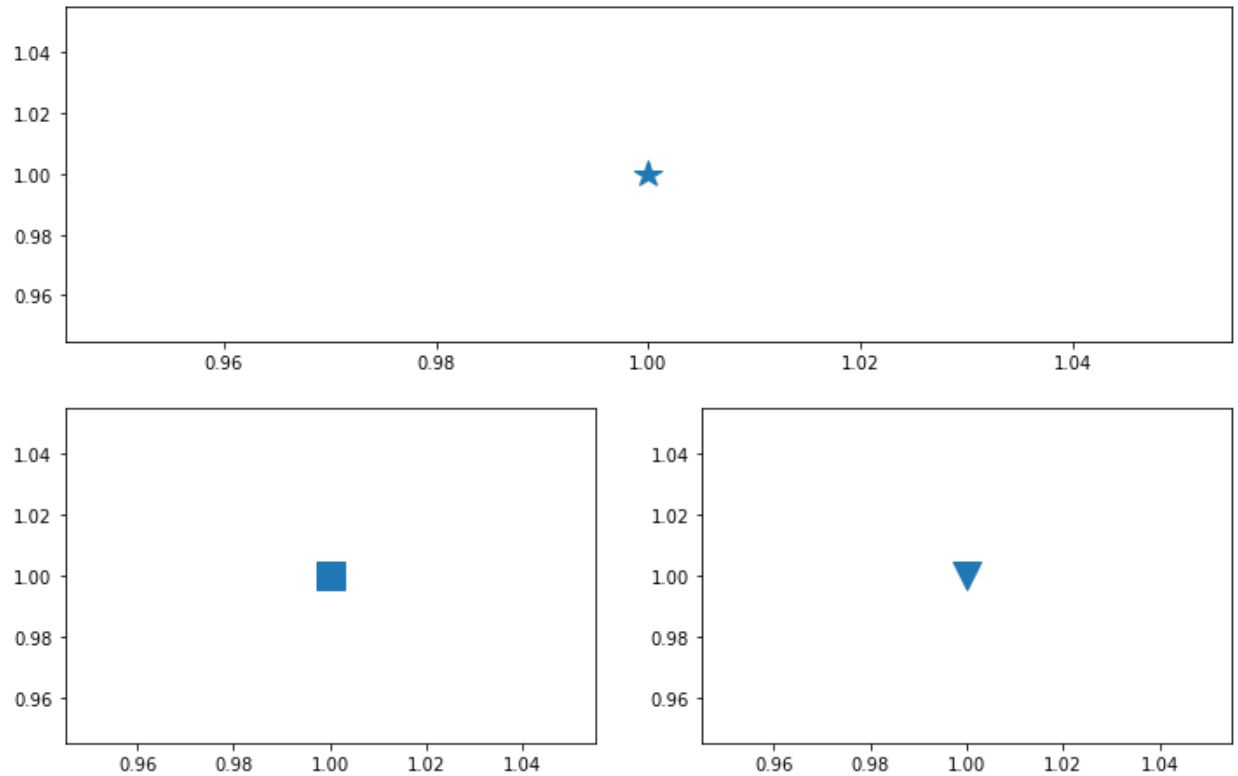
# Subplot 3

plt.subplot(2, 2, 4)

plt.plot(x, cubic)

plt.subplots_adjust(wspace=0.35, bottom=0.2)

plt.show()
```



Legends

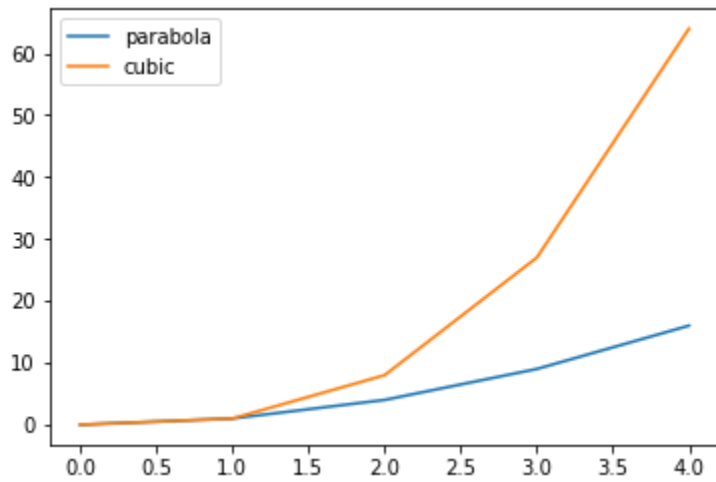
When we have multiple lines on a single graph we can label them by using the command `plt.legend()`.

The `legend` method takes a list with the labels to display. So, for example, we can call:

```
plt.plot([0, 1, 2, 3, 4], [0, 1, 4, 9, 16])  
plt.plot([0, 1, 2, 3, 4], [0, 1, 8, 27, 64])  
plt.legend(['parabola', 'cubic'])  
plt.show()
```

Copy to Clipboard

which would display a legend on our graph, labeling each line:



`plt.legend()` can also take a keyword argument `loc`, which will position the legend on the figure.

These are the position values `loc` accepts:

Number	Code	String
--------	------	--------

0		best
---	--	------

1		upper right
---	--	-------------

2		upper left
---	--	------------

3		lower left
---	--	------------

4		lower right
---	--	-------------

5		right
---	--	-------

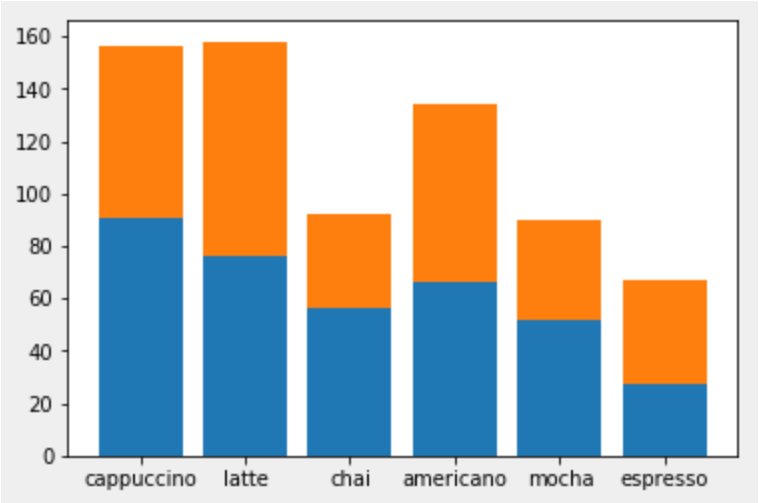
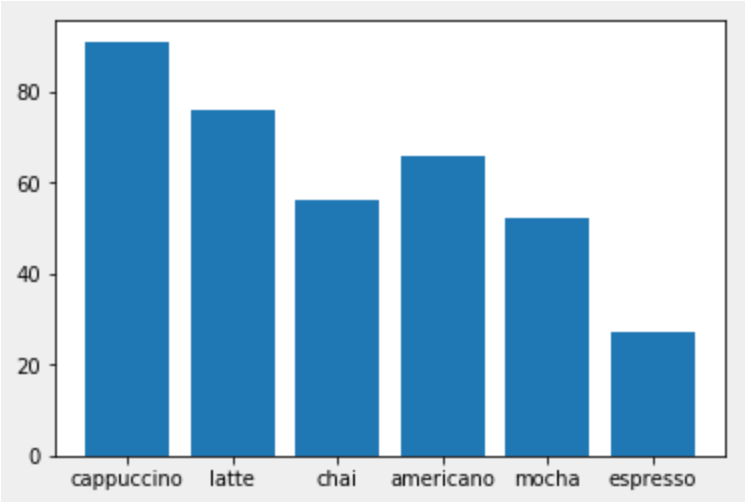
6		center left
---	--	-------------

7		center right
---	--	--------------

8		lower center
---	--	--------------

9		upper center
---	--	--------------

10		center
----	--	--------



Summary Statistics

Summary statistics are an important component of Exploratory Data Analysis (EDA) because they allow a data analyst to condense a large amount of information into a small set of numbers that can be easily interpreted. In order to decide what kind of summary statistic to use, it is important to consider two things:

- The question (and how many variables that question involves)
- The data (is it quantitative or categorical?)

Univariate Statistics

Summary statistics that focus on a single variable are called univariate statistics. They are useful for answering questions about a single feature in tabular data.

Quantitative Variables

Mean: Also called the “average”; calculated as the sum of all values divided by the number of values.

Median: The middle value of the variable when sorted.

Mode: The most frequent value in the variable.

Trimmed Mean: The mean excluding x percent of the lowest and highest data points.

Central Location

The central location (also called central tendency) is often used to communicate the “typical” value of a variable. Recall that there are a few different ways of calculating the central location:

Mean: Also called the “average”; calculated as the sum of all values divided by the number of values.

Median: The middle value of the variable when sorted.

Mode: The most frequent value in the variable.

Trimmed Mean: The mean excluding x percent of the lowest and highest data points.

Spread

Spread, or dispersion, describes the variability within a feature. This is important because it provides context for measures of central location. Like the central location measures, there are a few values that can describe the spread:

Range: The difference between the maximum and minimum values in a variable.

Inter-Quartile Range (IQR): The difference between the 75th and 25th percentile values.

Variance: The average of the squared distance from each data point to the mean.

Standard Deviation (SD): The square root of the variance.

Mean Absolute Deviation (MAD): The mean absolute value of the distance between each data point and the mean.

Categorical Variables

Categorical variables can be either ordinal (ordered) or nominal (unordered). For ordinal categorical variables, we may still want to summarize central location and spread. However, because ordinal categories are not necessarily evenly spaced (like numbers), we should NOT calculate the mean of an ordinal categorical variable (or anything that relies on the mean, like variance, standard deviation, and MAD).

For nominal categorical variables (and ordinal categorical variables), another common numerical summary statistic is the frequency or proportion of observations in each category. This is often reported using a frequency table and can be visualized using a bar plot.

Common numerical summary statistic:

Frequency

or

Proportion

Categorical variables either ordinal (ordered) or nominal (unordered) **we may** still want to summarize **central location** and **spread**.

However, because ordinal categories are not necessarily evenly spaced (like numbers), we should NOT calculate the mean of an ordinal categorical variable (or anything that relies on the mean, like variance, standard deviation, and MAD).

Bivariate Statistics

In contrast to univariate statistics, bivariate statistics are used to summarize the relationship between two variables. They are useful for answering questions like:

Do manual transmission cars tend to cost more or less than automatic transmission?

Do older cars tend to cost less money?

Are automatic transmission cars more likely to be sold by individuals or dealers?

Depending on the types of variables we want to summarize a relationship between, we should choose different summary statistics.

One Quantitative Variable and One Categorical Variable

If we want to know whether manual transmission cars tend to cost more or less than automatic transmission cars, we are interested in the relationship between transmission (categorical) and selling_price (quantitative). To answer this question, we can use a **mean** or **median** difference.

Use the different of categories from:

Mean

or

Median

Two Quantitative Variables

If we want to know whether older cars tend to cost less money, we are interested in the relationship between year and selling_price, both of which are quantitative. To answer this question, we can use the Pearson correlation.

Use:

Correction - Cross-tabulation or two-way table

Two Categorical Variables

If we want to know whether automatic transmission cars are more likely to be sold by individuals or dealers, we are interested in the relationship between transmission and seller_type, both of which are categorical. We can explore this relationship using a contingency table and the Chi-Square statistic.

Use:

Contingency table - Pearson

or

Chi-Square statistic

Group Project: Life Expectancy and GDP

You can learn a lot on your own, but you can learn even more when you collaborate with others!

Group Projects in the Data Science Paths

As part of this Path, you will have the opportunity to connect with other learners to practice and apply your new skills. This is optional! We will sometimes prompt you to work with peers, but you should feel free to work with others as often as you like to gain practice!

How to Conduct a Group Project

Step One – Understand the Project

Select what you would like to work on together. We suggest [Life Expectancy and GDP](#), which will help you to collaborate on creating a full data analysis project using pandas, Matplotlib, and seaborn. Alternatively, you can work on your own data analysis project that uses pandas, Matplotlib, and seaborn.

Assess the prompt or project requirements:

What technical skill sets do you need to complete the project? Will you need data analysis, data engineering, visualization and subject matter expertise?

How advanced a skillset in each is required and how much work is there to go around?

From here you'll know which roles your project needs and how many people are necessary. Depending on the size and complexity of the project, you may only need one other person or you may need four or more collaborators.

Step Two – Find Your Teammates

Find collaborators! [Visit the projects forum](#). You can also find collaborators in the [data-science channel in Codecademy's Discord server](#). If you are still looking for others to work with, check out the [Codecademy Facebook group](#), your local [Codecademy Chapter](#), or local meetups.

When you arrive, be sure to introduce yourself with the following information:

What % of the path you have completed

What timezone or country you're in

A link to your Codecademy Profile

One fun fact about yourself

Step Three – Build Your Team

Now that you've found people to work with, the next step is to figure out how each of you would like to contribute to this project.

These are some different roles that you and your teammates could take. Also, one person can have more than one role and more than one person could have the same role.

Product Manager who also codes. This person makes decisions about what features will be built and in which order. Data Science Product Managers keep the team organized and on track to be sure everything gets done. To learn a lot more about being a team leader, [please see this post](#)

Data Engineer. This person manages the data, making sure it is clean, reliable, and accessible. This role typically maintains the data standards and ensures consistency across a project.

Data Scientist. This person draws on one or more data sets to generate insights, and uses hypothesis testing to answer questions. They may create visualizations, conduct exploratory analyses, or use summary statistics to describe data. They may also use predictive statistics and machine learning to make predictions from the data.

Subject Matter Expert who also codes. This person situates datasets within their real-world context to ensure that the team is asking the right questions and interpreting the results accurately.

Think you may want to build something more complex? [Check out this article to see what a more specialized team looks like.](#)

Step Four – Get to Work

Get to it! Have a kick-off meeting, ideally over a video chat platform, so you can all meet one another and discuss the project.

Remember that if and when you get stuck, you can lean on each other to get through problems. In the real world, analysts troubleshoot their own work first with tried and true methods, but they often ask teammates for help too. If your teammates can't help, reach out in the forums or in [the Discord server](#).

Sometimes, life happens. Even in the “real world” workplace, people get sick or leave projects. If a learner leaves and you need a replacement, post in the Discord server and the community will try to help you out.

Step Five – Share!

If you completed the [Life Expectancy and GDP](#) Project, share it with fellow learners on its [forum page](#). If you choose to do another project, post your project in [the Projects category](#) and share it with the other learners in the forums or on Discord! Getting feedback is a vital part of the development process and will help you grow your skills. Remember to give back to other people wanting feedback or guidance, not just because it's a good thing to do, but you'll also benefit by explaining to others what you've learned.

If you're feeling generous, don't just share your work but share your insights and guidance for the next team tackling something similar. Paying it forward and giving back is how the developer world works – [entire systems have been built this way](#).

What is a Portfolio Project?

This project is a little different from other Codecademy projects you've encountered. In this project, you will bring together what you have learned in previous lessons to build a project in your own development environment and publish it to the web!



Create your own files
outside of the
Codecademy platform



Write your own code
with less guided
instructions



Use common project
management processes to
track your progress

How Do Portfolio Projects Work?

We'll provide you with high-level tasks to guide your project to completion, but you will be responsible for deciding how to implement them in your code.

There are many possible ways to correctly fulfill all of these requirements, and you should expect to use the internet, Codecademy, and other resources when you encounter a problem that you cannot easily solve.

Note that there are hints that can assist you, but they will only provide one potential implementation. Do not worry if your program looks different from ours!



Project Prompt

Life Expectancy and GDP

During this project, you will analyze, prepare, and plot data about GDP and life expectancy from the World Health Organization in order to answer questions in a meaningful way.

After you perform your analysis, you will create a blog post to share your findings on the World Health Organization website.

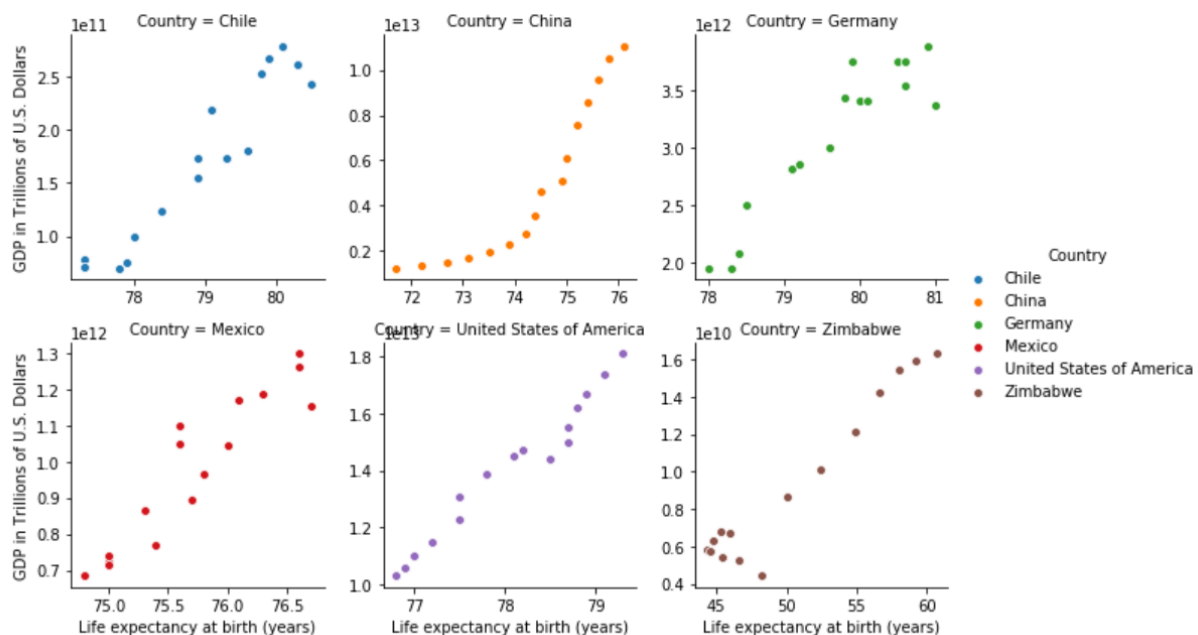
Click Begin to get started!

For this project, you will analyze data on GDP and life expectancy from the World Health Organization and the World Bank to try and identify the relationship between the GDP and life expectancy of six countries.

During this project, you will analyze, prepare, and plot data in order to answer questions in a meaningful way.

After you perform your analysis, you'll be creating a blog post to share your findings on the World Health Organization website.

Example Project



Project Objectives:

- Complete a project to add to your portfolio
- Use seaborn and Matplotlib to create visualizations
- Become familiar with presenting and sharing data visualizations
- Preprocess, explore, and analyze data

Prerequisites:

- Data Acquisition
- Data Visualization
- Hypothesis Testing
- Summarizing Quantitative Data
- Data Wrangling and Tidying
- Data Manipulation with Pandas

https://www.codecademy.com/paths/data-science-foundations/tracks/dsf-data-visualization-portfolio-project/modules/dsf-data-visualization-portfolio-project/kanban_projects/life-expectancy-and-gdp-portfolio-project

Introduction: Data Wrangling and Tidying

See what's covered in [Data Wrangling and Tidying](#).

Goals of this Unit

The goal of this unit is to learn common industry tools and best practices for data wrangling and tidying.

After this unit, you will be able to:

- Understand the difference between data wrangling and data tidying
- Identify and select specific data using regular expressions
- Clean datasets using pandas
- Apply Tidy Data best practices to facilitate data analysis
- Begin developing expertise in exploratory data analysis

You will put all of this knowledge into practice with an upcoming Portfolio Project. You can complete the Portfolio Project either in parallel with or after taking the prerequisite content—it's up to you!

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

Introduction to Data Wrangling and Tidying

A brief overview of the data wrangling process and tidy data

Frequently when we work with data, we encounter unstructured and/or *messy data*. Messy data can take a variety of forms. This could mean any of the following:

- The columns are mislabeled or do not have variable names.

- The dataset contains nonsensical data.

- Variables are stored in both the columns and rows.

While the data may be messy, it is still extremely informative. We need to clean, transform, and sometimes manipulate the data structure to gain any insights. This process is often called *data wrangling* or *data munging*.

At the final stages of the data wrangling process, we will have a dataset that we can easily use for modeling purposes or for visualization purposes. This is a *tidy* dataset where each column is a variable and each row is an observation.

Data Wrangling

Let's look at a subset of restaurant inspections from the [New York City Department of Health and Mental Hygiene](#) (NYC DOHMH) to work through some data wrangling processes. The data includes seven different columns with information about a restaurant's location and health inspection. Here is a description of the dataset's variables.

Pos.	Var. Name	Var. Description
------	-----------	------------------

```

--
0      DBA      Restaurant name
1      BORO      Borough
2      CUISINE DESCRIPTION      Type of cuisine
3      GRADE      Letter grade
4      LATITUDE      Latitude coordinates of restaurant
5      LONGITUDE      Longitude coordinates of restaurant
6      URL      URL link to restaurant's website

```

Let's use the `read_csv()` function in pandas to load our dataset as a pandas dataframe and take a look at the first 10 rows out of the 27 total.

```

import pandas as pd
restaurants = pd.read_csv("DOHMH_restaurant_inspections.csv")

# the .head(10) function will show us the first 10 rows in our dataset
print(restaurants.head(10))

```

Output:

	DBA	BORO	CUISINE DESCRIPTION	GRADE	Latitude	Longitude	URL
0	RICO POLLO II RESTAURANT	Brooklyn	Spanish	A	40.683726	-73.871872	https://www.seamless.com/menu/rico-pollo-ii-re...
1	ABC BEER CO.	Manhattan	Sandwiches	NaN	40.723757	-73.979079	https://www.abcbeer.co/
2	JADE GARDEN	Queens	Chinese	A	40.673972	-73.839694	https://www.seamless.com/menu/jade-garden-9627...
3	IHOP	Manhattan	Pancakes/Waffles	A	0.000000	0.000000	https://www.ihop.com/en
4	YEAH SHANGHAI DELUXE	Manhattan	Chinese	NaN	40.715528	-73.998539	http://www.yeah-shanghai-deluxe.com/
5	SEAMORE'S	Manhattan	Seafood	NaN	40.754501	-73.975950	https://www.seamores.com/
6	SEAMORE'S	Manhattan	Seafood	NaN	40.754501	-73.975950	https://www.seamores.com/
7	DANTE	Manhattan	Italian	NaN	40.728771	-74.001533	https://www.dante-nyc.com/
8	GINA'S ITALIAN BAKERY & PASTRY SHOP	Bronx	Bakery	A	40.854434	-73.865133	NaN
9	BONBON BAKERY	Queens	Bakery	A	40.733068	-73.804645	NaN

The `.shape` method in pandas identifies the number of rows and columns in our dataset as (rows, columns):

```
restaurants.shape
```

Output:

```
(27, 7)
```

When we look closely at the table, we see some missing data. In both `GRADE` and `URL` columns, we have missing values marked as NaNs. In the `Latitude` and `Longitude` columns, we have a missing set of coordinates marked as (0.000, 0.000) for IHOP. (0.000, 0.000) is the label for missing coordinates because no restaurants in New York City are at the equator. Other common indicators used for missing values are values that are NA and -.

Preliminary data cleaning

There are also duplicate rows for the restaurant labeled Seamore's. To remove any duplicate rows, we can use the `drop_duplicates()` function

```
# the .drop_duplicates() function removes duplicate rows
restaurants = restaurants.drop_duplicates()

# the .head(10) function will show us the first 10 rows in our dataset
print(restaurants.head(10))
```

Output:

	DBA	BORO	CUISINE DESCRIPTION	GRADE	Latitude	Longitude	URL
0	RICO POLLO II RESTAURANT	Brooklyn	Spanish	A	40.683726	-73.871872	https://www.seamless.com/menu/rico-pollo-ii-re...
1	ABC BEER CO.	Manhattan	Sandwiches	NaN	40.723757	-73.979079	https://www.abcbeer.co/
2	JADE GARDEN	Queens	Chinese	A	40.673972	-73.839694	https://www.seamless.com/menu/jade-garden-9627...
3	IHOP	Manhattan	Pancakes/Waffles	A	0.000000	0.000000	https://www.ihop.com/en
4	YEAH SHANGHAI DELUXE	Manhattan	Chinese	NaN	40.715528	-73.998539	http://www.yeah-shanghai-deluxe.com/
5	SEAMORE'S	Manhattan	Seafood	NaN	40.754501	-73.975950	https://www.seamores.com/
7	DANTE	Manhattan	Italian	NaN	40.728771	-74.001533	https://www.dante-nyc.com/
8	GINA'S ITALIAN BAKERY & PASTRY SHOP	Bronx	Bakery	A	40.854434	-73.865133	NaN
9	BONBON BAKERY	Queens	Bakery	A	40.733068	-73.804645	NaN
10	YOLANDA RESTAURANT	Brooklyn	Spanish	NaN	40.680254	-73.892960	NaN

The `.shape` method again:

```
restaurants.shape
```

Output:

```
(25, 7)
```

After dropping duplicate rows, we are left with 25 rows and can now see “YOLANDA RESTAURANT” at the bottom when looking at our dataset’s first 10 rows.

If we look at the first four columns of our data: `DBA`, `BORO`, `CUISINE DESCRIPTION`, and `GRADE`. These column names are all capitalized, while the columns following it use both uppercase and lowercase. To have some consistency across column names, we will iterate over the column names of our dataset and convert them all to lowercase using the `map()` and `lower()` functions. We also need to make sure to include the `str` function to identify that we are working with strings.

```
# map() applies the str.lower() function to each of the columns in our
dataset to convert the column names to all lowercase
restaurants.columns = map(str.lower, restaurants.columns)

# the .head(10) function will show us the first 10 rows in our dataset
print(restaurants.head(10))
```

Output:

	dba	boro	cuisine description	grade	latitude	longitude	url
0	RICO POLLO II RESTAURANT	Brooklyn	Spanish	A	40.683726	-73.871872	https://www.seamless.com/menu/rico-pollo-ii-re...
1	ABC BEER CO.	Manhattan	Sandwiches	NaN	40.723757	-73.979079	https://www.abcbeer.co/
2	JADE GARDEN	Queens	Chinese	A	40.673972	-73.839694	https://www.seamless.com/menu/jade-garden-9627...
3	IHOP	Manhattan	Pancakes/Waffles	A	0.000000	0.000000	https://www.ihop.com/en
4	YEAH SHANGHAI DELUXE	Manhattan	Chinese	NaN	40.715528	-73.998539	http://www.yeah-shanghai-deluxe.com/
5	SEAMORE'S	Manhattan	Seafood	NaN	40.754501	-73.975950	https://www.seamores.com/
7	DANTE	Manhattan	Italian	NaN	40.728771	-74.001533	https://www.dante-nyc.com/
8	GINA'S ITALIAN BAKERY & PASTRY SHOP	Bronx	Bakery	A	40.854434	-73.865133	NaN
9	BONBON BAKERY	Queens	Bakery	A	40.733068	-73.804645	NaN
10	YOLANDA RESTAURANT	Brooklyn	Spanish	NaN	40.680254	-73.892960	NaN

You may have noticed that the first column of the dataset is called `DBA`, but we know it is a column with restaurant names. We can use the `rename()` function and a dictionary to relabel our columns. While we are renaming our columns, we might also want to shorten the `cuisine description` column to just `cuisine`.

```
# axis=1` refers to the columns, `axis=0` would refer to the rows
# In the dictionary the key refers to the original column name and the
value refers to the new column name {'oldname1': 'newname1',
'oldname2': 'newname2'}
restaurants = restaurants.rename({'dba': 'name', 'cuisine
description': 'cuisine'}, axis=1)

# the .head(10) function will show us the first 10 rows in our dataset
print(restaurants.head(10))
```

Output:

	name	boro	cuisine	grade	latitude	longitude	url
0	RICO POLLO II RESTAURANT	Brooklyn	Spanish	A	40.683726	-73.871872	https://www.seamless.com/menu/rico-pollo-ii-re...
1	ABC BEER CO.	Manhattan	Sandwiches	NaN	40.723757	-73.979079	https://www.abcbeer.co/
2	JADE GARDEN	Queens	Chinese	A	40.673972	-73.839694	https://www.seamless.com/menu/jade-garden-9627...
3	IHOP	Manhattan	Pancakes/Waffles	A	0.000000	0.000000	https://www.ihop.com/en
4	YEAH SHANGHAI DELUXE	Manhattan	Chinese	NaN	40.715528	-73.998539	http://www.yeah-shanghai-deluxe.com/
5	SEAMORE'S	Manhattan	Seafood	NaN	40.754501	-73.975950	https://www.seamores.com/
7	DANTE	Manhattan	Italian	NaN	40.728771	-74.001533	https://www.dante-nyc.com/
8	GINA'S ITALIAN BAKERY & PASTRY SHOP	Bronx	Bakery	A	40.854434	-73.865133	NaN
9	BONBON BAKERY	Queens	Bakery	A	40.733068	-73.804645	NaN
10	YOLANDA RESTAURANT	Brooklyn	Spanish	NaN	40.680254	-73.892960	NaN

Data Types

Great! Now we have a dataset that is cleaner and much easier to read!. Let's take a look at each column's data types by appending `.dtypes` to our pandas dataframe.

```
restaurants.dtypes
```

Output:

```
name          object
boro          object
cuisine       object
grade         object
latitude      float64
longitude     float64
url           object
dtype: object
```

We have two types of variables: object and float64. `object` can consist of both strings or mixed types (both numeric and non-numeric), and `float64` are numbers with a *floating point* (ie. numbers with decimals). There are other data types such as `int64` (integer numbers), `bool` (True/False values), and `datetime64` (date and/or time values).

Since we have both continuous (float64) and categorical (object) variables in our data, it might be informative to look at the number of unique values in each column using the `nunique()` function.

```
# .nunique() counts the number of unique values in each column
restaurants.nunique()
```

Output:

```
name          25
boro           4
cuisine       15
grade          1
latitude      24
longitude     24
url           16
```

We see that our data consists of 4 boroughs in New York and 15 cuisine types. We know that we also have missing data in `url` from our initial inspection of the data, so the unique number of values in `url` might not be super informative. Additionally, we have corrected for duplicate restaurants, so the restaurant name, latitude, longitude, and url should be unique to each restaurant (unless, of course, there are some restaurant chains at different locations).

Missing Data

From our initial inspection of the data, we know we have missing data in `grade`, `url`, `latitude`, and `longitude`. Let's take a look at how the data is missing, also referred to as *missingness*. To do this we can use `isna()` to identify if the value is missing. This will give us a boolean and indicate if the observation in that column is missing (True) or not (False). We will also use `sum()` to count the number of missing values, where `isna()` returns True.

```
# counts the number of missing values in each column
restaurants.isna().sum()
```

Output:

name	0
boro	0
cuisine	0
grade	15
latitude	0
longitude	0
url	9

We see that there are missing values in the `grade` and `url` columns, but no missing values in `latitude` and `longitude`. However, coordinates at (0.000, 0.000) are not valid for any of the restaurants in our dataset, and we saw that these exist in our initial analysis. To handle this, we will replace the (0.000, 0.000) coordinates with NaN values.

We will use the `.where()` method from the `pandas` library to replace the invalid coordinates. The `.where()` method keeps the values specified in its first argument, and replaces all other values with `NaN`.

```
# here our .where() function replaces latitude values less than 40
with NaN values
restaurants['latitude'] =
restaurants['latitude'].where(restaurants['latitude'] > 40)

# here our .where() function replaces longitude values greater than -
70 with NaN values
restaurants['longitude'] =
restaurants['longitude'].where(restaurants['longitude'] < -70)

# .sum() counts the number of missing values in each column
restaurants.isna().sum()
```

Output:

name	0
boro	0
cuisine	0
grade	15
latitude	2
longitude	2
url	9

Now we see that latitude and longitude each have two missing data points thanks to replacing the (0.000, 0.000) values with `NaN` values.

Characterizing missingness with crosstab

Let's try to understand the missingness in the `url` column by counting the missing values across each borough. We will use the `crosstab()` function in `pandas` to do this.

The `crosstab()` computes the frequency of two or more variables. To look at the missingness in the `url` column we can add `isna()` to the column to identify if there is an NaN in that column. This will return a boolean, True if there is a NaN and False if there is not. In our crosstab, we will look at all the boroughs present in our data and whether or not they have missing url links.

```
pd.crosstab(  
  
    # tabulates the boroughs as the index  
    restaurants['boro'],  
  
    # tabulates the number of missing values in the url column as  
columns  
    restaurants['url'].isna(),  
  
    # names the rows  
    rownames = ['boro'],  
  
    # names the columns  
    colnames = ['url is na'])
```

Output:

url is na	False	True
boro		
Bronx	1	1
Brooklyn	2	4
Manhattan	11	2
Queens	2	2

We see that most of the restaurants in Manhattan in our dataset have restaurant links, while most restaurants in Brooklyn do not have url links.

Removing prefixes

It might be easier to read what url links are by removing the prefixes of the websites, such as "https://www.". We will use `str.lstrip()` to remove the prefixes. Similar to when we were working with our column names, we need to make sure to include the `str` function to identify that we are working with strings and `lstrip` to remove parts of the string from the left side.

```
# .str.lstrip('https://') removes the "https://" from the left side of
the string
restaurants['url'] = restaurants['url'].str.lstrip('https://')

# .str.lstrip('www.') removes the "www." from the left side of the
string
restaurants['url'] = restaurants['url'].str.lstrip('www.')

# the .head(10) function will show us the first 10 rows in our dataset
print(restaurants.head(10))
```

Output:

	name	boro	cuisine	grade	latitude	longitude	url
0	RICO POLLO II RESTAURANT	Brooklyn	Spanish	A	40.683726	-73.871872	seamless.com/menu/rico-pollo-ii-restaurant-335...
1	ABC BEER CO.	Manhattan	Sandwiches	NaN	40.723757	-73.979079	abcbeer.co/
2	JADE GARDEN	Queens	Chinese	A	40.673972	-73.839694	seamless.com/menu/jade-garden-9627-linden-blvd...
3	IHOP	Manhattan	Pancakes/Waffles	A	NaN	NaN	ihop.com/en
4	YEAH SHANGHAI DELUXE	Manhattan	Chinese	NaN	40.715528	-73.998539	yeah-shanghai-deluxe.com/
5	SEAMORE'S	Manhattan	Seafood	NaN	40.754501	-73.975950	seamores.com/
7	DANTE	Manhattan	Italian	NaN	40.728771	-74.001533	dante-nyc.com/
8	GINA'S ITALIAN BAKERY & PASTRY SHOP	Bronx	Bakery	A	40.854434	-73.865133	NaN
9	BONBON BAKERY	Queens	Bakery	A	40.733068	-73.804645	NaN
10	YOLANDA RESTAURANT	Brooklyn	Spanish	NaN	40.680254	-73.892960	NaN

Amazing! Our dataset is now much easier to read and use. We have identifiable columns and variables that are easy to work with thanks to our data wrangling process. We also corrected illogical data values and made the strings a little easier to read.

In this example, we worked with data that was rather *tidy*, in the sense that each row was an observation (a restaurant) and each column was a variable. However, what if our dataset was not tidy? What if our columns and rows needed reorganization?

Tidy Data

Let's take a look at a dataset that has information about the average annual wage for restaurant workers across New York City boroughs and New York City as a whole from the years 2000 and 2007. The data is from the [New York State Department of Labor](#), Quarterly Census of Employment and Wages, and only contains six total rows.

```
annual_wage = pd.read_csv("annual_wage_restaurant_boro.csv")
print(annual_wage)
```

	boro	2000	2007
0	Bronx	16392	16195
1	Brooklyn	16713	17017
2	Manhattan	27020	27080
3	Queens	16692	16883
4	Staten Island	15143	14951

Output:

There are three variables in this dataset: `borough`, `year`, and `average annual income`. However, we have values (2000 and 2007) in the column headers rather than variable names (`year` and `average annual income`). This is not ideal to work with, so let's fix it! We will use the `melt()` function in pandas to turn the current values (2000 and 2007) in the column headers into row values and add `year` and `avg_annual_wage` as our column labels.

```
annual_wage=annual_wage.melt(

    # which column to use as identifier variables
    id_vars=["boro"],

    # column name to use for "variable" names/column headers (ie.
    2000 and 2007)
    var_name=["year"],
```

```

# column name for the values originally in the columns 2000 and
2007
value_name="avg_annual_wage")

print(annual_wage)

```

	boro	year	avg_annual_wage
0	Bronx	2000	16392
1	Brooklyn	2000	16713
2	Manhattan	2000	27020
3	Queens	2000	16692
4	Staten Island	2000	15143
5	New York City	2000	23496
6	Bronx	2007	16195
7	Brooklyn	2007	17017
8	Manhattan	2007	27080
9	Queens	2007	16883
10	Staten Island	2007	14951
11	New York City	2007	23537

Output:

Now we have a tidy dataset where each column is a variable (borough, year, or average annual wage), and each row is an observation! This dataset will be much easier to work with moving forward!

You now have the tools to continue to gather more information about New York City restaurants to answer questions you are interested in exploring. For example, we could explore what variables are most predictive of receiving a passing health score. We would need to gather some more data and go through the data wrangling process to end up with tidy data that is ready for analysis. Happy coding!

Regular Expressions

- Literals
- Alternations - `cat|dog`
- Charters Sets
 - - # Regex: `[chr]at`
 - # Possible Solution 2
 - # Regex: `[^ems]at`
- Wild for Wildcards
- **Ranges** - `[1-9]`, `[a-z]`, `[A-Z]` - `[c-e][l-u][b-k]`
- **Shorthand Character Classes**
 - `\d`
 - `\w`
 - `\s`
 - etc
- **Grouping** 0 `(baboons|gorillas)`
- **Quantifiers - Fixed**
 - `\w{3}`
 - `\w{4,7}`
 - # Regex: `squea{3,5}k`
- **Anchors**
 - `^Monkeys: my mortal enemy$`
-

Dealing with Multiple Files

Often, you have the same data separated out into multiple files.

Let's say that we have a ton of files following the filename structure: 'file1.csv', 'file2.csv', 'file3.csv', and so on. The power of pandas is mainly in being able to manipulate large amounts of structured data. We want to be able to get all of the relevant information into one table so that we can analyze the aggregate data.

We can combine the use of `glob`, a Python library for working with files, with `pandas` to organize this data better. `glob` can open multiple files using shell-style wildcard matching to get the filenames:

```
import glob
import pandas as pd

files = glob.glob("file*.csv")

df_list = []
for filename in files:
    data = pd.read_csv(filename)
    df_list.append(data)

df = pd.concat(df_list)

print(files)
```

This code goes through any file that starts with 'file' and has an extension of .csv. It opens each file, reads the data into a DataFrame, and then concatenates all of those DataFrames together.

`.head()` – display the first 5 rows of the table

`.info()` – display a summary of the table

`.describe()` – display the summary statistics of the table

`.columns` – display the column names of the table

`.value_counts()` – display the distinct values for a column

```
import pandas as pd
import glob

student_files = glob.glob("exams*.csv")

df_list = []

for filename in student_files:
    df_list.append(pd.read_csv(filename))

students = pd.concat(df_list)

print(students.head())
print(len(students))
```

Off-Platform Project: Stack Overflow Survey Trends

Understanding developer trends in Stack Overflow survey data

You've learned a lot about missing data and how to handle it, so now it's time to put these strategies into action!

Project Overview

You work for a staffing agency that specializes in finding qualified candidates for development roles. One of your latest clients is growing rapidly and wants to understand what kinds of developers they can hire, and to understand general trends of the technology market. Your organization has access to this [Stack Overflow dataset](#), which consists of survey responses by developers all over the world for the last few years.

Your project is to put together several statistical analyses about the community to educate your client about the potential hiring market for their company.

Project Steps

Explore data

You decide to start by performing some Exploratory Data Analysis (EDA). This will provide you with a high-level understanding of the data fields, as well as help you identify which columns have missing data. In this case, you load the dataset into a `pandas` DataFrame and call it `df`. Take a moment to explore which columns you have in the data.

```
import pandas as pd

df = pd.read_csv('developer_dataset.csv')

print(df.columns)
```

Output:

```
Index(['RespondentID', 'Year', 'Country', 'Employment',
      'UndergradMajor', 'DevType', 'LanguageWorkedWith',
      'LanguageDesireNextYear', 'DatabaseWorkedWith',
      'DatabaseDesireNextYear', 'PlatformWorkedWith',
      'PlatformDesireNextYear', 'MainBranch', 'Hobbyist',
      'EdLevel', 'OrgSize', 'YearsCodePro', 'JobSeek',
      'JobFactors', 'ConvertedComp', 'WorkWeekHrs', 'OpSys',
      'WelcomeChange', 'NEWJobHunt', 'NEWJobHuntResearch', 'NEWLearn'],
      dtype='object')
```

At an initial glance, you notice the following kinds of information: A variety of columns that identify the person (`RespondentID`, `Year`, `Country`) Information about their experiences (`LanguageWorkedWith`, `DatabaseWorkedWith`, `UndergradMajor`, etc.) Information about what they might want to do in the future (`LanguageDesireNextYear`, `DatabaseDesireNextYear`, etc.)

At this point, you want a good understanding of how much data you have. Since this is a survey where each question is optional, you don't expect every column to have a full set of data.

Run `df.count()` to see a row count for each column. It should look something like this:

RespondentID	111209
Year	111209
Country	111209
Employment	109425
UndergradMajor	98453
DevType	100433
LanguageWorkedWith	102018
LanguageDesireNextYear	96044
DatabaseWorkedWith	85859
DatabaseDesireNextYear	74234
PlatformWorkedWith	91609
PlatformDesireNextYear	85376
Hobbyist	68352
OrgSize	54804
YearsCodePro	94793
JobSeek	60556
ConvertedComp	91333
WorkWeekHrs	51089
NEWJobHunt	19127
NEWJobHuntResearch	18683
NEWLearn	24226

From here, you can perform some basic summary statistics on the dataset. This will allow you to understand things like:

This will only work for numerical columns, but that will still be helpful. Use the following code:

```
df.describe()
```

You should see the following output:

	RespondentID	Year	YearsCodePro	ConvertedComp	WorkWeekHrs
count	111209.00000	111209.00000	94793.00000	9.133300e+04	51089.00000
mean	19262.039709	2018.854832	9.547045	1.251777e+05	41.051670
std	11767.011322	0.777503	7.548931	2.461218e+05	13.833929
min	1.000000	2018.000000	0.000000	0.000000e+00	1.000000
25%	9268.000000	2018.000000	4.000000	4.600000e+04	40.000000
50%	18535.000000	2019.000000	8.000000	7.900000e+04	40.000000
75%	28347.000000	2019.000000	14.000000	1.200000e+05	42.000000
max	42857.000000	2020.000000	50.000000	2.000000e+06	475.000000

Based on the above information, what observations can you make about the dataset?

Delete highly missing data

You notice this dataset has a number of columns with a significant amount of missing data. With this much missing data, it is unlikely that any statistical analysis using that data would be accurate and representative of the developers who filled out the survey. Luckily, you recall that you can safely remove columns with ~60% or more missing data.

Run the below code to see the percentage missing data for each column.

```
maxRows = df['RespondentID'].count()
```

```
print('% Missing Data:')
```

```
print((1 - df.count() / maxRows) * 100)
```

Your output should look like this:

% Missing Data:

RespondentID	0.000000
Year	0.000000
Country	0.000000
Employment	1.604187
UndergradMajor	11.470295
DevType	9.689863
LanguageWorkedWith	8.264619
LanguageDesireNextYear	13.636486
DatabaseWorkedWith	22.794918
DatabaseDesireNextYear	33.248208
PlatformWorkedWith	17.624473
PlatformDesireNextYear	23.229235
Hobbyist	38.537349
OrgSize	50.719816
YearsCodePro	14.761395
JobSeek	45.547573
ConvertedComp	17.872654
WorkWeekHrs	54.060373
NEWJobHunt	82.800852
NEWJobHuntResearch	83.200101
NEWLearn	78.215792

Based on the above numbers, you assume that it is safe to remove the following columns:

NEWJobHunt

NEWJobHuntResearch

NEWLearn

Use pandas to drop those DataFrame columns.


```
df.drop(['NEWJobHunt', 'NEWJobHuntResearch', 'NEWLearn'],
        axis=1,
        inplace=True)
```

Analyze developers by country

Start thinking about the questions you want to ask of the data. You decide to investigate the distribution of employment and developer type from a geographical (i.e. `Country`) perspective.

Both the `Employment` and `DevType` fields have missing data, but not a very significant amount, both with less than 10% missing. This is going to be foundational for your analyses moving forward, so you want to ensure that there are no missing data points.

Determine what kind of missing data you have for employment and developer type. One way to do that is check, at a country level, where the data is missing for each field:

```
import seaborn as sns
import matplotlib.pyplot as plt

df[['RespondentID', 'Country']].groupby('Country').count()

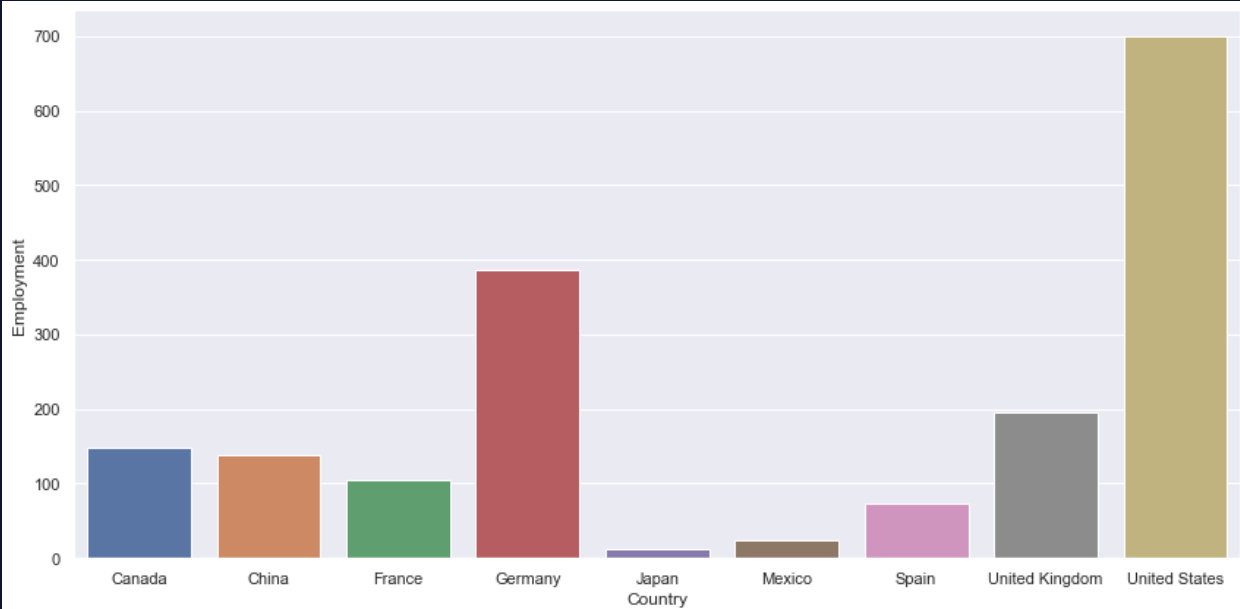
missingData =
df[['Employment', 'DevType']].isnull().groupby(df['Country']).sum().reset_index()

A=sns.catplot(
    data=missingData, kind="bar",
    x="Country", y="Employment",
    height = 6, aspect = 2)
B=sns.catplot(
    data=missingData, kind="bar",
    x="Country", y="DevType",
    height = 6, aspect = 2)
```

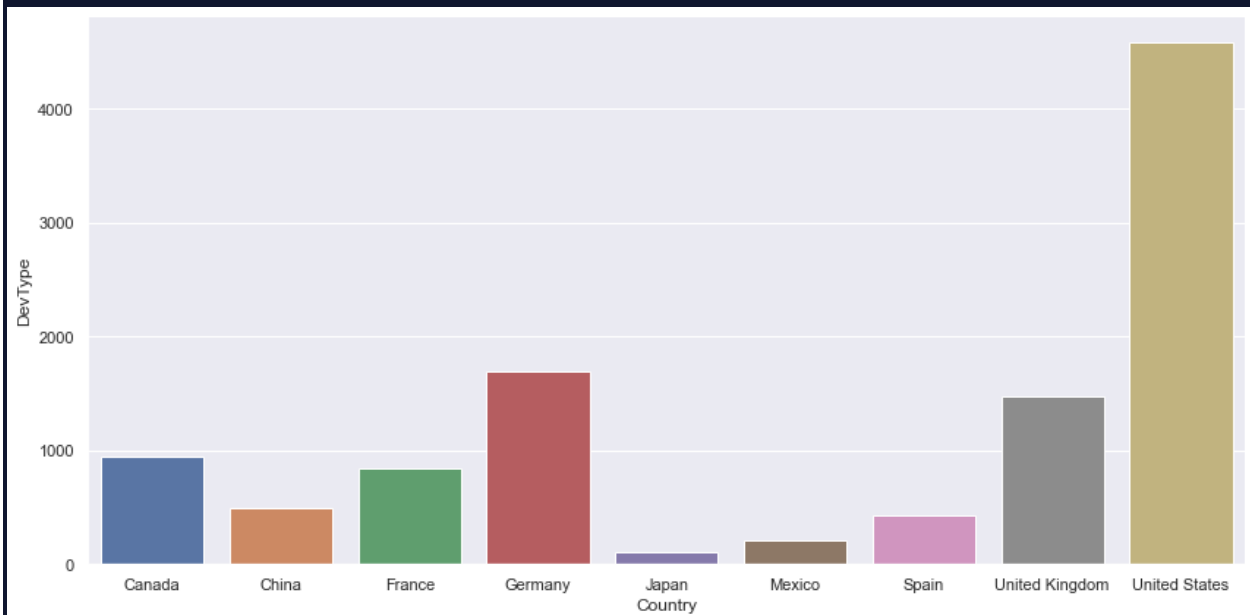
Output:

Country	RespondentID
Canada	8979
China	2072
France	6861
Germany	16215
Japan	1049
Mexico	1918
Spain	4534
United Kingdom	15854
United States	53727

Catplot [A](#) output:



Catplot [B](#) output:



Note: You can click on images to enlarge them.

As we can see from the above plots, the data doesn't appear to be missing for any country significantly more than any other. Using your domain knowledge, you understand that the missing data appears to scale with the relative size of each country (e.g. there is more missing data in the United States vs. Japan because there will be more respondents there). You also note that the United States and Germany have significantly more developers (on average) than the other countries, explaining why they have more missing data points.

You determine that the missing data for these two columns can be categorized as [MCAR](#). This means you can safely delete the rows that have missing data in these columns! This is a prime example of where you can employ Pairwise Deletion to only delete rows that have missing data for either

[Employment](#) or [DevType](#):

```
df.dropna(subset = ['Employment', 'DevType'],  
          inplace = True,  
          how = 'any')
```

Now you can analyze the distribution of employment and developer types by country. You decide to aggregate the employment data by key developer roles that align with major parts of the development lifecycle:

```
empfig = sns.catplot(x="Country", col="Employment",
                    data=df, kind="count",
                    height=6, aspect=1.5);

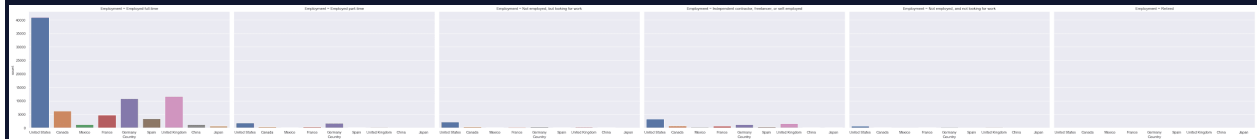
# Focus on a few of the key developer types outlined in the Stack
# Overflow survey
devdf = df[['Country', 'DevType']]
devdf.loc[devdf['DevType'].str.contains('back-end'), 'BackEnd'] = True
devdf.loc[devdf['DevType'].str.contains('front-end'), 'FrontEnd'] =
True
devdf.loc[devdf['DevType'].str.contains('full-stack'), 'FullStack'] =
True
devdf.loc[devdf['DevType'].str.contains('mobile'), 'Mobile'] = True
devdf.loc[devdf['DevType'].str.contains('administrator'), 'Admin'] =
True

devdf = devdf.melt(id_vars=['Country'],
                  value_vars=['BackEnd', 'FrontEnd', 'FullStack', 'Mobile', 'Admin'],
                  var_name='DevCat',
                  value_name='DevFlag')

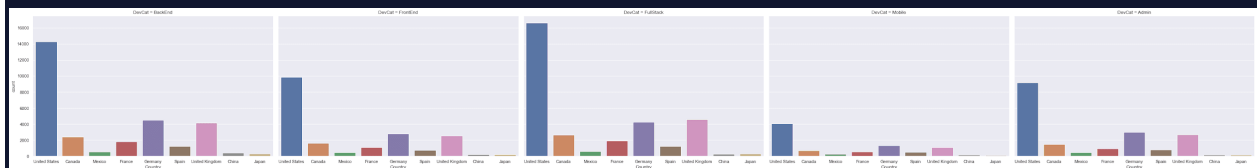
devdf.dropna(how='any', inplace=True)
```

```
devFig = sns.catplot(x="Country", col="DevCat",
                    data=devdf, kind="count",
                    height=6, aspect=1.5);
```

Catplot `empfig` output:



Catplot `devFig` output:



Note: You can click on images to enlarge them.

You see that the vast majority of respondents are employed full-time. Since these developers are mainly employed, this data will be relevant for a client who wants to see what developers look for in a potential job. You also see that the majority of developers will have skill sets in front-end, back-end, or full-stack development. This is interesting, and shows that the market values developers who can excel in at least a major part of the development lifecycle, if not the entire stack.

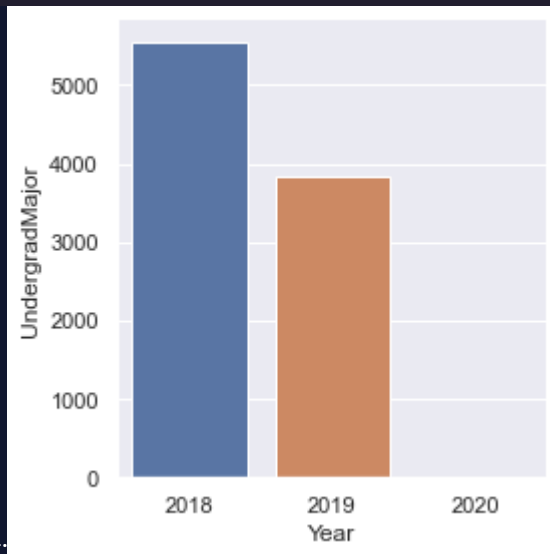
Investigate developer undergraduate majors

You decide to dive into the background for each type of developer to see trends in their educational experience that ultimately led to a career with technology. In particular, you look at the overall trend of majors year over year for respondents. As you saw before, you are missing about 11% of the data for `UndergradMajor`. Why do you think this data is missing? Could something have happened over the course of these three years? Is the fact that data is missing accurate?

To test your theory, take a look at the distribution of majors over each year:

```
missingUndergrad =
df['UndergradMajor'].isnull().groupby(df['Year']).sum().reset_index()
```

```
sns.catplot(x="Year", y="UndergradMajor",  
            data=missingUndergrad, kind="bar",  
            height=4, aspect=1);
```



Catplot output:

Note: You can click on images to enlarge them.

You see that all of the data for 2020 undergrad majors is filled in, indicating that each participant in these surveys had some level of decision for their undergrad major. For the purposes of your analysis, you are most interested in what major a person ultimately landed on, as this would be the educational background they would carry into a job search. You want to carry that value backwards for each participant to fill in any missing data. This is a great use for one of our Single Imputation techniques: NOCB! Fill in the gaps using NOCB:

```
# Sort by ID and Year so that each person's data is carried backwards  
correctly  
df = df.sort_values(['RespondentID', 'Year'])  
  
df['UndergradMajor'].bfill(axis=0, inplace=True)
```

From here, you analyze the major distribution for each year, using a vertical bar chart visualization:

```

# Key major groups outlined in the Stack Overflow survey
majors = ['social science', 'natural science', 'computer
science', 'development', 'another engineering', 'never declared']

edudf = df[['Year', 'UndergradMajor']]
edudf.dropna(how='any', inplace=True)
edudf.loc[edudf['UndergradMajor'].str.contains('(?!i)social science'),
'SocialScience'] = True
edudf.loc[edudf['UndergradMajor'].str.contains('(?!i)natural science'),
'NaturalScience'] = True
edudf.loc[edudf['UndergradMajor'].str.contains('(?!i)computer
science'), 'ComSci'] = True
edudf.loc[edudf['UndergradMajor'].str.contains('(?!i)development'),
'ComSci'] = True
edudf.loc[edudf['UndergradMajor'].str.contains('(?!i)another
engineering'), 'OtherEng'] = True
edudf.loc[edudf['UndergradMajor'].str.contains('(?!i)never declared'),
'NoMajor'] = True

edudf = edudf.melt(id_vars=['Year'],

value_vars=['SocialScience', 'NaturalScience', 'ComSci', 'OtherEng', 'NoMa
jor'],

    var_name='EduCat',
    value_name='EduFlag')

edudf.dropna(how='any', inplace=True)
edudf = edudf.groupby(['Year', 'EduCat']).count().reset_index()

eduFig = sns.catplot(x="Year", y='EduFlag', col="EduCat",
                    data=edudf, kind="bar",
                    height=6, aspect=1.5);

```

Catplot eduFig output:



Note: You can click on images to enlarge them.

You notice that the vast majority of people who enter the workforce for development have some background in a Computer Science major. Interestingly, however, the number of Computer Science majors significantly declined over the years surveyed, indicating that there could be other majors that have successfully entered the workforce for their desired job. This would require further analysis and could allow an individual to pursue a separate education path and still end up in some kind of developer role.

Examine the relationship between years of experience and compensation

At this point, you have studied the demographics of developers around the world, from where they live to the education paths they have taken. Now, you turn your focus to the various aspects that would influence the job-hunting process.

Years of experience are an important metric when looking to understand the general skill and technical capabilities of a potential candidate. Compensation is also important for our client to understand what the “going rate” for a particular developer is in today’s market. You might assume that there is a strong correlation between experience and job compensation, making it an excellent hypothesis to explore.

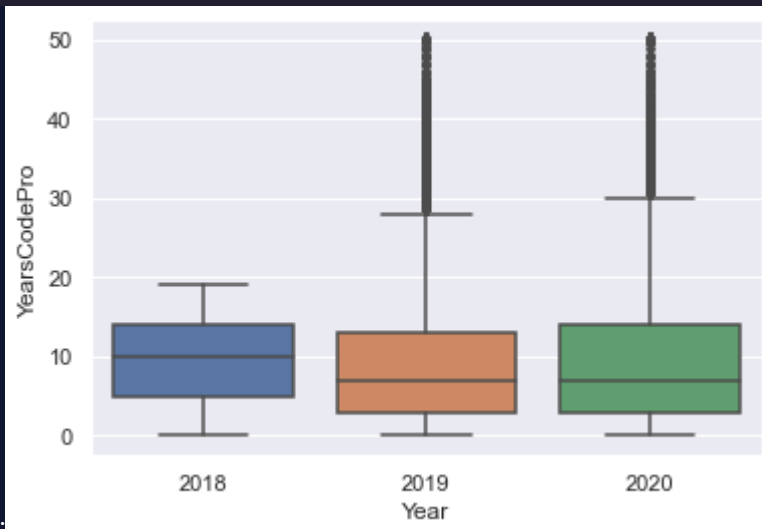
In order to understand a bit about the data for each of these two fields, perform some more exploratory analysis:

```
compFields = df[['Year', 'YearsCodePro', 'ConvertedComp']]

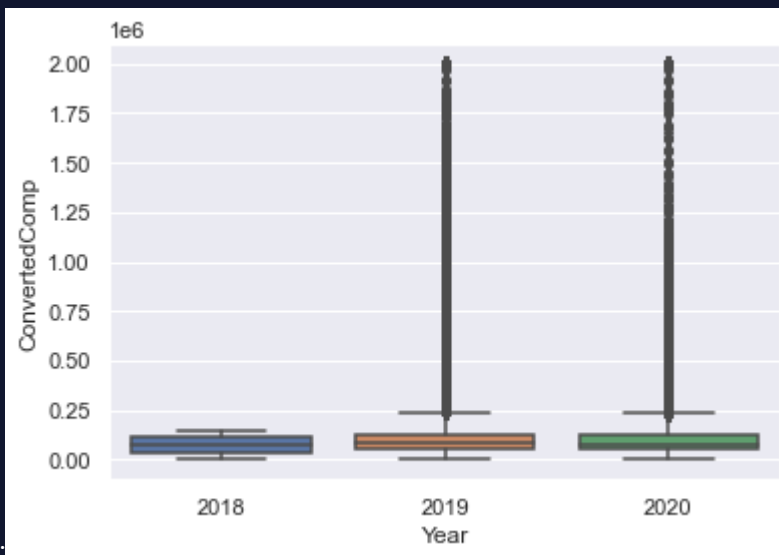
D = sns.boxplot(x="Year", y="YearsCodePro",
                data=compFields)
```



```
E = sns.boxplot(x="Year", y="ConvertedComp",  
               data=compFields)
```



Boxplot D output:



Boxplot E output:

Note: You can click on images to enlarge them.

You see that although there are some outlier data points for each column, the overall distribution is fairly consistent year-over-year. This indicates that there is a strong correlation between the data points, which should tell a good story about how experience can translate into compensation. Since

there is a clear trend with the data points, you decide the best method for filling in the missing data for these two columns is through Multiple Imputation:

```
import numpy as np
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
from sklearn.model_selection import train_test_split

imputedf = df[['YearsCodePro', 'ConvertedComp']]

traindf, testdf = train_test_split(imputedf, train_size=0.1)

# Create the IterativeImputer model to predict missing values
imp = IterativeImputer(max_iter=20, random_state=0)

# Fit the model to the the test dataset
imp.fit(imputedf)

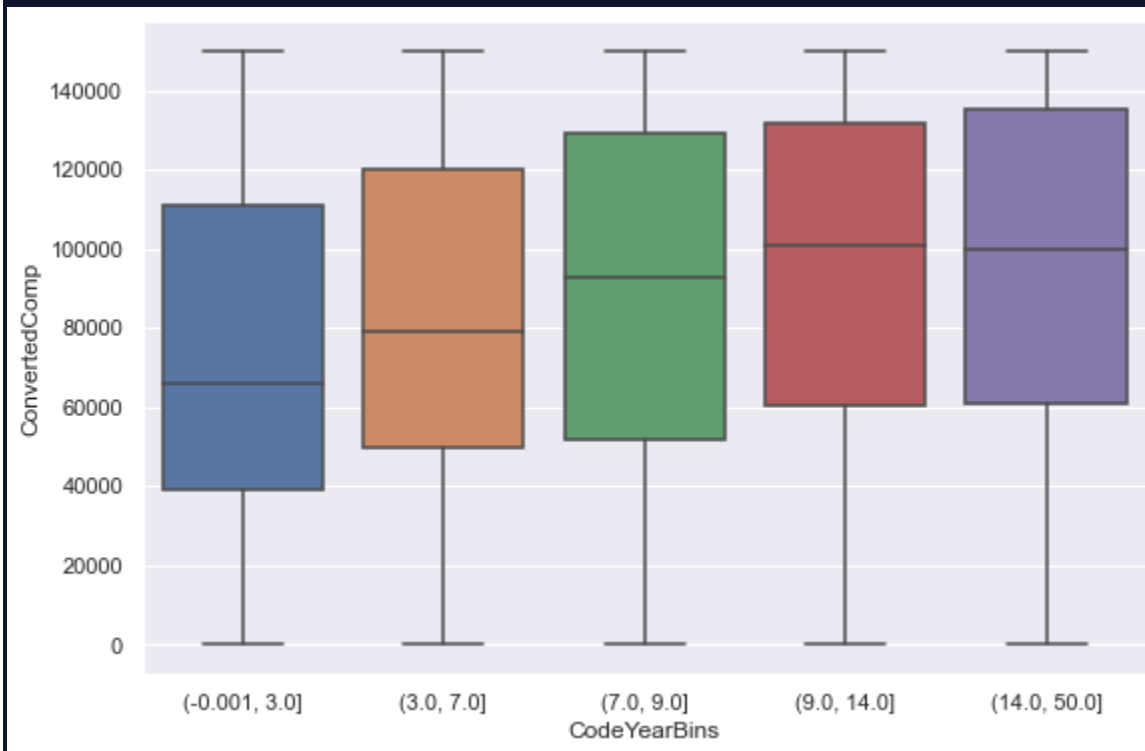
# Transform the model on the entire dataset
compdf = pd.DataFrame(np.round(imp.transform(imputedf), 0),
                      columns=['YearsCodePro', 'ConvertedComp'])
```

The above code will loop through (up to 20 times), and fill in the missing data based on the context provided by the other column. This should create data points that are indicative of the overall trend of the data. Now, you can analyze the relationship between `YearsCodePro` and `ConvertedComp` through the use of a boxplot like so:

```
compPlotdf = compdf.loc[compdf['ConvertedComp'] <= 150000]
compPlotdf['CodeYearBins'] = pd.qcut(compPlotdf['YearsCodePro'], q=5)

sns.boxplot(x="CodeYearBins", y="ConvertedComp",
            data=compPlotdf)
```

Boxplot output:



Note: You can click on images to enlarge them.

The plot above validates your hypothesis from before. While there are high (and low) earning developers at every experience level, experience appears to correlate with compensation. The more experienced a developer was, the more (on average) they were compensated.

Summary and Results

At this point, we have analyzed information about the developer community from a variety of points of view. Our client understands the global presence of the developer community, their varied backgrounds, and how their experience translates into compensation. Overall, these statistical analyses can guide actions in moving forward with a staffing plan that aligns with your client's growth plan and technical requirements.

By using a variety of techniques for handling missing data, you were able to reliably curate a cleaner dataset to fuel this set of analyses. These strategies allow you to salvage otherwise messy data, and should help you in the future with other datasets.

You can download an [example solution and the resulting datasets here](#).

Dataset Acknowledgements

The dataset provided is the result of a series of surveys hosted by [Stack Overflow](#) to understand their developer community. For the purposes of this project, the dataset has been slightly modified for the years 2018-2020.

Communicating Data Science Findings

[Learn how to effectively communicate data science analyses.](#)

Data Science is as much about storytelling as it is about coding and data wrangling. The reason Data Scientists are in such high demand recently is because of their abilities to translate the performance and health of a company by gathering previously unavailable insights from its data. It might seem like an easy and obvious thing at the moment, but in reality, a company's data might be so jumbled and unstructured that mere mortals are unable to make any sense of it.

Best Practices for Communicating Data Science Findings

A Data Scientist explaining to a group of Product Managers that the covariance between two variables shows no visible linear relationship is like Dumbledore teaching a group of Jedi how to conjure up a Patronus. Two different worlds. Does not compute.

The transformation of data analysis results by Data Scientists into easily accessible and understandable mediums for everyone to understand is actually one of the most important skills on the job.

After a Data Scientist completes an in-depth investigation on some set of data, the results of this investigation need to be converted somehow into a simplified summary for others to observe and understand. Usually, these kinds of summaries are written up as comments to one of your assigned tickets or more often as slides in a presentation that you then verbally discuss in further detail.

Coding question

Questions

To help guide and visualize these concepts, we're going to use a dataset about tipping and restaurant billing transactions to display different kinds of transformations.

The dataset can be easily sourced from the Python-based *Seaborn* visualization library as shown below. Feel free to take a look at the type of data it contains by running the code.

Code

```
import numpy as np
import pandas as pd
import seaborn as sns
pd.set_option('colheader_justify',
'center')
tips = sns.load_dataset("tips")
df = tips.head(10)
# Simply executing 'df' now will show the
same output as the following code does,
except to show the table within this
article we need to render it as HTML.
css = '''
    <html>
    <link rel="stylesheet"
    type="text/css" href="df_style.
    css"/>
```

Run

Check answer

Run your code to check your answer

Know Your Audience

Before starting the process of communicating your findings, it's important to put into perspective who your audience actually is. The entire tone of your presentation is shaped and changed by how individuals will perceive it.

Imagine you're presenting information about tipping and restaurant billing transactions to a group of restaurant owners. How would your presentation look? Now imagine you're showing the same data to a group of developers who are creating a food delivery app. How would your presentation be different for this audience than your first one?

If you put too much math/statistics jargon into a report or presentation for non-technical people, they're either going to ask you to define things for them or ask for you to clarify upon certain concepts. Hand it over too simplified to a technical audience, however, and they'll ask you how you came up with your results.

You need to find the perfect blend between these two to correctly adapt your communication.

Structure The Sequence Of Your Story

A structure defines in what sequence you reveal the results of your analysis. During this process, you also want to determine how much analytical depth you want to dive into. This is an opportunity to highlight only the most important observations you derived from the data through your exploration of it.

The type of observations you want to include is the kind that relates most to whatever the ultimate goal for the investigation is. If for example, you notice a completely unrelated trend in user behavior relative to your analysis, save it as an additional discussion point inside of an appendix at the end somewhere in your slides or report. Sometimes an audience member might also pick up on this additional detail and ask you about it during a Q&A session. This type of structure allows you to quickly go to your appendix and pick out a slide that goes more in-depth on a previously alluded detail.

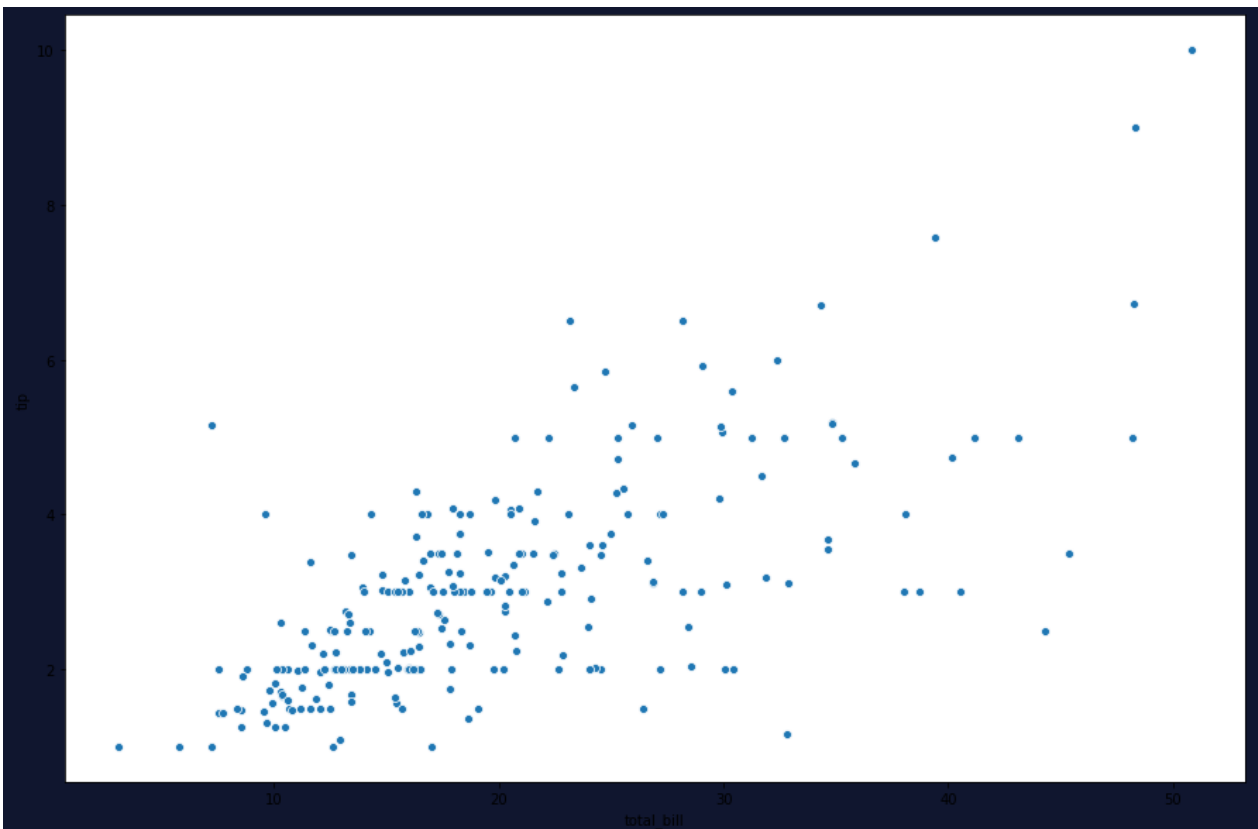
Find and pick relevant data

Are all of the insights you gathered from your data absolutely necessary to discuss? Does it help get your ultimate point across or is it just extra fluff you think is neat as a Data Scientist?

Sometimes, additional statistics that help a Data Scientist to take a particular investigative path may not be the ideal insights to share with the audience. If a number of features are vague in their immediate representation and don't explain some kind of requested result, it's better to pass on them.

The following is an example of a visualization of the tipping and restaurant transaction data. The visualization has had no adjustments and no added arguments. On the X-axis we have the total bill amount, and on the Y-axis we have the tip amount. The labels are small, there is no title, and the only immediate thing you can extract from it is the higher the bill, the higher the tip, which kind of seems obvious.

This is a rookie mistake in the Data Science field when trying to communicate findings through visualizations. We shouldn't look at a chart and think *What is this chart's purpose?*

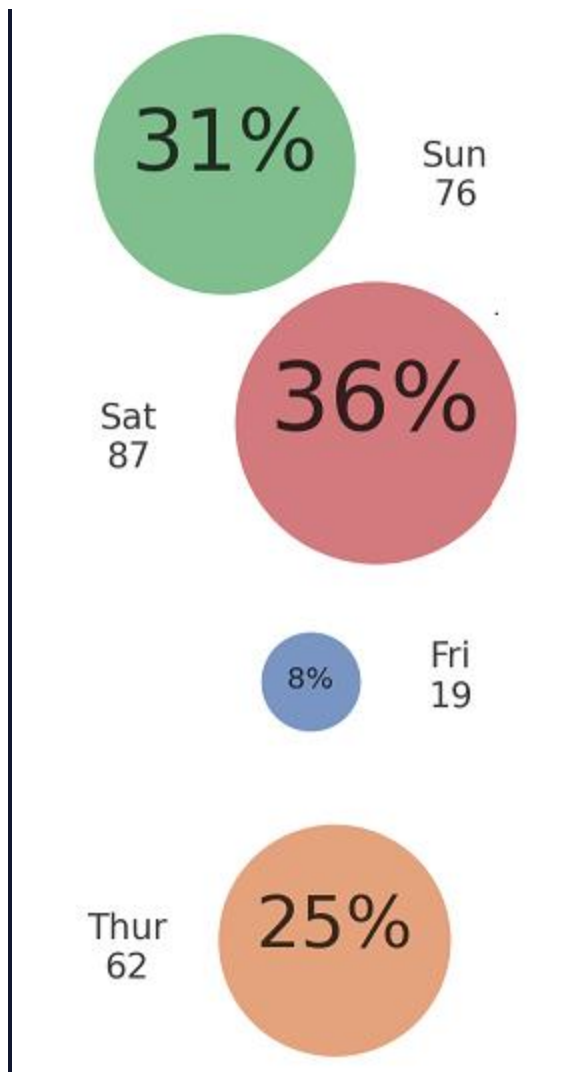


A little bland, no? Later in the article we'll focus on making it look a little more useful and easier to understand.

Choose proper methods of visualization

Contrary to popular belief, your data usually has a finite set of visualization types you should probably apply to it. Some examples include bar-charts and count-plots for categorical data, time-series charts for datetime type data, and lineplots/boxplots or histograms for continuous and numeric type data.

For example:



What is this visualization trying to tell us?

Using the wrong combination here will not only end up confusing the audience but could sacrifice the audiences' trust in you and your ability to deliver reliable analyses. For example, seeing a bubble plot type visualization on a website or in a magazine might look cool at first, but writing so much code to put circles behind 7 numbers is just silly. The visualization above could have just been easily put into a table without any effort and much quicker to read.

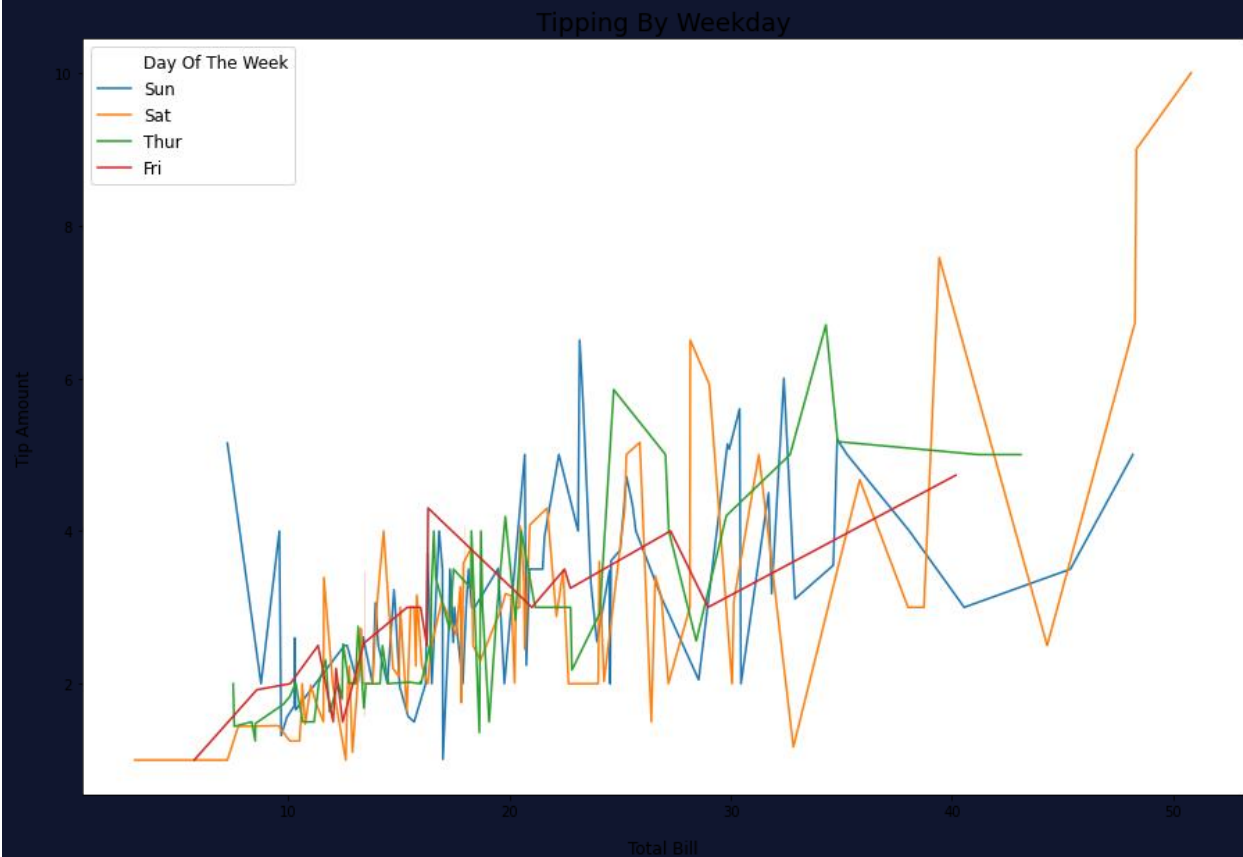
Complexity and Purpose

Don't let your visualizations become too noisy. You don't need to show a plot of every single possible combination of correlations for every single variable.

Focus only on the features important within your dataset to prevent introducing way too much information at one time to your audience. This includes defining maximum thresholds for how many segmented or grouped trends to picture in one visualization.

Another mistake made when trying to communicate a large magnitude of data is trying to fit it all in one visualization. In cases where you'd like to plot multiple trends in your data, it may sometimes be a better idea to just split it up into separate charts.

The following line plot shows the linear relation between bill amount and tips on different days. There are actually only four days included in this dataset, but if we took out the legend and asked what you see, the only response we would expect to hear is *spaghetti*:



Since the trend for the tip amount seems to stay relatively similar on different days, there's no real reason to visualize each day separately. The only reason Friday looks like a more stable behavior is that there is less data. If this dataset contained some kind of datetime variable and was much bigger, another approach would be to average out or sum the transactions by each date contained in the X-axis. In this case, you would have to add additional ticks in the X-axis to help signify the days of the week but would look much cleaner.

Conclusion

Data always starts out messy, ugly, and confusing. The simplest way of getting an audience to have a feel for a data-focused concept is through visualization. When using visualization to present your findings, remember the following:

- Visualizations are like jokes, if you don't immediately understand them... they're probably not very good.

- Know who your audience is.

- Make sure your visualization has a purpose and is somehow tied to the ultimate conclusion of your findings because irrelevant topics sometimes make your audience lose interest.

- Make sure you are using relevant data.

- Choose a proper method of visualization.

- Use large, short labels on your graphs so that people can quickly understand what the markers or line in a chart actually mean.

- Try to simplify concepts without using any technical jargon that makes the data look messy again.

Data Science Foundations Review

Congratulations on finishing the Data Science Foundations section of the Career Path – let's see what you learned!

At this point, you have a solid foundation in the technical knowledge, skills and abilities you need to get your start as a data scientist.

You are now able to:

- Create programs with Python 3
- Query and manipulate data with SQL and Python pandas
- Create data visualizations with Python matplotlib and seaborn
- Summarize and analyze datasets
- Conduct hypothesis testing
- Clean and tidy datasets
- Communicate your findings clearly

Plus, you now have several portfolio projects that showcase what you've learned in the first part of this Career Path, and can share these projects with your community and future employers.

In the next few articles and lessons, you will learn more about the different options available to you in your Career Path. Everything that you have covered up to this point is the same across all of the Data Scientist Career Path Specializations. So, if you choose to switch into another path now, you can pick up where you left off.

Data Scientist Specialization



Data Scientist Specializations Overview

The Codecademy Data Scientist Specializations

Data Science has evolved a lot since Harvard Business Review called it the [“Sexiest Job of the 21st Century”](#) in 2012.

Machine Learning Data Scientist Specializes in AI, algorithms, predictive analytics, and forecasting. Solves problems at scale and creates predictive models.	Analytics Data Scientist Derives insights from existing data, spots trends, and addresses business questions. Answers questions using data.
NLP Data Scientist Focuses on deriving meaning from large amounts of human language and generating new language in the form of chatbots and other AI applications.	Inference Data Scientist Identifies cause and effect, tests hypothesis, analyzes A/B tests. Contributes to decision-making.

Data Science teams have sprung up around the world, and employ people with a wide range of skills. Some Data Scientists work primarily in SQL and report on performance metrics (analytics). Others work primarily in Python and are involved in predictive modeling and machine learning (machine learning). Others work mostly in R and try to figure out what factors cause others (inference).

Of course, the size of a team matters, too. For example, if you are the only Data Scientist in an organization, you might get to do a little bit of everything. If you are a member of a large team with a lot of structure, you might have a more specific set of responsibilities.

Our Career Paths are designed to address both ends of the spectrum. With a solid foundation in the core principles, you will be able to cross data science functions, and with your specialization, you will be ready to contribute to specialized teams.

All of the Data Scientist career paths begin with the same core content (which is also our Data Science Foundations skill path). After building that foundation, you can keep or change your specialization to fit your goals — and transfer your progress automatically.

Codecademy divides Data Science into four specializations:

- [Machine Learning](#)
- [Inference](#)
- [Analytics](#)
- [Natural Language Processing](#)

These roles may go by different names in different places, but the focus is the same.

Throughout this lesson, you will learn more about each specialization, what they do, what job titles to look for, and what is covered in the Career Path.

At the end of this lesson, you will have the opportunity to change specializations if you choose to (or stay in this one).

Let's dive in and learn more about these specializations.

Machine Learning Specialization

When Data Science first came into its own,

Preview: Docs Loading link description

[Machine Learning](#)

was at the center of it. In the years since, we've realized that there is a lot to learn from data beyond machine learning, but this specialization is still at the heart of a lot of Data Scientist roles. If you are looking for a generalist role, we recommend starting here.

Overview of the Role

Data Scientists who specialize in Machine Learning fill a lot of roles on a Data Science team. As a Machine Learning Data Scientist, you might work with predictive modeling and

Preview: Docs Simulates human intelligence in computers, enabling learning, reasoning, and problem-solving to provide solutions across various tasks.

[artificial intelligence](#)

to solve problems at scale. These are also opportunities to be involved in Decision Science and leveraging machine learning to develop new technologies such as disease identification, customer churn, artificial intelligence, and recommendation systems.

How much will you make?

Estimated Avg. Total Pay \$132,507 (Glassdoor, 2022) Estimated Salary Range with 0-1 years experience: \$53,000 - \$340,000 (Glassdoor, 2022) Note that we find both ends of this range to be uncommon but have never heard of someone new to Data Science making \$340k, so interpret these estimates cautiously.

What will you do?

Lead the development and improvement of algorithms to solve business problems.

Work with vast amounts of data and build data-driven products.

Apply knowledge of

Preview: Docs Loading link description

[statistics](#)

, machine learning, programming, data modeling, simulation, and advanced mathematics to recognize patterns, identify opportunities, and make valuable discoveries leading to product development and improvement.

Prototype production-grade algorithms and models that improve the experience of customers and clients.

Research and develop algorithms to strengthen our (anomaly detection / recommendation / forecasting / etc.) system.

Use a flexible, analytical approach to design, develop, and evaluate predictive models and advanced algorithms that lead to optimal value extraction from data.

Work closely with adjacent teams (product engineers, marketing, sales, HR, and leadership) to understand their use cases, develop models, and balance scientific rigor and strategic considerations.

Provide data visualization support to help the team make decisions.

Find new ways to drive business impact and solve complex problems.

Learn new tools, systems, and languages quickly.

What the Machine Learning Specialization covers

- Data Science Foundations
- Supervised Machine Learning Models
 - Linear Regression
 - Logistic Regression
 - K Nearest Neighbors

- Decision Trees
 - Naive Bayes Classifier
 - Support Vector Machines
 - Random Forests
- Unsupervised Machine Learning Models
 - K-Means Clustering
 - Principle Component Analysis
- Deep Learning with TensorFlow
 - Regression
 - Classification
- Feature Engineering
 - Data Transformation
 - Feature Selection
- Evaluation Metrics
 - Normalization
 - Training, Validation, and Test Datasets
 - Accuracy, Recall, Precision, F1
 - AUC, ROC
- Handling Big Data with PySpark

Data Scientist: Analytics Specialization

Analytics Data Scientists are needed in nearly every company to draw insights from available data sources. They are at the forefront of working with data and analyzing it to develop meaningful insights.

Overview of the Role

Data Scientists who specialize in Analytics generate meaning and insights through data analysis. As an Analytics Data Scientist, you will work primarily in SQL and other databases and might be responsible for helping adjacent teams frame their business questions into questions that can be answered by data. You will likely create reports that highlight summary

Preview: Docs Loading link description

[statistics](#)

and exploratory data analysis. You may work across tools, moving from working with or aggregating data in Python or SQL and reporting on it with BI Tools such as Tableau or Excel. You may be responsible for making data visualizations and communicating about data to address specific business problems or goals. Many Analytics Data Scientists work directly with the data that companies have collected on their clients, customers, or the public. You might be responsible for reconciling data from multiple sources to address specific questions and draw insights.

How much will you make?

Estimated Avg. Total Pay \$122,514 (Glassdoor, 2022) Estimated Salary Range with 0-1 years experience:

\$54,000 - \$263,000 (Glassdoor, 2022) *Note that we find both ends of this range to be uncommon but have never heard of someone new to Data Science making \$263k, so interpret these estimates cautiously.*

Sample Job Description

Evaluate and define product and business metric

Preview: Docs Loading link description

[framework](#)

.

Build key data sets/pipelines to empower operational and exploratory analysis.

Democratize data by building and socializing decision tools (dashboards, reports).

Experiment design, measurement, and analysis.

Effective communications with key stakeholders, including Product Managers, Engineers, Designers, and Operation Managers.

Build key data sets to empower operational and exploratory analysis

Define, forecast, evaluate and report on key business metrics.

Work closely with adjacent teams (product engineers, marketing, sales, HR, and leadership) to understand their use cases, develop models, and balance scientific rigor and strategic considerations.

Provide data visualization support to help the team make decisions.

Find new ways to drive business impact and solve complex problems.

Learn new tools, systems, and languages quickly.

What the Analytics Specialization covers

Data Science Foundations

Advanced SQL

Preview: Docs A database is a collection of structured information stored so it can be easily accessed and updated. In a computer system, databases are commonly accessed through a database management system, or DBMS.

[Database](#)

Design

Advanced Data Visualization

Data Visualization with Tableau

Analyzing Spreadsheets with Excel

Data Scientist: Inference Specialization

Inference Data Scientists go by a lot of different titles. This role can be called a

Preview: Docs Loading link description

[Statistics](#)

Data Scientist, Data Analyst, Causal Scientist, Marketing Analysts, etc. It is essential to read the job description to understand what a given job will entail.

Overview of the Role

Data Scientists who specialize in Inference leverage statistics to understand data. As an Inference Data Scientist, you will work with and influence decision-making by developing and testing hypotheses and conducting experiments. You may conduct and own A/B tests, explore datasets to identify patterns and causal relationships. The key differentiator of an Inference Data Scientist is the emphasis on statistical methods to evaluate the relationship between

Preview: Docs Loading link description

[variables](#)

and how manipulating real-world products or features can be measured in the resulting data.

How much will you make?

Estimated Avg. Total Pay \$123,325 (Glassdoor, 2022) Estimated Salary Range with 0-1 years experience: \$54,000 - \$299,000 (Glassdoor, 2022) *Note that we find both ends of this range to be uncommon but have never heard of someone new to Data Science making \$299k, so interpret these estimates cautiously.*

Sample Job Description

- Use statistical methodologies to analyze datasets, build new products and evolve existing models.
- Apply stratification, variance reduction, and other advanced experiment design techniques to create A/B tests to efficiently measure the impact of your innovations while minimizing risk to the broader system.
- Apply various statistical methods, including causal inference, to understand the intricate ecosystem of our clients, customers, users, products, partners, and revenue.
- Generate and test hypotheses and analyze and interpret the results of experiments.
- Use statistical tools to identify specific, high-impact opportunities to build new and improve existing solutions.
- Build key data sets to empower operational and exploratory analysis.
- Define, forecast, evaluate and report on key business metrics.
- Work closely with adjacent teams (product engineers, marketing, sales, HR, and leadership) to understand their use cases, develop models, and balance scientific rigor and strategic considerations.
- Provide data visualization support to help the team make decisions.
- Find new ways to drive business impact and solve complex problems.
- Learn new tools, systems, and languages quickly.

What the Inference Specialization covers

- Data Science Foundations
- Advanced Statistics
- Causal Inference
- Deeper dive into A/B Testing & Hypothesis Testing
- Experimental Design
- Advanced SQL
- Advanced Data Visualization
- Data Visualization with Tableau
- Analyzing Spreadsheets with Excel

Data Scientist: Natural Language Processing Specialization

Natural Language Processing (NLP) is the most niche specialization we offer, but also a growing field. It is a good application of deep learning that is both widespread and transferable to solving other types of problems.

Overview of the Role

Data Scientists who specialize in (NLP) focus on managing the quirks of working with unstructured text and speech data. As an NLP Data Scientist, you might work with chatbots and support agents. You also might work with recommendation systems, search, social media analysis, sentiment analysis, topic modeling, translation, and generally extracting meaning from human language. Typically, you will be less concerned with measuring Key Performance Indicators (KPIs) and other business metrics than other Data Scientists might be. However, as an NLP Data Scientist, you also might be called on to be a generalist and therefore gather and present data on business metrics as well.

How much will you make?

Estimated Avg. Total Pay \$133,505 (Glassdoor, 2022) Estimated Salary Range with 0-1 years experience: \$54,000 - \$337,000 (Glassdoor, 2022) *Note that we find both ends of this range to be uncommon but have never heard of someone new to Data Science making \$337k, so interpret these estimates cautiously.*

Sample Job Description

Use

Preview: Docs Loading link description

[Machine Learning](#)

and Natural Language Processing to analyze speech and text data from customer and client interactions.

Use NLP to improve our search functions and build recommendation systems.

Build and deploy NLP/ML models for conversational AI products (chatbot).

Lead the development of our chatbot: Applying active learning, semi-supervised learning, weak label generation, and data augmentation strategies to improve the customer and/or client experience

Build machine learning and NLP models through all phases of development, from design through training, evaluation, and validation. Partner with engineering teams to operationalize models.

Work closely with adjacent teams (product engineers, marketing, sales, HR, and leadership) to understand their use cases, develop models, and balance scientific rigor and strategic considerations.

Find new ways to drive business impact and solve complex problems.

Learn new tools, systems, and languages quickly.

What the Inference Specialization covers

- Data Science Foundations
- Supervised Machine Learning Models
 - Linear Regression
 - Logistic Regression
 - K Nearest Neighbors
 - Decision Trees
 - Naive Bayes Classifier
 - Support Vector Machines
 - Random Forests
- Unsupervised Machine Learning Models
 - K-Means Clustering
 - Principle Component Analysis
- Deep Learning with TensorFlow
 - Regression
 - Classification
- Evaluation Metrics
 - Normalization
 - Training, Validation, and Test Datasets
 - Accuracy, Recall, Precision, F1
 - AUC, ROC
- Natural Language Processing
 - Text pre-processing
 - Bag of Words Models
 - Tf-idf
 - Word embeddings
 - LSTMs
- Building Chatbots

Natural Language Processing

NLP

Natural Language Processing (NLP) is the most niche specialization we offer, but also a growing field. It is a good application of deep learning that is both widespread and transferable to solving other types of problems.

Overview of the Role

Data Scientists who specialize in (NLP) focus on managing the quirks of working with unstructured text and speech data. As an NLP Data Scientist, you might work with chatbots and support agents. You also might work with recommendation systems, search, social media analysis, sentiment analysis, topic modeling, translation, and generally extracting meaning from human language. Typically, you will be less concerned with measuring Key Performance Indicators (KPIs) and other business metrics than other Data Scientists might be. However, as an NLP Data Scientist, you also might be called on to be a generalist and therefore gather and present data on business metrics as well.

How to Showcase your Data Science Skills

And start building your data science portfolio.

If a tree falls in the forest and no one is around to hear it, does it make a sound?

– Philosophy students everywhere

Hiring managers have their own version of that question: **If a data scientist doesn't have a portfolio, do they really have the skills?**

You need to have a portfolio. A portfolio is evidence that you have the technical skills and knowledge that you say that you do. But what goes into a portfolio and what does that mean in data science?

What Goes Into a Portfolio

Ultimately, exactly what you put in your portfolio is up to you. This [project at the end of our Interview Prep Skill Path](#) will help guide you through making sure that you've covered all of the major categories, but the projects themselves are completely up to you.

We recommend that you demonstrate the ability to:

Throughout this Career Path, you will encounter Capstone and Portfolio projects. These can absolutely be put into your portfolio. However, thousands of people take this Career Path. To really make yourself stand out, you will want to add projects about things you are interested in.

If you like sports, use sports data, if you like science, use scientific data, etc. Showing a little bit of your interests and what you are curious about will not only make the project more fun for you, but also let you showcase your own domain knowledge or curiosity. Interest and enthusiasm in your projects will help set you apart from other candidates when you are applying and interviewing.

There are a lot of places to look for datasets. We have an entire lesson on where to find data later in this Career Path. For right now, the best thing you can do is be curious about the world around you, and keep track of some things that you might want to explore as you build your portfolio.

What a Data Science Portfolio Looks Like

A data science portfolio can take a lot of shapes, you could create your own professional website, upload documents to your LinkedIn or other social media site, or create a repository on GitHub. We highly recommend GitHub. It is simple, lightweight, and by far the most popular place to house a portfolio.

You have a few choices in terms of what kinds of files you want to upload. A general rule is the simpler, the better. You can upload [Jupyter notebooks](#) of your projects, `.Rmd` or Python files of your code, or PDFs, or presentations of summary reports.

The kind of files you upload will depend on the kind of Data Scientist role you are applying for. A role that is more programming-focused (i.e., a Machine Learning specialization) should have a more programming-heavy portfolio. A role that is more focused on analysis and communication should have at least one summary report.

The Culture of Portfolios

There are a few unspoken rules about portfolios that you should keep in mind. Throughout the Career Path, we've tried to be explicit about when these things come up, and we have presented best practices throughout. However, we want to call out some of the potential pitfalls and make sure you avoid them.

Datasets

There are a few datasets that are used for teaching a lot. Specifically, the [iris dataset](#) and the [titanic dataset](#) are the most common. We use them in the Path, and you will likely find them in the wild. These are so popular that almost everyone who works in data science has seen them. These do not make good portfolio projects because employers might read them as highly scaffolded student projects or as simply replicating a tutorial, and they won't set you apart from other candidates.

Formatting

It is important to pay close attention to presentation. Using the right file types, and presenting content in the right format is essential to getting that first interview.

For example, if you have written some code to analyze a dataset, it should be stored as a `.py`, `.ipynb`, `.R`, or `.Rmd` file. We teach you how to do this in the Career Paths, but it is worth mentioning again that code should almost never be stored in a text document (e.g., `.txt`, `.doc`, etc.) unless it is formatted as a code snippet and used as an example or evidence.

Another example is Tableau (or PowerBI) visualizations and dashboards.

There are two ways to store these that demonstrate proficiency with these tools. First, both platforms offer repositories. [Tableau Public](#) is both a great place to show off your work and see what other people are making. The other way is to embed them into a website with an `iframe`.

You should **not** take screenshots, download them as jpegs, pdfs, or otherwise transform the rich, interactive dashboards you've made into static images.

You can (and should) embed static visualizations (that you make in Python, R, or even Excel or Tableau) into written reports, but the skill you are demonstrating is communicating visually about data (not proficiency in Tableau).

Finally, reports and summaries can be woven into your Jupyter Notebooks, presented as written documents (and stored as PDFs), or compiled into slide decks (also stored as PDFs).

Code Hygeine

It takes a long time and a lot of practice to write clear, well-structured code. However, you can take massive steps in that direction by writing well-commented and organized code. Just like writing an essay, writing good code often requires editing. Not all of your projects need to be edited. However, if you choose to highlight one or two (which you can do by pinning them in GitHub), you'll want to be sure that you've written in some comments and reorganized your code for clarity as much as possible.

Grammar

Just like in your resume and cover letter, you want to proofread your portfolio projects to be sure they are free from grammatical or spelling errors. Unlike in your resume/cover letter, a couple typos probably won't disqualify you. However, every Data Scientist needs to communicate about data, and you want to demonstrate that you can do that clearly. Even simply running any reports you write through a spellchecker will help in this respect.

Conclusion

Ultimately, your portfolio is how you verify that you have all of the skills you've worked so hard to acquire. Have fun with it by including some projects that

excite you, and be sure to be ready to talk about those projects with recruiters when you get that first interview!

Introduction to Advanced EDA

Explore what you will learn in Advanced Exploratory Data Analysis!

Exploration is great, but exploration with a map is even better. This unit provides a map to get you started.

What will Advanced Exploratory Data Analysis cover?

After this unit, you will be able to:

Why did we build this?

Exploratory Data Analysis is the backbone of most data analysis. It's all about discovering what is in your data. It's the best place to start formulating questions and getting curious and excited about the potential of your dataset. Likewise, EDA can itself be the answer to a lot of data questions. Often, all we are looking for is a description of the world we collected data about, and the process of EDA can deliver that.

Who are my classmates?

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

Variance

Finding the mean, median, and mode of a dataset is a good way to start getting an understanding of the general shape of your data

However, those three descriptive

Preview: Docs Loading link description

[statistics](#)

only tell part of the story. Consider the two datasets below:

```
dataset_one = [-4, -2, 0, 2, 4]
dataset_two = [-400, -200, 0, 200, 400]
```

Copy to Clipboard

These two datasets have the same mean and median — both of those values happen to be 0. If we only reported these two statistics, we would not be communicating any meaningful difference between these two datasets.

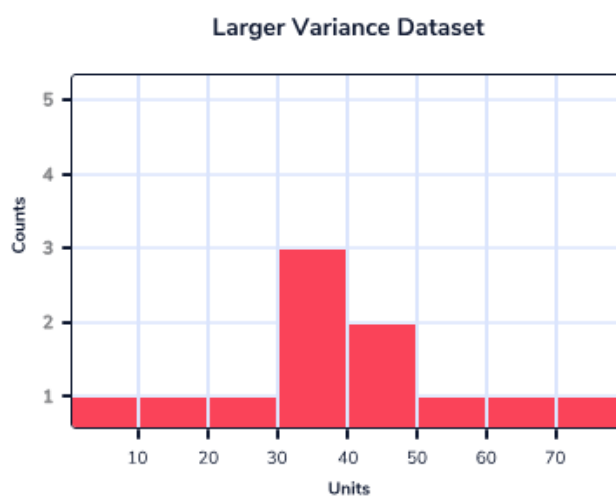
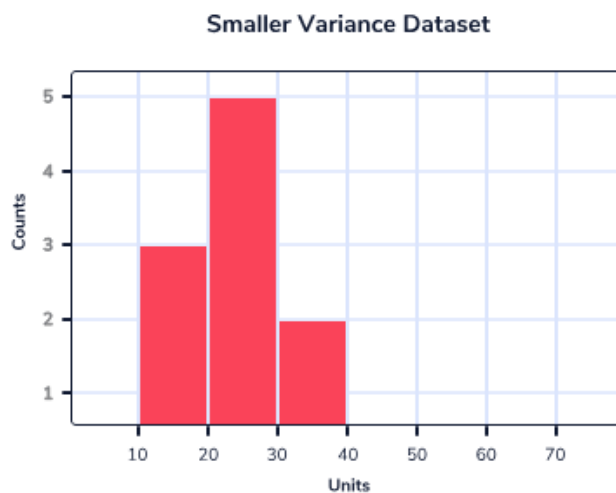
This is where *variance* comes into play. Variance is a descriptive statistic that describes how spread out the points in a data set are.

Distance From Mean

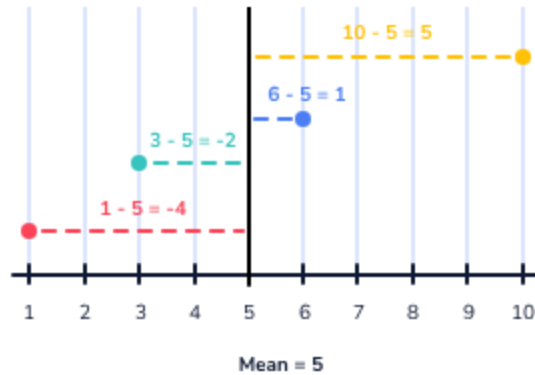
Now that you have learned the importance of describing the spread of a dataset, let's figure out how to mathematically compute this number.

How would you attempt to capture the spread of the data in a single number?

Let's start with our intuition — we want the variance of a dataset to be a large number if the data is spread out, and a small number if the data is close together.



A lot of people may initially consider using the range of the data. But that only considers two points in your entire dataset. Instead, we can include every point in our calculation by finding the difference between every data point and the mean.



If the data is close together, then each data point will tend to be close to the mean, and the difference will be small. If the data is spread out, the difference between every data point and the mean will be larger.

Mathematically, we can write this comparison as

$$\text{difference} = X - \mu$$

$$\text{difference} = X - \mu$$

Where $\bar{x}$ is a single data point and the Greek letter μ is the mean.

Instructions

Checkpoint 1 Failed, try again

1.

We've given you a very small dataset of five values named `grades`. These are five randomly chosen grades from the first teacher's class. We've also calculated the mean of this small dataset and stored it in a variable named `mean`.

Let's find the difference between each of these data points and the mean. We've created a variable for each difference. Start with `difference_one`. Change the value of `difference_one` — it should be equal to the first value in the data set minus `mean`.

Average Distances

We now have five different values that describe how far away each point is from the mean. That seems to be a good start in describing the spread of the data. But the whole point of calculating variance was to get one number that describes the dataset. We don't want to report five values — we want to combine those into one descriptive statistic.

To do this, we'll take the average of those five numbers. By adding those numbers together and dividing by 5, we'll end up with a single number that describes the average distance between our data points and the mean.

Note that we're not *quite* done yet — our final answer is going to look a bit strange here. There's a small problem that we'll fix in the next exercise.

```
import numpy as np

grades = [88, 82, 85, 84, 90]
mean = np.mean(grades)

difference_one = 88 - mean
difference_two = 82 - mean
difference_three = 85 - mean
difference_four = 84 - mean
difference_five = 90 - mean

#Part 1: Sum the differences
difference_sum = difference_one + difference_two + difference_three +
difference_four + difference_five

#Part 2: Average the differences
average_difference = difference_sum / 5

#IGNORE CODE BELOW HERE
print("The sum of the differences is " + str(format(difference_sum, ".16f")))
print("The average difference is " + str(format(average_difference, ".16f")))
```

Square The Differences

We're almost there! We have one small problem with our equation. Consider this very small dataset:

```
[-5, 5]
```

Copy to Clipboard

The mean of this dataset is 0, so when we find the difference between each point and the mean we get $-5 - 0 = -5$ and $5 - 0 = 5$.

When we take the average of -5 and 5 to get the variance, we get 0!

Now think about what would happen if the dataset were `[-200, 200]`. We'd get the same result! That can't possibly be right — the dataset with `200` is much more spread out than the dataset with `5`, so the variance should be much larger!

The problem here is with negative numbers. Because one of our data points was `5` units below the mean and the other was `5` units above the mean, they canceled each other out!

When calculating variance, we don't care if a data point was above or below the mean — all we care about is how far away it was. To get rid of those pesky negative numbers, we'll square the difference between each data point and the mean.

Our equation for finding the difference between a data point and the mean now looks like this:

```
variance = difference_sum / 5

print("The sum of the squared differences is " + str(difference_sum))
print("The variance is " + str(variance))
```

Variance In NumPy

Well done! You've calculated the variance of a data set. The full equation for the variance is as follows:

$$\sigma^2 = \frac{\sum (x_i - \bar{x})^2}{N}$$

Let's dissect this equation a bit.

Variance is usually represented by the symbol sigma squared.

We start by taking every point in the dataset — from point number `1` to point number `N` — and finding the difference between that point and the mean.

Next, we square each difference to make all differences positive.

Finally, we average those squared differences by adding them together and dividing by `N`, the total number of points in the dataset.

All of this work can be done quickly using Python's NumPy library. The `var()` function takes a list of numbers as a

Preview: Docs Loading link description
[parameter](#)
and returns the variance of that dataset.

```
import numpy as np

dataset = [3, 5, -2, 49, 10]
variance = np.var(dataset)
```



```

import numpy as np
import matplotlib.pyplot as plt
import codecademylib3_seaborn

teacher_one_grades = [80.24, 81.15, 81.29, 82.12, 82.52, 82.54, 82.76, 83.37, 83.42,
83.45, 83.47, 83.79, 83.91, 83.98, 84.03, 84.69, 84.74, 84.89, 84.95, 84.95, 85.02,
85.18, 85.53, 86.29, 86.83, 87.29, 87.47, 87.62, 88.04, 88.5]
teacher_two_grades = [65.82, 70.77, 71.46, 73.63, 74.62, 76.53, 76.86, 77.06, 78.46,
79.81, 80.64, 81.61, 81.84, 83.67, 84.44, 84.73, 84.74, 85.15, 86.55, 88.06, 88.53,
90.12, 91.27, 91.62, 92.86, 94.37, 95.64, 95.99, 97.69, 104.4]

#Set these two variables equal to the variance of each dataset using NumPy
teacher_one_variance = np.var(teacher_one_grades)
teacher_two_variance = np.var(teacher_two_grades)

#IGNORE THE CODE BELOW HERE
plt.hist(teacher_one_grades, alpha = 0.75, label = "Teacher 1 Scores", bins = 7)
plt.hist(teacher_two_grades, alpha = 0.5, label = "Teacher 2 Scores", bins = 30)
plt.title("Student test grades in two different classes")
plt.xlabel("Grades")
plt.legend()
plt.show()

```

Standard Deviation

Variance is a tricky statistic to use because its units are different from both the mean and the data itself. For example, the mean of our NBA dataset is `77.98` inches. Because of this, we can say someone who is `80` inches tall is about two inches taller than the average NBA player.

However, because the formula for variance includes *squaring* the difference between the data and the mean, the variance is measured in *units squared*. This means that the variance for our NBA dataset is `13.32` inches squared.

This result is hard to interpret in context with the mean or the data because their units are different. This is where the statistic *standard deviation* is useful.

Standard deviation is computed by taking the square root of the variance. `sigma` is the symbol commonly used for standard deviation. Conveniently, `sigma` squared is the symbol commonly used for variance:

$$\sigma = \sqrt{\frac{\sum (x_i - \mu)^2}{N}}$$

In Python, you can take the square root of a number using `** 0.5`:

```

num = 25
num_square_root = num ** 0.5

```

Standard Deviation in NumPy

There is a NumPy function dedicated to finding the standard deviation of a dataset — we can cut out the step of first finding the variance. The NumPy function `std()` takes a dataset as a

Preview: Docs A parameter is the name of a variable passed into a function. Parameters allow functions to accept inputs. An argument, on the other hand, is the actual value of the variable (also known as the parameter) passed into a function. Suppose we have a function called tripleThis(): js function tripleThis(x) { return x * 3; } tripleThis(6);

[parameter](#)

and returns the standard deviation of that dataset:

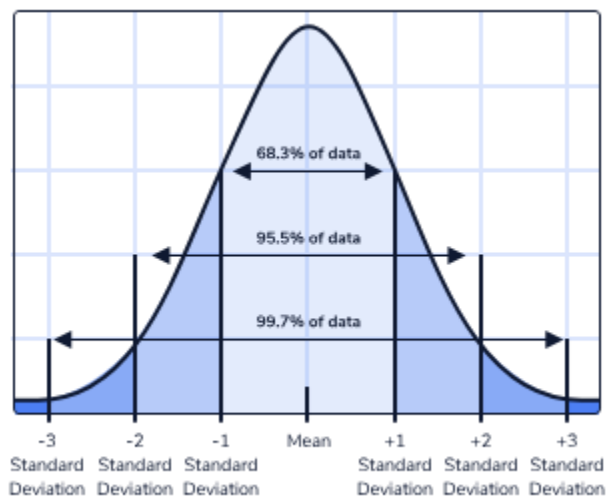
```
import numpy as np

dataset = [4, 8, 15, 16, 23, 42]
standard_deviation = np.std(dataset)
```

Using Standard Deviation

Now that we're able to compute the standard deviation of a dataset, what can we do with it?

Now that our units match, our measure of spread is easier to interpret. By finding the number of standard deviations a data point is away from the mean, we can begin to investigate how unusual that datapoint truly is. In fact, you can usually expect around 68% of your data to fall within one standard deviation of the mean, 95% of your data to fall within two standard deviations of the mean, and 99.7% of your data to fall within three standard deviations of the mean.



If you have a data point that is over three standard deviations away from the mean, that's an incredibly unusual piece of data!

Introduction to Statistics

Statistics is the science that is concerned with methods for collecting, organizing, analyzing, and interpreting data.

[Statistics](#) is often pitched as a way to find certainty through data. As you'll learn in this lesson, the power of statistics is more often used to communicate that certainty doesn't really exist. Instead, it provides tools to communicate how uncertain we are about a problem.

There's no better tool to visualize the uncertainty and chaos in data than a histogram. A histogram displays the distribution of your underlying data.

Histograms reveal, through numbers, interpretable trends in your data. They don't provide a yes or no answer, but are often used as a starting point for discussion and informing an answer to your data.

Whether you're brand new to statistics or know it well, this lesson will teach you how to use histograms to visualize your data and inform decision-making.

```
# Import packages
import codecademylib
import numpy as np
import pandas as pd

# Import matplotlib pyplot
from matplotlib import pyplot as plt

# Read in transactions data
mu, sigma = 800, 100 # mean and standard deviation
burrito_calories = np.random.normal(mu, sigma, 320)

# Save transaction times to a separate numpy array
plt.hist(burrito_calories, range=(250, 1250), bins=100, edgecolor='black')
plt.title("Calories in a Burrito Bowl", fontsize = 24)
plt.xlabel("Calories", fontsize=18)
plt.ylabel("Count", fontsize=18)

plt.show()
```

Summarizing Your Data

The purpose of a histogram is to summarize data that you can use to inform a decision or explain a distribution.

While a histogram is one of the most useful tools for communicating trends, people often use the *average* of a dataset to make broad claims about its underlying trends.

While the average value of data may be useful to interpret where the data is centered, it can also be misleading.

Average grocery store time

Throughout this lesson, you will analyze grocery store data to understand how a histogram may be a better tool for communicating trends in your data. You will use this data to determine the best times for a manager to staff their store.

Let's start by looking at our data and calculating the average time that a person will enter the grocery store.

Range

Histograms are helpful for understanding how your data is distributed. While the average time a customer may arrive at the grocery store is 3 pm, the manager knows 3 pm is not the busiest time of day.

Before identifying the busiest times of the day, it's important to understand the extremes of your data: the minimum and maximum values in your dataset. With the minimums and maximums, you can calculate the *range*.

The range of your data is the difference between the maximum value and the minimum value in your dataset.

$$\text{range} = \max(\text{data}) - \min(\text{data})$$

$$\text{range} = \max(\text{data}) - \min(\text{data})$$

Exercise Class Example

In the example below, we have a NumPy

Preview: Docs Loading link description

[array](#)

with the ages of people in an exercise class. Before looking at the data, let's think about what minimum, maximum, and range values are reasonable for a group of people in an exercise class:

The minimum cannot be below 0, because people don't have negative ages
The maximum is probably lower than 122 (the oldest person ever).

Now, let's take a look at our data.

```
exercise_ages = np.array([22, 27, 45, 62, 34, 52, 42, 22, 34, 26])
```

Copy to Clipboard

The minimum age in `exercise_ages` is 22, the maximum age is 62, and the range is 40.

You can use the following Python commands to verify this result:

```
min_age = np.amin(exercise_ages) # Answer is 22
max_age = np.amax(exercise_ages) # Answer is 62
age_range = max_age - min_age
```

```

# Import packages
import numpy as np
import pandas as pd

# Read in transactions data
transactions = pd.read_csv("transactions.csv")

# Save transaction data to numpy arrays
times = transactions["Transaction Time"].values
cost = transactions["Cost"].values

# Find the minimum time, maximum time, and range
min_time = transactions["Transaction Time"].min() # <-- Replace with min calc
max_time = transactions["Transaction Time"].max() # <-- Replace with max calc
range_time = max_time - min_time # <-- Replace max - min

# Printing the values
print("Earliest Time: " + str(min_time))
print("Latest Time: " + str(max_time))
print("Time Range: " + str(range_time))

```

Bins and Count I

In the previous exercise, you found that the earliest

Preview: Docs Loading link description

[transaction](#)

time is close to 0, and the latest transaction is close to 24, making your range nearly 24 hours.

Now, we have the information we need to start building our histogram. The two key features of a histogram are *bins* and *counts*.

Bins

A bin is a sub-range of values that falls within the range of a dataset. In the grocery store example, a valid bin may be from 0 hours to 6 hours. This bin includes all times from just after midnight (0) until 6 am (6).

Additionally, all bins in a histogram must be the same width.

If the range of values in our dataset is from 0 to 24, and the first bin in our grocery store example is from 0 to 6, can you figure out the minimums and maximums of the other bins?

The grocery store bins are:

- 0 to 6 hours
- 6 to 12 hours
- 12 to 18 hours
- 18 to 24 hours

Bins and Count II

A *count* is the number of values that fall within a bin's range. For example, if 100 customers arrive at your grocery store between midnight (0) and 6 am (6), your count for that bin is equal to 100.

Exercise Class Example

In the example below, we have an array with ten values in it, each representing the age of a person in an exercise class. We want to calculate the number of students who are in their 20s, 30s, 40s, 50s, and 60s.

```
exercise_ages = np.array([20, 27, 45, 69, 34, 52, 42, 22, 34, 26])
```

Copy to Clipboard

Question: What is our range?

Answer: Between 20 and 69.

Question: How many bins do we need?

Answer: The bins are 20s, 30s, 40s, 50s, and 60s, so we need five bins, each covering ten years.

The table below shows the number of people in each binned age group:

20-29	30-39	40-49	50-59	60-69
4	2	2	1	1

Histograms

While counting the number of values in a bin is straightforward, it is also time-consuming. How long do you think it would take you to count the number of values in each bin for:

- an exercise class of 50 people?
- a grocery store with 300 loaves of bread?

Most of the data you will analyze with histograms includes far more than ten values.

For these situations, we can use the `numpy.histogram()` function. In the example below, we use this function to find the counts for a twenty-person exercise class.

```
exercise_ages = np.array([22, 27, 45, 62, 34, 52, 42, 22, 34, 26, 24, 65, 34, 25, 45, 23, 45, 33, 52, 55])  
  
np.histogram(exercise_ages, range = (20, 70), bins = 5)
```

Copy to Clipboard

Below, we explain each of the function's inputs:

`exercise_ages` is the input array
`range = (20, 70)` — is the range of values we expect in our array. Range includes everything from 20, up until but not including 70.
`bins = 5` is the number of bins. Python will automatically calculate equally-sized bins based on the range and number of bins.

Below, you can see the output of the `numpy.histogram()` function:

```
(array([7, 4, 4, 3, 2]), array([20., 30., 40., 50., 60., 70.]))
```

Copy to Clipboard

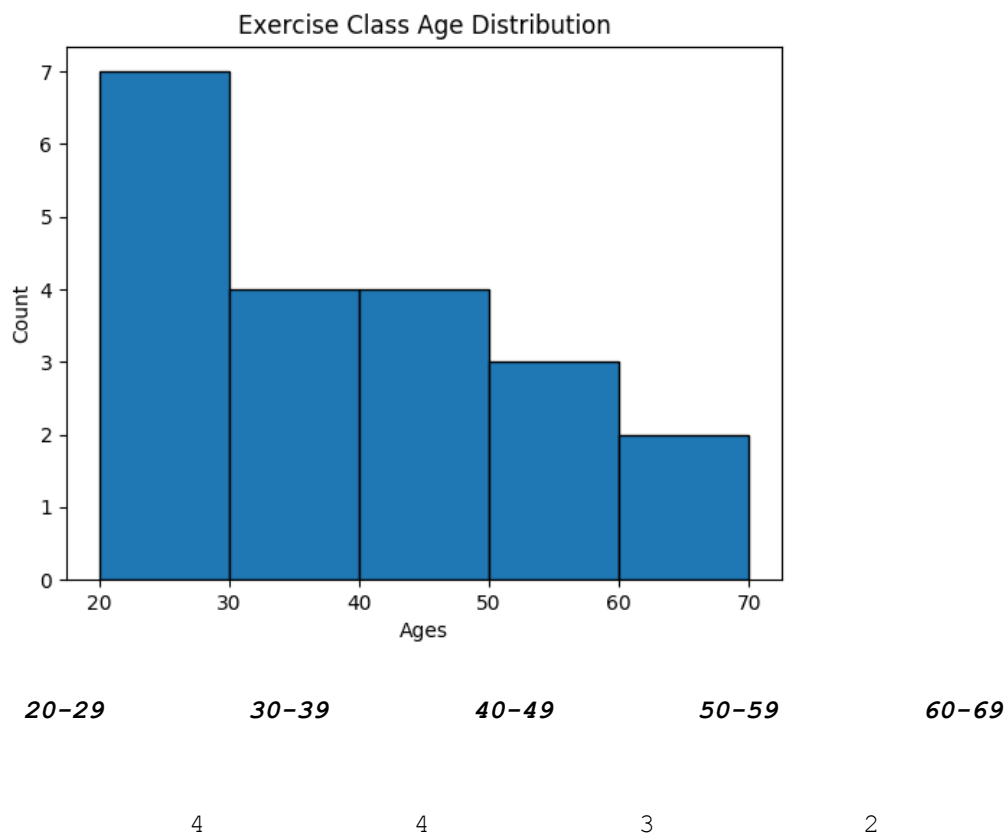
The first array, `array([7, 4, 4, 3, 2])`, is the counts for each bin. The second array, `array([20., 30., 40., 50., 60., 70.])`, includes the minimum and maximum values for each bin:

- **Bin 1:** 20 to <30
- **Bin 2:** 30 to <40
- **Bin 3:** 40 to <50
- **Bin 4:** 50 to <60
- **Bin 5:** 60 to <70

Plotting a Histogram

At this point, you've learned how to find the numerical inputs to a histogram. Thus far the size of our datasets and bins have produced results that we can interpret. This becomes increasingly difficult as the number of bins in a histogram increases.

Because of this, histograms are typically viewed graphically, with bin ranges on the x-axis and counts on the y-axis. The figure below shows the graphical representation of the histogram for our exercise class example from last exercise. Notice, there are five equally-spaced bars, with each displaying a count for an age range. Compare the graph to the table, just below it.



Histograms are an easy way to visualize trends in your data. When I look at the above graph, I think, “More people in the exercise class are in their twenties than any other decade. Additionally, the histogram is skewed, indicating the class is made of more younger people than older people.”

We created the plot above using the `matplotlib.pyplot` package. We imported the package using the following code:

```
from matplotlib import pyplot as plt
```

We plotted the histogram with the following code. Notice, the `range` and `bins` arguments are the same as we used in the last exercise:

```
plt.hist(exercise_ages, range = (20, 70), bins = 5, edgecolor='black')

plt.title("Decade Frequency")
plt.xlabel("Ages")
plt.ylabel("Count")

plt.show()
```

In the code above, we used the `plt.hist()` function to create the plot, then added a title, x-label, and y-label before showing the graph with `plt.show()`.

Describe a Histogram

Introduction

At this point, you should be familiar with what a histogram displays. In this lesson, we're going to build on those skills by learning the best way to describe a statistical distribution.

While many people know the functions to plot a histogram, few spend the time to learn how to fully, and concisely communicate what it means.

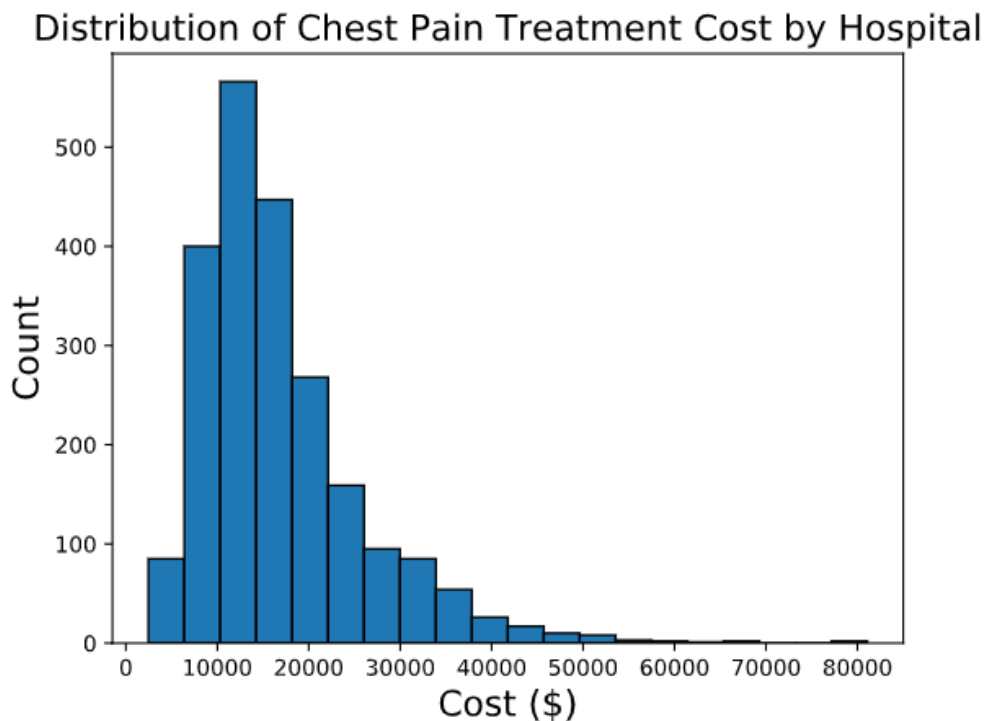
In this lesson, you will learn how to interpret a distribution using the following five features of a dataset:

- Center
- Spread
- Skew
- Modality
- Outliers

If you're one for mnemonics, maybe this will help:

Cream **S**hoes are **S**tylish, **M**odern, and **O**utstanding.

Throughout this lesson, we will use [data from the United States Health and Human Services Department](#) to compare the cost of the same medical procedure at over 2,000 hospitals across the country.

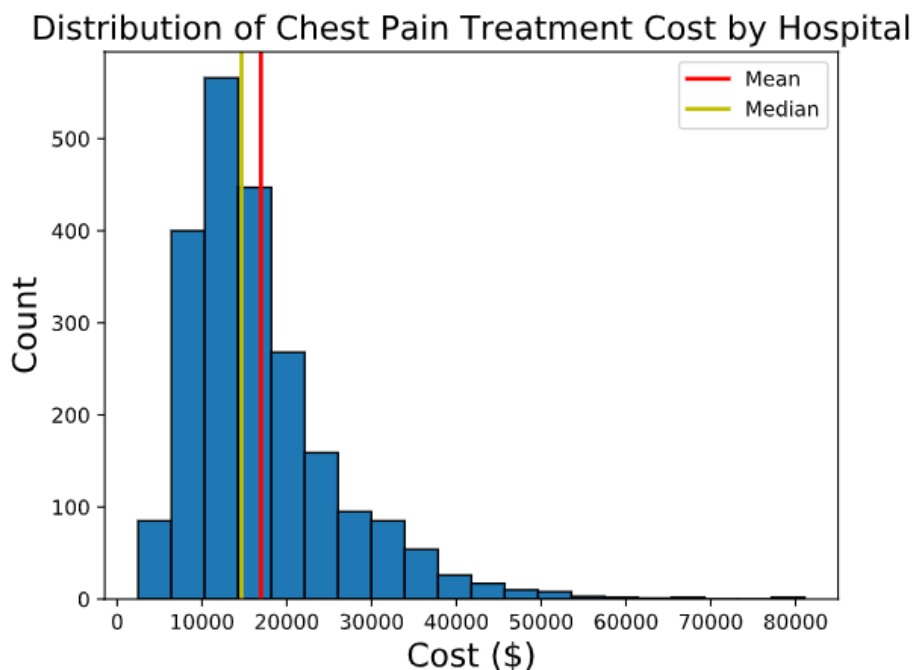


Describe a Histogram

Center

One of the most common ways to summarize a dataset is to communicate its center. In this lesson, we will use average and median as our measures of centrality. Take the Codecademy lessons on [average](#) and [median](#) if you're interested in how to calculate them by hand or using NumPy functions.

The figure below shows the average and median ages of a dataset of 100 authors. As expected, the average and median values are near the center of the distribution.



Spread

Once you've found the center of your data, you can shift to identifying the extremes of your dataset: the minimum and maximum values. These values, taken with the mean and median, begin to indicate the shape of the underlying dataset. Take the histogram below as an example:

The minimum value of this data is 18, and the maximum value is 76.

You can calculate the range using the following:

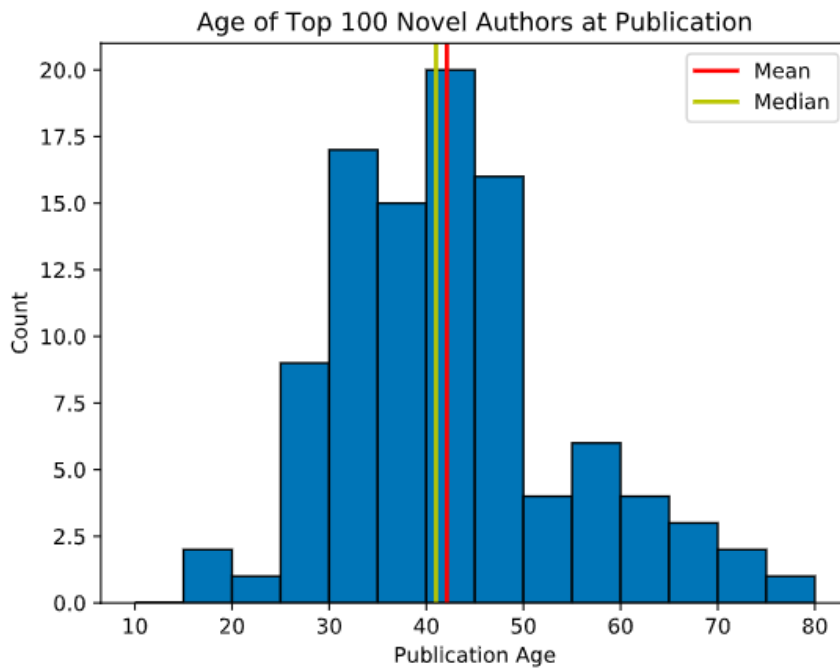
$$\text{range} = \text{max} - \text{min}$$

$$\text{range} = \text{max} - \text{min}$$

The range of this dataset is

$$\text{range} = 76 - 18 = 58$$

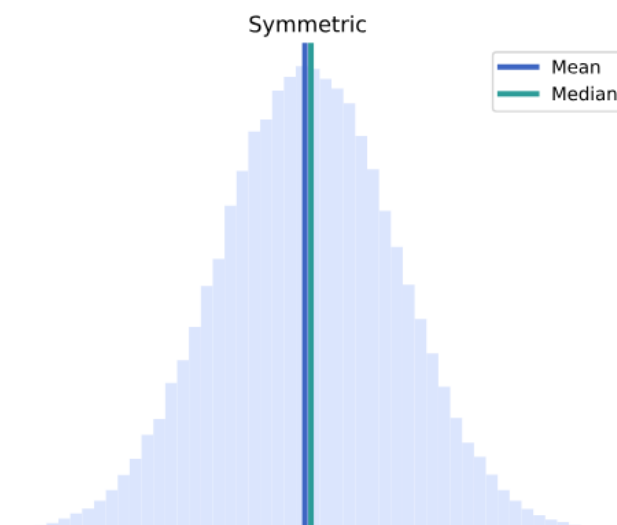
$$\text{range} = 76 - 18 = 58$$



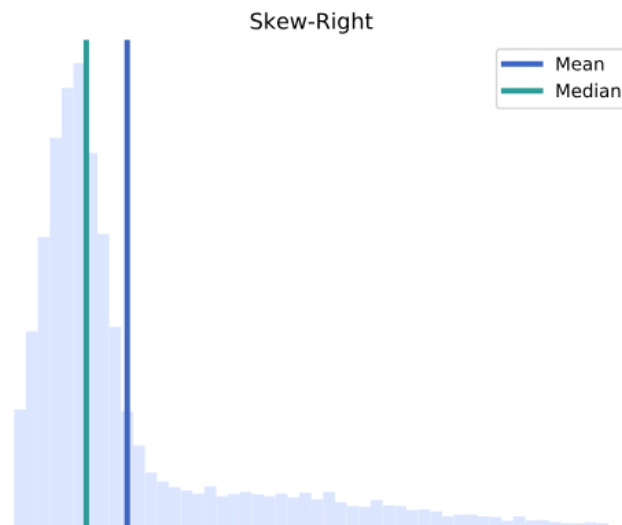
Skew

Once you have the center and range of your data, you can begin to describe its shape. The skew of a dataset is a description of the data's symmetry.

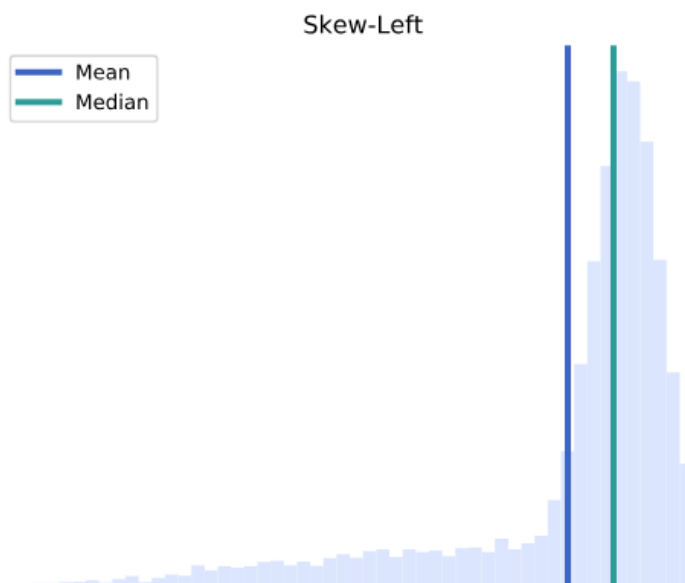
A dataset with one prominent peak, and similar tails to the left and right is called symmetric. The median and mean of a symmetric dataset are similar.



A histogram with a tail that extends to the right is called a right-skewed dataset. The median of this dataset is less than the mean.



A histogram with one prominent peak to the right, and a tail that extends to the left is called a left-skewed dataset. The median of this dataset is greater than the mean.



Instructions

Checkpoint 1 Enabled

1.

Replace the question marks in `?????????-skew` with `left`, `right`, or `no`.
`no` in this circumstance represents a symmetric dataset.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

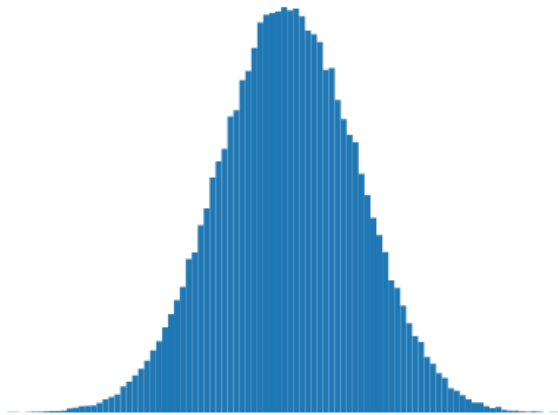
2.

Replace the question marks in `average is ?????????? than the median with greater or less.`

Modality

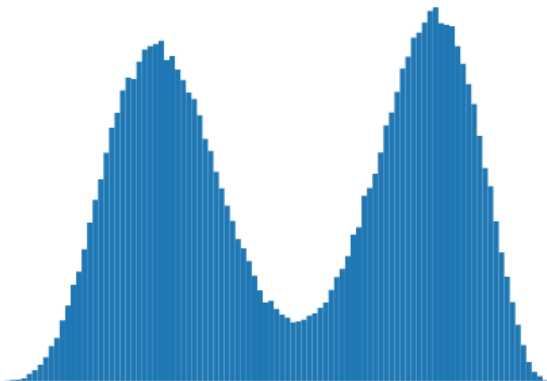
The modality describes the number of peaks in a dataset. Thus far, we have only looked at datasets with one distinct peak, known as *unimodal*. This is the most common.

Unimodal Distribution



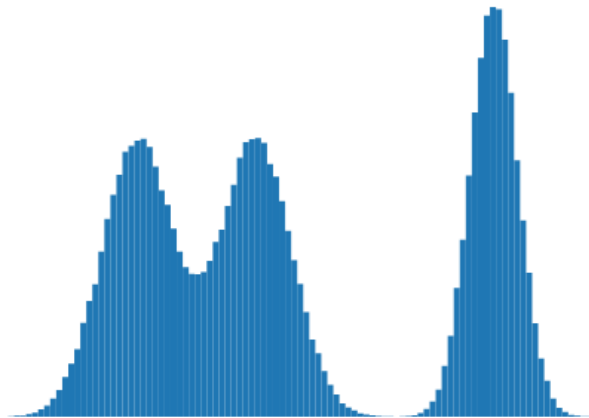
A *bimodal* dataset has two distinct peaks.

Bimodal Distribution



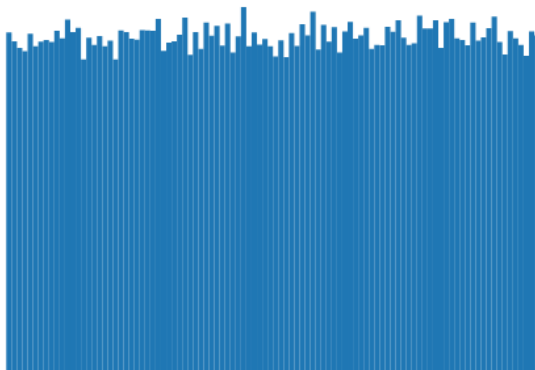
A *multimodal* dataset has more than two peaks. The histogram below displays three peaks.

Multimodal Distribution



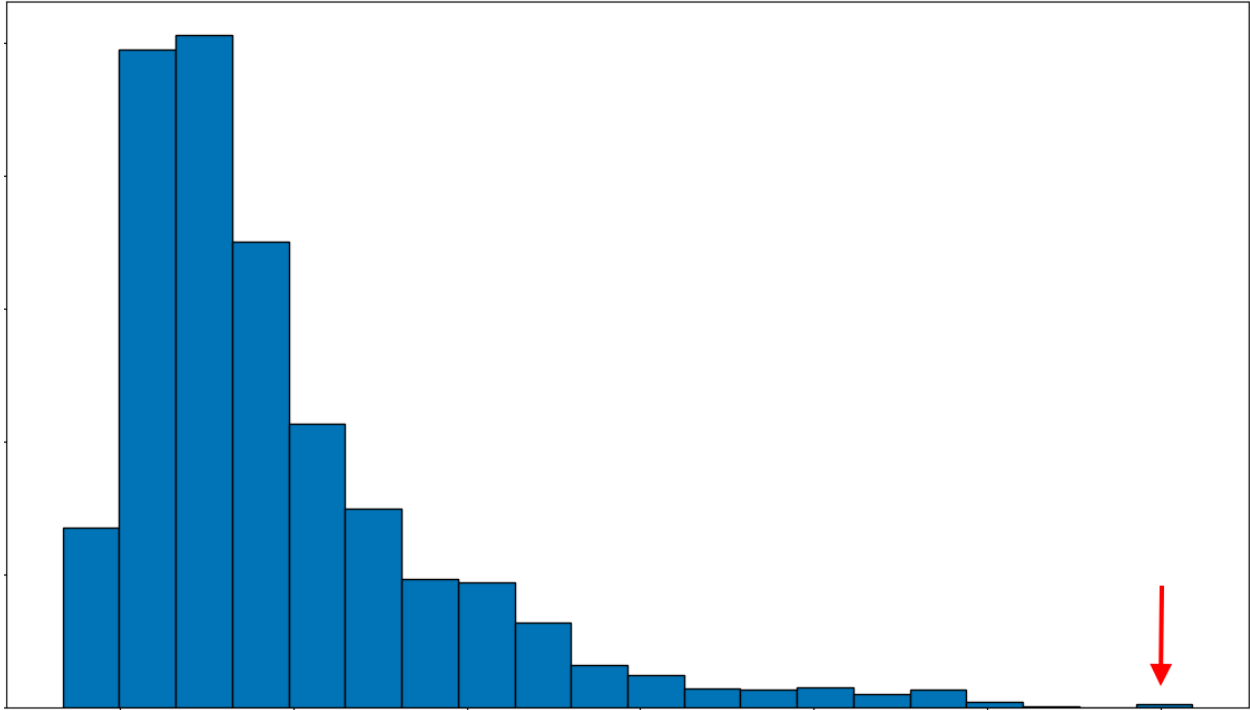
You may also see datasets with no obvious clustering. Datasets such as these are called *uniform distributions*.

Uniform Distribution



Outliers

An outlier is a data point that is far away from the rest of the dataset. Outliers do not have a formal definition, but are easy to determine by looking at histogram. The histogram below shows an example of an outlier. There is one datapoint that is much larger than the rest.



If you see an outlier in your dataset, it's worth reporting and investigating. This data can often indicate an error in your data or an interesting insight.

Review

In this lesson, you learned a

Preview: Docs Loading link description

[framework](#)

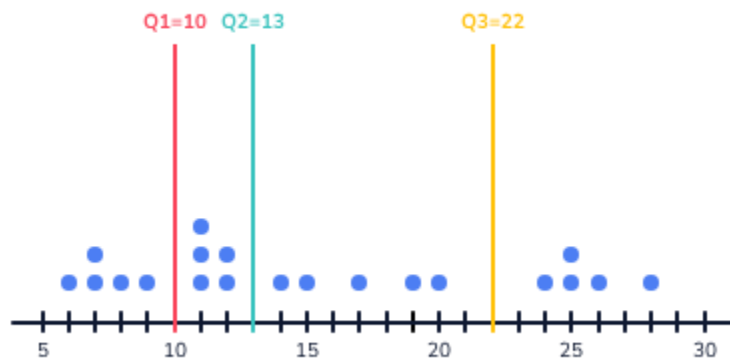
for describing the distribution of a dataset, which includes the following five features:

- Center
- Spread
- Skew
- Modality
- Outliers

Quartiles

A common way to communicate a high-level overview of a dataset is to find the values that split the data into four groups of equal size.

By doing this, we can then say whether a new datapoint falls in the first, second, third, or fourth quarter of the data.



The values that split the data into fourths are the *quartiles*.

Those values are called the first quartile (Q1), the second quartile (Q2), and the third quartile (Q3)

In the image above, Q1 is 10, Q2 is 13, and Q3 is 22. Those three values split the data into four groups that each contain five datapoints.

In this lesson, you will learn to calculate the quartiles by hand, and by using Python's NumPy library.

Instructions

In this lesson we'll be looking at a dataset about music. We've plotted a histogram of song lengths (measured in seconds) of 9,975 random songs.

Look up the length of a favorite song of yours. Do you think that song falls in the first, second, third or fourth quarter of the data?

For example, we've picked one of our favorite songs, *Chicago* by Sufjan Stevens. *Chicago* is 364 seconds long — we've plotted it as a red vertical line. It looks like *Chicago* is in either the third or fourth quarter of the data, but it's hard to say for sure. Let's find the quartiles of the dataset!

The Second Quartile

We'll come back to the music dataset in a bit, but let's first practice on a small dataset.

Let's begin by finding the second quartile (Q2). Q2 happens to be exactly the [median](#). Half of the data falls below Q2 and half of the data falls above Q2.

The first step in finding the quartiles of a dataset is to sort the data from smallest to largest. For example, below is an unsorted dataset:

```
[8,15,4,-108,16,23,42]
```

```
[8,15,4,-108,16,23,42]
```

After sorting the dataset, it looks like this:

```
[-108,4,8,15,16,23,42]
```


`[-108,4,8,15,16,23,42]`

Now that the list is sorted, we can find Q2. In the example dataset above, Q2 (and the median) is 15 – there are three points below 15 and three points above 15.

Even Number of Datapoints

You might be wondering what happens if there is an even number of points in the dataset. For example, if we remove the -108 from our dataset, it will now look like this:

`[4,8,15,16,23,42]`

`[4,8,15,16,23,42]`

Q2 now falls somewhere between 15 and 16. There are a couple of different strategies that you can use to calculate Q2 in this situation. One of the more common ways is to take the average of those two numbers. In this case, that would be 15.5.

Recall that you can find the average of two numbers by adding them together and dividing by two.

Q1 and Q3

Now that we've found Q2, we can use that value to help us find Q1 and Q3. Recall our demo dataset:

`[-108,4,8,15,16,23,42]`

`[-108,4,8,15,16,23,42]`

In this example, Q2 is 15. To find Q1, we take all of the data points smaller than Q2 and find the median of *those* points. In this case, the points smaller than Q2 are:

`[-108,4,8]`

`[-108,4,8]`

The median of that smaller dataset is 4. That's Q1!

To find Q3, do the same process using the points that are larger than Q2. We have the following points:

`[16,23,42]`

`[16,23,42]`

The median of *those* points is 23. That's Q3! We now have three points that split the original dataset into groups of four equal sizes.

Quartiles in NumPy

We were able to find quartiles manually by looking at the dataset and finding the correct division points. But that gets much harder when the dataset starts to get bigger. Luckily, there is a function in the NumPy library that will find the quartiles for you.

The NumPy function that we'll be using is named `quantile()`. You can learn more about quantiles in our [quantiles lesson](#), but for right now all you need to know is that a quartile is a specific kind of quantile.

The code below calculates the third quartile of the given dataset:

```
import numpy as np

dataset = [50, 10, 4, -3, 4, -20, 2]
third_quartile = np.quantile(dataset, 0.75)
```

Copy to Clipboard

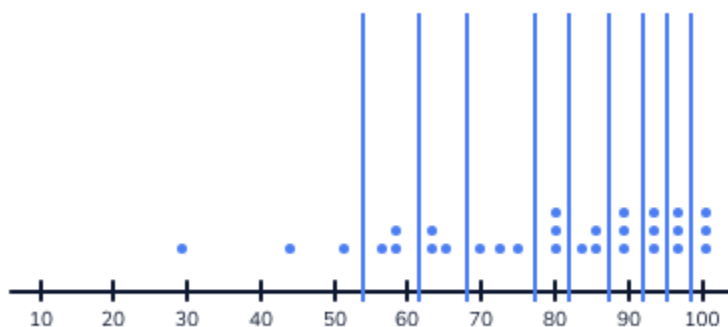
The `quantile()` function takes two parameters. The first is the dataset you're interested in. The second is a number between 0 and 1. Since we calculated the third quartile, we used 0.75 — we want the point that splits the first 75% of the data from the rest.

For the second quartile, we'd use 0.5. This will give you the point that 50% of the data is below and 50% is above.

Notice that the dataset doesn't need to be sorted for NumPy's function to work!

Quantiles

Quantiles are points that split a dataset into groups of equal size. For example, let's say you just took a test and wanted to know whether you're in the top 10% of the class. One way to determine this would be to split the data into ten groups with an equal number of datapoints in each group and see which group you fall into.



There are nine values that split the dataset into ten groups of equal size — each group has 3 different test scores in it.

Those nine values that split the data are quantiles! Specifically, they are the 10-quantiles, or deciles.

You can find any number of quantiles. For example, if you split the dataset into 100 groups of equal size, the 99 values that split the data are the 100-quantiles, or percentiles.

The [quartiles](#) are some of the most commonly used quantiles. The quartiles split the data into four groups of equal size.

In this lesson, we'll show you how to calculate quantiles using NumPy and discuss some of the most commonly used quantiles.

```
import codecademylib3_seaborn
from song_data import songs
import matplotlib.pyplot as plt
import numpy as np

q1 = np.quantile(songs, 0.25)
q2 = np.quantile(songs, 0.5)
q3 = np.quantile(songs, 0.75)

plt.subplot(3,1,1)
plt.hist(songs, bins = 200)
plt.axvline(x=q1, c = 'r')
plt.axvline(x=q2, c = 'r')
plt.axvline(x=q3, c = 'r')
plt.xlabel("Song Length (Seconds)")
plt.ylabel("Count")
plt.title("4-Quantiles")

plt.subplot(3,1,2)
plt.hist(songs, bins = 200)
plt.axvline(x=np.quantile(songs, 0.2), c = 'r')
plt.axvline(x=np.quantile(songs, 0.4), c = 'r')
plt.axvline(x=np.quantile(songs, 0.6), c = 'r')
plt.axvline(x=np.quantile(songs, 0.8), c = 'r')
plt.xlabel("Song Length (Seconds)")
plt.ylabel("Count")
plt.title("5-Quantiles")

plt.subplot(3,1,3)
plt.hist(songs, bins = 200)
for i in range(1, 10):
    plt.axvline(x=np.quantile(songs, i/10), c = 'r')
plt.xlabel("Song Length (Seconds)")
plt.ylabel("Count")
plt.title("10-Quantiles")

plt.tight_layout()
plt.show()
```

Quantiles in NumPy

The NumPy library has a function named `quantile()` that will quickly calculate the quantiles of a dataset for you.

`quantile()` takes two parameters. The first is the dataset that you are using. The second

Preview: Docs A parameter is the name of a variable passed into a function. Parameters allow functions to accept inputs. An argument, on the other hand, is the actual value of the variable (also known as the parameter) passed into a function. Suppose we have a function called tripleThis(): js function tripleThis(x) { return x * 3; } tripleThis(6);

[parameter](#)

is a single number or a list of numbers between 0 and 1. These numbers represent the places in the data where you want to split.

For example, if you only wanted the value that split the first 10% of the data apart from the remaining 90%, you could use this code:

```
import numpy as np

dataset = [5, 10, -20, 42, -9, 10]
ten_percent = np.quantile(dataset, 0.10)
```

Copy to Clipboard

ten_percent now holds the value -14.5. This result *technically* isn't a quantile, because it isn't splitting the dataset into groups of equal sizes — this value splits the data into one group with 10% of the data and another with 90%.

However, it would still be useful if you were curious about whether a data point was in the bottom 10% of the dataset.

Many Quantiles

In the last exercise, we found a single “quantile” — we split the first 23% of the data away from the remaining 77%.

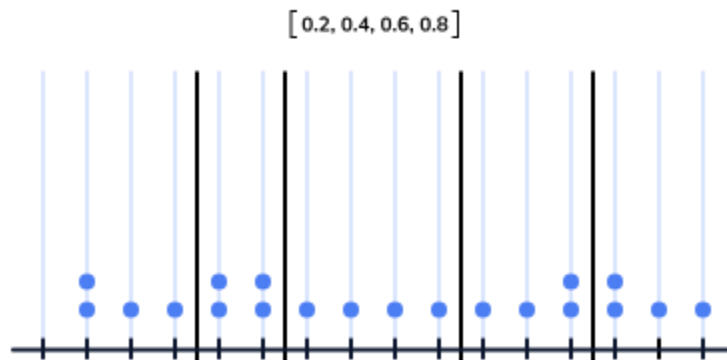
However, quantiles are usually a set of values that split the data into groups of equal size. For example, you wanted to get the 5-quantiles, or the four values that split the data into five groups of equal size, you could use this code:

```
import numpy as np

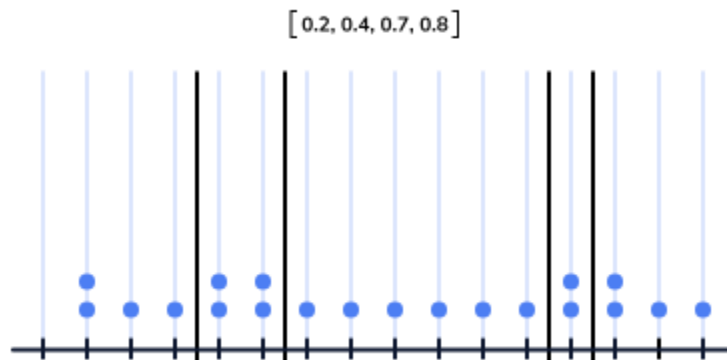
dataset = [5, 10, -20, 42, -9, 10]
ten_percent = np.quantile(dataset, [0.2, 0.4, 0.6, 0.8])
```

Copy to Clipboard

Note that we had to do a little math in our head to make sure that the values [0.2, 0.4, 0.6, 0.8] split the data into groups of equal size. Each group has 20% of the data.



If we used the values [0.2, 0.4, 0.7, 0.8], the function would return the four values at those split points. However, those values wouldn't split the data into five equally sized groups. One group would only have 10% of the data and another group would have 30% of the data!



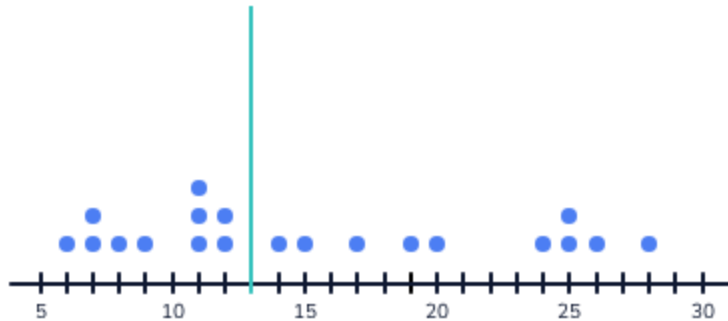
```
from song_data import songs
import numpy as np

# Define quartiles, deciles, and tenth here:
quartiles = np.quantile(songs, [0.25, 0.5, 0.75])
deciles = np.quantile(songs, [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9])

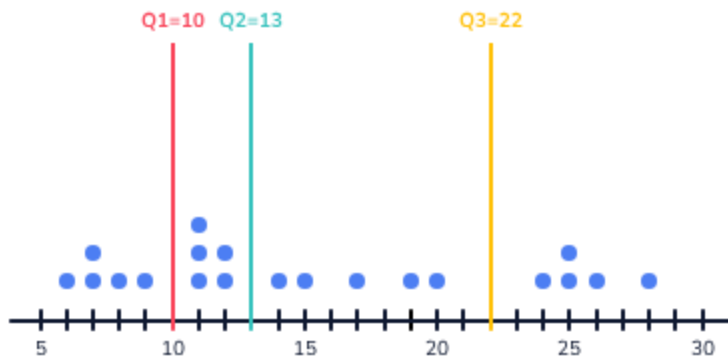
#Ignore the code below here:
try:
    print("The quartiles are " + str(quartiles) + "\n")
except NameError:
    print("You haven't defined quartiles.\n")
try:
    print("The deciles are " + str(deciles) + "\n")
except NameError:
    print("You haven't defined deciles.\n")
```

Common Quantiles

One of the most common quantiles is the 2-quantile. This value splits the data into two groups of equal size. Half the data will be above this value, and half the data will be below it. This is also known as the [median](#)!



The 4-quantiles, or the [quartiles](#), split the data into four groups of equal size. We found the quartiles in the previous exercise. Options



Finally, the percentiles, or the values that split the data into 100 groups, are commonly used to compare new data points to the dataset. You might hear statements like “You are above the 80th percentile in height”. This means that your height is above whatever value splits the first 80% of the data from the remaining 20%.

```
from song_data import songs
import numpy as np

# Define percentile and answer here:
percentile = np.quantile(songs, 0.32)
answer = "below"

# Ignore the code below here:
try:
    print("Your percentile is " + str(percentile) + "\n")
except NameError:
    print("You haven't defined percentile")
```

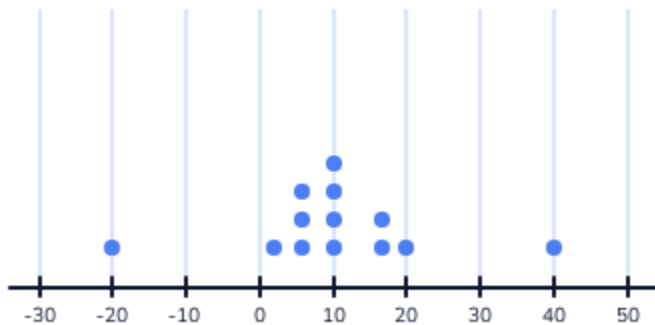
Range Review

One of the most common

Preview: Docs Statistics is the science that is concerned with methods for collecting, organizing, analyzing, and interpreting data.

[statistics](#)

to describe a dataset is the *range*. The range of a dataset is the difference between the maximum and minimum values. While this descriptive statistic is a good start, it is important to consider the impact outliers have on the results:



In this image, most of the data is between 0 and 15. However, there is one large negative outlier (-20) and one large positive outlier (40). This makes the range of the dataset 60 (The difference between 40 and -20). That's not very representative of the spread of the majority of the data!

The *interquartile range* (IQR) is a descriptive statistic that tries to solve this problem. The IQR ignores the tails of the dataset, so you know the range around-which your data is centered.

In this lesson, we'll teach you how to calculate the interquartile range and interpret it.

```
import codecademylib3_seaborn
from song_data import songs
import matplotlib.pyplot as plt

maximum = max(songs)
minimum = min(songs)
#Create the variable song_range here:
song_range = maximum - minimum

# Ignore the code below here
plt.hist(songs, bins = 200)
plt.xlabel("Song Length (Seconds)")
plt.ylabel("Count")
plt.show()

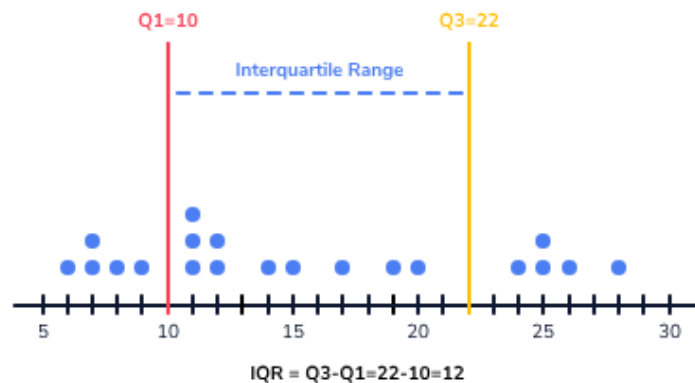
try:
    print("The range of the dataset is " + str(song_range) + " seconds")
except NameError:
    print("You haven't defined the variable song_range yet")
```

Quartiles

The interquartile range is the difference between the third quartile (Q3) and the first quartile (Q1). If you need a refresher on quartiles, you can take a look at our [lesson](#).

For now, all you need to know is that the first quartile is the value that separates the first 25% of the data from the remaining 75%.

The third quartile is the opposite – it separates the first 75% of the data from the remaining 25%.



The interquartile range is the difference between these two values.

```
import codecademylib3_seaborn
from song_data import songs
import numpy as np

q1 = np.quantile(songs, 0.25)
#Create the variables q3 and interquartile_range here:
q3 = np.quantile(songs, 0.75)
interquartile_range = q3 - q1

# Ignore the code below here
try:
    print("The first quartile of the dataset is " + str(q1) + "\n")
except NameError:
    print("You haven't defined q1 yet\n")

try:
    print("The third quartile of the dataset is " + str(q3) + "\n")
except NameError:
    print("You haven't defined q3 yet\n")

try:
    print("The IQR of the dataset is " + str(interquartile_range) + "\n")
except NameError:
    print("You haven't defined interquartile range yet\n")
```

IQR in SciPy

In the last exercise, we calculated the IQR by finding the quartiles using NumPy and finding the difference ourselves. The SciPy library has a function that can calculate the IQR all in one step.

The `iqr()` function takes a dataset as a

Preview: Docs Loading link description

[parameter](#)

and returns the Interquartile Range. Notice that when we imported `iqr()`, we imported it from the `stats` submodule:

```
From scipy.stats import iqr

dataset = [4, 10, 38, 85, 193]
interquartile_range = iqr(dataset)
```

Boxplots

Boxplots are one of the most common ways to visualize a dataset. Like [histograms](#), boxplots give you a sense of the central tendency and spread of the data.

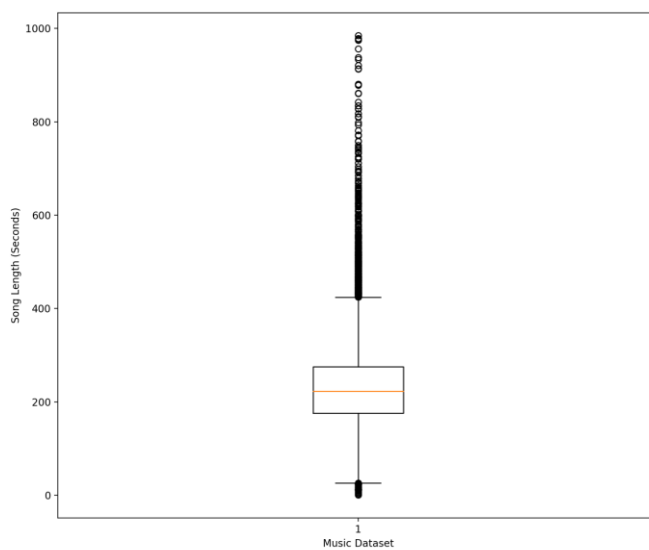
Take a look at the boxplot on this page. This boxplot is visualizing a dataset containing the lengths of 9,975 songs. We'll highlight some of the features of the boxplot and will dig into them in more detail in this lesson:

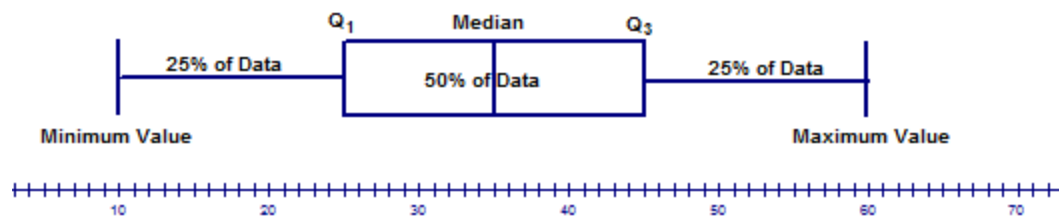
The line in the center of the box is the [median](#).

The edges of the box are the first and third [quartiles](#). This makes the length of the box the [interquartile range](#) — the middle 50% of your data.

The whiskers of the boxplot extend to include most of the data. There are many different ways to calculate the length of the whiskers.

Outliers are points that fall beyond the whiskers. Those points are represented with dots. In the boxplot we're showing, there are *many* outliers.





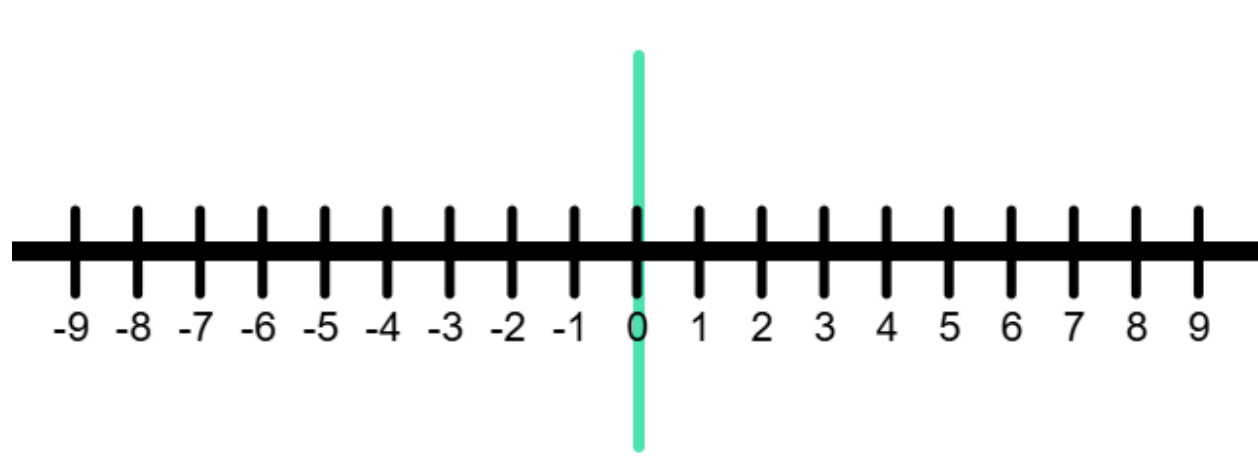
Median

When making a box plot, the easiest place to start is the line that is inside the box. This line is the [median](#) of the dataset. Half of the data falls above that line and half falls below it.

We can find the median of a dataset by using NumPy's `median()` function.

```
import numpy as np

dataset = [4, 8, 15, 16, 23]
dataset_median = np.median(dataset)
# dataset median stores the value 15
```



Interquartile Range

Now that we've drawn the median, let's draw the edges of the box. The box extends to the first and third [quartile](#) of the dataset.

This visually splits the data into fourths. One-quarter of the data will fall outside of the box to the left. Another quarter of the data will fall between the left side of the box and the median line. A third quarter of

the data falls between the median line and the right side of the box. And the last quarter of the data falls outside the box to the right.

By drawing the edges of the box using the first and third quartiles, the [interquartile range](#) is visualized. The length of the box represents this commonly used descriptive statistic.

We can use NumPy's `quantile()` function to find the first and third quartile:

```
dataset = [4, 8, 15, 16, 23]
first_quartile = np.quantile(dataset, 0.25)
third_quartile = np.quantile(dataset, 0.75)
```

Whiskers

The whiskers of a boxplot display information related to the spread of the dataset.

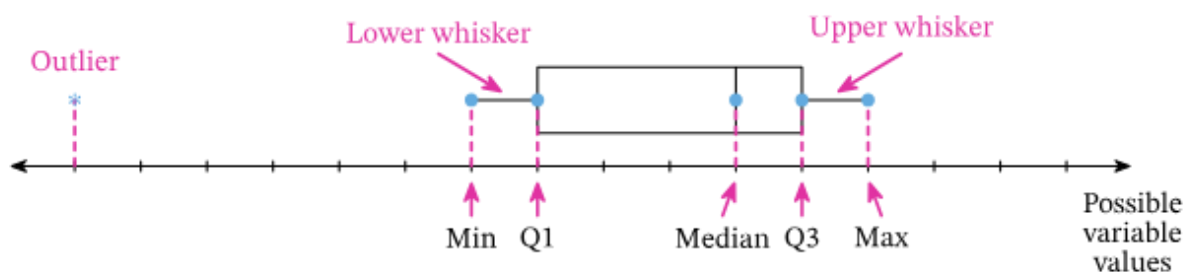
There are many different ways to plot the whiskers of a boxplot. You might see some boxplots where the whiskers extend to the minimum and maximum values. Some boxplots have whiskers that extend one standard deviation away from the mean of the data.

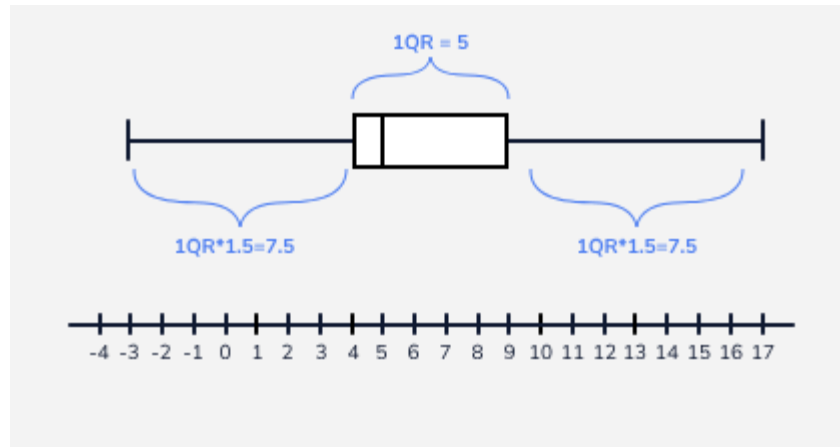
However, one of the most commonly used methods of drawing the whiskers is to extend them 1.5 times the interquartile range from the first and third quartile.

For example, let's say you had a dataset where the first quartile was 4 and the third quartile was 9. The interquartile range for this dataset is 5.

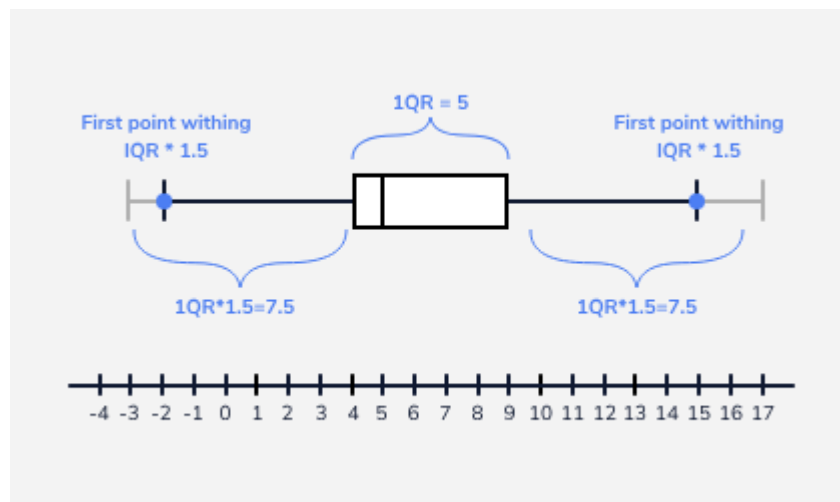
The whiskers would extend 1.5 times the length of the IQR. In this case, that is $1.5 * 5$, or 7.5.

We know the whiskers extend 7.5 units, but where do they start? They start at the edges of the box, or the first and third quartiles. In this case, the left whisker starts at the first quartile (4), and extend 7.5 units to the left. So the left whisker extends to -3.5. The right whisker starts at the third quartile (9) and extends to 16.5.





One more small detail to note – the whiskers usually don’t extend all the way to 1.5 times the IQR. Instead, they extend to the point closest to 1.5 times the IQR in the direction of the median. This means that instead of extending to -3.5 and 16.5, the whiskers would actually extend to the first point greater than -3.5 and the first point less than 16.5.



Outliers

The final piece of a boxplot is the representation of outliers. An outlier is a point in the dataset that falls outside of the whiskers. Outliers are usually represented with a dot or an asterisk.

Boxplots in Matplotlib

We’ve spent this lesson building a boxplot by hand. Let’s now look at how Python’s Matplotlib library does it!

The `matplotlib.pyplot` module has a function named `boxplot()`. `boxplot()` takes a dataset as a

Preview: Docs Loading link description

[parameter](#)

. This dataset could be something like a list of numbers, or a Pandas DataFrame.

```
import matplotlib.pyplot as plt

data = [1, 2, 3, 4, 5]
plt.boxplot(data)
plt.show()
```

Copy to Clipboard

One of the strengths of Matplotlib is the ease of plotting two boxplots side by side. If you pass `boxplot()` a list of datasets, Matplotlib will make a boxplot for each, allowing you to compare their spread and central tendencies,

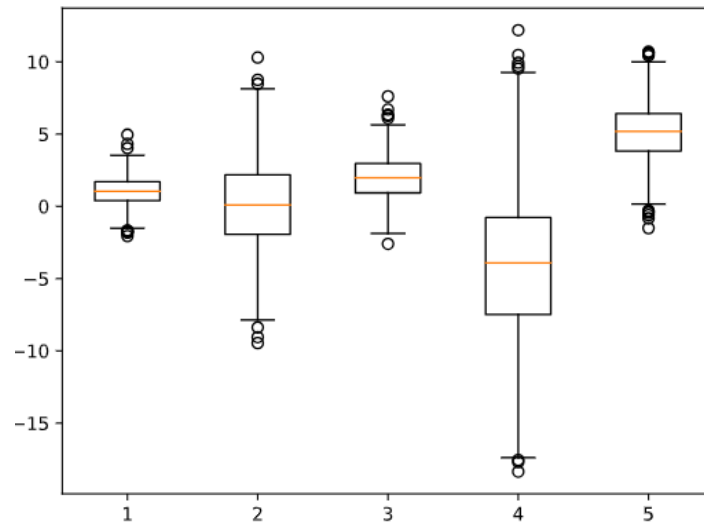
```
import matplotlib.pyplot as plt

dataset_one = [1, 2, 3, 4, 5]
dataset_two = [3, 4, 5, 6, 7]
plt.boxplot([dataset_one, dataset_two])
plt.show()
```

Review

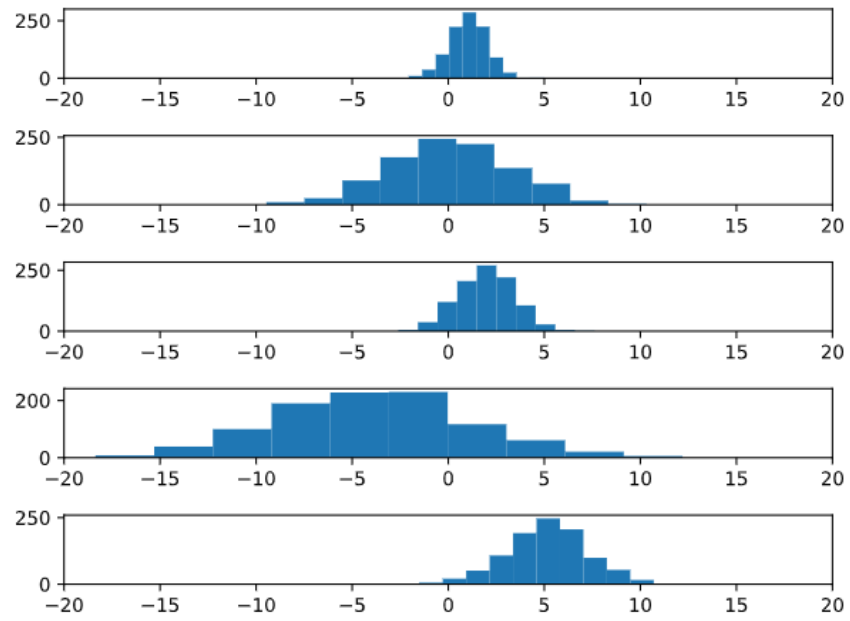
Great work! Here are some of the major takeaways from boxplots:

- The box of a boxplot visualizes the median, first quartile, and third quartile of a dataset.
- The length of the box in a boxplot visualizes the interquartile range.
- The whiskers extend from the box 1.5 times the size of the interquartile range.
- Outliers are points that fall outside of the whiskers. They're represented by dots.
- Boxplots are especially useful for comparing the spread of multiple datasets.



```
import codecademylib3_seaborn
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(1)

a = np.random.normal(1, 1, 1000)
b = np.random.normal(0, 3, 1000)
c = np.random.normal(2, 1.5, 1000)
d = np.random.normal(-4, 5, 1000)
e = np.random.normal(5, 2, 1000)
plt.boxplot([a,b,c,d,e])
plt.show()
plt.subplot(511)
plt.hist(a)
plt.xlim([-20,20])
plt.subplot(512)
plt.hist(b)
plt.xlim([-20,20])
plt.subplot(513)
plt.hist(c)
plt.xlim([-20,20])
plt.subplot(514)
plt.hist(d)
plt.xlim([-20,20])
plt.subplot(515)
plt.hist(e)
plt.xlim([-20,20])
plt.tight_layout()
plt.show()
```



Categorical Data

When exploring data, we're often interested in summarizing a large amount of information with a single number or visualization.

Depending on what we're trying to understand from our data, we may need to rely on different

Preview: Docs Statistics is the science that is concerned with methods for collecting, organizing, analyzing, and interpreting data.

[statistics](#)

. For quantitative data, we can summarize central tendency using mean, median or mode and we can summarize spread using standard deviation, variance, or percentiles. However, when working with categorical data, we may not be able to use all the same summary statistics.

For example, here are the first five rows and some selected columns of a dataset from the 1994 U.S. census:

age	education	marital.status	race
90	HS-grad	Widowed	White
82	HS-grad	Widowed	White
66	Some-college	Widowed	Black
54	7th-8th	Divorced	White
41	Some-college	Separated	White

Age is a quantitative variable, so we can calculate the average (or mean) age. However, for a variable like `marital.status`, we can't calculate something like "average marital status" because the possible values of marital status are categories rather than numbers (e.g. "Married", "Widowed", "Separated", etc.). This lesson will cover summary statistics specifically for exploring categorical data.

Nominal Categories

Depending on the data, some of the summary

Preview: Docs Loading link description

[statistics](#)

we use for quantitative data can still be meaningful for categorical data. Let's first consider a *nominal categorical variable*. A nominal categorical variable is a categorical variable with no intrinsic ordering to

the categories. Examples from the census dataset introduced in the previous exercise include `marital.status` and `race`.

Because these variables' categories have no ordering or numeric equivalents, it's impossible to calculate a mean or median. It would also be impossible to describe spread with statistics like variance, standard deviation, a range, IQR, or percentiles, because these statistics all rely on being able to order the data in some way. However, it is still possible to calculate the *mode*, the most common value in the dataset.

We can do this in Python using the `.value_counts()` function. The `.value_counts()` function calculates the count of each value in a column and returns the result as a series. By default, `.value_counts()` orders categories in descending order by frequency, thus the top row in the output will be the mode.

In the code below, we use `.value_counts()` to extract the most common responses in the field `marital.status`.

```
counts = df['marital.status'].value_counts()
print(counts)
```

Copy to Clipboard

Output:

Married-civ-spouse	14976
Never-married	10683
Divorced	4443
Separated	1025
Widowed	993
Married-spouse-absent	418
Married-AF-spouse	23

This means that the most common value of `marital.status` in this dataset is 'Married-civ-spouse' (married to a civilian spouse), with 14976 observations in that category. We can also extract the name of the modal category by taking the first value from the series `.value_counts()` returns.

```
modal_cat = counts.index[0]
print(modal_cat) # Output: Married-civ-spouse
```

Ordinal Categorical Variables - Central Tendency I

Ordinal categorical variables have ordered categories. For ordinal categorical

Preview: Docs Loading link description

[variables](#)

, we can find the modal category just like in the previous exercise — but we can also calculate other summary

Preview: Docs Loading link description

[statistics](#)

that are not possible for nominal categorical variables. For central tendency, this means we can also calculate a median.

In order to calculate numerical statistics for ordered categories, we need to first assign numerical values to the categories. Consider the variable `education` from the census data. We can inspect the unique categories in this variable using `.unique()`:

```
print(list(df['education'].unique()))
```

Copy to Clipboard

Output:

```
['HS-grad', 'Some-college', '7th-8th', '10th', 'Doctorate', 'Prof-school',  
'Bachelors', 'Masters', '11th', 'Assoc-acdm', 'Assoc-voc', '1st-4th', '5th-  
6th', '12th', '9th', 'Preschool']
```

Copy to Clipboard

Then, we can associate each of these categories with a numerical value, indicating an individual's "education level". In Python, the easiest way to do this is to convert the variable to type `'category'` using `pandas.Categorical()`. When converting a column to type `'category'`, we can also pass a list with the column's categories (and `True` to the `ordered` parameter) to indicate the desired ordering.

```
correct_order = ['Preschool', '1st-4th', '5th-6th', '7th-8th', '9th', '10th',  
'11th', '12th', 'HS-grad', 'Some-college', 'Assoc-voc', 'Assoc-acdm',  
'Bachelors', 'Masters', 'Prof-school', 'Doctorate']  
  
df['education'] = pd.Categorical(df['education'], correct_order,  
ordered=True)
```

Copy to Clipboard

Variables stored as type `category` have an attribute (`cat.codes`) that converts the categories to numbers. This allows us to perform numerical operations on this categorical field. This allows us to calculate the median category using numpy's `median()` function:

```
median_index = np.median(df['education'].cat.codes)  
print(median_index) # Output: 9  
  
median_category = correct_order[int(median_index)]  
print(median_category) # Output: Some college
```

Copy to Clipboard

By using `.cat.codes` on `education`, we're able to calculate that the median value for education level is `'9'` which translates to `'Some college'`.

Ordinal Categorical Variables - Central Tendency II

In the previous exercise, we used `.cat.codes` to find the median category for an ordinal categorical variable. We can use `cat.codes` to return numeric values and perform a wide range of operations on categorical data as well. However, before performing any operations, you should check to make sure they make sense in the context of the data.

For example, remember that the categories for `education` (in order) are as follows:

```
education_levels_ordered = ['Preschool', '1st-4th', '5th-6th', '7th-8th', '9th', '10th', '11th', '12th', 'HS-grad', 'Some-college', 'Assoc-voc', 'Assoc-acdm', 'Bachelors', 'Masters', 'Prof-school', 'Doctorate']
```

Copy to Clipboard

While we can represent these categories with equally spaced numbers, there's not equal spacing between categories. Some gaps between educational attainment levels represent up to four additional years of schooling (e.g. '1st-4th' to '5th-6th'), while others represent a single additional year of schooling (e.g. from '9th' to '10th').

When we use `.cat.codes` to translate these categories into integers, those integers have equal spacing. While translating categories to numbers is often necessary to store and use the order of the categories (for calculating a statistic like the median, which only relies on ordering, not spacing), we should not use those numbers to calculate

Preview: Docs Loading link description

[statistics](#)

– such as the mean – for which the distance between values matters.

In practice, researchers sometimes (albeit, incorrectly) report means for ordinal categories. For example, a researcher might want to analyze survey responses to the question "Rate your happiness on a scale from 1 to 5 where 1 means 'very unhappy' and 5 means 'very happy'".

If that researcher calculates 'mean happiness score', they are assuming that the difference in happiness between a rating of 1 and 2 is the same as the difference in happiness for a rating of 3 and 4. In practice, this assumption is likely not true and should be acknowledged if reporting a mean of an ordinal categorical variable.

```
import pandas as pd
import numpy as np

# Read NYC Trees Data
nyc_trees = pd.read_csv("nyc_tree_census2.csv")

nyc_trees.tree_diam_category = pd.Categorical(nyc_trees.tree_diam_category,
['Small (0-3in)', 'Medium (3-10in)', 'Medium-Large (10-18in)', 'Large (18-24in)', 'Very large (>24in)'], ordered=True)

# Get Mean Diam of diameter variable, `trunk_diam`
mean_diam = np.mean(nyc_trees.trunk_diam)
print(mean_diam)

# Get Mean Category of `tree_diam_category`
mean_diam_cat = np.mean(nyc_trees.tree_diam_category.cat.codes)
print(mean_diam_cat)
```

Ordinal Categories: Spread

In the last exercise, we learned that the mean is not interpretable for ordinal categorical

Preview: Docs Loading link description

[variables](#)

because the mean relies on the assumption of equal spacing between categories.

Many other

Preview: Docs Statistics is the science that is concerned with methods for collecting, organizing, analyzing, and interpreting data.

[statistics](#)

we might normally use for numerical data rely on the mean. Because of this, these statistics aren't appropriate for ordinal data. Remember that the standard deviation and variance both depend on the mean, without a mean, we can't have a reliable standard deviation or variance either!

Instead, we can rely on other summary statistics, like the proportion of the data within a range, or percentiles/quantiles. For example, consider the education variable from earlier. To calculate a range containing 80% of the data, we can use `np.percentile()`:

```
tenth_perc_ind = np.percentile(df['education'].cat.codes, 10)
tenth_perc_cat = correct_order[int(tenth_perc_ind)]
print(tenth_perc_cat) # output: 11th

nintietth_perc_ind = np.percentile(df['education'].cat.codes, 90)
nintietth_perc_cat = correct_order[int(nintietth_perc_ind)]
print(nintietth_perc_cat): #output: Bachelors
```

Copy to Clipboard

This tells us that at least 80% of respondents range in "education level" from 11th grade to a Bachelor's degree.

```
import pandas as pd
import numpy as np

# Read NYC Trees Data
nyc_trees = pd.read_csv("nyc_tree_census2.csv")

size_labels_ordered = ['Small (0-3in)', 'Medium (3-10in)', 'Medium-Large (10-18in)', 'Large (18-24in)', 'Very large (>24in)']

nyc_trees.tree_diam_category = pd.Categorical(nyc_trees.tree_diam_category,
size_labels_ordered, ordered=True)

# Calculate 25th Percentile Category
p25_ind = np.percentile(nyc_trees['tree_diam_category'].cat.codes, 25)
p25_tree_diam_category = size_labels_ordered[int(p25_ind)]
print(p25_tree_diam_category)

# Calculate 75th Percentile Category
p75_ind = np.percentile(nyc_trees['tree_diam_category'].cat.codes, 75)
p75_tree_diam_category = size_labels_ordered[int(p75_ind)]
print(p75_tree_diam_category)
```

Table of Proportions

You've already seen that we can use the `.value_counts()` function to get a table of frequencies for a categorical variable. A table of frequencies is often the first approach a data scientist might use to summarize a categorical variable; however, it is sometimes useful to instead look at the proportion of values in each category.

For example, knowing that there are 14976 people in the census dataset who are married to a civilian spouse is hard to interpret without the context of knowing the numbers in the other categories. Instead, if we know that 32% of the surveyed population is married to a civilian spouse, we have more context about the relative frequency of this category. We can calculate proportions by dividing the frequency by the number of observations in the data.

```
df['education'].value_counts()/len(df['education'])
```

Copy to Clipboard

We can also calculate proportions using `.value_counts()` by setting the `normalize`

Preview: Docs Loading link description

[parameter](#)

equal to `True`:

```
df['education'].value_counts(normalize = True).head()
```

```
import pandas as pd
import codecademylib3

# Read NYC Trees data
nyc_trees = pd.read_csv("./nyc_tree_census.csv")

# Calculate a table of proportions for `status`
tree_status_proportions = nyc_trees['status'].value_counts(normalize=True)
print(tree_status_proportions)
```

Table of Proportions: Missing Data

One thing to keep in mind when calculating the proportion of data in a particular category: how are you dealing with missing data? For example, consider the `workclass` variable from the census data. This column contains 1836 missing values, coded as `NaN`. By default, those missing values are not counted by `.value_counts()`.

Therefore, the results of `df['workclass'].value_counts()/len(df['workclass'])` and `df['workclass'].value_counts(normalize = True)` will be slightly different. You can set the `dropna`

Preview: Docs Loading link description

[parameter](#)

in `.value_counts()` to determine how `NaN` values are handled in summaries of data.

When we divide the frequency of each category by `len(df['workclass'])`, we're calculating the proportion of a specific `workclass` group as a portion of all people in the dataset. This is equivalent to setting `dropna = False` in the call to `value_counts()`.

```
df.workclass.value_counts(dropna = False, normalize = True)
```

```
import pandas as pd
import numpy as np

# Get NYC Trees Data
nyc_trees = pd.read_csv("nyc_tree_census.csv")

# Create proportions - Excluding Null Responses
health_proportions = nyc_trees['health'].value_counts(normalize=True)
print(health_proportions)

# Create proportions - Include Null Responses
health_proportions_2 = nyc_trees['health'].value_counts(normalize=True,
dropna=False)
print(health_proportions_2)
```

Binary Categorical Variables

Preview: Docs Binary is a number system of 1s and 0s which serves as a textual representation method for giving instructions to a computer's CPU. In computing and telecommunications, binary codes are used for various methods of encoding data such as turning the character strings of source code, into bit strings or instruction sets for the CPU to execute.

[Binary](#)

categorical

Preview: Docs Loading link description

[variables](#)

have only two categories. In Python, these variables are often coded as 0/1 or True/False. This makes it easy to calculate the frequency/proportion of these variables in a dataset. For example, consider a variable `income > 50K`, which is equal to 1 if a person makes more than 50k U.S.D per year, and 0 otherwise. If we add up all the 1s and 0s in this column, the sum will be exactly equal to the number of 1s (people making more than 50k):

```
np.sum(df['income > 50K']) #output: 7841
```

Copy to Clipboard

In Python, the same behavior holds for columns coded as True/False because True gets coerced to 1 and False gets coerced to 0 (this is also true in most other programming languages used by data scientists). Similarly, we can calculate the proportion equal to 1 or True by taking the mean of the column. This works because the mean is just the sum of all values in the column (which is the frequency of 1s or Trues) divided by the number of values in the column:

```
np.mean(df['income_>50K']) #output: 0.24
```

Copy to Clipboard

Finally, we can make use of this nifty trick for any variable by using a conditional to translate a non-binary variable into `True` and `False` values. For example, recall the `workclass` variable from the previous exercise. Suppose that you want to calculate the number (or proportion) of people who work in local government. We could translate the `workclass` column to a binary variable indicating whether a person works in local government (`True`) or not (`False`) by using a conditional.

```
print(df.workclass == 'Local-gov')
```

Discretizing Numerical Data and Collapsing Categories

An introduction to binning numerical data and combining categorical data

In this article, you will learn simple transformation techniques for numerical and categorical data to help make your data easier to analyze, including:

- What it means to bin numerical data and why it's important

- How to bin numerical data in Python

- Why combining categorical data can be useful for analysis/visualization

- How to combine categorical data in Python

Let's get started!

Binning Data

As a data analyst or scientist, you will sometimes deal with numerical data that you would like to arrange into different categories. For example, imagine you are looking at the ages of a set of

individuals. You might care more about the age category each person falls under (20-29, 30-39, 40-49, etc) than the exact age. The process of transforming numerical variables into categorical counterparts is called “binning.”

Binning is a way to group a number of continuous values into a smaller number of “bins”. We see this in the real world quite often. For example, a student’s letter grade is determined by the percentage “bin” they fall under. Moreover, an individual’s tax rate is determined by the income “bin” they fall under.

Why bin data?

Binning data can be useful in many situations, whether it be to create more appealing visualizations or to improve a machine learning model.

In areas like machine learning, binning data can be helpful to improve the accuracy of predictive models by reducing the noise of the data. For example, if you’re a machine learning engineer working at an investment firm, you might use binning as a technique to improve your stock prediction models by smoothing the data to reduce the impact of small, short term price changes.

Now that you have a better understanding of why we bin data, let’s explore how to bin numerical data in Python!

Binning Numerical Data in Python

In this example, we will look at the ages of students in a dance class (stored in a file called `dance_class_data.csv`).

First, we import pandas – a Python library to perform data manipulation and analysis. Then, we load in our data and take a peek at the first 10 rows of the data:

```
import pandas as pd

# Load in data
dance_class = pd.read_csv('dance_class_data.csv')

# Print the first 10 rows
print(dance_class.head(10))
```

The output looks like:

	Name	Gender	Age	Experience
0	Chris Shelton	M	23	beginner
1	Douglas Watson	M	28	intermediate
2	Martha Gomez	F	45	beginner
3	Amos Moore	M	63	beginner
4	Valentina Sen	F	35	beginner
5	Billy Woods	M	53	advanced
6	Oscar Barker	M	43	intermediate
7	Marie Sandoval	F	23	beginner
8	Nancy McBride	F	35	beginner
9	Lindsay Bowen	F	27	beginner

The `dance_class` dataframe has four columns – `Name`, `Gender`, `Age`, and `Experience`. We can find out the data types of these columns by using the `dtypes` property:

```
print(dance_class.dtypes)
```

The output looks like:

Name	object
Gender	object
Age	int64
Experience	object

Notice that the `Age` column is of type `int`. This is the column we'll be exploring.

We're curious to see which age range each student in the class falls under. To do this, we can bin the values to specific age groups. Before we do that, let's store the age of the students and find out the minimum and maximum ages:

```
# Store the ages
student_ages = dance_class['Age']

# min() method returns the lowest value
print(student_ages.min()) # 23

# max() method returns the highest value
print(student_ages.max()) # 65
```

The youngest student in the class is 23 years old and the oldest is 65 years old. We can therefore define our bin boundaries by decade (20-29, 30-39, 40-49, 50-59, 60-69).

Let's first create a `bins` variable to store these values:

```
# Store the boundaries
bins = [20, 30, 40, 50, 60, 70]
```

Next, we can create the bins using `pd.cut(df['column_name'], bins)` where `bins` is either:

- An integer specifying the number of evenly spaced bins, or

- A list of bin boundaries

Let's bin the value of the `Age` column into equally sized bins in a new *column* called `binned_age`. Then, let's print out the first few rows of the data:

```
# Create new binned_age column that bins the values of the 'Age' column
dance_class['binned_age'] = pd.cut(dance_class['Age'], bins)

# Print the first few rows of the data
print(dance_class[['binned_age', 'Age']].head())
```

The output looks like:

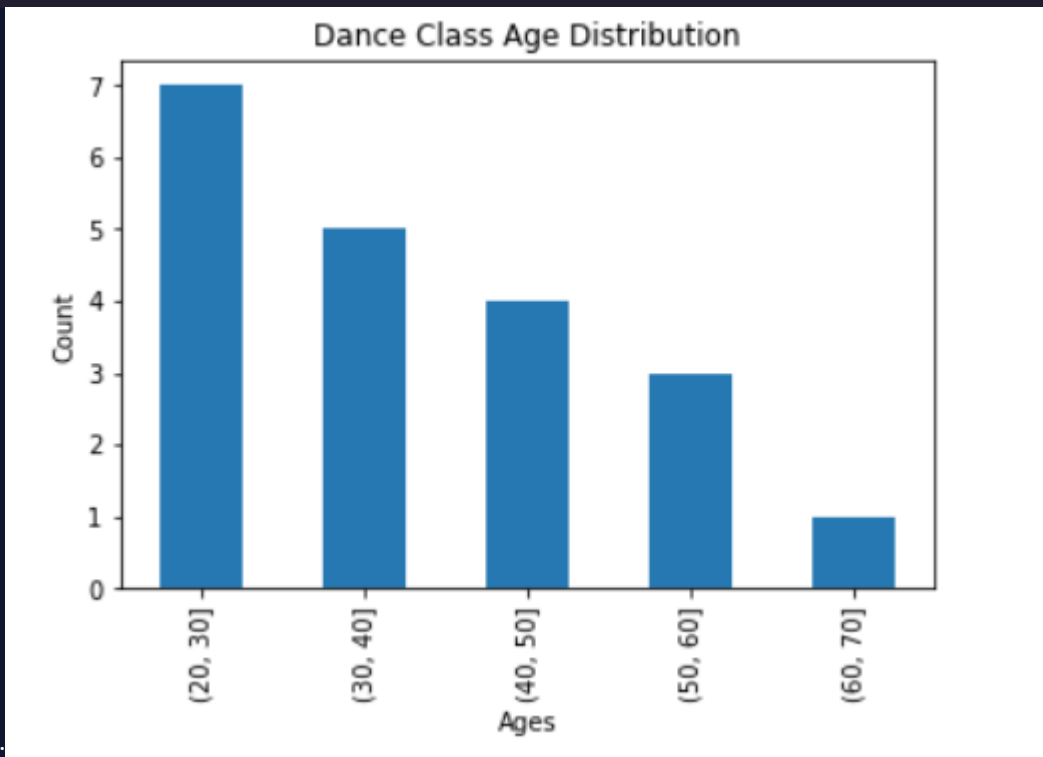
	binned_age	Age
0	(20, 30]	23
1	(20, 30]	28
2	(40, 50]	45
3	(60, 70]	63
4	(30, 40]	35

Notice how each age now falls under a specific age group. We can also plot our data in a bar graph to get a better visualization:

```
# Plot the bar graph of binned ages
dance_class['binned_age'].value_counts().plot(kind='bar')

# Label the bar graph
plt.title('Dance Class Age Distribution')
plt.xlabel('Ages')
plt.ylabel('Count')
```

```
# Show the bar graph
plt.show()
```



Output:

Notice how the data is categorized into different age groups, allowing us to see the big picture of our data without focusing too much on the precise ages of each individual. Based on this plot, we can see that the class skews toward younger ages.

If we want, we can also specify labels for our bins using a `labels` argument:

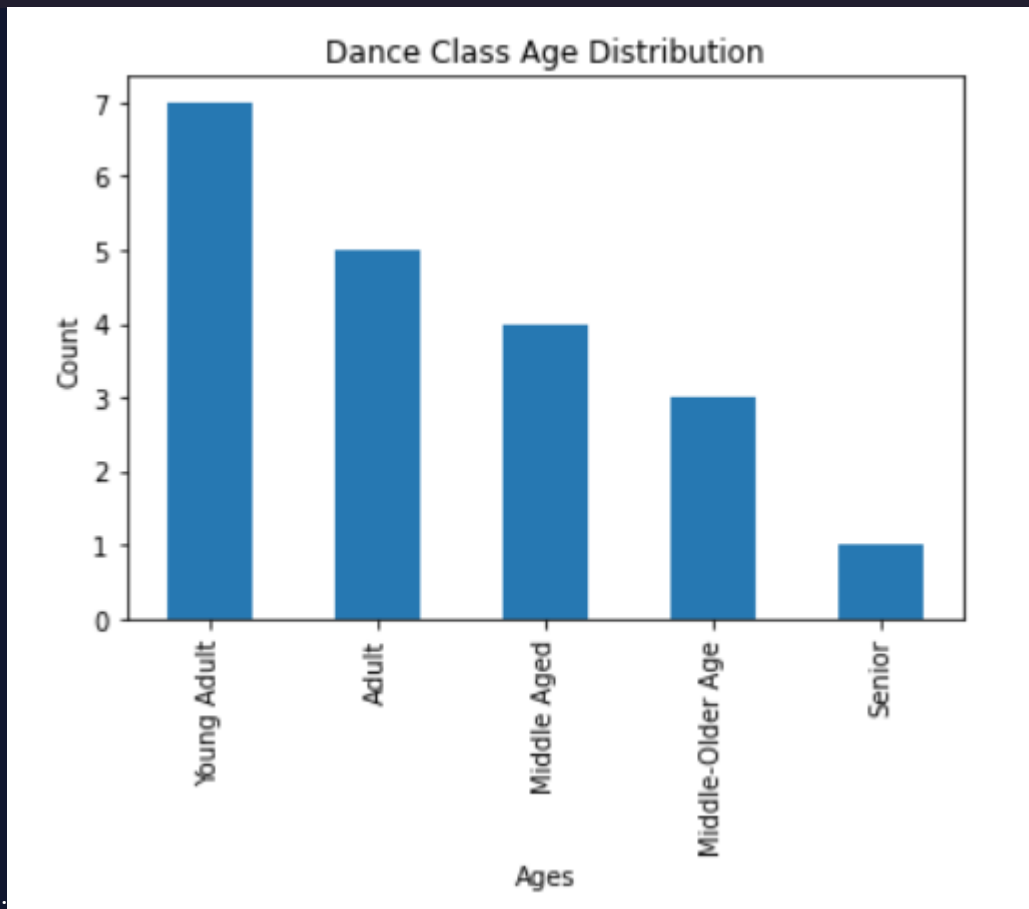
```
# Store the boundaries
bins = [20, 30, 40, 50, 60, 70]

# Store the labels for our bins
age_labels = ['Young Adult', 'Adult', 'Middle Aged', 'Middle-Older Age', 'Senior']

# Bin the values of the 'Age' column and specify the labels
dance_class['binned_age'] = pd.cut(dance_class['Age'], bins, labels = age_labels)
```

Now when we print the first few rows of our data or plot a bar graph of the data, each numeric range that we specified earlier is replaced with the assigned label:

```
   binned_age  Age
0  Young Adult   23
1  Young Adult   28
2  Middle Aged   45
3      Senior   63
4      Adult    35
```



Output:

Multiple choice

Questions

Which of the following is the correct way to bin the value of the `Grades` column in dataframe `student_data` into five equal bins?

Answer Choices

```
pd.bin(student_data, 5)
```

```
pd.cut(student_data['Grades'], 5)
```

```
pd.bin(student_data['Grades'], 5)
```

```
pd.cut(student_data, 5)
```

Combining Categorical Data

As a data analyst or scientist, you will sometimes work with categorical data. Just as a refresher, categorical data are variables that contain label values rather than numeric values. For example,

A “color” variable with the values: “blue”, “red” and “yellow”.

An “education” variable with the values: “high school”, “college”, and “no education”.

When dealing with categorical data, you may sometimes want to combine/remove categories for easier analysis. For example, imagine you’re looking at a data set of the most popular sports in a region:

Sport	Count
Basketball	500
Football	400
Baseball	8
Tennis	7
Cricket	4
Sailing	3

Notice that there are several categories of sports here but a pretty uneven distribution of their occurrences. Basketball and football make up an overwhelming majority of the data, while the other

categories only have a few occurrences. Therefore, it may make sense to combine all of the other sports into a category called “Other.”

Sport	Count
Basketball	500
Football	400
Other	22

Combining Categories in Python

Let’s explore how to combine categories of categorical data in Python. We’ll be working with a data set called `election_poll.csv` to see the most popular candidates for a city council election. This data set has a column called `votes` that we will use.

First, let’s import our data and see the counts for each of the candidates (categories) in our data:

```
import pandas as pd

# read in data
election_data = pd.read_csv('election_data.csv')

# get the counts for each candidate
votes = election_data['Vote'].value_counts()
print(votes)
```

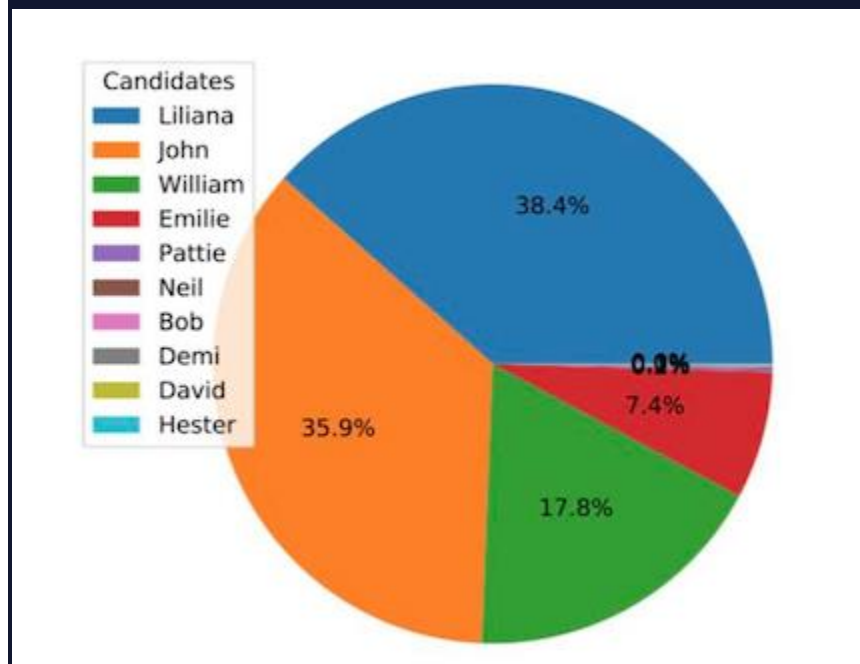
The output looks like:

```
Liliana    1067
John       998
William    494
Emilie     196
Pattie      6
Neil        3
Bob         2
Demi        1
```

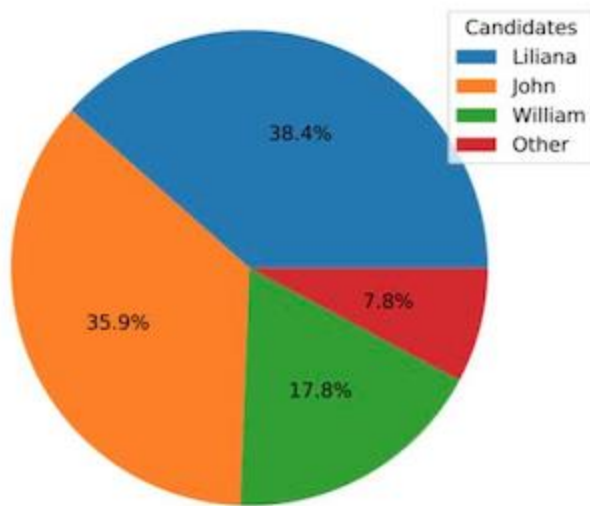
```
David      1
Hester     1
```

What do you notice about the data? Are there specific candidates we should focus more on and others who we should focus less on?

You may notice that Liliana, John, and William make up most of the data while the other candidates make up a smaller percentage of the overall vote count. If we're trying to understand the likelihood that each of these candidates is elected in an upcoming vote, it makes sense to collapse the other candidates into a category called "Other." If we decided not to do this, then our analysis might be more difficult to parse. For example, if we were to plot this data in a pie chart, it would look like this:



That isn't a pleasant pie chart to look at. Liliana, John, and William make up 92% of the data, while the other seven categories make up just 8% of the data. Once we combine those seven categories in an "Other" category, the pie chart looks like:



How can we do this in Python? We can create a mask for values occurring less than a specific number of times in `votes`. In our case, we'll check for values occurring less than 200 times. We do this using the `isin()` function:

```
mask = election_data.isin(votes[votes < 200].index)
```

Next, we can label these other categories as "Other" and print the updated vote count:

```
election_data[mask] = 'Other'  
print(election_data['Vote'].value_counts())
```

Output:

```
Liliana    1067  
John       998  
William    494  
Other      210  
Name: Vote, dtype: int64
```

Notice how much cleaner the data is now that we have combined the seven low frequency categories into one "Other" category.

Remember that the cleaner your data is, the simpler it will be to analyze and visualize, so taking the time to understand your data and apply simple transformations like this when appropriate will go a long way.

Multiple choice

Questions

Which of the following is not a benefit of combining categories of categorical data?

Answer Choices

x - It makes your data harder to read

It can simplify the data analysis process

It helps with data visualization

It improves data readability

Data Centering and Scaling

An introduction to data centering, min-max normalization, and standardization

As a data scientist, you will often need to transform your data before visualizing, analyzing, or modeling it. In this article, you will learn how to center and scale your data using common techniques. You will learn:

How to center data and interpret centered data

Why scaling your data is important

How to scale data using two common methods:

- **Min-max Normalization**

- **Standardization**

When to choose normalization vs. standardization

Let's get started!

Data Centering

Data centering involves subtracting the mean of a data set from each data point so that the new mean is 0. Mathematically, this looks like:

$$X_{centered_i} = X_i - \mu$$

$$X_{centered}$$

$$i$$

$$= X$$

$$i$$

$$-\mu$$

where x_i is a datapoint and the Greek letter μ is the mean of all the x values.

For example, let's take a look at a data set of ages for five individuals:

```
ages = [24, 40, 28, 22, 56]
```

The mean age in this data set is 34 years old.

To center our data, we subtract the mean from each data point in `ages`:

```
centered_ages = [-10, 6, -6, -12, 22]
```

This centered data is useful because it tells us how far above or below the mean each data point is, giving us additional insight that we can't get just by looking at the initial data set. For example, the age of the first individual is ten years below the average.

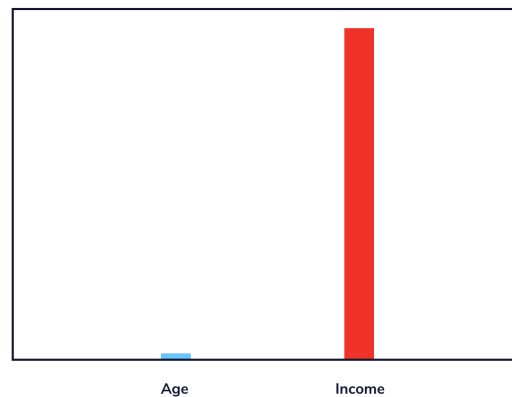
Note that, because the sum of the centered values is 0 ($-10 + 6 - 6 - 12 + 22 = 0$), the mean of the centered data is 0.

Data Scaling

A common task for data analysts and scientists is to find trends in data by comparing features of data points. However, this task is made difficult when the features are on drastically different scales.

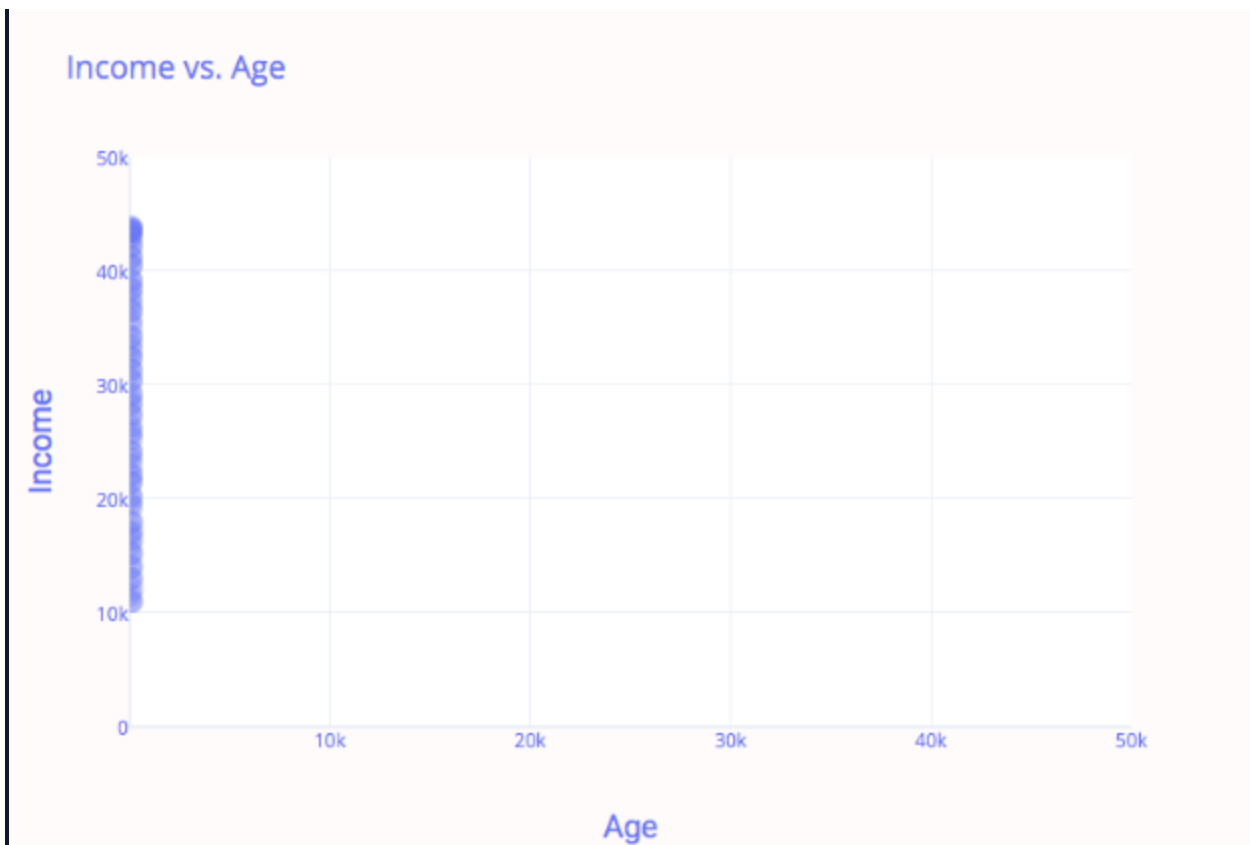
For instance, let's consider a data set containing two features, **age** and **income**.

In general, a person's age usually ranges from 0 to about 100 years. A person's income, on the other hand, usually ranges from 0 to large amounts measured in the thousands of dollars. Clearly, age and income are two features that have vastly different ranges.



This presents issues when trying to use many machine learning algorithms, which treat all dimensions equally regardless of their scale. The difference in one year of age is interpreted as exactly equal to the difference in one dollar of income. That makes no sense!

In other words, the income feature outweighs the importance of age because income is on a relatively huge scale. Take a look at the following image:



From this chart, it is impossible to notice any relationship between income and age – all of the data is squished to the left. This is because the feature with the larger scale (income) dominates the smaller feature (age).

We would like every datapoint to have the same scale so each feature contributes equally to the relationship. Data scaling lets us achieve this.

Two of the most commonly used data scaling techniques are:

Min-Max Normalization

Min-max normalization is one of the most simple and common ways to scale data.

For every feature in a data set, the minimum value of that feature is transformed into 0, the maximum value is transformed into 1, and every other value is transformed into a decimal between 0 and 1.

For example, if the minimum value of a feature is 10, and the maximum value is 30, then 20 would be transformed to 0.5 since it is halfway between 10 and 30.

The formula for *min-max normalization* is as follows:

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

X

$norm$

$=$

X

max

$-X$

min

$X - X$

min

Using min-max normalization, our previous chart of income vs. age looks like the following:



Notice that the data is now properly scaled – both age and income fall in the range of [0,1].

Let's get some practice implementing min-max normalization in Python:

Coding question

Questions

Complete the implementation of min-max normalization in the function `min_max_normalize()` using the formula given above.

Code

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14

def min_max_normalize(lst):
    minimum = min(lst)
    maximum = max(lst)
    normalized = []
    # code goes here
    return normalized
# Uncomment these function calls to test
your function:
#print(min_max_normalize([0, 25, 50, 75,
100]))
# should print [0.0, 0.25, 0.5, 0.75, 1.0]
#print(min_max_normalize([10, 12, 14]))
# should print [0.0, 0.5, 1.0]
```

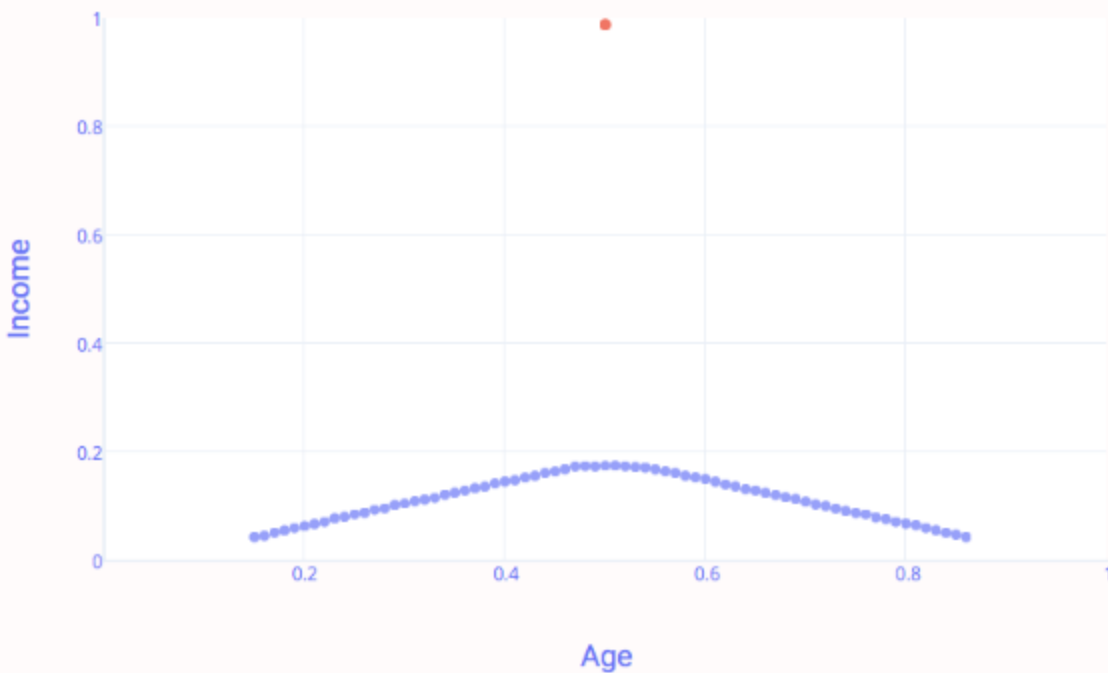
Run

Check answer

Run your code to check your answer

One downside of min-max normalization is that it does not handle outliers very well. For example, if you have 99 values between 0 and 20, and one value is 100, then the 99 values will all be transformed to a value between 0 and 0.2 while the outlier is transformed to 1. This results in skewed data:

Income vs. Age (normalized)



In this example, normalizing the data fixed the skewing problem on the x-axis, but now the y-axis is causing problems.

As we'll see shortly, standardization is a more robust data scaling method for dealing with outliers.

Standardization

Standardization (also known as *Z-score normalization*) is another common data scaling technique.

Standardization involves subtracting the mean of each observation and then dividing by the standard deviation:

$$z = \frac{\text{value} - \text{mean}}{\text{stdev}}$$

$$z =$$

$$\frac{\text{value} - \text{mean}}{\text{stdev}}$$

$$\text{value} - \text{mean}$$

Once standardization is complete, all the features will have a mean of zero, a standard deviation of one, and therefore, the same scale.

Unlike normalization, standardization does not have a bounding range. This means that even if you have outliers in your data, your standardized data will not be affected. Therefore, if your dataset has outliers, standardization is the preferred scaling technique.

Let's see if you can use the formula above to implement standardization in Python:

Coding question

Questions

Complete the implementation of standardization in the function `standardize()` using the formula given above.

Code

```
def standardize(lst, mean, std_dev):  
    standardized = []  
    # code goes here  
    return standardized  
# Uncomment these function calls to test  
your standardize function:  
# print(standardize([1, 2, 3, 4, 5], 3.0,  
1.41))  
# should print [-1.418, -0.709, 0.0, 0.  
709, 1.418]  
# print(standardize([10, 15, 20], 15.0, 4.  
08))  
# should print [-1.225, 0.0, 1.225]
```

Run

Check answer

Run your code to check your answer

When to Normalize vs. Standardize?

Min-max normalization and standardization both have a similar goal of transforming features in data to have the same scale so that each feature is equally important. So when should you use min-max normalization vs. standardization?

There is not always a clear answer. Both normalization and standardization have their strengths as well as their drawbacks. For example, if you need your data to be on a 0-1 scale, then it makes sense to use min-max normalization. If you have outliers in your data, then it is best to use standardization (*Z-score normalization*) since it does not have a bounding range like min-max normalization does.

Keep in mind that not every data set requires normalization or standardization. If your data features do not have vastly different ranges, then scaling your data might not be necessary.

Python Implementation

As you saw, it is possible to implement min-max normalization and standardization by writing your own Python functions. However, in practice, most data analysts and scientists use popular libraries such as `scikit-learn`, which makes it very easy to scale your data.

For example, to normalize your data you can import `MinMaxScaler` from the `sklearn.preprocessing` package and then make a simple function call:

```
from sklearn.preprocessing import MinMaxScaler
import pandas as pd

# read in data
data = pd.read_csv('data.csv')

# normalize data
scaler = MinMaxScaler()
normalized_data = scaler.fit_transform(data)
```

Likewise, standardizing your data is easy to do in just a few lines of code:

```
from sklearn.preprocessing import StandardScaler
import pandas as pd

# read in data
data = pd.read_csv('data.csv')

# standardize data
scaler = StandardScaler()
standardized_data = scaler.fit_transform(data)
```

Later on, you'll get practice with transforming your data in Python using real-world data. For now, let's test your knowledge of min-max normalization and standardization with a few questions:

Multiple choice

Questions

Which of the following is **not true** about *min-max normalization*?

Answer Choices

Min-max normalization transforms features so that the maximum value is 1.

Min-max normalization does not handle outliers well.

Min-max normalization handles outliers well.

After min-normalization, every feature has the same scale

Multiple choice

Questions

Which of the following is correct about *standardization*?

Answer Choices

Standardization does not handle outliers well.

Standardization is the same thing as normalization.

Standardization involves subtracting the standard deviation and then dividing by the mean.

Standardization involves subtracting the mean of each observation and then dividing by the standard deviation.

Multiple choice

Questions

Which of the following is correct about min-max normalization?

Answer Choices

Min-max normalization is also called Z-score normalization.

For every feature in a data set, the minimum value of that feature gets transformed into a 0, the maximum value gets transformed into a 1, and every other value gets transformed into a decimal between 0 and 1.

Min-max normalization is more widely used than standardization.

For every feature in a data set, the minimum value of that feature gets transformed into a -1, the maximum value gets transformed into a 1, and every other value gets transformed into a decimal between -1 and 1.

Review

As you learned in this article, normalization and standardization are scaling techniques that are slightly different but have a similar motive: to put data features in the same scale so that no feature is dominated by the other. These techniques are widely used by data analysts and scientists.

In this article, you learned:

Data centering:

Min-max normalization:

The goal of normalization is to put features with different ranges onto the same scale.

For every feature in a data set, the minimum value of that feature gets transformed into a 0, the maximum value gets transformed into a 1, and every other value gets transformed into a decimal between 0 and 1.

A downside of normalization is that it does not handle outliers well

Formula:

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

X

$norm$

$=$

X

max

$-X$

min

$X - X$

min

Standardization:

Standardization, also known as *Z-score normalization*, involves subtracting the mean of each observation and then dividing by the standard deviation:

$$z = \frac{value - mean}{stdev}$$

$z =$

$stdev$

$value - mean$

Once standardization is complete, all standardized features will have a mean of zero, a standard deviation of one, and therefore, the same scale. Unlike normalization, standardization does not have a bounding range. This means standardization can deal with outliers.

As a data analyst or scientist, it's important to understand the data you're working with and how to transform that data. You now have a better grasp of common data transformation techniques and why it's important to transform your data in the first place.

Advanced Data Transformations

An introduction to log transformation and other advanced data transformations

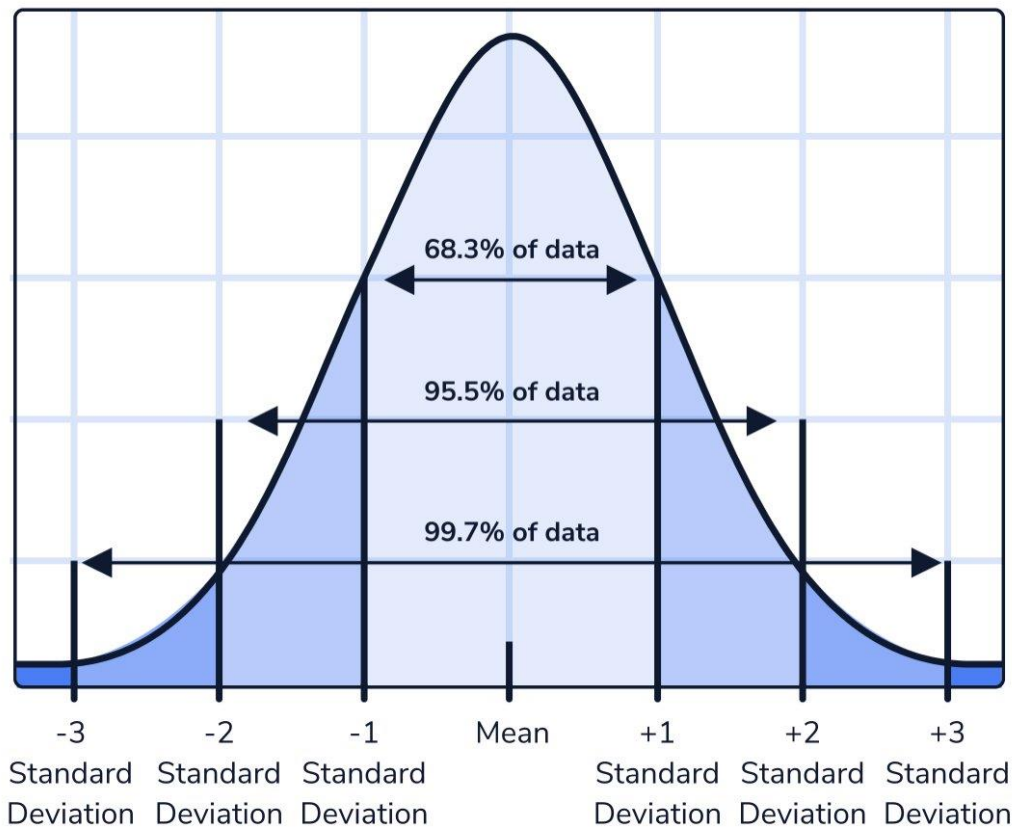
In this article, you will learn how to transform data in particular situations, such as when the data you're working with is skewed. You will learn:

- How to transform data that is skewed
- What is log transformation and why is it useful?
- How to implement log transformation in Python
- Different kinds of advanced data transformations

Let's get started!

Skewed Data

As a data analyst and scientist, you will often work with normally distributed data. Just as a refresher, normally distributed data is data where most data points are close to a central value with fewer instances as you get further away from the center. Visually, this looks like a "bell curve":



There are several reasons why having normally distributed data is useful:

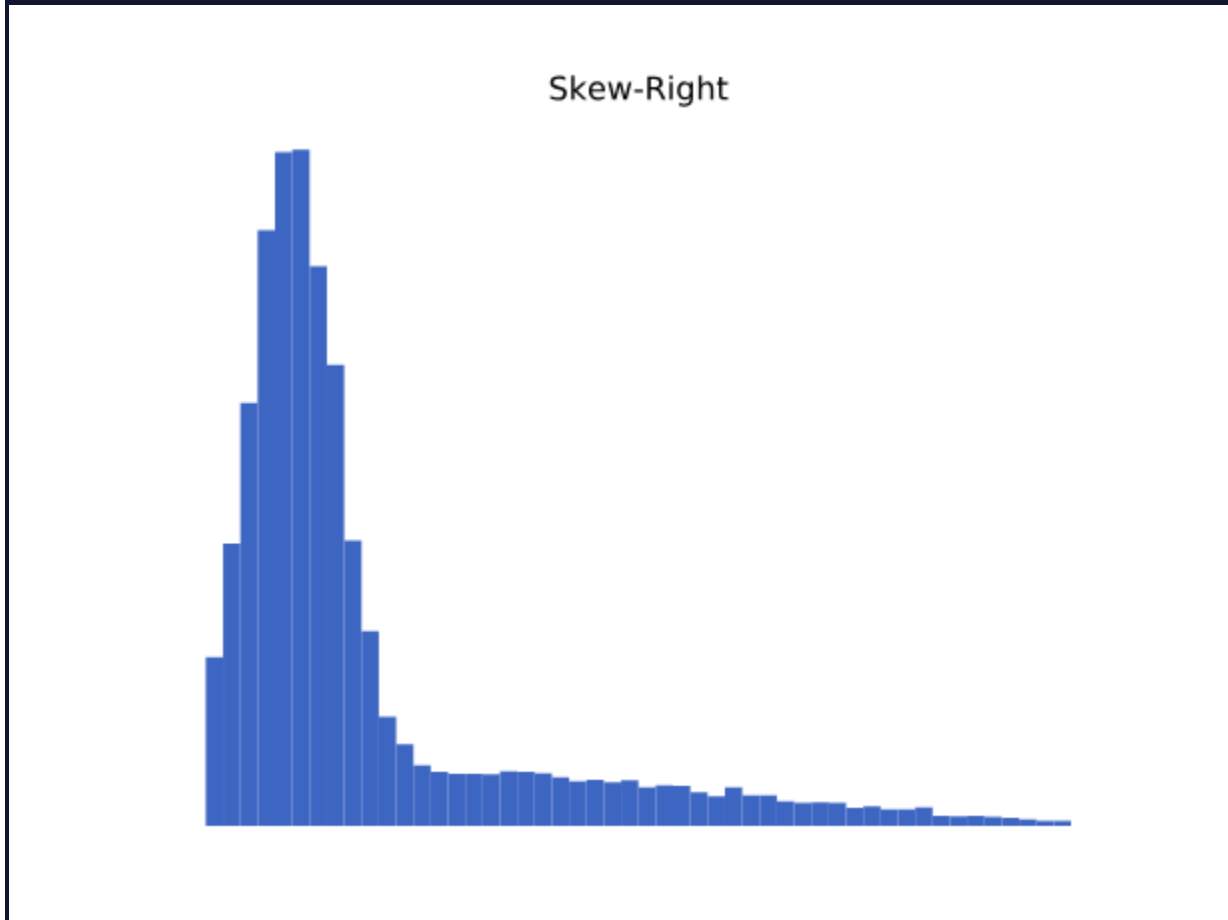
Normal distributions are symmetric around their mean. Moreover, the mean, median, and mode of a normal distribution are equal. These properties make the data easier to analyze because we can make some assumptions about how many datapoints are on either side of the mean.

As shown above, approximately 68% of the data falls within 1 standard deviation of the mean, and approximately 95% of the data falls within 2 standard deviations of the mean. Again, these properties make it easier to make generalizations about the data you're working with.

Many machine learning algorithms (such as linear regression) assume that the data distribution is close to normal. Therefore, transforming data so that it is normally distributed can enhance your ability to fit an accurate predictive model.

Oftentimes, real-life data is messy and does not conform to a normal distribution. Instead, the data might be skewed to the left or right.

For example, imagine you're a data analyst looking at income data in a population. You may find that a lot of the data is centered around lower values but that there are high values in the data as well. This results in a skewed distribution:



This is an example of a *right-skewed* data set. Lower incomes, which make up most of the data, fall on the left side of the diagram, while higher incomes fall on the right.

Feeding this data to some machine learning or statistical models will present issues if the model assumes normality, because the model will be trained on a much larger number of lower incomes. The skewness of the data violates the assumptions of the model.

In situations like this, we would like to make our data less skewed so that it conforms to model assumptions and is easier to work with. To help with this issue, we can use *Log Transformation*.

Log Transformation

Log transformation is a data transformation method that replaces each variable x with $\log(x)$.

When log transformation is applied to data that is not normally distributed, the result is that the data will be less skewed, or more “normal” than before. For example, after applying log transformation to the right-skewed data set above, we would see something similar to the following:



Notice how the shape of the data changed. The logarithm function squeezes together the larger values in your data set and stretches out the smaller values, making the transformed data more “normal” than before.

Let’s see how we can apply this on real data in Python.

Python Implementation

Python contains libraries that make it simple to transform your data using log transformation.

One quick method is to use `NumPy`, a Python library that makes it easy to work with arrays of data and perform statistical analysis. Numpy has a natural log transformation function that you can apply to an array of data, like so:

```
import numpy as np

data = [1, 1, 1, 1, 2, 2, 2, 3, 3, 4, 4]

# log transform
log_data = np.log(data)

print(log_data)

# 0, 0, 0, 0, 0.693, 0.693, 0.693, 1.098, 1.098, 1.386, 1.386
```

Data scientists and machine learning engineers will generally usually use powerful machine learning libraries such as `scikit-learn` to apply log transformation. You can import `PowerTransformer` from sklearn's `preprocessing` module and then pass in your data to a simple function call:

```
from sklearn.preprocessing import PowerTransformer

# log transform
log_transform = PowerTransformer()
log_transform.fit_transform(data)
```

Let's try to perform log transformation on real data in Python. We'll be working with home price data from Kaggle.

First, we will need to import several Python libraries:

`numpy` (to perform log transformation)

`pandas` (to read in a CSV file and perform data analysis)

seaborn (to plot our data)

```
import numpy as np
import pandas as pd
import seaborn as sns
```

Next, we need to read in our data file and store the home price data in a new variable:

```
import numpy as np
import pandas as pd
import seaborn as sns

# read in data file
home_data = pd.read_csv('home_data.csv')

# store home price data
home_prices = home_data['SalePrice']
```

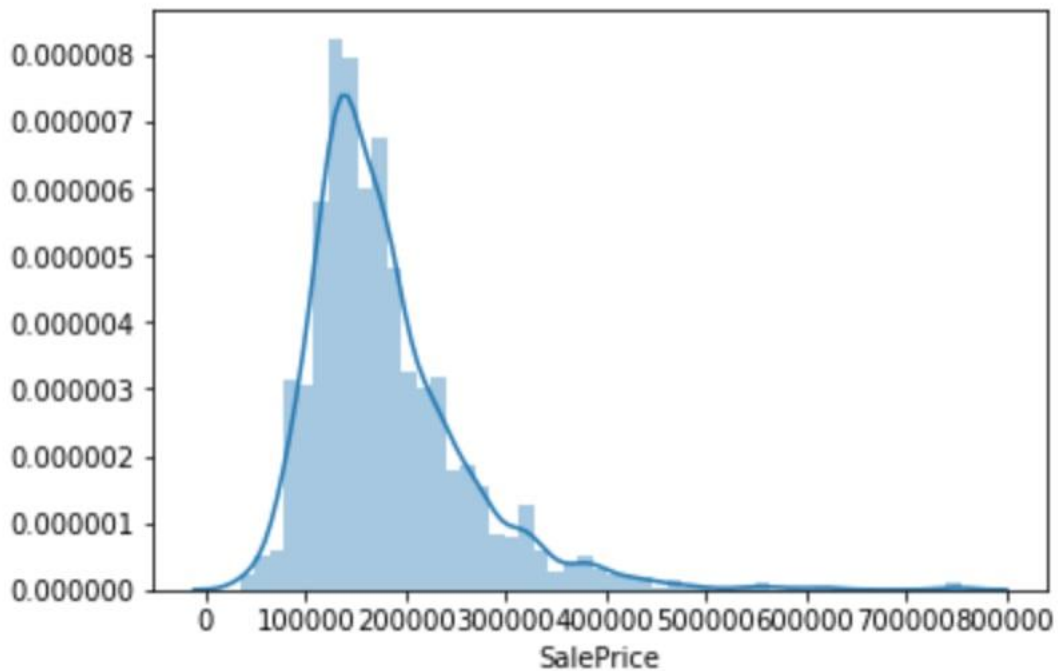
Now that we have our data stored, we can plot it using `seaborn`:

```
import numpy as np
import pandas as pd
import seaborn as sns

home_data = pd.read_csv('home_data.csv')
home_prices = home_data['SalePrice']

# plot data
sns.distplot(home_prices)
```

Distribution plot:



The first thing we notice is that our data is right skewed. Most of our data consists of lower and medium home prices, while higher home prices make up a small part of the data. Let's see exactly how skewed our data is:

```
print(home_prices.skew())
```

Output:

```
1.88
```

Generally, if skewness is less than -1 or greater than 1, the distribution is highly skewed. Our distribution has a skewness of 1.88, so it is highly skewed.

If we were to use this skewed data to predict home prices, we would run into some issues. The prediction model would be trained on a much larger number of lower and moderately priced homes so it would have trouble predicting the prices of expensive homes.

To help with this, we can apply log transformation to our data:

```
import numpy as np
import pandas as pd
import seaborn as sns

home_data = pd.read_csv('home_data.csv')
home_prices = home_data['SalePrice']

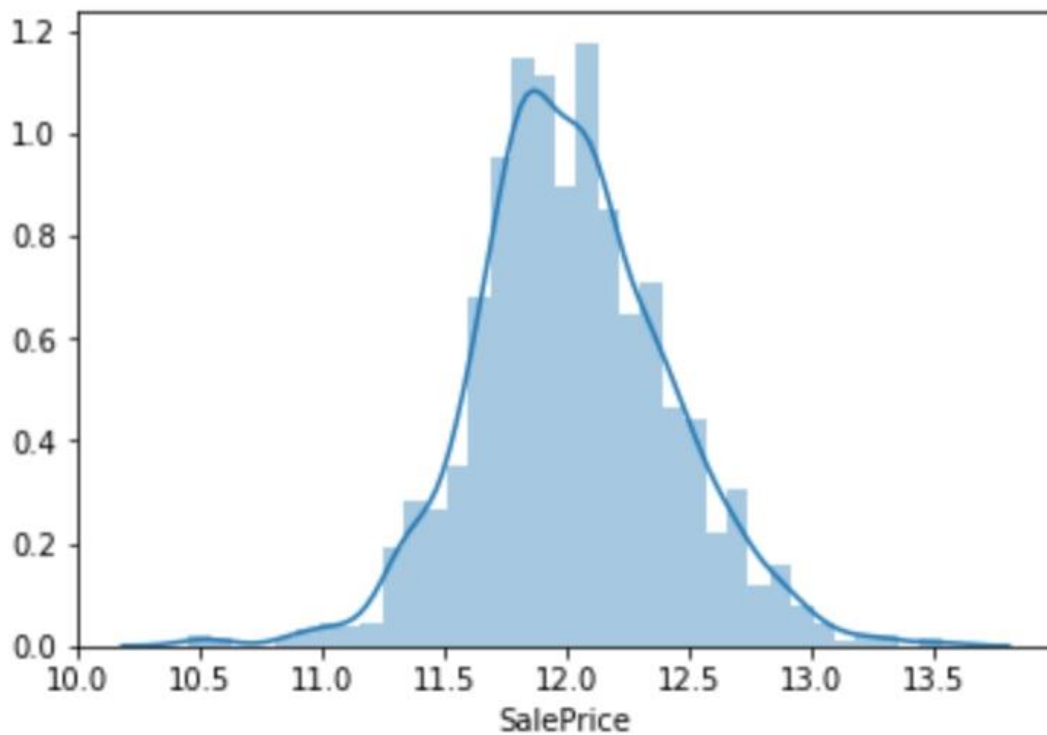
sns.distplot(home_prices)

# apply log transformation
log_home_prices = np.log(home_prices)
```

Next, let's plot the transformed data to see the shape of the distribution:

```
sns.distplot(log_home_prices)
```

Distribution plot:



Notice how the shape of the distribution is much more symmetrical (normal) than before.

Let's see how skewed our data is now:

```
print(home_prices.skew())
```

Output:

```
0.12133506220520406
```

Applying log transformation reduced the skewness from 1.88 to 0.12 — that's a significant change!

Now that our transformed data is less skewed and more normally distributed, we can use this data in our prediction model to more accurately predict home prices.

Other Transformations

In this article, we discussed only one advanced data transformation method – log transformation. But different situations call for other types of transformations.

In the examples above, we dealt with income and price data, which cannot be negative. But what happens if you have negative numbers in your data? You cannot take the log of a negative number, so log transformation will not work.

In this case, you would have to explore other data transformation techniques, such as *cube root transformation*, which involves converting x to $x^{(1/3)}$. This transformation reduces the right skewness but also has the benefit of working with zero and negative values.

To reduce left skewness, you might look at a technique such as *square transformation*, which involves converting x to x^2 .

An important part of your job as a data analyst and scientist is understanding the data you're working with and the shape of that data. Once you know this, it becomes easier to figure out how you should transform that data.

As you progress in your journey, you'll learn many techniques to deal with unique situations. For now, feel free to explore other data transformation methods [here](#).

Multiple choice

Questions

Log transformation:

Answer Choices

Is the most effective data transformation method

Works with any type of data

Reduces the skewness of your data

Removes outliers from your data

Fill in the blank

Questions

Fill in the code to apply log transformation on `data`:

Code

```
import
blank 1
as
blank 2

data = [10, 20, 30, 40, 50]
log_data = np.
blank 3
(
blank 4
)
```

Answer Choices

numpy

data

log

np

Click or drag and drop to fill in the blank

Check answer

Multiple choice

Questions

Which of the following is a limitation of log transformation?

Answer Choices

There are no limitations

It cannot deal with outliers

It makes your data more skewed

It does not work with zero or negative values

Introduction to Visualization for Data Science Applications

Visualize your future as a data visualization aficionado!

You can only see what you can visualize

-A famous Data Scientist (or we made it up)

This unit will give you a little extra practice with data visualization for data science.

What will Visualization for Data Science Applications cover?

After this unit, you will be able to:

Apply your matplotlib data visualization skills to new problems

Why did we build this?

Data visualization is at the core of what Data Scientists who focus on Analytics (and Data Analysts) do. These projects will give you an idea of what is possible, and hopefully inspire you to create your own portfolio projects with the skills you develop here.

Who are my classmates?

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

Welcome to Learn Tableau for Data Visualization

What will you learn?

In this course, you will learn the basics of importing, managing, and visualizing data in Tableau.

After this course, you will be able to:

- import and join datasets
- choose the best type of visualization for your data
- use Tableau's most important features and tools
- customize your dashboard to tell a data story
- contextualize your visualizations for your audience.

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for peer reviews on your project work and providing reviews to others in the [projects category](#), which can help to reinforce what you've learned.

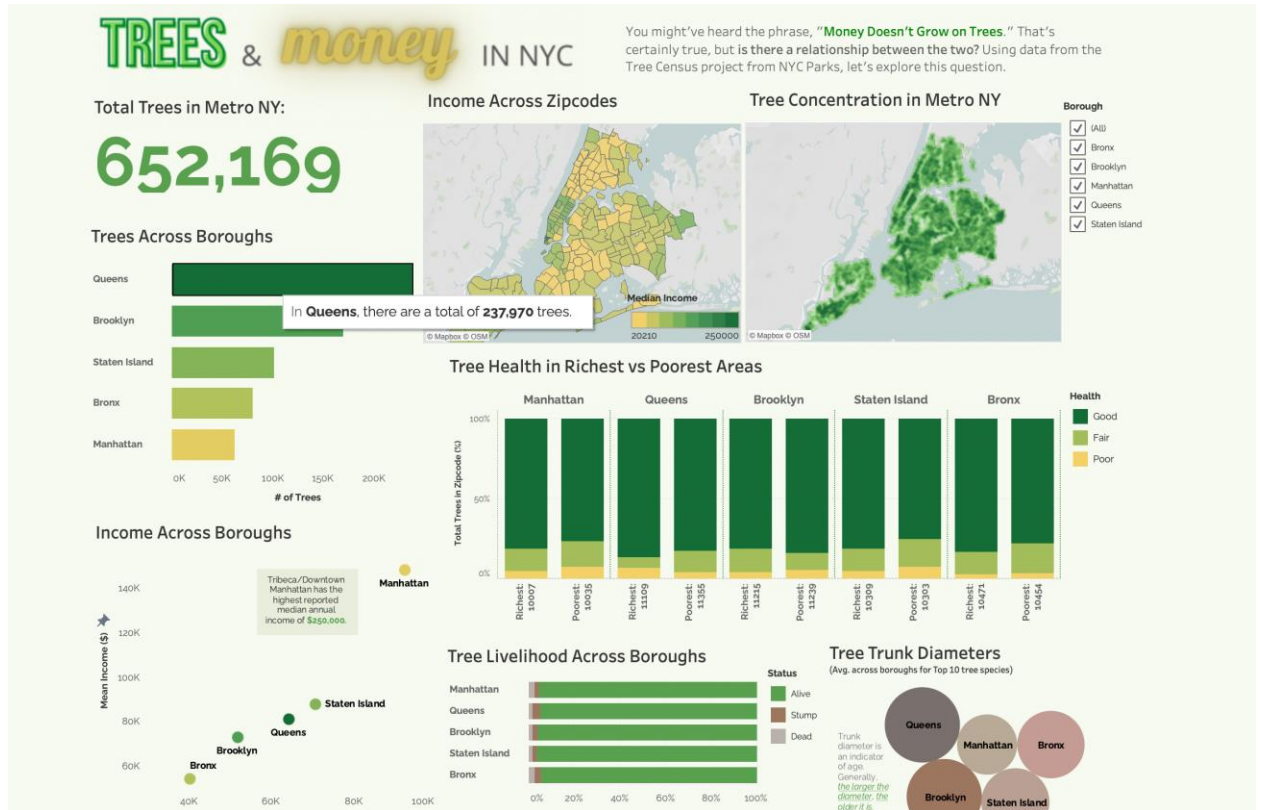
What should you know before starting this course?

This course does not require any prerequisite knowledge in either Tableau or data visualization!

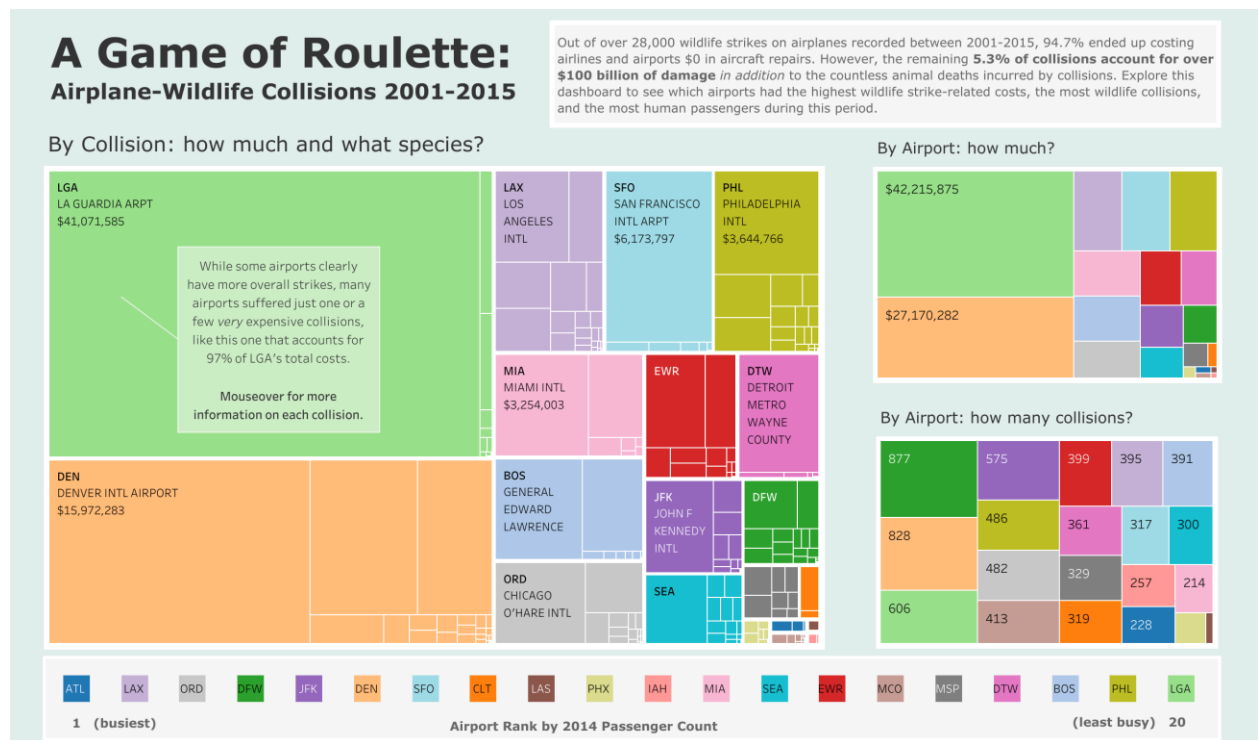
What will you do?

Throughout this course, you will be applying the concepts you learn in real data visualization projects:

[explore the NYC Tree census](#)



[analyze airplane wildlife strikes](#)



We're excited for you to start your journey as a data visualization specialist in Tableau!

Why Do People Use Tableau?

Welcome to this course on Tableau! Let's start with a couple of basic questions: what is Tableau and why do people like it?

Tableau is an analytics platform. It is often referred to as the visualization layer of an analytics ecosystem. This makes it the place users go to see and create presentable data charts, dashboards, and stories. Any heavy modification or preparation of the data is generally done before it is viewed in Tableau.

Tableau's core strength is making it easy for us to visualize information. It allows us to view and interact with data at a high-level *and* to drill into the same data at a granular level. It's like viewing a map at the city or state level and then zooming in all the way to view the street names and addresses when necessary. Tableau does this so well that it gives us many opportunities to answer questions in real-time without having to go somewhere else for the data or request additional reporting.

"Tableau" doesn't mean just one program, though. The Tableau platform includes free and paid products for data connection and preparation, data exploration, and ways to share key findings. In this course, we're using Tableau's powerful free offerings: Tableau Public for making visualizations, and its corresponding website (public.tableau.com) for sharing them.

The basic differences between Tableau Public and its paid counterpart Tableau Desktop amount to:

What kind of files we can connect to
Where we can save our work

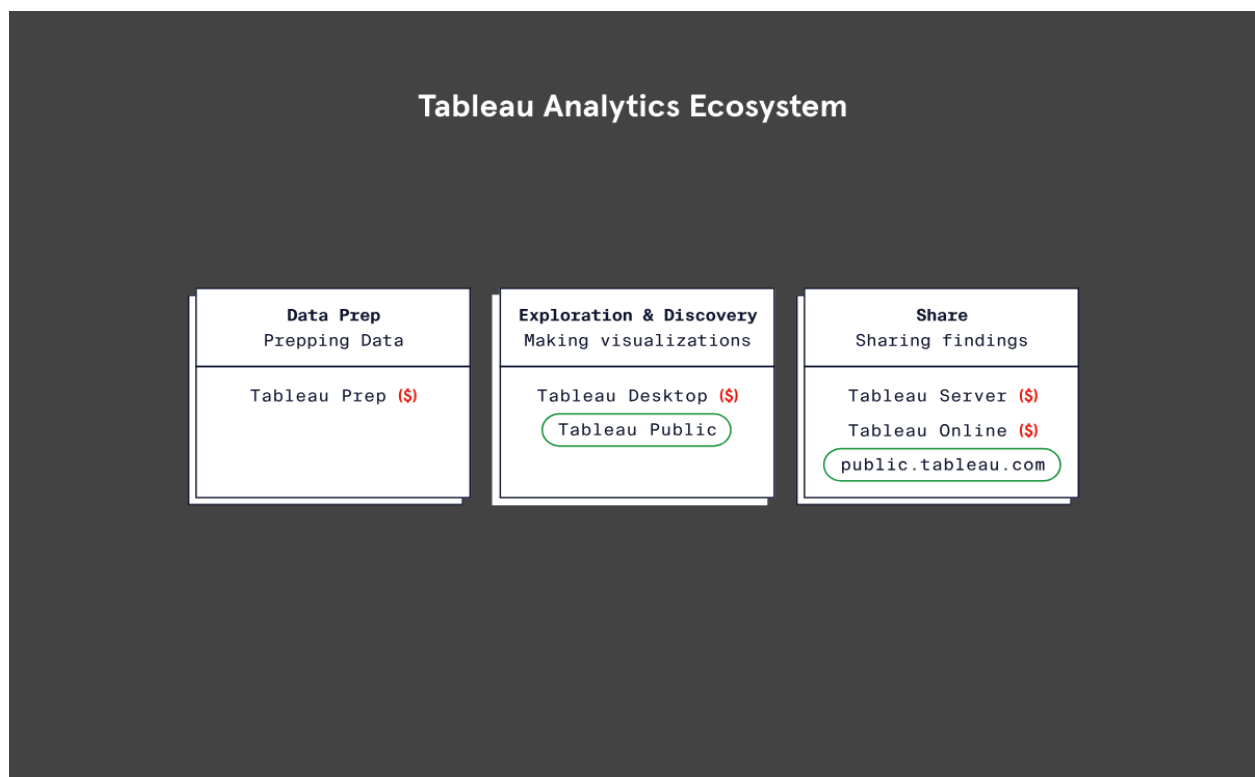
Tableau Public can connect to local flat file types such as .xlsx or .csv files, or to Google Sheets. Tableau Desktop can connect to over 70 data sources.

When it comes to saving work, Tableau Desktop can save work on a local computer, Tableau

Preview: Docs Loading link description

[Server](#)

, Tableau Online, or Tableau Public. Tableau Public saves work to a Tableau Public profile, which we'll set up in the next exercise.



Setting Up a Tableau Public Account

In order to use Tableau Public, we'll need a Tableau Public account. The account will provide us with an online portfolio to showcase all of our presentation-ready work, and it can also store hidden in-progress work until it's ready to be shared publicly.

For someone making a career change into data visualization with Tableau, Tableau Public is one of the best places to showcase dashboards and visualizations. If that applies to you, we suggest personalizing Tableau Public settings such as "Bio" and "Available For Hire" to make the most of your Tableau Public portfolio.

While it is possible to create visualizations in the browser by clicking "Create A Viz", we also recommend using the desktop version of Tableau Public. (This is not the same as Tableau Desktop, which is Tableau's paid version of the software.) The desktop version of Tableau Public provides a more user-friendly experience for dashboard building, as well as the ability to work offline.

With that, let's set up an account and download the desktop version!

Instructions

Navigate to [Tableau Public](#) and click the "Sign Up Tableau Public" button. This will prompt you for some details including your name and email as a requirement to create your public profile.

After signing up, go to the email that you received from Tableau Software Website Support, and click on the link to activate your account. You will then be sent to the Tableau Public home page. Click the "Sign In" button at the top right of the page and sign in (if you don't see a "Sign In" button, click on the toggle menu with the three small lines). After signing in, you will then be asked to set your location and agree to the terms and data policy.

Next, you will see a gray profile icon at the top right of the page. Click on this and a

drop-down menu will appear. Click on "My Profile".

Take a look at your profile settings. You can change your "Bio" and "Available for Hire" options found at the bottom of your Edit Profile page. (Once you have some work to showcase, the "Available for Hire" option will add this button to your profile so potential employers can contact you.)

Under the “Vizzes” tab in your profile you should see the following:

codecademy

Edit Profile

Vizzes ◦ Favorites ◦ Following ◦ Followers ◦



Get Started

Connect to your data to create a visualization and save it to your profile. Need some data?
Explore our sample data sets for inspiration.

Create a Viz

Explore Sample Data

Want to use Tableau Public offline?
[Download the free Desktop app](#)

Select the linked words at the bottom that read, “Download the free Desktop app”.
Then on the next page, click “Download Tableau Public”.

2. Note: if it logged you out it may prompt you to either sign up or sign in before you can download. Sign in and click the “Download Tableau Public” button again. If the file still doesn’t download, there should be a green pop-up message that says to click “Mac” or “Windows” (depending on your system) to download Tableau Public.

Once you have downloaded the software, install it on your machine by following the prompts.

Now that we’ve downloaded and installed Tableau Public, let’s move on to the next exercise to open it and see what we can do!








Lindsay Betzendahl

Senior Consultant, Design Lead at HealthDataViz | Pennsylvania, United States

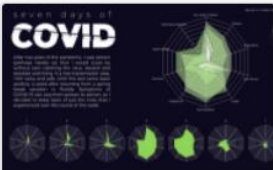
I am a 2x Tableau [#Visionary](#) (aka [#ZenMaster](#)) and 3x [#PublicAmbassador](#), working as a consultant at [#HealthDataViz](#). I am passionate about [#healthcare](#) and data visualization. I founded and run a community...
[Read more](#)

Following

Vizzes 187 Favorites 1,321 Following 476 Followers 3,687



The Great Modern Art Heist - #IronViz
Lindsay Betzendahl
☆ 22 👁 603



Seven Days of COVID
Lindsay Betzendahl
☆ 7 👁 149



Generating Vizzes - My Tableau Public Data
Lindsay Betzendahl
☆ 42 👁 1,657



Simple Yet Effective KPIs - #B2VB
Lindsay Betzendahl
☆ 39 👁 823



US Nursing Salaries 1998-2020 - #ProjectHealthViz
Lindsay Betzendahl
☆ 18 👁 470



The Path to Awakening - One Year of Sleep Data
Lindsay Betzendahl
☆ 43 👁 1,248



Annual Calendar Sunburst Template
Lindsay Betzendahl
☆ 13 👁 255



#ProjectHealthViz Submission Tracker
Lindsay Betzendahl
☆ 32 👁 1,249

Loading Data

When we open the Desktop version of Tableau Public, we'll see a window that allows us to connect to our first data source. As discussed, we can connect to local flat files and Google Sheets with Tableau Public. The two most common files we'll connect to are Microsoft Excel and .csv files.

In Tableau, *Microsoft Excel* refers to any file with an .xls or .xlsx extension. *Text file* refers to a .csv or .txt file. When a Microsoft Excel or Text file is selected, Tableau will only show the files relevant to that selection. Data files loaded into Tableau are also called tables or data sources.

Once we have the data loaded, Tableau will make some helpful changes to the original file. One of these changes is to make the field names more readable, for example changing `customer_name` to `Customer Name`. Another thing Tableau will do before we even see the data loaded is assign a data type to each field. This allows us to jump right into the analysis as soon as the data is loaded. In this exercise, we'll see how to change these names and types ourselves.

Let's load some data!

Instructions

First, you will need to download the zip file that contains all the data for this course. Go to the [Github repository](#) and click on the green button that says “Code”. When the drop-down window

appears, select “Download Zip” at the bottom.

Once the zip file is finished downloading, open the zip file and save the `datasets` folder somewhere outside of the zip file. Tableau cannot access a zip file directly.

2. We'll first work with our condensed version of the Tree Census Data from NYC (the original file can be downloaded [here](#) if you'd prefer the entire dataset).

Open Tableau Public and select the “Text file” option in the left pane. This will allow you to navigate to and open the `tree-census-NYC_2015.csv` file. The next screen we'll see is the data viewer. Look at the image to the right of this pane to understand the layout of the data viewer. From this screen in Tableau, we can see a preview of our csv, its metadata (field names), any other data sources we've connected to, and can even join data tables with one another.

For now, let's change a field name to make it clearer for ourselves and other viewers in the future. Take a look at the Data Dictionary by either opening the `TreeCensus-2015_data-dictionary.PDF` file in the `datasets` folder or [clicking here](#). For this dataset, we know that `tree_dbh` means a tree's “trunk diameter at breast height,” about 54in or 137cm above the ground. Just by opening this file in Tableau, the name has already been improved to `Tree Dbh`. Let's make it more immediately readable. Double-click the `Tree Dbh` field name and change it to `Trunk Diameter` and press Enter. Now let's change a field type. In the Metadata pane (the sections are labeled in the image to the right), we can see a list of all fields in the data source. Tableau is treating the `Tree Id` field as numerical data when it should be categorical data. Find the row that has `Tree Id` as the “Field Name” and click on `#` under “Type”, and then select `String` type. Once changed, you should see “Abc” under the “Type” (instead of `#`) We'll explore data types in more detail soon so don't worry if you have questions at this point.

The screenshot shows the Tableau Public interface with the following sections:

- Data Sources:** A list of connected data sources on the left sidebar, including `tree-census-NYC_2015` (Text file).
- Relationships Panel:** A central area showing the `tree-census-NYC_2015.csv` file and a prompt to "Need more data? Drag tables here to relate them. [Learn more](#)".
- Data Metadata:** A table showing the fields in the data source, including `Tree Id` and `Tree Dbh`.
- Data Preview:** A table showing a preview of the data, including columns for `Tree Id` and `Tree Dbh`.

Type	Field Name	Physical Table	Rem...
#	F1	tree-census-NYC_2015...	F1
#	Tree Id	tree-census-NYC_2015...	tree_id
#	Tree Dbh	tree-census-NYC_2015...	tree d...

tree-census-NYC_2015.csv	tree-census-NYC_2015.csv	tree-census-NYC_2015.csv
F1	Tree Id	Tree Dbh
65,536	240049	15
	388	29
	3633	1
65,539	230589	5
65,540	265424	29
65,541	265441	27

Joins, Unions, and Relationships

Okay, we've covered how to load a data file into Tableau. But what if a single data file doesn't have all the

Preview: Docs Loading link description

[variables](#)

we need to answer a question?

Maybe we have all the data we need but it's in multiple files or tables. Tableau allows us to use data from multiple files using its Relationship feature to automatically (or manually) join files on a common field.

Sticking with our Tree Census dataset, let's say we wanted to be able to show whether trees are native species or not. If we just so happen to have [a dataset with each tree's Latin name and its native species status](#), we can do just that! All we have to do in Tableau is ensure that the two tables form a Relationship based on their shared column.



Dimensions and Measures, Discrete and Continuous

Okay, there's a lot to take in here, but this exercise is one of the most integral to understanding Tableau's data language. Take your time and refer back to the chart on the right as needed.

During the Loading Data lesson, we saw how Tableau assigns different field types to data (numeric,

Preview: Docs Stores a sequence of indexed characters that can be of any length and is contained within a pair of single or double quotes.

[string](#)

, geographic, etc.). Tableau also categorizes each of these fields by their role in our visuals. Tableau calls the two roles *dimensions* and *measures*.

Dimension icons are blue and represent discrete fields: categories and descriptive

Preview: Docs Loading link description

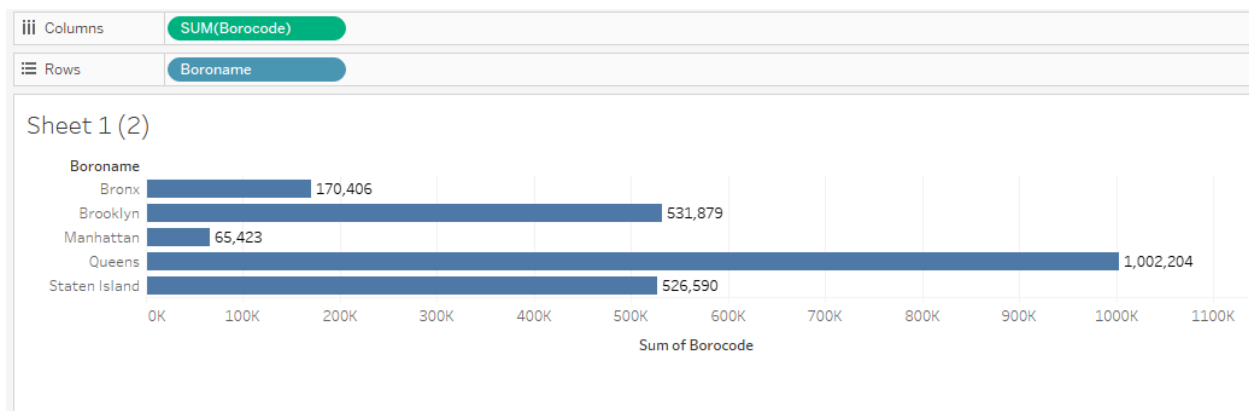
[variables](#)

Measure icons are green and represent continuous fields: numbers that we can perform calculations on.

Tableau is usually pretty accurate in assigning dimensions or measures to a data type, but it's always good to double-check. Remember when we changed the `Tree ID` field from a number to a string? Tableau had assigned this field as a *measure* because it contained numbers. But we knew that the numbers actually represented unique IDs, and we changed the data type to a string so that Tableau will treat this variable as a dimension.

Why does this matter? Let's check out `Borocode` – a field with numbers 1-5 to represent the boroughs of New York City: Manhattan, the Bronx, Queens, Brooklyn, and Staten Island – and see what happens if we let Tableau treat `Borocode` as a measure instead of a dimension.

When `Borocode` is read as a Number and Measure, Tableau aggregates the `Borocode` field in the `SUM()` formula. (Notice that the `Borocode` Pill is green and has been wrapped in the `SUM()` formula.)



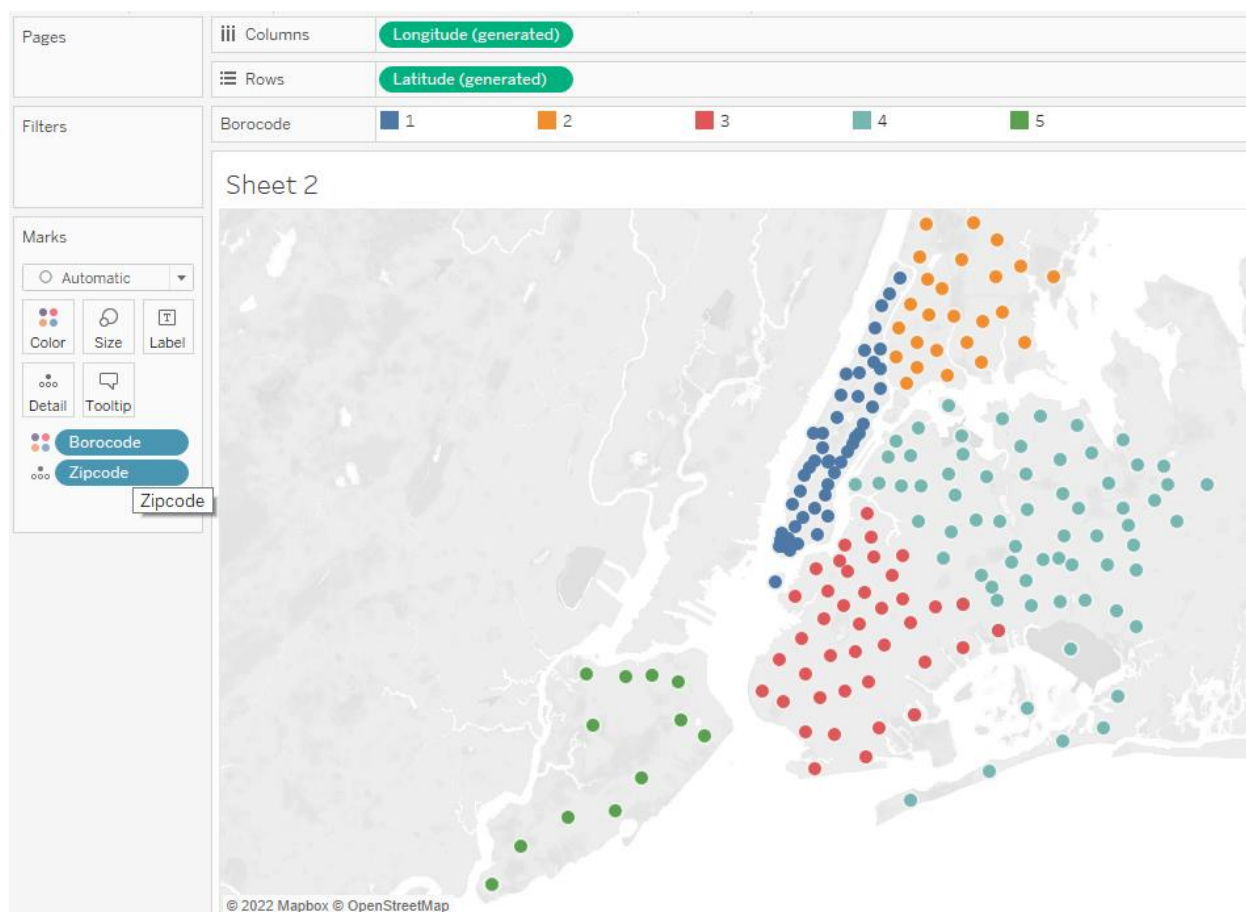
Since the field was "numerical" and set as a measure, Tableau summed the values in that field and created a bar graph to represent the totals. This doesn't tell us anything meaningful because the numbers represent categories! Manhattan has just a small bar in the graph because its `Borocode` is 1 rather than 4 or 5.

When we change the data type so that Tableau reads `Borocode` as a String and Dimension, however, `Borocode` now functions as an ID or label for each borough. Tableau will no longer attempt to do any calculations on this field, and we can see here how it corresponds with the variable `Borocode`.

Columns		
Rows		
Borocode		
Borocode		

Sheet 1		
Borocode	Borocode	
Bronx	2	Abc
Brooklyn	3	Abc
Manhattan	1	Abc
Queens	4	Abc
Staten Island	5	Abc

With that sorted out, we can use the field to make a map that uses the `Borocode` variable to apply color to each borough, for example.



Let's open our workbook with the Tree Census Data so we can explore another example together.

Instructions

You should still have the Tree Census data opened in the data viewer. To open a new workbook, click on the `Sheet1` tab at the bottom left of the window. It may take a few seconds to

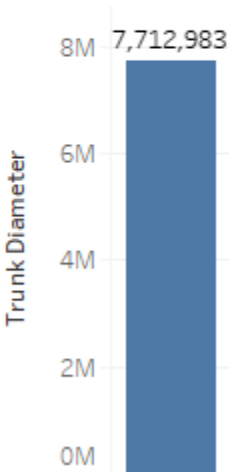
load.
Just as we did in the example in the narrative above, let's change the `Borocode` field type to String and Dimension. In the left pane, you should see the tree census file name with the column

names under it. There is a # next to the `Borocode` column name. Click on this number sign and a drop-down menu will appear. Select `String` so that the data will no longer be understood as a number. The # sign will then change to `Abc.` You must also convert `Borocode` to dimension so that the data is categorical instead of continuous. Hover over the `Borocode` column name and you'll see a small drop-down arrow. Click this arrow and then select `Convert to Dimension`. Once it is converted, it will turn blue and join the

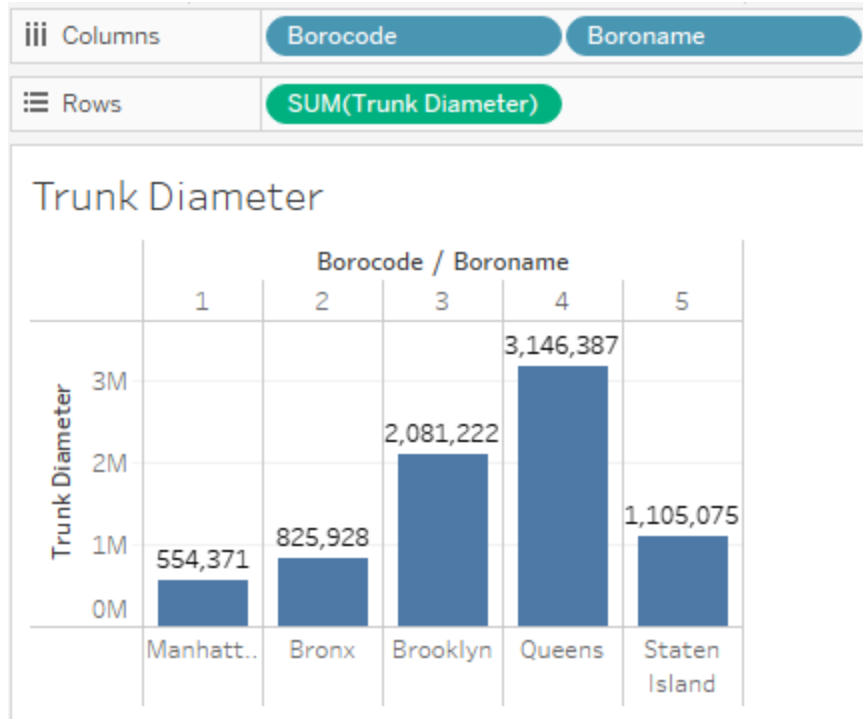
other blue columns at the top of the list.
Now, let's look at the `Trunk Diameter` field (previously the `Tree Dbh` field). It represents tree trunk diameter, measured in inches - that's not obvious from the name, but rather something we can look up in the [data dictionary](#).
Drag the `Trunk Diameter` field to the Rows Shelf at the top of the window. Tableau will `SUM` all the values that are available. It tells us that the sum of all the trees in the dataset is just under 8 million inches in diameter - a totally useless calculation.

Columns	
Rows	SUM(Trunk Diameter)

Trunk Diameter



Now pull the `Borocode` and `Boroname` pills to the Columns Shelf, and we'll see the sum of trunk diameters by borough.

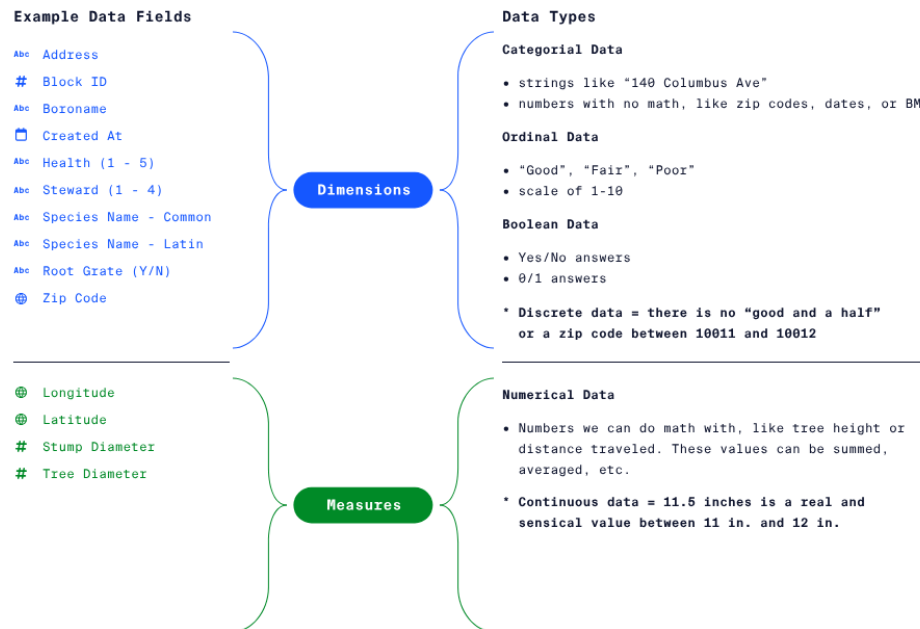


That's more information, but this aggregation still doesn't tell us whether Queens has *more* trees, trees with larger trunks, or both. So let's compare the difference between measures and dimensions for the "Trunk Diameter" field.

Let's make a new sheet. Select the icon to the right of the `Sheet1` button in the bottom left of the

window. A new worksheet named `Sheet2` will then appear. Pull the `Trunk Diameter` field to the Rows Shelf again.

This time, click the arrow on the green pill to show the dropdown menu. Select `Dimension` and `Discrete` and *voila*, you'll see all of the trunk diameters listed out one by one. Tableau now knows to show each of these values rather than summarizing them in the `SUM()` formula.



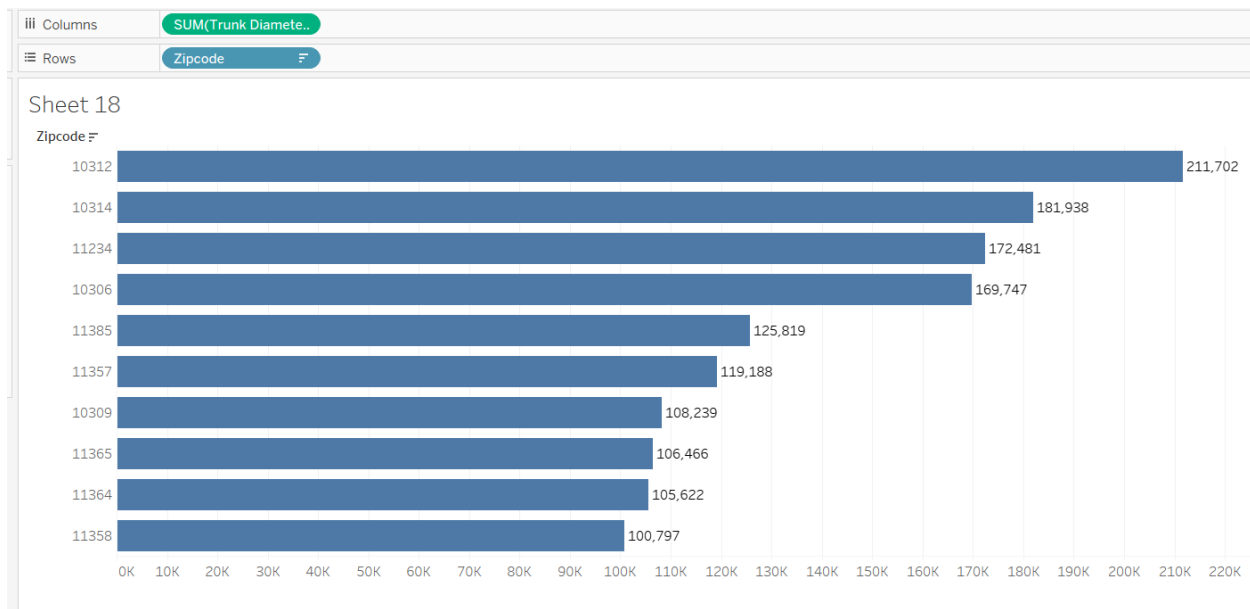
Sums, Counts, and Aggregates

When we talked about Measures (continuous, numerical data), we touched on aggregates. Let's look more into what aggregates are and how we can work with them in Tableau.

The most common aggregates are `SUM()`, `AVG()`, `MEDIAN()`, `COUNT()`, and `COUNTD()`. Check out examples here and in the table to the right.

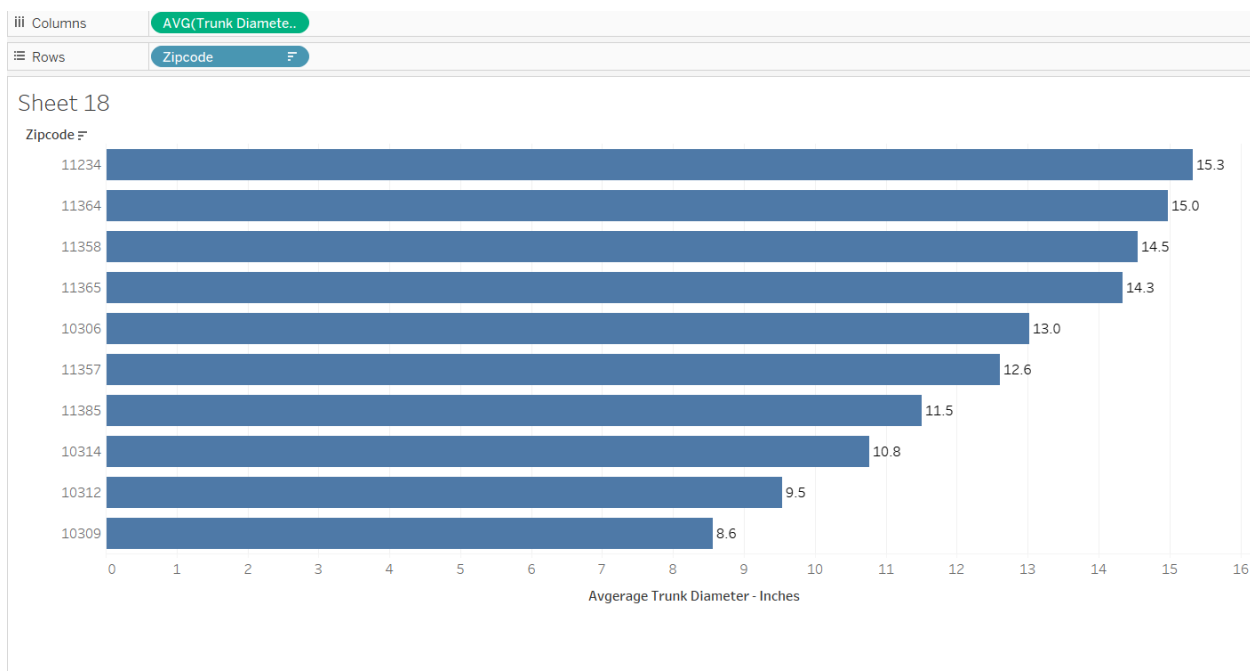
`SUM()` adds all the values in a field.
`SUM(Trunk Diameter)` = 7,712,983 inches
`AVG()` takes the average, or mean, of all values in a field.
`AVG(Trunk Diameter)` = 11.28 inches
`MEDIAN()` takes the median, or middle value, of all values in a field.
`MEDIAN(Trunk Diameter)` = 9.0 inches
`COUNT()` returns the number of items in a field.
`COUNT(Trunk Diameter)` = 683,788 trees with trunk diameter measurements
`COUNTD()` returns the number of distinct or unique items in a field.
`COUNTD(Trunk Diameter)` = 146 different measurements of trunk diameters.

What if we wanted to ask, "which zip code has trees with the biggest trunks?" We can drag the `Zipcode` pill onto the Rows shelf, and Tableau will automatically show us the sum of trunk diameters for all trees by `Zipcode`.

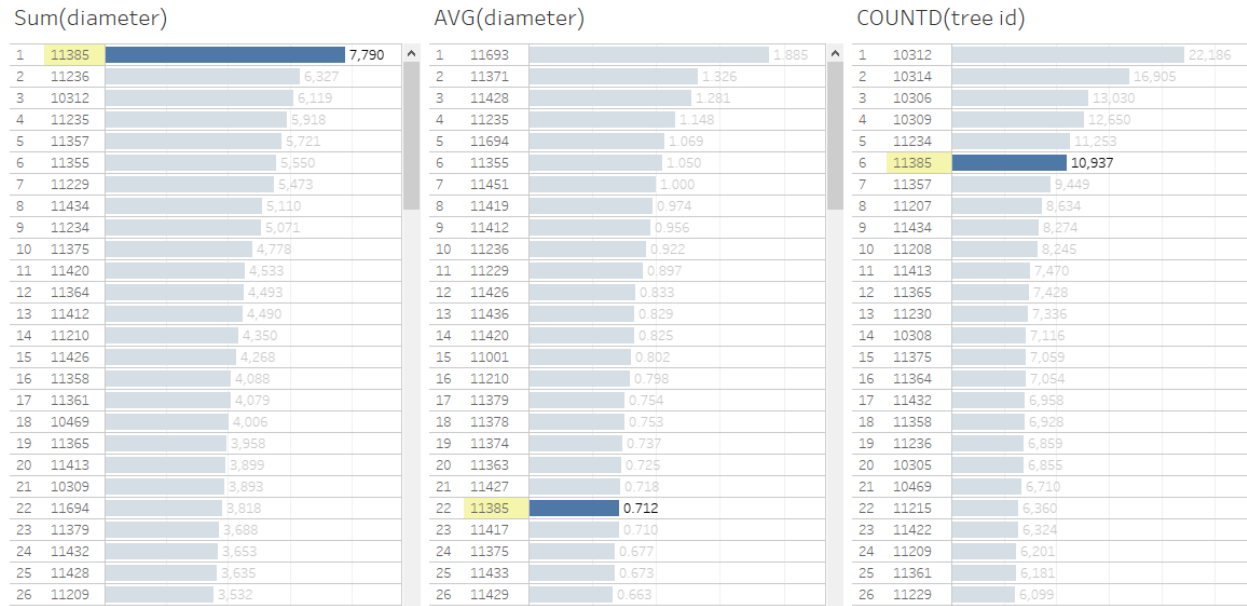


This doesn't answer our question, though, because summing the diameters will be impacted by *how many* trees are in a zip code, not just how big each trunk is. A better aggregation, in this case, would be the Average since that divides the diameter sum by the number of trees. We can see which zip code has the highest average diameters by changing the default aggregate.

Here are the top zip codes by average diameter.



This brings up another important aggregate that we can make. We hypothesized that if the sum of all tree diameters by zip code were sorted that the top zip code would likely be the zip code with the most trees. We can check that with the `COUNT()` aggregate. This counts each record of a given field and even more helpful will be the `COUNTD()` aggregate. The `D` indicates a distinct count of the given field. We will try this by returning a distinct count of `Tree ID` by zip code.



Instructions

Let's create some of these charts to understand aggregates better. Open Tableau and connect to your tree census data (`tree-census-NYC_2015.csv`). Then open a workbook. You may open a new workbook for every new chart we create (to change the title of your worksheet, double-click the title where it says, "Sheet 1" and enter the new title).

Drag the `Trunk Diameter` field to your Rows shelf. (You can also double-click the field name and a pill will be created for you on the Rows Shelf.)

Pull `Borough` to Columns and click the Swap Rows and Columns button.

Drag a new `Borough` pill to the Color Shelf. (You can hold CTRL while dragging to make a copy of a pill from the Rows Shelf. Command+drag on a Mac.) Your view should look like this so far:

You'll notice that the aggregation is still using `SUM(Trunk Diameter)`, as this is Tableau's default, but we know that it isn't accurate for finding the borough with the largest trunks. Let's switch that to `AVG(Trunk Diameter)`. Hover on the pill and click the dropdown arrow to open the pill's menu.

Select `Measure (Sum)` and then `Average`.

Hold down CTRL (or Command on a Mac) and drag the `AVG(Trunk Diameter)` pill to the Label shelf.

Hover over `Borough` in the Rows shelf and select `Sort`. Sort by Field in Descending order with the

aggregation of Average.

Now we see that Queens has the highest average trunk size in the city with 12.6 inches in

diameter. Queens also has the greatest SUM() of diameters but by a much wider

margin than when compared by average.

We can also see how many distinct trees are in the data. Remove Trunk Diameter from the columns shelf and the label shelf (you should still have Boroname on the Rows shelf and Boroname on the Color shelf). Next drag and drop the Tree Id column in the columns shelf (you will see a warning but just click Add all members). Then click the drop-down menu on Tree Id and select Measure and then Count (Distinct). This will count all the distinct trees for each Boroname. Feel free to CTRL and drag CNTD(Tree

Id) to the Label shelf again.

You can also make each value a percent of the total number of trees by clicking the drop-down on CNTD(Tree Id), selecting Quick Table Calculations, and then Percent of Total. You will need to change the label as well. Remove CNTD(Tree Id) from the label shelf, then CTRL and drag CNTD(Tree

Label shelf again.

We can see that Queens has about 11% more trees than Brooklyn, which accounts for the much larger share of the Sum in the top chart. While the order is the same with each aggregation (this isn't always the case!), we can understand tree trunk size in NYC better by comparing multiple aggregates.

AGGREGATE	What does it do?	Example	Graph												
SUM	adds all the values in a field	<code>SUM(Trunk Diameter)</code> = 7,712,983 inches	SUM (Trunk Diameter) <table><thead><tr><th>Borough Name</th><th>Sum (Trunk Diameter)</th></tr></thead><tbody><tr><td>Bronx</td><td>120,000</td></tr><tr><td>Brooklyn</td><td>180,000</td></tr><tr><td>Manhattan</td><td>100,000</td></tr><tr><td>Queens</td><td>150,000</td></tr><tr><td>Staten Island</td><td>110,000</td></tr></tbody></table>	Borough Name	Sum (Trunk Diameter)	Bronx	120,000	Brooklyn	180,000	Manhattan	100,000	Queens	150,000	Staten Island	110,000
Borough Name	Sum (Trunk Diameter)														
Bronx	120,000														
Brooklyn	180,000														
Manhattan	100,000														
Queens	150,000														
Staten Island	110,000														
AVERAGE	takes the average, or mean, of all values in a field	<code>AVG(Trunk Diameter)</code> = 11.28 inches	AVG (Trunk Diameter) <table><thead><tr><th>Borough Name</th><th>Average Trunk Diameter</th></tr></thead><tbody><tr><td>Bronx</td><td>10.5</td></tr><tr><td>Brooklyn</td><td>11.5</td></tr><tr><td>Manhattan</td><td>9.5</td></tr><tr><td>Queens</td><td>10.5</td></tr><tr><td>Staten Island</td><td>10.0</td></tr></tbody></table>	Borough Name	Average Trunk Diameter	Bronx	10.5	Brooklyn	11.5	Manhattan	9.5	Queens	10.5	Staten Island	10.0
Borough Name	Average Trunk Diameter														
Bronx	10.5														
Brooklyn	11.5														
Manhattan	9.5														
Queens	10.5														
Staten Island	10.0														
MEDIAN	takes the median, or middle value, of all values in a field	<code>MEDIAN(Trunk Diameter)</code> = 9.0 inches	MEDIAN (Trunk Diameter) <table><thead><tr><th>Borough Name</th><th>Median Trunk Diameter</th></tr></thead><tbody><tr><td>Bronx</td><td>8.5</td></tr><tr><td>Brooklyn</td><td>9.5</td></tr><tr><td>Manhattan</td><td>8.0</td></tr><tr><td>Queens</td><td>9.0</td></tr><tr><td>Staten Island</td><td>8.5</td></tr></tbody></table>	Borough Name	Median Trunk Diameter	Bronx	8.5	Brooklyn	9.5	Manhattan	8.0	Queens	9.0	Staten Island	8.5
Borough Name	Median Trunk Diameter														
Bronx	8.5														
Brooklyn	9.5														
Manhattan	8.0														
Queens	9.0														
Staten Island	8.5														
COUNT	returns the number of items in a field	<code>COUNT(Trunk Diameter)</code> = 683,788 trees with trunk diameter measurements	COUNT (Trunk Diameter) <table><thead><tr><th>Borough Name</th><th>Count of Trunk Diameters</th></tr></thead><tbody><tr><td>Bronx</td><td>120,000</td></tr><tr><td>Brooklyn</td><td>180,000</td></tr><tr><td>Manhattan</td><td>100,000</td></tr><tr><td>Queens</td><td>150,000</td></tr><tr><td>Staten Island</td><td>110,000</td></tr></tbody></table>	Borough Name	Count of Trunk Diameters	Bronx	120,000	Brooklyn	180,000	Manhattan	100,000	Queens	150,000	Staten Island	110,000
Borough Name	Count of Trunk Diameters														
Bronx	120,000														
Brooklyn	180,000														
Manhattan	100,000														
Queens	150,000														
Staten Island	110,000														
COUNT DISTINCT	returns the number of distinct or unique items in a field	<code>COUNTD(Trunk Diameter)</code> = 146 different measurements of trunk diameters	COUNTD (Trunk Diameter) <table><thead><tr><th>Borough Name</th><th>Distinct count of Trunk Diameter measurements</th></tr></thead><tbody><tr><td>Bronx</td><td>85</td></tr><tr><td>Brooklyn</td><td>95</td></tr><tr><td>Manhattan</td><td>65</td></tr><tr><td>Queens</td><td>90</td></tr><tr><td>Staten Island</td><td>80</td></tr></tbody></table>	Borough Name	Distinct count of Trunk Diameter measurements	Bronx	85	Brooklyn	95	Manhattan	65	Queens	90	Staten Island	80
Borough Name	Distinct count of Trunk Diameter measurements														
Bronx	85														
Brooklyn	95														
Manhattan	65														
Queens	90														
Staten Island	80														

Filtering

If you've ever encountered the so-called "[rainbow spinning wheel of death](#)," then you know what it looks like when an application or computer is struggling with the amount of work it's been given. Obviously, that's something we want to avoid, and luckily Tableau has built-in tools to help us manage the data load, even when working with hundreds or thousands of lines of data.

Tableau's filter function allows us to improve performance as well as remove null values or other extraneous data points.

During dashboard development, filtering helps Tableau's performance when adding new elements to our viz. Each time we add a pill to the view, Tableau will recalculate and redraw all the currently existing marks and elements. Filtering during development reduces the number of marks that Tableau has to redraw for each of our actions, meaning less waiting time during development.

Whenever we make a visualization that plots every individual data point, rather than an aggregate, filtering can be an important first step. We'll see an example of this when trying to make a map of all trees in NYC, but the same principle applies to a scatterplot as well.

Let's return to where we left off and use some filtering best practices to improve the speed of development and user interactions with our new dashboard.

Instructions

Open your Tableau workbooks and find the “Count D” visualization we created. This tab should show a horizontal bar chart of `CNTD(TreeID)` and `Boroname`. See the drop-down if you need to create it again.

Count D Viz Instructions

The `Health` field will give us the status and health of each tree. It also provides a good filter, since we can choose to only show trees of one status. Drag `Health` to the Filters

Shelf and select *Good*.

Now all the counts are filtered for only trees in good health. To further improve performance or develop a visualization quickly, we can add additional filters.

Now let’s make a map to see how much filters help when we have a *lot* of data points. Open a new worksheet in your workbook and name the sheet “Tree Locations”.

Add `Longitude` and `Latitude` measures to the view by double-clicking on each field name (`Longitude` should go to the columns shelf and `Latitude` should go to the rows shelf). You should see a map with a single point that represents the average of all the latitude/longitude values for each tree.

We can see the latitude and longitude for *each* tree by adding the `Tree ID` to the *Detail* shelf BUT before we do this let’s reduce that wait time by adding a filter. Pull the `Boroname` field to the Filters shelf and make a single selection to show just one Borough at a time – try `Bronx`. Now any development we do, whether adding color or looking at `Health` on the map, will be much quicker to load, since we’ve reduced the number of marks from almost 700k to about 85k.

Now pull `Tree ID` to the Detail card. In the pop-up choose *Add all members*. This will almost certainly take some time for Tableau to render. Then drag `Health` to the Color card.

To make the map more readable, click on the “Size” box inside the “Marks” box. Slide

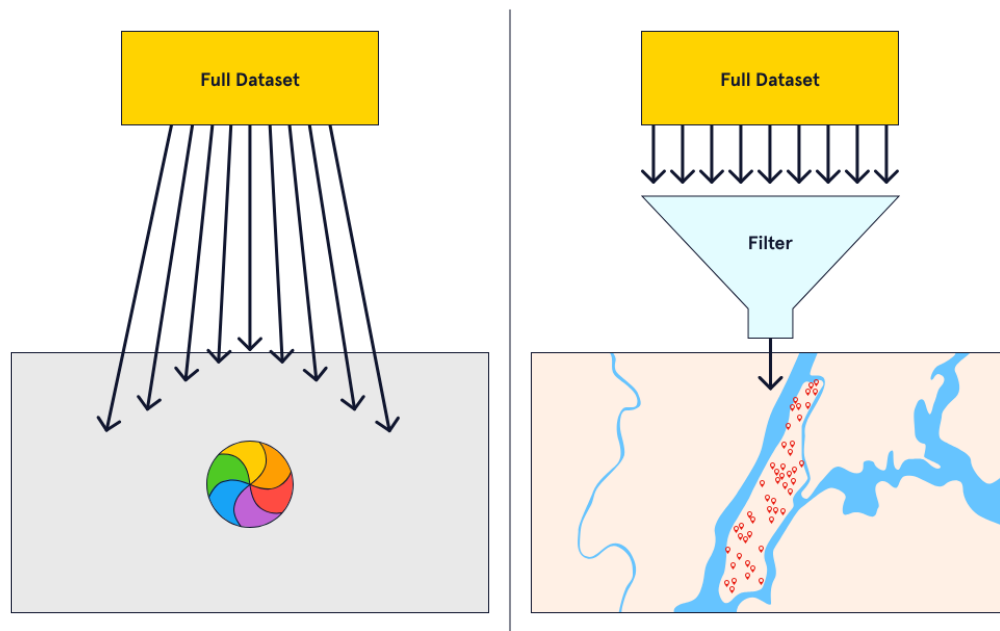
the lever all the way to the left.

This will make each data point smaller on the map. When the data points are larger, it seems the map is mostly filled with blue data points (Null data). After we make the data points smaller, we can see the blue

dots are actually the minority.

2. If you want things to move even faster or are concerned about your computer’s processing power, filter by a few `Zip CodeS` instead of `Boroname` to further reduce the number of points you’re plotting.

Lastly, In the Filters Shelf, click the dropdown on the `Boroname` pill and select Show Filter. This will add a window with a selectable checkbox on the view, allowing viewers to change the filter on the visualization.



Aesthetic Properties, aka Marks Cards

We learned how to generate a bar chart in the previous lesson. Now we'll enhance that chart with more information using the Marks Card. We'll walk through the cheat sheet on the right to see what each component does.

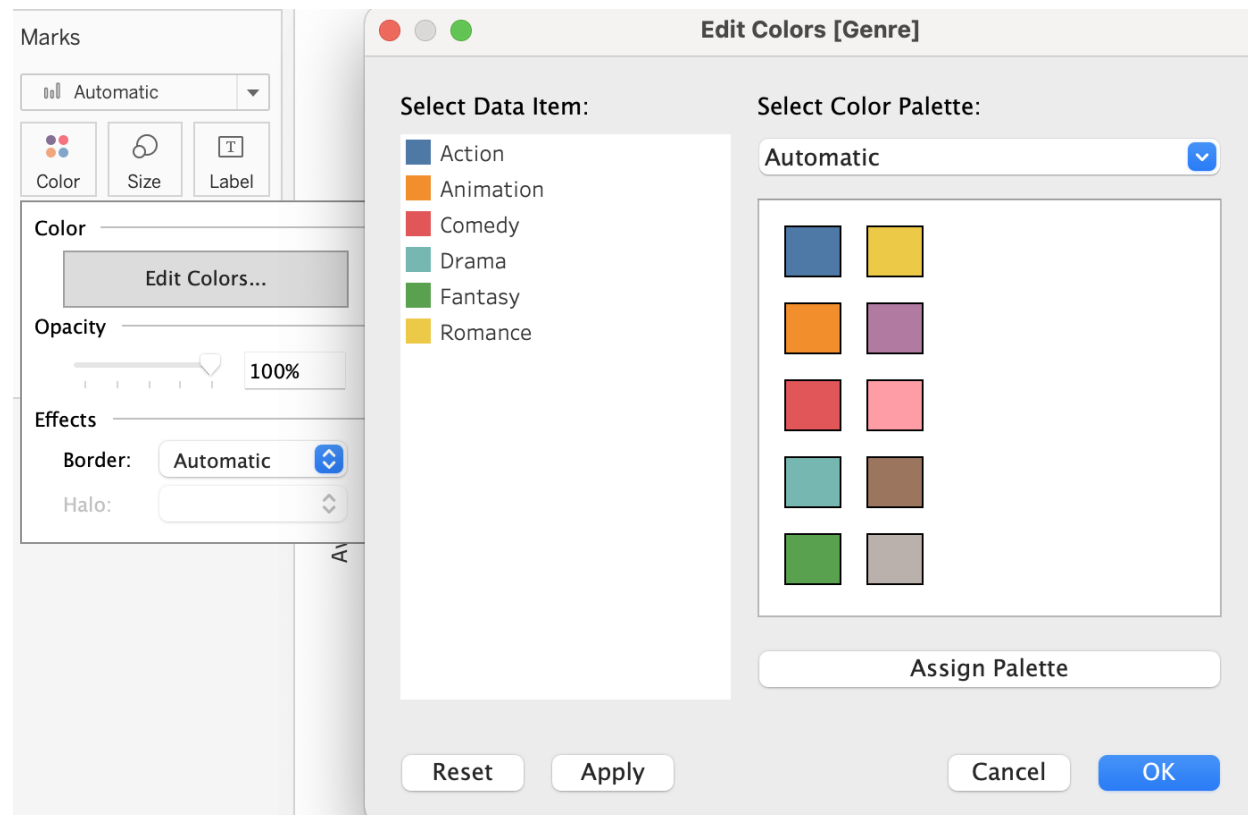
In Tableau, we can change aesthetic properties by dragging a field to one of the tiles on the Marks Card: Color, Size, Label, Detail (sometimes labeled Text), or Tooltip. We can change the chart type by selecting a chart in the Chart Dropdown. Let's try some out!

Instructions

Let's make some customizations to our bar chart from the Movie Profitability datasheet. First, make (or remake) the simple bar

chart by dragging `Genre` to the Columns Shelf and `AVG(Ratings)` to the Rows Shelf.

Drag `Genre` to the Color Mark and click the Color icon to assign a color palette to the data items.

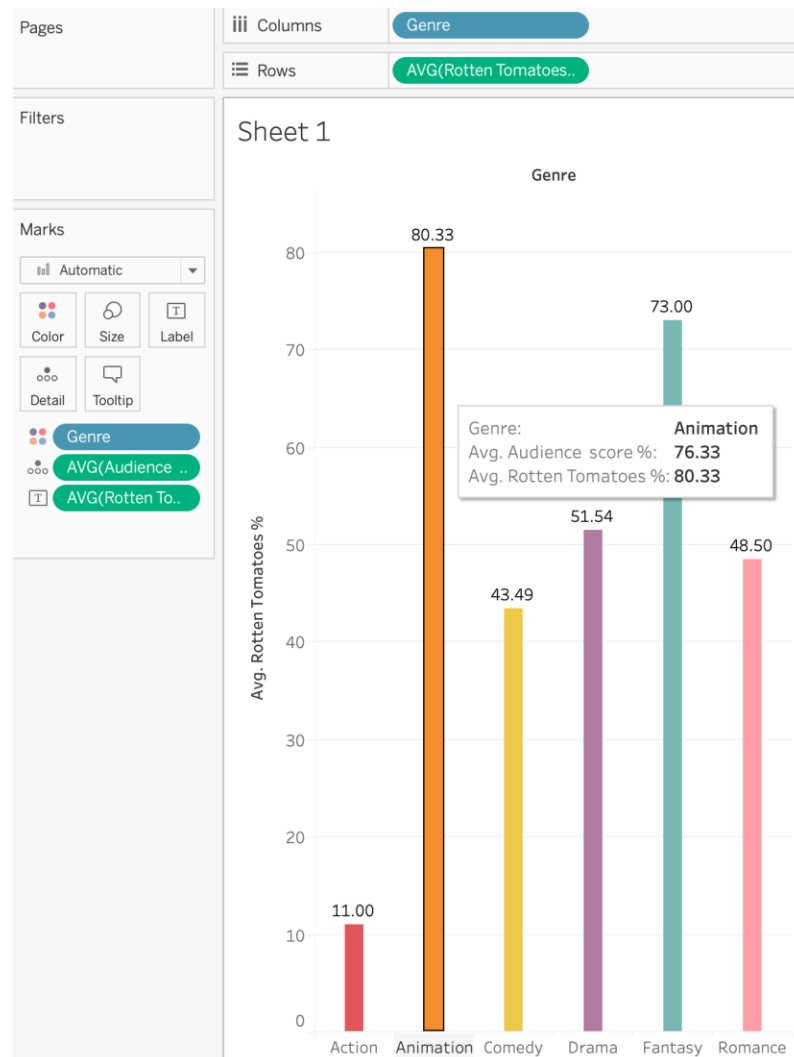


Next, click the Size mark and adjust the slider to make our bars thinner.

Drag the `Rotten Tomatoes %` field to the Label mark to display the average % labels with the corresponding bars. (Remember to change it to `AVG()` instead of `SUM()` so that the correct information is displayed.

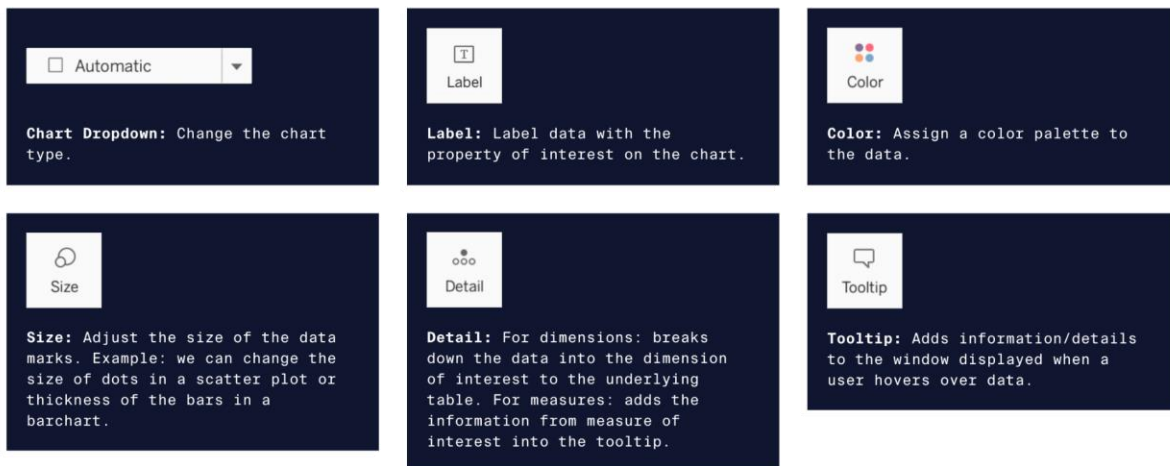
To add information about audience ratings, drag `Audience Score %` to the Detail mark. This will generate information about (again, remember to change to average so the measure is consistent with

Rotten Tomatoes %) audience ratings. Your bar chart should look something like this:



Congratulations! You customized your first viz.

Optional: Using the same dataset, create another viz with data fields of your choice. Make customizations to at least 2 marks properties of your choice. What did you choose and why? Think about how these modifications improve your viz. We're now getting the hang of vizzing! Next up, we'll learn about adjusting presets to make our visualizations look even more original.



Adjusting Presets

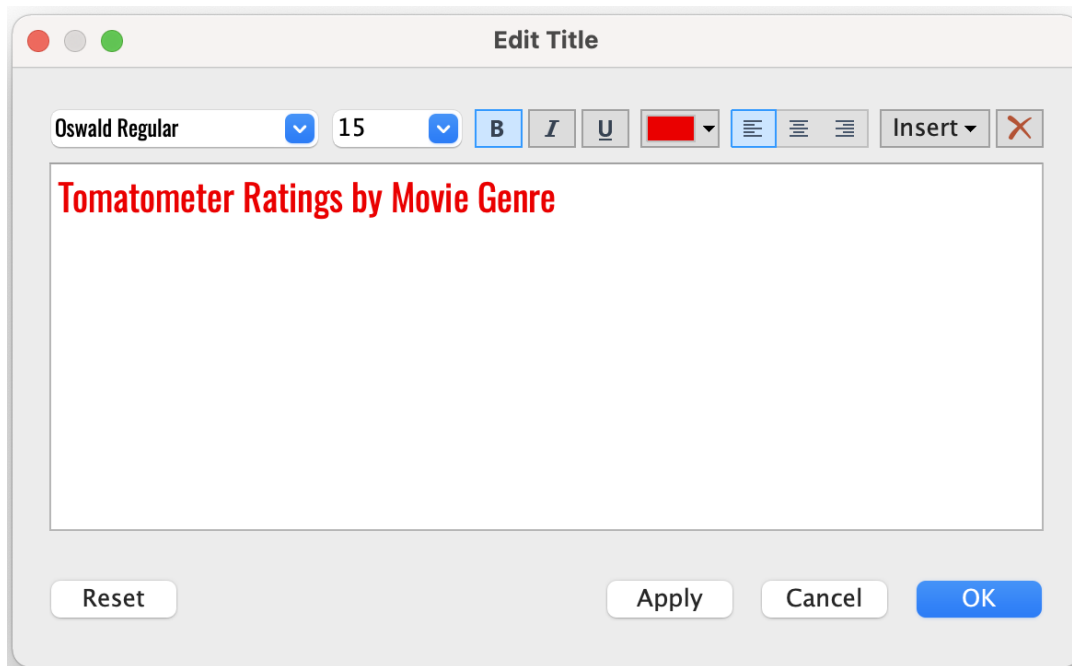
So far, we've created vizzes with Tableau's existing presets and learned how to use the Marks Card to change certain properties. This can create helpful and informative visualizations, but to take our visual communication to the next level, we can change up the default settings for a custom look.

Thoughtful formatting choices when it comes to font, color, spacing, and placement make a big difference to the final presentation. Using the same bar chart we created in the prior lesson, let's explore some of the adjustments available to us in Tableau.

NOTE: while custom fonts can make a huge visual impact, keep in mind that they may not show up for viewers who don't have fonts downloaded. The safest fonts are those in the Tableau font family, and any that are considered web safe ([example workbook here](#)).

Adjust the default chart title:

Right click `Sheet 1` and select Edit Title to adjust the font, size, weight, color, and alignment.



Adjust worksheet, axes, and tooltip fonts:

Click Format on the top menu, and navigate to Font to adjust the properties. We can choose which sections to apply these changes to.

Format Font

×

A

≡

🔍

📊

≡

Fields ▾

Sheet

Rows

Columns

Default

Worksheet:

Raleway, 9pt

Pane:

Raleway, 9pt

Header:

Raleway, 9pt

Tooltip:

Tableau Bo..

Title:

Tableau Lig..

Total

Pane:

Raleway, 9pt

Header:

Raleway, 9pt

Grand Total

Pane:

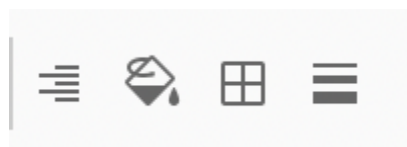
Raleway, 9pt

Header:

Raleway, 9pt

Adjust other chart properties:

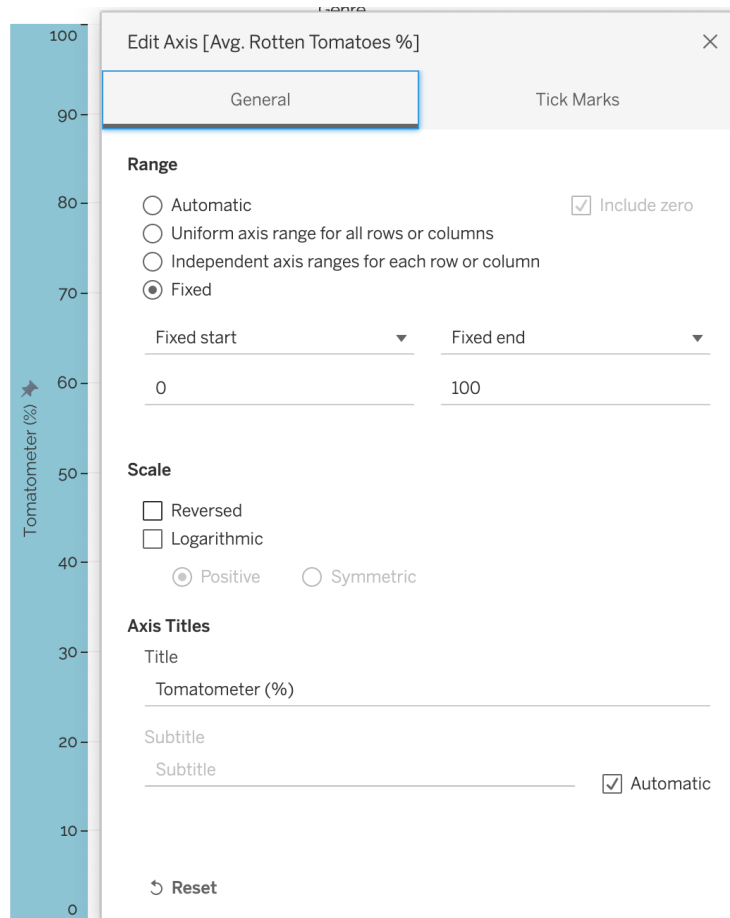
On the same Format Font menu, navigate to the other icons:



From left to right: change alignment, shading, borders, and lines. (Feel free to click around and undo if you don't like the change.)

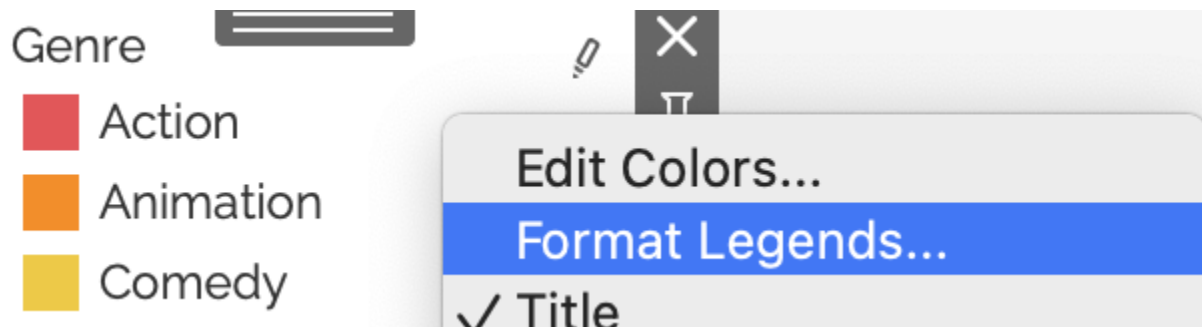
Adjust Axes

Right click on the axis of interest to adjust the range, scale, title, and tick marks. (Feel free to click around and undo if you don't like the change.)



Adjust Legends:

If your chart has a legend, right click and select Format Legends to adjust.



The number of options might seem overwhelming, but never fear: the next exercise will help us to design with an audience and story in mind!

Instructions

Now that we've discovered the world of adjusting presets, let's put our new customization skills to work. Customize the title of your viz. Try to use color and font style that speaks to the theme you want your visualization to convey. Adjust your axis titles: Rather than using the default data field names, change the axis titles so that they're not too redundant, but give the viewer just enough context.

Adjust your axes: You may also want to adjust the range to capture all of your data. For example, if the largest data point is 106.8, you may consider ending the range at a human-friendly number like 110 or 115.

Change the background shading and fonts of the worksheet. Again, keep your theme in mind when making your choices. For example, in the movies dataset, we might want to incorporate a colorful palette and keep things fun. How does your new viz compare to your original? Do you think the changes made the viz more effective? How so?

Storytelling Tools

So far we've been using the Rotten Tomatoes dataset in our vizzing journey. Let's close that out for now and apply the principles we've learned so far toward a new dataset for some additional practice. We'll be working with a [dataset on volcanic eruptions](#) this time. Scroll down to "significant-volcanic-eruption-database.csv" and then you should see a download icon (an arrow pointing downward to a horizontal line) to the right of the file name.

In the previous exercise, we adjusted presets to make our visualizations more effective. Similarly, we can use tools in Tableau to strengthen our story (you'll read more about this in an upcoming article as well). We can use tooltips to fill in the details, add annotations to give more context, or even sort data to present it in a more effective way.

Take the bar charts to the right as an example: we can apply a new "volcano-themed" color palette and title that better matches the topic of the viz. And by changing the tooltip text to

provide context and sorting the data from greatest to least value, we better illustrate the impact of volcanoes.

Little tweaks to our vizzes can make our story that much more powerful.

Instructions

Let's add storytelling elements to a viz. Using the volcanoes dataset, create a chart of your choice.

Customize the tooltip of your viz. (Note: the default tooltip only displays the fields used in the chart - if you want to add context by using another field, drag that field into the tooltip mark.) The default tooltip displays the data field(s) title as text followed by the dynamic field or calculation in the <> brackets. Here, we've modified the tooltip to a sentence that reads, "In 1815, volcanoes caused 10,000 deaths", rather than just "Year: 1815, Deaths: 10,000."

Montserrat 10 B I U [Color Picker] [List Icon] [List Icon] [List Icon] Insert X

Year: <Year>
Deaths: <SUM(Deaths)>

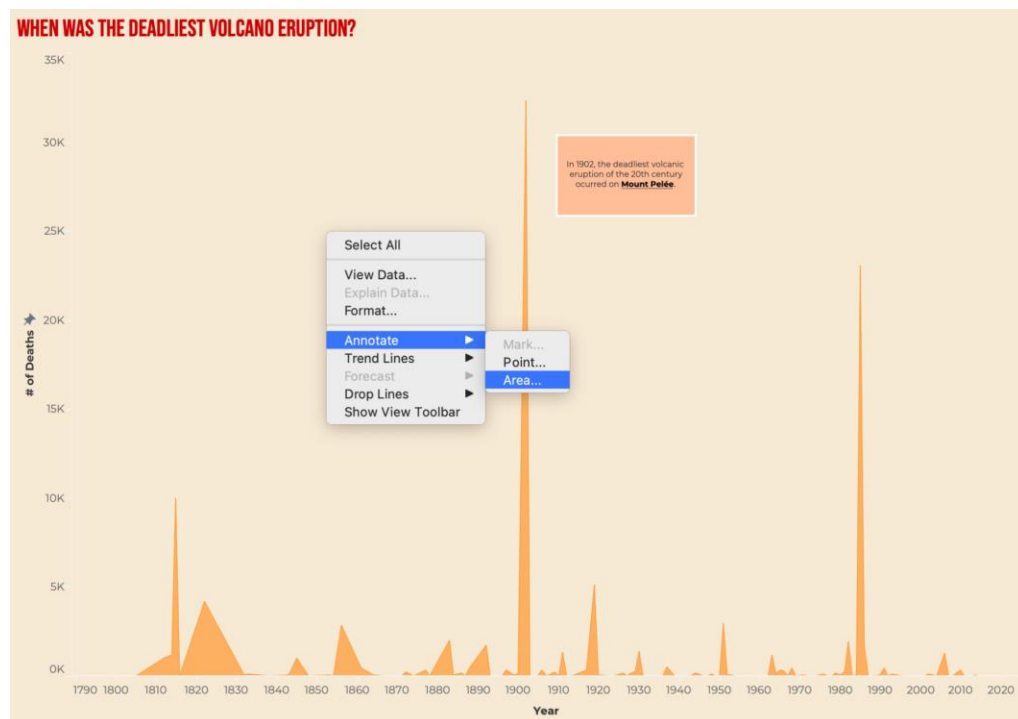
In <Year>, volcanoes caused <SUM(Deaths)> deaths.

☒ Show tooltips Responsive – Show tooltips instantly [Dropdown]
☒ Include command buttons
☒ Allow selection by category

Reset Preview Cancel OK

Identify a datapoint you want to provide additional context for. This can be an outlier or any particular datapoint you find interesting. To annotate the point,

right click on it and navigate to `Annotate > Area`. A text box will appear to type into. Once you're done, an annotation object will appear on the viz - you can right click the annotation object to format as desired. In this area chart, we visualized the number of deaths caused by volcanoes across years. Notice how there was a significant spike around the early 1900s? With a little bit of research, we learned that a volcano named Mount Pelée erupted then. This annotation will help the audience understand the sudden uptick in deaths and answer a question that will likely arise for many viewers.



Feel free to continue customizing or create a new custom chart from this dataset. In the next exercise, we will learn to put all of our story pieces together in a dashboard.

Dashboarding 101

Dashboards are a collection of several visualizations or views. They are particularly useful when we want to display multiple views at once.

We'll create a dashboard using the volcanoes dataset by clicking the dashboard button on



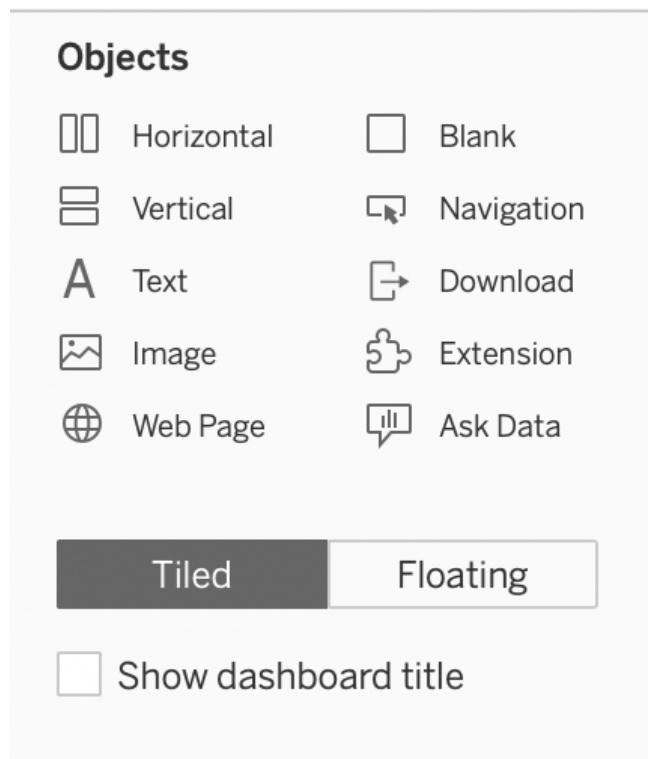
the bottom menu.

A blank dashboard window will appear, and we can begin dragging Sheets with our vizzes into the pane. (Tip: hold the shift key while dragging to maintain the aspect ratio while adjusting the size.)

When repositioning and resizing charts, we'll try to maintain a visual balance and make sure no elements of a visualization are obscured. For reference, we've got a "Before and After" dashboard on the right hand side of the screen. Notice how, in the Before image, legends are incorrectly placed, one chart cuts another off, and title alignment is inconsistent. The After dashboard is balanced but not symmetrical, dynamic but not crowded, and uses both graph space and blank space.

To add elements like text and imagery, use this menu:

Clicking and dragging the **Text icon** to the dashboard pane lets us add a title and a few sentences of context. Don't skip this step! A little bit of verbal context is one of the most helpful data viz tools at our disposal



Dragging in the **Image icon** allows us to upload images as an element on the dashboard. This one is more optional, but a logo, simple cartoon image, or icon related to the subject matter can help the viewer to instantly intuit what the dashboard is about. It's best to stick to iconography and avoid images taken with a camera.

Notice how we've played with font, color, and chart elements to tie the viz back to the volcano theme.

As always, keep **your** audience and story in mind, and adjust your aesthetic decisions accordingly - all the resources from the Storytelling exercise we just finished also apply to dashboards.

Let's get into it! For additional dashboard inspiration, check out our Rotten Tomatoes dashboard and others on our [Tableau Public page](#).

Instructions

Begin by dragging at least 2 vizzes that you've created into the dashboard pane. Resize and reposition as needed to ensure a balanced composition with nothing cut off.

Add a dashboard title using the Text button from the Objects menu.

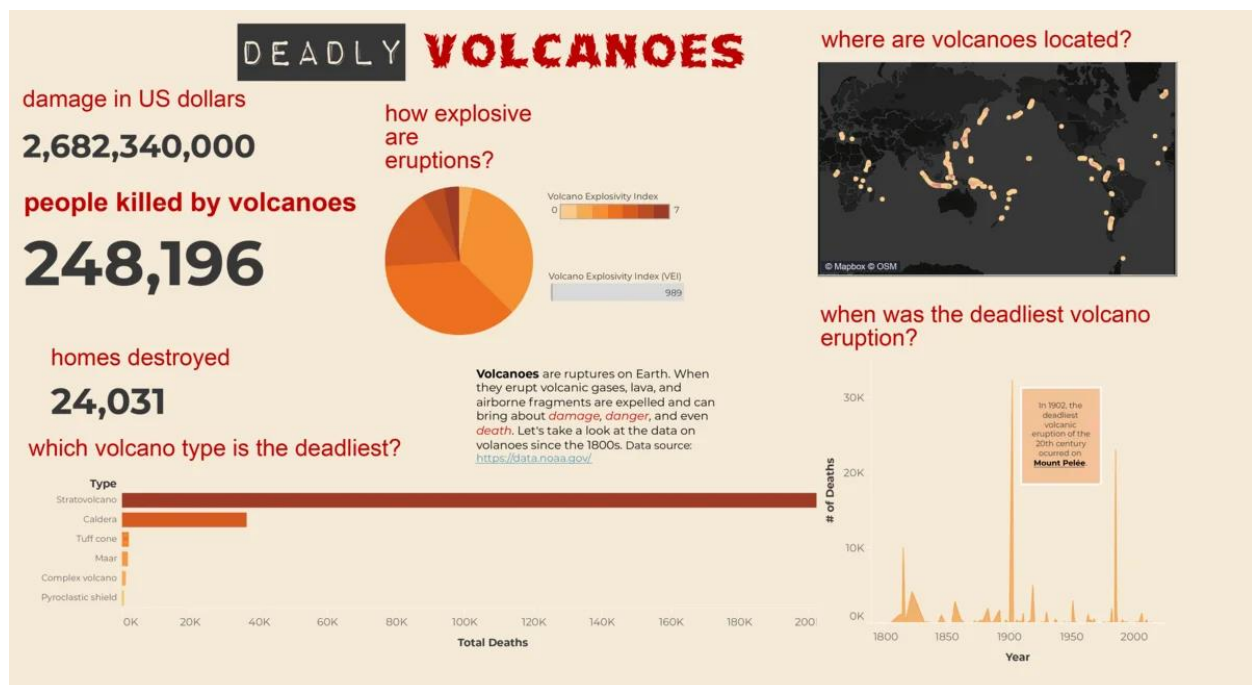
Customize this so it matches your theme.

Using the same Objects menu, optionally add some image elements related to your theme.

Change the background color of your dashboard.

Make any additional changes as you see fit, remember to have fun with it.

Amazing! You've just created your first ever Tableau dashboard. Next up, we will review key takeaways from this lesson so you can continue creating beautiful visualizations.



Takeaways

Phew, that was a lot of Tableau! Here's a refresher with everything we've covered - you can use this page as a reference in all your future Tableau endeavors.

Essential components of Tableau

Drag fields from the data pane to the columns and rows shelf to create visualizations, and filter out data in the Filters Shelf.

Stylize and adjust the chart type using the Marks Card. The Show Me tab shows us all the charts available. To break the viz into different views, we can use the Pages Shelf.

Starting on Paper

Sketch on paper first and consider the following questions: What is the purpose of this viz and what are the research questions, who is the audience, and what actionable insights can be taken?

Dimensions and Measures

We can configure dimensions and measures in numerous ways, resulting in different visualizations, by dragging them into different shelves or marks.

Marks Cards

Use marks to customize the chart type, chart-specific properties, color, size, label, detail and tooltips.

Adjusting Presets

For a customized look, we can make changes to default Presets including fonts, axes, legends, and worksheet/chart properties.

Storytelling

Seven approaches to data storytelling are change over time, drill down, zoom out, contrast, intersections, factors, and outliers.

Keep the viz simple with the purpose, audience, and actionable insights in mind, while utilizing aesthetic properties to strengthen our story's theme.

Dashboarding

Dashboards are a collection of vizzes created by dragging different worksheets into the view.

Practice strategic positioning, sticking within 4 views and having a main focal point to maintain good visual balance.

Congratulations on completing the Data Visualization with Tableau lesson!

More on Storytelling

Learn more about how to use Tableau to effectively tell a visual story.

Tableau is a fantastic tool with many options for visualizing data. With so many choices at our disposal, it's helpful to go through the steps of designing a story and review best practices.

Part I: Designing a Story

Many successful data visualizations and dashboards start by working on paper first. This is where the storytelling process begins.

Think about the following, and write or sketch out what comes to mind:

With these questions in mind, we can begin to carve out the viewer's journey. If we're making an argument, we may want to identify supporting data and think about the best sequence to present it in. For a narrative, we may want to give viewers the option to explore the data to come up with their own conclusion, or we might choose to lead the viewer to a specific conclusion.

Generally, the best way to present a story in Tableau is using a dashboard, which is a combination of several visualizations that can be arranged in a single presentation, like an interactive poster. We may also opt to use a Tableau story, which is a series of visualizations or dashboards. Stories allow us to use buttons to navigate through the views, like turning pages of a book.

Tableau offers some commonly used **approaches to storytelling with data** that can help us find focus in a visualization or dashboard.

Change Over Time: Uses temporal data to illustrate trends or shifts

Drill Down: Starts with the bigger picture, then gets into finer details

Zoom Out: Starts with a smaller detail then gets into broad and high-level picture

Contrast: Depicts differences between 2+ subjects

Intersections: Explores how 2 initially separate dimensions may converge

Factors: Takes a broader subject and divides it into factors (types/categories)

Outliers: Highlights and profiles outlier in the data

Whichever approach is right for your data, remember to check in as you create and tweak visualizations. Try to take on a beginner's mindset, and imagine that you're seeing your

dashboard for the first time. Does it answer the questions you hoped to address? If there's a central argument, is it clear to the viewer? If the dashboard is made for exploration, is it easy to navigate?

Part II: Best Practices

Now that we have a more focused direction on Storytelling, let's cover some best practices for communicating through data visualizations.

Choose the Right Chart

This one might seem basic, but it's a crucial part of the overall design and readability of the final project. Tableau gives us the flexibility to represent our data in numerous ways. Here are a few of Tableau's example chart options based on the relationship in the data.

Temporal changes: Line chart, area charts

Parts of a whole: Pie chart, Treemap

Relationship between variables: Scatterplot

Distribution: Histogram, box plot

Ranking or Magnitude: Bar chart, packed bubbles

With the right chart, everything else becomes easier. (On that note, if you feel like you're fighting your visualization at every step... consider trying a new chart type, or getting out the ol' pen and paper to see if you can make it work away from technological challenges.)

Don't Overcomplicate It

When crafting dashboards and stories, it's almost always better to keep things simple. Limit the number of vizzes in a dashboard and try to not overcrowd a view, keeping things simple for viewers to understand. It may be helpful to stick to 3-4 vizzes in a dashboard and eliminate any elements that do not add to the story.

Visual Balance

In art and design, visual balance is the sense of "weighted clarity in a composition." In other words, the way that visualizations are arranged should help them to make sense, not make

things look muddy, messy, or crowded. The most important things should be visually highlighted through size and positioning, and other elements should support without distracting or competing.

As designers, we should aim to maintain a sense of balance in our vizzes by utilizing elements such as positioning, size, color, shapes, and textures. There is an endless amount of information on good design, but if you're looking for a starter, read more about the basic forms of visual balance in the source linked at the bottom of the article.

Using Aesthetic Properties to Convey a Story Theme

Another step to effective storytelling is to strengthen our story by reinforcing a visual theme throughout. For example, if we're analyzing Olympic Games medals data, we might want to use colors of the medals and incorporate the logo of the Olympics in the title. We may also consider our theme when making font and chart choices. These choices can (subtly or forcefully) help the viewer to quickly understand what the dashboard is about.

Let's take the visualizations on the volcanic eruptions dataset as an example. When we think about volcanoes, that may evoke imagery of hot lava flow, thick ash, mountainous shapes, and fiery colors. Keeping in line with this imagery, we chose to use charcoal gray, reds, and oranges in the color scheme. The fonts used were bold and striking. We did a play on a hazard sign and a fire-like shape on the title. The use of an area chart with peaks and valleys was intentional and served to imitate the silhouette of volcanic terrain.

For more practice, check out [this dashboard](#) that uses Rotten Tomatoes data on movie profitability. See if you can pick up on which visual elements help to convey the theme in this data.

Tying It All Together

Once you've finished creating your viz, revisit it and ask yourself these questions:

Does the visualization serve its purpose, i.e. answer the core question posed?

Are my narrative approach and chart choices appropriate and easy to follow?

Am I following visualization best practices?

Have I used aesthetic properties to enhance my story?

The more you practice storytelling through visualizations, the easier and more intuitive it will become. We encourage you to keep making visualizations, keep asking yourself “Why does that look so *good*?” (or so “*not so good*”!) when you see striking data visualizations in the wild, and keep participating in our Codecademy forums and the Tableau Public community, which are fantastic resources for inspiration, feedback and motivation.

Sources:

Storytelling best practices: https://help.tableau.com/current/pro/desktop/en-us/story_best_practices.htm

Choosing the right chart: https://help.tableau.com/current/pro/desktop/en-us/what_chart_example.htm

Visual balance: https://wikieducator.org/Artistic_principles/Visual_balance

TABLEAU PROJECT

PROJECT I

https://www.youtube.com/watch?v=itLufvC00bk&ab_channel=Codecademy

PROJECT II

https://www.youtube.com/watch?v=gfQaMSP1VVI&ab_channel=Codecademy

PROJECT III

- <https://www.codecademy.com/paths/data-analyst/tracks/dsaly-tableau/modules/dsaly-tableau-capstone-project/projects/tableau-wildlife-airplane-strikes>
- <https://public.tableau.com/app/profile/codecademy/viz/LearnTableauforDV-FAAwildlifestrikes/roulettedashboard?publish=yes>
- <https://www.youtube.com/watch?v=yTyBmRPMmRc>

EXCEL

Welcome to Learn Microsoft Excel for Data Analysis

A brief overview of what you will learn in this course.

What will you learn?

In this course, you will learn the basics of handling, analyzing, and visualizing data in Excel.

After this course, you will be able to:

- explore datasets using sorting and filtering
- transform datasets with functions and PivotTable
- select the right visualization to communicate your findings
- import and format data
- protect sheets to maintain data integrity.

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for peer reviews on your

project work and providing reviews to others in the [projects category](#), which can help to reinforce what you've learned.

What should you know before starting this course?

This course does not require any prerequisite knowledge in either Excel or data analysis!

What will you do?

Throughout this course, you will be applying the concepts you learn in real data analysis projects:

You'll explore GDP data to investigate the rise of the computing industry

You'll visualize hotel data to better understand reservation cancellation patterns

You'll import, prepare, and protect hotel data to make sure analyses are using valid, unaltered data

At the end of the course, you'll combine your analysis, visualization, and handling skills to analyze Bitcoin price data to compare potential upsides to volatility

We're excited for you to start your journey as a data analyst in Microsoft Excel!

Exploring Data

Introduction to Exploring Data

Excel lets us store, organize, and analyze our data all in the same place. This makes Excel a great one-stop-shop for exploring a dataset prior to performing a formal analysis. In this lesson, we'll cover three standard Excel methods for data exploration:

- calculating averages and other

- Preview: Docs Statistics is the science that is concerned with methods for collecting, organizing, analyzing, and interpreting data.

- [statistics](#)

- using **formulas**

- organizing data using **sorting**

- zooming in on the data we care about by **filtering**

Each exercise in this lesson will come with an interactive Excel spreadsheet for you to download. The spreadsheet will give you instructions to practice what you've learned, provide hints, and give feedback on your answers.

Even old hands at Excel sometimes get stuck on seemingly "simple" tasks, so if you get stuck, please consult the solutions or ask a question in the forums, even if you think you "should" be able to do it.

Introduction to Excel

Let's start by taking a tour of the Excel interface! When you first open up Excel, the sheer number of features can be a little overwhelming. But once you understand the structure behind Excel's layout, you can find the features you need and put the flexibility of Excel to use.

Excel is structured around two basic purposes:

Storing data and analyses of data

Performing analyses of data

Let's walk through the schematic of Excel in the learning environment to the right. At the bottom of the schematic are tabs corresponding to three **sheets** named `Sheet1`, `Sheet2`, and `Sheet3`.

The actual sheets appear above the tabs with the sheet names. The sheets are where Excel stores data tables and the results of data analysis, like charts or calculations. The data analysis tools themselves are stored above the sheets.

The **formula bar** is directly above the sheets. This is where you can view and edit any calculations you are using in your analysis.

The area above the formula bar displays different analytics options depending on which tab is open. In the schematic, the **Home** tab is open and the **Insert** and **Data** tabs are closed. Each icon on a tab is connected to some analytics function like sorting a table or creating a chart.

Depending on your version of Excel, icons may appear differently, but they should always be in the same location. We'll give you directions in each exercise to find the tools you need!

Summarizing Numeric Data

Let's start by exploring numeric data! In the learning environment, we've loaded a dataset containing the number of new cars produced in the US by year (from the [Bureau of Transportation Statistics](#)). Often, exploring numeric data starts with answering basic statistical questions about the data such as

- what is the largest number of new cars in a year?
- what is the smallest number of new cars in a year?
- what is the average number of new cars in a year?

Excel can help us answer all of these. But first, we'll have to know a little more about how cells in a spreadsheet connect to each other. The connections between cells are based on cell "addresses" or "references".

Cell addresses are made up of a `column letter` followed by a `row number`. For example, the cell containing the number 10 in the following image is named B2 since it is in column B and

row 2. The number 10 is the **content** or **value** of the cell.

	A	B	C
1			
2		10	
3			

In addition to numbers, cells can also contain **formulas**, which are how we'll calculate our maximum, minimum, and average. A formula in Excel always starts with the **=** sign, so Excel knows that it should do some kind of computation.

For example, since B2 contains the number 10, the formula `=2+B2` would produce the value 12, since $2+10 = 12$. The cell name B2 gets replaced by its value when Excel does the computation.

We can also do subtraction, division, and multiplication in formulas using

- for subtraction
- / for division
- * for multiplication

This last one might seem like an unusual symbol, but it is pretty commonly used for multiplication on computers.

Here's an example of using ***** for multiplication. What would you change if you wanted to multiply B2 by 3?

	A	B	C
1			
2		10	<code>=2*B2</code>
3			

C2

✕

✓

fx

`=2*B2`

	A	B	C
1			
2		10	20
3			
4			

Now, you might be thinking to yourself that you could multiply 2 by 10 without a spreadsheet. That's right! But the magic of Excel formulas is that they update automatically when data changes. If the cell B2 was changed to contain the number 4, the cell C2 would automatically

update to contain the number 8, since $2 \times 4 = 8$. These dynamic formulas are the real power of Excel since they let us automatically update analyses with new data.

Ranges of Cells

To calculate an average we need to reference a whole collection of cells instead of a single cell at a time. To reference a range of data, we type the name of the first cell in the range (usually the top-left cell), a colon, and then the name of the last cell (usually the bottom-right cell).

For example, the range B1:C4 includes the cells in Columns B-C and Rows 1-4.

	A	B	C
1	=B1:C4		
2			
3			
4			
5			
6			
7			
8			

To calculate the maximum, minimum, and average of this range we use special Excel functions =MAX(), =MIN(), and =AVERAGE(), and we put the range containing the data between the parentheses.

In the example in the learning environment, the data is contained in the range B2:B7, so the full formulas for calculating these three

Preview: Docs Loading link description

[statistics](#)

are

```
=MAX(B2:B7)
=MIN(B2:B7)
=AVERAGE(B2:B7)
```

To display the average in cell B8, we

Select cell B8

Type the formula into the cell

Select `return` or `enter`

You can watch the full process in the slideshow in the learning environment. When you are ready to practice this yourself, follow the next set of instructions!

Instructions

1. When you're ready to practice what you've learned, download and work through [our interactive exercise spreadsheet](#). A couple of important points to keep in mind:
We've changed some of the default Excel formatting to make these easier to follow and more interactive. You'll learn how to do formatting in later lessons. We've locked all cells except the ones we're asking you to create formulas in. This is so we can automatically correct your answers and give you feedback without worrying that some data has been changed accidentally. If you are having technical issues with the Excel spreadsheets, check out [this post in our forums](#) for possible solutions!
If you get stuck, feel free to [download our solutions spreadsheet](#)
When you're done with the exercises, come back here and answer the reflection question!

Reflection question: when you restricted the range for the average, the average increased. Why do you think that happened, and when would you prefer to use one over

Transform Data

The raw data we get often isn't in precisely the form we would like to analyze. For example, our new vehicle data contains the number of new vehicles for each type of vehicle, but what if we wanted to see each type as a percent of total new vehicles instead?

We could calculate percentages one cell at a time using individual formulas, but this would result in a lot of time writing formulas instead of analyzing our data. Also, if we ever decided to change our analysis slightly, we'd be stuck updating each formula individually again. Fortunately, Excel lets us move, or drag, formulas while automatically updating cell references.

Let's take a look at what this means. Suppose the **cell** **B2** contains the **formula** **=A4**. If we move this formula from **B2** to **D1**, we moved the formula two columns to the right and one row up. Excel then updated the formula from **=A4** to **=C3** following the same "path": two columns to the right and one row up. A way to think of it is that Excel moved **everything** two steps over and one step up.

Sometimes, we won't want the column or row to update. For example, let's take a look at calculating the percentages of the new vehicle data in the next table.

A

B

C

1	Vehicle type	# of new vehicles	Percentage
2	car	1000	
3	SUV	1500	
4	truck	5000	
5	TOTAL:	7500	

We can calculate the first percentage in **C** by dividing the number of new cars by the total number of new vehicles. In Excel, we would type the formula `= B2 / B5` in the cell **C2**.

	A	B	C
1	Vehicle type	# of new vehicles	Percentage
2	car	1000	=B2/B5
3	SUV	1500	
4	truck	5000	
5	TOTAL:	7500	

What happens when we move this formula to the cell **C3**? The column stayed the same, and the row moved one cell down. Excel will update the references in the formula `=B2/B5` to follow the same path:

B2 becomes **B3**
B5 becomes **B6**

Now the cell **C3** will have the formula `=B3/B6`.

	A	B	C
1	Vehicle type	# of new vehicles	Percentage
2	car	1000	=B2/B5
3	SUV	1500	=B3/B6
4	truck	5000	
5	TOTAL:	7500	

This isn't what we want to calculate! The formula `=B3/B6` takes the number of new SUVs in `B3` and divides it by whatever is in cell `B6`, not by the total which is in cell `B5`.

We want Excel to update the reference `B2` to `B3`, so that we're calculating the SUV percentage. But we don't want Excel to update the reference to `B5`, since the total stays the same no matter which vehicle type we're working with.

To stop Excel from updating this row, we need to place a `$` before the original row reference. The first formula in `C2` becomes `=B2/B$5`. When we move it down to `C3` it updates to `=B3/B$5`. This takes the number of new SUVs and divides it by the total, which is what we want to do!

	A	B	C
1	Vehicle type	# of new vehicles	Percentage
2	car	1000	=B2/B\$5
3	SUV	1500	=B3/B\$5
4	truck	5000	

5 TOTAL: 7500

Note that if we wanted to stop the column from updating instead of the row, we would need to put the \$ sign in front of the column letter in the formula instead of the row number.

Errors and Best Practices

Forgetting to use a \$ when necessary sometimes results in an Excel error. These errors are displayed in the cell containing the reference.

The two most common errors are #DIV/0! and #VALUE!. Let's look at how these could occur in our example, where Excel updated the percentage formula to =B3/B6, pointing to a cell B6 that isn't in our table.

The #DIV/0! error indicates that Excel is trying to divide by 0. This could happen because the cell B6 contains a 0 or, more likely, because the cell B6 is empty. Excel assumes that empty cells are the same as 0.

The #VALUE! error likely indicates that the cell B6 contains some non-numeric value that Excel cannot use in the mathematical formula =B3/B6.

Even if no errors appear, our cell references might still be updating in a way we don't want. If there was a nonzero number in cell B6, Excel would calculate =B3/B6 without an error. Unless we're careful, we could use the wrong number in our analysis without realizing it.

In general, it is always good practice to audit every formula we intend to move, looking at every cell referenced to ask:

- Should the row of this reference update when moved up/down?
- Should the column of this reference update when moved left/right?

The learning environment has a slideshow illustrating each step of creating and moving a formula in Excel. You can advance the slides using the controls at the bottom. Once you feel ready to do this yourself, move on to the next set of instructions!

Instructions

1. When you're ready to practice what you've learned, download and work through [our interactive exercise spreadsheet](#). A couple of important points to keep in mind:
 - We've changed some of the default Excel formatting to make these easier to follow and more interactive. You'll learn how to do formatting in later lessons.
 - We've locked all cells except the ones we're asking you to create formulas in. This is so we can automatically correct your answers and give you feedback without worrying that some data has been changed accidentally.
 - If you get stuck, feel free to [download our solutions spreadsheet](#) to compare your work to ours!
 - When you're done with the exercises, come back here and answer the reflection question!

Question: how do new cars produced in 2019 compare to the other varieties, given what you calculated in the range I27:I31?

Check Our Answer

New cars are at only 67% of their 2015 production level, while new vans are at 85% and all other vehicle types are over 100% of their 2015 levels.

We have to be careful, however, because we are comparing each type to its own 2015 baseline. For example, while pickup trucks in 2019 are at 140% of their 2015 level, there are still only about half as many pickup trucks being produced as there are cars.

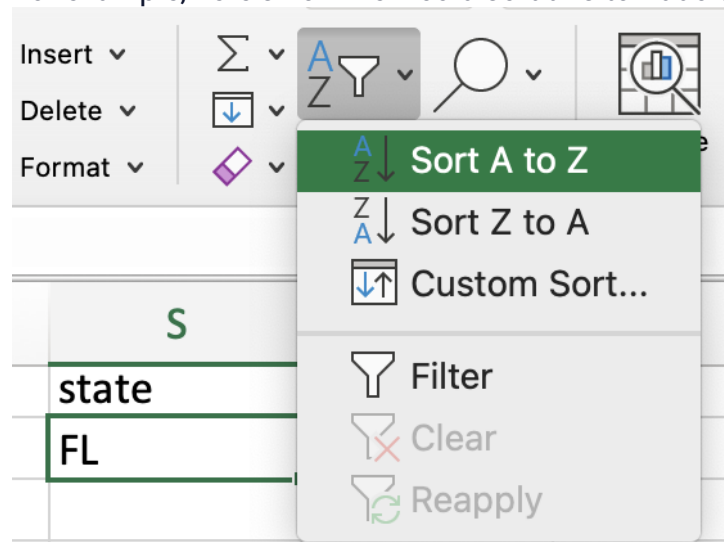
Sorting Data

Since Excel is often used to store and display data, we can scroll to explore, too. If the data is randomly arranged, it's hard to get any insight by just scrolling. But if the data is organized in a predictable way, we might be able to spot patterns that can guide our more formal analysis later on.

For example, look at the tornado data in the learning environment (from the [National Weather Service](#)). Notice that it is unsorted and difficult to make sense of. Once it is sorted, however, we can see all tornado measurements for each state together. This makes it much easier to visually inspect.

To sort a table of data in Excel by one of its columns, select a cell containing data for that column and then choose an order for sorting from the Sort & Filter menu on the Home tab.

For example, here's how we would sort this tornado data alphabetically by state:



This

Preview: Docs Loading link description
[method](#)

would result in a table starting with the tornados in Florida, since F comes before M and O alphabetically.

The option to sort **A to Z** or **Z to A** will only be displayed for text-based data. For numeric and time data, the options are **Sort Smallest to Largest**; or dates, **Sort Oldest to Newest** and vice versa. Sometimes, Excel might interpret the data type incorrectly and display the wrong option. We'll show you how to fix that in a later lesson.

Often, we want to sort by more than one column. For example, we might want to organize our tornado data so that all the tornados in each state are grouped together and only then sorted by date. The natural thing to try would be to sort by state and then sort by date. But, in case this wasn't already complicated enough, we actually need to use the reverse order.

This might seem unintuitive, but let's look at a small example to see why this happens. Consider the following table:

Date	State
1/4/20	FL
2/5/20	AL
1/31/20	FL
1/11/20	AL
2/6/20	FL

If we sort newest-to-oldest by date, we'll get the table rearranged by date:

Date	State
2/6/20	FL
2/5/20	AL

1/31/20	FL
---------	----

1/11/20	AL
---------	----

1/4/20	FL
--------	----

When we sort by state next, AL will come before FL. But which of the two AL rows will be first?

The way Excel deals with cells that have the same value is by maintaining the order they were already in. Since we've already sorted by date from newest to oldest, the 2/5/20 row for AL will come before the 1/11/20 row. Here's what the whole sorted table looks like:

Date	State
2/5/20	AL
1/11/20	AL
2/6/20	FL
1/31/20	FL
1/4/20	FL

In the learning environment, we've shown what happens if you sort a portion of this tornado dataset by time, and then by date, and then by state. When you're ready to try this yourself, move on to the next set of instructions.

Instructions

When you're ready to practice what you've learned, download and work through [our interactive exercise spreadsheet](#).

To allow you to sort freely, we've unlocked all cells of this spreadsheet. This is a great opportunity to practice Excel hygiene: do your best to only edit the cells indicated. If you think you might have accidentally erased some data, feel free to re-download the spreadsheet to start fresh!

If you get stuck, feel free to download [our solutions spreadsheet](#).

Once you've finished, think about the next question.

Question: could we use sorting to figure out how many tornados occurred on a given date?

Check Our Answer

Unfortunately, we can't. While sorting the data by date will group entries by date, we don't know for sure if different rows of this dataset correspond to different tornados. Even rows with different states or different times might correspond to the same tornado that has moved or grown. It is always important to not only think about the tools we have to analyze datasets, but also what limitations the dataset might impose.

Unsorted tornado data

date	time	state	length	width
1/10/20	10:34	OK	1.3	10
1/4/20	10:26	FL	4.39	265
1/10/20	14:27	MO	8.7	50
1/10/20	11:55	OK	1	50
1/4/20	9:29	FL	2.38	175
1/31/20	21:35	FL	4.68	50

1 - by time, early to late

date	time	state	length	width
1/4/20	9:29	FL	2.38	175
1/4/20	10:26	FL	4.39	265
1/10/20	10:34	OK	1.3	10
1/10/20	11:55	OK	1	50
1/10/20	14:27	MO	8.7	50
1/31/20	21:35	FL	4.68	50

2 - by date, new to old

date	time	state	length	width
1/31/20	21:35	FL	4.68	50
1/10/20	10:34	OK	1.3	10
1/10/20	11:55	OK	1	50
1/10/20	14:27	MO	8.7	50
1/4/20	9:29	FL	2.38	175
1/4/20	10:26	FL	4.39	265

3 - by state, A to Z

date	time	state	length	width
1/31/20	21:35	FL	4.68	50
1/4/20	9:29	FL	2.38	175
1/4/20	10:26	FL	4.39	265
1/10/20	14:27	MO	8.7	50
1/10/20	10:34	OK	1.3	10
1/10/20	11:55	OK	1	50

Filter Data

Sorting is excellent for organizing data, but finding specific data in a sorted table can still take a lot of scrolling. To answer questions about data with specific features, we're better off using **filters**.

Filtering a table consists of comparing each row of data to a list of criteria. Only those rows that match the criteria are included in the filtered table. For example, we could filter the tornado dataset by the following criteria:

state = AR

date is before 5/1/20

Applying this filter would result in a table of only those tornados in Arkansas before May 1st.

Sorting and filtering are often used together. For example, if we're interested in the length of tornados in Arkansas before May, we could take our filtered dataset and sort by length to see the longest and shortest tornados.

Let's look at an example of taking a question about data and breaking it down into filters. For example, let's imagine we've been asked to find out when the 2020 tornado season started in California (CA).

To break this question down, we start by identifying which columns of the dataset are being restricted. Since we're asked to find out *when* the tornado season *started*, this question will require some manipulation of the Date column. Since we're asked specifically about California, we'll also need to work with the State column.

Since the other columns aren't mentioned in the question, we won't need to filter or sort by them at all.

For the date, we're not asked to find a specific date, just to find the *start*, which will be the first date. This means we will want to sort the Date column from oldest to newest (further in the past to most recent).

For the state, we're given a specific value we want that column to have. For a tornado to be in California, it has to have CA in the state column.

To get the answer to this question we'll

filter by state = CA
sort Date from oldest to newest

The answer will be the date in the first row of this filtered and sorted table. We've put a walkthrough of applying these filters in the learning environment. Note that we're only showing a portion of the tornado data so you can see what happens. Follow along and see how the table changes with each step.

Instructions

When you're ready to practice what you've learned, download and work through [our interactive exercise spreadsheet](#).

To allow you to filter the data freely, we've unlocked all cells of this spreadsheet. This is a great opportunity to practice Excel hygiene: do your best to only edit the cells indicated. If you think you might have accidentally erased some data, feel free to re-download the spreadsheet to start fresh!

If you get stuck, feel free to download [our solutions spreadsheet](#) to compare your work with ours!

Once you've finished, think about the next question.

Question: What happens if you use a formula to calculate the average of a range, and then

apply a filter that removes most of the values in that range?

Check Our Answer

All a filter does is *hide* values that don't match the filter criteria. The formula for computing the average is still referencing the entire range, and so will continue to calculate the same number as before.

Let's Review

In this lesson, you've learned how to explore datasets in Excel using three key methods:

- formulas** to summarize and transform data

- sorting** to organize data for visual inspection

- filtering** to zoom in on data points with specified features

Specifically, you've learned how to

- use built-in Excel functions to find out the average, maximum, and sum of a set of numbers

- use draggable cell references and formulas to determine percents and baseline comparisons from raw data

- use sorting on multiple columns to gain insight into trends over time

- use filtering to answer questions about datapoints matching specific criteria

Introduction to Pivot Tables

Formulas, filtering, and sorting are powerful tools! But combining them to answer complex data questions can be tricky. Excel solves this problem for many common data questions through the `PivotTable` tool.

In this lesson you'll learn how to take a question about data and

- Determine if a pivot table is appropriate

- Translate the question into pivot table settings

- Create the pivot table in Excel

- Update the pivot table with new data

What is a Pivot Table

Pivot tables won't help you get your furniture through a door (unfortunately). They will help you *group* and *summarize* your data. Let's look at an example of how powerful this can be.

Suppose we're trying to calculate the total dollars lost to tornados in each state by month. Let's use the small example table below.

	Year	Month	Losses (\$)	Length (mi)	State
1	2020	March	5,000	.4	TX
2	2020	February	10,000	1.5	FL
3	2020	January	50,000	.8	AZ
4	2020	February	25,000	.9	FL
5	2020	January	30,000	1	AZ
6	2020	January	1,000	1.2	TX

We could answer this question by hand in three steps:

- Group together all rows with the same state and month.

- Collect the `Losses` values for all the rows in each group.

- Sum the `Losses` for each group to report the total losses by group.

A **pivot table** is a separate table collecting the results of this process. That might sound a bit abstract, so let's go through this step-by-step.

We start by creating a table of all the possible `Month` and `State` groups.

First, we use the values of the `State` column as column headers for the pivot table. Note that we don't repeat values. Even though `AZ` appears twice in the state column, it only appears once in the headers of our new table.

AZ

FL

TX

Second, we use the values of the `Month` column to label the rows of the new table. Once again, each month will only appear once.

	AZ	FL	TX
January			
February			
March			

The cells of this table correspond to the *groups* we need to form to answer the original question. Now, we need to gather and summarize the values in each group to fill in the corresponding cell.

For example, the upper-left cell of the new table is in the row `January` and the column `AZ`. When we look back at the original table, there are two rows in this group. To report the total losses, we collect the `Losses` values for those rows and add them up: $50,000 + 30,000 = 80,000$.

	AZ	FL	TX
January	80,000		
February			
March			

If we move one cell down, we'll be in the row `February` and column `AZ`. There are no rows of the original table corresponding to this group. We leave this cell blank, to tell someone reading the table that there were no rows of data in this group.

Here's the final pivot table — do you agree with the rest of our calculations?

	AZ	FL	TX
January	80,000		1,000
February		35,000	
March			5,000

This process is the same for any pivot table. Starting with a table of data, you pick

column labels: a column or columns of the original table to serve as column labels (or headers) for the pivot table

row labels: a column or columns of the original table to serve as row labels for the pivot table

cell values: a column or columns of the original table to provide the values for the cells of the pivot table

summary method: a

Preview: Docs Loading link description

[method](#)

for taking multiple values corresponding to a single group and producing one value for the pivot table

In our initial example, we picked

column labels: `State`

row labels: `Month`

cell values: `Losses`

summary method: `sum` (or total)

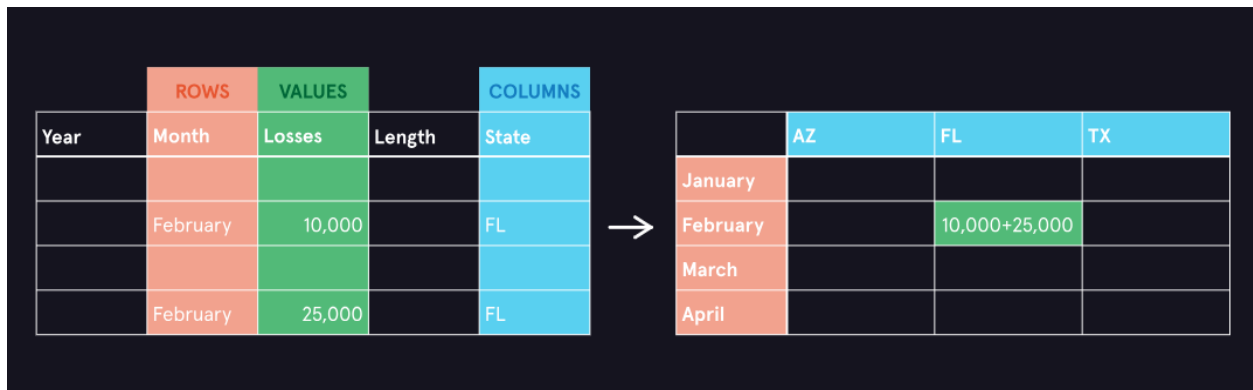
Instructions

1. You have a table of data from a pet adoption agency. The columns in the table are `Species`, `Age`, `Weight`, and `Date adopted`. The adoption agency has asked you to calculate the average weight of a pet by species and age. How would you set up a pivot table to answer this question? That is, what would you pick for

column labels:

row labels:

cell values:
summary method



	ROWS	VALUES		COLUMNS
Year	Month	Losses	Length	State
	February	10,000		FL
	February	25,000		FL

	AZ	FL	TX
January			
February		10,000+25,000	
March			
April			

Updating Pivot Tables

Unlike formulas, pivot tables in Excel do not update automatically if the original table changes. There are two methods for updating a pivot table manually, depending on how the original table has changed:

Refresh: if the size and location of the source dataset have not changed, select **Refresh** from the **PivotTable Analyze** tab.

Change data source: if the size and/or location of the source dataset have changed, select **Change Data Source** from the **PivotTable Analyze** tab and select the new source range.

Note that the **PivotTable Analyze** tab will only appear if you have selected a cell within the pivot table.

We've placed a slideshow illustrating both cases in the Learning Environment. When you're ready to try this yourself, get started on the next set of instructions!

Introduction to Visualizing Data

One of the most effective ways to identify and communicate trends in data is through **data visualization**. In Excel, we can create a wide variety of visualizations in the same sheets we've been using for data storage and analysis. In this lesson, you'll learn to visualize data by

using **conditional formatting** to visually explore the values in tabular data
creating **column charts** and **pie charts** to compare the sizes of categories
creating **histograms** and **scatterplots** to visualize numeric data
creating **line charts** and **sparklines** to identify trends in data
using Excel's **recommended chart type** feature to easily explore different possible visualizations

Each exercise will come with an Excel spreadsheet for you to download. The spreadsheet will give you instructions to practice what you've learned, explore customization options, and check your answers.

Conditional Formatting

Often, we think of data visualization as being all about charts and

Preview: Docs Loading link description

graphs

. In Excel, data visualization starts with tools to help us explore the data tables themselves! For example, let's take a look at a table of average gas mileage (mpg) for different types of vehicles over time (from the [Bureau of Transportation Statistics](#)).

	A	B	C	D
1	Year	Car	Car SUV	Pickup truck
2	2010	26.2	23.0	16.9
3	2011	25.8	23.5	17.2
4	2012	27.6	23.3	17.2
5	2013	28.4	24.3	17.5
6	2014	28.4	24.4	18.0
7	2015	29.0	25.1	18.8

What if we wanted to see at a glance whether gas mileage was improving over time, or which years met a certain threshold for efficiency? We'll look at two techniques: color scales and cell rules.

Color scales

In many cases, we want to see which rows and columns of a table have larger or smaller values. For example, we might want to see if vehicles have tended to get more efficient over time or identify the most and least efficient vehicles in the MPG dataset. A way to solve this using data visualization is by creating a **heatmap**, which colors each cell in the data table depending on its value.

In the heatmap below, for example, the smallest values are colored dark red, the highest values are dark green, and the values in-between are shaded based on where they fall in that spectrum (with yellow in the middle).

The car column is all shades of green, showing that they are more efficient than the other columns (yellow for car SUVs and red for trucks). Within the car column, the green is getting darker over time, indicating that cars have continually improved in average MPG!

	A	B	C	D
1	Year	Car	Car SUV	Pickup truck
2	2010	26.2	23.0	16.9
3	2011	25.8	23.5	17.2
4	2012	27.6	23.3	17.2
5	2013	28.4	24.3	17.5
6	2014	28.4	24.4	18.0
7	2015	29.0	25.1	18.8

We've placed a slideshow in the Learning Environment illustrating how to apply a heatmap in Excel using conditional formatting!

Cell rules

Sometimes, we have more specific questions about the values in our data. For example, we might want to see if every value in a table meets a particular threshold or not.

In our dataset, let's assume that a minimum of 26 mpg is considered "good" gas mileage for a vehicle. We can visualize this by coloring cells that meet this criteria green and all other cells red.

The result would look something like this:

In Excel, we can do this by defining custom rules to conditionally format the cells. The full process is illustrated in the slideshow in the learning environment.

Instructions

1. When you're ready to practice what you've learned, download and work through our [exercise spreadsheet](#). A couple of important points to keep in mind:

Unlike formulas and pivot tables, we can't automatically assess your visualizations. We have placed solutions to each exercise within the same spreadsheet. Feel free to compare your work to ours if you get stuck or to check if your solution is correct. Your visualization may not look identical to ours and that's okay!

There are lots of options we haven't covered — feel free to play around and see how different options work!

You can always re-download the spreadsheet if you want to start fresh.

Once you've finished, think about the next question.

Question: What do you notice about the first "good" years for each vehicle type?

Column and Pie Charts

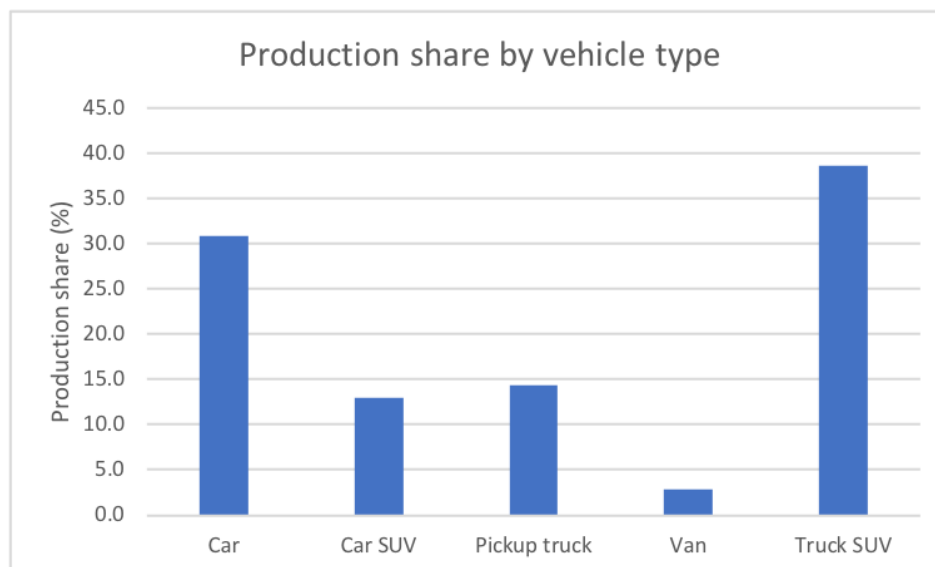
A common task in data visualization is to compare the sizes of categories. For example, in our vehicle data we might want to compare how many cars are being produced as opposed to SUVs or trucks. **Column charts** and **pie charts** are simple ways to visualize categorical data and make comparisons.

For this exercise, we'll look at a dataset containing data on the production shares (%) for different types of vehicles in 2020 (from the [Bureau of Transportation Statistics](#)).

	A	B
1	Vehicle type	Percent
2	Car	30.9
3	Car SUV	13.0
4	Pickup truck	14.4
5	Van	2.9
6	Truck SUV	38.7

Column charts

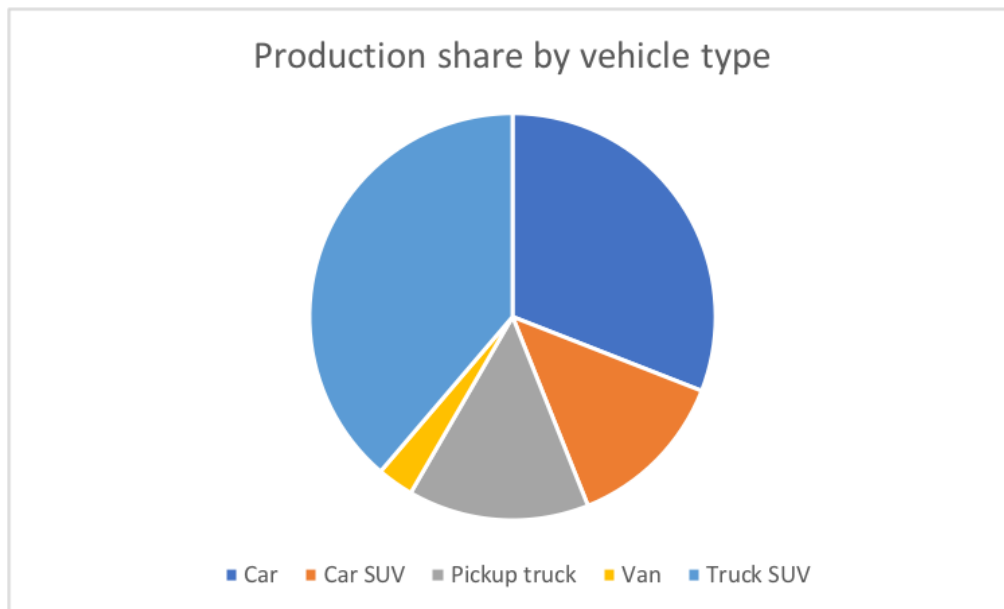
In column charts, each category is represented by a column in the plot and we can compare them by assessing the length of the columns or bars. Note that these are commonly also called bar charts, but the tool in Excel calls them column charts.



Pie Charts

In pie charts, each category is viewed as a “slice” or `sector` of a pie, with the size (or area) of the slice corresponding to the relative size of that category. Because the categories as slices make up the whole pie, pie charts should *only* be used to visualize categories that are pieces of some whole. In other words, the area of the sectors (slices) should add up to 100%.

In our dataset, the different vehicle categories are all part of the same whole: the collection of all vehicles in 2020.



Let’s look at a related dataset that might not be very suitable for pie charts. This table shows the production share of cars from 2015 to 2020:

	A	B
1	Year	Percent
2	2015	47.2
3	2016	43.8
4	2017	41
5	2018	36.7
6	2019	32.7
7	2020	30.9

While we can certainly plot a pie chart, it would be misleading, because these percents are not pieces of the same whole. They are each “slices” of their own year, and shouldn’t be

placed together in a pie. Make sure to evaluate your data to see if a pie chart is a right choice for visualization!

We've placed a slideshow illustrating how to create both types of charts in the learning environment. When you're ready to practice this yourself, move on to the next set of instructions!

Instructions

1. When you're ready to practice what you've learned, download and work through our [exercise spreadsheet](#). A couple of important points to keep in mind:

Unlike formulas and pivot tables, we can't automatically assess your visualizations. We have placed solutions to each exercise within the same spreadsheet. Feel free to compare your work to ours if you get stuck or to check if your solution is correct. Your visualization may not look identical to ours and that's okay!

There are lots of options we haven't covered — feel free to play around and see how different options work!

You can always re-download the spreadsheet if you want to start fresh.

Once you've finished, think about the next question.

Question: The column chart and pie chart you've created reflect the same data. What is the advantage of using each type of visualization?

Histograms and Scatterplots

Column charts and pie charts are excellent for comparing the sizes of categories. But they won't help us understand more general numeric columns. For example, suppose we wanted to use the following table of [housing price data](#) (in thousands of US dollars) to analyze the impact of wealth inequality on housing.

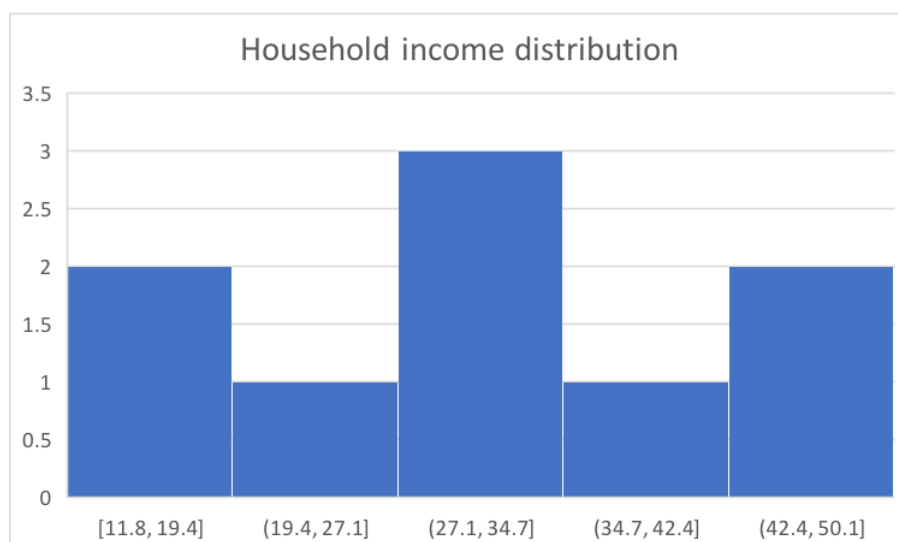
	A	B
1	Income	Housing price
2	40.9	2489.0
3	20.8	975.0
4	30.9	1399.0
5	28.8	875.0
6	15.2	518.0
7	50.1	2320.0
8	44.8	1907.0
9	11.8	561.0
10	31.6	3667.0

We'll look at two methods for visualizing numeric data in this exercise: histograms and scatterplots. We'll use histograms to visualize one column at a time, and scatterplots to understand the connection between the two columns.

Histogram

Histograms are used to understand the “shape” of a single column of numeric data. A histogram breaks the data-points into **bins** and then counts the number of data points in each bin.

To better understand what a histogram shows, let's visualize the median household income data for just the first 9 rows of data shown above:

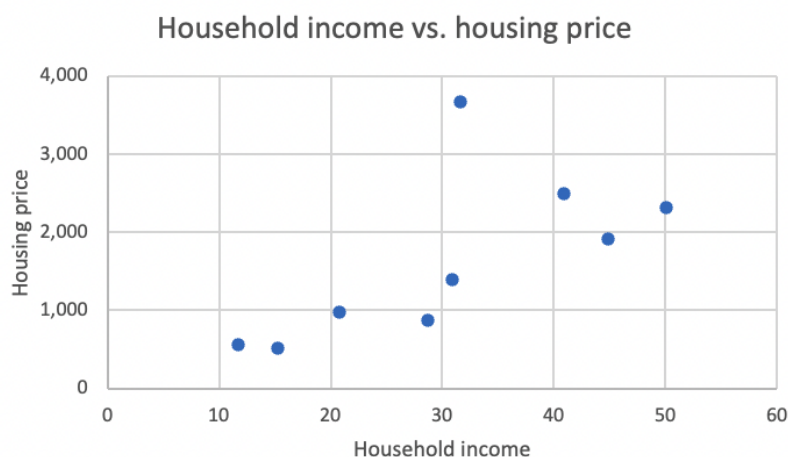


The smallest and largest household incomes are 11.8 and 50.1. To create this chart, Excel broke that range into five equal pieces. The range of values for each piece is labelled along the x-axis. The column above each piece represents the number of data points that fall within that range. For example, the largest column has a height of 3, indicating that there are 3 data points in the table between 27.1 and 34.7 (not including 27.1). Check the table above to confirm that Excel got that right!

Scatterplots

Scatterplots are used for visualizing **correlation** between two columns. As one column increases, does the other also increase? Decrease? Fluctuate? For example, in our dataset, we might want to know if higher-income households own more expensive houses: as income increases, does housing price also increase? It's important to remember that the existence of a correlation like this does not mean the one column is directly influencing the other.

Here is what the scatterplot looks like using the first 9 rows of data shown above, with household income on the x-axis and housing price on the y-axis:



To create this chart, Excel has placed a dot for each row of the table. The dot is positioned horizontally by the income value in that row and vertically by the housing price for that row. For example, the first dot in the bottom left corresponds to the ninth row with an income of 11.8 and housing price of 561.

We can see a general trend from this plot: as incomes increase (moving to the right in the plot), so do housing prices (moving up in the plot.) We refer to this as a positive correlation when the

Preview: Docs Loading link description

[variables](#)

trend in the same direction. If one variable increases while another decreases, this is referred to as a negative correlation.

We've placed a slideshow illustrating how to create these plots in Excel! When you're ready to try it yourself, move on to the next set of instructions.

Instructions

1. When you're ready to practice what you've learned, download and work through our [exercise spreadsheet](#). A couple of important points to keep in mind:

Unlike formulas and pivot tables, we can't automatically assess your visualizations. We have placed solutions to each exercise within the same spreadsheet. Feel free to compare your work to ours if you get stuck or to check if your solution is correct. Your visualization may not look identical to ours and that's okay!

There are lots of options we haven't covered — feel free to play around and see how different options work!

You can always re-download the spreadsheet if you want to start fresh.

Once you've finished, think about the next question.

Question: What can you say about the general shape of the household income histogram? What about the relationship between household income and housing price, and where the points are more densely scattered?

Line Charts and Sparklines

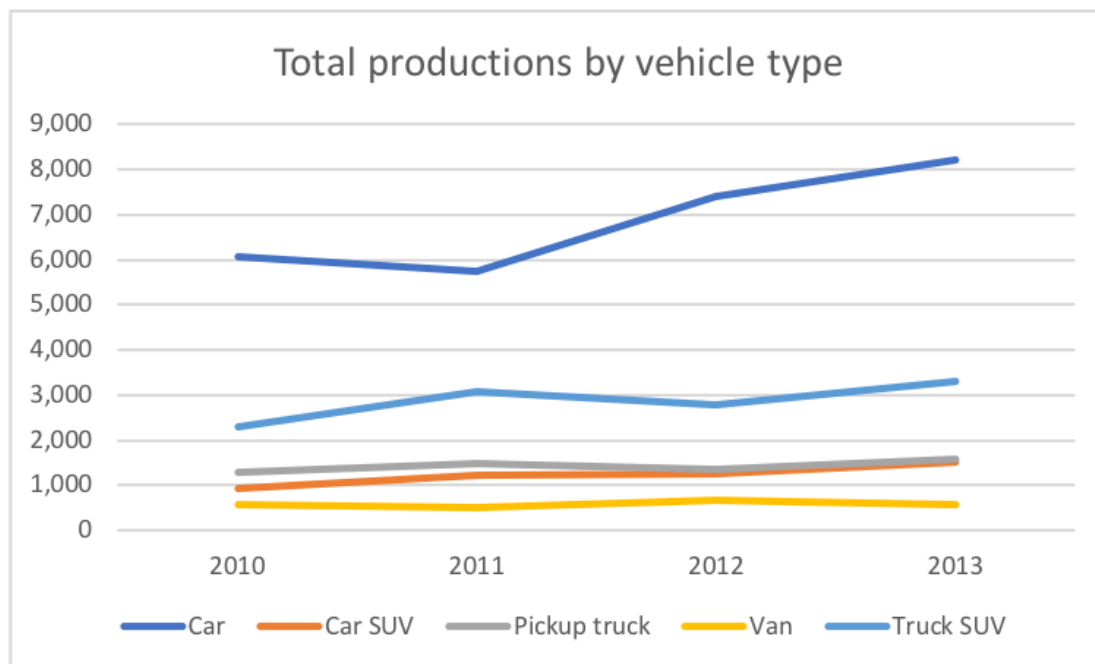
The last two visualizations we'll look at are charts that help us see the change in a numeric column over time.

For this exercise, we'll look at a dataset containing data on the total production (thousands of vehicles) for different types of vehicles over time (from the [Bureau of Transportation Statistics](#)). Here are the first few columns of this dataset:

	A	B	C	D	E
1		2010	2011	2012	2013
2	Car	6061	5743	7393	8226
3	Car SUV	915	1207	1265	1514
4	Pickup truck	1276	1479	1357	1577
5	Van	559	521	662	571
6	Truck SUV	2305	3069	2771	3310

Line charts

Line charts plot points in the data connected with a line to visualize the trend. Multiple lines can be plotted on the same graph to compare trends between groups. For example, here is a line chart of the production numbers above:



It looks like most vehicle types increased production over these four years, but Vans stayed fairly constant in comparison to the rest.

Sparklines

Sparklines are a feature in Excel that places a trend line (like a mini line chart) next to each row of a table. For example, here is what the sparklines would look like for our vehicle production data:

	A	B	C	D	E	F
1		2010	2011	2012	2013	
2	Car	6061	5743	7393	8226	
3	Car SUV	915	1207	1265	1514	
4	Pickup truck	1276	1479	1357	1577	
5	Van	559	521	662	571	
6	Truck SUV	2305	3069	2771	3310	

Sparklines can be helpful when we want to see general trends for individual categories. In the line chart, the fluctuations in Van production weren't visible, since they were so small compared to the fluctuations for cars. In the sparkline, we can see them much more clearly.

We've placed a slideshow illustrating both methods in the learning environment. When you're ready to practice this yourself, move on to the next set of instructions!

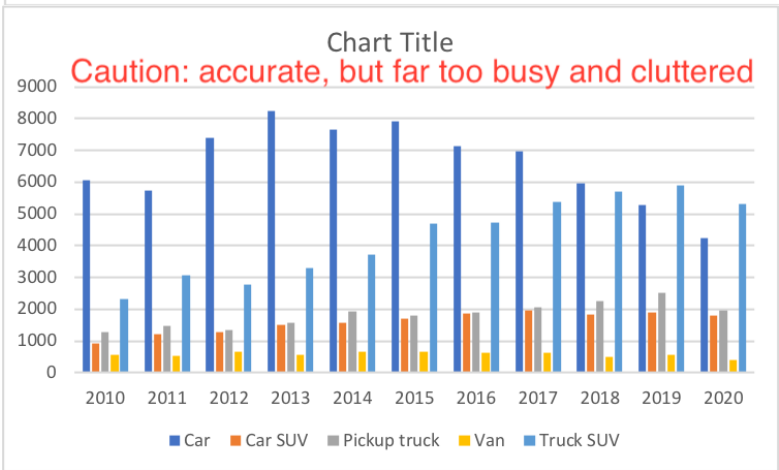
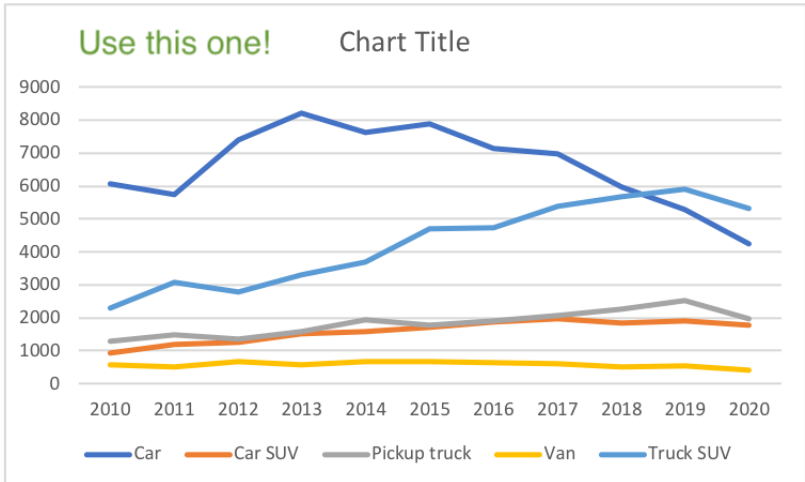
Instructions

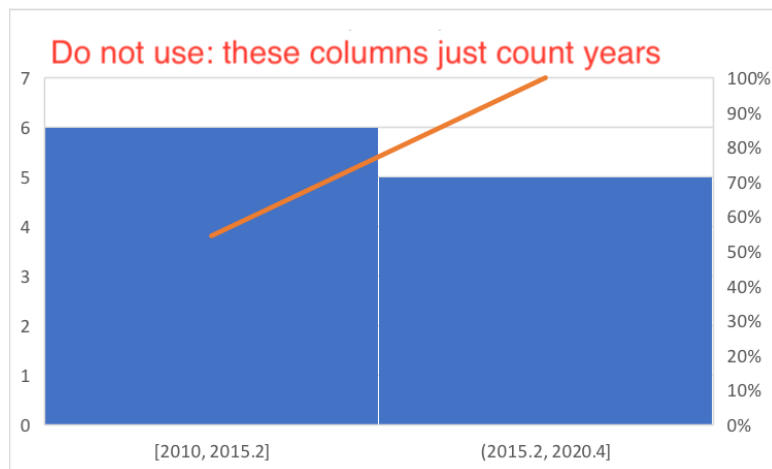
- When you're ready to practice what you've learned, download and work through our [exercise spreadsheet](#). A couple of important points to keep in mind:
 - Unlike formulas and pivot tables, we can't automatically assess your visualizations. We have placed solutions to each exercise within the same spreadsheet. Feel free to compare your work to ours if you get stuck or to check if your solution is correct. Your visualization may not look identical to ours and that's okay!
 - There are lots of options we haven't covered — feel free to play around and see how different options work!

You can always re-download the spreadsheet if you want to start fresh.

Once you’ve finished, think about the next question.

Question: How did the total production of the different vehicle types trend over the time period from 2010 to 2020?





Introduction to Handling Data

Even the most advanced analytics techniques are only as good as the data being used and the clarity with which results are presented. So handling your data properly is one of the most important skills you can learn in data analysis.

In this lesson, you'll learn how to prepare for analysis by

- importing external datasets from
Preview: Docs Loading link description
[CSV](#)
files
- cleaning and validating data prior to analysis

You'll also learn how to present your analysis by

- formatting tables for human readability
- protecting data when sharing spreadsheets with others

Importing Data

Many datasets are stored in formats other than Excel's **XLSX** format. One of the most common storage formats used across the data science world is the

Preview: Docs Loading link description

CSV

or **comma-separated values** format.

A CSV file is a text file containing data where the columns of data are separated by commas. Here's a sample of what a CSV file might look like if you open it up in a text editor:

```
month,sales, costs
Jan,1000,500
Feb,2000,800
```

Copy to Clipboard

Here's what this same CSV file looks like interpreted as a table:

month	sales	cost
Jan	1000	500
Feb	2000	800

Other Delimiters

CSV files typically use commas as **delimiters**, the symbol that indicates where one column ends and another begins. However, in some cases this doesn't work very well. For example, in some countries the `.` is used as a decimal marker. In these countries, using a `,` as a delimiter will cause problems, since Excel can't tell the difference between `,` as a decimal and `,` as a delimiter. In cases like this, it's better to use another symbol, like the semi-colon `;`, as a delimiter. Even with a different delimiter, the format is still called CSV.

Excel provides an import tool that lets us select which symbol is being used as a delimiter. We've placed a slideshow illustrating this process in the learning environment. Note that some of the buttons and menus might look slightly different depending on your operating system and version of Excel.

When you're ready to practice this yourself, get started on the instructions below!

Instructions

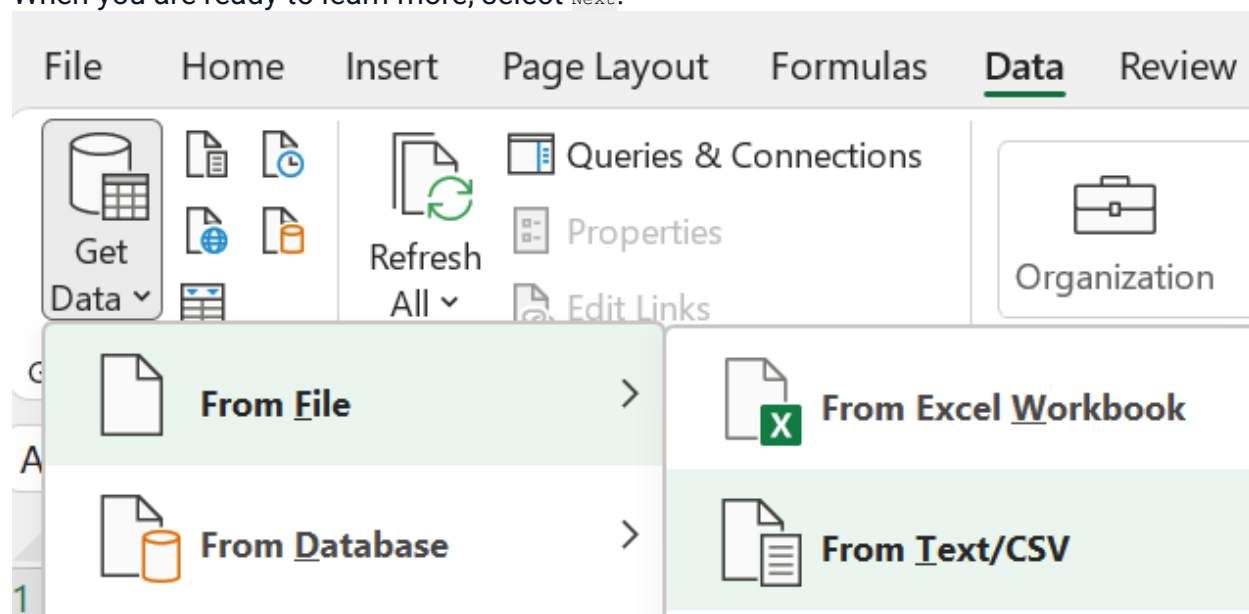
When you're ready to practice what you've learned, download and work through [our exercise spreadsheet](#).

Note that for this exercise, you will also need to download a CSV file. We've provided two:

if your computer uses `.` for decimals, [download this comma-delimited file](#)

if your computer uses `,` for decimals, [download this semicolon-delimited file](#)

Feel free to download our [solutions spreadsheet](#) to compare your work to ours! Note that you might get an `External data connections have been disabled` warning. When we imported the CSV, that established a connection between Excel and the CSV file. You can ignore this warning. When you are ready to learn more, select `Next!`



Cleaning Text Data

After importing a data file, we're often eager to get started with exploratory data analysis. Unfortunately, raw data is often messy data. If we don't take some time to inspect and clean a dataset, we may get meaningless or misleading results.

For example, let's take a look at this table of monthly sales:

	A	B	C
1	month	sales	
2	jan	1000	
3	jan	100	
4	february	50	
5	february	30	
6	april	30	
7	march	22	
8	mar	1000	
9	january	500	
10			

Suppose we want to know the sales associated with `january`. If we just dive in and filter the table on `month equals january`, we'll get a value of 500.

	A	B
1	month 	sales 
4	january	500

But there are several other rows of the original table that ought to be associated with `january`! Unfortunately, Excel can't tell that `jan` and `january` really mean the same thing. Even harder to spot is the fact that one of the rows labeled `jan` has three

spaces at the end. We can't see this extra whitespace easily, but Excel will treat it as a different category.

In a small dataset, these inconsistencies can be edited by hand. For larger datasets, Excel provides text cleaning functions that we can write once and then drag down a column like any other formula. You can even nest these functions (like in the last slide to the right), though nesting is outside the

Preview: Docs Loading link description

[scope](#)

of this course.

We've placed a slideshow demonstrating common text-cleaning functions in the learning environment. When you're ready to try these out yourself, move on to the next set of instructions!

Instructions

When you're ready to practice what you've learned, download and work through [our interactive exercise spreadsheet](#). Feel free to download our [solutions spreadsheet](#) to compare your work to ours!

Reflection Question: in the spreadsheet, why did we trim the data first?

	A	B
1	raw data	cleaned data
2	january	jan
3	JAN	jan
4	jan	jan
5	JAN	=LOWER(TRIM(A5))
6		

Inspecting Numeric Data

Before moving on to analysis, we also need to inspect and validate numeric data. There's no one rule for what valid numeric data looks like, since the definition of "invalid" data depends on what we expect "valid" data to look like. For example, an entry of `-500` might be suspicious in a column of Celsius temperatures but reasonable in a column measuring the sea floor relative to sea level.

While there is no one rule for cleaning data, there are some common questions that can help guide our inspections:

Are there any blank or missing data points?

What range of numbers (or dates) are we expecting?
Are there any data points that don't meet those expectations?

We've already learned the tools we need to answer these questions! For example:

missing data points: use the `Filter` menu and select `(Blanks)`

unexpected values: use sorting or functions to inspect minimum and maximum values or trends over time

Of course, identifying invalid or messy data is only the first half of the process. Once we identify these datapoints we have to decide what to do with them. There is a whole science to handling missing data, so be sure to check out our course on [Handling Missing Data](#) before working with missing data.

For this course, imagine that you were asked to review the datasets to the right. You've opened them in Excel and are inspecting the data. What would you do in each case? Think critically about if you can (or should) use the data and what types of analysis would be valid, and what wouldn't.

Flip the cards to read our answer.

When you're ready to practice inspecting data for missing or suspect entries, move on to the next set of instructions!

Instructions

When you're ready to practice what you've learned, download and work through [our interactive exercise spreadsheet](#).

Even if you get an exercise correct on the first try, select the other possible answers to see our explanations of why they are incorrect.

If you get stuck, feel free to [download our solutions spreadsheet](#) to compare your work to ours!

Reflection question: If there are no missing or suspect entries, are we good to proceed with analysis?

Missing data	
Temperature (F)	Date
10	1/1/20
12	1/2/2020
	1/3/2020
9	1/4/2020

Suspect data	
Temperature (F)	Date
10	1/1/20
12	1/2/2020
50	1/3/2020
9	1/4/2020

Missing Data	
Temperature (F)	Date
	1/1/20
	1/2/2020
	1/3/2020
9	

Suspect Data	
Temperature (F)	Date
10	1/1/2072
12	1/2/2072
10	1/3/2072
9	1/4/2072

Formatting Data

Because Excel is used to visually explore datasets and present those datasets for others, we often want to change the way tables are displayed to improve human readability.

We've loaded a table of data in the first slide in the learning environment. This table is perfectly accurate, but it isn't very easily readable. We want to present tables that provide insight, not tables that provide headaches!

We can solve this problem using Excel's formatting tools. These tools change the way data is displayed without changing any of the underlying data to maintain data integrity. Play through the slideshow in the learning environment to see how we can reformat this data using Excel. What do you think of the end result?

We've emphasized that formatting doesn't change the value of a cell. Let's look at an example of why this is important to remember.

In the Excel table below, the cell **B3** is adding the values in **B1** and **B2**.

A

B

1		1.1
2		1.4
3	sum:	2.5

If we format these numbers to have no decimal places, the table will be displayed like this:

	A	B
1		1
2		1
3	sum:	3

It seems as if Excel is calculating $1+1=3$! But this is just a consequence of the rounding, the underlying summation is still $1.1+1.4 = 2.5$.

When you're ready to practice formatting for yourself, get started on the next set of instructions!

Instructions

When you're ready to practice what you've learned, download and work through [our interactive exercise spreadsheet](#). Note that our solutions are contained in the same file for this exercise!

Reflection question: what would happen if a cell was formatted as a number and then you type a date into it?

Protecting Data

Okay, so we've explored a dataset and have some preliminary results to share with a team member. But our team member doesn't use Excel much, and we don't want them to accidentally overwrite a crucial formula.

Excel provides built-in tools for protecting different aspects of a spreadsheet. It is important to note that none of these will stop a malicious actor from accessing or altering data. All

anyone has to do to circumvent protected sheets and cells is to open the file in another program like Google Sheets.

However, protected and hidden sheets are still useful for preventing accidental data overwrites by ourselves or by others, controlling input to a spreadsheet, and cleaning up a spreadsheet for presentation.

Three common options for protecting and hiding data in Excel are

- Protecting an entire sheet from editing.
- Unprotecting specific cells to allow user input or data updates.
- Hiding sheets to keep raw data and background calculations out of the way.

We've placed a slideshow in the Learning Environment illustrating how to do each of these in Excel.

Best practices for protecting and hiding ranges

Protected and hidden ranges can be very helpful. They can also be very frustrating. Pretty much every Excel user has a story of hours lost hunting for hidden data in a large spreadsheet. We recommend the following practices for avoiding these frustrations:

- Give hidden sheets meaningful names. Nobody wants to spend hours hunting for data hidden on `Sheet45321`.

- Highlight any unlocked cells in an otherwise protected sheet. Highlighting unlocked cells will both warn users that those cells can be overwritten and will also direct them to the cells they are supposed to change. For example, this is why we included "input your answer here" messages in the exercise spreadsheets for this course!

- Establish norms for spreadsheet hygiene with your team. If everyone follows the same conventions for protecting data, it will be easy to open and use each other's spreadsheets!

Instructions

When you're ready to practice what you've learned, download and work through [our interactive exercise spreadsheet](#). Note that our solutions are in the same spreadsheet!

Once you're done, select `Next` to keep learning!

Let's Review

Congratulations! You're now ready to analyze data from start to finish in Excel. You've learned how to prepare for analysis by:

- importing raw data into Excel from CSVs
- cleaning inconsistent text data
- inspecting numeric data for missing or potentially invalid entries

After performing your analysis, you've learned how to

- make tables human-readable with custom formatting
- protect data from being accidentally overwritten
- clean up the presentation of a file by hiding unneeded sheets

Virtual Environment

Pipenv

- <https://www.codecademy.com/paths/data-science/tracks/dsml-python-fundamentals-part-iii/modules/dsdp-python-modules/videos/learn-python3-pipenv>
- https://www.youtube.com/watch?v=BVr-6Ki96XM&ab_channel=Codecademy

SETUP VIRTUAL ENRONMENT PYTHON - PIPENV ON WINDOWS PC

```
> cmd
> cd dir_path #setup directory path
> pip --version
> pip install --user pipenv #install in user level, not global
> pipenv
= warning? The scripts pipenv-resolver.exe and pipenv.exe are installed in 'dir_path/Python/Python37/Scripts' which
is not on PATH.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.

--> copy this: dir_path/Python/Python37/Scripts
    then go to > windows > type environment > edit the system environment variables > environment variables >
path > edit > new > paste > save>save>save

> pipenv --version
```

It is DONE

NOW OPEN ANY editor - PyCharm, Visual Studio, ATOM, etc. and start working in
Create or go to directory

```
> cd dir_path
> pipenv --three #python 3
> pipenv install numpy #ny version of numpy --> numpy = "*" but install the latest version of module
> pipenv install requests==2.18.1
```

https://www.youtube.com/watch?time_continue=391&v=BVr-6Ki96XM&embeds_referring_euri=https%3A%2F%2Fwww.codecademy.com%2F&embeds_referring_origin=https%3A%2F%2Fwww.codecademy.com&source_ve_path=Mjq2NjY

SETUP VIRTUAL ENRONMENT PYTHON - PIPENV ON MAC PC

```
> cmd #command line
> pip --version
> pip install --user pipenv
```

/* run pipenv -> if says like: ...C:/../bin which is not on PATH. consider adding directory PATH

```
If >
> pipenv > command not found
```

```
Then >  
> vi ~/.bash_profile > enter button
```

```
ADD TO open Notebook  
export PATH="/Users/file_directory/Library/3.7/bin:$PATH"
```

```
AT the end > write
```

```
> :wq! > enter_button
```

```
> more ~/.bash_profile
```

```
RESTART CMD
```

```
>pipenv --version
```

EXERCICES I

- <https://www.codecademy.com/paths/data-analyst/tracks/dsaly-excel/modules/dsaly-exploring-data-in-excel/projects/explore-gdp-in-excel>
- <https://www.codecademy.com/paths/data-analyst/tracks/dsaly-excel/modules/dsaly-visualizing-data-in-excel/projects/visualizing-hotel-data-in-excel>

- <https://www.codecademy.com/paths/data-analyst/tracks/dsaly-excel/modules/dsaly-handling-data-in-excel/projects/handling-hotel-data-in-excel>
- <https://www.codecademy.com/paths/data-analyst/tracks/dsaly-excel/modules/dsaly-capstone-project-bitcoin-prices-in-excel/projects/capstone-project-bitcoin-prices-in-excel>

Done11 / 11 done

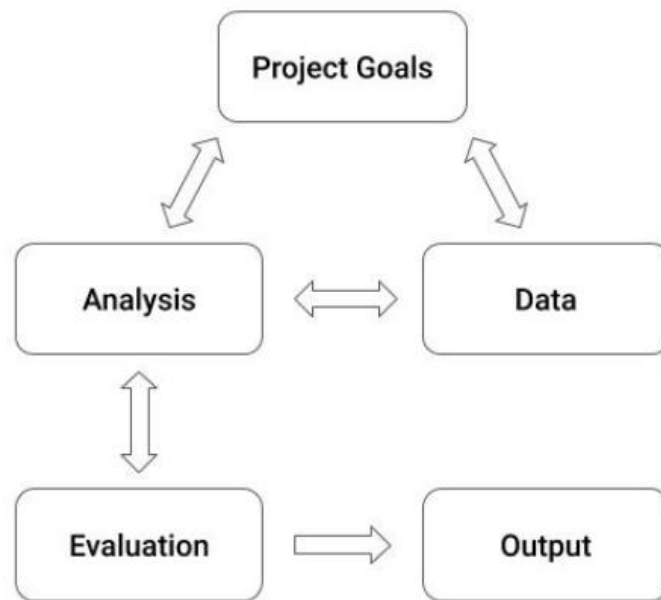
Setting up your Git Repository

Create a new Git repository for this project.

Project Scoping

Properly scoping your project will greatly benefit you; scoping creates structure while requiring you to think through your entire project before you begin. You should start with stating the goals for your project, then gathering the data, and considering the analytical steps required. A proper project scope can be a great road map for your project, but keep in mind that some downstream tasks may become dead ends which will require adjustment to the

scope.



ABC

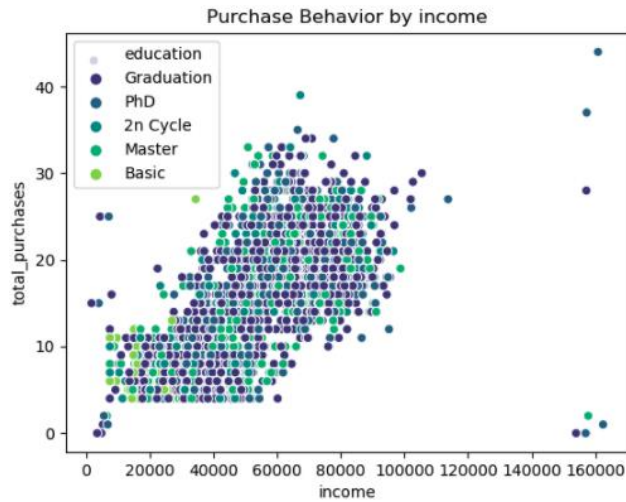
Your data analytics portfolio is your secret weapon when applying for potential positions. It should not only show off your skills and experience but give your interviewers insight into how you tackle data analytics problems. Your portfolio should highlight to hiring managers how you thought about, worked through, and visualized your results. In this final portfolio project, you'll use your understanding of data analytics and visualization to answer a question about a topic of your choosing.

For this project, you will conduct your own analysis in a Jupyter Notebook and share your results through your personal GitHub page. This project can be included as part of your portfolio when you apply for data scientist and data analyst jobs. It's good practice to showcase a variety of skills in order to make your application shine.

[Example Project](#)

```
In [21]: # add new total_purchases column to marketing_data
marketing_data["total_purchases"] = marketing_data["num_deals_purchases"] + marketing_data["num_catalog_purchases"] + max

In [22]: plt.figure()
# scatterplot of total_purchases vs. income
sns.scatterplot(y=marketing_data["total_purchases"], x=marketing_data["income"],
                hue=marketing_data["education"], palette="viridis")
plt.title("Purchase Behavior by income")
plt.show()
```



Project Objectives:

- Complete a project to add to a portfolio
- Use Git version control
- Use Jupyter Notebook to communicate findings
- Run an end-to-end data analytics project
- Become familiar with data analytics workflows

Prerequisites:

- Python Fundamentals
- Data Acquisition
- Data Manipulation with Pandas
- Data Wrangling and Tidying
- Summary Statistics

Hypothesis Testing

Data Visualization

Communicating Data Analysis Findings

Select Data Analyst Solvable Problem

First choose a topic of interest; then, within your topic, select a problem that you can answer using a data analytics approach. Make sure that this problem can be solved in a reasonable amount of time and with available data. If the problem is too complex, it might be worth breaking it down into smaller units.

Collect Data

If you didn't already have a dataset in mind, search the internet and see if it's freely available. If data is not available, go back to the previous step and find a new problem that will have data readily available.

Load and Check Data

Once you obtain your data, check to see if you can identify what each variable is measuring. Make sure it has the variables that you need for your exploratory data analysis and that there is enough data.

Determine what kinds of analysis you need

Do the questions you are asking require you to conduct hypothesis tests, look for correlations or build models? Knowing that in advance will help guide your data exploration and shape how you clean your data.

Data Wrangling and Tidying

Clean your data. Make sure that all the variables and values make sense. Decide how you will handle missing data. Concatenate or split values as necessary to work with your variables. Group and merge as needed to get data at the aggregation level you will need.

Explore and Describe your Data

Once your data is cleaned and ready to go, explore it through the use of summary statistics and data visualizations. Make sure to document findings along the way.

Communicate Findings

Make your visualizations and documented findings easy to decipher for potential stakeholders and/or the general public.

Conclusion

Finally, we can wrap up the project. You can write a conclusion about your process and findings.

Next Steps

Lastly, being a data analyst is about making continual progress. What are future-looking takeaways and limitations? If you were to do this project again, how can you extend this analysis?

NTILE - SQL partition of SQL

- **NTILE(5)** - return data divided in 5 equal part; **NTILE(8)** - divided in 8 parts

The last window function we are going to learn is called `NTILE` and this can be used to find quartiles, quintiles or whatever ntile your data-driven heart desires.

`NTILE` allows you to break your data into roughly equal groups, based on what ntile you'd like: so if you were using *quartile*, it would divide the data into *four groups* (quarters).

When using `NTILE` you are required to provide a *bucket*, which represents the number of groups you'd like your data broken down into: `NTILE (4)` would be four "buckets" which would represent **quartiles**.

Instructions

Checkpoint 1 Passed

1.

Run the following query:

```
SELECT  
    NTILE (5) OVER (
```

```
ORDER BY streams_millions DESC
) AS 'weekly_streams_group',
  artist,
  week,
  streams_millions
FROM
  streams;
```

Copy to Clipboard

Think about the following:

What does each `NTILE` represent?

If you're in `weekly_streams_group 1`, are you the most or least streamed artist?

What artists make up the most streamed grouping?

Hint

Because we have 5 buckets, this `NTILE` would represent *quintiles* or fifths. The top quintile is made almost entirely of Drake. The Weeknd and Bad Bunny also had enough weekly streams to land them a few spots in the top quintile.

Checkpoint 2 Passed

2.

Change our `NTILE` query to be a *quartile*, name the column `quartile` and run the query. How does this change our resultant data? What artists are now in the top quartile?

Hint

With quartiles, each group is now **larger** and contains more weekly streams. Our top quartile is still mostly Drake and The Weeknd, with one week from Bad Bunny and one week from Lady Gaga (who was not in the previous top quintile).

Checkpoint 3 Passed

3.

Now change our query to also `PARTITION BY week`.

How does this change our data?

Hint

We now have quartiles for each week. Each quartile contains two artists because we have eight artists total for each week in our dataset.

Table SQL - streams

Query Results

artist	week	streams_millions	chart_position
Drake	1	288.2	1
Drake	2	160.9	1
Drake	3	140.0	2
Drake	4	131.8	2
Drake	5	123.7	1
Drake	6	126.2	1
Drake	7	122.6	1
Drake	8	117.6	1
The Weeknd	1	76.3	6
The Weeknd	2	72.6	9

The Weeknd	3	69.9	9
The Weeknd	4	71.4	9
The Weeknd	5	66.1	9
The Weeknd	6	67.3	8
The Weeknd	7	63.6	8
The Weeknd	8	60.6	8
Taylor Swift	1	47.7	15
Taylor Swift	2	48.0	18
Taylor Swift	3	54.1	13
Taylor Swift	4	51.0	15
Taylor Swift	5	47.9	16
Taylor Swift	6	48.9	14
Taylor Swift	7	47.8	14
Taylor Swift	8	48.3	15
Doja Cat 1	41.7	17	
Doja Cat 2	39.5	23	
Doja Cat 3	38.5	23	
Doja Cat 4	34.5	25	
Doja Cat 5	28.9	42	
Doja Cat 6	28.2	43	
Doja Cat 7	26.5	49	
Doja Cat 8	24.8	57	
Luke Combs	1	55.8	9
Luke Combs	2	52.1	14
Luke Combs	3	54.5	12
Luke Combs	4	55.0	12
Luke Combs	5	52.8	13
Luke Combs	6	52.4	12
Luke Combs	7	50.5	12
Luke Combs	8	50.2	11
Bad Bunny	1	33.7	25
Bad Bunny	2	69.6	11
Bad Bunny	3	59.0	11
Bad Bunny	4	49.7	16
Bad Bunny	5	42.3	20
Bad Bunny	6	42.8	19
Bad Bunny	7	41.7	20
Bad Bunny	8	39.3	21
Beyoncé 1	26.3	42	
Beyoncé 2	25.9	47	
Beyoncé 3	26.6	46	
Beyoncé 4	26.5	50	
Beyoncé 5	26.3	53	
Beyoncé 6	27.9	45	
Beyoncé 7	27.3	46	
Beyoncé 8	34.3	27	
Lady Gaga	1	15.4	106
Lady Gaga	2	15.2	112
Lady Gaga	3	16.6	98
Lady Gaga	4	21.0	75
Lady Gaga	5	64.0	10
Lady Gaga	6	36.0	24
Lady Gaga	7	30.5	36
Lady Gaga	8	27.0	47

SOLUTION 1

weekly_streams_group	artist	week	streams_millions
1 Drake 1	288.2		

1	Drake	2	160.9
1	Drake	3	140.0
1	Drake	4	131.8
1	Drake	6	126.2
1	Drake	5	123.7
1	Drake	7	122.6
1	Drake	8	117.6
1	The Weeknd	1	76.3
1	The Weeknd	2	72.6
1	The Weeknd	4	71.4
1	The Weeknd	3	69.9
1	Bad Bunny	2	69.6
2	The Weeknd	6	67.3
2	The Weeknd	5	66.1
2	Lady Gaga	5	64.0
2	The Weeknd	7	63.6
2	The Weeknd	8	60.6
2	Bad Bunny	3	59.0
2	Luke Combs	1	55.8
2	Luke Combs	4	55.0
2	Luke Combs	3	54.5
2	Taylor Swift	3	54.1
2	Luke Combs	5	52.8
2	Luke Combs	6	52.4
2	Luke Combs	2	52.1
3	Taylor Swift	4	51.0
3	Luke Combs	7	50.5
3	Luke Combs	8	50.2
3	Bad Bunny	4	49.7
3	Taylor Swift	6	48.9
3	Taylor Swift	8	48.3
3	Taylor Swift	2	48.0
3	Taylor Swift	5	47.9
3	Taylor Swift	7	47.8
3	Taylor Swift	1	47.7
3	Bad Bunny	6	42.8
3	Bad Bunny	5	42.3
3	Doja Cat 1	41.7	
4	Bad Bunny	7	41.7
4	Doja Cat 2	39.5	
4	Bad Bunny	8	39.3
4	Doja Cat 3	38.5	
4	Lady Gaga	6	36.0
4	Doja Cat 4	34.5	
4	Beyoncé 8	34.3	
4	Bad Bunny	1	33.7
4	Lady Gaga	7	30.5
4	Doja Cat 5	28.9	
4	Doja Cat 6	28.2	
4	Beyoncé 6	27.9	
4	Beyoncé 7	27.3	
5	Lady Gaga	8	27.0
5	Beyoncé 3	26.6	
5	Doja Cat 7	26.5	
5	Beyoncé 4	26.5	
5	Beyoncé 1	26.3	
5	Beyoncé 5	26.3	
5	Beyoncé 2	25.9	
5	Doja Cat 8	24.8	

5	Lady Gaga	4	21.0
5	Lady Gaga	3	16.6
5	Lady Gaga	1	15.4
5	Lady Gaga	2	15.2

SOLUTION 2

weekly_streams_group	artist	week	streams_millions
1	Drake 1	288.2	
1	Drake 2	160.9	
1	Drake 3	140.0	
1	Drake 4	131.8	
1	Drake 6	126.2	
1	Drake 5	123.7	
1	Drake 7	122.6	
1	Drake 8	117.6	
1	The Weeknd 1	76.3	
1	The Weeknd 2	72.6	
1	The Weeknd 4	71.4	
1	The Weeknd 3	69.9	
1	Bad Bunny 2	69.6	
1	The Weeknd 6	67.3	
1	The Weeknd 5	66.1	
1	Lady Gaga 5	64.0	
2	The Weeknd 7	63.6	
2	The Weeknd 8	60.6	
2	Bad Bunny 3	59.0	
2	Luke Combs 1	55.8	
2	Luke Combs 4	55.0	
2	Luke Combs 3	54.5	
2	Taylor Swift 3	54.1	
2	Luke Combs 5	52.8	
2	Luke Combs 6	52.4	
2	Luke Combs 2	52.1	
2	Taylor Swift 4	51.0	
2	Luke Combs 7	50.5	
2	Luke Combs 8	50.2	
2	Bad Bunny 4	49.7	
2	Taylor Swift 6	48.9	
2	Taylor Swift 8	48.3	
3	Taylor Swift 2	48.0	
3	Taylor Swift 5	47.9	
3	Taylor Swift 7	47.8	
3	Taylor Swift 1	47.7	
3	Bad Bunny 6	42.8	
3	Bad Bunny 5	42.3	
3	Doja Cat 1	41.7	
3	Bad Bunny 7	41.7	
3	Doja Cat 2	39.5	
3	Bad Bunny 8	39.3	
3	Doja Cat 3	38.5	
3	Lady Gaga 6	36.0	
3	Doja Cat 4	34.5	
3	Beyoncé 8	34.3	
3	Bad Bunny 1	33.7	
3	Lady Gaga 7	30.5	
4	Doja Cat 5	28.9	
4	Doja Cat 6	28.2	
4	Beyoncé 6	27.9	

4	Beyoncé 7	27.3	
4	Lady Gaga	8	27.0
4	Beyoncé 3	26.6	
4	Doja Cat 7	26.5	
4	Beyoncé 4	26.5	
4	Beyoncé 1	26.3	
4	Beyoncé 5	26.3	
4	Beyoncé 2	25.9	
4	Doja Cat 8	24.8	
4	Lady Gaga	4	21.0
4	Lady Gaga	3	16.6
4	Lady Gaga	1	15.4
4	Lady Gaga	2	15.2

SOLUTION 3

weekly_streams_group	artist	week	streams_millions
1	Drake 8	117.6	
1	The Weeknd	8	60.6
1	Taylor Swift	8	48.3
1	Doja Cat 8	24.8	
1	Luke Combs	8	50.2
1	Bad Bunny	8	39.3
1	Beyoncé 8	34.3	
1	Lady Gaga	8	27.0
1	Drake 7	122.6	
1	The Weeknd	7	63.6
1	Taylor Swift	7	47.8
1	Doja Cat 7	26.5	
1	Luke Combs	7	50.5
1	Bad Bunny	7	41.7
1	Beyoncé 7	27.3	
1	Lady Gaga	7	30.5
2	Drake 6	126.2	
2	The Weeknd	6	67.3
2	Taylor Swift	6	48.9
2	Doja Cat 6	28.2	
2	Luke Combs	6	52.4
2	Bad Bunny	6	42.8
2	Beyoncé 6	27.9	
2	Lady Gaga	6	36.0
2	Drake 5	123.7	
2	The Weeknd	5	66.1
2	Taylor Swift	5	47.9
2	Doja Cat 5	28.9	
2	Luke Combs	5	52.8
2	Bad Bunny	5	42.3
2	Beyoncé 5	26.3	
2	Lady Gaga	5	64.0
3	Drake 4	131.8	
3	The Weeknd	4	71.4
3	Taylor Swift	4	51.0
3	Doja Cat 4	34.5	
3	Luke Combs	4	55.0
3	Bad Bunny	4	49.7
3	Beyoncé 4	26.5	
3	Lady Gaga	4	21.0
3	Drake 3	140.0	
3	The Weeknd	3	69.9

3	Taylor Swift	3	54.1
3	Doja Cat 3	38.5	
3	Luke Combs	3	54.5
3	Bad Bunny	3	59.0
3	Beyoncé 3	26.6	
3	Lady Gaga	3	16.6
4	Drake 2	160.9	
4	The Weeknd	2	72.6
4	Taylor Swift	2	48.0
4	Doja Cat 2	39.5	
4	Luke Combs	2	52.1
4	Bad Bunny	2	69.6
4	Beyoncé 2	25.9	
4	Lady Gaga	2	15.2
4	Drake 1	288.2	
4	The Weeknd	1	76.3
4	Taylor Swift	1	47.7
4	Doja Cat 1	41.7	
4	Luke Combs	1	55.8
4	Bad Bunny	1	33.7
4	Beyoncé 1	26.3	
4	Lady Gaga	1	15.4